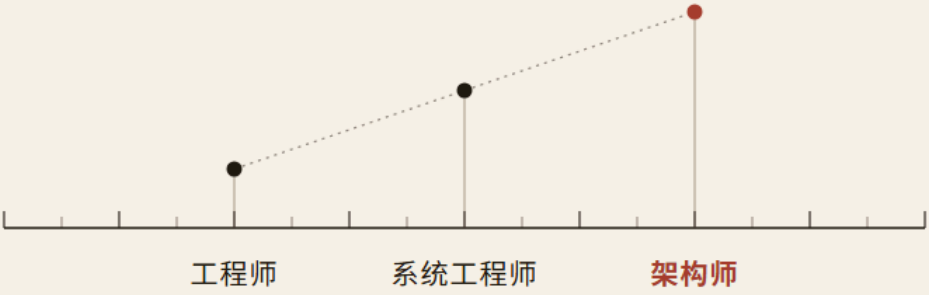
「大部分人在不准确的地基上盖房子」

# 架构师是个怎样的物种?

## —— 从工程师到架构师




学名 Architectus Systematis

特征：兼具架构活动能力和职责的工程师

把话说准

周拥军 著

系统工程出版社

# 架构师是个怎样的物种？

——从工程师到架构师

著：周拥军

2026 年 8 月

系统工程出版社 · EPUB / A5 打印稿

ISBN：—（待定）

书中两个团队与其中每一个人物，姓名均为化名；情节经合成，不指涉任何真实个人与机构。

# 前言：先立枢纽

这本书写了很久。起因不是“缺一本架构书”，是一个越来越强的疑问：**工具、流程、方法年年进步，项目的失败为什么没有变少？**

我给的答案是：多数失败不是技术失败，是概念失败。“需求”和“规定需求”，“验证”和“确认”，“架构”和“架构描述”，“配置”和“参数文件”——这些词天天挂在嘴边，能用、不会定义、各指各的。地基不准确，上面盖多少层都白搭。

这就是这本书的起点，也是它的终点：**把地基打准**。

所以它不是教科书——教科书给你完整的体系，让你按图索骥；也不是参考手册——手册给你现成的答案，让你随时翻。它是一本有观点的书，观点只有一句：**概念先要说准，方法才有地方站**。

判断一个概念讲没讲透，我只看三样：有没有可检验的判据、有没有失效的边界、有没有反例。三样缺一，就没讲透。这是这本书对自己的要求，也是读者可以拿来回敬它的尺子——**我写的每一把尺子，都欢迎你来检验**。

也说说它不做什么。它不承诺读完就能造出好系统——把词说准只是第一步，后面还有判断、取舍、时机，那些要在真实项目里练。它也不把概念当真理——概念是共识，共识可以修正。

为了说清这些词，我用了两个虚构的团队：造温控箱的冰链，做审批系统的恒信。它们不指涉任何真实公司，只是这个行业最常见的两种形态——一个造有形的设备，一个造无形的系统。同一个概念，让它在两个世界里各长各的样子。**概念不挑行业**，这是这本书想证明的第一件事。

两个团队一共十三个人。他们不是布景：从第 1 章那两场会起，这十三个人陪着你走完十八章，各长各的一截。他们是谁、两家各造什么，下一页写全了。

另有一件事，你会在很多章里撞见：有人把一份东西写下来、补起来，或者守住。它叫什么，第 7 章会给它名字；在那之前，先看他们怎么做。

全书十八章。第 1 章先立方法：把话说准，本身要练成一种能力。其余分四篇。

上篇·概念的地基（第 2~5 章）把地基上最常用的词钉住：工程，需求、设计开发、质量，需求工程，需求的完整性。

中篇·方法与判定（第 6~14 章）走完产品开发那条路：设计开发落地，产品数据，评审、验证与确认，工程师；再把架构这一族说清：架构概念、架构定义、架构决策、架构描述、架构与设计。

下篇·架构与治理（第 15~17 章）让判断受控、有效力、一路贯通。终篇·回到人（第 18 章）把前面各章合成一个人：架构师。

每章排成同一副骨架：先说这个词在工位上怎么被用错，再把它磨到边界清楚，最后看它往哪去。章末是认知反转与能力级，接五个抽屉：本章的概念总图、能力级、自查、练习、参考文献。全书的五个抽屉收在书末附录——术语表、概念档案、误解清单、参考文献、练习答案。再往后是后记。

读的时候，把精力分三档。**开场的现场是引子，一遍读可以快些过去**；**正文的论证要慢读**——那里每个判断都挂着出处，是这本书的地基；**章末的能力级与自查要做**——能力级告诉你这一章练到第几级，自查把这章的判据反过来问你。旁注是术语与出处的落脚处，读到拿不准的词，回看旁注比往前猜省事。

若只有时间读一样，就读能力级与自查：**它们是这本书真正想交到你手里的东西**。

想入门，从第 1 章读起，先握住那把四问的尺子，再往下走——每一章都是把第 1 章的方法用在一个词上。想救火，直接翻附录：五个抽屉收着全书所有的尺子和判据，查得到，就不算没掌握。想带团队，把第 9 章和第 18 章连起来读，选人、定岗、该不该给“架构师”这个头衔，都在这两张清单里。

正文用两个角色收束：第 9 章的工程师，第 18 章的架构师。选它们一头一尾，因为头衔最容易被误当成能力，“架构师”尤其如此。书里反复讲的是一件事：**头衔不定义人，能力才定义人**。一个物种，先是工程师，再往系统级长；把整条链看在眼里、落成决定的，是架构师。

而能力与价值，最终都体现在他们交出的产品数据上。

正文之外还有一篇**后记**：两年后，两间屋的人各自摊开手上的产品数据，把四个角色（系统架构师、系统工程师、模块架构师、模块工程师）和四份产品数据清单，交到你手里。

周拥军 2026 年 8 月 19 日

# 书中产品与人物

书里有两支虚构的团队：一个造有形的设备，一个造无形的系统。同一个词，在他们两家各长一个样——这是这本书想证明的第一件事。

**冰链科技：造温控箱的那一家**

冰链造医疗冷链运输温控箱。产品 M3 把硬件、嵌入式、云端和 App 联在一只箱子上：疫苗与药品从冷库到接种点，箱温守在 -20℃~8℃，全程留得下温度记录；断链 30 秒内告警，断电保温 72 小时。这本书用它演示造有形设备的那一路。

| 姓名 | 岗位 | 在书里管哪一摊 |
| --- | --- | --- |
| 林涛 | 产品经理 | 需求、评审、对客户 |
| 赵刚 | 产品开发经理 | 产品开发全局、评审签字 |
| 石军 | 嵌入式硬件工程师 | 温控硬件、固件 |
| 吴浩 | 云端软件 | 数据采集、App、追溯平台 |
| 季昭 | 测试 | 验证与确认 |
| 小张（女） | 系统工程师 | 整条链受控与可追溯 |
| 小周 | 制造与供应链 | 量产、物料、批次 |

**恒信集团：做审批系统的那一家**

恒信做跨部门的合同审批系统。法务、财务、业务三家在同一份合同上签字，一套流程管住合同版本与台账：月底对账要在两小时内对得平、一笔不差，合同数据留档十年。这本书用它演示造无形系统的那一路。

| 姓名 | 岗位 | 在书里管哪一摊 |
| --- | --- | --- |
| 顾言 | 需求分析师 | 需求获取、运行概念、规定需求 |
| 曹峰 | 产品开发经理 | 合同平台的上线、验收与业务价值 |
| 罗建 | 系统架构师 | 系统架构、数据模型 |
| 苏婷（女） | 开发 | 前后端、接口 |
| 王默 | 运维 | 部署、监控、版本发布 |
| 郑亮 | 业务方 | 财务、法务、业务的审批链接口 |

# 目录

前言：先立枢纽 . . . . . 4
书中产品与人物 . . . . . 7
第 1 章 概念体系：学习能力的杠杆 . . . . . 29
1.1 引言 . . . . . 29
1.1.1 误解现场 . . . . . 29
1.1.2 本章要回答什么 . . . . . 30
1.1.3 本章的骨架 . . . . . 31
1.2 先说准哪个词：真正要紧的词很少 . . . . . 32
1.2.1 概念是知识单元，不是术语 . . . . . 32
1.2.2 概念有主次：一般的、核心的、根上的 . . . . . 34
1.2.3 先抓哪个：从枢纽走到源头 . . . . . 36
1.3 概念体系能力：本事不在懂得多 . . . . . 38
1.3.1 挑枢纽、连关系、搬得走、划得清 . . . . . 38
1.3.2 根概念能力：先抓源头，再展结构 . . . . . 40
1.3.3 记忆可以外包，把话说准不能 . . . . . 40
1.4 凭什么说回报最高：方向与表征 . . . . . 41
1.4.1 学习产出是一道乘法 . . . . . 41
1.4.2 力臂、施力、支点：只负责放大 . . . . . 42
1.4.3 方向：挑错了词，全盘皆输 . . . . . 43
1.4.4 一个词能说到多深：能力四级 . . . . . 44
1.4.5 表征：第一级拿不到，上面全是零 . . . . . 47
1.5 有什么证据：一项压缩学习实验 . . . . . 50
1.5.1 差距在压缩，不在记性 . . . . . 50
1.5.2 实验：先学枢纽的人更快 . . . . . 51
1.5.3 把压缩用在自己身上 . . . . . 52

1.5.4 实验证明了什么，没证明什么 . . . . . 52
1.6 怎么练：一条循环转起来 . . . . . 53
1.6.1 什么叫学会：会用才算掌握 . . . . . 53
1.6.2 循环：从立枢纽到用起来 . . . . . 53
1.6.3 练的是哪一样，卡在哪一环 . . . . . 55
1.6.4 校准：概念是共识，不是真理 . . . . . 56
1.7 怎么找：挑出枢纽与根概念 . . . . . 57
1.7.1 概念是点，原理是线 . . . . . 57
1.7.2 怎么挑出枢纽 . . . . . 57
1.7.3 判定矩阵与演绎链 . . . . . 59
1.7.4 根概念长什么样：阻抗 . . . . . 60
1.7.5 根概念长什么样：产品数据 . . . . . 60
1.8 怎么验：答不全，就只是见过 . . . . . 62
1.8.1 假懂长什么样 . . . . . 62
1.8.2 怎么问才算问全 . . . . . 63
1.8.3 词汇病：每种都有药 . . . . . 65
1.8.4 查全：概念档案 . . . . . 67
1.8.5 认知反转 . . . . . 67
第 2 章 工程：让产品开发可重复 . . . . . 76
2.1 引言 . . . . . 76
2.1.1 误解现场 . . . . . 76
2.1.2 本章要回答什么 . . . . . 77
2.1.3 本章的骨架 . . . . . 78
2.2 工程的定义：每一样都不能少 . . . . . 79
2.2.1 先给工程一个定义 . . . . . 79
2.2.2 逐条拆开：各是什么，各管什么 . . . . . 80
2.2.3 权威定义：不同年代，同一个内核 . . . . . 81

2.2.4 交付能不能离开某个人 . . . . . 85
2.3 工程常被误当成什么 . . . . . 87
2.3.1 工程没有单一对立面 . . . . . 87
2.3.2 不是手艺：靠人，还是靠方法 . . . . . 88
2.3.3 不是方法论：方法论只是其中一层 . . . . . 89
2.3.4 不是机械化：系统化不等于算法化 . . . . . 93
2.4 部件全对，系统未必对 . . . . . 98
2.4.1 系统工程：不造部件，只造整体 . . . . . 98
2.4.2 涌现属性：合起来才有的行为 . . . . . 100
2.4.3 工程系统包含技术系统 . . . . . 101
2.4.4 验收参照：上线不等于被用起来 . . . . . 104
2.4.5 回头给恒信量一遍 . . . . . 106
2.4.6 认知反转 . . . . . 107
第 3 章 需求、设计开发、质量：产品开发的图式 . . . . . 115
3.1 引言 . . . . . 115
3.1.1 误解现场 . . . . . 115
3.1.2 本章要回答什么 . . . . . 116
3.1.3 本章的骨架 . . . . . 117
3.2 需求、设计开发、质量各管什么 . . . . . 118
3.2.1 需求：没说出口的也算 . . . . . 118
3.2.2 需要、期望与需求：内容与呈现方式 . . . . . 120
3.2.3 目的、目标与需求：价值、结果与规格 . . . . . 121
3.2.4 设计开发：需求怎么变得更详细 . . . . . 123
3.2.5 设计开发：每一级都在做同一件事 . . . . . 126
3.2.6 质量：固有特性满足需求的程度 . . . . . 128
3.2.7 概念卡片 . . . . . 129
3.3 起点、终点与参照，各有一把判据 . . . . . 133

3.3.1 设计开发不始于点子 . . . . . 134
3.3.2 起点的判据：需求要可转化 . . . . . 134
3.3.3 需求开发的边界：特性有没有被规定 . . . . . 136
3.3.4 终点的判据：为什么不是可验证 . . . . . 137
3.3.5 终点的样子：方案配齐，指令可执行 . . . . . 138
3.3.6 参照的判据：等级不是质量 . . . . . 141
3.4 读出程度，解开冲突，合起来看 . . . . . 144
3.4.1 程度怎么认定：从特性值到结论 . . . . . 144
3.4.2 分值为什么靠不住 . . . . . 146
3.4.3 质量与进度：改需求，不改质量 . . . . . 147
3.4.4 互相定义，所以拆不掉 . . . . . 151
3.4.5 认知反转 . . . . . 154
第 4 章 需求工程：把诉求变成需求规格 . . . . . 163
4.1 引言 . . . . . 163
4.1.1 误解现场 . . . . . 163
4.1.2 本章要回答什么 . . . . . 164
4.1.3 本章的骨架 . . . . . 164
4.2 开发与管理并列，不是包含 . . . . . 165
4.2.1 需求工程：开发做出来，管理看住 . . . . . 165
4.2.2 为什么是并列，不是包含 . . . . . 167
4.2.3 需求开发：把诉求变成规定需求 . . . . . 169
4.2.4 需求的来路 . . . . . 172
4.2.5 规定需求：SRS（需求规格说明书）里有什么 . . . . . 173
4.2.6 需求管理：看住基线，管住变更 . . . . . 176
4.2.7 开发与管理在哪里交接 . . . . . 180
4.3 最容易混的边界，分开看 . . . . . 184
4.3.1 明示与可采：说出来了，就能当依据吗 . . . . . 184

4.3.2 开发与管理：需求乱，是谁的账 ..... 185
4.3.3 需求文档：能评审，能验证，做不了确认 ..... 186
4.3.4 留档与受控：改名算不算版本管理 ..... 187
4.4 需求工程怎么做 ..... 189
4.4.1 运行概念（ConOps）：系统将被怎么用 ..... 189
4.4.2 运行概念的位置：分析之后，规格之前 ..... 190
4.4.3 一份运行概念里有什么 ..... 191
4.4.4 一条好需求的判据 ..... 192
4.4.5 需求文档体系 ..... 194
4.4.6 认知反转 ..... 198
第 5 章 需求的完整性：全不全，责任在承制方 ..... 208
5.1 引言 ..... 208
5.1.1 误解现场 ..... 208
5.1.2 本章要回答什么 ..... 209
5.1.3 本章的骨架 ..... 209
5.2 全不全，看谁在问 ..... 210
5.2.1 一份全的需求：覆盖什么 ..... 210
5.2.2 需求分类：同一特性，各有各的主人 ..... 212
5.2.3 功能需求：从业务用例走到场景 ..... 212
5.2.4 环境适应性需求：先把环境集定全 ..... 213
5.2.5 可实现性需求：客户不问，承制方提 ..... 215
5.2.6 每一类需求：都要有指标 ..... 217
5.3 先立载体，再分清混的说法 ..... 219
5.3.1 验证载体：各管一类需求 ..... 219
5.3.2 累加的是交付物，不是验证 ..... 221
5.3.3 样机与产品：通过不等于成功 ..... 222
5.3.4 功能与非功能，分得开吗 ..... 222

5.3.5 隐含需求：算不算单独一类 ..... 223
5.4 缺的需求从哪儿补 ..... 225
5.4.1 谁主导，谁确认 ..... 225
5.4.2 主导不等于替客户拍板 ..... 228
5.4.3 没人催的需求，承制方得记住 ..... 229
5.4.4 承制方靠资产积累，不靠现场救火 ..... 229
5.4.5 从症状反推缺了哪条需求 ..... 230
5.4.6 成本是需求，不是结果 ..... 233
5.4.7 认知反转 ..... 234
第 6 章 设计开发：需求与方案结对 ..... 242
6.1 引言 ..... 242
6.1.1 误解现场 ..... 242
6.1.2 本章要回答什么 ..... 244
6.1.3 本章的骨架 ..... 244
6.2 先把设计开发的对象看清 ..... 246
6.2.1 设计开发的单元是一对 ..... 246
6.2.2 设计文档要能定位、承接、追溯 ..... 247
6.2.3 产品数据凭什么能成树 ..... 249
6.2.4 框架定义管结构，详细定义管内容 ..... 252
6.2.5 同一棵树，停在哪个深度 ..... 255
6.2.6 框架定义不是架构定义 ..... 257
6.2.7 树装不下的内容 ..... 259
6.3 先立规矩，再磨动作 ..... 262
6.3.1 每一对先定规则 ..... 262
6.3.2 先做准备，再进循环 ..... 266
6.3.3 分配生成：从需求长出骨架 ..... 268
6.3.4 分析迭代：一条需求一个归宿 ..... 273

6.3.5 分解分析：切到可独立验证 . . . . . 278
6.3.6 覆盖检查：接住的需求一定被实现 . . . . . 283
6.3.7 干枝与叶节点分开维护 . . . . . 288
6.4 一条链上的逐对接力 . . . . . 289
6.4.1 上一份的叶节点，下一份的需求 . . . . . 289
6.4.2 链上只有分配步在做承接 . . . . . 292
6.4.3 链尾交给构件设计方 . . . . . 293
6.4.4 一对从接需求走到冻结基线 . . . . . 294
6.4.5 设计开发的验收与追溯 . . . . . 299
6.4.6 认知反转 . . . . . 300
第 7 章 产品数据：开发产品就是开发数据 . . . . . 311
7.1 引言 . . . . . 311
7.1.1 误解现场 . . . . . 311
7.1.2 本章要回答什么 . . . . . 313
7.1.3 本章的骨架 . . . . . 314
7.2 产品不只是实物 . . . . . 315
7.2.1 产品：物理存在与信息存在 . . . . . 315
7.2.2 产品：需求、价值与过程 . . . . . 317
7.2.3 产品数据：形式化的信息表示 . . . . . 319
7.2.4 产品数据不等于什么 . . . . . 322
7.2.5 产品数据该有的样子 . . . . . 323
7.2.6 实现方案：最重的产品数据 . . . . . 324
7.3 产品从定义到复制 . . . . . 328
7.3.1 开发止于可实现的需求定义 . . . . . 328
7.3.2 两域三态：产品的存在方式 . . . . . 330
7.3.3 设计态里，开发就是定义生成 . . . . . 332
7.3.4 软件没有独立的制造环节 . . . . . 333

7.3.5 产品质量的上限由数据质量决定 . . . . . 334
7.3.6 沿工位流动的是产品数据 . . . . . 337
7.4 产品数据能装下什么 . . . . . 341
7.4.1 利益相关方围绕产品数据协作 . . . . . 341
7.4.2 价值三层：哪一层落进了数据 . . . . . 344
7.4.3 职责为什么挂在工件上 . . . . . 346
7.4.4 活动值不值：产品数据增值判据 . . . . . 351
7.4.5 产品会死，数据体系活着 . . . . . 352
7.4.6 数字化不等于转型 . . . . . 355
7.4.7 认知反转 . . . . . 356
第 8 章 评审、验证与确认：各有一个参照 . . . . . 365
8.1 引言 . . . . . 365
8.1.1 误解现场 . . . . . 365
8.1.2 本章要回答什么 . . . . . 367
8.1.3 本章的骨架 . . . . . 367
8.2 判定：对什么，凭什么 . . . . . 368
8.2.1 病根：译名看着像近亲 . . . . . 368
8.2.2 参照落在三个不同的世界 . . . . . 370
8.2.3 差别不只在参照 . . . . . 373
8.2.4 评审：适宜、充分、有效 . . . . . 375
8.2.5 验证：对照、证据、证实 . . . . . 378
8.2.6 确认：换参照，不换判法 . . . . . 382
8.3 误用在哪，射程到哪 . . . . . 386
8.3.1 评审不是核对需求 . . . . . 387
8.3.2 测试通过不等于验证通过 . . . . . 390
8.3.3 业务方点了不等于确认过了 . . . . . 391
8.3.4 验证够不着没写下来的需求 . . . . . 393

8.3.5 确认的参照在场景里 . . . . . 394
8.3.6 评审早拦，确认终审 . . . . . 397
8.4 一份需求走完全程 . . . . . 399
8.4.1 先把三关串成一条链 . . . . . 400
8.4.2 评审贯穿整条流程 . . . . . 403
8.4.3 只做验证，会漏掉什么 . . . . . 406
8.4.4 评审怎么评 . . . . . 409
8.4.5 验证怎么验 . . . . . 413
8.4.6 确认怎么确认 . . . . . 415
8.4.7 认知反转 . . . . . 416
第 9 章 工程师：一份能力清单 . . . . . 425
9.1 引言 . . . . . 425
9.1.1 误解现场 . . . . . 425
9.1.2 本章要回答什么 . . . . . 425
9.1.3 本章的骨架 . . . . . 426
9.2 一份清单：能力与活 . . . . . 427
9.2.1 清单不是拍脑袋列的 . . . . . 427
9.2.2 ①科学素养：凭什么能行 . . . . . 429
9.2.3 ②方法论素养：把工作组织成方法 . . . . . 431
9.2.4 ③实践能力：亲手把方案变成现实 . . . . . 432
9.2.5 ⑤工程判断：在约束里取舍 . . . . . 433
9.2.6 ⑥可复现意识：让判断被接住 . . . . . 434
9.2.7 ④职业伦理：给能力定方向 . . . . . 435
9.2.8 工程师干的活：从需求到判定 . . . . . 437
9.2.9 清单不挑领域：变的只是权重 . . . . . 438
9.2.10 沟通与团队，是能力的执行手段 . . . . . 439
9.3 往外划边界，往里分角色 . . . . . 442

9.3.1 不是技术员 . . . . . 442
9.3.2 不是科学家 . . . . . 443
9.3.3 不是手艺人 . . . . . 443
9.3.4 不是“会写代码的人” . . . . . 444
9.3.5 不是头衔、证书、年限 . . . . . 444
9.3.6 不止一种角色：专责与终责 . . . . . 445
9.4 从会做，到能判 . . . . . 449
9.4.1 亲手做出来 . . . . . 449
9.4.2 从我会做到别人会做 . . . . . 450
9.4.3 从照做到会取舍 . . . . . 450
9.4.4 知道什么不能牺牲 . . . . . 451
9.4.5 从模块工程师走到系统工程师 . . . . . 451
9.4.6 从失败看缺项 . . . . . 453
9.4.7 拿产品数据检验 . . . . . 454
9.4.8 认知反转 . . . . . 458
第 10 章 架构概念：定义、价值与误解 . . . . . 467
10.1 引言 . . . . . 467
10.1.1 误解现场 . . . . . 468
10.1.2 本章要回答什么 . . . . . 469
10.1.3 本章的骨架 . . . . . 469
10.2 架构的定义 . . . . . 470
10.2.1 架构这个词的来路 . . . . . 470
10.2.2 架构是基本概念或属性，加管控原则 . . . . . 472
10.2.3 架构与架构描述：内容与载体 . . . . . 476
10.2.4 “基本”是对目的必要且充分 . . . . . 479
10.2.5 元素与关系不在定义里 . . . . . 482
10.2.6 假判据冒充真判据 . . . . . 484

10.3 架构的价值 ..... 486
10.3.1 架构站在“要什么”与“怎么做”之间 ..... 486
10.3.2 不补架构，连“好不好”都无从判 ..... 487
10.3.3 架构管到哪儿为止 ..... 489
10.3.4 架构凭什么管得住下游 ..... 490
10.4 架构的误解 ..... 493
10.4.1 不是一张图 ..... 494
10.4.2 不是一组决定 ..... 494
10.4.3 不是更粗的设计 ..... 495
10.4.4 不是一堆架构 ..... 496
10.4.5 不是框架与风格 ..... 503
10.4.6 不是“高层级” ..... 505
10.4.7 认知反转 ..... 505
第 11 章 架构定义：把判断变成判据 ..... 518
11.1 引言 ..... 518
11.1.1 误解现场 ..... 518
11.1.2 本章要回答什么 ..... 519
11.1.3 本章的骨架 ..... 520
11.2 吃进什么，吐出什么，中间加什么 ..... 521
11.2.1 吃进去的不只是一份需求 ..... 521
11.2.2 架构相关的留下，其余各有去向 ..... 523
11.2.3 需求与架构：双向的必要且充分 ..... 524
11.2.4 变换：同一批输入，能立出不同架构 ..... 525
11.2.5 架构师写目的陈述，立概念与原则 ..... 529
11.2.6 产出到哪儿为止 ..... 530
11.3 一句判断，凭什么让人非依不可 ..... 531
11.3.1 判断怎么才不只是意见 ..... 532

11.3.2 先定要支撑哪些决策 . . . . . 534
11.3.3 关切怎么变成判据 . . . . . 535
11.3.4 候选从哪来 . . . . . 536
11.3.5 什么样的想法才算候选 . . . . . 538
11.3.6 概念、属性和原则从哪来 . . . . . 539
11.4 素材：资产、知识与经验 . . . . . 541
11.4.1 资产：起点与素材 . . . . . 541
11.4.2 知识：能不能交给别人用 . . . . . 543
11.4.3 经验：只帮着找，不帮着选 . . . . . 544
11.4.4 资产稀缺时怎么办 . . . . . 545
11.4.5 认知反转 . . . . . 547
第 12 章 架构决策：在承重处做选择 . . . . . 558
12.1 引言 . . . . . 558
12.1.1 误解现场 . . . . . 558
12.1.2 本章要回答什么 . . . . . 559
12.1.3 本章的骨架 . . . . . 560
12.2 权衡：把候选摆进同一个空间 . . . . . 562
12.2.1 候选摆到一起，才看得见此消彼长 . . . . . 562
12.2.2 判据只有一个：目的 . . . . . 565
12.2.3 强制项：不是可选项 . . . . . 566
12.2.4 把谁列为候选，本身是一条决策 . . . . . 568
12.3 筛与选：先筛掉，再选定 . . . . . 570
12.3.1 筛掉看不出差别的候选 . . . . . 570
12.3.2 强制项：留得下，未必选得中 . . . . . 571
12.3.3 选定有时在架构过程之外 . . . . . 572
12.3.4 选定不是没缺点，是认下缺点 . . . . . 575
12.4 排序与时机：什么时候做 . . . . . 577

12.4.1 “改起来贵”只用来排序 . . . . . 577
12.4.2 影响半径、信息成本、修改风险 . . . . . 579
12.4.3 定节奏：早做、晚做与显式推迟 . . . . . 581
12.4.4 威胁目的，还是我们不熟 . . . . . 582
12.5 记录：一组选择与它的理据 . . . . . 583
12.5.1 决策是一组选择，不是一个设计 . . . . . 584
12.5.2 必须记什么，应该记什么 . . . . . 586
12.5.3 一份记录该有的栏位 . . . . . 589
12.5.4 认知反转 . . . . . 591
第 13 章 架构描述：把架构定义交出去 . . . . . 600
13.1 引言 . . . . . 600
13.1.1 误解现场 . . . . . 600
13.1.2 本章要回答什么 . . . . . 601
13.1.3 本章的骨架 . . . . . 602
13.2 一份工作产品，不是架构 . . . . . 605
13.2.1 架构无形，描述有形 . . . . . 605
13.2.2 一份描述由什么构成 . . . . . 606
13.2.3 规则先订，产物后造 . . . . . 606
13.2.4 为什么非有视图不可 . . . . . 608
13.2.5 不属于任何视图的信息 . . . . . 609
13.3 视角：从惯例到规格 . . . . . 610
13.3.1 视角是一组约定 . . . . . 610
13.3.2 视角管什么：框定关切，治理视图 . . . . . 611
13.3.3 一份规格该写什么 . . . . . 612
13.3.4 视角从哪来：先复用，后自建 . . . . . 613
13.3.5 视角怎么定出来 . . . . . 614
13.3.6 视角怎么用 . . . . . 615

13.4 视图：关切怎么被应对 . . . . . 616
13.4.1 视图是一个信息部件 . . . . . 616
13.4.2 图只是组件的一种形态 . . . . . 616
13.4.3 框定与应对：一次分工 . . . . . 617
13.4.4 一份视图该带什么 . . . . . 618
13.4.5 视图怎么造出来 . . . . . 619
13.4.6 视图怎么用 . . . . . 620
13.5 常用的几类视图 . . . . . 622
13.5.1 清单是例子，不是要求 . . . . . 622
13.5.2 语境视图：外部给了什么条件 . . . . . 624
13.5.3 概念视图：架构立在哪些思想上 . . . . . 624
13.5.4 功能视图：要做什么 . . . . . 625
13.5.5 逻辑视图：业务上，数据与变换是什么 . . . . . 626
13.5.6 物理视图：落在哪些元素上 . . . . . 627
13.5.7 技术视图：由哪些技术实现 . . . . . 628
13.5.8 组织视图：谁负责，谁有权 . . . . . 628
13.6 对账：缺哪一条都不算齐 . . . . . 629
13.6.1 动笔之前，手上该有什么 . . . . . 629
13.6.2 交件之前，该交出什么 . . . . . 630
13.6.3 自检：逐条问一遍 . . . . . 631
13.6.4 认知反转 . . . . . 632
第 14 章 架构与设计：不是概要和详细 . . . . . 641
14.1 引言 . . . . . 641
14.1.1 误解现场 . . . . . 641
14.1.2 本章要回答什么 . . . . . 642
14.1.3 本章的骨架 . . . . . 643
14.2 架构和设计，到底差在哪儿 . . . . . 645

14.2.1 先把名字分清楚 . . . . . 645
14.2.2 “粗一点的设计”，错在哪 . . . . . 646
14.2.3 架构要什么，设计要什么 . . . . . 647
14.2.4 design（设计）：宽义与窄义要分开 . . . . . 649
14.2.5 深浅说不通：独立性要求就是判据 . . . . . 649
14.2.6 位置不同：设计在链上，架构不在链上 . . . . . 650
14.3 交出去什么，接住什么 . . . . . 653
14.3.1 入口是检查，不是接收 . . . . . 653
14.3.2 转化什么，分配什么 . . . . . 654
14.3.3 出口是能拿去采购或实现 . . . . . 655
14.3.4 交接从来是双向的 . . . . . 658
14.3.5 分到哪一层停，看信息够不够 . . . . . 659
14.4 视图上的元素是谁家的 . . . . . 661
14.4.1 先给元素分层 . . . . . 662
14.4.2 描述里的零件，与被描述的对象 . . . . . 662
14.4.3 架构实体不是系统元素 . . . . . 664
14.4.4 架构实体可以是名义的 . . . . . 665
14.4.5 不必一一对齐 . . . . . 666
14.4.6 系统分解结构不是架构的产物 . . . . . 667
14.4.7 描述层到实现层，全是关系，没有粗细 . . . . . 668
14.5 视图上的元素都要“基本”吗 . . . . . 670
14.5.1 “基本”管的是谁 . . . . . 670
14.5.2 “基本”在视角上定，不在图标上 . . . . . 671
14.5.3 不必基本，不等于不必相关 . . . . . 672
14.5.4 换个位置就不基本了 . . . . . 673
14.6 能不能进，是谁的，够不够格 . . . . . 675
14.6.1 先问能不能进视图 . . . . . 675

14.6.2 进不去的，去哪 . . . . . 677
14.6.3 再问：图上的元素属于哪一层 . . . . . 678
14.6.4 最后问：拿掉一条，目的还达得成吗 . . . . . 679
14.6.5 各管一段，谁也替不了谁 . . . . . 679
14.6.6 认知反转 . . . . . 680
第 15 章 配置管理：让架构判断受控、可复现 . . . . . 688
15.1 引言 . . . . . 688
15.1.1 误解现场 . . . . . 688
15.1.2 本章要回答什么 . . . . . 689
15.1.3 本章的骨架 . . . . . 690
15.2 配置、版本、基线各管什么 . . . . . 692
15.2.1 配置：记下产品是什么样 . . . . . 692
15.2.2 什么算配置：不是所有特性都算 . . . . . 694
15.2.3 版本：给每一次改动编号 . . . . . 695
15.2.4 基线：被批准、被冻结，能当参照 . . . . . 696
15.2.5 先分清配置基线与产品基线 . . . . . 697
15.3 别把配置、版本、基线混着用 . . . . . 700
15.3.1 配置不是参数设置 . . . . . 700
15.3.2 版本不是基线 . . . . . 701
15.3.3 变体不是版本 . . . . . 702
15.3.4 没有基线，改动说不清 . . . . . 703
15.4 收下判断，守住基线 . . . . . 705
15.4.1 正式化：把草稿定成基线 . . . . . 705
15.4.2 定稿之前就受控 . . . . . 707
15.4.3 配置管理平时在做什么 . . . . . 708
15.4.4 变化总得跟基线比 . . . . . 710
15.4.5 候选是谁挑的，基线是谁定的 . . . . . 711

15.4.6 不做配置管理会怎么样 . . . . . 712
15.4.7 设计依从的是架构基线 . . . . . 713
15.4.8 配置管理替架构定义做了什么 . . . . . 714
15.4.9 认知反转 . . . . . 716
第 16 章 架构治理：立规矩，对结果问责 . . . . . 728
16.1 引言 . . . . . 728
16.1.1 误解现场 . . . . . 728
16.1.2 本章要回答什么 . . . . . 729
16.1.3 本章的骨架 . . . . . 730
16.2 治理、管理、架构定义各站哪儿 . . . . . 730
16.2.1 治理：一套制度，一个动作 . . . . . 731
16.2.2 治理平时做什么 . . . . . 733
16.2.3 治理和管理差在哪儿 . . . . . 737
16.2.4 治理与架构定义并行 . . . . . 738
16.2.5 治理这个词，从哪儿来 . . . . . 739
16.3 治理为什么非有不可 . . . . . 740
16.3.1 架构得一直符合目的 . . . . . 741
16.3.2 给出指导，评定，问责 . . . . . 744
16.3.3 治理对组织意味着什么 . . . . . 745
16.4 架构判断怎么才算数 . . . . . 746
16.4.1 架构判断的效力：谁来给 . . . . . 747
16.4.2 架构管理管什么 . . . . . 748
16.4.3 指令发下去，谁看有没有照办 . . . . . 750
16.4.4 谁能改架构自身 . . . . . 752
16.4.5 什么样的治理是假的 . . . . . 754
16.4.6 认知反转 . . . . . 755
第 17 章 一路贯通：从需求到上线运行 . . . . . 766

17.1 引言 . . . . . 766
17.1.1 误解现场 . . . . . 766
17.1.2 本章要回答什么 . . . . . 768
17.1.3 本章的骨架 . . . . . 768
17.2 概念怎么分，缝在哪儿，怎么合上 . . . . . 769
17.2.1 每一步都做对了，系统还是失败 . . . . . 769
17.2.2 顺着时间切，还是横着时间切 . . . . . 771
17.2.3 架构、配置、治理，不能当一站做 . . . . . 772
17.2.4 接缝：切开之后靠什么连上 . . . . . 773
17.2.5 缝合清单：写得出来，才缝得上 . . . . . 775
17.3 逐站问一句，一站不跳 . . . . . 777
17.3.1 恒信和冰链，走同一条时间线 . . . . . 777
17.3.2 需求：先弄清要成为什么 . . . . . 778
17.3.3 设计：需求怎么变成方案 . . . . . 779
17.3.4 把关：各对一个参照，各出一份证据 . . . . . 780
17.3.5 架构：定什么、权衡什么、写什么 . . . . . 782
17.3.6 配置：哪一版是基线 . . . . . 783
17.3.7 治理：谁有资格定，按什么程序 . . . . . 784
17.3.8 运维：上线之后，直到退役 . . . . . 786
17.4 换一个行业，这套概念还成立吗 . . . . . 787
17.4.1 民机平视显示器：坏了却看着正常 . . . . . 787
17.4.2 企业运营平台：能力固定，用法可变 . . . . . 789
17.4.3 认知反转 . . . . . 791
第 18 章 架构师：还是一份能力清单 . . . . . 802
18.1 引言 . . . . . 802
18.1.1 误解现场 . . . . . 802
18.1.2 本章要回答什么 . . . . . 803

18.1.3 本章的骨架 . . . . . 804
18.2 架构师会什么，每天做什么 . . . . . 805
18.2.1 每一条能力，都能追到一章 . . . . . 805
18.2.2 工程师的能力，一样都不能少 . . . . . 806
18.2.3 比工程师多出来的能力 . . . . . 807
18.2.4 配置和治理，也算架构能力 . . . . . 809
18.2.5 每天在做的活：从构思到改进 . . . . . 809
18.2.6 架构师和工程师，不一样在哪儿 . . . . . 811
18.2.7 标准只认架构活动，不认头衔 . . . . . 812
18.3 架构师的边界在哪儿 . . . . . 814
18.3.1 不是更高级的工程师 . . . . . 814
18.3.2 不是系统工程师：专责和终责 . . . . . 814
18.3.3 不是技术负责人，也不是项目经理 . . . . . 816
18.3.4 不是领域专家，也不是画图匠 . . . . . 817
18.3.5 不是目的的定义者 . . . . . 817
18.3.6 评估和认证，各管什么 . . . . . 819
18.3.7 架构师会怎么失败 . . . . . 820
18.4 架构师怎么长成，拿什么量 . . . . . 822
18.4.1 从工程师到架构师，变的是什么 . . . . . 822
18.4.2 被任命不等于成为 . . . . . 823
18.4.3 来路不同，功课也不同 . . . . . 824
18.4.4 说“不”：拦下的不是错，是没想清楚 . . . . . 824
18.4.5 再往上走，管的是组织 . . . . . 825
18.4.6 能力最终落在产品数据上 . . . . . 826
18.4.7 架构师在生存周期里的位置 . . . . . 828
18.4.8 换个人，能不能接住 . . . . . 830
18.4.9 认知反转 . . . . . 831

后记 · 两间会议室，两年后 ..... 841
附录：把尺子收进五个抽屉 ..... 852
附录 A 术语表：全书概念的英中对照 ..... 852
附录 B 概念档案集：33 张完整九格 ..... 897
附录 C 误解清单：全部反直觉命题速查 ..... 921
附录 D 参考文献总表 ..... 931
D.1 标准 ..... 931
D.2 经典文献（非标准） ..... 945
附录 E 练习答案 ..... 952
第 1 章 概念体系：学习能力的杠杆 ..... 952
第 2 章 工程：让产品开发可重复 ..... 953
第 3 章 需求、设计开发、质量：产品开发的图式 ..... 955
第 4 章 需求工程：把诉求变成需求规格 ..... 956
第 5 章 需求的完整性：全不全，责任在承制方 ..... 957
第 6 章 设计开发：需求与方案结对 ..... 958
第 7 章 产品数据：开发产品就是开发数据 ..... 959
第 8 章 评审、验证与确认：各有一个参照 ..... 961
第 9 章 工程师：一份能力清单 ..... 961
第 10 章 架构概念：定义、价值与误解 ..... 962
第 12 章 架构决策：在承重处做选择 ..... 965
第 13 章 架构描述：把架构定义交出去 ..... 966
第 14 章 架构与设计：不是概要和详细 ..... 967
第 16 章 架构治理：立规矩，对结果问责 ..... 969
第 17 章 一路贯通：从需求到上线运行 ..... 971
第 18 章 架构师：还是一份能力清单 ..... 972
附录收束 ..... 973

# 第 1 章 概念体系：学习能力的杠杆

## 1.1 引言

本章从一个现场讲起：一个词没对齐，两个项目停了工。接着回答一个问题：凭什么说“把话说准”是学习能力里回报最高的那一步？最后给出一副骨架，让你看清后面的路怎么走。一路读下来，落到手上的，是一套能用的本事。

### 1.1.1 误解现场

一个词，让两个项目停了工。

冰链的 M3 二代，模块边界划了两个月没划下来；恒信的合同系统，改造方案评了三个月没评完。一硬一软，卡住的原因一模一样：“架构”这两个字一在会上说出口，会就开不下去了。

冰链科技是造 M3 医疗温控箱的工程团队。这天他们开会，议题是 M3 二代怎么分块：

产品开发经理赵刚把会拉起来：“M3 二代的边界划了两个月。今天先把'架构'这词说清楚，再谈分块。”

产品经理林涛先把话钉死：“架构就是模块怎么分——产品架构。分块方案定下来，后面的活才好派。”

嵌入式硬件工程师石军摇头：“模块边界是设计的事。我说的是硬件架构——传感器、主控、通信怎么布，那是元件怎么划。”

云端软件的吴浩接话：“元件只是元素。系统架构在云端——服务边界、数据所有权，那是跨元素的约定。”

测试季昭皱眉：“那我按哪一层拆测试用例？验收判据挂在哪一层的定义上，我得先知道。”

制造侧的小周敲桌：“架构定不下来，物料清单和批次追溯也无从谈起。M3 二代每拖一个月，一代的订单就在外面压一批——架构对你们是张图，对我是一整条产线停不停。”

散会时，M3 二代的边界还是没划下来。

恒信集团是做合同审批系统的设计团队。这天他们在评改造方案：

产品开发经理曹峰把会议室占了：“改造方案评了三个月。今天先把'架构'这两个字对齐——对不齐，方案没法评。”

业务方的郑亮先抱怨：“我们公司的架构不行，审批链跨三个部门扯皮。”

系统架构师罗建纠正：“你说的那是组织架构。系统架构是服务边界和数据所有权——这些定不下来，图都画不出来。”

开发组的苏婷追问：“那我这边的接口清单，按哪张图对齐？前端等着开工。”

运维王默补一句：“还有部署架构呢？部署在哪、谁运维、上线怎么回滚——这一摊子总得有人说了算吧？”

需求分析师顾言放下笔：“停——'架构'在我们嘴里已经是五件事了。先各自说清你指的是哪一件。”

这一问，没人接得住。同一个词，两个世界，都是五张脸。问题不在人，在词。

可把词当场对齐，就真懂“架构”了吗？就算十张脸都同意它指系统架构，让他们各答四问：它是什么、不是什么、从哪来、往哪去，多半还是答不齐。**词可以当场对齐；概念要花功夫，才能真正拥有。**

## 1.1.2 本章要回答什么

本章从头到尾只为回答一个问题，另有六个小问题替它铺路：

1. **为什么“把话说准”**，是学习能力里回报最高的那一步？——“把话说准”，一半是**说准哪个词**，先抓枢纽与根；另一半是把它说到底。
2. 一句话说不准，凭什么能让两个项目一停就是两三个月的工？（§1.1、§1.8）
3. 说准哪个词：概念为什么是一套三层包含的体系？枢纽与根各是什么？（§1.2）
4. 谁来说准：这副能力由哪几样组成，又怎么检验它强不强？（§1.3）
5. 凭什么说回报最高：这条乘法律的两个归零项是什么，有没有证据？（§1.4.1、§1.5）
6. 说到多深才算说到：能力分哪四级，“表征是 1，上面三级是 0”又是怎么来的？（§1.4.4、§1.4.5）
7. 怎么练、怎么找、怎么验：五环是什么？三步识别法怎么找枢纽？四问法怎么验“真拥有”？（§1.6~§1.8）

### 1.1.3 本章的骨架

本章要立的，是一个主张和一把量它的尺子。

> **把话说准，是学习能力里回报最高的那一步。**

这句话里有两件事：先**说准哪个词**，抓住枢纽与根；再把这个词说到底，从一个词说到一整片。

凭什么说回报最高？因为这条乘法律的两个归零项，都落在“把话说准”上：**方向**（先抓错了词）与**表征**（话说得不准）。这两项里只要有一项是零，整条式子就是零——冰链和恒信的两场会，失守的正是这两项。

说到什么程度？同一个词，你能说到好几级：先把词说准（表征）→ 看得出套路（图式）→ 能推演（心智模型）→ 能判断什么值得相信（解释框架）。量“说到多深”的那把刻度尺，本章 §1.4.5 交给你。

后面七节就是答这一问的七步：先说准哪个词（§1.2）→ 这副能力是什么（§1.3）→ 它凭什么决定成败（§1.4）→ 有什么证据（§1.5）→ 怎么练（§1.6）→ 怎么找（§1.7）→ 怎么验（§1.8）。

## 1.2 先说准哪个词：真正要紧的词很少

> 上一问：两个项目为什么停工？因为“架构”这个词没拿准。那该说准哪个词、先抓哪几个？——一个领域里真正要紧的词很少，它们连成一张有主次的网。

### 1.2.1 概念是知识单元，不是术语

“工程”“需求”“质量”“架构”“评审”“治理”，这些词有什么共同点？它们都是各自领域的概念；而概念是什么，术语学标准里有一条严整的定义（ISO 1087:2019 §3.2.7，**经研习报告转引，正本不在库**）：**知识单元**。冰链和恒信那两场会里，“架构”就是这么个词：人人拿它当概念使，却没有一个人说得准它。

> **概念（concept）：由特性的独特组合而形成的知识单元**（ISO 1087:2019 §3.2.7）。判据：可指称（一个词或短语）＋可界定（内涵与外延）。概念与术语的区别（能指与所指），见 §1.8.3 病二。

一个概念拆开来看，有三个角。能指是词或符号，也就是概念的名字；所指是概念本身，你脑子里那个意思；指称（指称对象）是这个

词指向的那个东西。三个角里，这本书说的“概念”是中间那个：它既不在嘴上，也不在物上。冰链和恒信那两场会吵的，正是中间这个角：嘴上说的是同一个词，脑子里装的是各家的意思；能指与所指、涵义与指称这两套分法的出处，见 §1.8.3 病二。

> 术语注：别把两个“指称”混成一个。判据里的“可指称”与 ISO 1087 的 designation 一样，是“给概念命名”义；上文的“指称（指称对象）”是“概念指向的客观事物”义——同名两个角，恰是病二（§1.8.3）要治的同词异义。

概念从哪里来？多数不是哪位天才发明的，它们的来路不外乎创造、发现与凝结：

| 方式 | 含义 | 例子 |
| --- | --- | --- |
| 创造（invented） | 人造的工具，无自然界对应物 | 变量（笛卡尔/韦达）、图灵机 |
| 发现（discovered） | 规律本来就在，被揭开 | 万有引力、能量守恒 |
| 凝结（distilled） | 共同体长期实践经验的结晶被固化 | baseline（源自军品工程实践，后由 ISO 10007 固化） |

baseline 不是哪位科学家发明的：它来自军品工程“版本失控、改坏了找不回来”的教训，工程共同体摸出“受控快照”的做法，最后才被标准写成条文（15288:2023 §3.8 基线）。多数要紧的概念都是这样“酿”出来的。这也预先交代了一条后文要用的性质：概念是共识，不是真理（§1.6.4）。

关键在于：概念不是孤立的词，而是一张网。一个词的意义，由它与别的概念的关系决定；冰链和恒信那两场会之所以开不下去，就

是因为十张脸说的是同一个词，各自脑子里却是十张不同的网。这张网也不是平的：它有骨架，也有承重柱。

### 1.2.2 概念有主次：一般的、核心的、根上的

一个领域里的概念，不是一堆平级的名词，而是一套三层包含的体系：

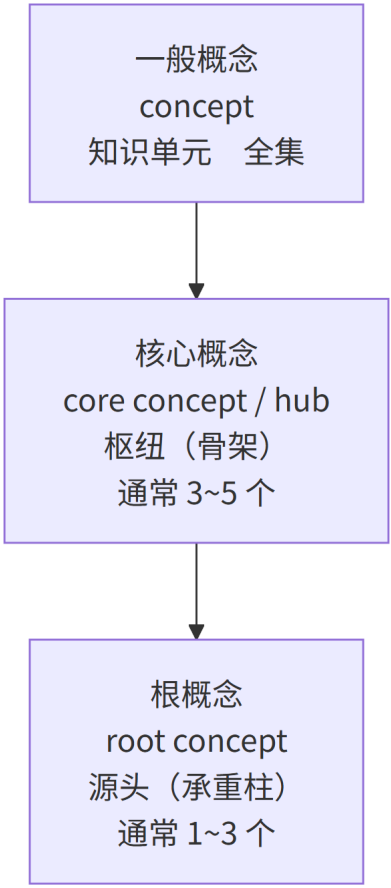



图 1-1

| 层级 | 名称 | 管什么 | 判据 | 数量 |
| --- | --- | --- | --- | --- |
| 全集 | 一般概念 | 可界定 | 可指称 + 可界定 | 全部 |
| 骨架 | 核心概念 | 连接多 | 反事实检验 + 词频／被引用度／前置数（后三条为代理判据） | 通常 3~5 个 |
| 承重柱 | 根概念 | 是源头 | 原子性 + 生成性 + 不可替代性 + 递归性 | 通常 1~3 个 |

这三层合起来，就是**三层体系**：一般概念 ⊃ 核心概念 ⊃ 根概念。层级越深、判据越严、数量越少。

它们的关系要连起来看。第一，一层套一层。根概念 ⊂ 核心概念 ⊂ 一般概念：每个根概念必然被全域依赖，每个核心概念必然可指称、可界定；反过来都不成立。大多数概念、甚至大多数核心概念，都不是根。

第二，越往里，门槛越高。一般概念只要求“能指称、能界定”，领域里绝大多数词都满足；核心概念还要过“枢纽性”：去掉它，领域不成立（反事实检验）；它被引用最多，前置最少。根概念再加一条“源头性”：不可再向下归约，其余概念由它导出，领域因它成立，多层反复生效。一句话：能界定，是概念；被依赖得多，是核心概念；依赖它的一切，它却不依赖域内任何概念，才是根概念。

第三，视角不同，不冲突。这三层比的不是谁更重要，是看你说的那个概念在网里站得靠不靠中心：核心概念（hub，轮毂）管“连接多不多”，根概念（root，根）管“是不是源头”。一个概念可以连

得多却非源头：“治理”很重要、被反复引用，但它由“对什么治理”定义，是派生的；质量也一样，在产品开发里被到处引用，却是需求与系统的派生。重要和源头，是两回事。

> 领域边界注记：一个概念算哪一层，要看在哪个领域里算。同一个词换个领域，位置就可能变——先看清是哪个领域，再判它站哪一层。

### 1.2.3 先抓哪个：从枢纽走到源头

三层里最该先动手的是哪一层？先看中间那一层。

> **核心概念（core concept / hub）**：概念网络中处于枢纽地位的概念——解释领域内其他概念都要用到它，而理解它自己只需要很少前置知识。判据：反事实检验 + 词频／被引用度／前置数（后三条为代理判据；前置数，就是理解它所需的前置知识量，越少，越是枢纽）。

要解释“符合性”，得先解释“需求”；要解释“架构”，得先解释“系统”。“需求”“系统”被反复调用，自己却几乎不需要前置：它们就是枢纽。章首冰链那场会上，三个人各说各的“架构”，谁也没有先问一句“系统”指什么；枢纽没立住，边界就划不下去。

枢纽的作用，每一处都关着整个领域。它决定这个领域用什么词说话（没有共同词汇，就没有精确沟通），也管着新知识往哪挂（新词要挂到已有的概念上才有意义），还管换个场景灵不灵（靠概念学到的本事能带走，靠死记硬背的带不走）。

再往深一层，是根概念——整张网的承重柱。

> **根概念（root concept）**：领域中不可再向下归约的源头概念，其余概念（含核心概念）都由它导出。四条判据：
>
> - **原子性**：不可由域内其他概念定义；
> - **生成性**：其余概念从它派生；
> - **不可替代性**：去掉它，领域崩塌；
> - **递归性**：多个抽象层级反复生效。

根概念 ⊂ 核心概念。一个领域里通常只有 1~3 个。概念是网上的节点，原理是连接它们的边；边也有层级：概念之间反复出现的规律（如“基线一经批准，变更须走受控流程”）是**核心原理**；再往上，领域中不可再被推导的法则，是**基本原理**——把根概念里本来就含着的道理，写成一句明白话。

> 术语注：一个概念是否含子概念，先讲父概念的内涵、把子概念作为其中一项，再讲子概念的内涵——本章 §1.2 正是这样做的。“根概念”四条判据、“基本原理”（root principle）为本方法论命名，不是任何标准的条文；“概念体系能力”是本书的操作化命名，源出 Katz 1955 的概念技能研究（见参考文献 R-51）。

为什么先抓根？因为它是源头：先立它，新知识有地方挂，后面的学习越来越省力（§1.5 有实验证据）。因为它管方向：它辨对了，往哪使劲才不错（§1.4.3）。还因为它能生成一片：把它立住，领域知识由它导出，换个场景也能重新展开（§1.7.4、§1.7.5）。还有一条时间账：根概念变动的速度远慢于外围知识，立得越早越省事，立错了则后面每一层都跟着错。

怎么算立住了？拿判定矩阵与演绎链过一遍，证明它是根；答全四问（是什么、不是什么、从哪来、往哪去）；再拿它给一个陌生领域立骨架。工具见 §1.7.3 与 §1.8.2。

这一问答完就是一句话：先说准源头上的那几个词，再展开骨架，最后填细节。

## 1.3 概念体系能力：本事不在懂得多

> 接上一问：谁来把词说准？——识别枢纽、织关系、迁移、辨边界，这几样合起来才是它。学习里最容易白费力气的地方也就在这两处：记了一堆点，却织不成网；学会了套路，却搬不到新场景。

### 1.3.1 挑枢纽、连关系、搬得走、划得清

前面讲了概念这张网的结构；现在讲在这张网上干活的能力：概念体系能力（conceptual ability），简称“概念能力”。冰链的边界划了两个月、恒信的方案评了三个月。这些工不是卡在方案上，是卡在这张网没人搭起来。

> **概念体系能力（conceptual ability）**：以概念为单位的压缩与组织能力——识别枢纽 × 织关系 × 迁移 × 辨边界。判据（三级，一级比一级硬）：转述测试（§1.5.3 又叫“压缩测试”；做到“建构”这一级，§1.6.1）＋辨伪测试（要有人来回对质才算）＋迁移测试（最高档）。

> 术语注：标准里另有一条“知识—技能—能力”的链（ISO 9000:2026 §3.10.4 知识／§3.10.5 技能／§3.10.6 能力）：

> 知识是使人能够做出有效的、依情境而定的决策与行动的领悟；技能是把一项任务做到规定期望的习得能力；能力是运用知识与技能达成预期结果的本领。那条链说的是人员能力由什么构成，本章说的是同一种能力练到多深——两把尺子，别并成一把。

它不是一样单独的本事，而是四样：

| 构件 | 做什么 | 反面对照 |
| --- | --- | --- |
| 识别枢纽 | 抓出决定其余部分的核心概念 | 被细节淹没，分不清主次 |
| 织关系 | 把概念连成有结构的网络 | 孤立记点，学一个丢一个 |
| 迁移 | 把网络搬到新场景 | 绑定场景，换个题就失灵 |
| 辨边界 | 用反例划清适用与失效范围 | 只举正例，不知何时失效 |

这四个构件合起来是什么成色，用转述、辨伪、迁移就能检验：

- **转述测试**（C 级，即“建构”那一级，§1.6.1）：陌生领域，三五句话讲清楚——讲不清，说明结构没成型（织关系没到位）；讲到卡壳处，说明边界没划清（辨边界没到位）。注意它的上限：讲给外行听只到“建构”这一级（§1.6.1），外行提不出有效质疑；
- **辨伪测试**（要到 I 级，即“互动”那一级，§1.6.1）：能举反例划清边界——举不出反例，说明只见过正例（辨边界没到位）；而且要**有人挑战**才算数，自己跟自己辩不算；
- **迁移测试**（最高档）：换一个情境，换行业、换粒度，你还认得出同一个结构吗？

> 概念体系能力强，不是“懂得多”，而是组织得好：同样的信息量，有人堆成一盘散沙，有人织成一张网。判断能力强弱，看的是这张网，不是信息量。

四构件之外，还有一个管方向的子能力，下一节专门说：**根概念能力**（root-concept ability）。

### 1.3.2 根概念能力：先抓源头，再展结构

四构件之外，还有一个管方向的子能力，它叫根概念能力（root-concept ability）。它是概念体系能力的一部分，做的是识别根概念、验证根概念、运用根概念。

说得再直白一点：它决定你先说哪个词。先从领域里挑出源头上的那几个词，再顺着它们把核心概念清单和彼此的关系铺开。四个构件负责“说出来之后怎么组织”，它负责“开口之前先站在哪儿”。

怎么算练到了？给你一个能当场试的判据：20 分钟，对一个陌生领域说出一组“根概念＋核心概念清单及其关系”，并且能为每一个候选说出理由。判定矩阵和演绎链就是用来辩护的，工具见 §1.7.3（候选清单的收集见 §1.7.2）。

不过这条判据有几处边界，先说清楚，免得你照着练却练歪：

- 20 分钟是练出来的，不是读出来的。第一次做，半小时能理出个头绪就算不错；
- 它是给陌生领域用的。对自己已经有底子的领域，一上来就跳枢纽，反而会让你把该看的细节漏掉；什么时候该换一种学法，§1.6.4 会讲；
- 它只管“先抓哪个词”，不管“拿主意”。把词说准和做决定，不是同一级能力；§1.4.4 会把这四级摊开给你看。

### 1.3.3 记忆可以外包，把话说准不能

这副能力在今天比过去更值钱，原因只有一句：**记忆与信息处理已经外包给机器**，剩下的识别枢纽、织关系、迁移、辨边界，才是你自己的活。

机器记得比你多、算得比你快，也写得出你没读过的句子；但它不替你判断“这个词在这里指什么”。恒信那场会就是现成的例子：“架构”在五个人嘴里是五件事，机器替你分不出谁在说哪一件。工具越强，方向与表征上的一次错被放得越大：这正是那条项项相乘、一处为零就全盘归零的说法（§1.4.1）。

一条边界同时立在这里：**AI 是好助手，不是判据**。你完全可以让它给候选清单，§1.7.2 就把“问 AI”列为收集候选的一条正路；但候选要自己用硬判据筛、用演绎链辩护。它能替你收集，不能替你拥有。

## 1.4 凭什么说回报最高：方向与表征

> 能力有了名字，接下来要问：它凭什么决定学习的成败？——因为学习产出是一个乘积，而它两头各握着一个能让结果归零的开关。

### 1.4.1 学习产出是一道乘法

为什么要给这副能力单独立一个名字？因为别的能力都没有它这个位置——它是学习能力的杠杆。章题这么说，本章就得论证到底：凭什么说把话说准回报最高？

先看这条式子：

> 学习产出 = 根概念能力（方向）× 概念体系能力（力臂）× 学习投入（施力）× 实践场景（支点）

它们的分工不同：方向决定往哪儿使劲，力臂决定同样的力气能撬动多少，施力是投入，支点是概念落脚的真实任务；后三样只负责放大，下一节逐个说。

关键在乘号。加号里的项可以互相补偿：这一项弱一点、那一项强一点，总分还过得去。乘号里不能：任何一项为零，整条式子就是零。同一门知识、同样多的时间，一个人先抓源头，一个人从细节啃起。过一阵子再看，差距不是努力拉开的，是方向拉开的。后者的方向那一项接近零，后面的力气全乘在零上。冰链和恒信那两场会就是这样：十张脸都在使劲，可各使各的方向。

这条式子因此叫**乘法律**。一错就让别的力气全白费的，却只有两个归零项：方向（先抓错了词，§1.4.3）和表征（话说得不准，§1.4.5）；其余都只负责放大——投入再多、场景再好，也只是把已经错了的东西放大。

这是“回报最高”的第一层理由：把话说准，是这条乘法律的两个归零项之一，归零项就是方向与表征这两根轴。它不是让学习好一点，而是决定学习有没有。而它同时是最便宜的一步：立枢纽只要 20 分钟，给一个词建一张概念档案只要一张纸（§1.8.4）。

“杠杆”这个说法，讲的是这条律的另一半——放大。所以章题没有说错，只是这个说法自带边界：杠杆放大的是力，却不分方向；枢纽选错，学得越快偏得越远；杠杆需要支点，概念不落真实任务，力臂再长也撬不动什么；杠杆自己不发力，没有投入，力臂无穷长也是零产出。

### 1.4.2 力臂、施力、支点：只负责放大

式子里的另外三项，性质不一样：它们只负责放大，不负责指方向。

- 力臂，就是概念体系能力。把枢纽连成关系、搬到新场景、划清边界；同样一份材料，会组织的人拿走的是结构，不会组织的人拿走的是一堆点；

- 施力，就是学习投入。时间、注意力、练习量，缺一样，力臂再长也不动；
- 支点，就是实践场景。概念要落在真实任务上才使得上劲；只记不用，前面两样也一起白费（§1.6.2 第 5 环、§1.6.3 迁移）。

它们有一个共同点：放大的都是已经确定的东西。方向对了，力臂越长撬得越多；方向错了，力臂越长偏得越远。冰链那边的力气和经验都不缺，施力、支点都在，卡住的只是最初的那一项。同一门知识，两种学法的差别就在这里：

| | |
|---|---|
| 先学核心概念（先立骨架） | 只背结论和招式（无框架） |
| 知识有地方挂：新概念迅速入网 | 碎片化记忆：学完就忘 |
| 举一反三：跨场景自由迁移 | 绑定具体场景：换个题就失灵 |
| 能辨伪：识别概念偷换 | 易被带偏：分不清概念边界 |

先立骨架再填细节，后面的学习越来越省力；先切入细节，越来越吃力。这不是经验之谈，§1.5 给出的实验会证明这一点。

### 1.4.3 方向：挑错了词，全盘皆输

四个部件里，最根本的不是力臂，是方向。为什么？

因为其他三个部件都在“放大”，只有方向在“指路”。放大是没有方向感的，方向一旦错了，后面每一份力气都加在错误上。压缩学习实验里，枢纽优先组的全部优势，正来自“先识别枢纽”这一下。其后 800 轮正式学习，都只是这个方向上的展开（§1.5）。

方向不是一次选定的，而是三层递进：

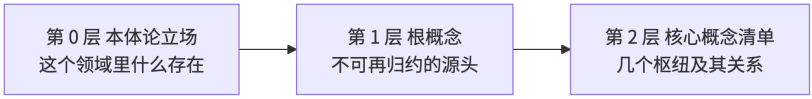



图 1-2

先问本体论问题：这个领域里，什么东西是存在的？再抓根概念：哪几个是不可再归约的源头？最后展开成核心概念清单及其关系。

第 0 层答错，上面两层再漂亮也是错的：把“产品”当成企业运行领域的根，与把“产品数据”当成根，导出的是两张完全不同的网（§1.7.5）。根概念能力就是把这三层都定准的那一下。

这就是本章标题的一半意思：概念体系能力是学习能力的杠杆，而杠杆的第一性是方向。所以本书反复强调“先立枢纽”——它不是一条技巧，是把方向握在手里的那一下。

一个澄清，防止误读：概念体系能力管的是学习的头一步和放大那一步，不是学习的全部。那条训练循环（立枢纽、织关系、逼输出、勤校准、落实践）的每一环都在练它，其中立枢纽与织关系练得最直接；逼输出、勤校准、落实践这三环还要配另外三样：把学过的拿出来用、随时检查自己哪里想错了、以及坚持下去。它决定学得多快、看得多深；那三样决定学不学得动、学不学得远。

### 1.4.4 一个词能说到多深：能力四级

方向选定了，接下来要问的是说得有多准。同一个词，能说到好几个深度；说得浅，等于没说。

本章后面有一场评审会，正好是这件事的例子。恒信的罗建把“架构”答了个滴水不漏：是什么、不是什么、从哪来、往哪去；可轮到“这个系统要不要拆”，他一个字也说不出来。不是他没准备，是他的能力只到第一级。

我们在大脑里操作的从来不是真实世界，而是它的代用品。你思考的不是经济活动本身，是那些概念和指标；你理解的不是历史，是某种历史叙事。能做多大的事，取决于手里的代用品长成了什么样。代用品本身有粗有细，粗到能判断，细到只是一个词。能力四级，就是

这四种粗细：表征、图式、心智模型、解释框架。它不是四种知识，是同一种能力的四档深浅：

| 级 | 是什么 | 一句话判据 |
| --- | --- | --- |
| ① 表征 | 现实事物在头脑中的代用品；最小单位——概念、名词、对象、关系和边界 | 能提取关键表征，并把它画成关系图，才算入门 |
| ② 图式 | 表征聚起来、认得出的一类结构；一眼能认出的套路 | 看事情一块一块看；一听就知道这是哪一出，还能预判下一步 |
| ③ 心智模型 | 会动的图式——内部有变量、有因果、有反馈、有边界条件 | 能推演“如果 A 则 B”，能说出它的失效边界在哪 |
| ④ 解释框架 | 对一整个领域的整体看法：把一堆模型和套路攒成的一整套 | 能判断什么值得相信、什么只是叙事包装 |

第一级是表征。“苹果”这个词就是表征；真实的苹果是一堆原子，你脑子里的苹果是红的、圆的、甜的。零散地记住一批词，不叫入门；抓住关键表征、认出它们之间的关系、画出图来，才叫入门。

第二级是图式。表征聚成结构，就成了图式：一看就知道这是哪一出，还能预判下一步。没图式的人看什么剧情都新鲜；图式多的人看什么都是套路。反过来说，恒信那五张“架构”的脸，连表征这一级都还没对齐——更谈不上图式。

第三级是心智模型。图式回答“这是什么”，心智模型回答“它怎么运转”。万有引力定律是一个心智模型，费曼“在积分号下求导”的绝招是一个心智模型；本书后面讲的每一条判据，说到底也是一个心智模型。

第四级是解释框架。它是当前某一派意见对一整个领域的系统性看法：回答的是“这一大类现象该怎么解释”。摸到这一级，你才开始

像行家那样理解世界：不是追故事，而是问机制；不是摆立场，而是比解释力。

> 术语注：前三级的说法在认知科学里是通行的；**第四级“解释框架”是课程作者的自命名**（原话：“这个词是我发明的”），不是标准术语——本书借用它的分法，读者不必把它当成定论。

四级共用同一种能力，只是粗细不同：识别枢纽，第一级是挑出最该先认的那几个词，第四级是认出“这一派把什么当枢纽”；辨边界，第一级是说清这个词不指什么，第三级是说出模型的失效边界。同一个构件，换一级来用，做的事是同一件。

本章的工具停在第一级。四问是第一级（表征）用的工具，管的是“把词看准”：这个词指什么、不指什么、从哪来、往哪去。这是最底层的一级，也是最便宜、最容易被跳过的一级，但它不负责拿主意。罗建的四问答得再好，也只是把几个词说准了——他还没有能推演的机制，更谈不上判断该不该拆。本章后面给出的判据，也都还在这一级；它们长成能推演的机制，要到后面几章。

这里有一个可以自查的崩样：

> 能把一个概念的四问答全，却在一个真实的取舍面前说不出话——这不是表达能力问题，是能力层级问题。

那上面三级谁来建？本书后面每一章都在建，只是你此前不知道它们各在哪一级：

| 级 | 每一级处理什么、怎么算做到了 |
| --- | --- |
| ① 表征 | 本章的四问（是什么／不是什么／从哪来／往哪去）与术语工作；第 3～5 章的领域概念——判据：能提取关键表征，并画出关系 |
| ② 图式 | 本章的三层体系与“织关系”；第 2 章的工程四层与对立面；第 6、7 章的结构与产品数据——判据：能认出结构，预判下一步 |
| ③ 心智模型 | 第 10～12 章（架构、架构设计、架构决策）——判据：能推演，能说出失效边界 |
| ④ 解释框架 | 第 17 章整合、第 18 章架构师——判据：能判断什么值得相信 |

所以这本书不是一个一个知识点，是同一种能力逐级往上练：先在表征这一级把话说准，再在图式这一级把结构认出来，然后建能推演的心智模型，最后逼近判断。

这四级里，前三级可以讲、可以写、可以画出来；第四级有一块是讲不出来的。有一种知识只能在现场长在身上：老师傅调火候、老工程师看一眼就知道哪不对，那不是天赋，是在现场待久了长出来的本事。本书教你把话说准、把结构理出骨架；但第四级里有一块，写不进书里。认下这条边界，比多背几条判据有用。

### 1.4.5 表征：第一级拿不到，上面全是零

“回报最高”还有第二层理由，就藏在这四级之间：

> **价值律**：**表征是 1，上面三级是 0**（本书命名。“1 和 0”说的是成不成立、以及量级，不是分数。）

先说“1 定成立”这一半：第一级决定的是**成不成立**，词指错了东西，上面三级再漂亮也是零。前言里那句话说的正是这一级：“地基

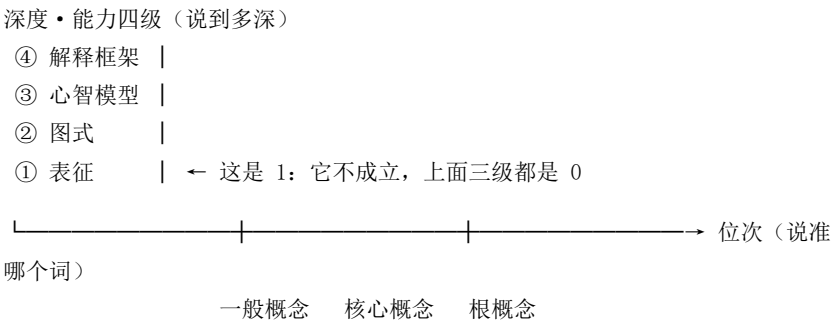
不准确，上面盖多少层都白搭”。本书反复讲的“先把话说准”，不是风格要求，是这条式子里最靠前的那个数。

再说“0 定量级”这一半：上面三级不是不重要，它们决定的是**能做多大事**：从“看准一个词”，到“看出一套套路”（图式），到“能推演”（心智模型），再到“能判断什么值得相信”（解释框架），每加一级，同样一份信息能做的事就翻着倍往上涨。**1 定成立，0 定量级**。这就是为什么第一级最便宜，也最不能省。省掉 1，后面全是 0；有了 1，后面每一级都实打实地加量级。

于是这条乘法律的两个归零项到齐了：方向（§1.4.3：先抓哪个词）与表征（本节：话说得准不准）。方向管往哪使劲，表征管拿的是不是那个东西；两者都过，才轮到力臂与投入去放大。这就是章题的完整意思——概念体系能力是学习能力的杠杆，因为它同时管住了这条律的归零项与放大项。

两个归零项，两根轴，合成一个坐标系：位次轴（一般 ⊃ 核心 ⊃ 根）回答“该抓哪个词”，深度轴回答“这个词你要说到多深”，它的刻度就是能力四级。它就是你量“说到多准、说到多深”的那把刻度尺。

下面这把刻度尺，量的就是“你说到多准、说到多深”：




两根轴各管一件事——一个概念可以位次不低、深度很浅，也可以位次不高、深度很深。罗建的“架构”就是前一种：位次上落在核

心概念，深度上只到表征。枢纽抓到了、话也答全了，可能力只到第一级。所以谈到任何一个概念，都要同时报出它的两个坐标。

> 术语注：位次轴上的“三层”（一般／核心／根）与深度轴上的“四级”（表征／图式／心智模型／解释框架）是两个量词，别混：论位次说“第几层”，论能力说“第几级”。

最后交代一句“概念”这个词。四级处理的东西，本书一律叫概念。这是本书自己的用法，比 §1.2.1 的严谨定义宽：概念是头脑里对现实事物的代用品；它在四级里各是一个样子：表征这一级是**现实事物在头脑中的代用品**，即概念、名词、对象、关系和边界；图式这一级是**认得出的一类结构**；心智模型这一级是**会动的图式**；解释框架这一级是**对一整个领域的系统性看法**。

> 术语注：严谨定义（ISO 1087:2019 §3.2.7 的知识单元、§1.2.1 的可指称＋可界定）管的是术语意义上最窄的那一层；四级通称“概念”，是本书的操作读法。

## 案例回场｜答得全，却说不出该不该拆

恒信的评审会上，方案要过一道关：合同系统要不要拆成两个。

罗建先把“架构”答了一遍。它是什么：一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）；它不是什么：不是一张架构图，不是一次技术选型，不是组织架构；它从哪来：从建筑学借入软件与系统领域，1970 年代结构化设计、1990 年代软件架构热，后由标准定稿；它往哪去：不可逆决策被做出之前用，变更影响评估时用。

四条一条不差。

顾言问了一句：“那这次拆不拆?”

罗建停了几秒：“……我得再看看。”

顾言没催他，把那份标准翻到术语章：“42010 §3.2 那条定义，答的是这个实体是什么。它管定义，不管决策。要判拆不拆，另有三件事：服务边界切在哪、数据归谁、拆完哪条链会断。这三件答不出来，方案写得再漂亮也过不了评。”

罗建把架构图合上：“这三件我一件都答不上来：我手上只有定义，没有判据。”

他不是没准备。他手上的“架构”停在第一级：词说准了，关系也画得出；顾言那三件事，一件都还没做。

**四问答得全，只是拿到了那个 1**

## 1.5 有什么证据：一项压缩学习实验

> 道理讲完了，该给证据。凭什么说“先立枢纽”不是经验之谈——2026 年北京大学团队的一项实验，证明了枢纽优先的学习结构为什么占优。

### 1.5.1 差距在压缩，不在记性

2026 年，北京大学心理与认知科学学院等团队在《自然·通讯》发表了一项研究。它给出了一个反直觉的答案：高手和普通人的差距，不在记性好，而在擅长“压缩”。所谓压缩，就是把全量信息转化成关键信号。

> 术语注：这是本书对实验的读法。实验直接测量的是“枢纽优先的知识结构更容易掌握”（§1.5.2）；从“结构更易学”引申到“高手与普通人的差距在压缩”，是本书的一步引申——它经得起检验，但不是论文的原话。

经验医生把成千上万种疾病的知识，压缩成几个关键信号；资深调查记者在堆成山的文件里，只盯着“异常”的部分。他们都在做同一件事：压缩学习。章首那十张脸反着来——十套没压缩过的说法，谁也接不住谁。

压缩为什么有效？机制在带宽上：工作记忆一次只能同时处理 4 到 7 个信息元素（常被引用的经验值；机制依据见 R-97 第 38 讲引的认知负荷理论）。学习失败的常见原因不是“存不下”，而是新信息过这道窄门时堵住了。

压缩的单位叫**图式**（schema，认知心理学里的既有概念，即§1.4.4 四级里的第二级）：把多个相关元素打包成一个整体，打包得越多，一次能过的信息就越多。北大团队这一步的贡献，是把“什么样的结构最容易被压缩”量了出来。

### 1.5.2 实验：先学枢纽的人更快

团队设计了几类结构相同、但连接方式不同的人工知识网络，找一批人来学。结果很清楚：少数枢纽占据大量连接的网络（无标度网）最好学；均匀对称、没有主次的结构反而难掌握。

另一个实验更有说服力。同一批知识，一组先学枢纽再学细节（枢纽优先），一组先学边缘再学枢纽（边缘优先），还有一组随机学。进入完全相同的正式学习后，枢纽优先组的优势随时间越来越大，而且扩散到整张网络，包括那些从没直接学过的纯细节节点。

脑成像也印证了这一点：枢纽优先的学习者在大脑里维持着更稳定的知识结构表征；先学边缘的人，早期也有表征，但迅速衰减。先搭框架的人，新信息进来只需判断“挂在框架哪个位置”；先切入细节的人，后期积压太多零散节点，整合不动。

### 1.5.3 把压缩用在自己身上

研究给了三步可以直接照做的动作。**找到枢纽**：学任何新东西前，先花 20 分钟找枢纽节点，翻书目录、读百科开头、问 AI、问行家都行；比如《思考，快与慢》的枢纽就是系统一、系统二、偏见。**建立联系**：先弄清枢纽之间的关系，再填细节，关系没通，细节学一个丢一个。**转述**：讲给完全外行的人听，三五句话讲清楚，就是压缩成功；转述是“压缩测试”，费曼技巧的机制正是它。

它们不是孤立的技巧，而是后面那条循环的头三环。§1.6 把它们接成一条完整的练法。

### 1.5.4 实验证明了什么，没证明什么

论文也有它的边界，每一条都要说清：

- **生态效度**：实验用的是抽象的知识网络，真实领域（教科书、标准、行业惯例）的结构复杂得多；怎么识别真实领域的枢纽仍是手艺活，论文没有替你解决——这正是 §1.7 要做的事：三步识别法＋判定矩阵；
- **测的是预测，不是应用**：范式测的是“预测下一张图片”这类隐含结构的学习，没有直接测应用与迁移。所以“转述能验证掌握”是本书补上的一步，不是论文的结论；
- **先有鸡还是先有蛋**：枢纽优先的前提是先知道枢纽在哪；面对完全陌生的领域，只能靠二手信号（目录、百科、专家、AI）先取候选，边学边修正（§1.7.2）。

另外，别把“框架形成后表征更稳定”过度引申成“框架不可改”。框架可改，见 §1.6.4 校准。

## 1.6 怎么练：一条循环转起来

> 道理和证据都有了，接下来是手上的活。这副能力不是天赋，怎么把它练出来——一条可以一圈一圈转的循环。

### 1.6.1 什么叫学会：会用才算掌握

先说“掌握”是什么意思。教育学界有一个公认的认知目标阶梯，叫布鲁姆分类学：知道 → 理解 → 应用 → 分析 → 评价 → 创造，六级。

大多数人的学习停在“知道”和“理解”：能复述，能听懂，考试能过；换个场景，就用不出来。分界线画在“应用”上：会用才算掌握。你背得出“质量＝客体的固有特性满足需求的程度”，不算掌握质量；你拿着它评一份验收标准、判断一次分歧算不算质量问题，才算掌握。恒信评审会上顾言那句“那这次拆不拆”，问的正是“会不会用”。

还有一条更细的分界：同样一段时间，参与的方式决定收效。被动听讲（P）最弱，主动复述（A）强一些，**建构（C）**才算真学习，能生成材料里原本没有的新信息；能与人来回对质（互动，I）则更强。这条次序说明了：为什么“讲给外行听”只是入门线（外行提不出有效质疑），以及为什么第 3 环要配第 4 环（§1.6.2）。

### 1.6.2 循环：从立枢纽到用起来

掌握专业知识有一条循环，就叫**五环模型**：立枢纽 → 织关系 → 逼输出 → 勤校准 → 落实践。每转一圈，骨架更准、网络更密、输出更顺：

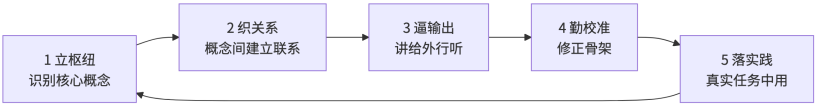



图 1-3

| 环节 | 做什么 | 为什么（依据） |
| --- | --- | --- |
| 1 立枢纽 | 学新领域前先花 20 分钟识别枢纽概念：翻目录、读百科开头、问 AI、问行家 | 压缩学习：先学枢纽，好处会扩散到整张网；一上来要记的东西最少（§1.5） |
| 2 织关系 | 弄清枢纽间关系（包含/依赖/对立）再填细节 | 知识是网络不是列表；记忆靠连接检索（§1.2.1） |
| 3 逼输出 | 三五句话讲给外行、写成文档、做题、教别人 | 自己回想一遍，比再看一遍记得牢（测试效应）；一讲就露馅，缺口自己冒出来 |
| 4 勤校准 | 随时准备推翻自己：枢纽选错、关系画反就改 | 想错的地方，得用更准的说法顶掉（概念转变） |
| 5 落实践 | 在真实任务中用：做项目、审文档、解真问题 | 只记不用的知识是死的（惰性知识）；真本事＝概念结构×在场景里练出来的经验 |

这张表里最容易被跳过的是第 3 环和第 4 环，而它们恰好是**刻意练习**的要件：目标明确到能判对错、反馈快到能马上改。没有反馈的重复不叫练习，叫重复：这也是“讲出去”和“勤校准”要挨着放的原因。恒信那场会上，顾言那声“停”就是一次现成的反馈，可惜没人接住（§1.8.5）。

掌握 = 用结构消化信息，用输出验证结构，用实践把结构磨出来。学习不是把知识装进脑子，而是长出结构，让新知识有处安放。

### 1.6.3 练的是哪一样，卡在哪一环

五环的每一环都有名字：立枢纽 → 织关系 → 逼输出 → 勤校准 → 落实践。能力不是天生的，是练出来的。这不是说天赋不存在，而是说天赋与练习是乘法关系：天赋决定每次更新的利率，练习决定你有没有在更新。

专家与新手之间，差别也出在这两样上。一部分在正确性标准的分辨率，专家能分辨出新手看不出的差别；一部分在内化了的身体图式，手艺长在身上，说不出来但做得出来。都不是天赋，是路径。

怎么练？这条从立枢纽到落实践的循环，本身就是练法，四个构件在五环里各就各位：

- 识别枢纽 → 第 1 环“立枢纽”：四构件的第一个，就是五环的第一步；
- 织关系 → 第 2 环“织关系”：把概念连成网，正是织关系构件；第 3 环“逼输出”讲不清，说明结构没成型，也是织关系没到位；
- 辨边界 → 第 3 环“逼输出”：讲到卡壳处，就是边界没划清的地方；第 4 环“勤校准”：用反例推翻旧骨架，也是辨边界；
- 迁移 → 第 5 环“落实践”：换场景用，正是迁移构件。

所以五环不是五件事，是四构件在循环里转。每转一圈，四构件都更准一点：这是概念体系能力最实在的训练路径，也是“能力是练出来的，不是天生定死的”这句话怎么落到做上。

第 5 环“落实践”最常失效的地方在**迁移**。从学到用，中间有三道锁：**表征锁**是学的时候就没抓住结构，只记住了字面；**触发锁**是知道该用，但那个场景里想不起来用；**生态锁**是真到了现场，环境不给你用的条件。

对应的两个策略很朴素：**拥抱**，主动把新知识往自己已有的场景里塞；**桥接**，拿旧场景里的经验去接新知识。迁移测试（§1.3.1 第三级）量的就是表征锁、触发锁、生态锁这三道锁开没开。（这三道锁是课程对迁移研究的提炼，本书沿用。）

### 1.6.4 校准：概念是共识，不是真理

五环的最后一根柱子，是第 4 环“勤校准”。它凭什么立得住？因为概念是共识，不是真理（本书底色：反绝对真理）。

化学的“燃素”、物理的“以太”，都曾是各自时代的核心概念，后来被推翻。库恩把它叫做“范式转换”：每个时代的枢纽清单，是那个时代认知的天花板，不是终点。

所以，学核心概念，就是花最小的力气，把这一行几百年攒下的共识接过来；但它是共识，不是真理：枢纽是起点，不是天花板。这也正是“勤校准”的意思：连领域公认的枢纽，都可以怀疑、修正。

对管方向的概念体系能力来说，这条尤其要紧：方向不是一次定死的，要一路校准。你今天立的枢纽，明天可能被证明是错的。那不是失败，是这条乘法律在正常纠偏。枢纽清单是活的，方向就永远是活的。

最后一条边界：先立枢纽不是一刀切。对完全陌生的领域，先花 20 分钟找枢纽最划算；对已经有基础的领域，一上来就跳枢纽反而会压住细节：同一种学法，在新手身上有效，在行家身上会失效。练的时候还要挑**可取的困难**：在自己扛得住的范围内跟材料较劲（先自己回想再翻书、把话说给别人听），而不是跟疲劳和干扰较劲。

# 1.7 怎么找：挑出枢纽与根概念

> 说准哪个词、凭什么、怎么练，都答完了。落到手上还有一问：具体怎么找到那个词？——三步识别法、判定矩阵与演绎链，最后走三个真实领域。

## 1.7.1 概念是点，原理是线

找枢纽，先分清两个层级：

- 核心概念 = 节点（名词）：领域的“词汇”，如需求、配置、基线、版本；
- 核心原理 = 边/规则（一句判断）：概念之间反复出现的规律，通常用“当……必须……”“……取决于……”表达，如“基线一经批准，变更须走受控流程”“验证以规定需求为参照”。原理是更高级的枢纽——掌握原理，概念自动串联。

原理也有层级：核心原理之上，是基本原理，也就是原理侧的源头（见 §1.2.3）。

## 1.7.2 怎么挑出枢纽

找核心概念（枢纽），可以顺着三步走：

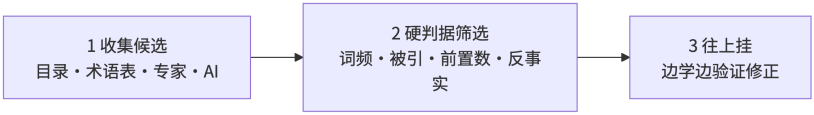



图 1-4

第一步，收集候选，借现成的材料，20 分钟就够。领域共同体已经帮你压缩过，直接“抄”就行：

| 来源 | 为什么有效 |
| --- | --- |
| 教材目录/章节标题 | 好教材的顺序本身是枢纽优先的（先基础后应用） |
| 专业术语表 Glossary | 共同词汇的浓缩，出现即被共同体认可 |
| 课程大纲/综述论文 | 专家设计并经多年检验；综述是高手压缩好的结构 |
| 问 AI/问专家 | 直接要“10 个核心概念及其关系”，再交叉验证 |

收集阶段宁多勿少：判据要求“20 分钟给出根概念与核心概念清单及其关系”，那是收敛之后的答案；收集时先要“10 个”，多出来的正好用来交叉验证、往上挂。

第二步，硬判据筛选，用枢纽的定义去验证：

- **词频**：领域文本里被反复提到的名词，多半是枢纽；
- **被引用度**：统计概念定义中被引用的其他概念，被引用最多者最基础——比如“需求”的定义，被整个质量体系引用；
- **前置数**：理解它需要的前置知识越少，越像枢纽——数一数“要讲清它，得先讲几个别的概念”；
- **反事实检验**：问一句“如果这个概念/原理不存在，这个领域还成立吗?”经济学少了“供需”，整个框架就塌了——它是枢纽。反事实检验是四条里最狠的一条。

> 术语注：前三条是**代理判据**——用可数的量代替真正要判的东西，方便，但会骗人。本书早期草稿就写错过一次：把“被引用度＝出度、前置数＝入度”当成枢纽的判据（出度是它连出去的关系，入度是连进来、指向它的关系），那测的其实是**根**（被全部依赖），不是**枢纽**（连接最多）。真要判谁是根，回到**根概念的四条判据**与判定矩阵（§1.2.3、§1.7.3）。

第三步，往上挂。每学一个新概念，都问一句“它挂在哪个枢纽下?”挂不上去，说明枢纽清单有漏洞。清单是活的，随理解加深增删；专家与新手的区别，不是清单长度，而是清单准不准。

### 1.7.3 判定矩阵与演绎链

三步识别法给出的是核心概念清单（骨架）；要辨出其中哪几个是根概念（承重柱），还得让每个候选过两道关：先横向打分，再纵向论证。根概念 ⊂ 核心概念，正好对应 §1.2.2 的包含关系。

识别根概念不能靠感觉，要用两件能查、能对质的工具：**判定矩阵**（横向打分）与**演绎链**（纵向论证）。

判定矩阵：四条判据（原子性／生成性／不可替代性／递归性）× 候选概念，逐格打分并附证据：

| 候选 | 原子性 | 生成性 | 不可替代性 | 递归性 | 判定 |
| --- | --- | --- | --- | --- | --- |
| 候选 A | ✅/⚠/❌ + 一句证据 | …… | …… | …… | 根 / 派生 / 子类 |

演绎链：把结论写成能一条条查的论证链。每个前提（P）附出处，结论（C）由前提逐条推出：

P1：候选 = 三步识别法收集的候选清单（§1.7.2）
P2：根概念 = 不可再归约的源头概念，四条判据判定（§1.2.3）
P3：判定证据 = 标准族 / 工程现实（可核验）
P4：候选 X 通过四条判据的判定矩阵（见上表）
——→ C：X 是该领域的根概念

> 演绎链的意义：把“我觉得是”变成“因为 P1、P2、P3、P4，所以是”——每一环都可以被反驳、被替换，结论因此可辩护。这正是根概念能力判据里“为每个候选辩护”的完整操作。

### 1.7.4 根概念长什么样：阻抗

这两件工具落在真实领域里长什么样？先看两个领域，各用几句话走完。

高速信号传输，根是阻抗。说起高速信号，你多半先想到速率、带宽，但它们都是结果，不是根。信号一上高频就失真：反射、串扰、衰减、损耗、时序，根子都在阻抗。

用四条判据核验：阻抗由传输线本身的特性（线宽、介质、间距）决定，不依赖域内其他概念，这就是原子性；特性阻抗、反射、串扰、损耗、时序都由它派生，这就是生成性；去掉它，信号完整性领域不成立，这就是不可替代性；从芯片引脚到背板到系统，层层都生效，这就是递归性。所以这个领域的根概念是阻抗（1 个）。

产品开发，根是需求、系统、设计开发。很多人以为根是“质量”，但质量是需求被满足的程度，是派生的；也有人以为是“产品”，那只是一个交付物的名称。三个源头各答一问：需求回答“要什么”，系统回答“承载在什么上”，设计开发回答“怎么把它一级一级说到可实现”。恒信要评的改造方案、冰链要分的 M3 二代，说到底都挂在这三个词上。

质量、功能、接口、组件、架构，都从这三者派生；四条判据逐条同样过：都不可再定义、都生成一片、去掉一个领域就失去立足点、从产品级到模块级层层生效。所以产品开发的根概念是需求、系统、设计开发，共 3 个。

### 1.7.5 根概念长什么样：产品数据

第三个领域，走一遍完整的判定：它是本书的主场。

企业运行体系的根概念是产品数据。为什么不是“产品”？设计与制造之间传递的不是实物而是模型与图纸，质量与合规证明的是数据

记录，产品的“身份”就是它的数据档案：在企业运行层面，产品以数据形态存在。

更反直觉的是，根也不是流程。企业运行不就是跑流程吗？但流程由“数据的流转次序”定义：它不原子；去掉流程，数据仍在，只是无序。流程只是产品数据按时间排出来的样子；同理，组织是它按空间摆出来的样子，绩效是它按价值算出来的样子；配置管理管的是产品数据的受控状态，IT 系统是它的数字化载体：投影可再造、可重组，都不是根。

真正过得了这四条判据的是产品数据：自身即事实，原子；一切由它派生，生成；去掉它企业只剩空壳制度，不可替代；概念级、流程级、实例级都生效，递归。

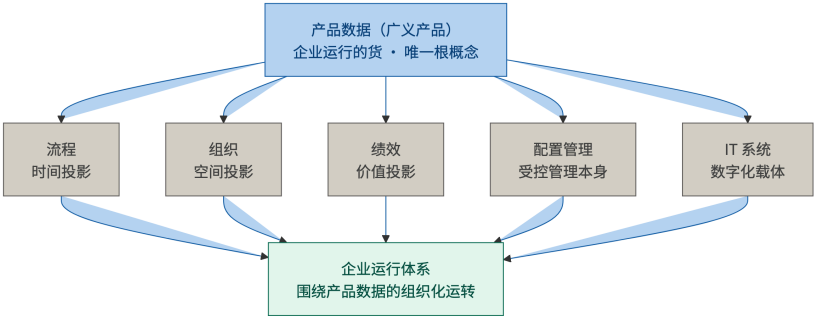



图 1-5

> 图注：流程、组织、绩效是产品数据在时间、空间、价值三个维度上的投影；配置管理、IT 系统是它的受控管理与数字化载体——投影可重组，产品数据这一个根不变。

所以企业运行体系的根概念，是产品数据（1 个）。这个域的核心概念则是流程、组织、绩效这三个投影，以及被反复引用的需求、质量、治理；它们都由产品数据派生，都不是根。

注意：同一个词换个领域，位置就可能变。需求在产品开发域里是根，在企业运行域里只是核心。产品数据这根承重柱，第 7 章会把它展开为“开发产品就是开发产品数据”。

三个示例是本书按四条判据判定的结果，供对照，不是任何标准的条文。

## 1.8 怎么验：答不全，就只是见过

> 最后回到手上最容易含糊的一步：一个词说到什么程度，才算说准了？——一把尺子（四问法）、几种最常见的“没拿准”（词汇病），以及要查全时去哪儿查。

### 1.8.1 假懂长什么样

怎么才算真正说准了一个枢纽概念？先看“假懂”长什么样：能背原文，却举不出反例；不知道它何时失效；孤立记住，不知道它挂在哪个枢纽下；不关心它从哪来；只在考试里答得出来。

真懂反过来：能用自己的话重述，能举正例也能举反例，知道它不是什么、何时不成立，知道它挂在哪个枢纽下、与谁冲突，知道它解决什么问题、为何被发明，讲给外行能懂、换个情境能用。

理论依据是维特根斯坦的一句话：“意义即用法”（meaning is use）。概念的意义不在字典里，在它的用法里。理解一个概念，就是会用它，而不是会背它。按这条看，章首那十张脸各说各的“架构”，说的确实是十个不同的概念：词是同一个，用法各是各的。**但十种用法不等于十个架构**——同一个实体只有一个架构；用法多出来的那些，是**关注面与描述**，不是多出来的架构（第 10 章 §10.4.4）。

### 1.8.2 怎么问才算问全

怎么判断自己真懂了？把上面那些差别压成四个问题：

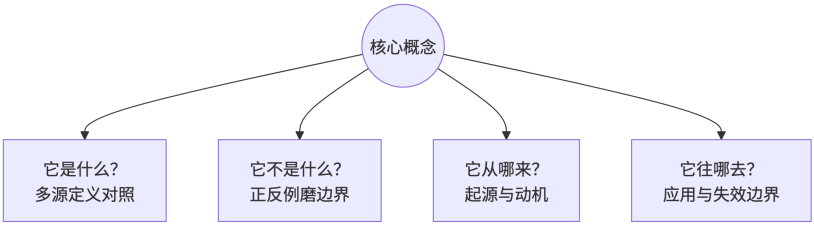



图 1-6

| 问 | 操作 | 理由 |
| --- | --- | --- |
| 它是什么？ | 找三四个来源（标准、教材、百科、高手著作）对照定义，差异处即本质处 | 定义的差异暴露概念本质；同一个词，标准与教材表述不同，差的那一点往往就是本质 |
| 它不是什么？ | 找正例和反例，用反例磨边界 | 边界靠反例定型；概念靠典型例子活，靠反例定型 |
| 它从哪来？ | 回到概念被创造时的问题现场：解决什么问题？没有它之前人们怎么受苦 | 知道它当初解决什么问题，才记得住；词源能让它活起来 |
| 它往哪去？ | 挂在什么流程、被谁使用、何时失效 | 用得越熟越知道适用条件；掌握失效边界＝深度理解 |

这四个问题都答全，才算拥有它；只能答第一个，只是见过它。这背后，是本书底色的反记忆崇拜——懂不等于记住。大多数人的“懂”，其实是“见过”：背得出定义，却答不出它“不是什么”的边界，也答不出它“往哪去”。

同一道门槛，本章量过三次：答全上面这四问，是给这个词做体检；把关键表征画成关系图，是把它摆进网里（§1.4.4）；三五句话讲给外行听，是压缩测试（§1.3.1）。三次都过，才算把它提到第一级。

问这四个问题之前还有一步，专门管词的身份，叫**术语锁定三问**：① 这个词出自哪部标准的哪个条款？② 那部标准的术语章里有没有定义它？③ 没定义的话，它是按通用语意用的吗？“概念”就是个现成的例子：它在 ISO 1087:2019 §3.2.7 有正式定义，在 ISO/IEC/IEEE 42010 里却是按通用语意使用的（§1.2.1 的两个“指称”之争就是这么来的）。词的身份没锁住，那四个问题会问在错的对象上。

拿章首那个被吵翻天的词练一遍。量的是本书认定的那个所指，也就是系统架构：

- 它是什么：一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）；
- 它不是什么：不是一张架构图（图会过时，架构的基本概念持续起作用）；不是一次技术选型；更不是组织架构、产品形态；
- 它从哪来：从建筑学借入软件与系统领域；1970 年代结构化设计、1990 年代软件架构热，后由 ISO/IEC/IEEE 42010（现行 2022 版）定稿为系统与软件工程标准；
- 它往哪去：在不可逆决策被做出之前用；在变更影响评估时用；在团队边界定义时用。失效边界——当组织架构、产品形态混进来时，它就说不清了。

答完四问，章首那两场吵架的解法已经出来了：他们吵的是各自不同的“它”。四问先帮他们分清在谈哪一个，再谈要不要改。

## 案例回场｜先分清在谈哪一个

回到章首那两场会。这一次，屋里有人带了四问：它是什么、不是什么、从哪来、往哪去。

冰链那边，林涛先说自己指的是**产品架构**（他关心的是模块怎么分）；石军说他指的是**硬件架构**（传感器、主控、通信怎么布）；吴浩说他指的是**系统架构**（服务边界与数据所有权）。三个人把各自的四问答完，才看清说的是三个不同的所指——其中只有系统架构是本书认定的（同一个实体只有一个架构，第 10 章 §10.4.4）；另两个是同一个系统的不同**关注面**。先分清在谈哪一个，再谈要不要改。

恒信那边同样：郑亮说的是**组织这个实体的架构**；罗建、王默说的是**本系统架构的两个关注面**；苏婷要的是**接口那一张视图**。四问答下来，每个人都说清了自己指的那一件；先谈哪一个、后谈哪一个，也就排得出来了。

散会时，郑亮把那叠审批单往前一推："三个月，十一场会，方案一页没改。这三个月不是花在方案上，是花在'架构'这一个词上。下一回先把这个词对齐，再谈改哪一页；省下的是返工，不只是时间。"

**没有人先把"在谈哪一个"问出来**

### 1.8.3 词汇病：每种都有药

四问法为什么必要？因为"见过"的样子是有规律的，最常见的那几种，合起来叫"词汇病"：

| 病 | 症状 | 例子 |
| --- | --- | --- |
| 病一 说不出定义 | 用例子代替定义，问不到判据 | "评审就是开会过一遍" |
| 病二 同词异义 | 同一个词，各指各的概念 | 章首的"架构" |

| 病 | 症状 | 例子 |
| --- | --- | --- |
| 病三 翻译伪朋友 | 英文原词不同，中文翻译形近 | 评审 / 验证 / 确认 |

- 病一，说不出定义，只能举例。例子是插图，不是定义：定义回答“它是什么”，例子只回答“它长这样”。要判断“一次不走流程的聊天算不算评审”，例子答不了，判据才答得了。治病的药是给定义，而且定义要带可检验的判据——这背后是本书的**判据主义**。“评审”的定义是“为达成既定目标，对客体的适宜性、充分性或有效性所作的确定”（ISO 9000:2026 §3.11.2），判据便是：有没有得出结论？有没有留下记录？
- 病二，同词异义，各指各的。同一个能指（词），挂在不同的人身上，就是不同的所指。这正是章首“架构”吵架的机制。两个来源说的是同一件事的不同侧面：索绪尔的能指与所指、弗雷格的符号／涵义／指称。词还会漂移：“架构”从建筑学漂到软件、组织、数据，每次借用叠一个新所指。漂移无害，不对齐才可怕。治病的药是把词和概念分开——不谈“架构是什么意思”，改谈“你说的架构，指称的是什么”。
- 病三，翻译伪朋友。几个英文原词完全不同，中文翻译后听起来像一类。本书第 8 章的主角评审（review）、验证（verification）、确认（validation），在 ISO 9000 里的定义动作词不同：评审用“确定”（determination，§3.11.1）定义，验证与确认用的是“证实”（confirmation）；“证实”没有自己的条款号，在这两条定义句里以句首词出现，另有 §3.11.7 的 metrological confirmation，那是计量领域的另一个词。中文都带“评/验/认”像近亲，其实不是。治病的药是回英文原词、查词根、看定义的动作词。

冰链和恒信那两场会，病二占了大头；这本书每章章首的“误解现场”，大多也落在这几种词汇病里。比词更深的还有“乱用”，概念本身没认错，却用在不该用的场景，那是“往哪去”没答全，见 §1.8.4。

### 1.8.4 查全：概念档案

这四问好记、也够用，但它记住的是四个方向，不是一张可以逐项打勾的清单。要查全，本书另备一样东西，叫**概念档案**：书末附录 B 给每张核心概念配一张档案卡，把四问展开成固定的几栏，一栏一栏填过去，漏在哪一栏一目了然。

全书约 45 张核心档案，其中 33 张完整版收在书末附录 B，其余散见各章。恒信那场会如果按这张档案过一遍，“架构”的几栏里一定有空着的。空着的那栏，正是他们卡了三个月的地方。

一条使用规则，必须遵守：档案不是每一栏都必须填，薄栏可省，但不可被跳过。跳过的栏，往往是误解潜伏的地方。本书各章给概念“钉死定义”，用的都是这张档案；你自己给手头的概念建档案，也照它来。

### 1.8.5 认知反转

回到章首那两场会。

其实十张脸都没有错。冰链的五张脸，产品、硬件、云端、测试、制造，各有各的立场；恒信的五张脸，组织（另一个实体）、系统、接口、部署、点醒，也各有各的道理（其中只有**系统架构**是本书认定的）。错的是这两场会本身：一场没有对齐概念的会议，注定是一场各说各话的表演。

如果当时有人带着四问，会议会是另一个样子：

> “等一下，我们说的'架构'**不是五件不同的事，是同一个架构的五张面孔**。先各自答四问：它是什么、不是什么、从哪来、往哪去？——**产品架构看模块边界，硬件架构看元件布局，系统架构才管服务边界和数据所有权**（组织架构则是另一个实体）。分清在谈哪一面，再谈要不要改。”

本章的认知反转，比“一场各说各话的表演”更深一层：你不是不懂那个词，你是还没拥有它。能把“架构”背出来、能画出架构图，都只是“见过”它；能回答它是什么、不是什么、从哪来、往哪去，才算拥有它。四问答不出后两问的人，和那两场会里的十张脸，站在同一条起跑线上。

顺手把一个更深的念头也放进你的口袋。如果连“架构”这么核心的枢纽词，都只是“见过”而非“拥有”，那你脑子里其他天天用的词呢？你真正拥有几个？位次上，你追到根了吗？深度上，你是刚把词说准（表征），看得出套路了（图式），能推演了（心智模型），还是能判断什么值得相信了（解释框架）？

两个坐标一起报，才是完整的答案。本章的工具，把你送到第一级。

一个在造温控箱的工程世界，一个在做审批系统的设计世界：同一种能力量过来，同一个词都是五张脸。从第 2 章起，每章帮你把一个枢纽概念从“见过”变成“拥有”，这是第一级的活；上面那三级，从第 6 章起接着建。

---

## 小结

本章不给你一个新概念，给你一套本事：概念体系能力，学习能力的杠杆。

说到底就两句话。一是挑对该先说准的那个词：先找根，再铺骨架；二是把这个词从“背得出”说到“用得上”：把话说准是那个 1，能推演、能判断是它后面添的 0。

这两步看着便宜，却最不能省：词挑错了、说歪了，后面花多大力气，都只是在把错的东西放大。

从第 2 章起，我们用这套坐标去量第一个枢纽概念，工程：工程是什么，不是什么，以及为什么“用敏捷 + CI/CD”不等于“在做软件工程”。

## 附录

### A 本章概念总图

主张：把话说准，是学习能力里回报最高的那一步

├── 一个现场 ──────────────── 一个词，让两个项目停了工；词可以当场对齐，概念要花功夫拥有 §1.1

├── 先说准哪个词 ──────────── 概念是一张网：一般概念 ── 核心概念（枢纽）── 根概念（源头） §1.2

│ └ 为什么先抓根：它是源头（新东西靠它挂）/ 方向（管往哪使劲）/ 能生成一片 / 立得越早越省事

├── 概念体系能力 ──────────── 识别枢纽 × 织关系 × 迁移 × 辨边界；判据分三级 §1.3

│ └ 子能力：根概念能力（先抓源头，再展结构；20 分钟立骨架是练习目标）

├── 凭什么说回报最高 ── 一条乘法律：方向 × 力臂 × 施力 × 支点；两个归零项：方向、表征 §1.4

│ ├── 放大项：力臂、施力、支点——只放大，不指方向 §1.4.2

| ├── 归零项一·方向：先抓哪个词（第 0 层本体论立场 → 根概念 → 核心概念清单） §1.4.3

| ├── 深度轴：能力四级（表征 → 图式 → 心智模型 → 解释框架） §1.4.4

| ├── 归零项二·表征：表征是 1，上面三级是 0；位次轴 × 深度轴＝坐标系 §1.4.5

| └── 证据：压缩学习实验——枢纽优先的优势随时间扩散到整张网 §1.5

├── 怎么练 ——— 一条循环：立枢纽→织关系→逼输出→勤校准→落实践；迁移有三道锁 §1.6

├── 怎么找 ——— 三步识别法（收集候选→硬判据→往上挂）＋判定矩阵 / 演绎链 §1.7

└── 怎么验 ——— 四问法（是什么 / 不是什么 / 从哪来 / 往哪去）＋术语锁定三问 §1.8


## B 本章的能力级

**第 ① 级为主**

- **表征**——把六个词说准：概念／核心概念（枢纽）／根概念／基本原理／概念体系能力／根概念能力（书末附录 B 六张卡）；用的工具是四问＋术语锁定三问
- **图式**——给出两个可识别的结构：三层包含（一般 ⊃ 核心 ⊃ 根）与能力四级（表征 ⊃ 图式 ⊃ 心智模型 ⊃ 解释框架），以及两轴合成的坐标系（§1.4.5）
- **心智模型**——给出三个能推演的机制：三步识别法＋判定矩阵／演绎链、乘法律与价值律、五环模型
- **解释框架**——本章不建此级；由第 17、18 章承担

> 能答全四问，才算拥有；只能答第一问，只是见过。记住这条次序：表征是 1，上面三级是 0 ——1 定成立，0 定量级。

## C 自查 · 这些说法对吗?

1. “概念是专业术语，背熟定义就算懂了。”——**错**。懂＝能答四问、能用、能迁移；背熟定义只是“见过”。
2. “概念能力＝知识渊博，懂得多能力就强。”——**错**。组织得好才关键；知识多不等于会抓枢纽。
3. “概念体系能力＝根概念能力，两个词是一回事。”——**错**。概念体系能力是全集（四构件）；根概念能力是它的子能力（方向盘：先抓承重柱，再展骨架）。
4. “杠杆的力臂越长越好，投入越多产出越大。”——**错**。乘号里有两个归零项：方向与表征。方向错了，力臂越长偏得越远；表征没拿到，上面盖多少层都白搭（§1.4.3、§1.4.5）。
5. “先学细节再搭框架，殊途同归，反正最后都学全了。”——**错**。压缩学习：先立枢纽，后面的学习越来越省力；先切细节，越来越吃力。
6. “把概念讲给外行听是浪费时间。”——**错**。转述是压缩测试，讲不清＝结构没成型，正好暴露缺口。
7. “四问法答出'它是什么'就够了。”——**错**。能答全四问才算拥有；答不出“不是什么/往哪去”＝只是见过它。
8. “核心概念一旦记住就不用再改。”——**错**。概念是共识不是真理，枢纽是起点，随时可修正（勤校准）。
9. “掌握一个领域＝记住更多信息。”——**错**。学习不是往脑子里堆信息，而是先搭骨架，再让输入各归其位。
10. “识别枢纽靠天赋，没有方法。”——**错**。有可操作的三步识别法：收集候选、硬判据筛选、往上挂；辨根概念还要过判定矩阵＋演绎链。
11. “核心概念都是科学家发明的。”——**错**。三种出生方式：创造/发现/凝结；多数是共同体长期实践“酿”出来的。

12. “枢纽就是最重要的那一个词，记住它就够。”——**错**。核心概念是骨架（通常 3~5 个）；之上还有根概念（承重柱通常 1~3 个）；还有原理一层。
13. “根概念就是最重要的概念。”——**错**。根概念是最“源头”的概念（原子/生成/不可替代/递归），未必最“重要”——质量很重要，但它不是根。
14. “概念只有节点，没有边。”——**错**。概念是节点、原理是边；核心原理之上还有基本原理（§1.2.3），掌握原理，概念自动串联。
15. “先把图式建起来更要紧，表征差一点没关系。”——**错**。表征是 1，上面三级是 0：1 定成立，0 定量级；1 没拿到，上面盖多少层都白搭（§1.4.5）。
16. “能答全四问，就说明我这一级已经修满了。”——**错**。四问是第一级（表征）用的工具；上面还有能推演的心智模型、能判断的解释框架（§1.4.4）。答全四问是**入场**，不是到顶。

## D 练习

1. 用四问法量你工作里天天用的一个词（架构/需求/质量/评审……）——哪一问最薄？为什么最薄的那一问，恰恰是你最容易出错的地方？量完再给它报个坐标：位次在哪一层、能力练到第几级？
2. 用三步识别法找出你领域的核心概念：收集候选（目录/术语表/专家/AI）、硬判据筛选（词频/被引/前置数/反事实）——“如果没有它，这个领域还成立吗？”再试着用判定矩阵辨出其中哪几个是根概念。
3. 把某个概念讲给一个完全外行的人，三五句话讲清楚。卡在哪一步？暴露了什么缺口？
4. 用五环模型给最近一次学习做个体检：你停在第几环？下一环是什么？为什么卡在那？

5. 用四问法重开章首那两场会：十张脸对“架构”各持所指，哪些是同一个所指、哪些不是？怎么让工程团队和设计团队先各自对齐，再谈别的？
6. 你领域里哪个概念是“凝结”出来的？它的共同体是谁？哪个是“发现”或“创造”的？
7. 你有没有“曾经坚信、后来修正”的概念？是五环哪一环让你改的？
8. 用乘法律给你的学习做个诊断：方向（根概念能力）、力臂（概念体系能力）、施力（学习投入）、支点（实践场景）各缺什么？两个归零项——方向与表征——哪一项你更没底？
9. 拿你最近学到的一个概念做一次**迁移测试**：换一个行业、换一个粒度，这条判据还成立吗？认出“同一个结构换了张脸”了吗？（§1.4.4 四级里，这是从图式走向心智模型的那一步。）

（答案要点见书末附录，与训练教材题卡同源）

---

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**


- [1] **R-01** ISO 1087:2019 —— 概念定义依据（§3.2.7：“由特性的独特组合而形成的知识单元”）
- [2] **R-04** ISO/IEC/IEEE 42010:2022 —— 架构定义（§1.8.2 演示、概念档案·架构）

- [3] **R-08** ISO 10007:2017 —— baseline 由军品工程实践固化（凝结三方式举例）
- [4] **R-38** ISO 9000:2026（第 5 版）—— 评审/验证/确认（§3.11.2 / §3.11.12 / §3.11.14，伪朋友辨析）

**经典文献**

- [5] **R-43** 索绪尔《普通语言学教程》—— 符号=能指+所指；语言（langue）与言语（parole）
- [6] **R-44** 弗雷格《论涵义与指称》(1892) —— 符号→涵义→指称
- [7] **R-45** Xiangjuan Ren 等《Compressive learning scaffolds higher-order network structure to enhance human knowledge acquisition》（Nature Communications, 2026, DOI: 10.1038/s41467-026-75843-7）—— 压缩学习实验
- [8] **R-46** Bloom 教育目标分类学（Anderson & Krathwohl 修订版, 2001）—— 五环掌握线“会用才算掌握”
- [9] **R-47** Roediger & Karpicke (2006) —— 测试效应
- [10] **R-48** Posner 等 (1982) —— 概念转变
- [11] **R-49** Whitehead《The Aims of Education》(1929) —— 惰性知识
- [12] **R-50** Lave & Wenger《Situated Learning》(1991) —— 情境学习
- [13] **R-51** Katz《Skills of an Effective Administrator》(1955) —— 概念技能（“概念体系能力”的管理学源头）
- [14] **R-66** 维特根斯坦《哲学研究》(1953) —— 意义即用法（假懂 vs 真懂）
- [15] **R-67** 库恩《科学革命的结构》(1962) —— 范式转换（共识 ≠ 真理）

- [16] **R-68** Kahneman《思考，快与慢》(2011) —— 系统一/系统二（“找枢纽”的读者参照）
- [17] **R-78** Miller《The Magical Number Seven, Plus or Minus Two》（Psychological Review, 1956） —— 工作记忆容量的经典文献（“7±2”的来路）
- [18] **R-97** 万维钢《现代思维工具 100 讲》（得到 App 课程，第 38—47 讲） —— 工作记忆带宽与压缩单位、能力四级（表征→图式→心智模型→解释框架）、参与度分级、迁移三道锁、默会知识边界；本章“4 到 7 个”这一口径与压缩单位“图式”的说法，依据都在第 38 讲

# 第 2 章 工程：让产品开发可重复

## 2.1 引言

本章先用一场季度会把问题摆出来，再把本章要回答的问题说清楚，最后给出一副整章的骨架。看完这三段，本章后面在争什么、凭什么说“有流程、有工具”还不算在做工程，也就清楚了。一路答下来，一条能当场用的判据就落到你手上：换个人、换团队，同样的需求与约束，还能不能同等交付。

### 2.1.1 误解现场

恒信集团 IT 部的季度会，上半年最后一天。

系统架构师罗建站在投影前，语气里有压抑不住的兴奋：

> “各位，我们敏捷转型成功了。双周迭代、CI/CD 全绿、自动化测试覆盖率 80%。按 IEEE 610.12 那三个词——systematic、disciplined、quantifiable——我们一条不缺。我们已经是工程化团队了。”

会议室里响起稀稀拉拉的掌声。这个团队过去一年上了 CI/CD、拆了微服务、引入了敏捷站会，每一步都踩在教科书的说法上。

产品开发经理曹峰接过话头：“系统上线了，指标也全绿——我们交付的东西跑起来了，工程就算成了。”

运维王默低头看了一眼手机。监控上，月末高峰的曲线又拱了起来：今天是上半年的最后一天，也是本月最后一个工作日。他想起上个月那场事故，想说什么，又忍住了。

只有需求分析师顾言，在掌声落下去之后，低声问了一句：

“那……上个月上线那次事故，是为什么?”

罗建愣了一下：“那是人手不够，临时赶工。跟工程化没关系。”

顾言张了张嘴，没再问。

如果你坐在这间会议室里，你大概也会点头。大多数人对“工程”这个词的直觉，和罗建一模一样：有流程、有工具、有测试、有敏捷，就是在做工程。而罗建那句“事故跟工程化没关系”，恰恰是本章要推翻的东西。上个月的事故，几乎可以肯定是“不是工程”的证据，而不是“工程之外”的意外。

本章要做的，是先回答一个最基础的问题：我们到底在做什么?是工程，还是包装成工程的手艺? 这两个词的区别，决定你交付的东西能不能“换个人也交得出来”。

---

### 2.1.2 本章要回答什么

读完本章，你应该能回答：

1. 一句定义能不能把“工程”说准? 定义里的每一样各管什么? 为什么约束不是麻烦，而是定义的一部分?
2. 不同年代、不同领域的权威定义，凭什么走到同一个内核上?
3. 换个人、换团队，还能不能同等交付? 这条判据测的是能力高低，还是可复制性?
4. 为什么工程没有单一对立面? 科学、手艺、艺术各自站在哪个维度上?
5. 工程不只是方法论。“敏捷 + CI/CD + 测试”占了其中哪几层? 缺的会出什么事，为什么价值与伦理那一层最致命?

6. 系统化和算法化是一回事吗？自动化程度越高，是越接近工程，还是越远离工程？
7. 系统工程和软件工程是什么关系？工程系统和技术系统呢？“系统上线了”算不算工程成功？（“工程师”头衔为什么不等于在做工程，第 9 章专章回答。）

---

### 2.1.3 本章的骨架

本章要立一个主张。工程是把交付从人身上挪到方法上的一次转移，挪得走的是方法，挪不走的是判断。

> **工程 = 把“做成一次”变成“谁来做都成”。**

这件事分两半。挪得走的那一半是原理、流程、工具、标准，也就是工程科学、方法论、工具与实践。而工程之所以是工程，靠的是工程科学、方法论、工具与实践、价值与伦理这四层同时在场。挪不走的这一半是每一步上的判断，价值与伦理守的就是这条挪不走的线，所以工程不能被算法替掉。

本章要说清的是这么几件。先给“工程”一个定义，再给一条能当场用的判据：换个人、换团队，还能不能同等交付。然后把工程和最容易混的那些东西分开：工程不等于方法论，缺了哪一层，就有一样东西重新落回某个人身上；系统化也不等于算法化，挪不走的那部分就是判断。最后交代验收看什么：工程系统包含技术系统，成不成，由“被用起来”说了算。

用第 1 章的话说，罗建只是“见过”工程：他背得出那三个形容词，却答不出“它不是什么”。有流程、有工具、有敏捷，唯独没问过：这套做法离了人，还成不成立。

§2.2 钉“工程”这个词本身，§2.3 把它和容易混的东西分开，§2.4 落到怎么用：从造部件到造整体，从上线到被用起来。

---

## 2.2 工程的定义：每一样都不能少

> 这一节把“工程”看清：先立一句定义，再把定义里的每一样拆开看，看不同年代的权威机构怎么走到同一个内核上，最后把全章反复使用的那条判据交给你。

### 2.2.1 先给工程一个定义

“工程”这个词用得太滥：信息工程、管理工程、社会工程，什么都能挂上“工程”二字。但在系统工程、软件工程、硬件工程里，它有一个很硬的共同内核，可以用一句话说完：

> **工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西。**

> 术语注：这句定义是本书对 ECPD、ABET、INCOSE、IEEE 610.12 几份权威定义的**凝练**，不是任何标准的原文，原定义见§2.2.3；本书的判据与命名同样是作者主张，欢迎检验、欢迎校准。
>
> **先说清“挪不走的”指什么**：指的是**判断**，每一步取舍上都要用的那个动作，从工程科学到价值与伦理层层都有。方法能把工程科学、方法论、工具与实践挪走，这个动作始终留在人身上。

这句定义里，每一样都各司其职。注意它的结构：科学原理 × 判断创造 是“料”，约束 是“框”，系统方法 是“路”，有用且安全 是“果”。缺了“料”是空谈，缺了“框”是失控，缺了“路”是碰运气，缺了“果”是自娱自乐。后面每一节，都是它们的展开。

### 2.2.2 逐条拆开：各是什么，各管什么

① 科学原理 × 判断创造——地基。工程是应用原理，不是发现原理。物理定律、数学模型、算法理论，是工程的依托。没有力学定律，土木工程师只是个会搭积木的人；没有算法理论，软件工程师只是在瞎调参数。这一要素回答了“为什么这样能行”。

那个“×”不是装饰：判断创造是把原理用活的动作，包括选哪个模型、怎么取舍、何时偏离标准做法。它不单独占位，却贯穿每一步，下一项“约束”里会再见到它。

② 约束——精髓。成本、工期、资源、风险、法规、物理极限。注意一个反直觉的事实：约束不是工程的附加项，是定义的一部分。1941 年 ECPD 那份最早的工程定义里，就同时写着“运行经济性”和“生命财产安全”两条。一个没有约束的“创造”不叫工程，叫发明或艺术。

没有约束，就没有判断的空间。工程里的每一个决策，本质都是“在约束下做取舍”：成本有限，所以要权衡；工期有限，所以要排序；安全有红线，所以不能让。**约束越多，判断越值钱**。反过来也要防过头：无意义的限制、纯官僚式的流程不产生判断价值，那只是限制，不是约束。

③ 系统方法——过程。需求 → 设计 → 实现 → 验证 → 维护。它保证工作可计划、可度量、可复现。可计划，才知道每一步花多少资源；可度量，才知道每一步做没做对；可复现，才谈得上“换个人也能交付”。这几样缺一样，方法就只剩墙上一张流程图。

这也是本章主张里“挪得走”的那一半：方法可以写成文档、变成流程、交给工具，让别人接住。

④ 有用且安全的人造物——目的。一切为了交付能用的东西。工程不追求“有意思”，追求“有用且安全”。再看“人造物”这个词：工程交付的是人造的东西，不是自然的东西；光“有个想法”不算数。想法是发明或艺术的起点，只有被约束、被系统方法造出来、对人有用的，才是工程的产品。这也是为什么工程可以复现、可以检验，而灵感不能。

空气动力学之父冯·卡门（Theodore von Kármán）有一句话常被引来，说工程和科学的分工：

> **“科学家研究现存的世界，工程师创造从未存在的世界。”**
> Scientists study the world as it is; engineers create the world that never has been.

这里还有一条更深的推论要交代：判断不能写全成规则，因为判断就是在约束下做取舍，而约束一变，写好的规则就失效。把“哪种工况要降额”写进对照表，换了供应商、遇到新工况，表就罩不住了。所以“完全可复现”不是工程的极致：纯可复现是算法（见 §2.3.4），工程是“可复现的方法 × 不可复现的判断”。**方法是挪得走的，判断挪不走**，它只能长在人身上。

定义里的每一样都不是论文里的装饰，是可以逐条打勾的诊断清单。恒信怎么打这个勾，见案例进展①。

### 2.2.3 权威定义：不同年代，同一个内核

“工程”有多个国际权威机构的正式定义，年代不同、领域不同，却走到了同一个内核上。

ECPD（美国工程师职业发展委员会，ABET 前身，1941）：> 创造性地应用科学原理，设计或开发结构、机器、装置、制造过程……着眼于预期功能、运行经济性以及对生命财产的安全。

ABET（1979，现行认证标准）：> 将通过学习、经验和实践获得的数学与自然科学知识，运用判断力加以应用，以经济地利用自然的物质与力量，造福人类。

IEEE 610.12-1990（IEEE 标准术语，词条是软件工程）：> The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.（把系统化的、规范的、可度量的方法应用于软件的开发、运行与维护——也就是把工程应用于软件。）

INCOSE（国际系统工程学会，2019）：> 系统工程是一种跨学科、整合性的方法，运用系统原理以及科学、技术和管理方法，使工程系统的成功实现、使用与退役成为可能。

把 ECPD、ABET、INCOSE 三家的说法并排看，共同的东西就浮出来了：科学原理是知识基础；判断创造是把原理用活的动作；约束权衡是取舍的范围；造福目的是“有用且安全的人造物”；系统方法保证这件事可复现。这不是巧合，是“工程”这个概念的稳定内核。IEEE 610.12 那条说的是软件工程，只从软件这一侧贡献了“系统方法”这一面。

> 出处：以上四家定义见 R-15。其中 INCOSE 那一条的英文逐字见 ISO/IEC/IEEE 15288:2023 §3.50：ISO/IEC/IEEE 15288:2023 §3.50 收了这一定义，原文作 transdisciplinary and integrative approach to enable the successful

realization, use, and retirement of engineered systems using systems principles and concepts and scientific, technological and management methods，来源条注明 INCOSE-TP-2020-002-06。**正本另有中译**，作“一种跨学科、综合性的途径……使工程化系统得以成功实现、使用和退役”；本节所引的中译取 INCOSE 2019 的转述版。（IEEE 610.12-1990 同），故一律带年代引用，不写成“标准说”。

术语注：IEEE 610.12 定义的是**软件工程**，不是“工程”本身——它的原话是“把工程应用于软件”（the application of engineering to software），把“工程”当作已知概念引用。严谨的表述应该是：“据 IEEE 610.12 对软件工程的定义，工程化的过程表现为……”（它的三个形容词 systematic / disciplined / quantifiable，即“工程化的过程特征”，见 §2.3.4）。这个出处的分辨，正是第 1 章“翻译伪朋友”式的严谨：**同一个词，出处不同，所指不同**。

**概念卡片 · 工程**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西 |
| **它不** | ⑥⑦ | 对立：手艺（方法论维度）· 科学（认识论维度）· 艺术（目标维度）——工程没有单一对立面；辨析：工程不 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **是什么** | | 等于方法论（方法论只是其中一层）、工程不等于科学（创造对发现） |
| **它从哪来** | ②③⑨ | 工业时代大规模建造中“凝结”出来的实践——铁路、桥梁、工厂撑不住单靠个人手艺，共同体把造东西的经验压成可教可查的专业；演化：1941 ECPD 首次成文定义 → 1979 ABET → 2019 INCOSE，三家几十年间走到同一个内核上；1990 年的 IEEE 610.12 另从软件这一侧贡献“系统方法”一面；engineering ← 拉丁 ingenium（天生的才智、巧思） |
| **它往哪去** | ④⑤⑧ | 回答“如何把经验变成可复现的创造”；在团队交接、招聘、复盘时用试金石量它；误区 ✗ “有流程有工具 = 在做工程”（缺了哪一层都不算）✗ “工程 = 写代码 / 画图纸”（载体不是判据） |
| **判据** | 试金石 | 换个人、换团队，用同样的需求和约束，能交付质量相当的成果——能，是工程；不能，那是手艺 |

顺着“判断创造”这半个要素再往下走一步。ABET 还给 engineering design（工程设计）下过定义：为系统、部件或流程设计出满足需要的形态，一个**想出办法**（devising）、**反复定夺**（iterative decision-making）、**创造性、开放式**的过程。

第 3 章 §3.2.5 会把这句话展开：构思打开可能性，定义终结漂浮，设计开发才是转换到底的那一段。“判断创造”里的“创造”，只是第一棒。那里还有一处要留意：这里说的“设计”是广义的，构思、

定义都算在里头；等构思、定义、设计并排站的时候，“设计”要退到下游的狭义上，而三者的先后是逻辑上的蕴含，不是日历上的三步。

至于“创造”算不算“创新”，那是另一把尺子：够新又成了，才追认创新。

> 术语注：这句对 engineering design 的表述，是本书对 ABET 工程设计定义的凝练，不是 ABET 原文逐字（见 R-15）。

### 2.2.4 交付能不能离开某个人

定义里的每一样、权威机构的每一份定义，最后都指向同一条可检验的判据。这条判据，本书叫它**试金石**，全章反复使用：

> **换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果？能，是工程；不能，那是手艺——哪怕它碰巧也用了代码、图纸或电路。**

试金石可以在这些场合直接使用：

- 团队交接时：“如果我走了，这套流程还能跑吗？”答不上来，就是手艺；
- 招聘时：判断一个人带来的是“方法”还是“经验”，方法可传播，经验只在本人；
- 复盘时：这次交付能复现吗？复现不了，就是一次性的手艺成功，不是工程能力。

试金石的妙处在于它测的不是能力高低，而是可复制性，而这正是工程与手艺的分野。它也是第 1 章反事实检验的落地：去掉那个人，交付还成立吗？

## 案例进展①｜会后打勾：恒信缺了哪一样

季度会散会后，几个人还留在会议室里。顾言把定义里的那几样写在了白板上。

“我按 §2.2.2 那几样，挨个打一遍勾。”他说，“科学原理——他们的'为什么'有依据吗？容量评估是工程科学，砍掉它等于砍掉第一层。约束——他们没有识别'月末高峰'这个性能约束，等于没有约束可守。系统方法——CI/CD 在。有用且安全的人造物——系统上线了，可新流程没人用。”

运维王默把一摞监控曲线摊在桌上，指着每月末冲顶的那几段：“这条我最有发言权——曲线每个月最后几天都会冲顶，我报过好几轮，可没人把它当成约束来守。头一年我还较真，把报警写成整改建议，追着立项；罗建说'先记着，忙完这阵'，忙完就没下文了。后来我就只发数据，不再追了。我那时候以为，把监控做好、把数据准时发出去，就算尽到了运维的责任。”

顾言问他：“你报上去的是什么？”王默一愣：“报警记录。”

“报警只是数据。”顾言说，“约束要有人识别、有人命名、有人立项，写成一条有人守的规则才算数——按这条判据，你递了三年数据，约束一次也没递上去。”

王默想了好一会儿，点头：“所以我报的是数据，不是约束。我今天才知道，一条没人管的数据，等于没提。”

业务方的郑亮也接了茬：“没人用我最有体会——法务那边问电子签怎么走，财务那边问台账接不接，业务部嫌系统绕。上线是上线了，用起来是另一回事。你们说的技术系统，我承认它做完了；可我这边的账是：没人用，就等于没交付。”

---

## 2.3 工程常被误当成什么

> 这一节把“工程”和它容易混的东西分开：科学、手艺、方法论、机械化，每个挨着的词都变成一句可检验的判据。

### 2.3.1 工程没有单一对立面

一个反直觉的答案：**“工程”没有单一对立面**，它的对立面取决于你站在哪个维度看。维度不同，对手就不同：

| 维度 | 对立面 | 问什么 |
| --- | --- | --- |
| 认识论 | 科学 | 发现世界 vs 创造世界 |
| 方法论 | 手艺 | 可复现 vs 依赖个人 |
| 目标 | 艺术 | 满足用户 vs 表达自我 |

认识论维度：工程 vs 科学。科学问“世界是什么”，工程问“应该造出什么”。科学家的失败可贵（推翻假说），工程师的失败是灾难（垮了就垮了）。这一对最容易被混淆，因为“科学”听起来比“工程”高级。但工程不是科学的应用附录，它们是两种目标：一个求真，一个求有用。

目标维度：工程 vs 艺术。艺术为表达自我，功能让位于审美；工程为满足用户，一切决策向可用、可靠、可维护汇报。两者的分界线不是“好不好看”，而是“为谁服务”。梵高的画不供人“可用”，但工程师造的桥，是要被人走的。**好不好看是加分项，能不能用是及格线**。

理解“工程没有单一对立面”，就不会再用“有没有用代码”来判断“是不是工程”。

### 2.3.2 不是手艺：靠人，还是靠方法

这一对最实用，因为它直接给出“你在不在做工程”的判断：§2.2.4 的试金石，在这里第一次用起来。

手艺靠个人经验与技巧，结果依赖人；工程靠流程、标准、文档、评审、测试，换个人也能交付。注意：手艺不是贬义词。一个经验丰富的老工程师本身就是最值钱的资产。但团队的能力和工程的能力是两回事。手艺依赖具体的人，工程依赖可复现的方法。你团队里最值钱的那个人走了，交付还能不能维持？能，你有工程；不能，你有手艺。

## 案例进展②｜石军的手艺，恒信的流程

冰链科技的嵌入式硬件工程师石军，做了几十年温控产品。他的车间里没有一份完整的测试方案，但他“心里有数”，哪个批次容易出问题、哪种工况下要降额、哪家供应商的电容不可靠，全在他的脑子里。

去年石军请了两周假，温控箱的一个批次出了问题，产线愣是没人能定位——因为“知道该查哪里”的只有石军一个人。石军回来，十分钟找出问题。

这就是手艺：结果依赖具体的人。石军在，质量在；石军不在，质量跟着走。

制造侧的小周看得更直接。他管产线，最怕的就是这种“人一走、线就停”：“试金石这一问，我们车间天天在答——这条线的工艺参数，是写在作业指导书里，还是只在老师傅手上？写在纸上的，换班照样跑；只在人手上的，他一休假我就得陪着熬夜。这跟谁手艺高没关系，是本事存在谁身上的问题。”

冰链的系统工程师小张管的是整条链：箱体能不能造、能不能验、交不交得出去，都从她那儿过一道。她把石军那些“该查哪里”逐条

抄下来，抄得下去的写进清单，抄不下去的几条圈出来，旁边注一句“只在人身上”。她拿着那张清单问了一句：

“这张单子换个人拿着，还能不能做出同样的判定？能，它是方法：为什么这么判、依据写没写下来，都查得到；不能，它是手艺：每一次都得回来问同一个人。”

现在把镜头切回恒信。罗建的团队上了 CI/CD、有了测试，这算工程了吗？还不一定。如果这套 CI/CD 跑起来靠的是张三调试的三条隐藏命令、李四记得的两个环境变量、王五没写进文档的部署顺序，那它仍然是手艺，只是手艺的载体从“脑子”换成了“脚本”。工具不会自动把人从手艺变成工程，可复现才会。

工程化不是要把石军的手艺废掉，而是要把他的判断显性化成可教的规则、可查的文档、可检验的标准：把个人能力变成组织能力。这才是试金石的意思：它测的不是“你牛不牛”，是“你不在的时候，团队还牛不牛”。

### 2.3.3 不是方法论：方法论只是其中一层

先分三个容易混的词：**方法（method）**是“一条具体做法”，比如“排序用快速排序”；**方法论（methodology）**是“关于怎么组织工作、怎么选方法的系统学说”——瀑布模型、敏捷、CMMI；**工程（engineering）**比前两个都大——方法论只是它的一部分。工程不等于方法论。

工程是一个完整的人类实践，从上往下立着这么几层：

```
第四层 价值与伦理   为了什么？安全 / 可靠 / 负责     ← 最容易被忽略的一层
第三层 工具与实践   用什么做？CAD / 编译器 / 测试链
第二层 方法论       怎么组织？需求 → 设计 → 实现 → 验证 → 维护
第一层 工程科学     依据什么？原理 / 模型 / 定律
```

每一层都是一道把交付从人身上挪走的关。缺了哪一层，就有一样东西重新落回某个人身上：

- 缺工程科学：没人能回答“为什么这样能行”，只能靠某位老师傅的直觉，别人学不走；
- 缺方法论：做法只能师徒相传，人走了做法跟着走；
- 缺工具与实践：方法论悬在半空，落地全靠个人技巧；
- 缺价值与伦理：压力面前松不松手，全靠某个人的良心。

恒信那个“先上再说”的接口就是现成的样子：容量评估被砍，缺的是工程科学；变更没有控制、没有基线，“受控”这一项不及格（§2.3.4）；为进度让掉可靠性还不留记录，缺的是价值与伦理。缺的两层加上这一项，事故就来了。前三层缺了是“不专业”，第四层缺了是“会出事”。

前几层挪走的是“本事”，价值与伦理守的是“底线”。这里要跟 §2.2.1 说过的“判断”分开放：判断是每一步取舍上的动作，四层里层层都有；底线是判断不可越过的那条线（安全、可靠、负责）。一个是动作，一个是界限。正因为判断始终留在人身上，那条线才需要有人守着。

底线挪不到方法上，但这不等于只能靠良心，它可以用“让它看得见”来管住：把每一次妥协摆到台面上。下一节那几条判据，就是从这里来的。

回头看 §2.2.2 那句定义：科学原理 × 判断创造是“料”，其中科学原理对应工程科学那一层；系统方法是“路”，展开成方法论和工具与实践；有用且安全是“果”，正是价值与伦理守住的底线；约束是“框”。约束和判断这两样，都不单独占一层：约束是每一层做取舍时的范围，判断是每一层做取舍都要用的动作。定义说的是工程由什么构成，层次说的是它怎么立起来，两套说法说的是同一件事。

敏捷不等于软件工程。回到章首罗建那句话的意思：敏捷、CI/CD、测试齐了，就是在做软件工程。现在可以精准地指出它错在哪：敏捷、CI/CD、测试，只落在方法论和工具与实践这两层上。

而工程还有两层没兑现。先看工程科学：你的“为什么”有依据吗？性能压测的结论能复现吗？再看价值与伦理：赶进度时，安全需求会让位吗？罗建的团队，方法论和工具与实践这两层是新的；工程科学、价值与伦理那两层是旧的。只讲方法论不讲伦理，那不叫工程，叫“有组织的赌博”。

事故就出在这一层。为什么说价值与伦理最致命？因为每一次重大工程事故背后，几乎都不是方法论出错，而是这一层失守：

- 泰坦尼克号：号称“不沉之船”，救生艇数量按“不会沉”的假设配，为成本和进度牺牲了安全裕量；
- 挑战者号：O 形环在低温下会失效，发射前工程师就提出过反对，管理层也知情。但为了发射窗口和压力，管理者把风险“合理化”了。技术和检查都在场，缺的只有这一层。
- 波音 737 MAX：MCAS 系统（自动防失速）的设计缺陷，在进度与市场的压力下没有被认真对待，安全需求让位。（自动化怎样压缩了飞行员夺回控制权的余地，见 §2.3.4。）

> 出处：这三起事故的经过据公开报道与研究文献整理，库内没有一手调查报告。737 MAX 那两起坠毁的经过，可参 **JATR《Boeing 737 MAX Flight Control System》（2019 年 10 月提交 FAA）**、**印尼 KNKT 狮航 JT610 最终调查报告（2019）**与 **NTSB 安全建议（2019-09-26）**。

这些事故的共同点：不是“不会做”，是“做的时候没守住”。怎么判断一次妥协是失守，而不是客观原因？问三句：利益相关方知不

知情？有没有评审？有没有记录？客观原因不会专门留记录，失守恰恰发生在“没有记录”的地方。这是本章第一样能当场打勾的东西。

## 案例进展③｜有工具，为什么还不是工程

上个月那场事故，第一个被惊醒的是运维王默。

月末高峰，监控曲线冲顶，报警响了——数据库连接池耗尽，合同审批系统宕机。王默第一时间扩容、重启、回滚，把系统救回来，花了四十分钟。法务和财务几百个审批卡住。

他翻到那个新上线的接口，心里一沉：“这个接口，我从没见过它的容量基线——工程科学这一层，我们等于没做。”

顾言事后把上线日志翻出来，找写接口的开发苏婷核对。苏婷盯着那行提交记录，没有回避：“评估在计划里排过，可月底要上，罗建说先上再说。我当时也想着，上线后再补来得及。”她顿了顿，又补了一句：“结果，没来得及。方法论那一层我们没缺，缺的是压住进度的那一层。”

“你们没有容量评估、没有压测，我连它正常水位是多少都不知道，”王默说，“出了事我只能重启、回滚、赌一把——这不是救火，是撞运气。”

顾言把根因钉死：“按 §2.3.3 说的那几条对一遍：利益相关方知不知情？没有。有没有评审？没有。有没有记录？冲刺计划上写过一行，后来又划掉了。一条不占。这就是价值与伦理这一层失守，不是客观原因。”

他接着说：“敏捷、CI/CD、覆盖率 80%，这些都在。可一个团队工程化没有，不看它的流程有多完整，看它在压力面前会不会为质量让路。这个接口是罗建在站会上说了一句'先上再说'就上的——那句话的分量，比任何一层都重。”

罗建那句“跟工程化没关系”，正好说反了。事故恰恰证明：这个团队有工程的方法论和工具，还没有工程的价值与伦理这一层。

### 2.3.4 不是机械化：系统化不等于算法化

所谓工程化，说的是工程做起来是什么样。上一节讲的是工程由哪几层构成；这一节把它落到“怎么做”上。IEEE 610.12 的软件工程定义里用了三个形容词，给“工程化”画了像：

- **systematic（系统化）**——做事有章法；
- **disciplined（受控 · 有纪律）**——守规矩、受约束；
- **quantifiable（可度量）**——有证据、可检验。

这三样里，本节先说系统化。

> 出处提醒：这三个形容词出自 IEEE 610.12-1990 对**软件工程**的定义，所以本节讲的是“工程化的过程表现”，不是“工程的定义”。严谨的说法应该是：据 IEEE 610.12 对软件工程的定义，工程化的过程表现为……。**本书把这几样过程特征当尺子用，但它不是工程本身的定义**，工程本身的定义见 §2.2.3 的 ECPD、ABET、INCOSE 三家（IEEE 610.12 只答软件工程这一侧）。这条分辨见 R-14。

但“系统化”这个词，在中文里最容易翻乱：systematic（系统化）和 systemic（系统性）两个同源却不同的词，中文都译成“系统的”；而 algorithmic（算法化，也就是“机械化”）又常被当成“系统化”的本义，以为系统化就是把做法固化成算法、交给机器自动跑。它们各回答各的问题：

| 词 | 含义 | 回答的问题 |
| --- | --- | --- |
| **systematic（系统化）** | 做事有章法：步骤清晰、方法规范、可检查可复现 | 怎么做事 |
| systemic（系统性）／systems thinking（系统思维） | 看问题有全局：关注部分间关系、涌现行为、与环境交互 | 怎么看问题 |
| algorithmic（算法化） | 确定性执行：输入相同、输出必相同，无需判断 | 能否自动执行 |

> 出处：三行辨析是本书对用词的整理，无单一外部出处；系统化与系统性的分野另见 R-14 对软件工程过程特征的说明。三处与原报告的出入须说明。一处是前两行的“含义”格系改写，原报告作“过程有条理——步骤清晰、方法规范、可检查可复现”与“视角看整体——关注部分间关系、涌现行为、与环境交互”，后者语出同报告 §6 末句。另一处是原报告两行所带的括注，第一行是“(IEEE 定义用词)”、第二行是“(INCOSE 定义的落脚点)”，此处没有引。**第三处是“可重复”的释义**。05 号 §6 释作“换人执行结果仍稳定”，本书按附录 C 的两条分工改。**成色·可重复＝过程本身能重复执行**（本节表、§2.3.3、§2.4.5 三处同口径）；而“换执行者仍稳定”归系统化自问那一问。

systematic 是“做事有章法”，systemic 是“看问题有全局”。好的工程两者都要：用系统思维发现问题，用系统化方法解决问题。顺带统一说法：本章说的“机械化”，指的就是 algorithmic（算法化）。

举个 systemic 的例子：恒信的合同审批系统，如果只用系统化的眼光盯住“审批流跑通了”，就会漏掉“法务、财务、业务三家的数

据定义对不上”，而后者得用系统性的眼光看整体才看得见。系统化解决“这一步对不对”，系统性解决“整个系统对不对”。本书第 10、11、12 章（架构）就是 systemic 的主场。

那系统化（systematic）本身到底是什么？拆到底，它有这些可操作的特征：

- 有结构：分解为清晰步骤与阶段；
- 有章法（即“有方法”，含判断）：每步有标准与检查点，但不是机械执行，留判断的位置；
- 可重复：过程本身能重复执行；
- 全覆盖：从需求到退役，无遗漏环节。

反面是即兴发挥：无结构、无章法、不可复现、有盲区，想到哪做到哪，同一问题每次处理都不一样。

怎么判断一个流程是系统化还是装饰？问三句：每一步有明确的输入和输出吗？每一步有可检验的检查点吗？换个执行者，结果还稳定吗？都答“是”，才是系统化；答不上来，那只是一张流程图。

拿这几句去照同一个“给软件加新功能”的例子。非系统化的做法是：想起需求直接开写，写一半发现要改数据库，顺手改了表结构，没测旧功能，上线后报表挂了，连夜回滚，三个月后没人知道为什么多了个字段。系统化的做法是：需求写清楚并评审，评估影响范围，设计评审，编码加单元测试，集成测试回归旧功能，预发布验证，上线，记录变更。

前者每一步都没有输入输出、没有检查点、换个执行者就抓瞎；后者每一步都有明确的输入、输出和检查点。同样是加一个功能，一个越加越乱，一个越加越稳，这就是系统化和非系统化的差别。

顺带把“系统化”的误解磨掉：系统化不是流程繁琐，是可检查。系统化的本质不是“多”，而是每一步都有输入、输出、检查点；每个

决策都可追溯；整个活动可复制。它管的不只是“做没做”，而是“做没做对、能不能证明做对了”。流程是系统化的载体，可检查才是系统化的目的。

恒信那个没做容量评估就上线的接口，见案例进展③，非系统化也能上线；但只有系统化能让你回答“改了什么、为什么、影响谁、怎么验证”。

系统化不等于算法化。“系统化”就等于“有结构、有算法、可重复、全覆盖”吗？这几样里只有“有算法”站不住，该换成“有章法（有方法，含判断）”：算法是确定性指令，输入相同输出必相同，无需判断；工程方法有章法，但每一步都留着判断的位置，材料公差、需求变化、成本权衡，没有唯一答案。

更准确地说，系统化不等于按固定程序走：它有流程，但流程里留着判断的位置。

系统化的目的，从来不是消灭不确定性。需求会变、环境会变、人会犯错，工程面对的世界从来不是确定性的。它做的是把不确定性纳入可控范围：用检查点、评审、测试、反馈回路把这些变化管起来。敏捷就是典型：流程完整，但每一步都依赖人的判断，高度系统化，却一点谈不上算法化。

工程化的完整图景如下。系统化只是其中之一；把 systematic、disciplined、quantifiable 三个词都展开，systematic 展开成有结构、有方法、可重复、全覆盖，disciplined 展开成“受控”，quantifiable 展开成“可验证”：

| # | 方面 | 出处 | 一句话 |
|---|---|---|---|
| 1 | 有结构 | systematic | 分解为清晰步骤与阶段 |
| 2 | 有方法（含判断） | systematic | 有章法，非算法执行 |

| # | 方面 | 出处 | 一句话 |
|---|---|---|---|
| 3 | 可重复 | systematic | 过程本身能重复执行 |
| 4 | 全覆盖 | systematic | 从需求到退役无遗漏 |
| 5 | 受控（有纪律） | disciplined | 有基线、有变更控制 |
| 6 | 可验证（可度量） | quantifiable | 有证据、可检查、可度量 |

这些方面，是把“工程做起来是什么样”钉住的判据。回看恒信：有结构、有方法、可重复，前几样沾了边；但“受控”和“可验证”呢？上个月那个“先上再说”的接口，既没有变更控制，也没有容量验证，连“评估环节”这个维度也一起缺位。这不是清单，是关卡：缺一样，就是“差一点”，而工程里“差一点”往往就是事故。

顺着有结构、有方法、可重复、全覆盖、受控、可验证这几样再往上，有一条链：系统化 → 工程化 → 流程化 → 信息化。系统化是方法底子，加纪律与可验证是工程化，再固化是流程化，系统承载是信息化。反直觉的是：流程化不等于工程化，有流程不等于有工程。

恒信工单录入有固定流程，可录完没人核对、出错查不到记录，那仍是即兴；把“录入→核对→抽检”写成规程、设标准与检查点、留台账，换个人也能稳定交付，才是工程化。

自动化把判断推到了哪里。这些方面里没有“算法化”，有的是“有方法（含判断）”。可算法化的诱惑不会消失：它在现实里以“自动化”的面目出现。既然算法化是“确定性执行、无需判断”，而工程是“有章法的判断”，那么自动化程度越高，是越接近工程，还是越远离工程？

- 自动化不是减少判断，是把判断提纯。执行交给机器，人手上剩下的是最难的判断：异常怎么处置、边界在哪里、目标有没有变。一个全自动发布系统，人剩的不是“不用判断”，而是“判断密度最

高”。把“阈值设多少、异常怎么处置”也交给机器，机器不知道“为什么这个阈值”，换个场景就抓瞎，这缺的正是工程科学。

- 自动化拿走的是人阻止事故的能力。737 MAX 的 MCAS 本为防失速而自动接管控制；两次坠毁中，飞行员都没能在有限时间内夺回控制权（事故经过的出处见 §2.3.3 末注）。这里有两道关：权限上，机器优先于人，留给人的反应时间极短；能力上，长期不接管会失去情境意识，就算把控制权交回给你，你也不知道怎么接。

把这两件事合起来，就得到自动化与工程的分野：工程要的是“错误可见”（系统化），自动化追求的是“零错误”（把错误掩盖掉），方向相反。全自动的终点是消灭判断，而消灭判断，就是消灭工程。

---

## 2.4 部件全对，系统未必对

> 这一节把前面那些判据用起来：从“造部件”到“造整体”（系统工程），从“上线”到“被用起来”（工程系统），最后回头把恒信完整地量一遍。

### 2.4.1 系统工程：不造部件，只造整体

“系统工程 = 构建系统的工程，软件工程 = 构建软件的工程”，这个方向正确，但“系统”这个词是递归的：

- 软件工程 = 构建软件系统的工程。软件本身就是一个系统：模块、接口、数据流、运行时环境、版本演进……软件工程内部同样需要系统思维；
- 系统工程 = 构建复杂系统的工程。这里的“系统”特指由多个异构部件（硬件 + 软件 + 人 + 流程 + 数据）组成的整体。系统工程的重心不在“造部件”，而在整合。

系统工程不造任何部件，它负责“把部件整合成整体”：把总需求分解分配给各部件工程、定义接口、在系统层面完成集成，再做两道判定，即系统级验证与系统级确认。这两道判定怎么分、为什么不可互推，见 §2.4.4。这就是为什么叫“元工程”（meta-engineering）：它是工程之上的工程。

递归还不止一层：当软件足够大（操作系统、微服务集群），软件工程内部也会长出系统工程的子集：系统架构师、接口契约、集成测试、端到端验证。这就是“软件系统工程”，软件世界里的元工程。

把“整合”说透一点：系统工程在“接口管理”上花的精力，比任何部件工程都多。总需求分解给各部件之后，部件之间的接口契约、数据定义、时序依赖，这些“缝”才是系统工程的战场。

恒信合同审批系统里，法务系统的“合同状态”和财务系统的“合同状态”是两张字典。集成时才发现同一个词两套定义，这就是接口层面的系统工程问题，也是第 1 章“词汇病”在系统级的重演。系统工程交付的是接口契约、集成方案、系统级验证与确认的方案，这些看不见摸不着的东西，恰恰决定系统能不能转起来。

> 术语注：这些方案都是工作产品。**对工作产品不是只能评审**：评审对目标、靠判断，不要求客观证据；而**验证的对象并不限于实物**——ISO/IEC/IEEE 15288:2023 §6.4.9 明写验证的是“系统、系统元素**或人工制品**”（如系统需求、架构描述、设计描述），它的客观证据可以是检验结果、替代计算或评审文件（ISO 9000:2026 §3.11.12 注 1）。评审的结论有三档，“条件性合规”那一档要带补项、责任方与期限。**别把评审通过当成验证通过**；三个词各自对什么、在哪儿用，第 8 章展开。

### 2.4.2 涌现属性：合起来才有的行为

**单个部件都合格，不等于整体合格**。这叫**涌现属性（emergent behavior / emergent property）**：每个发动机都正常、每块机翼都达标，飞机未必能飞。整体行为不是部分行为的简单相加。

> 出处：这不是本书的发明。ISO/IEC/IEEE 15288:2023 §3.46 给“系统”下的定义就是这件事——原文作 arrangement of parts or elements that together exhibit a stated behaviour or meaning that the individual constituents do not（各部分或各元素的安排，它们合在一起呈现出单个构成部分所不具备的既定行为或意义）。**涌现这件事，正本写在 §5.2.1**：“系统的属性（作为整体）都源自或涌现于部件或元素及其各自的属性，以及各部件之间、系统与其环境之间的关系和相互作用。”标准对“系统”的定义本身就写着涌现属性，所以它在工程里无处不在。

涌现属性为什么在工程里无处不在？因为工程把一个大问题拆成许多小问题，分给许多人做。系统工程的职责，就是守住那些“部分之和的溢出”：接口、定义、时序，它们不属于任何一个部件，却决定整体成不成立。

这里要补一句：**这正是确认侧的题眼**。验证逐条对的是部件自己的规定需求，全过了也只说明“每一条都做对了”；系统整体在预期用途上成不成立，要由确认来答。恒信集团的三个模块单看都能验收：法务、财务、业务各自都做得不错；但三个部门的数据定义对不上、审批流跨部门断点没人管，集成那天就是事故那天。

微服务世界每天上演同一出：每个服务单独健康，但调用链、数据一致性、超时熔断没人管，上线必出事。这些“灰色地带”就是系统工程的地盘。

### 2.4.3 工程系统包含技术系统

再往上，还要分清两个常被混为一谈的“系统”，其中一个包含另一个。

> **工程系统（engineered system）**：被设计或改造的、与预期运行环境交互、达成预期目的、遵守适用约束的系统（INCOSE 2019：a system designed or adapted to interact with an anticipated operational environment to achieve one or more intended purposes while complying with applicable constraints）。INCOSE 还说明，工程系统可以由人、产品、服务、信息、流程、甚至自然元素组成。

> 出处：这条定义见 R-63（INCOSE《系统工程手册》第五版中文译本），所引英文**定义句**不在译本中；INCOSE-TP-2020-002-06 号文件库内也没有。故带年份 INCOSE 2019 引用。库内可直核的相邻处有两处：ISO/IEC/IEEE 15288:2023 §3.46 给“系统”下的定义，以及 §5.2.1 那句“系统的属性……源自或涌现于”各部分及其关系——后者正是 §2.4.2 那条涌现属性的正本依据。**与底本的一处分歧须说明**：05 号 §8 把软件系统直接列为工程系统，本书按这条定义里“人、流程、组织是否算进来”改判为技术系统（§2.4.3）。

工程系统的内核，是**技术系统**：仅由人工技术组件（硬件、软件、机械、电气）构成的系统，人只是使用者。

技术系统的特征是：纯人工制品，没有自然物、没有“活的”组件；功能靠技术实现，性能可量化、可测试（吞吐量、负载、响应时间）；人只是外部使用者，司机、操作员都在系统之外，不是组成部分。（这三条特征文字见 R-63（二手）；划分依据＝INCOSE 2019 的工程系统定义。）

一句话：技术系统是设备的集合；工程系统是整个社会技术系统，是技术子系统、人的子系统、组织子系统耦合在一起的整体，技术系统只是它的内核（§2.4.4）。

**概念卡片 · 工程系统**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 被设计/改造、与环境交互、有目的、守约束的系统（INCOSE 2019）——人、产品、服务、信息、流程**甚至自然元素**都是它的一部分 |
| **它不是什么** | ⑥⑦ | 对立：自然系统（无设计者）· 社会系统（非被设计，如习俗、市场）· 概念系统（无运行环境）；社会系统不等于社会技术系统——后者就是工程系统（见§2.4.4）；辨析：工程系统包含技术系统，技术系统只是内核 |
| **它从哪来** | ②③⑨ | 20 世纪 70—80 年代系统工程界从大量失败项目发现“系统造出来、性能达标却用不起来”——人、流程、组织必须算进系统；演化：INCOSE 2019 把定义从 technical system 改写为 engineered system，视野从设备扩到整个运行环境（见 R-63）；engineer ← 拉丁 ingenium（巧思） |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它往哪去** | ④⑤⑧ | 回答“如何让整个社会技术系统运转起来”；在验收、上线、生存周期管理时用；误区 ✗ “软件部署好 = 工程系统成功”（技术达标 ≠ 人会用、流程顺、组织撑得住） |
| **判据** | 被用起来 | 边界画在“整个运行环境”上，人、流程、组织、数据都算在里头；验收参照是“系统被用起来”，不是“设备跑起来”（§2.4.4） |

工程系统的生存周期：INCOSE 的系统工程定义里，工程系统要被“实现、使用、退役”，意味着工程系统有生存周期，要为“退役”做设计，让系统能安全关停、数据能迁移。恒信的合同审批系统，十年前的纸质台账怎么迁入？退役时的数据归档和权责移交呢？这两问，技术系统不问，工程系统必须答。

软件系统为什么不算概念系统：它虽然看不见摸不着，但它是被设计出来的，有明确目的，与操作系统、网络和用户交互，也守着性能、安全与法规这些约束。不过要留意：它**本身**仍是技术系统，人只是使用者，不在它之内；加上用户、业务流程和运维组织，才是工程系统（§2.4.4）。

术语注：软件系统不以“概念系统”归类。概念系统（数学理论那类纯信息形态）被判为非工程系统，理由是它**没有运行环境**；软件系统有运行环境、有生存周期，所以不归入概念系统；至于它算不算工程系统，要看人、流程、组织有没有算进

> 来（§2.4.4），单看它自己，仍是技术系统。两处都“看不见摸不着”，是两件事，不能互推。

最典型的工程系统例子是机场：跑道和航站楼（物理）、雷达和行李分拣（技术）、塔台和地勤（人）、安检和调度（流程）、航班数据（信息），还有气象条件（环境）。合起来才产生“机场顺利运行”这个涌现行为。单看任何一个子系统都合格，不等于机场能转起来。工程系统的边界，画在“整个运行环境”上，不是画在“设备的壳”上。

同样，冰链的 M3 温控箱不只是“设备集合”。硬件、固件、云端、App，那是技术系统；它要交付的是“疫苗、药品冷链运输”，全程温度可追溯、断链可告警，背后连着仓储物流流程、法规义务、批次追溯和各环节的操作员。石军把箱体做到极致，是把技术系统做到极致；箱体进了冷链、温度真被全程追溯、断链真告警出来，M3 才算被“用起来”。按技术系统看它达标了，按工程系统看，它才刚开始。

这个道理，制造侧的小周最清楚：

“产线上我把这箱子造出来，可箱子什么时候算'成了'？不是下线那刻。是它装进冷链车、温度被全程追溯、断链真告警了——那才叫被'用起来'。我们车间考核的是下线合格率，客户认的却是全程没断链。按技术系统看，我们把箱子做完了；按工程系统看，我们才交了一半。”

### 2.4.4 验收参照：上线不等于被用起来

把技术系统当成工程系统来验收，是系统工程史上最贵的教训之一，也是恒信集团的写照。

按技术系统视角：恒信合同审批系统部署好了、服务器跑起来了、模块全部上线，就是“系统成功”。

按工程系统视角：采购流程没改、法务人员不习惯电子签、财务数据没清洗、审批权责没理清，就是“系统失败”。

砸了几千万，最后发现“系统”根本没被用起来：法务还是线下签、财务还是手工台账、业务还是走邮件。这不是技术问题，是把技术系统误当成了工程系统来验收，忘了人、流程、组织也是这个系统的一部分。

这里还差最后一道分辨：就算系统真被用起来了，你怎么知道它“成”了？**验证（verification）对的是规定需求**：需求的每一条都实现了吗？测试过了吗？答的是“做对了吗”。**确认（validation）对的是预期用途**：这个系统在真实的使用场景里，解决了利益相关方的问题吗？答的是“做对了东西吗”。两者不可互推：**全部需求都验证通过，确认仍可能失败**。恒信的上线验收，只做了前者。

> 出处：ISO/IEC/IEEE 15288:2023 §3.54／§3.55 的注 1 与 ISO 9000:2026 §3.11.12／§3.11.14 两处定义说的都是这件事：确认对的是“针对特定预期用途或应用的需求”，验证对的是“规定需求”。**两句话只差一个词**：§3.55 注 1 末句作 The system was built right.（验证），§3.54 注 1 末句作 The right system was built.（确认），问的是两件事。

20 世纪 70—80 年代，系统工程界用大量失败项目发现：很多“系统”其实是社会技术系统，单独优化技术部分没有意义。这就是为什么 INCOSE 2019 特意把定义从 “technical system” 改写为 “engineered system”，把视野从设备扩到整个运行环境。

这个概念当年被系统理论家 Ropohl 等人点破：很多“系统”是技术子系统、人的子系统、组织子系统耦合在一起的整体。技术部分

是内核，但单独优化它没有意义，因为“用不起来”往往卡在人、流程、组织那一层。

软件系统本身是技术系统（代码、数据、服务器）；“软件 + 用户 + 业务流程 + 运维组织”才是工程系统。说“我们构建了一个技术系统”，说的是设备的集合；说“我们构建了一个工程系统”，承诺的是整个社会技术系统能运转起来：技术达标、人会用、流程顺、组织撑得住。恒信的敏捷转型要成为真正的工程，第一步是把验收的参照从“系统上线”换到“系统被用起来”。

这也给第 8 章埋下回声：把“结果当成功”的病，在确认环节的版本就是“UAT 跑了就是确认过了”，同一病的两种形态。

### 2.4.5 回头给恒信量一遍

把本章的判据串起来，对恒信的合同审批系统完整地量一遍：

项目一：交付能不能离开某个人。换个团队来运维恒信的 CI/CD，能一样跑吗？顾言把问题递给王默，他是运维，这话他最有发言权。

王默摇头：“换个人来，光那三条隐藏命令就够他找一天；还有两个环境变量，写在李四的备忘录里，不在任何文档里。这一项我认——按 §2.2.4 那条判据量，我们不合格。”

项目二：层次是否齐备。工程科学：容量评估被砍，这一层缺位；方法论：敏捷在，合格；工具与实践：CI/CD 在，合格；价值与伦理：为进度砍可靠性，这一层失守。结论：缺了两层，尤其缺了最致命的价值与伦理。

项目三：工程化的成色。有结构、有方法、可重复：前几样沾了边（可重复说的是过程本身能重复执行；“换个人还能不能交付”是项目一的事）；全覆盖：容量测试被砍，不及格；受控：“先上再说”，不及格；可验证：没有容量验证，不及格。结论：缺了全覆盖、受控、可验证，不是“差一点”，是差一半。

项目四：系统化到什么程度。每一步有输入输出吗？有检查点吗？换执行者稳定吗？——最后一句答不上来。结论：系统化的“可检查”没立住，差的就是那另一半。

项目五：整合度。法务、财务、业务三个模块各自都做得好，但数据定义对不齐——涌现属性在系统的“缝”里等着。结论：部件工程合格，系统级整合缺位。

项目六：验收的参照。按技术系统看，系统上线了；按工程系统看，法务线下签、财务手工台账、业务走邮件。该做的两道判定只做了一道：验证勉强跑了一半，确认根本没做。结论：验收参照用错了，拿“上线”当“成功”。

量下来，项项不合格。恒信的方法论和工具是真的，缺的在别处。这里给一条建议：

> **先把“验收的参照”从“系统上线”换成“系统被用起来”，再谈敏捷转型成功。否则，这套流程与工具只会让“不会出事”的错觉维持得更久。**

本章的判据不是彼此孤立的。试金石测可复制，层次测完整，工程化的成色测过程，系统化自问测可检查，系统工程（元工程）测整合，验证与确认测验收。合起来，才是一次完整的量。

### 2.4.6 认知反转

回到恒信那场季度会。罗建宣布“我们已经是工程化团队了”——会议室鼓掌，顾言问了句扎心的。

第一处：他把“有方法论、有工具”当成了“在做工程”。工程不止那两层，敏捷和 CI/CD 只占其中一部分。上个月的事故不是“工程

之外”的意外，恰恰是缺了价值与伦理那一层的证明——为进度悄悄牺牲可靠性，还不留记录，是这一层失守的标准动作。

第二处：他背得出 IEEE 610.12 的三个形容词，却没问过那三个词是在给谁下定义。那是一份软件工程的定义，说的还是“工程化的过程特征”——过程特征齐了，不等于工程齐了。

第三处：他默认了“系统上线了”就算工程成功。系统是技术系统还是工程系统，取决于它有没有被整个社会技术系统用起来——恒信的系统上线了，但法务还在线下签，这套系统就不是一个成功的工程系统。

而冰链的石军，给了另一种对照：几十年手艺，个人能力顶天立地，但交付依赖具体的人。换个人来做，交付就维持不住。

本章的反转是温和而彻底的：你可能没有在做工程，而你在做的那件事，可能比工程更难被复制。不要急着宣布“我们是工程化团队”。先问一句：换个人来做，交付还维持得住吗？再看那几层齐了没有。

用第 1 章的话说，对“工程”这个词，你原来只是“见过”：背得出定义、说得出“敏捷+CI/CD”，甚至背得出那几个英文形容词。本章把它推到“拥有”：能答全四问（它是什么、不是什么、从哪来、往哪去），能判一个系统是不是工程系统。能答全四问才算拥有——你当场就能判一个系统是不是工程系统。

> **工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西。它不靠流程的多少定义，靠工程科学、方法论、工具与实践、价值与伦理是否同时在场定义。**

用第 1 章 §1.4.4 的尺子量一下：本章把“工程”从**第 ① 级（表征）**推到了**第 ② 级（图式）**。你现在看得出那几层里缺的是哪一层了；至于能推演、能拿主意，那是**第 ③ 级（心智模型）**的事。

## 小结

本章回答的是全书最基础的问题：我们到底在做什么。工程不是流程的堆砌，是工程科学、方法论、工具与实践、价值与伦理同时在场的人类实践；它不靠工具定义，靠那条判据和那几层齐不齐来检验。它把交付从人身上挪到方法上，却把判断留在人身上。这条边界，既解释了它为什么可复现，也解释了它为什么不能被自动化替掉。

下一章，我们看产品开发里最要紧的几个词：需求、设计开发、质量。工程那层底子立起来之后，就该看它们各自是什么、边界画在哪、又是怎么合起来用的。

散会时，王默在运维台账上添了一栏：这条曲线归谁。以前它只活在他的监控屏上，他嘴上念叨、从不上纸；这回他把它写下来了，写下来，别人才接得住。

---

## 附录

### A 本章概念总图

主张：工程是把交付从人身上挪到方法上的一次转移——挪得走的是方法，挪不走的是判断；

挪走了就成，没挪走就不成

工程 = 科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西

├── 定义　科学原理 × 判断创造（料）• 约束（框）• 系统方法（路）• 有用且安全（果）§2.2

├── 权威定义　ECPD 1941 → ABET 1979 → INCOSE 2019：三家同一个内核；

│　　IEEE 610.12-1990 另给软件工程一侧的过程特征　§2.2.3

├── 试金石　换个人、换团队，用同样的需求和约束，能交付质量相当的成果吗——能，是工程；不能，那是手艺　§2.2.4

├── 对立面　认识论×科学　方法论×手艺　目标×艺术　§2.3.1

├── 层次　工程科学 → 方法论 → 工具与实践 → 价值与伦理（四道挪走的关）　§2.3.3

├── 近义词辨　systematic 系统化（有章法）≠ systemic 系统性（有全局）≠ algorithmic 算法化（确定执行）§2.3.4

├── 自动化　提纯判断 + 拿走人阻止事故的能力——要错误可见，不是零错误

├── 系统工程 = 元工程（整合部件，不造部件）→ 涌现属性：部件全对 ≠ 系统对　§2.4.1~2

├── 验证与确认　验证对规定需求（做对了吗）• 确认对预期用途（做对了东西吗）——不可互推 §2.4.4

└── 工程系统 包含 技术系统（人/产品/服务/信息/流程/自然元素都是它的一部分）　§2.4.3

## B 本章的能力级

**第 ② 级为主**

- **表征**——把“工程”这个词说准（科学原理 × 判断创造、约束、系统方法、有用且安全的人造物），并把挨着它的那些词分开：手艺、科学、艺术、方法论、系统工程、技术系统、社会技术系统（附录 A T-13~T-21；“工程”完整九格见**书末**附录 B，“工程系统”见本章 §2.4.3）
- **图式**——几个可指认的结构：工程的层次（工程科学 → 方法论 → 工具与实践 → 价值与伦理，四道把交付从人身上挪走的关）、对立面（认识论对科学 · 方法论对手艺 · 目标对艺术）、包含关系（工程系统包含技术系统）
- **心智模型**——几条能推演的机制：试金石（换个人还能不能同等交付）、系统化到什么程度（有输入输出 / 有检查点 / 换执行者稳定）、价值失守问什么（利益相关方知不知情 / 有没有评审 / 有没有记录）、验证与确认不可互推、自动化把判断推到了哪里

- **解释框架**——本章不建此级；由第 17、18 章承担

> 能答全四问，才算拥有；只能背出那句定义、那几层名字，只是见过。真拥有的人，是在压力面前守得住价值与伦理那一层的人。

## C 自查 · 这些说法对吗？

1. “我们用了敏捷 + CI/CD + 测试，所以我们在做软件工程。”——**错**。敏捷和 CI/CD 只落在方法论和工具与实践上；工程还有工程科学、价值与伦理这两层。
2. “工程的对立面就是科学。”——**错**。工程没有单一对立面：认识论上是科学、方法论上是手艺、目标上是艺术。
3. “系统化就是把流程固化成算法，让机器自动跑。”——**错**。系统化是“有方法（含判断）”；算法化是“确定性执行”。工程每一步都留有人判断的空间。
4. “微服务每个都健康，系统就是健康的。”——**错**。涌现属性：调用链、数据一致性、超时熔断这些灰色地带没人管，上线必出事。
5. “软件部署上线了，系统就成功了。”——**错**。那只是技术系统成功；工程系统成功要看人会不会用、流程顺不顺、组织撑不撑得住。
6. “需求全部验证通过了，这个系统就成了。”——**错**。验证对的是规定需求，确认对的是预期用途，两者不可互推；需求全过而没人用，一样是失败。
7. “老工程师经验丰富，这就是工程。”——**错**。那是手艺；按那条判据一量：换个人还能同等交付吗？把经验写成文档只是第一步，还要可检验：换个人能按文档交付同等结果，才算工程化。

8. “自动化程度越高，系统越安全。”——**错**。执行交给机器，判断留给人；剩下的判断反而更难、更密。而自动化也削弱了人拦事故的权限和本事。工程要的是“错误可见”，自动化追求的是“零错误”。

## D 练习

1. 用试金石检验你们团队最近交付的一个系统：换三个人来做，能交付质量相当的成果吗？如果不能，缺的是**哪一层**（工程科学／方法论／工具与实践／价值与伦理）？
2. “敏捷 + CI/CD + 测试”落在工程的哪几层上？缺的那几层，用你们公司的实际情况各举一个“没守住”的例子。
3. systematic、systemic、algorithmic 三个词，各自回答什么问题？举一个“有章法但需要判断”的日常工作例子。
4. 系统工程和软件工程是什么关系？为什么说系统工程是“元工程”？你们系统最近一次“部件全对、整体出事”，是哪个灰色地带没人管？
5. 用“技术系统”和“工程系统”两个视角检查你们最近上线的一个系统：按技术系统看成功了吗？按工程系统看呢？如果两个答案不一致，缺的是哪一块（人／流程／组织／数据）？
6. 把你们最近一次验收的记录翻出来：哪几条做的是验证，哪几条做的是确认？有没有“验证全过、确认没做”的？
7. 那几起工程事故（泰坦尼克／挑战者／737 MAX）的共同点是什么？在你们公司找一个“价值与伦理先松手”的真实例子。
8. 冰链的石军是“手艺”还是“工程”？如果让你给石军的手艺做“工程化改造”，第一步做什么？
9. 照着有结构／有方法／可重复／全覆盖／受控／可验证给恒信的敏捷转型打分，指出缺的是哪几样。

10. 你们团队最值钱的“手艺”是什么？如果要把它工程化，第一步是把哪个“心法”写成可检验的规则？
11. 拿那句定义给你最近的一个项目打分：科学原理、约束、系统方法、有用且安全的人造物，各自缺了什么？哪一条缺了最致命？
12. 用三句话，把“工程 ≠ 手艺”讲给一个完全不懂工程的人（比如你的家人）——卡在哪个词上？那个词就是你概念最薄的地方。

（答案要点见书末附录，与训练教材题卡同源）

---

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**


- [1] **R-14** IEEE 610.12-1990 —— 软件工程定义（“把工程应用于软件”；systematic / disciplined / quantifiable → 工程化的那些方面；**它定义的是软件工程，不是工程**，§ 2.3.4 有引用规范）。
- [2] **R-15** ECPD 1941 / ABET 1979 / INCOSE 2019 —— 工程权威定义（几家的定义走到同一个内核：科学原理／判断创造／约束／系统方法／造福目的）；INCOSE 2019 的工程系统定义与生存周期；INCOSE 那条见 15288:2023 § 3.50（§ 2.2.3）
- [3] **R-18** CMMI —— 工程／开发方法论举例（敏捷、瀑布、CMMI）
- [4] **R-37** ISO/IEC/IEEE 15288:2023 —— 本章几处一手锚：**§ 3.46** 系统定义；**§ 5.2.1** 系统属性“源自或涌现于”各部分及

其关系（涌现属性的依据，§2.4.2）；**§3.50** 系统工程定义（来源条注明 INCOSE-TP-2020-002-06，§2.2.3 那三家之一在此可回正本逐字核）；**§3.54 注 1 / §3.55 注 1**（The right system was built. / The system was built right.——确认与验证的分野，§2.4.4）

- [5] **R-38** ISO 9000:2026（第 5 版）—— **§3.11.12 验证**对“规定的需求”／**§3.11.14 确认**对“针对特定预期用途或应用的需求”（§2.4.4 验证与确认不可互推的条款锚）

**经典文献**

- [6] **R-60** 冯·卡门（von Kármán）——“科学家研究现存的世界，工程师创造从未存在的世界”（常被归于；另存一种英文措辞）
- [7] **R-63** INCOSE Systems Engineering Handbook —— 工程系统（**译者注中文释义**，PDF p46／印刷页 31）；系统工程术语（PDF p397）
- [8] **R-65** Ropohl —— 社会技术系统：技术子系统＋人的子系统＋组织子系统耦合，单独优化技术部分没有意义

> 工程师的能力清单见第 9 章。

# 第 3 章 需求、设计开发、质量：产品开发的图式

## 3.1 引言

本章立产品开发里最要紧的三个词：需求、设计开发、质量。三节走一条线：先把它们各自说准，再把相邻的边界一一画开，最后合起来看它们为什么拆不掉。这条线立住之后，从第 4 章起，每一章都是它在某一个方向上的展开。

### 3.1.1 误解现场

周五下午四点，冰链科技的产品评审会开了整整两个小时，会议室里飘着咖啡和争论的气味。

产品经理林涛把一页参数对比表拍到桌上，指着一个数字：

> “M3 的温度精度做到 ±0.3℃，竞品只有 ±0.5℃；断电保温 ≥ 72 小时，竞品 48 小时；重量还轻了 2 公斤。这几条都落到可测量的固有特性上了，设计规格全线优于竞品——**符合性没有问题**，质量这一关我看不出问题在哪。现在只剩一件事：对外怎么讲，是打'行业精度第一'，还是'保温时长远超竞品'。”

会议室里一片点头，有人甚至轻轻鼓了掌。坐在上首的产品开发经理赵刚也点了头，在产品评审结论栏里签下“通过”：

> “三个参数全线领先，需求里写的每一条都超了。质量这一关，我签字。”

只有坐在角落的测试季昭，弱弱地举起手：

> “那……测试判据是什么？接收准则写在哪份测试方案里？”

林涛摆摆手：“技术方案都定了，符合性判据后面自然会有。先定对外怎么讲。”

季昭张了张嘴，又闭上。会议继续。

如果你坐在这间会议室里，你大概也会点头。因为大多数人对“质量”和“设计开发”这两个词的直觉，和林涛一模一样：

- **质量 = 东西做得比人强、指标压过竞品**；
- **设计开发 = 做出技术方案**，方案定了，后面的测试、采购、生产、交付，都是“自然会有”的事。

这两个直觉，是本章要推翻的东西。错的不是林涛，错的是这两个词在他脑子里指向的那个东西。季昭那句被淹没的问话，其实已经给出了结论：设计开发根本没有结束，到章末我们回来读它。

### 3.1.2 本章要回答什么

读完本章，你应该能回答：

1. 需求、设计开发、质量各自是什么？它们为什么能构成产品开发的图式，彼此又不在同一个层级上？
2. 需要、期望、需求有什么区别？目的、目标、需求又有什么区别？
3. 需求里“明示的”“通常隐含的”“必须履行的”分别指什么？漏掉哪一种最要命？
4. 设计开发从什么时候开始、到什么时候结束？“技术方案做完了”够不够？
5. 需求开发和设计开发的边界画在哪？凭什么这么画？

6. 质量是“特性除以需求”吗？“等级高”等于“质量好”吗？特性超出需求是加分还是减分？
7. 质量能打分吗？为什么有些特性不能摊进平均数？
8. “质量与进度冲突”到底是一场什么冲突？该动的是什么，不该动的是什么？
9. 需求、设计开发、质量为什么互相定义、拆不掉？由它们合成的那条路径能推出什么，又推不出什么？

### 3.1.3 本章的骨架

先把本章要立的东西摆出来，后面三节，都是给它添内容。

本章的主张是一句话。

> 需求、设计开发、质量不是三个并列的词，是一条能走的路径——定需求 → 出规格 → 判满足；这条路径立起来，“东西好不好”就从一句印象，变成一次可复核的判定。

需求、设计开发、质量在这条路径上各有位置：

> 需求 = 定需求：定义“要什么”。它是设计开发的输入，也是质量判定的依据。
>
> 设计开发 = 出规格：把需求逐级转化为更详细的需求，一级出一份规格，直到可实现。规格就是规定需求的集合。
>
> 质量 = 判满足：认定客体的固有特性满足需求的程度。单项看符合不符合，整体看满足到什么程度。

它们的分量并不一样，第 1 章把领域里的概念分成三层：一般概念、核心概念、根概念，**根概念** ⊂ **核心概念**，越往里门槛越高。产品开发这一行的核心概念不止三个，本章只取这三个；第 1 章 §1.7.4 判

出的根是需求、系统与设计开发，本章取其中的需求与设计开发——它们是根，质量是派生的枢纽。

需求、设计开发、质量同属核心概念这一层，但三者并不平级。而它们聚起来形成的结构，就是本章要说的产品开发的图式：**三个词是表征，聚起来的那条路径才是图式**（第 1 章 §1.4.4）。

本章分三节走。第一节把需求、设计开发、质量各自是什么说准，包括它们的来处。第二节把相邻的边界一一画开：起点、终点、参照。第三节把定需求、出规格、判满足这条路径用起来：读出程度、解开冲突、看它们为什么拆不掉。

---

## 3.2 需求、设计开发、质量各管什么

> 这一节先看定义。它们都在这里出场：需求给三种形态，设计开发给一条转化的定义，质量给一个程度。看完定义，后面的边界与运用才有话可说。

### 3.2.1 需求：没说出口的也算

打头的是需求，ISO 9000:2026 给它的定义只有一行：

> **需求（requirement）**：明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）。
>
> need or expectation that is stated, generally implied or obligatory

> 术语注：requirement 本书一律作“需求”；“要求”是另一个汉语词（**本书口径**，ISO 9000:2026 里没有 demand 这个术语条目），不用来指 requirement。ISO 9000:2026 的中文译本把 requirement 译作“要求”，本书引用时按本书术语写作“需求”，条款号不变；需要逐字对照的，以英文原文为准。

注意后半句是并列关系，没有主次：明示的（stated，说出来了）、通常隐含的（generally implied，惯例默认、没说出口）、必须履行的（obligatory，法规规定的那一类），三个都是需求。

拿 M3 温控箱展开，三种形态全占。

**明示的**——写在纸上的。客户在合同里写明“箱温波动不超过 ±0.5℃”“箱门可单手打开”。白纸黑字，谁都能看到，谁都赖不掉。

**通常隐含的**——没人写，但行业默认。冷链运输箱“应该扛得住装卸时的磕碰”“外壳不能有锐角划伤搬运工”“电池要能撑过全程并显示余量”。这几条没人写进合同，但你要是漏了，客户在使用的第一天就会认定“这产品不行”。隐含需求漏了是缺陷，不是“没写下来所以不算”，它不会自己开口，需要组织主动去识别。这是起点判据里最贵的一条（见本章 §3.3.2）。

**必须履行的**——法规强制的那一类，无条件满足。法规依据是 ISO 9000:2026 §3.5.5 法定要求与 §3.5.8 法规要求。GSP（药品经营质量管理规范）把疫苗冷链的温度、断链与异常报警列为硬性需求（冷藏疫苗常见 2~8℃，冻干制品 −20℃ 冷冻），M3 按 −20℃~8℃ 宽范围设计，断链即报废。不满足这条，M3 连上市资格都没有；必须履行的需求没有讨价还价的余地。

镜照一眼恒信集团：合同审批系统的明示需求是“合同在三个部门间流转审批”，隐含需求是“审批人能在手机上快速通过”“离职审批

人自动移交待办”，必须履行的是“合同数据按公司法要求留档十年”。IT 世界里也是明示、隐含、必须履行，一个不缺。

三种形态的差异，到第 8 章会变得性命攸关：验证（verification）对照的是规定需求，也就是被明示的那一类（正本原注：例如写在成文信息中的）。确认（validation）对照的，是针对特定预期用途或应用的需求，也就是真实场景里的那些（ISO 9000:2026 §3.11.12、§3.11.14）。隐含需求没进文档，验证够不着它，只有确认拦得住。那是第 8 章的事，这里先埋一句。

“需求”这个词本身也有来处：requirement ← 拉丁 requirere（要求、寻求）。而这个词的所指经历过一次漂移，它原指“要的东西”，后来才分出明示、隐含、必须履行这三种形态。这正是第 1 章 §1.8.3 说的“同词异义”：一个词如何攒出多个概念。ISO 9000 把这场漂移固化成了标准定义。

### 3.2.2 需要、期望与需求：内容与呈现方式

英文在这里特别值钱，中文习惯把“需要”“期望”“需求”糊成一个词，英文是三个：

| 英文 | 中文 | 含义 | M3 的例子 |
| --- | --- | --- | --- |
| need | 需要 | 硬性诉求，缺了会痛 | 疫苗不能坏 |
| expectation | 期望 | 软性诉求，缺了会失望 | 精度越高越好、App 越顺手越好 |
| requirement | 需求 | 需要或期望 × 呈现方式 | 上面两者的全集 |

分清这一层，要害在于“需要”和“期望”是**内容**，而“明示”“隐含”“必须履行”是**呈现方式**。内容说的是“你要的那个东西本身”，呈

现方式说的是“这个内容是怎么出现的”。两个方向交叉，才是需求的完整样子：一个 need 可以是明示的，“我们要 72 小时保温”，也可以是隐含的，“要能扛磕碰”。一个 expectation 可以是隐含的，“手感要高级”，也可以是明示的，“外壳要金属拉丝”。

> 术语注：“需要和期望是内容、明示和隐含和必须履行是呈现方式”这个两维说法是本书的梳理主张——ISO 9000:2026 §3.5.1 的定义句并列给出 need／expectation／requirement 三个词，其中只有 requirement 是术语条目，不这样分方向；两维画法是本书为方便使用而做的凝练。

为什么要较真这个区别?因为后面所有的设计开发、质量判定、评审、验证与确认，对上的都是“需求”这个完整样子，而不是其中某一个形态。林涛心里想的“质量”，八成只有“期望”那一层，“越高级越好”；而法规、客户合同、行业惯例这些真正决定成败的部分，不在他的视野里。把需要、期望、需求糊成一个，等于把最重要的一半需求扔出视野。

镜照一眼恒信：业务方的郑亮，“需要”是月底账能对上，缺了会痛；“期望”是查询响应再快一点，缺了会烦；而“查询响应有明确指标、数据留档十年”才是落到纸上的需求。IT 世界里同样是“内容 × 呈现方式”这个两维样子。

### 3.2.3 目的、目标与需求：价值、结果与规格

比上一条更常见的混用，是需求、目标、目的不分：ISO 9000:2026 给了目标（objective）和需求（requirement）的标准定义，没有单列“目的”（purpose）。三者怎么分，用这一份定义对齐（本书的梳理主张，作为对标准定义的解释）：

> **目的（purpose）** = 系统为什么被研制的理由。落在价值侧，可以不量化；它是目标与需求的来源，在最上游。它的确立过程对应 ISO/IEC/IEEE 15288:2023 §6.4.1 业务或任务分析，任务与目的在那里定义；**ISO 9000 未给“目的”术语级定义**，15288 给的是“过程目的”（§3.29）。
>
> **目标（objective）** = 要实现的结果（ISO 9000:2026 §3.7.11）。落在系统语境，就是系统预期达成的可度量结果，是目的的可度量投影；主语是“要实现的结果”，可以是战略性的、战术性的或运行层面的，也可以是组织范围的、或针对某个项目、产品、服务或过程的（§3.7.11 注 1、注 2），不必是系统自身的属性。
>
> **需求（requirement）** = 系统自身的可验证规格。落在系统自身；三种形态见 §3.5.1。

对照着看：目的 = 存在理由，目标 = 要实现的可度量结果，需求 = 自身可验证规格。三者都是关于系统的陈述，区别在侧面，不在“内”和“外”。链条是：目的 →（展开）目标 →（转化）需求。

落到 M3（疫苗冷链）：目的 =“让疫苗在冷链运输中一路不失活、安全送达”（价值侧，可以不量化）。目标 =“到站疫苗活性 ⩾ 95%”（把“不失活”翻译成能盯着看的数）。需求 =“箱温 -20℃~8℃ 内偏差不超过 ±0.5℃、断链 30 秒内告警”（系统的可验证规格）。三者各占一个侧面：价值、结果、规格。

为什么不能混？目的定方向，需求定规格，目标是中间那层翻译。把目的当需求写进规格，设计开发会茫然：一张写着“疫苗不失活”的纸，没给任何可设计的数。把目标当需求，则跳过了“靠什么达成”这一步：活性 ⩾ 95% 是个结果，活性靠箱温守住，得倒推出箱温条款，设计才落得下去。

| | 目的 | 目标 | 需求 |
|---|---|---|---|
| 回答 | 为什么做 | 达成什么可度量的结果 | 做成什么样 |
| 侧面 | 价值 | 结果 | 规格 |
| 能不能量化 | 可虚 | 必量化 | 必须可判定 |
| 锚点 | 15288:2023 §6.4.1 | ISO 9000:2026 §3.7.11 | ISO 9000:2026 §3.5.1 |

比混用更隐蔽的，是**根本不立目的**。目的由利益相关方定义、由业务方签字确认，开发团队与架构师只承接、不拍板（第 18 章展开）。目的缺位的时候，团队不会停下来等，而是照自己猜的那个“目的”往前冲。做得越完美，离真实目的越远，这正是“正确地做错事”。先过目的这一关，再谈目标与需求。

恒信的合同审批系统，同一根链条也立得住：目的 =“让合同审批合规、月底账目清楚对得上”。目标 =“月底对账时间不超过 2 小时、逐笔一致率 100%”。需求 =“合同编号三方一致、25 号后签的合同归属下个统计月”。两个系统，同一套价值、结果、规格。

### 3.2.4 设计开发：需求怎么变得更详细

第二个词，也是本章的主角。ISO 9000:2026 的定义，几乎每个词都是关键词：

> **设计和开发（design and development）**：将某一客体（§3.5.3）的需求（§3.5.1）转换为该客体更详细需求的一组过程（§3.3.1）。

拆开看，四样东西各有各的讲究。

客体——被设计的东西；定义用的是“客体”，不是“产品”。这不是措辞讲究：设计开发的对象绝不限于硬件，产品设计开发、服务设计开发、过程设计开发都成立（§3.3.11 注 3）。M3 温控箱是客体，恒信的合同审批系统是客体，连一条“疫苗出库流程”也可以是客体。

把“过程设计开发”落到最熟悉的例子：工艺编制。图纸定下“装配扭矩 15±2 N·m、密封无泄漏”，这是构件做成什么样；工艺编制就是制造过程的设计开发，把“做成什么样”转成“制造环节怎么做”的可实现需求，输出工艺规程、工艺卡这些制造方案。

产品开发不只是图纸或代码，还包括构件的加工、装配、检验技术的开发。缺了工艺，产线不知道“怎么装配、用什么工装、检验标准是什么”，设计就没到可实现。

需求——输入必须是需求，不是点子、灵感、创意；创意不表述为需求，就进不了设计开发的流程。顺着这条边界再往外划一格：目的和目标也不在设计开发之内，它们是设计开发的输入端与前提（15288:2023 §6.4.1 业务或任务分析、§6.4.2 利益相关方需要与需求定义）。设计开发从系统需求（§6.4.3）起算，系统架构定义（§6.4.4）、设计定义（§6.4.5）才是它的核心。

转换为——动词才是灵魂，输入与输出必须是不同的东西。流程走完了，输出和输入一个样，那不是设计开发，那是走流程。这个“不同”体现在“更详细”上。

**更详细的需求**——唯一的判定标准；注意，定义没说“更正确”“更优化”，只说“更详细”。设计开发的职责是把需求展开、分解，落到可实现和可验证的粗细。方案好不好，由评审和验证把关，不是设计开发自己说了算。

辨析栏 · 设计、开发、设计开发

> 英语里 design（设计）、development（开发）与 design and development（设计开发）三者，有时完全同义，有时指整个流程的不同阶段：design 偏概念与方案阶段，development 偏详细化与实现阶段。用哪个词不重要，重要的是整体是一个需求转化过程（§3.3.11 注 2）。落到中文，“设计开发”对应 §3.3.11 的完整术语，别把它拆成“设计”加“开发”两件事。

“更详细”这三个字，拆开来看是三个可查的成分：

| 成分 | 回答的问题 | 判据 | 锚点 |
| --- | --- | --- | --- |
| **更具体** | 详细体现在哪 | 需求通常按特性来定义，特性可以定量也可以定性 | §3.3.11 注 1 |
| **更可实现** | 详细到什么程度够 | 够实现为止，能直接被生产和服务提供执行 | 本章 §3.3.5 终点 |
| **更可验证** | 详细得对不对方向 | 落到特性上（定量或定性都可），让验证有对照的东西 | §3.3.11 注 1 |

注意“更可验证”这一格只证明方向对，不能当终点。验证以规定需求为对照，而规定需求正是设计开发的输出，所以验证能发生，前提是设计开发已经结束（本章 §3.3.4 有完整推理）。

还有一个被排除的成分：“更可理解”。它若指“文字浅显”，与“详细”无关甚至相悖；若指“能被下游理解执行”，已经被“更可实现”吸收了。

一句话记：具体到特性，详细到实现，可验证指方向。

这套判据在实战里很好用。既然“更详细”是设计开发有没有发生的唯一判据，判断“这个变更算不算设计开发”就有了硬标准：输入输出同样详细，就是没发生设计开发。

只改外观颜色、不产生任何新需求细节的变更，严格说不构成设计开发，可以走快速通道。而“把温度控制从单路改成双路冗余”产生了新的可靠性需求、新的接口需求、新的测试需求，就是一次实实在在的设计开发，必须走完整流程。

这条标准在变更管理和审核时很管用：它把“改一下”和“重新设计”分开，后者不能图省事。

### 3.2.5 设计开发：每一级都在做同一件事

定义里还有一个注容易被忽略：**输入那份需求的表述，比输出“更宽泛、更概括”**，而一个项目中可以有若干个设计和开发阶段（ISO 9000:2026 §3.3.11 注 1）。后一句让设计开发在项目内部层层往下走：

客户需求（宽泛）→ 概念设计 → 系统需求（细化中）
→ 详细设计 → 部件规格（更细）
→ 工艺设计 → 可生产可检验的需求（最细）

概念设计把客户需求变成系统需求，详细设计把系统需求变成部件规格。工艺设计把部件规格变成可生产、可检验的需求，**每一级都在执行同一个 §3.3.11**。每个阶段的输出需求，就是下一阶段的输入需求，直到需求足以支撑实现和验证。

这对 M3 意味着什么？林涛以为“设计开发 = 做一次技术方案”。不对，M3 的温度控制系统需求 → 制冷子系统规格 → 压缩机与温控板件规格 → 生产工艺需求，是四五个设计开发阶段叠在一起。任何一级的“更详细”没做透，下游就在半成品上开工。这也是为什么终点那么要紧：总得知道整条路径在哪里算走完。

设计开发内部怎么运转？有三个动词常被误当成三个并列的流程：构思（conceiving）、定义（defining）、设计（design）。它们不是并列，是包含，构思和定义都发生在设计开发这个范围里面。

> **构思（conceiving）**：从利益相关方关切出发，生成多个候选方案的发散性活动：产物是“可能是什么”，不是“决定是什么”。
>
> **定义（defining）**：在候选之间画边界，使选定方案从“多义”变为“单义”、从“漂浮”变为“确定”的活动：把可能性收敛为一个被承诺的方案。

| | 构思 | 定义 | 设计开发 |
|---|---|---|---|
| 一句话 | 打开可能性 | 终结漂浮 | 转换到底 |
| 干什么 | 发散：生成至少两个候选 | 收敛：画边界、挑定一个 | 把需求转化为更详细的需求 |
| 产物 | 候选清单 | 选定的方案 | 更详细的需求（规格） |
| 位置 | 内·发散段 | 内·收敛段 | 整体 |

设计开发“把需求转化为更详细的需求”，转化不是一步到位的：先想，也就是构思，这个需求有哪些“怎么做”的候选；再选，也就是定义，在候选之间画边界、挑定一个。发散在前、收敛在后，合起来才是“转化”。

所以“设计开发包含构思与定义”，是包含关系，不是三个并列的流程。这落到第 6 章的结对六步，就是③出方案与⑤做裁决；**每一层设计开发内部，都重复这一轮构思、定义、再细化**，区别只在层级。

> 术语注：构思与定义这两个动词取自 ISO/IEC/IEEE 42010:2022 架构工作（architecting）§3.1 的动词串（正本中译作“构想”），**上面两条定义句是本书的梳理**；“设计”一词有宽窄两义（宽义是装下构思与定义的容器，狭义是 15288:2023

> §6.4.5 的向下细化），与 ISO 9000:2026 §3.3.11 的“设计开发”在这里按整体与部分对接：“设计开发”是整体，构思与定义是它内部的两段。

设计开发是递归的，落地时每一级都是一对“需求—方案”的结对，走完六步：接需求、验就位、出方案、对需求、做裁决、签定稿（见 §6.4.1）。

### 3.2.6 质量：固有特性满足需求的程度

第三个词，也是全章最容易被误解的一个。

> **质量（quality）**：客体（§3.5.3）的一组固有特性（§3.10.1）满足需求（§3.5.1）的程度（ISO 9000:2026 §3.5.2）。

三个关键词，一个都不能松。

**固有特性**——存在于客体本身的：功能、性能、材质。“固有”与“赋予”相对，是指存在于客体之中的特性（§3.5.2 注 2）。赋予特性不算——正本举的例是“某客体的价格”（§3.10.2 注 2），品牌、交付时间同属此类，后两例是本书的扩举。

这意味着设计开发的范围也被“固有”框住；价格、交付周期是其他过程的事，设计开发管不着，也不该管。但成本是另一回事：成本作为“经济可承受性”需求，仍会进入设计输入（见 §5.4.8）。管不着的是“价格”这个赋予特性，成本需求本身要接。

林涛在评审会上比“精度、保温时长、重量”，这些是固有特性，比对了；但要是比“价格比竞品便宜”，那不是质量，是经营。镜照一眼恒信：合同审批系统的固有特性是查询响应时间、正确性；“用的是哪家平台”“上线时间”是赋予特性，设计开发管不着。林涛式的“比

参数”，在 IT 世界里换成“比平台名气、比上线速度”，一样比错了地方。

**满足**——符合性判定，不是数量比。特性达到需求规定的水平就是满足，达不到就是不满足。这是一次判定，不是一道除法。质量问的是“结果在标准上的位置”，不是“标准除以结果”，两边不同质，一个在客体里，一个在客体外，除不了。

**程度**——差、好、优秀，形容词，不是数字（§3.5.2 注 1）。质量是被认定出来的，不是被计算出来的，靠验证与确认拿客观证据判定。

这里藏着全书的一个方法时刻：**判据主义**——不给没有判据的定义（第 1 章 §1.8.3）。质量的定义，恰恰把“质量好”从一个主观印象，变成了一条可检验的判据：特性满足需求的程度。程度可以用测量、比对、检验这些客观证据认定，因此“质量好不好”可以争论、可以复核、可以裁决。这不是给质量泼冷水，这是让质量第一次成为工程上可用的判据。整章后面每一次划线，用的都是它。

还有一处常被忽略：质量定义里的“需求”是广义的，明示、隐含、必须履行三种形态并列（§3.5.1），规定需求只是它的子集。定义这一层，质量已经把隐含需求罩进去了。但要衡量质量，就必须落到可衡量的规定需求上。隐含需求不显性化成文，就验不着、量不出。这个“定义宽、衡量窄”的落差，正是需求必须显性化的根源（第 5 章展开）。

“质量”这个词的来处，同样是一次漂移：quality ← 拉丁 qualitas（“如何”）。它原指“档次、用料”，好东西就是好质量；ISO 9000 把它纠正为“特性满足需求的程度”，与档次无关（呼应第 1 章 §1.8.3）。这一漂，是本章要立的最大一个反转。

### 3.2.7 概念卡片

需求、设计开发、质量都亮过相了，档案收在这里，按四问查：

## 概念卡片 · 需求

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）——定需求，定义“要什么” |
| **它不是什么** | ⑥⑦ | 对立：点子、创意——不表述为需求就进不了设计开发的流程；辨析：需要是内容里的硬性诉求、期望是内容里的软性诉求、需求是内容乘呈现方式；需求不等于目标，也不等于目的 |
| **它从哪来** | ②③⑨ | requirement ← 拉丁 requirere（要求、寻求）；“需求”原指“要的东西”，后漂移出明示、隐含、必须履行三种形态（第 1 章 §1.8.3）；ISO 9000 把它固化为标准定义 |
| **它往哪去** | ④⑤⑧ | 它回答“要什么”，是设计开发的输入、质量判定的依据；在需求获取、需求评审、变更控制时用；误区 ✗ “需求 = 用户说出来的”（隐含需求与必须履行的需求漏掉就是缺陷）✗ “需求 = 目标”（目标回答“达成什么可度量的结果”，需求回答“做成什么样”；回答“为什么做”的是目的） |
| **判据** | 可转化 | 起点判据“可转化”：完整、无歧义、不冲突、隐含需求已识别（见本章 §3.3.2） |

## 概念卡片 · 设计开发

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 将某一客体的需求转换为该客体更详细需求的一组过程（ISO 9000:2026 §3.3.11）——逐级出规格，直到可实现 |
| **它不是什么** | ⑥⑦ | 对立：走流程——输入输出同样详细，就是没发生设计开发；辨析：设计、开发、设计开发三个词的关系；需求开发（ISO 9001:2015 §8.2）与设计开发（§8.3）的边界看“特性有没有被规定”（见本章 §3.3.3）；构思与定义是设计开发内部的发散段与收敛段，不是并列的流程 |
| **它从哪来** | ②③⑨ | design ← 拉丁 designare（标注、指明）；“设计开发”由“设计”“开发”两个词合成，但不是两件事的简单拼合——ISO 9000 用 design and development 完整表述需求转化这一件事，中文标准统一译作“设计开发”，强调它是整体 |
| **它往哪去** | ④⑤⑧ | 它回答“怎么把要什么变成做得出来、验得了的规格”；从需求进入设计，到规格足以支撑实现和验证时止；误区 ✗ “设计开发 = 画图或创意活动”（是需求转化）✗ “技术方案做完 = 设计开发完成”（下游方案配齐才算，见本章 §3.3.5） |
| **判据** | 可实现 | 终点判据“可实现”：还有哪个环节会在执行时反问“这到底怎么做”？有，设计开发就没结束（见本章 §3.3.5） |

## 概念卡片 · 质量

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 客体的一组固有特性满足需求的程度（ISO 9000:2026 §3.5.2）——判满足，判出满足到什么程度 |
| **它不是什么** | ⑥⑦ | 对立：等级——等级高不等于质量好；辨析：符合性回答“满足了吗”，质量回答“满足到什么程度”，等级回答“按哪套需求分类”（见本章 §3.3.6） |
| **它从哪来** | ②③⑨ | quality ← 拉丁 qualitas（“如何”）；“质量”从日常的“档次、用料”漂移到“符合需求的程度”——ISO 9000 纠正了“好东西就是好质量”的直觉（第 1 章 §1.8.3） |
| **它往哪去** | ④⑤⑧ | 它回答“东西做出来到底好不好”；在评审、验证与确认、接收时用；误区 ✗ “特性越多越高，质量越好”（超出需求范围是过度设计，见本章 §3.3.6）✗ “质量能算成一个分数”（见本章 §3.4.2） |
| **判据** | 等级不是质量 | 质量等于特性满足需求的程度，可检验；特性超出需求区间就是过度设计（见本章 §3.3.6） |

## 案例进展①｜没做调研的林涛

冰链科技立项 M3 的第七周，产品、研发、销售开联合评审。

林涛拿着方案书进场，标题是“M3：行业精度第一的冷链温控箱”。他开口第一句：“客户要的是精度——这条需求我们直接做到行业第一。”

销售小冯打断他：“谁告诉你的？上周我拜访了三个省疾控中心，他们问的第一句话全是——能不能塞进我们现有的疫苗运输车？你的 M3 尺寸比现在那款大了一圈，塞不进去。这条需求合同里没写，你也没问过。”

赵刚把笔放下了：“这条，我们谁都没问过客户。”

会议室安静了。

林涛愣住：“……合同里没写这条。”

“是没写，”小冯说，“是没写。但这是他们所有人都在问的——按 §3.5.1，通常隐含的也是需求。你立项前做过市场调研吗？”

林涛没做过。他的起点是一个点子，“把精度做上去，就能赢”，跳过了研究，直接进了设计开发。七周后，方案书被推倒重来，重新做尺寸、重新评估保温层厚度、重新谈供应链。

这一轮返工，损失的七周，就是起点没收好的全部代价。设计开发转化的是需求，起点给错了需求，转化得越彻底，浪费得越彻底。

---

## 3.3 起点、终点与参照，各有一把判据

> 这一节把相邻两个词之间最容易混的地方分开。概念靠正例活，靠反例定型：起点那处边界在需求与设计开发之间，终点那处在设计开发与质量之间，参照那处在需求与质量之间。

### 3.3.1 设计开发不始于点子

边界最容易打架的地方是交接处。第一处在需求与设计开发的交接处：设计开发从什么时候开始?

先说清“开始之前”。设计开发要拿到被研究确认过的需求，才谈得上开始：

> 点子、灵感、想法 →（市场调研、可行性分析、技术预研、客户需求识别）→ 确定的需求 → 设计开发开始

为什么必须插这一步? 因为设计开发的输入是需求，而输入的需求（按 §3.3.11 注 1）往往是研究的成果。一个没经过研究确认的点子，设计开发会把它老老实实做成一个“错误的需求”的完美实现。它不会自己发现方向错了，它只管把给它的东西转化到底。

很多项目“设计得很辛苦、产品没人要”，根源不在终点没管好，而在起点没收好。错的不是施工，是拿到的需求本身就错了。

### 3.3.2 起点的判据：需求要可转化

那“开始”这一步怎么算过? 设计开发的起点，是一份需求规格被正式确立为设计输入的时刻。而这份需求规格，必须先满足“可转化”判据，四条前提，一条都不能松：

1. 完整——功能、性能，以及由产品和服务性质导致的潜在失效后果，一个不缺。M3 不只是“能制冷”，还包括“制冷失效时自动告警”“断电后保温时长”“结霜影响”这些潜在失效的处置。缺一项，设计开发就可能在错误的方向上越走越远。
2. 无歧义——每条需求只能有一种理解。“精度要高”不是需求，“箱温波动不超过 ±0.5℃”才是。歧义就是留给未来的争吵。

3. 不冲突——需求之间自洽。“重量轻 2 公斤”和“保温 72 小时”可能打架（保温靠保温层，保温层厚则重），这要在需求阶段协调掉，而不是等设计开发做完再发现。
4. 隐含需求已识别——最容易被忽略、却最决定规格对不对的一条。它没写进规格，却决定规格对不对；它不会自己出现，需要主动去挖。

这四条前提不是拍脑袋凑出来的标准，而是“设计开发就是需求转化”这一定义想成立所必须具备的条件。逐条反推就知道它们躲不开：

- 需求有缺口，转化在残缺的输入上开工，输出必然残缺——转化不可靠；
- 需求有歧义，同一份输入生出多种理解——转化没有确定结果；
- 需求冲突，满足 A 就违反 B——转化无解；
- 隐含需求没识别出来，方向就错了，验证全绿、确认全红——转化白做。

所以可转化判据的正当性不来自任何更高层的东西，就来自定义本身。标准（ISO 9001:2015 §8.3.3）做的，只是把这四条前提写成了条款。

> 出处提醒：本章引 ISO 9001:2015 的条款号，**首次出现处带标准名与年份（ISO 9001:2015），其后同一节内可简写；表格与对白各自独立起算**。相邻的规范表述见 ISO 9000:2026 §3.3.11 注 1 与 §3.5.1 注 2。

> 起点没收好，转化的是错误需求——终点越完美，偏离越远。

### 3.3.3 需求开发的边界：特性有没有被规定

管理评审时一定会遇到一个问题：市场调研、可行性研究、URS（用户需求说明书）这些文档的开发，算不算设计开发?

概念上：需求开发也算设计开发，而且同构。§ 3.3.11 的定义不挑阶段：客户说“我要一台能低温运行的小型设备”，需求开发把它变成“工作温度 -40℃、体积不超过 0.5m³、……”，这本身就是一次“需求变详细”。需求开发也是需求转化，同样具备 § 3.3.11 的定义特征。它和设计开发是同一种认知活动的不同阶段，只是转化发生在更靠前、更粗的层级。

标准组织上：两者可合可分。ISO 9001:2015 把需求确定（§ 8.2：顾客沟通、需求确定、需求评审）和设计开发（§ 8.3）分成两条，不是因为“需求开发不是需求转化”，而是职责的组织方式不同。§ 8.2 管“要什么”（面向顾客与市场），§ 8.3 管“怎么实现”（面向技术与实现）。复杂组织常把需求开发作为设计开发的第一阶段纳入 § 8.3；简单组织放在 § 8.2 即可。两种策划都合规。

实用判据：看特性有没有被规定。这是把质量定义（§ 3.5.2“客体的一组固有特性”）用起来的硬判据：

> 需求开发阶段，客体还没确立，特性压根不存在，质量没有承载体；设计开发阶段才开始“把需求落到客体的固有特性上”。
>
> 特性还没开始规定，是需求开发（ISO 9001:2015 § 8.2）；特性开始被规定，才是设计开发（§ 8.3）。需求规格就是那道分界。

必须补的一句提醒：不管市场调研、可行性研究、URS 这些文档归 § 8.2 还是 § 8.3 的第一阶段，它们都是设计输入的来源，直接影响产品质量特性，审核时“这些文档不在受控范围”是过不去的。它们可以不属于“设计开发”，但不能不受控、不可追溯。林涛的“M3 立

项调研”如果能受控存档，案例进展①那七周返工至少不会连个“当时为什么这么做”的记录都没有。

### 3.3.4 终点的判据：为什么不是可验证

第一处边界在起点，第二处在设计开发与质量的交接处：设计开发什么时候算结束?

绝大多数团队（包括冰链科技）说“技术方案做完就结束了”。但结论不能拍脑袋，得把候选终点一个一个删掉：

| 候选终点 | 为什么被否定 |
| --- | --- |
| 确认通过 | 错位：确认的判定对象是已可使用的客体、发生在设计开发形式结束之后很久；而设计开发这一段锚定的是“需求” |
| 责任移交点 | 只是制度动作（设计基线发布），不是定义内部的条件 |
| 可验证 | 逻辑倒置——详见下面 |
| 可实现 | 定义内部唯一自然的终止条件 |

“可验证”为什么不对，这是全书最要紧的一处推理，值得单独看。

验证（verification）的定义是：通过提供客观证据，证实规定的需求已得到满足（ISO 9000:2026 §3.11.12）。注意：验证的对照物是规定需求，而规定需求正是设计开发的输出。也就是说：

> 验证能发生，前提是设计开发已经结束（已经输出了规定需求）。拿“可验证”当设计开发的终点，等于让设计开发结束在验证的起跑线上——循环倒置。

更致命的是第二层：验证的注 1 说，客观证据可以是检验的结果，也可以是其他确定形式的结果，如进行替代计算或评审文件（§3.11.12 注 1）。那么几乎所有写清楚的需求都“可验证”，这条判据区分不了完成与未完成。你说“需求写完了所以可验证了所以设计开发完了”，那到底是“写完了”还是“可验证”？绕回去了。

所以终点只能落在“可实现”：

> 设计开发止于“可实现”：需求被细化到足以支撑客体实现、生产和服务提供可直接执行的程度（输出成为完整的规定需求）。
>
> 验证与确认都在终点之后，各对一把尺子。验证对照这份规定需求；确认对照“针对特定预期用途或应用的需求”（ISO 9000:2026 §3.11.14 正本中文用字）。“可实现”是入场券（必要条件），验证通过是及格线（符合），确认通过才算在真实场景里站住——但终点在验证之前。

顺带把确认的位置摆正。它的判定对象是已可使用的客体，对照物是“针对特定预期用途或应用的需求”（§3.11.14），使用条件可以真实也可以模拟（§3.11.14 注 3）。它不是设计开发内部的终点，而是终点之后的终审。

### 3.3.5 终点的样子：方案配齐，指令可执行

“可实现”在实物产品上是什么样子？下游各个环节要用的方案，全部完成：

技术方案 • 物料采购方案 • 制造方案（工艺）• 测试方案
交付方案 • 部署方案 • 操作方案 • 运维方案

这些方案不是彼此独立的几件事，是一样东西的不同侧面。“规定需求”按下游环节分配下去：采购照着买、生产照着做、检验照着测、实施照着装、用户照着用、维护照着修。每个方案，都是给一个下游环

节的“可实现需求”。这正是“满足后续产品和服务提供过程的需要”（ISO 9001:2015 §8.3.5 b）的完整展开。

> 术语注：“止于可实现”与这份方案清单是本书的主张——本书对 ISO 9001:2015 §8.3.5 b（设计输出应满足后续产品和服务提供过程的需要）的操作化，不是 ISO 原文的列举。标准给的是判据方向；拆成哪几个方案、怎么命名，由组织按自己的下游环节裁剪。

每个方案对应的条款，都对着下游过程的需求：

| 方案 | 对应条款 | 下游环节 |
| --- | --- | --- |
| 技术方案 | ISO 9001:2015 §8.3.5 d：规定对预期目的至关重要的特性 | 设计本身 |
| 物料采购方案 | ISO 9001:2015 §8.4：外部提供的需求 | 采购 |
| 制造方案（工艺） | ISO 9001:2015 §8.5.1：生产和服务提供的受控条件 | 生产 |
| 测试方案 | ISO 9001:2015 §8.3.5 c：监视和测量需求 + 接收准则 | 检验与验证 |
| 交付、部署、操作、运维 | ISO 9001:2015 §8.5.1 及生存周期各环节 | 交付、实施、使用、维护 |

退役与回收不在默认清单内：本章未在库内核到 ISO 9001:2015 的退役环节对应条款；退役处置属 ISO/IEC/IEEE 15288:2023 的处置过程（§6.4.14）。它的需求侧由第 5 章的可回收性需求承接（§5.2.5），方案侧可以按下游环节增删。

还有一处用词要精确：“方案”不是“结果”。测试方案属于设计开发：它规定测什么、判据是什么；测试的执行和测试结论属于验证。停在“方案完成”这一步，正好守住“设计开发止于可实现”这条线。

> 终点由“最后一个配齐的方案”决定，不由“第一个完成的技术方案”决定。

为什么技术方案不够？因为技术方案只回答“客体本身长什么样、怎么实现”，它是这些方案里最先完成、最显眼的一个，但只是一个侧面。只做到技术方案就宣布终点，后果是下游连环停摆：

- 采购没有物料规格和接收准则——采购停在原地；
- 生产没有工艺依据——产线开不了工；
- 测试没有判据——验证无法启动，季昭那句“测试判据是什么”没人能答；
- 交付、部署、操作、运维没有指令——产品出门即失联，客户拿到手不知道怎么装、怎么用、怎么修。

镜照一眼软件世界。恒信集团的合同审批系统，这些方案长这样：技术方案、数据方案、接口方案（与法务、财务系统对接）、测试方案（验收标准）、部署方案、操作方案、运维方案、安全方案。

一个不少，只是名字从“物料、制造”换成了“数据、接口、部署”。凡是只做了技术方案就宣布“设计完成”的软件项目，和只有技术方案的 M3 一样，都会在下游连环停摆。

“可实现”这个词容易引起误会，好像设计开发完了，东西就能跑起来。所以要分清三层：

| 层次 | 谁给的 | 性质 |
| --- | --- | --- |
| 原理可实现 | 技术方案 | 纸上走得通 |
| 方案可实现 | 方案全部配齐 | 设计开发的终点——从“原理可行”跨到“可执行指令齐备” |
| 事实可实现 | 验证与确认 | 终点之后——“能不能真正实现并运行”由它们裁决 |

落到 M3：双压缩机冗余在纸面上完全成立，这是原理可实现。方案配齐、可执行指令齐全，这是方案可实现，也就是设计开发的终点。真造出样机、在真实冷链运输里跑通验证与确认，才是事实可实现，而这一层，冰链连边都没摸到。从“纸面走得通”到“真能跑”，中间隔着整个验证与确认。

设计开发交付的不是事实，而是一份声明，一份可信的可实现声明。方案齐备、逻辑自洽，靠评审、样机、仿真来提高把握，但最终裁决权在终点之后的验证与确认。守住这条界线，设计开发就不会把“我们觉得能做”当成“我们做完了”。

这就是第二处边界的全部内容：设计开发把裁决权交在终点之外，它只负责把规格做到“可实现”，把“到底行不行”交给质量判定。各管一段，边界在这里。

### 3.3.6 参照的判据：等级不是质量

第三处在需求与质量的交接处：中文里最容易和“质量”混的，是“等级”和“符合性”。符合性、质量、等级不是同一个意思的不同说法，而是回答三个不同的问题：

| 词 | 回答的问题 | 形态 | M3 的例子 |
|---|---|---|---|
| 符合性 conformity | 满足了吗? | 二者取一（符合或不符合） | 箱温波动是否在 ±0.5℃ 以内? |
| 质量 quality | 满足到什么程度? | 程度（差、好、优秀） | 一组特性综合下来，好不好? |
| 等级 grade | 按哪套需求分类? | 分级（高档、低档） | M3 是高配冷链箱，还是经济型? |

三者都以“需求”为对照，而需求是设计开发定出来的。等级的定义在这里给全：对功能用途相同但需求不同的客体所做的类别或级别划分（§3.5.4）。

关键的一处反直觉：等级高不等于质量好。高档车可以是差质量，功能设计有缺陷；低档车可以是好质量，它按自己的需求做到了极致。因为等级是“按不同需求分类”，质量是“满足各自需求的程度”。M3 把精度做到行业第一，它只是等级定得高；如果需求只要 ±0.5℃，那么超出的部分对“质量”没有贡献。

这条边界走到头，就得到过度设计的一条硬判据：特性超出需求范围的，不是质量加分，是过度设计，只加成本，不加质量。这给“砍需求”提供了一条干净的标准：砍过度设计，是唯一“既省时间又不伤质量”的操作，不是牺牲质量，是给质量做减法。

回到冰链那场评审会：林涛比的“±0.3℃ 对 ±0.5℃”是等级之争；要问质量，得先问需求写的是 ±0.5℃ 还是 ±0.3℃。需求范围之外的那 0.2℃，就是过度设计的起点。

镜照一眼恒信：合同审批系统的需求是“合同在三个部门间流转审批”。设计开发把这条需求做进审批流设计。质量读的是查询响应时

间、审批时效、正确性，同一套“定需求、出规格、判满足”，只是从“温度精度”换成了“审批流转”。

一个硬件、一个软件，起点、终点、参照这三处边界画开的方式一模一样。这正是本书用两条线讲故事的原因：这套概念不挑行业。

起点、终点、参照画完，“判据主义”的手法也亮了相，概念靠正例活、靠反例定型。等级靠“高档车可以是差质量”定型，过度设计靠“超出需求范围只加成本”定型，终点靠“可验证是循环倒置”定型。每一处边界，都有一条可检验的判据收口。

## 案例进展②｜只有技术方案的 M3

时间回到章首那场评审会之后。

M3 的技术方案确实漂亮：双压缩机冗余、±0.3℃ 恒温控制、全数字监控。林涛拿着它宣布“设计完成”，产品进入“等待量产”。

量产准备会上，采购先问：“压缩机我们按什么型号采？物料采购方案的接收准则是什么？”——采购方案：没有。

制造与供应链的小周顺着技术方案往下追。他翻到装配那页：“装配工艺规程呢？产线上按哪份文件装、用什么工装？设计开发止于可实现，这两条不落纸，规格就没走完。”林涛没接上。他又翻到测试那节：“低温密封测试的接收准则写在哪？不合格判到哪一档？”还是没接上。他敲着纸面说了最后一句：“本章 §3.3.5 说终点是方案配齐——制造方案和测试方案都不在，这条线没走完。”

季昭接着问：“验证的对照物是规定需求。规定需求在哪份文件里？接收准则写在哪份测试方案里？”——测试方案：没有。他章首那句被淹没的问话，在这里第二次冒出来，这次没人能再忽略。

系统工程师小张把技术方案翻回第一页：“季昭问的那份规定需求，不是一份文件，是每一级交出去的那一份。一条需求往下一级走，

要答两问：下游做不做得出来、验的时候拿什么判。两问答得出，这一级才算细化到可实现，这就是 §3.3.5 的终点判据。装配工艺规程和接收准则不是缺了两份方案，是这条链断了两级。”

交付团队问：“客户拿到箱子，怎么装、怎么部署、怎么培训？这几条下游需求，谁写？”——交付方案、部署方案、操作方案：都没有。

云端软件的吴浩问：“追溯平台的数据谁备份、App 断连了谁发现？运维方案里这两条需求写在哪儿？”——运维方案：没有。

五轮追问下来，八个方案里七个“没有”。会议室从“技术方案真漂亮”的气氛，跌进“我们离能交付还差得远”的沉默。

石军，那个一直不怎么说话的嵌入式硬件工程师，说了句：“我们以为设计开发已经结束了。其实按 §3.3.11，它连一半都没走完。”

设计开发止于“可实现”，方案配齐的那一刻。八个方案里冰链只完成了一个（技术方案），就宣布了胜利。

> 还有哪个环节会在执行时反问“这到底怎么做”？——有，设计开发就没结束。

---

## 3.4 读出程度，解开冲突，合起来看

> 这一节把定需求、出规格、判满足这条路径用起来：先读出结果，再解开冲突，最后看它们为什么互相定义、拆不掉。

### 3.4.1 程度怎么认定：从特性值到结论

上一节把“程度”这个词说清了：它是认定出来的，不是计算出来的。认定分三步，一步不缺：

第一步　确定特性值（§3.11.1）　　查明一项或多项特性及其特性值
手段：测量（定量）、检验（符合性）、试验、评审、计算
产出：客观证据（§3.8.6）

第二步　与需求比对　　判符合性（§3.5.9）
用接收准则——设计输出 ISO 9001:2015 §8.3.5 c 里提前写好的判定标准

第三步　认定满足程度　　得出质量（§3.5.2）
综合一组特性的符合情况，认定程度：差、好、优秀
动作上就是验证（§3.11.12）与确认（§3.11.14）

每一处锚点都缺不得：工具在 §3.11.1 确定、§3.8.6 客观证据，判定在 §3.5.9 符合性、结论在 §3.5.2 质量；动作落在 §3.11.12 验证与 §3.11.14 确认；对照物是 §8.3.5 的接收准则，写在测试方案里，它是终点那批方案中的一个。缺了测试方案里的接收准则，程度就出不来。

这里有一处重要的不对称：

- 单项是二者取一——每一条特性：符合，或者不符合；
- 整体是程度——综合一组特性：差、好、优秀。

整体程度按特性的重要程度综合评价，关键特性优先，即 §8.3.5 d 说的“对预期目的至关重要的特性”。这是第 8 章评审与质量判定的底子，这里先立起来。

“接收准则”落到纸面是什么样子？M3 的测试方案里必须写清楚：箱温波动在 ±0.5℃ 以内判“符合”还是“不符合”，用什么设备测、测多久、抽多少样本，判定结论写在哪份记录上。没有这几样，程度就出不来：你说 M3“质量好”，拿什么证据？哪条特性符合了、哪条没符合、符合到什么程度？

接收准则就是那份提前写好的判定标准，它让“质量好”从一句感觉，变成一次可复核的认定。这也正是测试方案必须占终点一席的原因：没有接收准则，质量永远只能“感觉”，不能“认定”。

镜照一眼恒信：合同审批的“查询响应”怎么读？测试方案里写清楚：在多大数据量下测、多少并发、响应超过几秒判“不符合”、结论记在哪张记录上。同一套三步，只是把“温度”换成了“响应”。

### 3.4.2 分值为什么靠不住

既然“程度”是形容词，那能不能给它打分，比如 M3“质量 85 分”？

标准这一层：没有。符合性、质量、等级三个近亲都不是分值，而且标准通篇没有给质量一个单点分值：特性是多维的、彼此不同质，加不成一个数。隐含需求无法完整数值化。§3.5.2 注 1 也明说，质量用形容词修饰。

组织这一层：分值可以造，但它是组织的度量，不是标准概念。打分等于三步：定权重，即特性重要程度；单项评分，把接收准则映射成分数；加权汇总，得到“85 分”。三个前提，缺一个分就是假的：需求完整、特性规定完整、证据可靠，三样分别落在起点、终点与验证。

最要命的一条：关键特性不能被摊平；ISO 9001:2015 §8.3.5 d 说“对预期目的至关重要的特性”要优先。加权公式若把安全性、可靠性这种一票否决的特性摊进平均数，安全项失了分，被“精度超强”补回来，结果还是 85 分。这个 85 分，就是用假象盖住了致命问题。分值的最大陷阱：它给了你一个精确的错误。

## 案例进展③｜平均分遮住的致命项

质量评审会上，季昭拿着刚按设计参数估算出来的评分表：“我给 M3 的质量打了 85 分——温度控制 92 分、保温 88 分、外壳做工 90 分、重量 70 分……”

石军打断他：“权重怎么算的?”

季昭：“平均。”

石军指着评分表一行小字：“你把'断电告警'放进去了吗?”

季昭：“放了。我把它当普通特性，权重和其他项一样，20 分里得 18 分。”

石军：“断电告警是保命的。ISO 9001:2015 §8.3.5 d 说它是'对预期目的至关重要的特性'，不是平均数里的一项。它不合格，产品就不该上市。你把它摊进平均数，安全项失 2 分，被'温度控制 92 分'补回来，最后 85 分——这 85 分，是在替一个致命缺陷打掩护。”

小张把这一条记进清单：「断电告警」单列，不进平均分。

会议室安静了。石军继续说：“要么给关键特性设一票否决，要么用加权而不是平均。否则这个分数，就是给了你一个精确的错误。”

季昭默默改评分规则。他在心里记下：打分看起来比“程度”精确，但精确的前提是把权重量对，量错了，精确就是假象。

### 3.4.3 质量与进度：改需求，不改质量

读出结果之后，一定会撞上一个绕不开的冲突：质量与进度。这场冲突是一场被误判的取舍。

先拆掉两个站不住的前提。

第一个：质量不是时间的函数。质量等于特性满足需求的程度，和花多少时间没有必然关系，你不能说“多给我三个月，质量就自动变好”。

第二个，也是更深的那个：进度本身就是一种需求。按 ISO 9001:2015 §8.2.2（R-03）：交付时间和交付条件也是顾客需求的一部分。所以“质量与进度”本质是同一份需求集合内部打架：性能与

功能需求，对上交付时间需求。不是两个系统在拔河，是一份需求清单自己起了内讧。

解法不是二选一，是重新平衡这份需求集合。

第一步：判真伪。一半以上的“质量进度冲突”是估算失真、计划虚报、范围蔓延造成的假冲突，进度算错了，却怪质量太慢。假冲突，改计划，不动质量。M3 的“八周做不完”里，有多少是因为七周返工浪费的？那一部分不是质量的问题，是起点欠下的账。

第二步：用需求谈判，不用质量谈判。调整的是需求集合（哪条可降、可删、可延），不是“质量原则”。必须走正式变更（ISO 9001:2015 §8.2.4 需求的更改、§8.3.6 设计更改）：书面、评审、通知利益相关方。口头“差不多就行”是最危险的变更形式。需求被悄悄换掉，验证与确认全在跟一个没被承认的靶子打。把“实时推送”降成“两小时同步”这类降级，必须落在变更记录里，而不是散会后的记忆里。

第三步：分清可妥协与不可妥协。

- 不可动：必须履行的需求（安全、法规，§3.5.1 里“必须履行的”那一类；法规来源见 §3.5.5／§3.5.8）、关键特性（§8.3.5 d 对预期目的至关重要的那些）。“断电告警”就在这一类；
- 可动：非关键特性、性能上的富余、过度设计（可砍，见本章 §3.3.6）。

第四步：度量环节可缩不可砍；最蠢的赶进度方式是“先交付，验证确认后补”。验证与确认是质量的度量，砍掉等于不度量就宣布合格。风险全部后移成现场返工，而失败成本是质量成本里最贵的。正确的做法是：把验证范围缩到关键特性，而不是把验证整个砍掉，只验关键的，别全验；但不验，不行。

三条禁忌，一条比一条贵：

1. 砍验证与确认赶进度——最贵，风险后移，返工爆炸；

2. 口头改需求——最危险，需求被偷换，整条定义链失效；
3. 把“降需求”说成“降质量”——概念错。等级（§3.5.4）不等于质量（§3.5.2）：降低需求水平（砍非关键）是合法变更；交付不满足已承诺需求的东西，才是真降质量。

正道：与利益相关方一起剪裁需求。

> 单方面砍需求叫偷换，和客户一起剪裁才叫变更，中间差着整个需求的所有权。

为什么“一起”是硬要求？四条理由，层层递进：

1. 需求的所有权在利益相关方手里。需求（§3.5.1）是 need 或 expectation 的表述，是利益相关方（尤其客户）的资产，组织只是受托转化。单方面剪裁等于拿别人的东西替别人做主：剪掉的那块可能恰好是客户最在乎的隐含需求——他没说，但你剪了，事后必炸；
2. 剪裁改变的是验证的靶子。验证（§3.11.12）对照规定需求——需求一剪，靶子就变了。不和利益相关方一起剪，验证就是在打一个自己偷偷移动的靶子；
3. 剪裁的正确动作是排序，不是删减。让利益相关方排优先级：命根子（不可动）、想要的（可降）、贪多的（可剪）；
4. 剪裁要留痕，剪下来的部分要存档。走 §8.2.4 与 §8.3.6 的正式变更：剪了什么、为什么剪、谁决定的、哪天恢复，全部记录。剪掉的需求不是消失了，是挂账：这一轮进度解了，下一轮迭代或改型时就是现成的输入。剪裁是暂存，不是销毁。

把需求、设计开发、质量放回原位看：质量等于特性满足需求的程度，需求集合变了，质量判定跟着变。所以“解进度冲突”从来不是牺牲质量，而是和利益相关方一起，重新把参照摆正。

## 案例进展④｜进度不够，客户被迫说真话

M3 的量产进度亮起红灯，逼出了第一次真正的剪裁：距离向省疾控中心首批交付还剩八周，可制造方案、测试方案都还没影。

小周把产线的日历摊开："装配工艺、低温密封测试、工装夹具，排进产线就要六周；压缩机供应商谈了两家，都卡在'接收准则是什么'。这两份方案一天不定，产线要的可实现需求就一天没落纸。"

林涛这才明白，"八周做不完"不是缺时间，是缺制造和测试两个方案，量产准备会上那七个"没有"里的两个。

会议室里，对面坐着疾控中心的刘主任，这位"客户"其实是采购方代表。

林涛硬着头皮开口："刘主任，进度不够了。我们可能得砍掉一部分需求才能按时交付。"

刘主任："砍哪部分？"

林涛："比如把'箱门单手打开'这条需求……"

刘主任打断他："不行。护士经常一手抱孩子一手开箱——这条是合同里明示的，也是保命的一条，不能砍。"

林涛："那'App 实时温度推送'这条需求呢？"

刘主任想了想："这个可以降级——不用实时推送，两小时同步一次就行。但断电告警必须保留，那是保命的；按 §3.5.1 的分类，它属于必须履行的需求，不在可谈的范围里，砍了这一条，交出去的就是另一台产品。"

林涛："还有'重量再轻 2 公斤'这条需求？"

刘主任笑了："那条本来就是你们销售自己吹出来的，我从没要过——超出需求范围的，就是过度设计，砍。"

三十分钟，需求清单重新排了序：命根子是断电告警、箱门单手开；想要的是实时推送改成定时同步；贪多的是重量减 2 公斤、宣传用的“行业精度第一”。季昭在旁边记录，每一条都写下谁决定、为什么、何时恢复。

散会后，季昭对林涛说：“你知道吗，我们开产品会开了七周，问客户'你还有什么需求'，刘主任每次都说'没有了'。今天让他砍需求，他十分钟就把真正在意的全说出来了。”

进度冲突，会把隐含需求逼出来。平时问“你还有什么需求”他答不出；你说“进度不够，必须砍一样，砍哪个”，他立刻答得出。

这也是本章最实在的一课。林涛一直以为“质量与进度”是两件事在拔河，真坐下来和客户一起排需求，才发现那是同一份需求集合的内部排序，参照重新摆正。

### 3.4.4 互相定义，所以拆不掉

需求、设计开发、质量各自是什么，起点、终点、参照的边界各画在哪，一场冲突怎么解，最后一步，是把它们放回一起看：

- 需求（§3.5.1）是输入，定义“要什么”；
- 设计开发（§3.3.11）是转化，把需求变成特性化的规定需求；
- 质量（§3.5.2）是判定，认定固有特性满足需求的程度。

> 一个产品开发流程，无非是：定需求 → 出规格 → 判满足。

两条案例线在这条路径上完全同构。冰链定需求：-20℃~8℃ 宽范围、断链即报废；出规格：方案配齐；判满足：箱温精度的符合性。恒信定需求：合同三部门审批流转；出规格：接口、数据、部署方案；判满足：查询响应、审批时效。一个是硬件，一个是软件，这条路径

纹丝不动，这正是这套概念不挑行业的证据，也是全书用两条线讲故事的原因。

说它们撑得起“核心概念”这个位置，得看证据。

证据一：定义互相引用，拆不掉。设计开发（§3.3.11）引用需求（§3.5.1）；质量（§3.5.2）引用需求和特性（§3.10.1）；§3.3.11 注 1 又说需求“通常按特性来定义”。抽掉任何一个，另外两个都悬空，自洽，且不可拆。

证据二：角色不可替代。需求管输入、设计开发管转化、质量管判定，三个角色各司其职，谁也替不了谁。

证据三：能推出领域里现有的全部方法。

- 需求管理、需求评审、变更控制 ← 从“需求”推出；
- 设计输入与设计输出、评审、验证与确认 ← 从“设计开发 + 质量判定”推出；
- 质量控制、质量策划、质量改进 ← 从“质量”推出。

再往外看，主流方法也各有各的位置：

| 类别 | 方法 | 与需求、设计开发、质量的关系 |
| --- | --- | --- |
| 直接展开 | 系统工程（V 模型）、软件工程、硬件工程 | 这条路径的工程化形态 |
| 流程框架 | IPD | 这条路径的流程化、组织化形态 |
| 运行容器 | 过程方法 | 这条路径的运行方式——设计开发本身就是一组过程 |
| 扩展学科 | 可靠性工程、安全性工程 | 质量里“随时间、随失效”的专门化 |

逐个检验。V 模型为什么是 V？因为需求从哪一级分解下去，验证就必须从哪一级收回来。左半边是需求逐级转化，右半边是质量判定逐级回收。

IPD 为什么必须设技术评审点与决策检查点？因为质量判定不能等到最后才看，要在每个关口用“特性满足需求的程度”检查一遍。需求为什么要有追溯矩阵？因为需求是质量判定的参照，参照断了，判定就无据。变更为什么要控制？因为改需求就是改参照，不改参照，验证就是打移动靶。

这些“为什么”都能从需求、设计开发、质量推出来，这就是“核心概念”和“几个相关概念”的区别。

但这里必须补一句刹车，否则前面那些“为什么”就被说满了：需求、设计开发、质量长不出技术本体。

> 工程 = 概念这一侧的方法（需求、设计开发、质量，管“要什么、怎么证明做对”）× 技术本体（物理、材料、算法、工艺，管“怎么做”）。

需求、设计开发、质量能推出的是“如何组织做什么”和“如何证明做对了”的全部方法；它长不出牛顿定律，也长不出快速排序。判别标准只有一条，看问题的主语是谁：

- 主语是“做不做得出、怎么做” → 技术本体（跟着客体走）；
- 主语是“做没做对、值不值得、怎么保证” → 概念这一侧（跟着需求与质量走）；
- 两者都在 → 两者杂交的产物。

真正的产品开发方法，几乎都是两侧杂交出来的，越靠中间，越值钱。

路径上还嵌着两个承重构件：客体（§3.5.3）和特性（§3.10.1）。没有客体，质量那条“客体的一组固有特性”无处安放，设计开发那条“某一客体的需求”也无处安放；没有特性，需求落不到客体上，质量也无从度量。

需求、设计开发、质量的层级也不完全平级。三者同属核心概念这一层，其中需求与设计开发是根概念，质量是派生的枢纽——它被到处引用，却由需求与系统派生出来。最底下站着的是需求与设计开发这两个根；质量架在其上，而把质量接回需求的那段转化，正是设计开发。

> 术语注：“需求与设计开发是根概念、质量是派生的枢纽”是本书对三者层级关系的凝练主张，不是任何标准的术语——ISO 9000 只分别定义它们，不划定层级；层级判定见第 1 章 §1.2.2 与 §1.7.4。

后面第 4 章到第 16 章，除了第 9 章回到“工程师”这个人的角色章，其余都是需求、设计开发、质量在不同方向上的展开。到第 17 章合起来落成一件真实的事，第 18 章回到那个把这条路径背在身上的人。

第 2 章把“工程”说清了：工程不是手艺，是可复现的方法。本章的需求、设计开发、质量，就落在那层底子上；它们不是终点，是后面所有章节的出发点。

### 3.4.5 认知反转

回到周五下午那场评审会。林涛拍着参数对比表说“符合性没有问题”。

现在我们知道，他说错了一个字：他想说的不是“质量高”，是“等级高”。M3 按更高的需求设计制造、精度行业第一，它是高档产品，

等级高。但等级高不等于质量好：高档车可以是差质量，低档车可以是好质量。M3 的精度远超需求（±0.3℃ 对 ±0.5℃），对“质量”没有贡献，只贡献了成本和复杂度，那部分不是质量，是过度设计。

而真正决定 M3 成不成的，是另外两件事，一件在终点，一件在起点。

终点没守住。冰链团队只做了技术方案就宣布“设计完成”，采购、制造、测试、交付、部署、操作、运维的方案一个都没有。季昭那句被淹没的“测试判据是什么”，不是一句无关紧要的问话，它宣告设计开发没有结束。到案例进展②里它第二次冒出来，这次谁都无法再忽略。

起点没收好。林涛跳过研究、从“把精度做上去”的点子直接开工，把七周浪费在一个没有对过需求的方向上。“箱体能塞进标准疫苗车”这种客户开口第一句就在问、合同里却没写的需求，反而在立项七周后才被销售带回来。

本章其实给了三个反转：一个属于术语，等级不等于质量；一个属于流程，设计开发止于可实现；一个属于起点，研究到需求再到设计开发这条链不能跳。它们都从同一个动作出发：

> 把一个你以为懂、其实各指各的词，摆到定义面前；把一条你以为是流程的路径，从起点到终点重新走一遍。

而画起点、终点、参照这三处边界、解这一场冲突用的工具，正是第 1 章那套方法里的**判据主义**，每个概念都给一条可检验的判据。质量等于特性满足需求的程度，设计开发止于可实现，需求要过“可转化”。需求、设计开发、质量从此合成一条能走的路径，“东西好不好”这一问也就有了可复核的依据。

用第 1 章的话说一句：对“需求”“设计开发”“质量”这三个词，你原来只是“见过”：背得出定义，说得出“质量好”“设计完成了”。

本章把它们推到“拥有”：能答全四问（各是什么、不是什么、从哪来、往哪去），能用“定需求、出规格、判满足”把任何一个项目从头走到尾，能说出每一处边界的判据。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：本章把“需求”“设计开发”“质量”三个词从**第 ① 级（表征）**推到了**第 ② 级（图式）**。你现在看得出“定需求、出规格、判满足”这条路径怎么接起来，而不只是背得出三个定义。

## 小结

本章是需求、设计开发、质量的入场，也是全书方法论的底子：定需求、出规格、判满足。第 2 章把“工程”说清了，它们就落在那一层上。

第一节把需求、设计开发、质量各自是什么说准：需求（明示、隐含、必须履行三种形态，还要和需要、期望、目的、目标分开）、设计开发（“更详细”是唯一判据，而且层层往下走）、质量（固有特性、满足、程度）。

第二节画开三处边界：起点（研究先于需求、“可转化”的四条前提、特性有没有被规定）、终点（止于可实现、方案配齐、声明不等于事实）、参照（等级不是质量、过度设计），判据主义在这里成了亲身体验。

第三节把路径用起来：读结果（从特性值到结论）、解冲突（质量与进度是同一份需求集合内部打架，与利益相关方一起剪裁）、合起来看（定义互相引用、角色不可替代、能推出领域里的全部方法，但长不出技术本体）。

它们互相咬合、推不翻，还能推出后面所有章节的方法。V 模型为什么是 V、IPD 为什么有技术评审点、为什么要求要有追溯矩阵、为什么变更要控制，答案都在需求、设计开发、质量里。

下一章，我们看需求是怎么被开发出来的：需求工程。

产线计划本上，添了两行字：制造方案、测试方案。以前产线排不动，大家只说“八周做不完”，两份方案从不上纸；本章之后小周明白了：方案不落纸，设计就没到可实现，产线排得再满，也是空转。

---

# 附录

## A 本章概念总图

需求、设计开发、质量的关系，一张图收住：

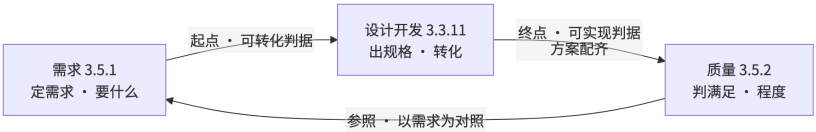



图 3-1

- 第一节 · 需求、设计开发、质量是什么：需求（明示、通常隐含、必须履行，以及需要、期望、需求的分别）、设计开发（“更详细”判据、层层递归）、质量（固有特性、满足、程度）；以及三者各自的来处；
- 第二节 · 三处边界：起点（研究先于需求、可转化的四条前提、特性有没有被规定）；终点（止于可实现、方案配齐、声明不等于事实）；参照（等级不等于质量、过度设计）；
- 第三节 · 运用：读结果（从特性值到结论、关键特性不能摊平）→冲突（质量与进度是同一份需求集合内部打架；与利益相关方一起

剪裁：排序、留痕、挂账）→ 合起来（定需求、出规格、判满足；客体与特性是承重构件；需求与设计开发是根概念，质量是派生的枢纽；能推出领域里的方法，但长不出技术本体）。

M3 走完这张图的正确姿势：定需求（-20℃~8℃ 宽范围、断链即报废的硬需求）→ 出规格（方案配齐）→ 判满足（箱温精度的符合性）→ 全程让利益相关方参与剪裁。恒信集团走的是同一张图，只是从“温度”换成了“审批流转”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把“需求”“设计开发”“质量”三个词说准：明示、通常隐含、必须履行三种形态；设计开发的输入、客体、转换、更详细的需求；质量的固有特性、满足、程度。三者同属核心概念一层，其中需求与设计开发在第 1 章 §1.7.4 的判定里是根概念（那里判出的第三个根是系统），质量是派生的枢纽；
- **图式**——需求、设计开发、质量聚起来形成的结构：“定需求 → 出规格 → 判满足”这条路径，以及路径上的起点、终点、参照三处边界；章题说的“产品开发的图式”，就是它；
- **心智模型**——起点判据“可转化”（完整、无歧义、不冲突、隐含需求已识别）；终点判据“可实现”（方案配齐、执行时没人反问你“这到底怎么做”）；参照判据“等级不是质量”（特性超出需求范围就是过度设计）；
- **解释框架**——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有；这把尺子现在装进你手里了。

## C 自查 · 这些说法对吗?

1. “我们把参数做得比竞品高一倍，质量肯定没问题。”——**错**。参数超出需求范围是等级高、是过度设计，不是质量好；质量等于特性满足需求的程度。
2. “技术方案完成了，设计开发就结束了。”——**错**。止于“可实现”，方案配齐才算；只到技术方案，下游环节连环停摆。
3. “质量进度冲突，砍掉测试先上线，后面再补。”——**错**。验证与确认是质量的度量，可缩不可砍；砍了等于不度量就宣布合格，风险后移成现场返工。
4. “客户没说要，我们就不用做。”——**错**。隐含需求也是需求，漏了就是缺陷；它没进文档，验证够不着，只有确认拦得住。
5. “这个变更只是改个颜色，不用走设计流程。”——对。输入输出同样详细，就是没发生设计开发，可以走快速通道；但“把温控改成双路冗余”这种产生了新需求细节的，就是设计开发，不能图省事。
6. “我们和客户一起砍需求，是降低质量。”——**错**。和客户一起剪裁需求是合法变更（排序、留痕、挂账）；单方面砍需求才叫偷换。
7. “把'成为行业第一'写进需求规格，设计开发就知道怎么做了。”——**错**。那是目的，回答“为什么做”；目标把目的翻译成可度量的结果（比如“到站疫苗活性 ⩾ 95%”），需求回答“做成什么样”，必须可判定。目的定方向，需求必须可判定。
8. “质量能打分，M3 质量 85 分，就是客观结论。”——**错**。程度是认定不是计算；关键特性（安全、可靠）不能摊进平均数——那个 85 分是“一个精确的错误”。
9. “需求开发是需求的事，设计开发是画图的事，两不相干。”——**错**。需求开发也是需求转化，和设计开发同构（同一个 §3.3.11）；边界只看一条——特性有没有被规定。

## D 练习

1. 设计开发的输入必须是“需求”。一个“点子”算不算输入？用你手上的一个产品举例，指出“点子”和“需求”的分界在哪。
2. 设计开发什么时候算结束？“技术方案做完了”够吗？列出至少三个还缺的方案，并说明为什么终点由“最后一个配齐的方案”决定。
3. 质量等于客体固有特性满足需求的程度。特性超出需求范围（过度设计），质量是升了还是降了？为什么？
4. “质量与进度”的冲突本质是什么？砍掉验证与确认赶进度为什么最贵？口头改需求为什么最危险？
5. “和客户一起剪裁需求”和“单方面砍需求”的本质区别是什么？为什么剪下来的需求算挂账，不算消失？
6. 符合性、质量、等级三者各自回答什么问题？“高档车可以是差质量”这句话为什么成立？
7. 用“特性有没有被规定”这条判据，判断你们公司一个正在进行的工作，是需求开发还是设计开发。
8. 检查你当前一个项目：“要什么”（需求规格）满足完整、无歧义、不冲突、隐含需求已识别吗？哪一条最弱？会造成什么后果？
9. 用“它从哪来”量一遍需求、设计开发、质量：各自为什么被造出来、如何演化成今天的定义？哪个词曾从“档次、用料”漂移成“符合需求的程度”？
10. 用“定需求、出规格、判满足”把你最近的一个项目走一遍：需求定全了吗？规格出到“可实现”了吗？读出来的是“程度”还是“感觉”？起点、终点、参照这三处边界，各在哪里容易混？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。


**标准**

- [1] **R-03** ISO 9001:2015 —— 需求确定（§8.2；§8.2.2 交付时间和交付条件也是顾客需求）、设计开发（§8.3：§8.3.3 设计开发输入的可转化判据、§8.3.5 b 满足后续产品和服务提供过程的需要、§8.3.5 c 监视和测量需求与接收准则、§8.3.5 d 对预期目的至关重要的特性、§8.3.6 设计更改）、§8.2.4 需求的更改、§8.4 外部提供、§8.5.1 生产和服务提供的受控条件
- [2] **R-37** ISO/IEC/IEEE 15288:2023 —— §6.4.1 业务或任务分析（任务与目的在那里定义，§3.2.3）、§6.4.2 利益相关方需要与需求定义（§3.2.4）、§6.4.3 系统需求定义（设计开发的起算点，§3.2.4）、§6.4.4 系统架构定义、§6.4.5 设计定义（§3.2.5）；§6.4.14 处置过程（退役，§3.3.5）
- [3] **R-38** ISO 9000:2026（第 5 版）—— 需求（§3.5.1：明示的、通常隐含的或必须履行的需求或期望）、设计开发（§3.3.11：把需求转换为更详细的需求）、质量（§3.5.2：固有特性满足需求的程度）、等级（§3.5.4）、符合性（§3.5.9）、特性（§3.10.1）、客观证据（§3.8.6）、确定（§3.11.1）、验证与确认（§3.11.12、§3.11.14）、目标（§3.7.11）、过程（§3.3.1）、项目（§3.2.11）

**经典文献**

- [4] **R-80** V 模型（V-Model）—— 开发过程模型：验证与确认的 V 形对应（从这条路径推出的主流方法）

- [5] **R-81** IPD（集成产品开发）—— 技术评审点 TR、决策检查点 DCP（从这条路径推出的主流方法）

# 第 4 章 需求工程：把诉求变成需求规格

## 4.1 引言

第 3 章把需求、设计开发、质量三个词立在了产品开发的路径上，需求是那条路径的起点。可需求不会自己站到纸上；它得有人去问、去分析、去写成条文，写成之后还得有人看着它不被悄悄换掉。本章看的就是这件事：一份需求是怎么被做出来的，做出来之后又靠什么一直是那一份。先把这件事分成两段看，再把最容易混的几处边界画开，最后看它在项目里怎么用。

### 4.1.1 误解现场

恒信集团合同审批系统项目启动后的第三周，管理层开了个会。

IT 总监拍板：“需求最近总是变来变去，合同评审、财务对账、法务条款，改一版崩一版。我们要加强需求管理。谁负责？顾言，你来。”

顾言是需求分析师，系统工程师团队里的需求侧专职。他没接话，先问了三个问题：

> “需求为什么会变？变之前，我们的需求开发做到位了吗——调研做了吗？分析做了吗？每条需求写得能当依据用吗？”

会议室安静了几秒。

IT 总监皱眉：“这些不都是需求管理的一部分吗？管好需求，自然就都做了。”

如果你坐在这间会议室里，大概也会点头。因为大多数人对需求的直觉，跟这位总监一模一样：需求乱，是管理没跟上；把需求管理做好，需求就稳了。

本章要推翻的正是这个直觉。它错在一个词上：**需求管理不包含需求开发**。需求是开发出来的，不是管出来的。

### 4.1.2 本章要回答什么

读完本章，你应该能回答：

1. 需求工程、需求开发、需求管理是什么关系？“需求管理包含需求开发”错在哪？
2. 需求开发有哪几个活动？最后一个活动为什么叫校核（评审），不叫确认？
3. 一条需求“说出来了”算什么？为什么口头说的不能当验证的依据？
4. 需求从哪来？除了从目标转化，还有哪些来路？
5. 需求“总在变”的时候，怎么先分清是需求变了，还是当初没说清？
6. 运行概念（ConOps）在整条链的什么位置？为什么“先写条款、后补故事”顺序反了？
7. 需求管理管的是什么？变更控制要走过哪几步才算完整？基线凭什么当分界线？
8. “另存为 v2”为什么不是版本管理？
9. 一条好需求的判据是什么？哪些文档任何规模的项目都不能省？

### 4.1.3 本章的骨架

> 需求工程，是把模糊的诉求做成一份能当依据用的规定需求——开发负责做出来，管理负责让它一直是那一份；两段并列，只在“批准 → 基线”这一处交接。

反过来说更刺人：**管理的严格补不了开发的欠账**。需求乱，多半不是没人管，是没人做。

两段各有各的活动、各有各的判据，中间只隔着一道分界：批准之前需求还在流动，批准之后一切改动都要走变更。本章把这两段分别看清，再把最容易混的几处画开。说出来了不等于能当依据（明示与可采）、需求乱得先分清是谁的账（开发与管理）、需求文档能评审也能验证，做不了确认（评审与验证确认）、给文件改名不等于版本管理（留档与受控）。

最后一节把这条链用起来：先用运行概念把“系统怎么被用”讲清楚，再写规格里的条款。这是第 1 章说的先立枢纽、再填细节在本章的现身：先立问题域，再写条款；顺序反了，条款写得再整齐也是悬着的。

---

## 4.2 开发与管理并列，不是包含

> 这一节先看定义。需求工程由哪两段构成、每一段各做什么、需求从哪来、做出来的是什么、两段在哪里交接。这些立住了，后面的边界与运用才有话可说。

### 4.2.1 需求工程：开发做出来，管理看住

先看它由哪两段构成，再看它凭什么配得上“工程”二字。

> **需求工程（requirements engineering）= 需求开发 + 需求管理**（并列，不是包含）

- **需求开发（requirements development）**：把利益相关方的需要与期望转化为规定需求，经过获取、分析、规格说明、需求校核。它是创造性的：从没有到有。
- **需求管理（requirements management）**：对规定需求进行基线化治理，经过基线、变更控制、追溯、状态与度量。它是控制性的：从有到一直是那一份。

一句话：**开发负责做出来，管理负责让它一直是那一份。**

这不是本书的分法，标准就是这么分的。CMMI 把需求管理（REQM）与需求开发（RD）列为两个并列的过程域；ISO/IEC/IEEE 29148（需求工程）按“开发 + 管理”两条线组织全篇（此处只锚标准与名称，不引条款号）。

> 出处提醒：本章另引八家按名称锚定的来源：CMMI（REQM／RD）、ISO/IEC/IEEE 29148、IEEE 1362、IEEE 830-1998、IEEE 610.12-1990（经第 2 章转引）、MoSCoW、Kano、Boehm 一系的研究；其中 IEEE 830-1998 与 IEEE 610.12-1990 带版本年。本章只锚标准与名称，不引条款号；正文里给出条款号的只有 ISO 9000:2026 与 ISO/IEC/IEEE 15288:2023。

在 ISO/IEC/IEEE 15288:2023 的过程清单里，这条链有三步：业务或任务分析（§6.4.1）定义任务与目的，利益相关方需要与需求定义（§6.4.2）把需要与期望做成利益相关方需求，系统需求定义（§6.4.3）再把它做成系统需求。需求工程做的是后两步，第一步是它的输入。第 3 章 §3.2.4 已经把这个位置划开过。

那为什么配得上“工程”二字？因为这件事有方法（访谈、原型、排序、评审）、有过程（每一步都有输入、输出、角色和准则）、有纪

律（基线、变更控制、追溯），而且可复现，换个人来做，结果不会差太远。

第 2 章讲“工程化的成色”时列的那几条（有结构、有方法、可重复、全覆盖、受控、可验证），需求工程一条不缺：获取讲方法，规格讲结构，规定需求讲可验证，管理讲受控。这套说法的出处是 IEEE 610.12-1990 对软件工程的定义，出处层级第 2 章 §2.2.3 交代过。它和方法论、手艺的分野也在这里：可复现、可检验，换个人也能做。

### 4.2.2 为什么是并列，不是包含

流行一种说法：“广义的需求管理包含需求开发。”这个说法错在把两类性质相反的活动揉成了一团。

| | 需求开发 | 需求管理 |
|---|---|---|
| 性质 | 创造性：收集、分析、建模、写规格 | 控制性：冻结、变更、追溯、跟踪 |
| 回答 | 需求是什么、合理吗 | 需求定了吗、变了吗、谁改的 |
| 失败模式 | 来源没摸清、分析没做透 | 基线乱、变更失控 |

把开发包进管理，会盖住一个关键事实：**需求是开发出来的，不是管出来的**。管理再严，开发没做好，一样出烂需求。章首恒信那场会，就是“只谈管理、不谈开发”的典型。

两种欠账的后果也不一样。开发欠账出在源头：来源没摸清、话没说清，之后再怎么严管，管的也还是那些没说清的东西。管理欠账出在过程：需求原本写清楚了，却被无序变更一点点磨掉。

恒信“改一版崩一版”，如果只去加强管理，那些没说清的需求依然没说清，只是从此被更严格地看住而已。**先把开发的欠账补上，再谈管理的严**：顺序不能反。

**概念卡片 · 需求工程**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 需求开发与需求管理两段并列的总称（CMMI 的两个并列过程域、29148 的两条线）：开发负责做出来，管理负责让它一直是那一份 |
| **它不是什么** | ⑥⑦ | 对立：广义的“需求管理”，把创造性活动包进控制性活动里，会盖住“需求是开发出来的”；辨析：需求工程是两段的总称，需求开发与需求管理在它下面；需求工程不等于需求管理 |
| **它从哪来** | ②③⑨ | 从系统工程的需求侧过程演化来：ISO/IEC/IEEE 15288:2023 §6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义是它的过程锚点；CMMI 与 29148 把它分成并列两段 |
| **它往哪去** | ④⑤⑧ | 它回答“需求从哪来、做出来之后怎么才算一直是那一份”；在需求获取、评审、变更控制时用；误区 ✗ “加强需求管理 = 往流程上加东西”（先问源头清了吗）✗ “需求工程 = 写需求文档”（文档只是载体） |
| **判据** | 两段并列 | 能说清“做出来”和“一直是那一份”分别归谁吗？——需求是开发出来的，不是管出来的 |

### 4.2.3 需求开发：把诉求变成规定需求

需求开发走四个活动：

需求获取 → 需求分析 → 需求规格说明 → 需求校核（评审）
收集需要与期望　　澄清/分解/排序/建模/协商　　写成可验证条文　　对工作产品做评审

| 活动 | 做什么 | 典型输出 |
| --- | --- | --- |
| 获取 | 访谈、调研、竞品分析，收集需要与期望 | 利益相关方登记册、访谈调研记录 |
| 分析 | 澄清、分解、排序、建模、协商冲突 | 需求池、优先级排序表、业务规则清单 |
| 规格说明 | 把分析结果写成可验证条文 | SRS、术语表、追溯矩阵初版 |
| 需求校核（评审） | 对工作产品判断适宜、充分、有效 | 需求评审报告、需求评审·确认记录、批准记录 |

> 术语注：表内“确认”是**利益相关方确认**，指签字批准前那句“是不是用户要的”；它与 ISO 9000:2026 §3.11.14 的确认（validation）不是同一件事——后者要在使用中兑现：对一份需求文档，能做的是评审与验证，**确认要等它被实现、被用起来**（§4.3.3）。

四个活动不是跑一遍就完，它们是一个循环：分析时发现缺信息就回获取，规格时发现问题就回分析。为什么是循环？因为“模糊的诉求”不会一次被看清。访谈时对方说“重要合同要审批”，分析时发现“重要”没定义；回去再问“多重要算重要”，得到“金额 ⩾ 500 万或三年期以上”，才进得了规格。

需求开发的每一步都可能推翻前一步的假设。循环不是低效，是正视“需求天然是渐进的”这个事实。

最后一个活动为什么叫校核（评审），不叫确认？因为需求文档是一份工作产品，还不是能用的东西。一份还没实现的文档，能对它做的只有评审。三把尺子的并排对照在 §4.3.3，这里先把名字钉住。

> 出处注：「工作产品」这个范畴取自 ISO/IEC/IEEE 15288:2023。它用“工作产品”统称过程产出，人工制品是其中一类：§3.6 人工制品条写的是“在项目期间产生并用于捕获和传递信息的工作产品”（原文其后还有一串示例，此处只引定义语），表 B.1 列的则是“一组可能的工作产品，包括人工制品与信息部件”；ISO 9000:2026 的定义里只有评审一条点了“客体”（§3.11.2）；验证说“规定需求”、确认说“针对特定预期用途或应用的需求”，三条都不说工作产品。

所以需求开发的末位活动，准确的名字是**需求校核（评审）**：评审人对着目标，判断这份需求适不适宜、够不够、有没有用。真正的验证与确认，要等到有东西可用了才做。那是第 8 章的活，本章只把它挂在链上。

> 术语注：需求工程标准（ISO/IEC/IEEE 29148:2018）里有一个说法叫 requirements validation，指的是“需求本身对不对”，与 ISO 9000:2026 §3.11.14 的确认（validation）同名不同物：后者判的是已实现客体在特定用途下满不满足。此处只锚标准与版本，不引条款号。

获取与分析：来源与澄清。获取是需求的第一手证据。恒信的合同审批系统，获取的对象是三个部门的人：法务（条款、合规）、财务

（金额、账期、对账）、业务（审批流、时效）。获取做没做到位，判据只有一条：**每条需求有没有来源**，无来源需求不得入需求池。

分析是第一道加工，五件具体的事：

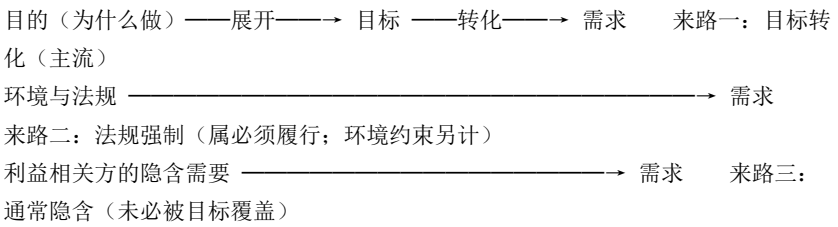
1. 澄清：把“重要合同要审批”问成“金额 ⩾ 500 万或涉及三年期以上的合同须审批”；
2. 分解：把父需求切出子需求。这与第 6 章的分解是同一条规则、两处现身：切点都由消费方的语义范围定，不存在“需求侧”“设计侧”两套分解；
3. 排序：MoSCoW、Kano 决定先做什么、后做什么；第 3 章“与利益相关方一起剪裁需求”在这里落地；
4. 建模：把澄清过的关系画成模型（原型、状态图、业务规则图），让分歧先在图上暴露；
5. 协商冲突：三个部门对“合同状态”的理解不一致，在这里拍平，而不是等集成时炸。

排序值得单独说一句：MoSCoW（必须／应该／可以／不要）和 Kano（基本型／期望型／兴奋型）不是两张玩具表，它们直接决定“需求不够时砍哪条、保哪条”。恒信的“合同数据留档十年”（公司法强制）在 MoSCoW 里是 Must，在 Kano 里是基本型。两套框架指向同一个结论：这条需求不在被砍的清单里。

需求缺陷的成本。常被引用的一组数据：同一个缺陷在需求阶段修复成本是 1，发布后修复是 30 到 70 倍。需求阶段写错一个字，可能意味着设计、开发、测试全部返工。需求工程的每一分投入，都在给下游省十倍百倍的返工钱。这组数据出自 Boehm 一系的研究（附录 E）。

### 4.2.4 需求的来路

第 3 章 §3.2.1 把需求分成三种形态：明示的、通常隐含的、必须履行的。三种形态不是分类游戏，它们各自对应一条来路。问“需求从哪来”，答案就是这三条。




来路一：从目标转化。目的展开成目标、目标转化成需求，这是主流来路。第 3 章 §3.2.3 已经把目的、目标、需求三个词摆开，这里只钉一处用词：**目标到需求是“转化”，不是“分解”**。

分解是同类拆分，语义守恒，整体等于部分之和；目标→子目标、需求→子需求才是分解。目标回答“要实现的结果”，需求回答“做成什么样”，两者不同类。

这条链是跨类导出：一条目标往往导出一组需求，导出的每一条都要判断；阈值是权衡出来的，不是从目标里分出来的。ISO 9000:2026 §3.3.11 说设计开发是“将某一客体的需求转换为该客体更详细需求的一组过程”，那是同类细化；目标到需求比这更远一步，本书按跨类导出处理，理由是语义不守恒。

这不是本书的私分法：ISO/IEC/IEEE 15288:2023 把任务与目的、利益相关方需要与需求、系统需求分放在三个过程里，业务或任务分析（§6.4.1）、利益相关方需要与需求定义（§6.4.2）、系统需求定义（§6.4.3）。三段各是一个过程，不是同一条链上的粗细三档；它们之间是跨类导出，不是同类拆分。

往下走，需求到子需求是分解，需求到方案是求值（第 6 章）；分解与求值都以语义可守恒为前提。目标到需求这两样都不占，所以那一步要判断，不能照着分。

来路二：从环境与法规。这一支要分成两条走：法规强制那一支属“必须履行”，环境那一支另立一类。恒信“合同数据留档十年”是公司法强制：组织就算没立过“合规目标”，这条需求照样成立，它不经过组织目标这道转化。气候、地域、文化等环境适应性要求，归第 5 章承接。

来路三：从利益相关方的隐含需要。案例进展②里财务那句“25 号之后签的合同能不能算下个月”，就是来路三：行业惯例默认，没人写进文档，但漏了就要出事。

三条来路合起来说明一件事：**需求比目标转化的结果更宽**。目标的完备性靠评审的充分性把关（目标有没有被需求覆盖全，第 8 章）；另外两条来路不经过目标，只能靠需求工程自己补齐。利益相关方的需要要收集全，法规要列成清单（第 5 章展开）。

> 术语注：“需求三形态对应三条来路”是本书为方便使用做的梳理，不是标准原文；标准只并列给出需求的三形态，不给来路。

### 4.2.5 规定需求：SRS（需求规格说明书）里有什么

规格说明把分析结果写成可验证的条文，核心交付物是需求规格说明书（SRS）。一份 SRS 通常有六章：

引言（目的 • 范围 • 读者）→ 总体描述（产品视角 • 环境 • 约束）
→ 功能需求（逐条编号 REQ-xxx）→ 非功能需求（性能 • 安全 • 可用性）
→ 外部接口（用户 • 硬件 • 软件 • 通信）→ 验证方法（每条需求的接收准则）

各章各有职责：引言把读者领进来；总体描述给出产品视角；功能需求逐条编号，无歧义、可验证；非功能需求写性能、安全、可用

性；外部接口定义与外部世界的边界；验证方法给每条需求配接收准则。最后一章最容易被砍，但它正是“规定需求可验证”的兑现处：没有接收准则，需求就退化成愿望。

> 承接第 3 章 §3.3.3：那一节说过“特性还没开始规定，是需求开发；特性开始被规定，才是设计开发”。那份分界文档就是 SRS。写完之后，才谈得上把需求落到客体的固有特性上。

这里的枢纽概念是规定需求。

> **规定需求（specified requirement）**：被明示的需求（ISO 9000:2026 §3.5.1 注 2）。

正本的原话是 A specified requirement is one that is stated, e.g. in documented information（库内中译作“规定的要求是明示的要求，例如写在成文信息中的要求”；本书把 requirement 一律作“需求”，下同）。要紧的是那个 e.g.：写在成文信息里是最常见的一种明示方式，不是“规定”的条件。说出来了就是规定需求，口头说的也算，它已经进了这个集合。

那口头说的和成文的差在哪？差在能不能拿来当依据。规定需求要当验证或合规的参照，还得过两道关：先**可判定**，有编号、有对象与适用范围、有断言、有判定方法。

第二道关是**可采**，四条都要过：**相关性**，能回溯到目的；**可检验性**，自带判定方法与准则；**时效性**，属于已冻结的那一版；**同一性**，有唯一标识。口头那句“页面要快一点”是规定需求，但不可采：四条里它只对得上相关性，没有判定方法、不可复现、也没有版本可查。

出处注：“可判定”四件套与“可采”四条件为本书工作名（术语表 T-186）。

所以“说了不等于规定”这句话，准确的说法是：说出来了就是规定需求，但**说出来了不等于能当依据**。本书对这句话的改法只换理由，不换结论。口头需求照样不能当验证的靶子，不是因为它“不算规定需求”，而是因为它不可采。

这也顺带解释了需求工作的两道动作：需求评审要把口头内容写成条目，配置管理要把条目冻进基线。成文与基线不是“升格为规定”的仪式，是取得可采性的两步：写下来才有判定方法与唯一标识，冻进基线才谈得上时效。

**概念卡片 · 规定需求**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 被明示的需求（ISO 9000:2026 §3.5.1 注 2）：验证的参照；成文与基线是它的可采条件，不是它的构成条件 |
| **它不是什么** | ⑥⑦ | 对立：不可采的明示，口头说了但没判定方法、不可复现、无版本，够不上“依据”；辨析：明示 = 规定（一个等号），成文 ⊂ 明示（最常用的一种方式） |
| **它从** | ②③⑨ | ISO 9000:2026 §3.5.1 把需求按“明示的、通常隐含的、必须履行的”三形态并列，其中“明示”这一支被单列，因为验证需要一个能对账的稳定参照；speci- |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **哪来** | | fied ← 拉丁 specificare（明确指明）；**“可验证”是从验证定义（§3.11.12）推出来的，不是注 2 的原话** |
| **它往哪去** | ④⑤⑧ | 它回答“拿什么当验证的参照”；在需求规格、评审、测试设计、变更控制时用；误区 ✗ “没写成文档就不算规定需求”（不算的是可采，不是资格）✗ “口头说了就能当依据” |
| **判据** | 可采 | 一条需求能提供客观证据吗（测试只是其一）？能，就可采、可当依据；只能“感觉”，就还得写下来、定下来 |

### 4.2.6 需求管理：看住基线，管住变更

需求开发把需求做出来了，之后归需求管理看着。它保证的只有一件事：谁动了、动在哪、为什么动，都说得清。

标准侧给过一条过程锚：利益相关方需求要记录下来，记成适合整个生存周期做需求管理的形式。这些记录建立需求基线，保留需要的变化及其来源，并成为向业务或任务分析的决策、利益相关方需要、系统需求和后续系统元素追溯的基础（ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 24）。记录、基线、变更、追溯四件事，在那一条注里是一次到位的。

四个活动：

基线管理 → 变更控制 → 可追溯性 → 状态与度量

| 活动 | 解决什么问题 | 典型产出 |
| --- | --- | --- |
| 基线管理 | 需求“定了吗” | 需求基线、基线登记表 |

| 活动 | 解决什么问题 | 典型产出 |
| --- | --- | --- |
| 变更控制 | 需求“变了吗、谁改的” | 变更请求单 CR、CCB 评审记录 |
| 可追溯性 | 需求“从哪来、到哪去” | 追溯矩阵 RTM |
| 状态与度量 | 需求“整体什么状态” | 需求状态报告、度量报告 |

四者不是均分的：**变更控制是核心战场**（大多数团队的需求管理问题都出在这里）；基线是它的前提，追溯让它查得清影响面，状态与度量让它看得见整体情况。合起来回答的是一串问题：定了吗、变了吗、影响谁、整体什么状态。

> 术语注：§4.2.1 那句“对规定需求进行基线化治理”里的“治理”，取“受控地管住”这一层意思，与第 16 章作为专名的治理（谁掌舵、谁划桨）不是同一个概念。那一个讲的是跨层级的校正机制，这一个讲的是把已批准的需求看住。

变更怎么走完一圈。需求管理最容易失控的地方是变更。一条标准的路径：

变更请求单 CR → 影响分析（经追溯矩阵反查）→ CCB 评审（批准/拒绝/暂缓）→ 实施变更（同步 SRS • RTM）→ 再验证（符合新的规定需求）→ 关闭（更新基线 • 登记日志）

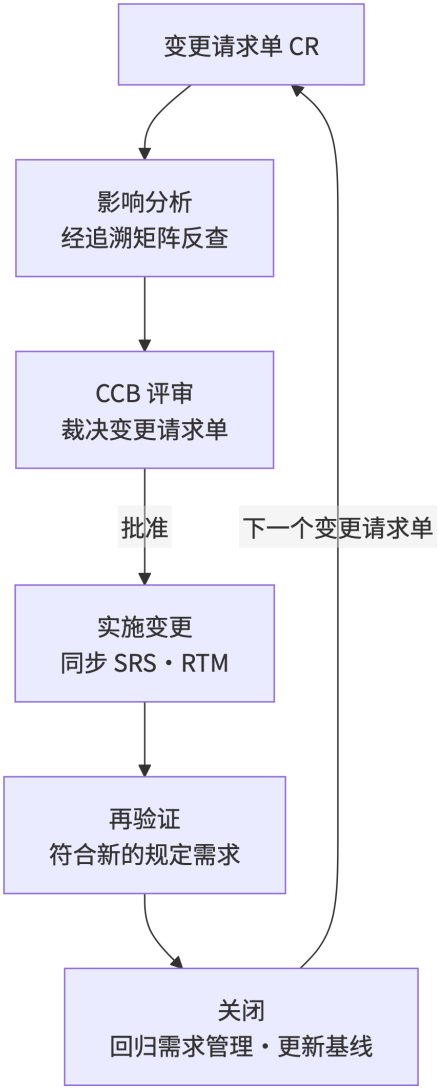



图 4-1

每个节点都是一个必须可判定的关卡：CR 有没有立案、影响分析有没有做、CCB 有没有结论、再验证有没有通过。任何一个答不上

来，变更就在流程外流窜。一圈走完，回到下一个变更请求单立案：变更控制是闭环，不是一条直线。

三条铁律：

> **无 CR 不得变更；不分析不得上会；验证通过方可关闭。**
>
> 术语注：这三条是本书对变更控制纪律的操作化凝练（源出 CMMI 配置管理与需求工程标准的变更管理需求，非标准原文照录），可当场检验。

变更还要分级：微小（需求负责人批）、一般（项目经理批）、重大（CCB 全体评审）。时限上有组织级约定，如 SRS 完成后 5 个工作日内评审、影响分析 3 个工作日、CCB 5 个工作日内决策、紧急变更 24 小时内追认；度量上有预警阈值：变更率 ≥ 15%、稳定性 ≤ 80%、追溯覆盖率 < 100%、变更平均周期 ≥ 10 天；触及其中任一条线，就启动根因分析。

这些数字是组织级示例，要害在于：变更控制的每一步都要可判定，“以什么标准”必须写死，否则流程就是摆设。

一个走完整条路径的例子：恒信法务提出“电子签章的合同也走审批流”。这是一条 CR。影响分析经 RTM 反查，发现影响审批状态机、签章模块、归档模块共 3 处；CCB 评审通过；实施后同步 SRS 与 RTM；再验证“电子签章合同审批通过后进入归档”；关闭，更新基线。整个过程留下 6 份记录，每个节点一份。任何一环断了，这条变更就成了查无实据的暗改。

基线是分界线。线左是需求开发的产物（还在流动），线右归需求管理（一切改动走变更控制）。

> **基线**：经批准、确立了产品或服务某一时点特性、作为生存周期参照的配置信息（ISO 9000:2026 §3.10.12；本书行文简作“被批准冻结的参照”）。
>
> 术语注：此定义取自配置管理的通用概念，第 15 章展开全貌。

第 15 章会把这个概念讲透。这里只需要记住它在需求链上的作用：基线是变更的出发原点。从基线出发，一切变更都可描述；离开基线，一切变化都不可言说。

基线和追溯是一体两面：基线给“变”一个参照原点，追溯给“变”一份影响清单。没有基线，追溯无从谈起；没有追溯，基线一建立就失效。恒信先建基线、再建 RTM，顺序对了。

判据问句：这一版需求被批准了吗？被冻结了吗？生存周期拿它当参照吗？三问全过，才是基线。

### 4.2.7 开发与管理在哪里交接

接口只有一处：批准 → 基线。需求开发的最后一步是需求批准记录（评审通过、利益相关方确认），批准之后建立需求基线，进入需求管理的领地。

> 这道分界是整条需求工程唯一的接口——批准前，需求还可以流动，归开发负责；批准后，一切改动走变更请求，归管理负责。这道分界不咬合，两段就各管一段，需求会在中间断掉。

需求管理计划（RMP）。程序文件（组织级“怎么管需求”的通用规定）与 RMP（项目级执行计划）是两个层级：每个项目启动时，按程序文件编制 RMP。RMP 通常有六块，每块回答一个问题：

| 模块 | 回答的问题 |
| --- | --- |
| 目的范围 | 这份计划管什么 |
| 角色职责 | 谁是 CCB、谁批微小变更 |
| 需求组织规则 | 需求怎么编号、怎么入池 |
| 基线策略 | 什么时候建基线、什么算重大变更 |
| 变更控制流程 | CR 怎么走、时限多久 |
| 追溯与度量安排 | RTM 怎么建、什么指标预警 |

RMP 是程序文件的项目版：把“组织怎么管需求”翻译成“这个项目怎么管需求”。

需求管理的正式产出，以台账和凭证为准。审计、扯皮、追责全靠它们：

| 活动 | 文档 |
| --- | --- |
| 计划 | 需求管理计划 RMP |
| 基线 | 需求基线 · 基线登记表 |
| 变更 | 变更请求单 CR · 影响分析报告 · CCB 评审记录 · 变更验证记录 |
| 追溯 | 覆盖分析记录 · 追溯矩阵 RTM |
| 状态与度量 | 需求状态报告 · 变更日志 · 需求度量报告 |

其中 CR、CCB 记录、变更日志最要紧，它们是“改了没有、谁批的、为什么改”的唯一凭证。**变更日志只追加不修改**，审计时调取。需求管理最忌口头的变更：落到文档层面，就是没有凭证的变更。

台账与凭证之外的东西见 §4.4.5：不可省的骨架文档、整套编号；这里只说交接处要交出去的那几件。

## 案例进展①｜顾言的“需求管理”升级会议

会后，顾言把“加强需求管理”这条指令翻成了两件事。

第一件，他调了恒信需求库里近三个月的变更记录。四十七条变更里，二十九条不是需求变了，而是当初就没说清：合同评审什么时候启动，写的是“重要合同要审批”，可“重要”从没定义过；财务要的“对账报表”从没说清统计范围。

第二件，他列了两张单子：一张是开发欠账（没调研、没分析、没写可验证的规格），一张是管理欠账（没基线、没变更流程、没追溯）。

他把两张单子并排摊开：

> “加强需求管理这件事，一半是补需求开发的课。管理能让一份需求一直是那一份，可它没办法让没说清的需求变清楚——说不清的，只有开发能补。”

系统架构师罗建盯着那张开发欠账的单子看了半天。

> “这四个星期我们加了三道流程、两张模板，变更数一条没降。现在看，是把两件事当成一件事了：管理管的是'定了的需求不变形'，开发管的是'需求本身立得住'。”

罗建把话往下说：

> “流程没错，是顺序反了。按 ISO 9000:2026 §3.3.11，设计开发是把需求转换成更详细的需求——那些连'重要'都没定义的条目，压根过不了第 3 章 §3.3.2 那条可转化的线。拿没过线的条目去开发，开发得越彻底，返工越彻底。”

两周后，顾言补了需求调研和需求池，建了第一版基线。罗建说“需求稳了一点”。不是需求变少了，是欠账开始还了。

这两张单子，正是本章要分的两笔账：§4.3 磨它们的边界，§4.4 看怎么用起来。

## 案例进展②｜合同的“隐含需求”

做合同审批系统需求调研时，顾言访谈法务、财务、业务各三场。每次最后他都问一句“还有别的吗?”，得到的都是“没有了”。

直到财务随口说了一句：“对了，我们月底要对账，合同如果 25 号之后才签，能不能算到下个月?”

顾言记下了。这句话暴露了一条隐含需求：合同审批的统计归属月份规则。没人把它写进任何需求文档。它“通常隐含”，是行业惯例默认。但如果漏了，月底对账会错，财务会觉得“这系统根本不能用”。

需求开发里，**隐含需求不会自己开口**。它需要主动去挖：访谈里的随口一句、业务流程里的惯例、前代产品的教训。挖出来的隐含需求要显性化、写进规格、变成规定需求。否则验证够不着它（验证对照规定需求），只有确认拦得住它（确认对照特定预期用途或应用的需求）。那是第 8 章的核心，这里先埋种子。

需求开发到底开发什么?不是把用户的话抄下来，是把不开口的、没成文的、说不清的，都挖出来、写下来、定下来。

---

## 4.3 最容易混的边界，分开看

> 这一节把最容易混的几处分开。概念靠正例活，靠反例定型：一处混在“说了”与“能当依据”之间，一处混在开发与管理之间，一处混在评审、验证与确认之间，一处混在留档与受控之间。

### 4.3.1 明示与可采：说出来了，就能当依据吗

“审批要严谨”“报表要一眼看懂”“导出要方便”，这些话客户说了，领导也说了。它们算需求吗？

算。按 ISO 9000:2026 §3.5.1 注 2，说出来了就是明示的需求，明示即规定。它们已经是规定需求了。

但它们还当不了依据。

验证要的是一个能对账的靶子，而一句口头的话对不了账：“快一点”是几秒？“好用”怎么证明？所以同一句需求，要从“说了”走到“能当依据”，中间还得满足可采的条件：

| | 口头明示 | 成文且已定版 |
|---|---|---|
| 是规定需求吗 | 是（明示即规定） | 是 |
| 能当验证的参照吗 | 不能：四条件里只对得上相关性，没有判定方法、不可复现、没有版本 | 能：四条都占，有编号、有判定方法、在已冻结的那一版上 |
| 传得下去吗 | 转述容易走样 | 白纸黑字不漂移 |
| 追得回来吗 | 没有版本可查 | 有版本可查 |

这条边界的判据问句：一条需求能提供客观证据吗（测试只是其一）？能，就可采、能当依据；只能“感觉”，就还得写下来、定下来。

恒信那句“重要合同要审批”就是标本。它是明示的需求，也是规定需求，可它不可采：“重要”有两种以上的理解，谁也没法拿它去

验证。把它规定成“金额 ⩾ 500 万或三年期以上”，它才第一次能当依据用。

> 承接第 3 章 §3.3.2：那一章从设计开发那一侧看同一件事，给的是“可转化”那条线（完整、无歧义、不冲突、隐含需求已识别），管的是能不能进厂加工。这一节从验证那一侧看，管的是能不能当对照的靶子。两条线不是一回事：可转化管进不进得去，可采管作不作数。

### 4.3.2 开发与管理：需求乱，是谁的账

案例进展①里顾言翻了三个月的变更记录，发现一多半“变更”根本不是需求变了，而是当初就没说清楚。

这条边界钉死的是一条诊断判据：当需求“总在变”，先别急着上管理手段，先分清它是哪个病。

| 病 | 症状 | 药 |
| --- | --- | --- |
| 开发欠账 | 没调研、没分析、没写可验证的规格 | 补开发：调研、分析、写规格 |
| 管理欠账 | 没基线、没变更流程、没追溯，于是变更失控、查无实据 | 补管理：基线、变更流程、追溯 |

两种欠账的症状不一样，问法也不一样：听到“这条需求当初是谁提的、为什么提”答不上来，多半是开发欠账，来源没留下；听到“这一版是谁批的、上一版长什么样”答不上来，多半是管理欠账，痕迹没留下。

判据问句：最近这次需求混乱，是“当初没说清”（开发欠账）还是“定了之后没人管”（管理欠账）？答错了病，药就开错了。章首那位总监开的就是这个药。

### 4.3.3 需求文档：能评审，能验证，做不了确认

需求评审开完，主持人常问一句：“验证了吗？确认了吗？”这句话问错了。

按 ISO 9000:2026，这三件事各有各的对照，对象也不一样：

| | 评审 review | 验证 verification | 确认 validation |
|---|---|---|---|
| 定义 | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（§3.11.2） | 通过提供客观证据，证实规定需求已得到满足（§3.11.12） | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（§3.11.14） |
| 对照 | 为客体确立的目标 | 规定需求 | 特定预期用途或应用的需求 |
| 对象 | 工作产品（文档、模型）——**能评审，也能验证** | 实现出来的客体，**或人工制品**（15288:2023 §6.4.9） | 已可使用的客体——**确认的现场在使用态** |

> 术语注：确认一条的正本中文用字是“针对特定预期用途或应用的要求”（§3.11.14），本书一律照正本引作“针对”。

需求文档是工作产品。**对它做的是评审与验证**：评审判断这份需求适不适宜、够不够、有没有用；验证认定它满足“这份文档自身应满足的规定需求”（如条目齐备、可判定），证据形态就是评审文件（15288:2023 §6.4.9 的验证对象含人工制品）。**确认做不了**——确认要在使用中兑现，那是第 8 章的三个看门人。

这一处为什么非分清不可？因为它决定了一句话能不能被承诺。把需求评审叫成“需求确认”，等于给一份还没实现的东西签了确认的字：评审会上说“确认通过”，验收时才发现用户要的不是这个。那句

“确认通过”就成了将来扯皮的证据。第 8 章会把这条线完整展开，这里先钉住一句：**需求文档做不了确认**——确认要在使用中兑现；能对文档做的是评审与验证。

> 术语注：需求工程标准里的 requirements validation（“需求本身对不对”）与这里的确认同名不同物，见 §4.2.3 的术语注。

判据问句：你现在要判的这份东西，是做出来的客体，还是纸面上的工作产品？是工作产品，**能评审，也能验证**（验证它满足文档自身应满足的规定需求）；要说“确认通过”，得先把东西做出来，并且用起来。

### 4.3.4 留档与受控：改名算不算版本管理

需求管理里最像版本管理的动作，是给文件改名，也最容易被当小事放过去。

**改名、留档，不等于版本管理**。版本管理要有三样：**基线**（批准的参照）、变更控制（怎么变、谁批）、追溯（从哪变起、波及谁）。

> 术语注：这里说的“版本管理”是**配置管理管到文件层面**的日常说法（第 15 章展开全貌）；**上面的三样，正是“留档”与“受控”的分界**。

“另存为 v2”只做到了留档，留下了每一版的快照，却没回答三个问题：哪一版是权威基线？这次改为什么、谁批的？这条需求从哪来、影响谁？

这是全书“另存为 v2”的首次出场。第 15 章会用合同版本 final_2.zip 再演一遍，同一个病，换了文件后缀。

判据问句：这次“版本管理”，能回答“哪一版是基线、这次为什么改、从什么变到什么”吗？答不上来，就只是改名。

## 案例进展③｜“另存为 v2”的合同需求

恒信的合同审批需求改了六版。v1 是没后缀的原稿，v2 以后靠文件名管版本：“合同需求 最终版 v3.doc”“合同需求 真最终版 v4.doc”。

后果是：需求池里同一份合同的需求散在六个文件里，追溯矩阵没法建，验证没有参照。苏婷是开发，她接口写到一半，对着六个文件名发愁：

> “我照着哪个版本写？接口这一半按 v4 的字段名写的，可 v6 里'合同状态'多出一个'已签章待归档'——六个文件三种说法。哪一版是批过的需求规格说明书，我看不出来，数据库的表结构就定不下来。”

王默是运维，他把发布记录也摊到桌上：

> “文件名记不下谁批的、为什么改。我们发布认的是可复现——哪个包配哪套环境参数，能复现出当时的运行形态，才算数。你们这六个文件，连哪一版是基线都对不上；没有变更请求单，也没有追溯矩阵，等于每次改动都没发生过。”

罗建终于承认：“我们不是需求变太多，是这几份需求文件从来没立过基线。”

顾言把这几份需求文件收进了基线管理，之后每个变更都走 CR 流程。六周后，变更失控率降了七成。不是需求变少了，是每次变更都留下了痕迹。

---

## 4.4 需求工程怎么做

> 这一节把这条链用起来：先用一份运行概念把“系统怎么被用”讲清楚（问题域），再写规格里的条款（解空间），最后看一条好需求的判据和不可省的文档。

### 4.4.1 运行概念（ConOps）：系统将被怎么用

SRS 回答“系统必须做什么”，可在这之前还有一个更根本的问题：系统怎么被用？法务怎么审合同、财务怎么对账、业务怎么发起审批；这些“怎么被用”的叙事，是 SRS 的上下文。

没有它，SRS 的每一条都像悬空的条款：你知道要做什么，却不知道为什么、为谁、在什么场景下。

运行概念（Concept of Operations，ConOps）就是回答“系统怎么被用”的那份文档（IEEE 1362 定义其结构）。它是需求工程里唯一一份叙事性的正式输出，别的文档都是条目和条款，只有它讲一段过程。标准侧的位置也是明的：ISO/IEC/IEEE 15288:2023 把“开发运行概念及其他生存周期概念”列为利益相关方需要与需求定义（§6.4.2）的一项活动；§6.4.2.3 的注 5 又给出展开的去处——运行概念的更多信息见 ISO/IEC/IEEE 29148:2018 第 5、6 章，系统运行概念的带注释大纲见该标准附录 A。

### 4.4.2 运行概念的位置：分析之后，规格之前

ConOps 不在流程最前，也不在最后：

业务目标 → 需求获取/分析 → 运行概念 ConOps（系统怎么被用）→ SRS（系统必须做什么）→ 设计 • 测试

链头那段“业务目标／系统目的”，由业务方定义并签字确认，对应后记里系统架构师那份《系统目的或任务理解确认单》（第 18 章）。目的、目标、需求三个词怎么分，第 3 章 §3.2.3 已经立过，这里只补分工：目的由业务方定；目标由需求侧帮它立成可衡量的结果；架构师在架构语境里把它当判据用，但立得对不对仍要业务方点头。

需求从三条来路上来（§4.2.4），三件东西各有一种落纸形态：目的落成确认单，目标落成操作目标清单，需求落成规格里的条目；评审的通过标准，正是会前从这份清单里选出的可判定目标（第 8 章）。

为什么 ConOps 必须在“分析之后、规格之前”？因为它的完整定稿（运行场景、异常、应急、角色）必须建立在已经摸清需要的基础上：先获取分析，再综合成运行概念，最后翻译成 SRS 条文。

跳过 ConOps 直接写 SRS，等于从问题域一步跨进解空间，中间那段没了。

ConOps 还有第二重身份，比“上下文”更要紧：**它是确认判据的载体**。确认对照的是“针对特定预期用途或应用的需求”，而这份需求的正文，恰恰写在运行场景里，并且主体是隐含需求那一支。隐含需求不在派生链上（派生链承载的是明示的那些），验证够不着它，只有确认拦得住它；没有 ConOps，确认就没有靶子。第 8 章会把这条线接上。

> ConOps 讲一段过程，SRS 讲一条条条款——先讲清楚系统怎么被用，才谈得上系统必须做什么。

这正是本章的方法底座时刻：**先立问题域，再写条款**。第 1 章叫它先立枢纽、再填细节。第一节立的“开发与管理并列”是第一个枢纽，ConOps 是第二个。先扎进条款、不立骨架，条款越多越悬空。

### 4.4.3 一份运行概念里有什么

IEEE 1362 给 ConOps 的结构列了六部分：

| 部分 | 内容 |
| --- | --- |
| 引言 | 目的、范围、读者 |
| 系统概述 | 系统的使命与地位 |
| 运行环境 | 部署环境、外部接口 |
| 运行场景 | 正常·异常·应急（核心） |
| 利益相关方与角色 | 谁用什么、谁管什么 |
| 运行支持 | 部署、培训、维护、升级 |

其中**运行场景是核心**，而且要写三种：正常（一份合同走完审批全流程）、异常（法务驳回、财务校验不过、审批人离职）、应急（系统宕机时的线下替代流程）。

写 ConOps 有个实用判据：让一个不熟悉这套系统的人读完，能不能画出审批流程图？画得出来，ConOps 合格；画不出来，故事没讲清楚。

它的失效边界也要说清：ConOps 服务的是“怎么被用”的复杂度。运行方式一眼看透的小系统（单机、单场景）可以省掉；场景越复杂、角色越多，越不能省。

**概念卡片 · 运行概念 ConOps**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 从用户与运营视角描述“系统将如何被使用”的文档（IEEE 1362 定义其结构）：需求工程里唯一一份叙事性的正式输出 |
| **它不是什么** | ⑥⑦ | 对立：SRS，ConOps 讲一段过程，SRS 讲一条条条款；辨析：不是运营手册（不含运维操作细节），不是需求清单（是运行叙事），不是系统设计（不落进解空间） |
| **它从哪来** | ②③⑨ | 系统工程早期发现：只写“系统必须做什么”，利益相关方对“系统将被怎么用”各想各的，于是把运行方式单独立成一份文档；Concept of Operations → IEEE 1362 定其结构 → 需求工程标准收编，缩写 ConOps |
| **它往哪去** | ④⑤⑧ | 它回答“SRS 的上下文是什么”，同时是确认判据的载体（隐含需求的正文写在运行场景里）；用在分析之后、规格之前；失效边界：运行方式一眼看透的小系统可省 |
| **判据** | 画得出流程 | 让一个不熟悉系统的人读完，能画出流程图吗？画得出来就合格，画不出来就是故事没讲清楚 |

### 4.4.4 一条好需求的判据

一条需求工程意义上合格的需求，须同时满足六条：

> 完整（该有的都有，含隐含需求）· 一致（互不打架）· 无歧义（只能有一种理解）· 可验证（能提供客观证据）· 可追溯（知道从哪来、到哪去）· 有优先级（知道先做哪个）

> 术语注：这六条不是任何标准的固定清单，是**本书对“SRS 写得好不好”的判据**。邻近的规范表述在 ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 26：那一条对利益相关方需求既分析单条需求的特性（必要的、实现独立的、无歧义的、完整的、单一的、可实现的、可验证的、符合的），也分析需求集的特性：完整的、一致的、可行的（或可负担的）、有界的；本节这六条，是本书按 SRS 编写与评审的用法从这些特性里取的。

六条不是并列的形容词，它们互相支撑：无歧义是可验证的前提，可验证是规定需求当依据的条件，可追溯是变更影响分析的依据，有优先级是“与利益相关方一起剪裁需求”（第 3 章）的操作基础。

六条里最常被牺牲的是可追溯，因为它不立刻显形，直到需求变更时才发现“这条需求当初为什么加、影响谁”全答不上来。

落地表现：SRS 里禁止“快速”“大约”“友好”这类不可验证的表述，这正是无歧义与可验证的检查项。恒信那句“重要合同要审批”为什么烂？它同时违反两条：无歧义（“重要”有两种以上理解）和可验证（提供不出客观证据）。

六条之上还有一套全集，分两层立。整体层管“这份规格能不能当设计输入”，第 3 章 §3.3.2 的可转化四条前提；单条层管“这一条写得好不好”，本节这六条。需求工程标准另给“单条好需求”列了一份更长的特性清单（必要性、恰当、正确、符合、单一、可实现等），本书只按标准与版本引用，不逐条照抄条款号，也不写成“标准说……”。

这些判据是怎么来的？不是标准组织凭空发明的，是失败模式的反面。工程界用几十年数据统计需求缺陷：约半数的软件错误要到后期才被发现，而且多数发生在写代码之前（Boehm 一系的研究，附录 E）。于是把“歧义、缺失、冲突、不可验证、不可追溯”这些缺陷类型逐条写成它们的对偶：“不许歧义”就成了无歧义。

标准制定走的是同一条路：收集实践 → 提炼失败模式 → 写成正向条款 → 投票评审 → 修订。判据不是被“想”出来的，是被“反推”出来的。

### 4.4.5 需求文档体系

需求工程的文档可以有十几份，按“素材 → 分析 → 正式交付 → 校核”四类组织：

| 类 | 文档 | 用途 |
| --- | --- | --- |
| 素材类 | 利益相关方登记册 · 访谈调研记录 · 竞品分析 · 用户故事 | 需要与期望的第一手证据 |
| 分析类 | 需求池 · 优先级排序表 · 业务规则清单 · 原型 · 可行性分析 | 需求怎么定 |
| 正式交付 | SRS · 术语表 · 追溯矩阵 RTM | 需求是什么 |
| 校核交接 | 需求评审报告 · 需求评审·确认记录 · 需求批准记录 | 对不对、交出去 |

素材类文档即使不写成正式文档，也要留原始记录，它们是追溯链的最底层。

骨架文档不可省。上面这些文档里，有五份不能省，任何规模的项目都一样：

SRS（需求规格说明书）• 需求池 • RTM（追溯矩阵）• 评审报告 • 基线

- SRS：规定需求的载体；
- 需求池：需求统一入口，无来源需求不得入池；
- RTM：追溯矩阵，变更影响分析的依据、测试覆盖的对照、合规审计的证据；
- 评审报告：校核的凭证；
- **基线**：变更的参照。

RTM 的终极检验：任取一条需求，能不能双向走通，往上游知道它从哪来，往下游知道它影响谁？走不通，RTM 就是一张漂亮的表格，不是追溯。

这套文档可以统一编号：开发域 QR-01~12（利益相关方登记册、访谈记录、需求池、优先级、业务规则、ConOps、SRS、术语表、RTM、评审报告、需求评审·确认记录、批准记录）、支撑文档 QRS-01~04（竞品分析、用户故事、原型模型、可行性分析）、管理域 QR-13~23（RMP、基线、基线登记表、CR、影响分析、CCB 记录、变更验证记录、覆盖分析、状态报告、变更日志、度量报告）。

编号让“哪份文档属于哪个活动”一目了然。**编号不是目的，可引用才是**：“这条需求在哪个文档、哪一条”要能指到具体位置，否则追溯就是空谈。

文档体系是不是太重了？判据很简单：没有文档的环节，就是不可追溯的环节。裁掉文档不是省事，是把追溯链砍断；追溯链断了，需求变更时你就只能靠记忆和运气。

用恒信走一遍 SRS 六章：引言（这份 SRS 覆盖合同审批系统 1.0 的功能与非功能需求）；总体描述（系统替代线下纸质审批，部署于内网，用户为法务、财务、业务三方）；功能需求 REQ-001~REQ-187（发起、初审、校验、终审、签章、归档）。

后三章是：非功能需求（月末高峰 500 并发、合同数据留档十年、操作日志不可篡改）；外部接口（与 OA、电子签章平台、财务系统的接口契约）；验证方法（每条需求的接收准则，如“审批通过后 5 分钟内通知利益相关方”）。连同非功能与接口，全份约三百条。六章写完，一个不熟悉项目的人能独立判断这份规格全不全。

恒信补了 RTM 之后，一次“合同金额校验规则”变更，五分钟内定位了受影响的 17 条需求、3 个模块；没有 RTM，这要翻遍六个版本文件找两天。

## 案例进展④｜恒信没人写的运行概念

恒信第一版 SRS 写得飞快：三个月，三百条。评审会上，法务负责人翻了两页就问：

> “你们写'合同审批通过后自动通知利益相关方'——哪个利益相关方？审批链上有几个人？通知不到会怎样？这三问答不上来，这一条就不是能验收的需求，是一句表态。”

没人答得上。因为没人写过 ConOps：“法务、财务、业务三方怎么用这套系统审批一份合同”这个叙事从没被写进过任何文档。三百条悬在半空，每条都缺上下文。

产品开发经理曹峰翻了翻那三百条：

> “条款一条条都对，可凑在一起，说不清一份合同到底是怎么被用起来的。评审到这儿只能一条条过——我们要的不是三百条各自成立的条目，是一份看完能明白这系统怎么运转的运行概念。”

顾言花三周补 ConOps：把一份 200 万的采购合同从发起、初审、校验、终审、签章到归档，写成故事，逐环节标注角色、异常与应急。

业务方的郑亮陪着顾言一段段对审批链。对到一半他提醒：

> “审批人要是离职了，谁接得上？流程图上这一格是空的。运行场景里'异常'那一栏不写，上线第一个月就会有人拿着纸质单子来找我们——这一栏不是补充材料，它就是需求。”

写到“运行支持”，顾言去问运维王默：

> “系统宕机了，审批走什么替代流程？数据怎么备份、监控什么、谁负责升级？这些都得写进运行概念——运行支持那一栏不落纸，上线那天我们就是临时摸索。”

写完，三百条 SRS 里有一半需要改写：先写条款、后补故事，顺序反了。

规格的问题，往往要到运行概念里才看得出来。

镜照一眼冰链：M3 的 ConOps 是“疫苗从出库到接种全程温度不断链”。运行场景里写着“运输车在高速上抛锚 2 小时”这种应急，直接决定了“断电保温时长”这条硬需求。ConOps 写不出来的场景，就是将来要出事故的场景。

这份运行概念是系统工程师小张执笔的。她在每节末尾留一行“谁认”：签了字的那一段才算数。

冰链的另一面在需求管理：产品经理林涛在评审会上随口一句“精度差不多就行”，没有 CR、没有记录，一句口头变更把验证的靶子换掉。一个缺的是把系统怎么被用写下来，一个缺的是把改动管起来。

### 4.4.6 认知反转

回到章首那场会。IT 总监说“我们要加强需求管理”。现在我们知道，他说错了一半。

需求工程是需求开发与需求管理两段并列，不是包含。恒信的需求乱，一大半是需求开发的欠账：没调研（来源不清）、没分析（“重要合同”没澄清）、没写可验证的规格（三百条悬空条款）。这些欠账，管理补不了：管理能让一份需求一直是那一份，不能让一份没说清的需求变清楚。

顾言补的三样东西，各自补在了正确的位置上：ConOps 补上问题域（先讲清怎么被用，再写条款），需求基线补上变更的参照原点（改名留档不等于版本管理），CR 流程补上变更的痕迹（无 CR 不得变更）。

本章的反转是：“需求管理”这四个字被用得太顺口，以至于人们忘了它不包含“需求开发”。需求是开发出来的，不是管出来的。

再深一层：恒信以为“加强需求管理”是往流程上加东西，结果一半是补开发的课。这提示一条普遍的次序。当一个团队说“要管起来”的时候，先问一句：源头清了吗？管理的价值建立在开发的质量之上，这是需求工程里最容易被倒置的一对关系。

本章还改了两句流行话的说法，结论没变、理由变了：

- “说了不等于规定”。说出来了就是规定需求（ISO 9000:2026 §3.5.1 注 2 是一个等号）；要紧的是**说出来了不等于能当依据**，缺的那一样叫可采。
- “需求评审就是需求确认”：需求文档是工作产品，**能评审，也能验证，做不了确认**——确认要在使用中兑现。

用第 1 章的话收一次束：本章从头到尾演示的是同一只手。先立枢纽、再填细节。第一节立“开发与管理并列”这个枢纽，ConOps

立“问题域”这个枢纽，四处边界一条条磨开，都在对“陷细节而不立枢纽”说不。

对“需求工程”这个词，你原来只是“见过”，背得出“需求工程 = 开发 + 管理”；本章把它推到“拥有”，能答全四问：它是什么，开发与管理两段并列；不是什么，明示与可采、开发与管理、评审与验证确认、留档与受控这四处边界；从哪来，15288 的需求侧过程、CMMI 与 29148 的两分；往哪去，先问题域再条款。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：本章把“需求工程”推到了**第② 级（图式）**。你现在看得出“开发做出来、管理看住、批准与基线交接”这条结构，而不只是背得住那个并列式。

## 小结

承接第 3 章，本章讲需求是怎么被开发出来的：需求工程把模糊的诉求变成能当依据用的规定需求。需求开发负责做出来（获取、分析、规格说明、需求校核；以 ConOps 为问题域、SRS 为载体、规定需求为枢纽），需求管理负责让它一直是那一份（基线管理、变更控制、可追溯性、状态与度量），两段并列，交接只在“批准 → 基线”这一处。

它磨开了四处边界：明示与可采（说出来了不等于能当依据）、开发与管理（需求乱先分清是谁的账）、评审与验证确认（需求文档做不了确认）、留档与受控（改名不等于版本管理）。全章反复演示的是同一个方法底座时刻：先立枢纽、再填细节，也就是先立问题域、再写条款。

需求域的那份产品数据，也在这几周里补齐了：ConOps、第一版需求基线、CR 流程，都是顾言一件件补起来的，第 7 章会看到，这些东西本身就是产品。

下一章看另一件事：需求要全到什么程度。功能、环境适应性、可实现性三类需求及其指标齐备，规格才算立全；而需求工程产出的规定需求怎么进设计开发的树形结构、怎么被分解和分配，是再下一章（第 6 章 设计开发落地）的事。

---

# 附录

## A 本章概念总图

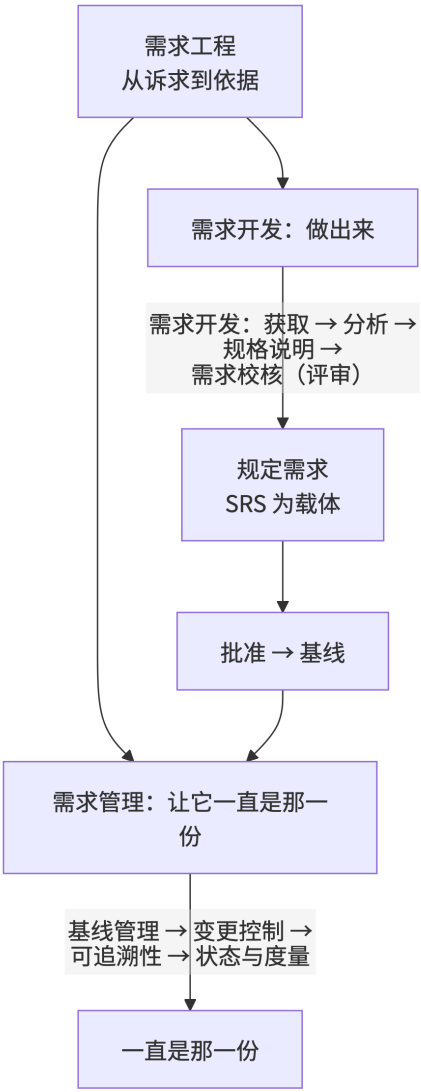

需求工程
从诉求到依据
需求开发：做出来
需求开发：获取 → 分析 → 规格说明 → 需求校核（评审）
规定需求
SRS 为载体
批准 → 基线
需求管理：让它一直是那一份
基线管理 → 变更控制 → 可追溯性 → 状态与度量
一直是那一份

图 4-2

- 第一节 · 开发与管理并列：需求工程 = 需求开发 + 需求管理（不是包含）；四个活动与循环；需求的来路；规定需求与可采条件；管理的四个活动、变更控制、基线；两段交接在“批准 → 基线”；
- 第二节 · 四处边界：明示与可采（说出来了不等于能当依据）／开发与管理（先分清是谁的账）／评审与验证确认（需求文档做不了确认）／留档与受控（改名不等于版本管理）；
- 第三节 · 怎么做：ConOps 在分析之后、规格之前（先立问题域，它同时是确认判据的载体）；一条好需求的判据；需求文档体系与不可省的骨架五份（SRS · 需求池 · RTM · 评审报告 · 基线）。

恒信走完这张图：顾言补 ConOps（问题域）→ 写 SRS（条款）→ 补 RTM 与基线 → 变更走 CR。冰链走的是同一张图，只是把“审批流转”换成了“温度不断链”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把“需求工程”“需求开发”“需求管理”“规定需求”“运行概念 ConOps”几个词说准：两段并列而非包含；规定需求是被明示的需求（明示即规定，成文属可采条件）；ConOps 是需求工程里唯一一份叙事性的正式输出；
- **图式**——两段路径合成的结构：“开发做出来 → 批准 → 基线 → 管理让它一直是那一份”，以及这条路径上的四处边界（明示与可采、开发与管理、评审与验证确认、留档与受控）；
- **心智模型**——欠账二分（需求乱先分清是开发的账还是管理的账）；可采条件四项——相关性、可检验性、时效性、同一性，缺判定方法、不可复现、没有版本，就当不了依据；变更控制怎么走完一圈、三条铁律（无 CR 不得变更、不分析不得上会、验证通过方可关闭）；

· **解释框架**——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有——分清“做出来”和“一直是那一份”：先补开发的课，再谈管理的严。

## C 自查 · 这些说法对吗？

1. “需求管理包含需求开发，管好需求自然就开发好了。”——**错**。两段并列不是包含；开发是创造、管理是控制；需求是开发出来的，不是管出来的。
2. “客户说了'页面要快一点'，这算不上需求吧。”——**错**。说出来了就是明示的需求，明示即规定；它缺的是可采——四条里只对得上相关性，没有判定方法、不可复现、没有版本，所以当不了验证的靶子。
3. “需求写完了开个评审会，就是确认通过了。”——**错**。需求文档是工作产品，**对它做的是评审与验证**（评审判适宜、充分、有效；验证认定它满足文档自身应满足的规定需求）；**确认**要等有东西可用，并且用起来才做。
4. “需求变更多，说明需求管理没做好。”——**错**。先分清是需求变了，还是当初没说清——后者是开发的欠账，加流程治不了。
5. “版本文件名改一改，就是需求管理。”——**错**。那是留档，不是受控；没有基线，变更就说不清“从什么变到什么”。
6. “ConOps 是给运营部门看的故事，跟开发没关系。”——**错**。它是问题域那一侧的正式输出，也是确认判据的载体；跳过它直接写 SRS，条款就悬空。
7. “需求分析就是开会讨论一下需求。”——**错**。分析有五件具体的事（澄清、分解、排序、建模、协商冲突），每件都有可检验的产出。

8. “写需求文档就是把用户的话抄下来。”——**错**。抄下来的是获取的原始记录；规格说明要把分析结果写成可验证的条文。
9. “隐含需求会自己冒出来，客户没提就是没有。”——**错**。隐含需求不会自己开口，要主动去挖；挖出来还得显性化、写进规格，否则验证够不着。
10. “需求开发四个活动跑一遍就完。”——**错**。它们是一个循环：分析发现缺信息回获取，规格发现问题回分析；需求天然是渐进的。

## D 练习

1. 用“开发做出来、管理让它一直是那一份”给恒信的困境做诊断：顾言列的两张单子分别是什么？你们团队最近一次需求混乱，属于哪一张？
2. 找出你们公司一条“明示但不可采”的需求（说了，但拿不出客观证据），把它改写成可采的规定需求。
3. 你最近一次写需求（或接收需求），有没有跳过分析直接进规格？跳过了，代价是什么？
4. 你们团队的“需求评审”有没有被叫成“需求确认”？如果改口叫评审，签字那一刻的责任归属会有什么变化？
5. ConOps 和 SRS 的区别是什么？“先写条款、后补故事”为什么顺序反了？
6. 用六条判据给一条你正在做的需求打分：完整、一致、无歧义、可验证、可追溯、有优先级——哪一条最弱？后果是什么？
7. 你们当前项目的需求基线在哪？没有基线的话，最近一次变更“从什么变到什么”说得清吗？
8. 需求变更走完一圈（CR → 影响分析 → CCB → 实施 → 再验证 → 关闭），你们哪一环最常被跳过？跳过之后那条变更还查得到吗？

9. “隐含需求不会自己开口”——用恒信财务那句“25 号之后签的合同算下个月”举例，说明隐含需求怎么被挖出来、怎么显性化。
10. 用 MoSCoW 给恒信合同审批系统的需求排一次序（必须/应该/可以/不要），“合同数据留档十年”属于哪一档？为什么？
11. 需求“从哪来”？用三条来路（目标转化、环境与法规、隐含需要）各举一例，说明哪一条最容易被漏掉。

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-03** ISO 9001:2015 —— §8.3.3 设计开发输入（可转化判据在标准里的位置）
- [2] **R-12** ISO/IEC/IEEE 29148:2018 —— 需求工程（需求开发与需求管理并列；单条需求质量特性与需求集特性；shall 语句形式规则）
- [3] **R-13** IEEE 1362 —— 运行概念 ConOps（结构六部分，运行场景为核心）
- [4] **R-18** CMMI（REQM / RD）—— 需求管理与需求开发是并列过程域
- [5] **R-29** IEEE 830-1998 —— 软件需求规格说明书推荐实践（好 SRS 八特性，29148 的前身经典）

- [6] **R-37** ISO/IEC/IEEE 15288:2023 —— §6.4.1 业务或任务分析、§6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义（需求工程的过程锚点）
- [7] **R-38** ISO 9000:2026（第 5 版）—— §3.5.1 需求（注 2：规定的需求是明示的需求，例如写在成文信息中的需求）、§3.11.2 评审、§3.11.12 验证、§3.11.14 确认、§3.3.11 设计和开发

**经典文献**

- [8] **R-69** MoSCoW 优先级方法（源自 DSDM）—— 需求优先级排序（Must / Should / Could / Won't）
- [9] **R-70** Kano 模型（1984）—— 基本型/期望型/兴奋型需求分类——“与利益相关方一起剪裁需求”
- [10] **R-96** Boehm & Basili，“Software Defect Reduction Top 10 List”（IEEE Computer, 2001）—— 需求缺陷的实证溯源（约半数软件错误到后期才被发现，判据从失败反推的经验锚点）

# 第 5 章 需求的完整性：全不全，责任在承制方

## 5.1 引言

第 4 章把规定需求做出来了，也让它一直是那一份；那一章没有回答的是：这一份需求本身，全不全。本章看的就是这件事。先把“全”拆成几类内容看清，再把最容易混的几处边界画开，最后看它在项目里怎么用。

### 5.1.1 误解现场

冰链科技的 M3 医疗冷链温控箱，原理样机做出来了。

嵌入式硬件工程师石军把一台样机摆在桌上：它能稳定把箱内温度控制在 2℃、波动不超过半度。金属箱体、半导体制冷片、一块单片机，走线是手焊的，电源是实验室的。它跑了一周，温度曲线漂亮极了。

产品经理林涛站在白板前宣布：“原理样机验证通过，我们的产品成功了。接下来就是放大生产。”

产品开发经理赵刚点了头：“样机跑通了，验证也过了——放大生产，我签。”

会议室一片掌声。只有角落里制造代表小周皱着眉。他翻着样机的零件清单，摆了三件事，最后问了一句：

> “制冷片我们买得到，可这颗控温芯片是定制料——供应商说最小起订量五万片，交期二十周。外壳是 CNC 一件件铣出来

> 的，量产要改注塑，这个结构和壁厚注不出来。还有，这个精度是靠手工校准的，产线上没人会做。你们管这些叫'成功'?”

会议室安静了。

如果你坐在冰链的会议室里，大概也会觉得林涛没错：样机都跑通了，验证也过了，怎么会不是产品成功?

本章要推翻的正是这个直觉。它错在一个词上：“原理样机验证通过”只验证了一类需求。

### 5.1.2 本章要回答什么

读完本章，你应该能回答：

1. 一个产品的需求要“全”，要覆盖哪几类内容? 为什么每一类都必须有指标?
2. 给需求分类，看什么? 为什么同一个特性会同时落在两类需求里?
3. 为什么“原理样机验证通过”不等于“产品成功”? 哪一类需求只有真正的产品才能验?
4. 三类内容各自由谁提出? 为什么可实现性需求客户根本不关心?
5. 为什么说承制方是需求开发的主导方?“引导”与“确认”的边界在哪?
6. 一个组织怎么才能不反复踩“可实现性需求缺失”的坑? 成本为什么是需求、不是结果?

### 5.1.3 本章的骨架

> 需求要全，全在三类内容上：功能需求、环境适应性需求、可实现性需求各有各的代言人，各有各的专属验证载体——缺哪

> 一类，交付物就降一格。而最容易被漏掉的那一类没有人替你要，所以“全不全”是承制方自己的责任。

反过来说更刺人：功能需求客户会要，环境需求法规会要；可实现性需求，只有你自己会要。

本章分三块走。第一块把“全”拆开：一份需求要覆盖哪些内容、每类由谁提出、为什么每类都要有指标。第二块把最容易混的几处画开：样机通过为什么不等于产品成功、“非功能需求”这个分法为什么不能用、隐含需求为什么不是单独一类。第三块回答最扎心的那个问题：可实现性需求客户不关心，那它靠谁提、靠什么补、缺了怎么诊。

---

## 5.2 全不全，看谁在问

> 这一块先立“全”的答案：需求要覆盖哪些内容、按什么判据分类、每一类各自包括什么、从哪里来。

### 5.2.1 一份全的需求：覆盖什么

第 3 章立过一句话：需求必须可判定。第 4 章立过另一句：需求是开发出来的，要能当依据用。本章往前走一步，问一个更基本的问题：一份需求要“全”，到底要覆盖哪些内容?

答案藏在“产品”这个词里。一份讲正向设计的培训材料里，“产品”与样机的分别写得很干净：

> **产品**：满足功能需求、环境需求和可实现性需求的成品。

也就是说，“产品”这个词的成立，本身就要求这三类内容都在场。反过来推：需求缺了哪一类，交付物就降一格。缺可实现性需求，交付物停在环境样机；环境适应性需求也缺，连环境样机都算不上。冰链那台 M3 缺的正是可实现性需求。

三类内容，每一样都不是一条笼统的要求，而是一组能展开、能设指标、能指派给利益相关方的条目：

| 问的问题 | 需求类别 | 一句话定义 |
| --- | --- | --- |
| 它能不能干我们要它干的事? | **功能需求** | 系统所具备的行为特征，以输入、处理、输出方式定义 |
| 它在真实环境里能不能活? | **环境适应性需求** | 对气候、国家、文化、风俗、法律等各类环境的适应 |
| 它能不能被经济、可重复地造出来? | **可实现性需求** | 生存周期各环节“方便实现”，加经济可承受 |

这套分法不只对系统成立。构件的需求同样由这三块构成，与系统完全同构。同一个模板，在系统级和构件级各复现一次。

> 术语注：本章把需求内容分成“功能／环境适应性／可实现性”三块，并让每一块各配一个专属验证载体，是本书对军品与工程界“完整需求”实践的**凝练与命名**，不是任何标准的原文。这个分法的一手来源是这份培训材料（§7.2 给出这三类内容的正式清单），本书在这份材料之上做了两件事：把分类判据统一到“谁在问”，把内容与载体的对应讲成一条因果。标准这边给的是另外两样东西：需求本身的定义与三种呈现方式（ISO 9000:2026 §3.5.1），以及需求过程的框架（ISO/IEC/IEEE 15288:2023 §6.4.2、§6.4.3，第 4 章已引）。欢迎检验、欢迎校准。

### 5.2.2 需求分类：同一特性，各有各的主人

这三类最容易搞混的地方，是拿“特性的名字”来分。但分类的判据不是特性本身，是**谁在问**：

- **功能需求**，是客户与使用者在问：我要的产品，做不做得到我要的事？
- 环境适应性需求，是使用环境在问（监管方、气候、法律）：你到了我的地盘，守不守规矩、活不活得下去？
- 可实现性需求，是承制方在问：我造得出来吗？造得起吗？交付得了吗？维护得了吗？

这条判据解释了为什么同一个特性会同时出现在两类需求里。拿“测试性”说：它在使用环境下被问，问的是“产品在役期间测得出状态、定得了位”，归环境适应性需求。它在承制方的生产与交付里被问，问的是“产线上测得方便、测得快”，归可实现性需求。同一个特性，两个主人，两种归类。

叫法也跟着主人走：在使用环境下被问，写“测试性”；在承制方的生产交付里被问，写“可测试性”。

### 5.2.3 功能需求：从业务用例走到场景

功能需求是系统所具备的**行为特征**，以输入、处理、输出方式定义。它不是“要有个温控功能”，而是“给定箱内目标温度 X，处理器采样、决策、输出制冷驱动，使实测温度稳定在 X±0.5℃”。有输入可构造，有输出可检查，这样才可验证。

它还包括一条从业务走到系统的展开链：

业务用例 → 业务场景 → 系统用例 → 用例场景
（用产品完成什么业务）（典型过程）（系统为支撑业务的行为）（可运行的具体过程）

这是三类内容里唯一自带展开通道的一块：环境适应性需求靠环境分析展开，可实现性需求靠实现方案展开，只有功能需求能从“利

益相关方要办什么事”一路走到“系统在什么场景下做什么动作”。这条链也是后面把需求往下分解时最主要的素材来源（第 6 章）。

**来源**主要有五个：

1. 客户与使用者的需要与期望——三类内容里最强的一个外部代言人，客户最愿意，也最能主动表达的就是这一块；
2. 业务用例与场景分析——不是听客户罗列功能，而是把业务建模成用例与场景，再从场景里推出系统该有的行为；
3. 需求分析师的专业捕获——客户以为自己在提需求，往往提的是“想要的东西”甚至“解决方案”，分析师要穿透到“真正的需要”。这一类需要常常连客户自己都没意识到，因而往往不成文；
4. 组织资产——同类业务的用例骨架、场景模板可以跨产品复用，但功能需求的主体仍是产品特异的；
5. 利益相关方确认（签认）——需求是对需要或期望的规范性陈述，并经利益相关方确认，才算成立。

功能需求的陷阱不是“没人提”，而是“提了但没提对”：客户自信满满说出的，常常是方案，不是需要。

### 5.2.4 环境适应性需求：先把环境集定全

环境适应性需求是对**各类环境**的适应。这里的“环境”是广义的：不只是自然环境，还包括法规环境与社会文化环境。

它通常展开成这么几项，每一项都答“在什么环境下，产品必须做到什么”：

1. **可靠性**——在规定条件与时间内完成规定功能、不失效，跑冷链车不出故障；
2. 安全性——运行与维护中不产生不可接受的伤害或风险，不漏电、不伤人；

3. 测试性——产品在役期间便于测试与故障定位，能快速确认自身状态；
4. 保障性——使用与维护所需的保障资源（备件、工具、人力）易于配备；
5. 适航性——符合航空器适航法规，可安全进入空中运行；
6. 气候适应性——在温度、湿度、盐雾、海拔等气候条件下保持正常功能；
7. 电磁兼容性——在电磁环境中既不受外界干扰，也不对外界产生超标干扰；
8. 法律合规性——符合投放地域的法律法规与强制性标准，例如环保回收的新规。

负责 App 与追溯平台的吴浩跟着补了一句：“冷链车进了山区连不上网，App 得先把温度数据暂存在本地，出了山区再补传——在弱网环境下，产品必须不丢数据。”

它与可实现性需求有一处重叠：测试性、保障性这一类特性，在那边也会出现（写作可测试性、可运维性）。这不是重复，是同一个特性同时服务两个主人：面向使用环境，产品在役期间要测得出、保得住。面向承制方，生产、交付、运维过程中要测得方便、维得方便。归到哪一边，还是看谁在问（§5.2.2）。

来源主要有五个：

1. 产品投放地域的自然与电磁环境本身——卖到哪、用在什么环境，需求就从哪来；
2. 强制性的标准、法律、法规——这些不是可选项，捕获的动作是“摘录”：从法规文本里摘进需求条目；
3. 文化、风俗、国家——使用地的社会文化环境，决定人机交互、内容、外观、操作习惯方面的要求；

4. 环境利益相关方，并经其确认——监管方确认合规，最终用户确认真实环境下的使用预期；
5. 组织的环境适应性需求资产库——这一类需求不应从零发明，应基于组织积累下来的资产开发。

捕获环境适应性需求的关键不在技术，在先把产品打算投放的完整环境集定下来，再逐项确认。

> 术语注：法规强制的那几条，属于需求三种呈现方式里的“必须履行的”。正本给过两个条目：法定要求是立法机构规定的必须履行的需求（ISO 9000:2026 §3.5.5），法规要求是立法机构授权的机构规定的必须履行的需求（§3.5.8）。三种呈现方式与本章的内容分类是两条轴，§5.3.5 会把它们厘清。

### 5.2.5 可实现性需求：客户不问，承制方提

可实现性需求是承制方对自己提的要求，一共八个方面，前七个各对生存周期的一个过程，第八个是横切这些过程的总账：

| 生存周期过程 | 可实现性需求 | 对应的实现方案 | 提出者 |
|---|---|---|---|
| 物料采购 | 物料可采购性 | 物料采购方案 | 采购业务代表 |
| 制造 | 可制造性 | 制造（集成）方案 | 制造业务代表 |
| 测试 | 可测试性 | 测试方案 | 测试业务代表 |
| 交付 | 可交付性 | 交付方案 | 交付业务代表 |
| 部署 | 可部署性 | 部署方案 | 部署业务代表 |
| 运维 | 可运维性 | 运维方案 | 运维业务代表 |
| 回收 | 可回收性 | 回收方案 | 回收业务代表 |
| 生存周期全程 | **经济可承受性** | 成本方案 | 市场代表 |

这张表有三处要点。

其一，三条线是锁死的：过程、需求、方案。实现方案是对可实现性需求对应内容的**符合性解答**：物料采购方案回答物料可采购性需求，制造方案回答可制造性需求，如此类推。一条可实现性需求缺了，与它对应的那条方案也就没有依据。

其二，经济可承受性虽然是第八条，却与前面七条不是并列关系：它是横切前面七项的总账，所以它是**需求项**，不是过程。七个过程加上这条总账，才是完整的八个方面（§5.4.6 单独说它）。

其三，构件级与系统级同构。可实现性需求也还是那些方面，只是构件级的“可运维性”在有些材料里写成“可服务性”。那是同一个方面在系统级与构件级的两种叫法，本章一律按系统级的叫法写。

> 术语注：第 3 章讲过设计开发的终点是“可实现”，那时说的是设计输出的完成度（下游各方案配齐、输出完整的规定需求）；本章讲的“可实现性需求”是需求集合里的一类内容（承制方在需求阶段提出的诉求）。一个是终点判据，一个是需求类别，两处不是一回事，别混。两张表合看才完整：第 3 章从设计输出面向下游，本章从承制方在需求侧提出诉求。另有一张表要分清：本书讲设计开发的终点“可实现”时，列的是下游各方案（技术、物料采购、制造、测试、交付、部署、操作、运维）；本章这八个方面多“回收”与“经济可承受性”、少“技术”与“操作”——**两者不是同一张清单的两种写法**。

来源主要有五个：

1. 承制方各业务代表——采购、制造、测试、交付、部署、运维、回收的业务代表，加市场代表。可实现性需求不是客户提的，是组织对自己提的；

2. 承制方现有的能力与资源约束——人、机、料、法、环，需求的指标边界据此设门槛；
3. 开发过程中的现实检验——产品开发本身就包括开发加工与装配技术、开发供应商、研制新设备与工装；一旦某条实现不了（物料买不到、技术不成熟、设备拿不到），必须回头改技术方案，而不是悄悄绕过这条需求；
4. 组织资产——工艺、供应商、设备方面的长期积累；
5. 市场与竞争研究——经济可承受性来自对竞品与替代产品的价格成本研究。

客户不关心这一块，所以得由承制方自己提。客户只关心结果：价格、交期、质量、售后；他不关心你的成本结构、物料好不好采购、好不好制造。更麻烦的是，客户还可能反向挤压它：定制化要求违背货架产品策略，不停加功能抬高制造难度，压缩报价挤压成本空间。可实现性需求常常是那个需要组织顶住客户压力才守得住的底线。

### 5.2.6 每一类需求：都要有指标

三类内容各自包括什么、从哪里来，到这里都看清了。还剩一条三类共用的规矩：需求等于描述加指标，缺了指标就不成立。

拿 M3 说：

- **功能需求的指标**：温控精度、响应时间、续航时长、容量；
- 环境适应性需求的指标：工作温度范围、防护等级、电磁兼容的限值、法规条款的符合性；
- 可实现性需求的指标：目标成本、关键尺寸公差、交期、服务周期。

一句“要控温准”是愿望；“箱温波动不超过 ±0.5℃”才是需求。没有指标的需求，三块都会退化成愿望。

**概念卡片 · 三类内容**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 产品需求要全，全在三类内容上：功能需求（以输入、处理、输出定义的行为特征）、环境适应性需求（对气候、国家、文化、风俗、法律等各类环境的适应）、可实现性需求（生存周期各环节“方便实现”加经济可承受）——每块都“及其指标” |
| **它不是什么** | ⑥⑦ | 对立面：只写功能需求，把“产品需求”当成“功能清单”，这是最常见的伪全。辨析：功能与非功能的二分是这三块的一个子集，不是它的替代（§5.3.4 展开）；隐含需求也不是单独一类，它是“怎么出现”的一种形态（§5.3.5 展开） |
| **它从哪来** | ②③⑨ | 背景：需求“全不全”在“产品与样机”的落差里暴露——从样机翻车、量产反咬的教训里逼出来。历史：源自军品与工程界的完整需求实践，本书凝练为三类内容加各自的专属验证载体，并把分类判据统一到“谁在问”。英文：functional／environmental adaptability／realizability requirement |
| **它往哪去** | ④⑤⑧ | 它回答“一份需求要全到什么程度”——缺可实现性需求，交付物停在环境样机；环境适应性需求也缺，连环境样机都算不上。用在需求捕获、需求评审、规格编写三处：每一块都要答出来源、利益相关方、指标、验证载体。误区 ✗ “需求写全了等于功能写全了”——功能写全只够造原理样机 |
| **判据** | 谁在问 | 每一块都能答出“谁在问”（客户、环境、承制方），每一块都有指标，每一块都有专属验证载体 |

## 5.3 先立载体，再分清混的说法

> 这一块把“全”和它容易混的样子分开：先立每类内容各配一个验证交付物，再逐条磨开四处边界。

### 5.3.1 验证载体：各管一类需求

三类内容各自对应一个**专属的验证交付物**，本书叫**验证载体**。这不是分法的巧合，是一条因果链：

需求类别 → 决定验证条件 → 决定交付物形态

| 需求类别 | 要在什么条件下验证 | 验证载体 | 确认方 |
| --- | --- | --- | --- |
| 功能需求 | 把功能从环境与量产成本约束里**剥离**出来，单独验证原理 | **原理样机** | 客户与使用者 |
| 环境适应性需求 | 把产品放进**真实环境**去跑 | **环境样机** | 环境相关方 |
| 可实现性需求 | 在**量产经济性**约束下实现出来 | **真正的产品** | 承制方 |

为什么必须各有一个？因为三类内容需要的验证条件互不相容：验证“原理对不对”，环境干扰越少越好。验证“真实环境下活不活得下来”，就必须上真实环境。验证“能不能经济、可重复地造出来”，只能在量产的约束下见真章。一个载体替不了另一个。原理样机跑通了，也证明不了你量产能做出来。

负责测试的季昭点头应道：“那我手里的活也跟着载体走——原理样机上我只能测功能，环境样机上测真实环境，能不能上产线，得等真正的产品出来。按本章的说法，每一类需求各有一个专属载体；样机不换，测来测去还是那一块。”

工程界有一个同构的刻度可以借来看这件事：美国航空航天局的技术就绪水平（TRL），从“基本原理被观察到”到“在真实环境中运行”，一共九级。原理样机的演示通常落在中间几级，离“真实环境运行”还隔着距离。而“能不能造出来”归另一个刻度（制造就绪水平 MRL）管。这两种就绪水平只借来说明一件事：样机通过，不等于产品就绪。

概念卡片 · 验证载体

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 每一类需求的专属验证交付物：功能需求对原理样机、环境适应性需求对环境样机、可实现性需求对真正的产品 |
| **它不是什么** | ⑥⑦ | 对立面：用一个载体验证全部需求——拿原理样机的通过宣布产品成功。辨析：验证载体与“评审、验证、确认”（第 8 章）不是一回事——载体回答“每一类需求用什么证明”，**评审、验证、确认各自回答“对照什么”**；另外，确认的参照物是使用情境、运行概念、用例这一族东西，它与三类验证载体并列，不能互相替代 |
| **它从哪来** | ②③⑨ | 背景：三类内容需要三种互不相容的验证条件——原理要脱离环境与成本单独验，环境要放进真实环境验，可实现要在量产经济性约束下验。历史：技术就绪水平（TRL）把技术成熟度分成九级，原理样机演示对应中间几级、真实环境运行对应最高两级（二手）；生产就绪是制造就绪水平（MRL）管的另一个刻度。英文：prototype（原理样机）；validation vehicle |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它往哪去** | ④⑤⑧ | 它回答“每一类需求用什么证明”——用原理样机的通过就宣布产品成功，会漏掉环境适应性还没验、可实现性根本没进需求清单。用在交付验收、里程碑决策、需求评审三处：判定“交付物是什么”，看三类载体的验证完成到哪一步。误区 ✗ “样机验证通过了等于产品成功了”——那只验证了功能需求 |
| **判据** | 各管一类 | 每一类需求都能答出“用什么交付物验证它”——答不上来的那一类，就是需求没写全的证据 |

载体、验证、确认，这三样不是一回事，这里先划一条边界。三类载体承载的是验证，判据是明示的规定需求（ISO 9000:2026 §3.5.1 注 2）。

而“确认”对准的不是条目，是为特定预期用途或应用的需求。确认要在真实或模拟的使用条件下做一次（§3.11.14 注 3：使用条件可真实可模拟）。所以环境样机上“逐条环境指标是否达标”属验证，“放进真实使用环境看合不合用”属确认。确认的参照物是使用情境这一族东西，与三类验证载体并列，不能互相替代。展开留给第 8 章。

### 5.3.2 累加的是交付物，不是验证

细看会发现，交付物是一级级**累加**起来的：环境样机当然得先把功能做全（不工作就谈不上环境适应），真正的产品当然得先满足功能与环境。所以很多人顺口就把“原理样机、环境样机、真正的产品”当成需求逐级累加的三段，以为做到最后一段，前面几块内容就都验过了。

累加的是交付物，**不是验证**。每一块内容有且只有一个专属载体：功能需求靠原理样机验证，环境适应性需求靠环境样机验证，可实现性需求靠真正的产品验证。原理样机不验证可实现性需求，环境样机也不验证它。

可实现性需求，只有真正的产品（量产、交付、运维跑过一遍）才能验证。这就是“原理样机通过”与“产品成功”之间那段距离的来源。

### 5.3.3 样机与产品：通过不等于成功

回到章首那场会。林涛说“原理样机验证通过，我们的产品成功了”，他做对了什么？他确实验证了功能需求：温控原理成立，这一关过得漂亮。

他漏掉的是什么？环境适应性需求还没验：真上了冷链车，温差、振动、零下二十度的库房，曲线还平不平？可实现性需求根本没写进需求清单：定制料断供、注塑结构做不出来、手工校准上不了产线。

精确地说，他把**一个载体**的验证结果，当成了**全部载体**的验证结果。而那些“不方便”，压根不是“问题”，是可实现性需求缺失的症状。

一句话：样机通过只证明了它是一台好的原理样机。把它当产品，是需求的完整性欠账在交付环节结的账。

### 5.3.4 功能与非功能，分得开吗

工程界最常见的需求分类不是三分，是两分：功能需求与非功能需求。这个分法听着顺手，却是需求分类里最误导人的一个。

“非功能”是一条排除法划出来的：凡是不能当场演示“它能干什么”的，全被划到这个名目下。性能、可靠性、环境适应性、可维护性、可部署性、成本，它们的主人、来源、验证载体全不一样，却在同一个名字下被当成同类需求处理。

拿“谁在问”这条判据一量就清楚了：这个名目底下装着两块，一块是**环境适应性需求**，环境在问、用环境样机验；另一块是**可实现性需求**，承制方在问、用真正的产品验。

它的害处有三条。一是判据丢了：排除法答不出“谁在问”，你问“这条非功能需求该归谁提”，没人答得上来。二是验证载体糊了：环境样机与真正的产品是两回事，装进同一个名目就分不清该用哪个去验。三是“全”的标准消失了：“非功能”没有上限，只能越列越多；而“内容分几块、每块及其指标”有判据、有终点。

所以本章不用“非功能需求”来给需求分类：功能与非功能的二分，是三类内容的一个子集，不是它的替代。判据主义在这里磨出的一条边是：别用排除法给需求分类，要用“谁在问”。

### 5.3.5 隐含需求：算不算单独一类

前面磨掉了一条边：别用“功能与非功能”给需求分类。这里再磨一条，专治“三”字撞车：“怎么出现”与“归哪一类”，是两条不同的轴。

第 3 章讲过需求的三种呈现方式：**明示的**，说出来了；**通常隐含的**，惯例默认、没说出口；必须履行的，法规强制的那一类。三个都是需求。本章讲的是按内容分类：功能、环境适应性、可实现性，判据是“谁在问”。

两条轴互不相干。一条功能需求可以是明示的（合同写明“箱门可单手打开”），也可以是隐含的：谁都没写，但你漏了，客户第一天就会说这产品不行。一条环境适应性需求多半是必须履行的（法规摘录），也可以是惯例默认的。

三种呈现方式落到哪一支，大致有偏向：环境适应性需求多为必须履行的那一支，可实现性需求多为明示的那一支（组织自设），功能需求则是明示与通常隐含两支的混合。

所以隐含需求不是单独一类。它不是“谁在问”的一种答案，而是“怎么出现”的一种形态，任何一块内容都可能以隐含形态出现。

隐含需求的来路是一条旁路。需求的上游大致有三条通道：从目标转化而来、从环境与法规而来、从利益相关方没说出口的需要而来。隐含需求正是第三条通道的产物。它不经目标转化，未必被目标覆盖，而且常常不成文。客户自己都没意识到的需要，写不进目标，也就无从“转化”，只能由分析师从利益相关方的需要里挖出来（§ 5.2.3 来源三）。

隐含需求不成文，靠文档做的验证够不着；拦得住它的，是用真实使用场景做的确认（第 8 章）。这也给本章的判据补了一条边界：“谁在问”只回答“归哪一块”，不回答“怎么出现的”。归好类之后还得追问一句：这一条是明示的，还是隐含的？隐含的那一条，才最容易被漏掉。

---

## 案例进展①｜恒信的上线翻车，翻的不是功能

恒信的合同审批系统没有物理样机，但同一副框架照样成立。需求分析师顾言复盘时发现，公司里反复出现的“系统上线后一堆问题”，很多不是功能问题，而是三类内容在 IT 世界里以另一种形态缺位：

- **功能需求**（合同能发起、能审批、能归档）——恒信做得最早，原型阶段就把流程跑通了；
- **环境适应性需求**（月末五百人并发、审计留痕、内网安全合规）——只在试点（放进真实业务环境跑）时才验；
- 可实现性需求（可部署、可运维、可升级、成本可承受）——只有真正的产品（投产、运维跑过一年）才验。

恒信的“样机”是原型、试点、真正的产品；冰链的“样机”是原理样机、环境样机、真正的产品。同一副框架，两个世界，长一个样。

复盘会上，顾言把投影关了：“原型跑通那天，三个验证载体，我们签过几个字?”

没有人接话。他转身在白板上写下：“恒信大部分'上线翻车'，翻的不是功能，是可实现性需求——部署文档缺失、运维没人接、升级要停机、一年运维成本是预算的两倍。原型跑通那天，这几样一条都验不了；要等真正的产品跑过一年，它们才会浮出来。”

---

## 5.4 缺的需求从哪儿补

> 这一块回答最扎心的问题：可实现性需求客户不关心，那它靠谁提、靠什么补、缺了怎么诊。

### 5.4.1 谁主导，谁确认

前面把三类内容的来源一条条列过了。需求开发其实是一场三方戏：

| 角色 | 是谁 | 干什么 |
| --- | --- | --- |
| **来源方** | 客户与使用者、监管与环境、组织资产 | 提供原始素材：需要与期望、强制性标准、经验与问题 |
| **主导方** | **承制方** | 驱动需求开发全过程：捕获、分析、建模、引导范围、形成条目 |

| 角色 | 是谁 | 干什么 |
| --- | --- | --- |
| **确认方** | **利益相关方** | 终审：需求条目成立，以利益相关方**确认（签认）**为准 |

来源方提供原料，承制方主导加工，利益相关方负责确认。客户说了算的不是“需求是什么”，而是“这条需求成不成立”：内容由承制方主导，成立由利益相关方确认。

> 术语注：这里的“确认”是**签认受理**——需求条目成不成立；不是第 8 章的 validation（对预期用途的确认，ISO 9000:2026 §3.11.14）。同一个词两个义，本章分开写。

承制方凭什么主导？因为客户知道**自己要什么**（他的业务、他的痛点），却不知道另外四件事，而这四件恰好是承制方的强项：

1. **什么技术可能**——有没有更优的实现路线；
2. 行业横向视野——服务过很多客户与产品，知道同类业务普遍缺什么、普遍踩什么坑，单个客户永远没有这个广度；
3. 标准与环境——法规、环境、行业规范，是承制方在摘录；
4. 资产积累——组织长期攒下来的经验、建议、问题，这是专业性的物质基础。

正因为这四条，承制方不只是被动接收客户需求，而是主动引导需求：把客户“想要的”穿透成“需要的”；提出客户没想到的需求（行业惯例、标准、未来趋势）；建议技术路线与范围；把可实现性约束主动注入需求。

引导能力本身也是一条判据：只能逆向模仿的承制方，是被动接收客户规格的执行者，谈不上引导。只有具备理论推导内核的承制方，

才可能提出客户自己都想不到的需求。正向设计的能力，从需求捕获那一刻就开始用了。

概念卡片 · 承制方主导

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 需求开发由承制方主导（捕获、分析、建模、引导范围、形成条目），利益相关方保留确认终审权 |
| **它不是什么** | ⑥⑦ | 对立面：客户主导，把需求开发当成“客户说、承制方记”；承制方对产品更专业，是主导方而非转述方。辨析：承制方（主导，负责加工）与来源方（客户、环境、资产，提供原料）、确认方（利益相关方，终审）三分工；引导可以主动，确认不能缺席 |
| **它从哪来** | ②③⑨ | 背景：客户知道“自己要什么”，却不知道技术可能、行业视野、标准环境、资产积累这四件事，需求开发只能由承制方主导加工。历史：源自供需双方的契约视角——承制方是产品的实现方，也天然是需求开发的主导方。英文：承制方＝产品的实现方（realizing party／developer）；主导≈lead／drive，与 guide（引导）不是同一个词 |
| **它往哪去** | ④⑤⑧ | 它回答“需求由谁做出来、谁说了算”——客户说了算的不是需求是什么，而是需求成不成立。用在需求捕获、需求评审、范围沟通三处：承制方主动把可实现性约束注入需求。误区 ✗ “承制方就是听客户要什么就做什么”——它引导范围、塑造需求 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **判据** | 责任归属 | 需求全不全、明不明确、可不可度量，问责承制方；引导可以主动到拉满，利益相关方的确认一次不能省 |

### 5.4.2 主导不等于替客户拍板

承制方主导的是**过程与专业内容**，确认权始终在利益相关方手里。引导的**力度**可以拉满：该追问的追问到底，该提的主动提，该顶住压力挡回去的就挡回去。被限制住的从来不是力度，而是最后那一个动作：替对方做决定。

引导越界有几种常见的样子：为了可制造性，悄悄删掉一条功能；为了用货架件，把标准件硬塞进需求；用一句“客户其实不需要”把需求压下去。这些动作看着都是在替客户着想，实质是把自己的约束伪装成了客户的需求。一旦越界，需求就不再是“客户要办的事”，退化成承制方自己方便的设想。

所以这一层边界是一条铁律：

> 承制方引导可以主动到拉满，利益相关方的确认一次都不能省。

这条与第 16 章要讲的“目的的所有权在业务方”是同一件事：内容可以主导，目的与成立的终审在外部。后记里那份《系统目的或任务理解确认单》由业务方签字生效，走的正是同一条规矩。

### 5.4.3 没人催的需求，承制方得记住

既然是主导方，“需求全不全、明不明确、可不可度量”这三件事，就是承制方的责任，不是客户的。

客户可以说“我不知道自己要什么”，那是他的本分；承制方不能说“客户没提，所以需求不全”，那是失职。

这条责任在本章格外扎人，因为最容易漏的那一块内容，恰好是**没有外部代言人**的那一块。功能需求客户会主动提，环境适应性需求法规会强制提，可实现性需求，客户根本不关心，只有组织自己会要。没有人催的那一块，只能靠主导方自己记住。

### 5.4.4 承制方靠资产积累，不靠现场救火

可实现性需求还有一个性质，让它比另外两块好补：**它的复用度最高**。

功能需求随产品而变，环境适应性需求随投放市场而变。可实现性需求不一样：组织的供应链、工艺、设备、人员约束跨产品持续存在。过去的产品在制造上踩过的坑，下一个产品十有八九还会踩。采购难、装配难、运维难，这些痛点在同一个组织里几乎是常量。

| 类别 | 重新捕获的比重 | 复用组织资产的比重 |
| --- | --- | --- |
| 功能需求 | 高（随产品而变） | 低 |
| 环境适应性需求 | 中（随投放环境） | 中（法规与标准类可积累） |
| **可实现性需求** | **低** | **高（组织约束长期恒定）** |

所以这一块**最适合，也最应该**基于组织资产开发，而不是每个项目从零发明。它的来路是一个回路：

问题 / 经验 / 建议 →（转化）→ 可实现性需求资产 →（复用）→ 新产品需求 →（产生新问题）→ 回到起点

关键词是“转化”，问题不是照抄进需求，而是被提炼成一条规范的需求条目，并配上指标：

- 问题：“这批零件公差太严，装配车间返工率三成” → 转化 → 可制造性需求：“零件关键尺寸公差 ⩾ X，装配间隙 ⩾ Y”；
- 建议：“接口统一成标准件” → 转化 → 物料可采购性需求：“优先选用标准外购件（货架产品，COTS）”；
- 教训：“上一代低温试验反复失败” → 转化 → 环境适应性需求资产条目。

“自觉”两个字是这套积累成立的前提，也定义了它的反面：问题靠口口相传，只在现场救火，从不落成条目。这样的组织看着也在积累，其实那些经验是散的、隐性的、没法复用的。判断一个组织自不自觉，只看一条：它有没有一条制度化的回路，问题、分析、转成需求条目、入库、新产品复用、复盘更新。

于是有一个更硬的推论：没有资产积累机制的组织，天然捕获不到全面的可实现性需求。不是承制方的代表不肯说，是组织根本没有可复用的东西可以拿来提。“大部分团队都不考虑可实现性需求”，不是一次需求访谈的疏漏，是组织知识管理缺位的必然结果。

### 5.4.5 从症状反推缺了哪条需求

既然这一块内容这么重要，又这么容易被漏，那漏了之后怎么查出来？先看它为什么会被漏——四个错位：

1. **利益相关方错位**：可实现性需求的提出者是承制方，不是客户。团队习惯只访谈客户，自然捕获不到；
2. **时间错位**：功能需求缺失，原理样机阶段就爆；环境适应性需求缺失，环境样机阶段爆；可实现性需求缺失，要到量产、交付、运维才爆——最晚，也最贵，于是在短期评估里显得“不重要”；

3. 话语权错位：研发阶段由研发主导，采购、制造、运维的代表没有在需求阶段发声的机制，等产品定了稿才说“做不出来”；
4. 认知错位：团队把可实现性当成“后续工艺的事”，而可实现性其实是产品的固有特性，必须写进需求。

四个错位导致一个最常见的现场：团队在量产时吐槽“物料难买、零件难造、测试难做、交付难搞、部署难办、运维难弄、退役麻烦、成本失控”，然后互相指责采购、制造、运维“没做好”。

但按本章的框架，这些症状不是执行问题，是需求缺失问题：

| 事后症状 | 缺失的需求条目 |
| --- | --- |
| 物料不方便采购 | 物料可采购性 |
| 零部件不方便制造 | 可制造性 |
| 不方便测试 | 可测试性 |
| 不方便交付 | 可交付性 |
| 不方便部署 | 可部署性 |
| 不方便运维 | 可运维性 |
| 不方便退役 | 可回收性 |
| 成本很大 | 经济可承受性 |

团队每抱怨一个“不方便”，就等于供认了一条当初没写进需求的可实现性需求。诊断的方向整个反过来：不是“执行要不要改进”，而是“需求起点全不全”。

这一招正是第 1 章那套方法里的反事实检验在需求侧的动作：第 1 章问“去掉这个枢纽，这个领域还成立吗”，本章问“没有这条可实现性需求，量产会怎样”。它把“缺了会怎样”从一句抽象假设，变成一次从现场症状反推的查账。

## 案例进展②｜冰链的量产翻车清单

小周在会后列了一张单子，给林涛看。这张单子后来被冰链叫作“量产翻车清单”：

> 定制控温芯片——物料不方便采购，最小起订量五万片、交期二十周；箱体外壳 CNC 铣出来——不方便制造，注塑的模具与壁厚做不出来；精度靠手工校准——不方便测试，产线上没人会、也来不及；电源适配器没有认证——不方便交付，进不了冷链资质清单；固件没有远程升级通道——不方便部署，客户现场只能返厂；备件库里没有温控模块——不方便运维，坏一块换整机；电池按环保新规要回收——不方便退役，回收成本比电池还贵；全部加起来——成本超预算四成，经济可承受性崩了。

小周把单子往前推了推：“八条我都在产线上一条一条对过，不是听来的。”

石军接了一句：“这八条里有三条是固件侧的——不只是产线的事。”

林涛越看脸越白：“这八条，没有一条是'制造没做好'，全是'需求当初没写'——定制料对应物料可采购性，注塑做不出来对应可制造性，手工校准对应可测试性。我们一直在查执行，该查的是需求起点。”

赵刚站在旁边，脸色也没比林涛好到哪去：“放大生产是我签的字——我签的是'样机跑通了'，不是'产品造得出来'。第 3 章说设计开发止于可实现，第 4 章说需求是开发出来的；这两句今天落到我头上：这份需求从起点就缺着一块，而那一块，从来没有外部代言人。”

他把那页单子接过去，八条一条一条看，一条一条签。

系统工程师小张把那张单子按顺序又看了一遍：“这八条不是八件麻烦，是八条需求。八条写进需求清单，各配一个指标。验的时候照 §5.3.1 那张表走，它们的载体只有一个：真正的产品。今天写不进清单的那一条，到量产还会再翻一次车。”

### 5.4.6 成本是需求，不是结果

可实现性需求的八个方面里，最后一条要单独说，因为它的地位和其他几条不一样：**它是总账**。

经济可承受性差，几乎总是“物料不方便采购，加上不方便制造，加上不方便测试”等等共同把成本堆上去的结果。它有四处特殊。

第一，成本是需求，不是结果。产品的成本是相关需求经过分解、分配、分析之后的结果，不是单纯“设计出来”的。没有成本相关的需求，设计就没有依据和目标：每次选型、定工艺、定测试深度，都没有预算约束可以对着优化，成本只能失控。等量产报价发现太贵，再开“降本大会”换供应商、压工艺——每一次降本动作，都等于承认经济可承受性需求当初没写过。

第二，提它的不是客户，是市场代表。客户要的是报价越低越好，财务管的是账不是需求。提出经济可承受性这一条的是市场代表，依据是对竞品与替代产品的价格成本研究，给出价格与成本的建议，其他人负责承接和实现。团队没让市场代表在需求阶段发声，成本需求就缺位；就算设了一个成本目标，没确认利益相关方、没落到承接者，它仍然不是一条需求。

第三，它验证得最晚，几乎不可逆。成本要到真正的产品阶段才能用完整的成本记录来验证。而压成本的杠杆全是设计期的决定：优先选用标准外购件、避免定制化外包、把核心技术模块做成可复用资产。等发现贵了再改，架构基线与物料清单已经冻结。

第四，它同样需要指标。目标成本、生存周期成本上限、降本目标——不可衡量，同样不成为需求。

### 5.4.7 认知反转

回到章首那场会。林涛说“原理样机验证通过，我们的产品成功了”。现在我们知道，他把**一块内容**的验证通过，当成了**全部内容**的验证通过。

精确地说：石军的样机验证了功能需求，温控原理成立，这一关他过得漂亮；环境适应性需求还没验；可实现性需求根本不在需求清单里。那八条“不方便”，不是量产时冒出来的执行问题，是需求起点就缺位的证据。

冰链交付的不是产品，充其量是一台经过验证的原理样机：原理对、能控温，但造不出、交付不了、用不起。

本章的反转是：“产品需求”这几个字，大多数人以为等于“功能清单”。真相是功能只占一份，另外两份分别是环境适应性与可实现性；而最容易被漏掉的，恰恰是没有外部代言人的那一块——客户不关心你造不造得出来，只有组织自己会要，而组织自己又常常忘了要。

判断一份需求全不全，不要问“功能写全了吗”，要问“每一块内容，都能答出用什么交付物验证它吗”——答不上来的那一块，就是欠账。

还有更深的一层：需求全不全，是承制方的责任，不是客户的。客户可以不知道自己要什么；承制方不能说“客户没提，所以不全”。

而“承制方为什么不提可实现性需求”的答案，多半不是他们不肯，是组织没有积累下可提的东西。这给出一条普遍的提醒。当一个团队说“下次吸取教训”的时候，先问一句：这次的问题，有没有变成一条可复用的需求？问题不转成资产，教训就永远是教训。

用第 1 章的话收一次束：对“需求的完整性”，你原来只是“见过”，背得出“样机不是产品”，也喊得出“需求要全”。本章把它推到“拥有”：能用“谁在问”给任何一条需求分类，能用验证载体查出哪一块欠账，能从一声“不方便”反推出当初没写的那条需求。能答全四问，才算拥有。

用第 1 章 §1.4.4 的尺子量一下：本章把“需求的完整性”推到了第 ② 级（图式）。内容与载体那张对应表，你现在一眼看得出缺口在哪。

## 小结

承接第 3、4 章，本章回答的是“一份需求要全到什么程度”。答案是把需求拆成三类内容：功能（客户在问，以行为特征定义）、环境适应性（环境在问）、可实现性（承制方在问），每块及其指标；判据是“谁在问”。

然后立起内容与载体的对应：原理样机验证功能需求，环境样机验证环境适应性需求，真正的产品验证可实现性需求。“样机通过不等于产品成功”不是一句谨慎话，是需求的完整性在交付环节结的账。

最后回答最扎心的问题：可实现性需求为什么总被漏掉。答案不是能力，是四个错位、没有外部代言人，以及组织没有积累下可提的东西。而这一切的责任都在承制方——它是需求开发的主导方，引导可以主动到拉满，利益相关方的确认一次都不能省。

把这条链反过来用，就是本章给的现场工具。从症状反推缺了哪条需求：每一声“不方便”，都是一条缺失需求的自白。

散会那天，林涛把那句“我们的产品成功了”又说了一遍，说完自己先摇了头——他要的那份需求，从一开始就缺着一块，而那一块没有人替他提。他记住了自己欠的是哪一块账。

到这里，“要什么”这一问答完了：工程是什么，需求、设计开发、质量三个词，需求从哪里来、要全到什么程度。这些都立起来了，接下来看它怎么被用来造东西。

下一章是第 6 章 设计开发：需求与方案结对。它看的是这份完整的需求集合怎么被用来造东西：一份需求与一份方案结成一对，靠分配生成、分析迭代、分解分析，一层层变成可实现的方案。

# 附录

## A 本章概念总图

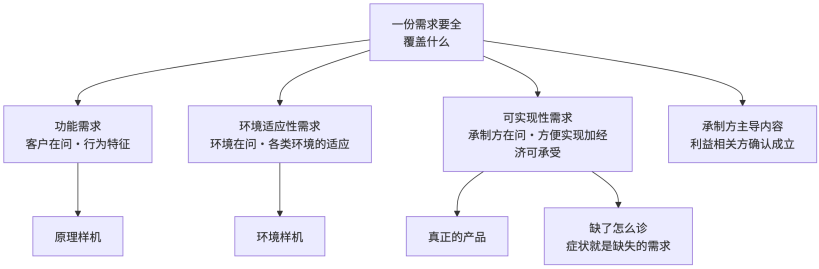



图 5-1

- 第一节 · 内容：功能需求（输入、处理、输出定义的行为特征，自带从业务到系统的展开链）、环境适应性需求（对气候、国家、文化、风俗、法律等各类环境的适应）、可实现性需求（生存周期各环节“方便实现”加经济可承受）；每块都“及其指标”；分类判据是“谁在问”；
- 第二节 · 边界：验证载体各管一类需求（原理样机、环境样机、真正的产品）；交付物累加而验证不累加；样机通过不等于产品成功；拿“非功能”给需求分类，分不出谁在问；隐含需求不是单独一类，它与内容分类是两条轴；

- 第三节·怎么用：承制方主导、利益相关方确认；责任在承制方；可实现性需求靠组织资产补，从问题转化而来；缺了就从症状反推；成本是需求，不是结果。

恒信走完这张图：原型验功能、试点验环境适应性、投产运维才验可实现性。冰链走的是同一张图，只是把“并发、审计、合规”换成了“温差、振动、断链”。

## B 本章的能力级

**第 ② 级为主**

- **表征**——把“功能需求”“环境适应性需求”“可实现性需求”“验证载体”“承制方主导”几个词说准：三类内容各有各的代言人；验证载体是每一类需求的专属验证交付物；内容由承制方主导、成立由利益相关方确认；
- 图式——内容与载体的对应结构：需求类别决定验证条件、验证条件决定交付物形态，缺哪一块内容，交付物就降一格；以及“谁在问”这条分类判据；
- 心智模型——从症状反推缺了哪条需求（每一声“不方便”对应一条缺失的可实现性需求）；四个错位（利益相关方、时间、话语权、认知）；问题、资产、需求这条回路；成本是需求、不是结果；
- 解释框架——本章不建，由第 17、18 章承担。

> 能答全四问，才算拥有——一份需求缺的是哪一块、该拿什么去验，你现在看得出来；样机通过，也不再等于产品成功。

## C 自查 · 这些说法对吗？

1. “原理样机验证通过了，产品就成功了。”——**错**。只验证了功能需求；环境适应性需求还没验，可实现性需求压根没进需求清单。

2. “需求写全了，就是功能写全了。”——**错**。功能只是三类内容里的一块；缺了可实现性需求，交付物是环境样机，不是产品。
3. “可实现性需求是后续工艺的事，不归需求管。”——错。它是产品的固有特性，必须写进经利益相关方确认的需求；没有它，设计就没有依据和目标。
4. “成本是算出来的，设计完再核算。”——错。成本是需求分解、分配、分析的结果；没设成本相关的需求，设计就没有优化的目标。
5. “客户不关心的需求，就是不重要的需求。”——错。可实现性需求客户根本不关心，可它决定组织能不能活下去。
6. “承制方就是听客户要什么就做什么。”——错。承制方是需求开发的主导方，更专业、引导范围；但引导不等于替客户拍板，利益相关方的确认不能缺席。
7. “需求不够全，是客户没提清楚。”——错。需求全不全是承制方的责任；客户可以不知道自己要什么，承制方不能说“没提所以不全”。
8. “量产翻车是采购、制造、运维没做好。”——错。每一个“不方便”，都是一条当初没写进需求的可实现性需求。
9. “一个特性归哪一类，看特性的名字。”——错。分类的判据是“谁在问”；同一个测试性，在使用环境下是环境适应性需求，在承制方的生产交付里是可实现性需求。
10. “原理样机、环境样机、产品是需求逐级累加的三段，做到最后一段就都验过了。”——错。累加的是交付物，不是验证；可实现性需求只有真正的产品能验。
11. “八条可实现性需求配不配实现方案，是后面设计的事。”——错。过程、需求、方案三条线是锁死的；实现方案是可实现性需求的符合性解答，一条缺了，对应的方案就没有依据。

12. “我们经验很丰富，问题口口相传，这就是在自觉积累。”——错。散的、隐性的、没法复用的经验不是资产；自觉积累指的是那条制度化的回路。
13. “'非功能需求'就是功能之外的需求，管性能、可靠性这些。”——错。功能与非功能的二分只是三类内容的子集；这个名目底下装着环境适应性需求与可实现性需求两块，它们的主人、来源、验证载体都不同。
14. “隐含需求是三类之外的又一类需求。”——错。隐含是“怎么出现”的一种形态，不是“谁在问”的一种答案；任何一块内容都可能是隐含的。

## D 练习

1. 拿冰链的八条“不方便”复盘：它们分别对应哪八条可实现性需求？如果重做需求，哪几条最该在需求阶段就写出来？
2. 找你们的一个产品，把它的功能需求、环境适应性需求、可实现性需求各列三条。哪一块最少？为什么？
3. 对你们的产品问一遍验证载体那一条判据：每一块内容，用什么交付物验证？答不上来的那一块，就是欠账。
4. 你们最近一次量产翻车或上线翻车，症状是哪一个“不方便”？反推出来缺的是哪条需求？当时的需求评审里，为什么没人提？
5. 用“测试性”与“可测试性”举例：同一个特性为什么有两个名字、两种归类？
6. 你们做需求捕获时访谈的对象都有谁？采购、制造、运维、市场的代表，有没有在需求阶段发过声？
7. 画一条你们组织的“问题、资产、需求”回路：最近一次事故，有没有变成一条可复用的需求条目？没有的话，卡在哪一步？

8. 经济可承受性这条需求，你们由谁提出、设没设指标、有没有经市场代表确认？如果没有，你们的“降本大会”是不是每隔一段时间就开一次？
9. 用恒信的镜照走一遍：你们的原型、试点、真正的产品分别对应什么？“上线成功”与“产品成功”之间差了什么？
10. 用四问法量一量“验证载体”或“承制方主导”：它是什么、不是什么、从哪来、往哪去？哪一问最薄，为什么最薄的那一问恰恰是你最容易出错的地方？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。


**标准**

- [1] **R-10** NASA TRL（技术就绪水平）—— 技术成熟度九级刻度：原理样机演示落在中间几级、真实环境运行落在最高两级，“样机通过不等于产品就绪”的工程刻度
- [2] **R-37** ISO/IEC/IEEE 15288:2023 —— §6.4.2 利益相关方需要与需求定义、§6.4.3 系统需求定义（需求过程锚点，第 4 章已引）
- [3] **R-38** ISO 9000:2026（第 5 版）—— §3.5.1 需求（注 2：规定的要求是明示的要求）、§3.5.5 法定要求、§3.5.8 法规要求、§3.5.9 合格、§3.3.11 设计和开发、§3.11.12 验证、§3.11.14 确认（注 3：使用条件可真实可模拟）

**经典文献**

- [4] **R-82** MRL（制造就绪水平）—— 生产就绪是另一把尺子（与 TRL 对应：TRL 管技术成熟度、MRL 管制造就绪）
- [5] **R-83** COTS（货架商用产品）—— 组织资产复用的一条通路（§5.4.4 那句“优先选用标准外购件”）

> 对照阅读：本章“内容与载体”呼应第 3 章“需求、设计开发、质量”与第 4 章“需求工程”，“从症状反推”是第 1 章反事实检验在需求侧的动作，验证与确认的界分留给第 8 章。

# 第 6 章 设计开发：需求与方案结对

## 6.1 引言

第 5 章把一份需求立全了：功能需求、环境适应性需求、可实现性需求，各有各的代言人，各有各的专属验证载体。立全之后要回答的是下一个问题：这份需求，怎么被用来造东西。

本章看的就是这一段，设计开发。一句话说清它的样子：一份需求配一份方案，结成一对。一对之内先定规则，再走三个动作：分配生成、分析迭代、分解分析。三个动作的活，是把每一条需求落到一个能负责它的节点上。上一对交出来的东西，就是下一对的输入，一对一对往下接。

### 6.1.1 误解现场

恒信集团合同审批系统项目评审会，下午两点，会议室坐满了人。

系统架构师罗建在投影上打开了《合同审批系统总体设计方案》，六十页，他花两个月写成，评审会前夜还在改格式。他清了清嗓子：“这份方案覆盖了需求调研的全部三百条需求，下面我按章节过一遍——”

需求分析师顾言问了一句：“它跟需求文档是什么关系？”罗建：“设计文档嘛，就是把需求写详细一点。”

法务部的人翻开打印稿，翻了十几页，打断他：“审批链上，谁能驳回、谁不能？”

罗建翻到那一页：“在这里，第 23 页的流程图。”顾言追问：“那这条需求，落在方案哪个节点上？”

罗建：“……图上没有它。”

财务的人翻到第 31 页：“月底对账的统计口径，合同 25 号之后签的算哪个月？”

罗建：“这个……在需求文档里写过。”

业务部的同事问：“我们上线之后，发起一份合同，先点哪里？”

罗建：“……界面上。”

运维的王默补了一句：“那它部署在哪、监控什么、怎么回滚？”

罗建翻了翻稿子，没翻到：“……这些后面会写。”

王默追问：“那我上线要部署几样、监控什么，稿子里数得出来吗？”

产品开发经理曹峰坐在后排，把六十页掂了掂：“两个月、六十页、三百条需求，都在了。可我们要的是一份'评审完就能开工'的方案，不是一本要通读的书。”

会议室安静下来。顾言坐在角落，把六十页从头翻到尾，又翻回第一页，轻声说了一句：

“罗建，这不是一篇方案。这是一本小说。”

“方案不该是一篇文章。”顾言把打印稿合上，“一篇文章是线性的，只能从头读到尾。可你的方案有五类读者——法务要看审批规则、财务要看校验和台账、业务要看怎么发起、开发要知道模块边界、运维要知道怎么部署。五个人的问题分散在六十页里，谁也找不到自己那一枝。”

罗建不服：“那这三百条需求，该往哪儿放？”

顾言说：“写成结构。三百条需求，一条一条落到该负责它的地方去——写成一篇文章，就没有地方可落。”

罗建没接话。他忽然意识到，自己两个月里做的每一件事都对：访谈做了、需求收了、方案写了、格式改了。可这份方案没有一个位置是给“这条需求由谁负责”准备的，它只有页码。

### 6.1.2 本章要回答什么

读完本章，你应该能回答：

1. 设计开发的对象为什么是“一对”文档，而不是一份？什么叫“由需求开发出方案、并由方案迭代需求”？
2. 记成一次 Y=F(X) 的求值，X、Y、F 各是什么？为什么说“单承接”就是函数的单值性？
3. 设计文档为什么必须能定位、能承接、能追溯？产品数据凭什么可以条目化、可以成树、可以按 N:1 分配？
4. 框架定义与详细定义各管什么？同一棵树为什么有框架、概要、定义三个形态，“最深形态”又指什么？
5. 为什么框架定义不是架构定义？
6. 每一对开工之前为什么先要定规则？三套规则各管什么，分解规则为什么必须带着消费方的语义范围来定？
7. 分配生成、分析迭代、分解分析这三个动作各在做什么，各自靠什么判据收敛？
8. 一条需求为什么只能归一个节点？违规了分几种情形、各回哪一步修？
9. 链上相邻两份之间，哪些步在承接、哪些步不做映射？链尾那一刀交给谁？

### 6.1.3 本章的骨架

本章要立的是“需求-方案”这一对。先把骨架立起来，后面三块，都是给它添东西。

> 设计开发的单元是一对“需求-方案”：上一份的叶节点，就是下一份的需求。一份需求配一份方案，一次 Y=F(X) 的求值（X 是需求，Y 是方案，F 是三套规则）。每份文档都是一棵树：干枝是框架定义，叶节点是详细定义；树按定义到多深，分框架、概要、定义三个形态。一对之内，先定规则，再走分配生成、分析迭代、分解分析三个动作。一条链上逐对接力，链尾把最后一刀交给构件设计方。

**一对**——设计开发的单元是一份需求文档配一份方案文档，有需求必有方案。结对是双向的：正向由需求开发出方案，反向由方案迭代需求。记成函数，就是一次 Y=F(X) 的求值（§6.2.1 展开）。

**一棵树**——每份文档都是一棵树，不是一篇文章：干枝管结构、叶节点管内容。同一棵树按定义到多深，分框架、概要、定义三个形态。最深的那个形态，就是这一对实际交出去的东西（§6.2.4、§6.2.5 展开）。

先定规则，再走三个动作，每一对开工先把 F 立起来：分解、构建、承接三套规则由人类主导制定。然后走三个动作。**分配生成**把需求叶节点落到下游能负责它的节点上，长出方案骨架。分析迭代守住“一条需求只归一个节点”，违规就回炉重切需求。分解分析让节点长出下层的叶节点，并检查“接住的需求一定被实现”（§6.3 展开）。

一条链——链上相邻两份之间，上一份的叶节点就是下一份的需求；一份一份往下接，追溯边自然生成。链尾那一份只到框架，把最后一刀交给构件设计方，执行换手，概念不分（§6.4 展开）。

本章的方法时刻，是第 1 章概念能力里的织关系：方案是结构，不是堆，把条目织成树、让每个部件各就其位彼此承接，而不是排成一列摞起来。

## 6.2 先把设计开发的对象看清

> 这一块把设计开发的对象看清：先立“一对”——一次 Y=F(X) 的求值；再看设计文档为什么必须能定位、承接、追溯；再推产品数据凭什么能条目化、能成树、能按 N:1 分配；然后把干枝与叶节点、树的三个形态、框架定义与架构的分界、树装不下的内容一一分开。

### 6.2.1 设计开发的单元是一对

先回答一个问题：设计开发的对象是什么？不是一份文档，是一对——一份需求文档，配一份方案文档。

有需求必有方案，有方案必有需求：需求说“要什么”，方案说“怎么做”。上一层的需求，靠下一层的方案承接；而这份方案接住了需求，转过身又成为更下一层的需求。中间每一层，既是方案，又是需求（§6.4.1 展开）。

结对是双向的。正向，由需求开发出方案，按规则把需求一步步转换成方案；反向，由方案迭代需求，方案做下去发现需求切得不合用，就回头重切需求、调整承接（§6.3.4）。正反交替，直到这一对收敛：需求是合规的需求，方案是合规的方案。

这个“由 X 按规则得到 Y”的过程，记成一个函数最清楚：**Y=F(X)**。X 是需求，Y 是方案，F 是三套规则，需求侧的树形结构与节点规则、方案侧的树形结构与节点规则、方案节点承接需求叶节点的承接规则。F 由每一对开工时先定（§6.3.1）。

函数有一条铁律：**必须单值**。多个 X 可以映射同一个 Y，多条需求叶节点落到同一个节点上，没问题；一个 X 映射多个 Y，一条需求叶节点归两个节点，F 就不成其为函数。单承接，就是函数的单值性（§6.3.4 磨开）。

> Y=F(X)：由符合规则的 X，按函数规则生成符合规则的 Y。
>
> 术语注：这一记法是本书对设计开发落地方法的凝练命名，非标准条文——欢迎检验、欢迎校准。

所以本章说“一棵树”时，先记住：树是对一份文档而言的。需求文档是一棵树，方案文档也是一棵树，一对“需求-方案”，就是两棵树。这一块先看清“每份文档都是一棵树”；下一块的三个动作，正是这两棵树之间的事。

### 6.2.2 设计文档要能定位、承接、追溯

现在回到罗建那份六十页。它为什么失败？不是他写得不好，他写得太像文章了。一篇文章有三样东西，恰好是设计文档不能缺的。

第一是不能定位。一篇文章是线性流动的，从第 2 页读到第 60 页。但“审批驳回的规则”在第几页？“对账的统计口径”在第几页？读者只能从头找起。

而方案从来不是给一个人从头读到尾的。方案是给五类人分别“查询”的：法务只找审批规则，财务只看校验与台账，开发只管模块边界。一篇文章逼着五类人都通读全文，白费功夫；更糟的是通读之后还是找不到，于是开始猜。评审会开到第四十分钟还在争“这条需求到底有没有被方案覆盖”，不是大家不认真，是文章没有给“覆盖”一个可定位的位置。

第二是不能承接。下游开发拿到六十页文章，怎么开工？他需要三件事：负责哪一块、要满足哪些需求、做到什么程度算完成。文章没有模块边界，开发只能“全文通读，然后自己揣摩”，各揣出各的边界，集成必然撞车。文章没有叶节点，需求没有地方落。

第三是不能追溯。需求是会变的（第 4 章刚讲完）。一条需求变了，影响方案的哪几页？文章时代只能重读全文、靠人肉找相关性，找不全就漏改。

三样合起来，其实是一件事：文章里没有“位置”。没有位置，读者定不了位，开发接不了手，变更时找不到该改的地方。

树的回答恰好是三件事：

| 部分 | 是什么 | 干什么 |
| --- | --- | --- |
| **根与层级** | 从根节点到枝梢节点的完整分级 | 层层细化，同层保持同一抽象级别 |
| **分类节点** | 非枝梢节点（如“审批业务域”“平台服务”） | 组织层级——像文件夹，本身不含实体内容 |
| **枝梢节点** | 框架骨架的终点 | 承接需求——需求落在这个节点上（一个节点可落多条） |

一篇文章与一棵树，差别不在长短，在有没有叶节点：

- 文章里，“审批驳回”是正文里的一个段落；树里，它是“审批流程”这个枝梢节点上挂着的一条需求。
- 文章的段落不能被“承接”，你不能说“这一段归张三做”；树的枝梢节点可以，枝梢节点是需求落脚的钩子，也是开发认领的任务单。
- 文章的段落不能被“追溯”，需求变了你不知道哪段要改；树的枝梢节点可以，沿着“需求→枝梢节点”这条边，一步命中。

所以树的关键，是枝梢节点与叶节点怎么咬合。它有两个判据，一个管“落”，一个管“搬”：

> 枝梢节点 = 需求落脚的主要落脚点（下游还能负责这条语义的节点，见 §6.3.3）——一个枝梢节点可以落多条需求（N:1）；

> **落点以语义定级**。叶节点（详细条目）= 能一对一搬走的原子条目——一个叶节点只能属于下一棵树的一个节点。
>
> 术语注：“干枝管结构、叶节点管内容”是本书（通用设计规范）的落地操作，非标准条文；**本章各术语的英译均为本书自拟，非标准术语**。后记四角色清单里，框架定义／详细定义这对文档为模块级（模块架构师／模块工程师），系统级对应物为系统架构师的架构描述。

“一对一”是本章最要紧的一个约束，先记住它，第二块会展开：一个叶节点，只能被搬到下一棵树的唯一一个节点上。

最后是最值得问的一句：既然树这么好，为什么大家天然写文章？因为写作是线性的，一支笔顺着思路走就行。而一棵树要有层级、要让每条需求落到枝梢节点、要让叶节点长出来，这需要方法。大多数团队没有这个方法，方案就退化成文章。翻一翻公司里任何一份“总体设计”，十有八九是按章节排列的文章，不是大家不愿意写树，是没人教过这套方法。

### 6.2.3 产品数据凭什么能成树

上一节回答“文章为什么不行”，是从读者角度看的。还有个更根本的问题：“树”为什么就行？树能成，是拍脑袋定的吗？不是，它底下有三条依据，每条都有出处。

依据一：产品数据可以被条目化。设计的每一步产物都要能被独立判定“是否被满足”，而通行的需求书写标准正是这样要求单条需求的：单数、可验证、无歧义。

出处是 ISO/IEC/IEEE 29148 对良构需求的要求；邻近的规范表述在 ISO/IEC/IEEE 15288:2023 §6.4.2.3 注 26 与 §6.4.3.3 注 15。其中“单数”一名就是“一条只讲一件事”，“唯一解释”是本书的读法。

于是最小的判定单元是条目：一条条目能独立定义、独立验证、独立分配、独立追溯。文章里没有这个最小单元，段落不能被单独验证，也不能被单独交出去。

依据二：产品数据可以被树形化。因为分层是复杂系统的常态结构：复杂系统研究里讲层级与近可分解性，见 H. A. Simon《The Sciences of the Artificial》1996 第 7 章。工程上也早有分解纪律，即“百分之百规则”：互斥且完备，本书转述。但要注意，“能组树”不是自动的，同一层的分组必须同时满足三条，否则得到的不是树：

| # | 条件 | 不满足则 |
|---|---|---|
| 一 | 判据单维——同一层只用一个判据分组（如“系统职责”），不得混用两个（如职责与业务域并用） | 同层出现交叉分类，两个判据交叉切出的是一张格，不是树 |
| 二 | 条目单归属——每条只归一个父节点 | 得到的是有向无环图，不是树 |
| 三 | 分组完备互斥——子层完整覆盖父层，且互不重叠 | 出现缺口或重叠，不是合格的树 |

依据三：产品数据可以由需求设计方案，按 N:1 分配。因为设计被建模为从需求域到方案域的映射：公理化设计讲的 FR 与 DP 映射，见 N. P. Suh《Axiomatic Design》2001。

这条映射的形状，由两端与一条约束定下来：定义域取需求侧的原子（叶节点）。粗粒度节点内部含着多条性质不同的需求，交出去后没法判定谁负责哪一部分。值域取方案侧当前能负责的实体。映射必须是函数：允许多对一，一个实体承担一族需求，这是内聚。不允许一对多：一条需求散在多个实体，那是变更传播与重复实现。

两侧的理由还不同源：定义域取原子，是为了可判定；值域取当前实体，是因为这一刻方案侧的叶节点还没产出。

> 产品数据能条目化、能成树、能按 N:1 分配——三条依据合起来，才撑得起一套可执行的生成方法。
>
> 术语注：三条依据引自通用设计规范与 EOS 产品数据分解结构（见章末参考文献与库内路径），本书只做转述与举例。

三条依据还有一层用处。它们把“相邻两份产品数据之间的步”分成三类：分配步、整段搬入步、形态轴步（本章叫它**链上三类步**），是第三块要用的分界（§6.4.2）。

条目化之后，还有一件事必须回答：条目一多，怎么组织？真实系统几百上千条，平铺着放不行：人脑装不下，需要“先看结构、再进细节”；一条需求变了、影响哪些条目，要聚类才能定位。要按领域把条目分给不同团队，也得先有分组。所以条目必须被组织。

而组织成什么形态?这里藏着树最深的理由，为什么偏偏是树，不是列表、不是表格、不是网状?

> 因为树是唯一能让“导航路径”与“承接路径”合一的形态。

列表没有层次，一多就应付不了。表格能表达“谁承接谁”，但那是映射表，不是内容组织。网状能表达复杂关系，但需求必须“从根落到叶节点”，网状没有唯一根，答不了“这条需求归谁”。只有树，从根到叶节点有唯一路径：既是读者找路的导航，也是需求落地的承接。顺着树走一遍，就是一次完整的承接验证。

这个“合一”，就是第 1 章概念能力里的织关系：把散点织成结构，让导航与承接走同一条路。设计做的是把三百条需求织成一棵树，每条需求有唯一的位置。方案是结构，不是堆：堆是线性排列，结构是有承接关系的网。

### 6.2.4 框架定义管结构，详细定义管内容

树组织好了，还差最后一步：细到什么程度？细到每条需求都被接住、每个叶节点都可验证，可整棵树都细到叶节点，你就看不见树了。管理与理解需要先看骨架（结构、分类、承接关系）；落实与验证需要逐个看叶节点（定义、指标、验收标准）。

两种抽象级别、两种目的、两种变更频率，于是树天然分成两大部分：**干枝**与**叶节点**——这不是谁规定的，是“既要看懂、又要做实”这两件事逼出来的。它们各有各的名字：

| 部分 | 叫什么 | 管什么 | 回答什么问题 |
| --- | --- | --- | --- |
| 干枝（结构） | 框架定义 | 组织与承接 | “结构是什么、承接了谁、往下分解出什么” |
| 叶节点（内容） | 详细定义 | 定义与验证 | “每个模块具体做什么、做到什么程度” |

框架定义就是树的干枝：从根节点一路分叉到枝梢节点。每个枝梢节点回答“结构是什么、承接了谁”：系统被组织成哪几大块，每个枝梢节点上要落哪些需求。

详细定义就是树的叶节点：长在枝梢节点上的每一条，回答“每个模块具体做什么”。一条可独立定义、可独立验证的条目，有编号、需求描述、性能指标、验收标准、映射目标。

最关键的一点：叶节点不是漂着的，它长在枝梢节点上。每个叶节点都明确属于某个枝梢节点。“顺序审批”这个叶节点长在“审批流程”这个枝梢节点上。树正是用“叶节点属于哪根枝”这件事，把结构与内容连成一个整体。

这也回答了“为什么缺一不可”：干枝上没有叶节点是枯树，叶节点没有干枝是落叶。如果只有干枝，树就是空骨架，枝梢节点上没有

内容；如果只有叶节点，叶节点就散落一地。一棵好树：干枝挂得住，叶节点长得出。

画出来，就是一棵实实在在的树，干枝部分，是框架定义：

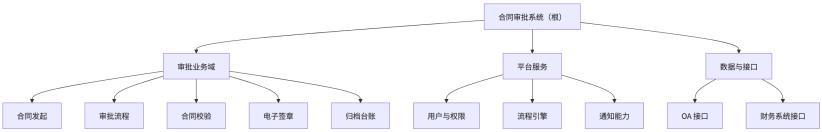



图 6-1

中间层的“审批业务域”“平台服务”是**分类节点**（干枝的分叉、负责组织的文件夹）；“合同发起”“审批流程”这些枝梢节点是框架树的终端。每个枝梢节点上长着叶节点，叶节点就是详细定义：

「审批流程」这个枝梢节点上，长着四条叶节点：

- SYS-002.01　顺序审批　　支持按法务→财务→业务顺序审批，审批后 5 分钟内进入下一节点
- SYS-002.02　驳回回退　　支持驳回，驳回后回到发起人并可修改重提，响应 < 1s
- SYS-002.03　会签　　支持多方同时审批，会签发起后同步通知各方
- SYS-002.04　超时提醒　　审批超过 48 小时未处理自动提醒，提醒后 24 小时内升级

注意这个例子里已经藏着三个动作的影子：“审批流程”这四条叶节点，是一把分解的刀切出来的。每条叶节点都要落到下一棵树的一个节点上，落靠在分配生成时对账（§6.3.3）。干枝与叶节点这两个部分，正是为这些动作而生的。

> 概念卡片 · 框架定义

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 设计树的干枝：从根到枝梢节点的树形结构 + 映射表 |
| **它不是什么** | ⑥⑦ | 对立＝详细定义（叶节点）——干枝组织与承接，叶节点定义与验证，合起来才是一棵完整的树；辨析＝框架定义 ≠ 架构（见 §6.2.6） |
| **它从哪来** | ②③⑨ | 从结构骨架与内容条目分层来：设计文档先立骨架、再填条目；为拆掉“架构＝文档”的词汇病定名（见 §6.2.6）；英文 preliminary definition，preliminary ← 拉丁 praeliminaris（门槛之前） |
| **它往哪去** | ④⑤⑧ | 没有它，方案不可整体看懂、需求没有地方落；在设计树结构搭建、需求落靠定位时用；误区 ✗“框架定义就是画一张系统结构图”——画图只是第一半，还必须有映射表说明每条叶节点承接了谁 |
| **判据** | 落枝问 | 任取一条需求，能说出它落在哪个节点上吗？说不出的，这棵树还没长全。失效边界：链首的需求侧是存量材料（开工时直接结构化成树）、链尾的方案侧止于框架（叶节点归构件设计方）——这两处退化成最简形态 |

概念卡片 · 详细定义

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 设计树的叶节点：长在枝梢节点下的条目列表，每条可独立定义、独立验证 |
| **它不是什么** | ⑥⑦ | 对立＝框架定义（干枝）——叶节点给干枝填内容，干枝给叶节点定位；辨析＝详细定义 ≠ 需求文档——它既写“是什么”也写“做到什么程度”（指标＋验收标准），是设计内容的最终落点 |
| **它从哪来** | ②③⑨ | 与框架定义同源孪生：把“可验证、可落实”的最小单元独立成层（见 §6.2.3 条目化）；“详细”之名与“框架”对称而成；英文 detailed definition，detail ← 拉丁 talis（如此） |
| **它往哪去** | ④⑤⑧ | 它回答“每个模块具体做什么、做到什么程度”——可逐条验证、可一对一搬走；在逐条定义、验收核对时用；误区 ✗“叶节点越多越详细越好”——叶节点必须长在枝梢节点上，散落的条目只是段落 |
| **判据** | 五件套 | 每条叶节点都有编号、需求描述、性能指标、验收标准、映射目标吗？缺一，叶节点还不完整 |

## 6.2.5 同一棵树，停在哪个深度

同一棵树，可以停在不同的深度上。按定义到多深，它有三个形态：

| 形态 | 包含什么 |
| --- | --- |
| **框架**（xx 框架） | 根 → 枝梢节点；叶节点只以枝梢节点的展开项露名字 |
| **概要**（xx 概要） | 根 → 叶节点，各级节点只到说明 |
| **定义**（xx 定义） | 概要 + 各节点的详细定义 |

同一棵树上，层名分三级：**干枝**（叶以外的各级）、**枝梢节点**（这一次框架的最深一级）、**叶节点**（树的末级）。本章下文单说“节点”时，指下游能负责这条语义的那个节点（可能是干枝，也可能是枝梢，§6.3.3 展开）。

三个形态由三级设计动作产出：框架设计出框架，概要设计出概要，详细定义出定义。相邻两级可以合做，一个设计动作一次交出两级；也可以按需省略或裁剪，某份产品数据不必每一级都停。

这一对实际交出去的东西，是各自最深的那个形态。本书因此不用“框架与完整态”这种两分说法：说“完整态”会把“概要”这一档挤没。它既不等于框架，也不等于定义，而是“根到叶节点、只到说明”的独立一档。“最深形态”才是这一对两端的正式说法。

有两处细节，越往后越要紧，先在这里立住：

- 框架有阶：枝梢取哪一级，由这一次框架设计定；取得越浅，框架的阶越高。“枝梢”始终指这一次框架的最深一级，不是一个固定的树位，同一份业务流程产品数据，可以给出构件级的高阶框架（枝梢＝产品构件），也可以给出文档级的高阶框架（枝梢＝文档级端到端任务），每一阶都是一份合法的框架。因为枝梢是承接的落点（§6.3.3），框架的阶越高，这一对承接时下游能提供的落点越浅。
- 叶节点有三种身份：在框架里，它只是枝梢节点上的一条展开项（名称级）；到概要，它才成为树节点（说明级）；到定义，它才带着详细定义。

还有一条随之而来的规矩：分解源判据附着于这一次形态的最深一级。框架停在哪一阶，判据就挂哪一级；到了概要，叶节点已是树节点，判据的内容就是叶节点本身，不再另设展开清单。

最后一点，叶不只是“最深的那一级”，它有判据：叶节点必须切到可被独立解释、可独立验证、可被唯一交出去的粒度。所以“某棵树到哪一级为止”是一个判断——不是一个位置约定。

### 6.2.6 框架定义不是架构定义

有人会问：在很多方法论文档里，这份树形文档常被叫作**架构定义**（architecture definition）。一棵树的形状，不就是架构吗？

按 ISO/IEC/IEEE 42010:2022，这项工作的规范名字是 **§3.3 架构描述**，即“用于表达架构的工作产品”；“架构定义”是习惯叫法。

正是这个“听着顺耳”，是个词汇病。这一节要竖起的又一个反直觉命题，就在这里：

> 框架定义 ≠ 架构定义。

三个理由，每一个都值得想清楚。

理由一：与“架构”撞车。“架构”（architecture）在全书极其重量级。第 10 章会依 ISO/IEC/IEEE 42010:2022 **§3.2** 给它一个精确的定义（“实体在其环境中的基本概念或属性，以及为实现和演化该实体及其相关生存周期过程的管控原则”）；本书把这个定义读作系统不可还原的涵义——**关于系统根本性质的那组基本概念或属性。管控原则是与它并列的第二半**，不在“涵义”之内。“决定”只是认出并记下它的抓手（第 10 章 §10.2.2 术语注）。

按 ISO/IEC/IEEE 15288:2023 §3.13，设计是“系统元素及其关系的规定，其完整程度足以支撑架构的合规实现”。**架构约束的是设**

**计的空间，不是替它挑做法**——管住下游的那一半是**管控原则**：它收窄取舍的范围，靠相容性把不相容的做法排掉，而不是指定一个做法（第 10 章 §10.3.4）。框架定义是一份文档，架构是系统本身的那组**基本概念或属性与管控原则**；**“决定”是记录（ADR），不是架构本身**。

镜照一眼冰链，同一个病，换了世界。冰链把 M3 的方案结构文档直接叫“架构定义”，这名字一听就重：架构都“定义”了，还动什么？

于是人人都以为架构已定、画完即可，文档再没人维护。可那棵树天天在长：加一点需求、改一条逻辑，它就得跟着长。等第 10 章把“架构”说清楚，嵌入式硬件工程师石军才明白：被他们叫“架构定义”的只是一棵还没长成的树。真正的架构（**那组“去掉就不是 M3”的基本概念或属性，以及与之并列的管控原则**）一直活着，只是从没人写下来。

用已经承担重量级含义的词，去命名轻量级文档，是给自己埋词汇病的坑。

理由二：不对称。它的孪生兄弟叫“详细定义”。既然另一半叫“详细”，按对称，这一半就该叫“框架”。叫“架构定义”，反而跟这对名字自己的对称关系打架。

理由三：名实相符。它回答“结构是什么、承接了谁”，是设计的结构骨架——比架构轻：初步、框架，用于组织与承接。真正“不可还原的涵义”确实藏在树的枝条走向里，但那是从树里读出来的判断，不是树本身。第 10、11、12 章会专门去挖。

> 框架定义是目录，详细定义是正文，架构是这本书真正要表达的思想。目录不是思想，但好的目录帮你定位思想。

不过，把“不等于”说死之前，得分清两层。

文档层面，框架定义不等于架构：框架定义是一份设计文档（结构骨架＋映射表），架构是系统不可还原的涵义（第 10 章），两者不在一个层面。

但在“把系统结构写下来”的描述层面，框架定义可以作为架构的一种描述——**它描述的是架构“可以关乎”的结构面（42010:2022 §5.2.2），同时它本身是一份设计定义文档**（第 7 章 §7.2.4：AD 与 DD 平级）。两层不打架：不等于，是在文档与涵义之间划线；可作描述，是在描述与涵义之间搭桥。

所以本章里，我们统一用框架定义这个名字。它比“架构定义”少一分误导，多一分老实。

罗建后来问顾言：“为什么不叫架构定义?”顾言反问：“你那六十页'总体设计方案'，叫不叫架构?”罗建哑然。顾言：“所以名字上就要分开，文档是文档，架构是架构。”

### 6.2.7 树装不下的内容

本方法有一个基础假设：需求或方案类产品数据都能组织成树形结构。这个假设成立，但依据是组织性的，不是内容性的：说“组织成树”，只保证每条内容有一个唯一的归属位置；不保证内容之间只有父子关系。

需求与方案的语义本质是图，有四类内容，纯树放不下：

| 内容类型 | 现象 | 树形处理 |
| --- | --- | --- |
| **横切** | 一条约束影响多个分支（“可用性 ≥ 99.9%”“所有操作需审计留痕”） | 给它独立的树位置（如非功能分支），用影响面标注勾连被影响的节点 |

| 内容类型 | 现象 | 树形处理 |
| --- | --- | --- |
| **共享** | 一个定义被多处引用（“客户”贯穿多个流程、接口契约被多个组件消费） | 归属到一处，其余引用——跨文档只记引用、不复制正文 |
| **多维** | 同一批需求可按功能／场景／性能／利益相关方多个维度切分 | 选一个稳定的主组织维度作树的轴，其余维度降为节点属性 |
| **循环／递归** | 业务流程自调用、嵌套，调用关系成环 | 树按归属组织层级，调用关系记在节点内容里（递归：每一级元素的设计开发都是一次完整的设计开发） |

由此读出本方法对“树”的读法：树是存储骨架，引用与追溯是关系载体。承接因此分两种并存。**归属式承接**是功能性分配，走单承接约束。**横切式承接**管非功能、约束与安全策略：留在独立分支作叶节点，用影响面标注表达对多个方案节点的承接，不走归属分配。

假设在两种情况下不成立。一是要求树同时表达全部关系，把横切、共享也塞进分支结构；解法已给出：横切走独立分支加标注。二是选不出稳定的主组织维度，树组织变成任意的，内容一增长就会反复重构。后者的解法是：每一对开工前先定死主组织维度（对应下一块的“每一对先定规则”）。

还有一层边界要提前说清。上面讨论的是“哪些内容树装不下”，另有一层是“哪些东西根本上不了链”：**确认的判据**（预期用途、使用情境，主体是隐含需求）与方针不在链上。因为本链按派生关系生成，

只承载明示的条目。这决定了这条链能支撑评审与验证，支撑不了确认（§6.4.3 收口）。

## 案例进展①｜六十页文章变成一棵树

评审会散了之后，罗建回去熬了一夜。

他把六十页文章揉碎了，提炼成一页树。合同审批系统分出三个枝：审批业务域、平台服务、数据与接口。三个枝上长出十个枝梢节点：审批业务域下是发起、流程、校验、签章、归档；平台服务下是权限、流程引擎、通知能力；数据与接口下是 OA、财务接口。

第二天他把这一页树拿给顾言看。顾言点头：“好多了。现在你告诉我，法务关心的审批规则，在哪个枝上?”

罗建指向“审批流程”那个枝梢节点。

“那这个枝梢节点上，落了哪几条需求?”

罗建愣住了。树还只是一张图，枝梢节点上还没有叶节点。

“这就对了。”顾言说，“树还只是骨架。要变成方案，得把三百条需求一条一条落到枝头上——落得准、落得全、落得能验证。”

“怎么落，先立骨架还是先切叶节点?”

“先定规则，再一个动作一个动作来。”顾言竖起一根手指，“第一个动作是分配生成——把三百条需求落到这十个节点上。先学落，再学查，最后学长叶节点。”

---

# 6.3 先立规矩，再磨动作

> 这一块先立规矩、再磨动作：每一对开工先定规则——三套规则由人类主导制定，分解规则必须带着消费方的语义范围来定。然后三个动作各靠反例定型：分配生成不是“简单分派”、分析迭代不是“返工丢人”、分解分析不是“多列几条”更不是“空话”；最后说清干枝与叶节点为什么不焊死在一起。

## 6.3.1 每一对先定规则

上一块的树，回答了“设计文档长什么样”。现在的问题更根本：树是怎么长出来的？答案是先定规则，再走下面三个动作。

三个动作不是三个先后的大阶段，而是每一对“需求-方案”都要走的功课，各有各的发生地：

| 动作 | 在哪儿做 | 做什么 | 自检 |
| --- | --- | --- | --- |
| **分配生成** | 需求叶节点 → 方案框架（两棵树之间，加树上） | 叶节点落到节点上（复用 → 改进 → 新增），新节点自下而上调树 | 三自检＋收敛判据 |
| **分析迭代** | 两棵树之间（查），加回需求侧（迭代） | 检查分配与生成的正确性：单承接成立吗、承载量合适吗、改进波及谁——违规就迭代需求分解 | 违规两分类处理 |
| **分解分析** | 方案树上（节点 → 叶节点） | 节点按分解规则分解为方案叶节点，再做承接覆盖检查 | 语义四查三场景＋覆盖公式 |

但在这三个动作之前，还有一件每对必做的事：**定规则**（分解、构建、承接三套）。规则不是从天上掉下来的通用教条，是这一对自己的本钱：没有它，三个动作没有可执行的依据。

规则由谁制定？人类主导，AI 辅助。定义各级节点的物理意义与父子关系、选定主组织维度、制定本对的具体规则，这些是判断性工作，不存在机械算法。语义归纳、边界界定的制定权在人。AI 承担三类辅助：候选生成、一致性检查、反例检验；并按规则执行设计动作与机械核对。父节点的生成是最典型的一例：只有人类制定的语义与质量判据下的检查，没有机械的生成算法。

三套规则按动作对象分三块：

| 规则块 | 内容 |
| --- | --- |
| **需求侧规则** | 需求侧的分解规则与构建规则（首对在这一步制定；链上各对沿用上一对——本对的需求树就是上一对的方案树） |
| **方案侧规则** | 方案侧的构建规则与分解规则（每对新立） |
| **承接规则** | 需求叶节点 → 方案节点（每对新立） |

三套执行规则各有一条机械判据：

| 执行规则 | 管什么 | 机械判据／收敛判据 |
| --- | --- | --- |
| **分解规则** | 节点 → 叶节点（两处现身：本对方案侧在分解分析中产出；需求侧叶节点是上一对同规则的产出） | 单承接 + 语义原子性（防止切得过碎） |

| | | |
|---|---|---|
| 执行规则 | 管什么 | 机械判据／收敛判据 |
| **构建规则** | 子节点 → 父节点（自下而上调树的依据） | 覆盖完整 + 兄弟互斥（收敛）；父语义三查：独立可定义、非兜底、最小覆盖 |
| **承接规则** | 需求叶节点 → 方案节点（分配依据） | 单承接（可 N:1、不可 1:N）+ 承载量约束（超限触发节点拆分） |

分解规则怎么定，是最要紧的一条：制定时必须把消费方的语义范围带进来。叶节点长出来是给下游用的，消费方就是承接、使用这些叶节点的一方。四条考量：

- 消费方的语义范围：把消费方承接结构的节点物理意义及其特征作为制定输入，切点落在其语义范围的边界上，叶节点天然可被单点承接。单承接由分解规则直接保证，而不是迭代试错的目标。本对内，需求的消费方是方案框架，方案的消费方是下一对的承接结构（§6.4.1）。
- 语义原子性：叶节点保有独立语义、可独立验证。它的作用是防止切得过碎：不为迁就消费方的切法把语义整体切碎。
- 切不动不硬切：消费方语义范围切不动的语义整体（一致性约束、接口语义等关系性内容）走两个出口，节点拆分、横切式承接（§6.2.7）。

- 随消费方结构演进：分配生成中消费方的节点被新增或改进时，受影响的分解规则随影响面复查同步修订，“先定”指对当前消费方结构的首次制定，不是一劳永逸。

构建规则再补两句。父节点的语义按三条质量判据自查：**独立可定义**——父语义能独立于子节点清单表述，回答“这一组共同是什么”，不是清单的拼接；**非兜底**——“其他／杂项”式语义筐不合格。没有独立语义范围的父节点，会在后续承接中变成语义黑洞。最小覆盖——选能覆盖子节点语义之并的最小类，只留生长余量、不留模糊空间。

收敛判据两条机械可查：覆盖完整（每个非叶节点语义包含其子节点语义之并）＋兄弟互斥（同层兄弟两两不重叠），两者满足即收敛。

定规则还有一层容易被忽略的分工：

| | 切法（内容规则） | 判据（形式判据） |
|---|---|---|
| 回答的问题 | 按什么维度切 | 切得好不好 |
| 典型例子 | 按利益相关方／按流程／按能力／按职责拆 | 语义四查、三场景、单承接、覆盖公式 |
| 性质 | 领域经验 | 逻辑要求 |
| 适用范围 | 结对特有（每对结对不同） | 全链通用（所有结对相同） |
| 该不该统一 | 不该——必须业务化 | 必须——不统一则对账失败 |

判据必须统一：分配要做对账，需要一套所有结对都认的账本语言，A 结对送来的叶节点，在 B 结对的下层能对上账。切法必须业务化：业务专家直接参与分解规则制定与评审，每次退回重切的裁决都沉淀成新的切法经验，切法指南是项目自己的记忆，越用越准。

还有两个把“切”与“落”接上的装置。一是候选声明：每个叶节点在切出时声明一个候选节点，这份声明是给分配的输入，不是切法的验收。二是骨架先行：对着已约好的下层节点清单切，候选声明才有节点可指。

> 定规则 = 人类主导（AI 辅助）× 切法业务化（结对独有）× 判据统一（全链共用）× 契约共享（候选声明 + 骨架先行）。
>
> 术语注：这套“切法／判据／契约”三分，是本书对通用设计规范“分解分配解耦”的凝练命名，非标准条文——欢迎检验、欢迎校准。

除了解耦，还有组织级约定：术语表、命名规则、验收模板、反模式案例库。判据全链通用、切法结对独有，但都要落到组织的本地语言，统一叫法、统一格式，否则各对写出的叶节点长得不一样，全链照样对不齐。

### 6.3.2 先做准备，再进循环

规则定完，进入干活。干活分两段：正式设计之前有一段准备，然后是**正式设计的循环**。

准备这一段做三件事。一是定规则，每一对必做（§6.3.1）。二是把存量材料结构化成树；入口材料已经是最深形态的，直通。三是处理可以直接承接的需求叶节点，准备阶段只做直接承接，不动已有的方案结构；需要改进或新增才能接住的，留给正式设计。

准备的第三件事也要过同一道关口：承接落定后，对被承接的节点执行承接覆盖检查（§6.3.6），没通过的叶节点不算直接承接，退回未处理。处理完能直接承接的，其余全部作为未处理的需求叶节点进入循环。

正式设计是一个叶节点驱动的循环，直到全部需求叶节点处理完。一轮走五步（走完一对是六步，见 §6.4.4）：

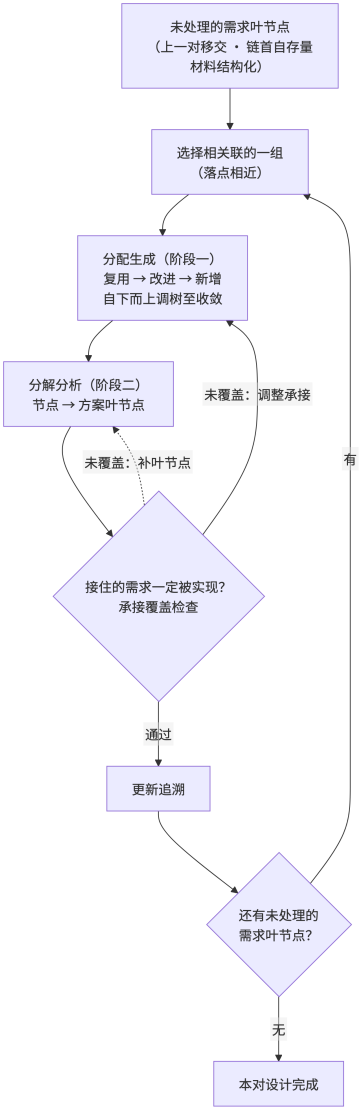



图 6-2

1. 选择待设计的一组：预计由同一个（或同一小簇）节点承载的叶节点归为一组，判据方向是落点相近，可查的信号是同一业务场景、同一功能主题、内容互相牵连；拆开设计会导致重复承接或互相等待。
2. 分配生成（阶段一）：这一组叶节点落到方案侧能负责它的节点上，走复用 → 改进 → 新增，改进伴随影响面复查；有新增或改进就自下而上调树至收敛。
3. 分解分析（阶段二）：把本轮涉及的节点按分解规则分解，产出方案叶节点；上一轮覆盖检查要求补叶节点的节点一并处理。
4. 承接覆盖检查：对本轮涉及的节点执行检查，该节点承接的所有需求的语义，包含于其叶节点语义之并；未覆盖就回第 2 步调整承接，或回第 3 步补叶节点。
5. 更新追溯——进入下一轮；没有未处理的需求叶节点时循环终止。

这里要追问一句：需求侧的“分解”去哪了？答案在整条链上看，本对的需求叶节点，正是上一对分解分析切出来的。本对开工时验收移交，只在分析迭代检出不合规时才重切。分解只有一条规则、两处现身（§6.3.5 收口）。

### 6.3.3 分配生成：从需求长出骨架

分配生成：把需求叶节点落到方案框架能负责它的节点上，并为新增或改进的节点自下而上调树。

它回答的问题：这条叶节点，归下一棵树的哪个节点负责？这棵树的骨架该怎么长？

先纠正一个容易想错的地方：一条叶节点不一定落在枝梢节点上。它落在下游能负责这条语义的那个节点上，是干枝还是枝梢，由语义定。“上游叶节点落到下游枝梢节点”只是“上游已到最深形态、

下游只到框架”这一种完成度差下的样子。往下游若已交出概要，也可以落在干枝上。所以落在哪儿是一次判断：判据是语义，不是位置。

恒信走到这里，罗建手上有“系统需求”这棵树的叶节点（SYS-002.01 顺序审批、SYS-003.01 金额校验……），下一棵树是“产品构成方案框架”：前端、审批引擎、合同台账、签章服务、通知服务。分配就是给每一条叶节点找一个归宿：

| 需求叶节点（系统需求详细条目） | → | 产品构成方案节点 |
|---|---|---|
| SYS-002.01 顺序审批 | → | 审批引擎 |
| SYS-002.04a 超时检测 | → | 审批引擎 |
| SYS-002.04b 提醒发送 | → | 通知服务 |
| SYS-003.01 金额校验 | → | 审批引擎 |
| SYS-005.02 电子签章 | → | 签章服务 |
| SYS-006.01 合同归档 | → | 合同台账 |

（SYS-002.04a／b 系由 SYS-002.04“超时提醒”重切而来，故事见 §6.3.4。）

分配完成不是结束，要过三关：

| 自检 | 问的问题 | 违反的后果 |
|---|---|---|
| **全覆盖** | 所有叶节点都落上了吗？ | 没落的叶节点在下层方案中无对应设计，需求直接遗漏 |
| **单承接（一对一）** | 每条叶节点都只落一个节点吗？ | 重复实现，浪费资源、无法验证 |
| **N:1 合理性** | 每个节点落的叶节点数合理吗？ | 太少，节点太碎；太多，节点承载过宽 |

- 全覆盖最容易被“觉得没问题”带过。罗建第二次分配，数一遍叶节点总数、再数一遍落上去的，数字对不上，有一条“审批历史查询”没地方放。方案树里压根没有“检索”这个节点，补也没处补，

这条叶节点还是从需求里消失了，直到上线后用户问“审批历史哪去了”才发现。

- **N:1 合理性**的经验值：一个节点落 2 到 5 条叶节点较优，超过 8 条就该考虑这个节点是否太宽、要不要触发节点拆分。但这是经验不是铁律。判断标准永远是语义相关：落在同一个节点上的叶节点，应该确实是一类事。

分配不是把叶节点贴到一张现成的树上，方案框架本身就是分配生成长出来的。三步，优先级从高到低：

| 动作 | 资产优先原则 | 说明 |
| --- | --- | --- |
| **直接承接** | Reuse（复用） | 现有节点的语义范围直接覆盖需求，落上即完成 |
| **改进后承接** | Improve（扩展／拆分／合并） | 现有节点扩展、拆分、合并后能承接，就落到拟改进节点，登记改进意图 |
| **新增承接** | Add（新增） | 基于需求语义与节点的物理意义，生成新的节点 |

优先级始终不变：能复用就不改进——改进有代价（影响面复查，§6.3.4）。但既有结构仍应先用尽，能改进就不新增，新增只留给改进也接不住的叶节点。

新增或改进之后，**自下而上调树**——归组子节点、归纳或修订各级父节点的物理意义，直至收敛：覆盖完整 + 兄弟互斥。调树由人类主导、AI 辅助；每长一个新父节点，先过三条质量判据：独立可定义、非兜底、最小覆盖。新节点归不进任何现有父节点的语义范围时，才在更高层新增父节点。

还有一层名字要分清：**分割、对齐、分配**这三个动词，在 ISO/IEC/IEEE 15288:2023 **§6.4.5.1 注 2** 里作 partitioning／alignment／allocation。正本说的是**设计定义过程把“架构实体→系统元素”的这三件事反馈回架构定义过程，供其合并或确认**；**§5.9 讲概念与系统定义的整体关系，不含这三个动词**，原注有误。

那是架构级把边界落到元素与契约上的动作，属第 11 章六步的第④步“分配与接口”。本章的“分配”是需求叶节点与方案节点之间的落靠：两级同名，说的是两件事。

概念卡片 · 分配生成

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 把需求叶节点落到方案侧能负责它的节点上（复用→改进→新增），并为新节点自下而上调树——阶段一：由需求长出方案骨架 |
| **它不是什么** | ⑥⑦ | 对立＝分析迭代（查与迭代）——分配生成负责“长”，分析迭代负责“查”，查出的违规才回炉；辨析＝分配生成 ≠ 简单分派——单承接是硬约束（一条叶节点一个归宿），落点由语义定级而非位置约定，且必须先复用再改进后新增 |
| **它从哪来** | ②③⑨ | 源自系统工程的需求分配与资产复用实践（复用优于改进、改进优于新增）；“分配生成”合称是本书对阶段一动作的凝练命名；英文 allocation & generation，allocation ← ad-（向）＋ locare（放置） |
| **它往** | ④⑤⑧ | 在两棵树之间接力时用；边界＝落点以语义定级（干枝或枝梢，不预设）、改进伴随影响面复查；误区 ✗“把 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **哪去** | | 叶节点贴到一张现成的树上”——方案框架本身就是分配生成长出来的 |
| **判据** | 三自检 | 三自检过了吗？全覆盖／单承接／N:1 合理。调树收敛了吗（覆盖完整＋兄弟互斥）？改进做影响面复查了吗？ |

## 案例进展②｜搬之前，先验叶节点

罗建撸起袖子，把系统需求的叶节点往产品构成方案框架的节点上搬。

顾言拦住他：“搬之前，叶节点验过了吗？父需求里'三年期以上要重点审'这条，你的叶节点齐吗？”

罗建一愣，回去一查：“合同校验”节点的叶节点只切了金额校验、条款校验、资格校验三个，期限校验漏了。“合同期 ≥ 3 年进入重点校验”这条要求，叶节点里没有对应。

“这批叶节点是上一对切出来交到你手上的，开工前就要验收。”顾言说，“漏一条叶节点，下层方案就不会做期限校验。将来五年期合同不走重点审，出了事追溯回来，就是这个漏洞。判据在 §6.3.5 的语义四查里。”

罗建补切了期限校验，又看了一遍，越看越不对劲：“我好像还多切了一个'合同打印'……这不是校验该干的。”

顾言：“那叫影子需求，切叶节点是精确切分，不是顺手多切。删掉。”

罗建删掉，又复查约束继承，把“数据留档十年”这条系统约束，落到归档那个节点上，确保有叶节点显式承接了它。

苏婷凑过来看了一遍，她是开发，问：“这批叶节点齐了，我是不是就能照着开工了?”

顾言：“还早。叶节点齐了、落上了，才算走完分配生成；接着要分解分析，让节点长出自己的叶节点，再查一遍'接住的需求一定被实现'。这两步没过，开工就是白干（§6.3.3、§6.3.5）。”

“好，我先把这批叶节点过一遍。”罗建说。

顾言：“现在，先把落上去的每一条都查一遍——下一个动作，分析迭代。”

### 6.3.4 分析迭代：一条需求一个归宿

分析迭代：检查分配与生成的正确性，并根据检查结果迭代，迭代的主要对象，是需求分解。

它回答的问题：叶节点落对了吗? 落得下吗? 错了，回哪一步修?

很多团队怕这两个字，“迭代”听着像返工、像前面的活白干了。恰恰相反：分析迭代是有方向的修正，不是推倒重来。方向规定只有一条：

> 节点是基准，需求是适配方。违规时重切需求，而不是迁就需求改框架。

为什么? 基准不只被动接收分配。分解规则预先定好了需求侧怎么切（切点带进消费方的语义范围，§6.3.1）；叶节点天然可单点落靠，迭代试错不是常态机制，而是兜底。改进与新增属正向承接设计，框架为承接切好的叶节点而生长，不是修正；修正只在检查出违规时发生，方向只指向需求侧。

**单承接：函数的单值性**

分析迭代守的核心约束只有一条，也是本章最重要的约束：

> 一条需求叶节点，只能落到下一棵树的唯一一个节点上（单承接：可 N:1、不可 1:N）。

这就是 Y=F(X) 的单值性：多条叶节点落同一个节点（N:1）是聚合，一条叶节点落两个节点（1:N）让函数不成其为函数。为什么这么严？三个理由：

1. 避免重复实现。一条叶节点（比如“金额校验”）同时落到“审批引擎”和“前端”，两个模块各实现一遍，同一逻辑两处实现，改一处漏一处，这是重复代码的根源。
2. 保证验证有唯一对象。“金额校验”对不对，该测哪个模块？单承接让验证有唯一目标；一对多让验证不知道该查谁。
3. 让变更可定位。需求改了金额校验，只改一个模块；分到两处就要找齐两处，漏一处就是缺陷。

注意一个对称：节点承接叶节点是 N:1（一个节点落多条叶节点），叶节点归宿节点是单承接（一条叶节点只有一个归宿）。前者是聚合，后者是定位。很多团队把方向搞反，让叶节点落多个节点（违反单承接），却让一个节点只落一条叶节点（浪费聚合能力）。

还有一条跟着落地的约定：承接落定的标志是一次明示化。被承接的需求叶节点以**规定需求**的形态落到目标节点上：有编号、有对象与适用范围、有断言、有判定方法。停留在“大致知道归它管”，不算落定。

**违规两分类：回炉，还是走出口**

仍出现一对多时，先判类型，别急着返工：

- **程序违规**——分解规则没按四条考量制定（切点没带进消费方的语义范围）。处理：回炉重定分解规则，再重切需求。把消费方节点的物理意义带进来重新定切点。
- **切不动的语义整体**：关系性内容怎么切都要跨节点（判据见§6.3.1 第三条考量）。走两个出口：承载量超限触发节点拆分（比如“超时提醒”拆成超时检测与提醒发送）；约束与策略类内容走横切式承接（留在独立分支作叶节点，用影响面标注表达，§6.2.7）。

顺带把“防切过碎”说全：重切不是越切越细。语义原子性判据守着下界：叶节点保有独立语义、可独立验证，防止为迁就节点的切法把语义整体切碎。

**改进有涟漪：影响面复查**

“改进后承接”不是局部操作。对节点的扩展、拆分、合并会影响已承接的其他需求：拆分后，原已承接的需求要重新分配到子节点；合并后，节点的语义范围变窄，某些既有需求可能失配。

任何改进操作后必须执行“影响面复查”：重新核对既有承接，失配则批量重分配或回退改进。否则会在循环内留下隐性失配，到符合性审查才暴露。改进同时改变分解规则的制定输入。新增或改动的节点物理意义即时更新消费方的语义范围，受影响的分解规则须随影响面复查同步修订。

概念卡片 · 分析迭代

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是** | ① | 检查分配与生成的正确性（单承接、承载量、影响面），并根据检查结果迭代——迭代的主要对象是需求分解 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **什么** | | |
| **它不是什么** | ⑥⑦ | 对立＝分配生成（长骨架）——迭代不是否定生成，是修正输入（重切叶节点）或补全结构（节点拆分、新增）；辨析＝分析迭代 ≠ 需求分析（第 4 章·加工）——需求分析往前看（需求清不清楚），分析迭代往回修（叶节点合不合用） |
| **它从哪来** | ②③⑨ | 源自单承接约束（Y=F(X) 的单值性）与资产变更管理实践；“分析迭代”是本书对阶段一关口的凝练命名，非标准条文；英文 analysis & iteration |
| **它往哪去** | ④⑤⑧ | 每轮分配生成后立刻用；违规两分类——程序违规回炉重定分解规则再重切需求，切不动的整体走节点拆分或横切式承接；改进伴随影响面复查；误区 ✗“检查不过就推倒重来”——迭代有方向：节点是基准，重切的是需求 |
| **判据** | 两问一查 | 单承接成立吗（可 N:1、不可 1:N）？承载量合适吗（超限触发拆分）？改进后既有承接核对了吗（影响面复查）？ |

## 案例进展③｜一个“悬空”的叶节点

罗建把系统需求的叶节点往产品构成方案框架上搬，搬到最后，剩下一条没人要：

SYS-004.01 合同全文检索，支持对已归档合同按全文检索，响应 < 3s。

“全文检索……”罗建先看系统需求树：三个干枝——审批业务域、平台服务、数据与接口；再看产品构成方案框架的节点——哪个都不像。硬塞给“合同台账”？台账只管存储，不做检索。（SYS 编号按需求流水编排、不对应节点序——004 是“数据与接口”域下的需求，不表示缺了第 4 个节点。）

顺带分清两层名字：系统需求树里叫“流程引擎”“通知能力”（能力），产品构成方案框架里叫“审批引擎”“通知服务”（构件），名字别跨层混用。

分析迭代检出了这条悬空叶节点。顾言让他先别硬塞：“一条叶节点没有归宿，只有两种可能：要么**叶节点切粗了**（欠切分，该重切成'建索引'和'查索引'两条），要么**方案树没长够**（缺一个'检索服务'的节点，生成新节点）。你先判断是哪种，两条路的判据在 §6.3.4。”

罗建想了半天：“这条叶节点拆不开，'对已归档合同全文检索'就是一件事，再拆就把'建索引'和'查索引'的接口写进去了，那是设计不是需求。所以是方案树缺节点。”

于是分配生成走了一步新增：产品构成方案框架新增一个节点“检索服务”，叶节点有了归宿。至此产品构成方案框架六个节点齐了。

王默一直在旁边听着，他是运维，接过话茬：“这根'检索服务'立住了，我运维就有得部署、有得监控了。一个节点，就是一样要上线伺候的东西？可反过来，树没长全，我连该部署几样都数不清。少一个节点，就少一样要伺候的东西，到上线那天才露馅。”

顾言点头：“对。方案里的节点，就是你运维要伺候的对象。你的部署清单，就是这棵树的全覆盖检查，你数得清几样，树就长了几个节点。”

他顿了顿，补了一句：“分析迭代检出来的问题，一半是分配的错，缺枝要生成新节点；一半是上游的错，叶节点切粗了要重切。三个动作是互相把门的。”

### 6.3.5 分解分析：切到可独立验证

分解分析：方案节点按分解规则分解为方案叶节点（阶段二），并做承接覆盖检查，接住的需求一定被实现。

它回答两个问题：这个节点要长出哪些叶节点，才能把它承接的叶节点全部实现？长出来的叶节点，真的把承接的需求都实现了吗？

先回头看一眼叶节点的来历。需求叶节点是上一对用同一条分解规则切出来的（首对由链首从存量材料结构化产出）。本对的方案叶节点，由本对在分解分析中产出，同一条分解规则，全链两处现身。链尾对例外：节点的分解移交构件设计方执行（§6.4.3）。

**切到哪为止**

罗建面对的“合同校验”这个节点（上一对的产物），落着几条需求。它被切成了四条叶节点。“拆成几条”谁都会，难的是拆得对。分解规则的执行，先记住切法：

> 切点沿消费方节点的语义范围边界切分——本对方案叶节点的消费方是下一对的承接结构，链尾对的消费方是构件设计方。叶节点天然可被消费方单点承接（单承接由分解规则直接保证，§6.3.1）；“再分就失去独立语义”只是下界——防止切得过碎；切不动的语义整体不硬切（走节点拆分或横切式承接两个出口）。

切出来对不对，过语义四查：

| 检查 | 问的问题 | 违反的样子 |
| --- | --- | --- |
| **覆盖完整性** | 父节点里的每条要求，叶节点都有对应吗? | 漏——父里有的，叶节点缺了 |
| **不超出性** | 叶节点引入父范围外的新内容了吗? | 超出——凭空冒出来的要求 |
| **逻辑一致性** | 叶节点之间互相矛盾吗? | 冲突——两条叶节点打架 |
| **约束继承** | 父的非功能约束，至少一条叶节点承接了吗? | 丢失——约束在分解中消失 |

- 覆盖完整性：“合同校验”要审“金额、期限、条款”三件事，只长出“金额校验”“条款校验”，期限校验就漏了，这是最常见的失败（案例进展②刚抓过一次），漏一条，下层就不知道要支持它。
- 不超出性：如果顺手长出第五条叶节点“合同打印”，这是“校验”节点上不该有的，分解不是多列几条，是精确切分。没经上游确认，就是范围蔓延（也叫影子需求）。
- 逻辑一致性：“金额 ≥ 500 万进入重点校验”和“所有合同一视同仁”不能同时存在，两条叶节点打架，下层方案无所适从。
- 约束继承：“合同数据留档十年”是系统级约束，分解时必须至少一条叶节点显式承接（比如归档的叶节点写“数据保存 10 年”）。约束在分解中丢了，下层根本不知道，直到审计才发现没留档。

四条检查里，“覆盖完整”与“不超出”是姊妹对：一条防漏，一条防多。漏了，需求在方案里消失；多了，长出没被授权的需求，方向相反，都让树偏离它该承接的需求。

**指标怎么跟着分**

分解还有第二个工作面：性能指标怎么分。父需求带着一个指标（比如“整笔审批在 48 小时内完成”），分解后指标怎么落到各条叶节点上？三种情况，按顺序判断。

情形一：可拆分，子指标之间有函数关系。响应时间、可靠度、功耗这类指标，子项可按运算关系拆。恒信的例子：“整笔审批 48 小时完成”拆成“发起 + 各级审批 + 签章 + 归档”各段加起来不超过 48 小时（串联可加），每个子项分到预算，总和不超过父值。

镜照一眼冰链，这是最漂亮的一类：M3 温控箱的“全程可靠度 ≥ 0.99”，拆成硬件可靠度 × 固件可靠度 × 云端可靠度 ≥ 0.99（**串联乘积**——任何一个环节断链，整条链就断）。于是硬件分到 0.995、固件 0.995、云端 0.999，三者乘积约 0.989，不够 0.99。

数字一旦落纸，立刻发现预算不够，就得回炉，这正是“可拆分”的价值：把账算清楚，不靠感觉。

情形二：下放，所有子需求各自独立满足同一指标。工作温度范围这类指标没法拆分。M3 的“工作温度 -20℃~60℃”（设备工作环境温度；箱内冷链货温是另一档，见第 3 章）要求每个部件都在 -20℃~60℃正常工作：传感器、电池、压缩机、主控板，一个都不能例外。这不是把温度区间分给不同部件。所以所有叶节点**同值继承**——恒信的例子：“操作日志不可篡改”是系统级约束，全部下放。

情形三：暂不分解，系统级耦合，需求阶段算不清。M3 的“整机噪声 ≤ 45dB”：噪声是“声源—传播路径—接收者”的系统级问题，需求分解阶段还没有结构数据，没法预先分给某个部件。这时候**不做假分解**——在分解记录里标注“向下一层传递”，指标不消失，留到方案设计阶段再算。

最怕的是第三种情况当成第二种，硬塞给一条叶节点，结果那条叶节点怎么都满足不了。

> 指标分解三情形：能拆就拆（按函数关系）、拆不动就下放（同值继承）、算不清就传递（标注“向下一层传递”）。最忌讳的是假装能拆。

**切法上的七种错**

分解的错误不止“漏”和“多”，一共七种，值得一次记全：

| 类 | 反模式 | 表现 | 后果 |
| --- | --- | --- | --- |
| 语义 | 过分解 | 把实现步骤当功能切（“先加热→再保温→最后冷却”） | 需求里混入实现细节，压缩下层设计空间 |
| 语义 | 欠分解 | 一条需求含多个独立功能（“上料、加工、检测、分拣一起做”） | 无法分配——一条要多个节点承接 |
| 语义 | 影子需求 | 凭空长出父需求没有的内容 | 范围蔓延，未经上游确认 |
| 语义 | 约束丢失 | 父的非功能约束在叶节点中消失 | 下层不知有约束，交付后才发现不满足 |
| 指标 | 指标 | 子指标口径变了（父“响应 < 2s（95 分位）”→ 子“< 2s（平均）”） | 名义继承、实质偏移，验证时对不上 |

| 类 | 反模式 | 表现 | 后果 |
|---|---|---|---|
| | 错配 | | |
| 指标 | 指标落空 | 父有量化指标，子只保留功能描述（“加热响应 < 5s” → “支持加热”） | 指标约束彻底丢失 |
| 指标 | 指标超预算 | 子指标之和超过父指标（成本 5000 → 2000＋2000＋1500） | 预算被突破，整体无法收敛达标 |

过分解最值得多说一句：它把“怎么做”混进“做什么”。第 3 章讲过需求答“要什么”、设计开发答“怎么做”，分解是精确切分，只切“要什么”，不写“怎么做”。“支持温度控制”是需求，“先加热再保温”是设计，后者一旦写进需求，下游就失去选“怎么实现”的自由。万一有更好的方案呢？这是需求被过度约束最常见的形式。

概念卡片 · 分解分析

| 四问 | 格 | 内容 |
|---|---|---|
| **它是** | ① | 节点按分解规则分解为方案叶节点（阶段二），并做承接覆盖检查——接住的需求一定被实现 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **什么** | | |
| **它不是什么** | ⑥⑦ | 对立＝分配生成（树间落靠）——分解分析在树内（节点→叶节点）；辨析＝分解分析 ≠ 多列几条——切点由消费方语义范围定，语义四查＋指标三情形防切过碎、七种反模式防切错 |
| **它从哪来** | ②③⑨ | 源自工程分解实践：把整体拆回组成部分，让复杂设计可逐个验证、逐个分配；同一条分解规则全链两处现身；“分解分析”是本书凝练命名；英文 decomposition & analysis，de-（反）＋ compose（组成） |
| **它往哪去** | ④⑤⑧ | 每个节点分配落定后用；链尾对例外——节点分解移交构件设计方（分解源判据就是规则接口，§6.4.3）；误区 ✗“叶节点越多越详细越好”——切过细把语义整体切碎，下游消费不了 |
| **判据** | 四查一公式 | 切点带消费方语义范围了吗？语义四查与指标三情形过了吗？覆盖检查公式过了吗（该节点承接的所有需求的语义，包含于其叶节点语义之并）？ |

### 6.3.6 覆盖检查：接住的需求一定被实现

叶节点切完、落好，是不是就结束了？没有。分配只保证“每条叶节点都有归宿”，不保证“每个节点都接住了叶节点”。这是分解分析的第二半，分析：

每个节点分解完成后，检查：

> 该节点承接的所有需求的语义，包含于其叶节点语义之并（满足度）

这是需求符合性审查第三层（满足度）的设计期预演——未覆盖就回分配生成调整承接，或回分解分析补叶节点。这防止“接住了但没实现”的空洞，这道关口，分配生成里落了承接的每个节点都要过。

覆盖检查是两阶段之间的关口：分配生成决定“谁承接谁”，分解分析决定“承接的有没有被实现”，两者在这一对会合。

覆盖检查落成文档动作，就是符合性分析：检查每个方案节点，是否充分满足了它落着的全部需求，分语义、指标两个维度。

先清一个词。“分析”这个词，到本章已经一分为三：第 4 章的“需求分析”是**加工**（往前看，需求清不清楚）。§ 6.3.4 的“分析迭代”是**查分配与生成**（往回修，叶节点合不合用）。这里的“符合性分析”是**查承接的实现**（往后看，方案接没接住）。同词三义，各指各的，评审会上听到“分析”，先问一句查的是什么。

语义符合性：方案的定义覆盖了需求吗？三问：

| 问 | 充分的样子 | 不充分的样子 |
|---|---|---|
| **覆盖度** | 方案节点的定义里，提到了每条承接需求的对应处理 | 定义只覆盖了部分需求 |
| **映射正确性** | 需求的术语和方案的术语能对应 | 需求是“安全审计”，方案是“操作日志”——范围不同 |
| **分析具体性** | “通过实现 X 机制满足 Y 需求，具体体现在 Z” | “本节点满足上述需求” |

“本节点满足上述需求”是最典型的空话——它什么都没说。合格的符合性分析长这样：

> 本节点（审批引擎）通过以下方式满足所承接的需求：
>
> - 针对 SYS-002.01（顺序审批）：实现状态机驱动的审批流，支持法务→财务→业务顺序流转，覆盖了“按序审批”的全部语义；
> - 针对 SYS-003.01（金额校验）：实现金额规则引擎，校验金额 ⩾ 500 万进入重点审，响应 < 1s。

每一条叶节点，都有一个“怎么满足它”的具体回答。写不出来的，就是没接住。

指标符合性：方案指标不低于需求指标吗？语义接住了，指标也要接住：

- 需求“审批响应 < 2s”，方案节点“审批引擎响应 1.8s” → 满足；
- 方案节点“审批引擎响应 2.5s” → 降级（必须说明理由、经评审确认，不能默默降）；
- 方案节点根本没提响应时间 → 丢失（指标在方案里消失了）。

和指标分解一样，比较前要先确认运算关系：需求 2s 是整体（可拆分），方案要算的是组合值；需求是下放（同值继承），方案直接比。指标分析最强的形式是把账算出来：“本节点各环节 0.6＋0.7＋0.5＝1.8s < 2s，结论：满足。”

> 符合性分析的基本句式：“针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足／降级／丢失。”写不出这句的，等于没分析。

时机：每一层分配完立刻做，不是末次全量验证。符合性分析最容易犯的错是“留到最后做”，等全部设计完成才做一次总检查，这是倒置的：分析越晚，缺陷离源头越远，返工越贵。正确做法是每一对

“需求-方案”分配完成，立刻对每个节点做分析。本章把一对“需求-方案”走完的功课叫结对六步，它是其中第⑥步“签定稿”内的检查关（§6.4.4）。逐层清账，而不是算总账。

这呼应着第 8 章要讲的评审、验证与确认，那时我们会看到，验证把“符合性分析”纳入了更完整的框架：设计期这一遍是预演——链尾构件规格评审上是正式审查（§6.4.3）。这里先记住：分析是每一层都要做的动作，不是最后的仪式。

概念卡片 · 符合性分析

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 检查承接的实现：每个方案节点是否充分满足所承接的全部需求（语义＋指标）——承接覆盖检查的落纸形态 |
| **它不是什么** | ⑥⑦ | 对立＝需求分析（第 4 章·加工）与分析迭代（查分配与生成）——符合性分析查的是“承接的实现”；辨析＝符合性分析 ≠ 评审——它是设计动作（每层分配后立刻做），评审是关口（第 8 章）；分析是评审的输入 |
| **它从哪来** | ②③⑨ | 源自验证实践：检查方案是否满足所承接的需求，从“评审验证确认”家族分出，成为每层分配后立即执行的动作；英文 compliance analysis，comply（符合）；区别于质量意义上的“符合性”（那是程度判定） |
| **它往** | ④⑤⑧ | 每层分配完成后立刻用；设计期是满足度预演，链尾构件规格评审上是正式审查（§6.4.3）；误区 ✗“本节点 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **哪去** | | 满足上述需求”——空话；要逐条叶节点说清怎么满足、指标多少 |
| **判据** | 满足句式 | 每个节点都有语义＋指标结论（满足／降级／丢失）吗？降级有理由并经评审确认吗？ |

## 案例进展④｜重要合同的审批，没人认领

恒信走到这一步，顾言组织了一次覆盖检查（符合性分析），对产品构成方案框架的每个节点，逐条核对“承接了的需求，它的定义接得住吗”，再报“满足／降级／丢失”。

报到“审批引擎”这个节点，罗建卡住了。节点上落着顺序审批、金额校验那几条。

但第 4 章顾言好不容易澄清的那条“重要合同要审批”。触发条件“金额 ⩾ 500 万或三年期以上”，在审批引擎的定义里，没有对应的处理。

“那条需求哪去了？”罗建翻了半天：“我把它归到'合同校验'去了，校验过了就触发重点审……”

顾言摇头：“那你看看'审批引擎'的承接判据，有没有写'承接重点审触发'？”没有。校验触发只是触发，“进入重点审批流程”这个动作本身，没有节点认领。覆盖检查失败：需求写着“重要合同要审批”，方案里“审批”这一环没人认领。

这不是小事。没有这个承接，重点合同会和其他合同一样走普通流程，三年期合同漏掉重点审，是合规事故。

郑亮坐直了，他是业务方："'金额 ≥ 500 万或三年期以上要重点审'，这是我们业务方定的红线。方案里没接住，业务上就是漏审。"

罗建在"审批引擎"的承接判据里补上：承接"重要合同触发重点审流程"，实现"校验结果 → 重点审路由"。

顾言合上本子："这就是为什么覆盖检查每一层都要做。分配让每条叶节点有归宿，但归宿了不等于接住了，树是不是真的长起来，要靠分解分析来验。"

### 6.3.7 干枝与叶节点分开维护

动作的边磨完了，还有一个容易焊死的地方，干枝与叶节点。

这是设计文档里最容易被低估的一件事。干枝和叶节点必须分开维护、各自演进，因为它们的变更频率和变更原因完全不同：

- **修剪枝条**（调整分类层级，比如"归档台账"从"审批业务域"挪到"数据与接口"）→ **只动框架定义**——叶节点一条都不用动。
- **长新叶节点**（新增需求，落在已有节点下）→ 动的是内容（详细定义加叶节点、框架定义映射表补一行承接），干枝结构不用动。
- 只有当节点本身变了（新增或删除节点）时，干枝和叶节点才同时动。

第 4 章刚讲过需求变更控制。这里你看到了"结构"与"内容"分离带来的变更弹性：干枝管结构，结构不轻易变；叶节点管内容，内容随时长。把结构和内容焊死在同一份文档里，每次内容变更都要拖着结构一起动，变更成本成倍上升。这呼应着第 15 章的配置管理：干枝和叶节点是两个配置项，分别纳入基线。

---

## 6.4 一条链上的逐对接力

> 这一块把一对放大成一条链：链上相邻两份是什么关系、哪些步在承接哪些步不做映射、链尾那一刀怎么交出去、一对之内要走完哪些功课、设计开发怎么验收。

### 6.4.1 上一份的叶节点，下一份的需求

前面两块在讲“一对需求-方案之间”怎么干活。分配生成让叶节点落到节点上、骨架长出来；分析迭代管违规，违规就重切需求；分解分析让节点长出自己的叶节点，再检查承接的实现。但真实设计不止一对“需求-方案”。

从一个真实的链条看起，恒信的项目从头到尾是这样：

- 老总一句话“合同审批要数字化”，**原始需求**——还没被组织；
- 顾言访谈法务、财务、业务，把“数字化”展开成各方需要的集合，**利益相关方需求**；
- 运行概念写清“一份合同怎么被三方审批”，**业务流程**；
- 业务流程翻译成系统必须提供的能力，系统需求（上一块的 SYS 叶节点）；
- 系统需求变成可交给构件设计方的构件骨架，产品构成方案框架。

从“一句话”到“可设计的构件”，一份一份往下接，每相邻两份之间都是一个“需求-方案”结对。而链首那一句还没被组织的话，要先进一段入口加工：把外部材料文本化、切分、澄清，加工成第一份需求产品数据。入口加工出来的，才是链上的第一份；“原始需求”本身不是链上的一份产品数据。

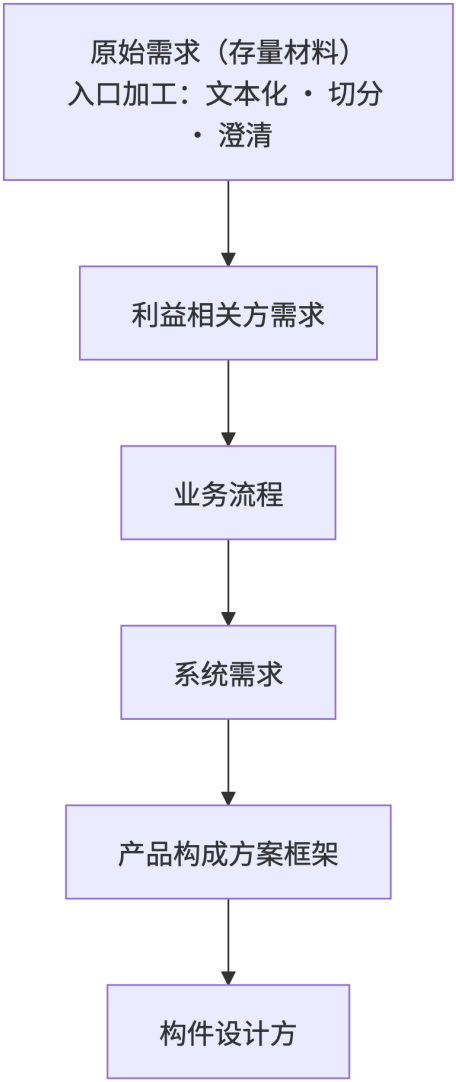



图 6-3

关键在中间层。“利益相关方需求”“系统需求”这两层，同时扮演两个角色：向上，它是方案；向下，它是需求。这就是“需求与方案相对”，需求不是文档的固有标签，而是相对相邻文档扮演的角色：

| 层 | 对上层 | 对下层 |
| --- | --- | --- |
| 原始需求（存量材料） | — | 入口加工的输入，不是链上产品数据 |
| 利益相关方需求 | 方案（承接原始需求） | 需求（驱动业务流程） |
| 业务流程 | 方案（承接利益相关方需求） | 需求（驱动系统需求） |
| 系统需求 | 方案（承接业务流程） | 需求（驱动产品构成方案框架） |
| 产品构成方案框架 | 方案（承接系统需求） | —（链尾：节点＝构件设计方可消费的单元，§6.4.3） |

这个相对关系不是纯概念。它是三个动作能接力工作的前提：上一对的分解分析刚切完叶节点，下一对的分配生成紧接着就从这批叶节点开工。利益相关方需求刚当完“方案”，转过身就当“需求”。

这也是为什么树结构在每一层都长得一样：树不挑角色，需求是树，方案也是树。

每一对“需求-方案”的正式名字叫文档结对：上层叶节点（详细定义条目）是需求方，下层节点（方案侧能负责这条语义的那个节点）是方案方。有需求必有方案，有方案必有需求，这是结对的第一条律。

每一对都遵循同样的两阶段工作法：

1. 骨架阶段：先约定下层方案的框架树，长哪几根枝、每个节点叫什么，并建立初步映射；

2. 填充阶段：上下层并行填充，上层完善叶节点（详细定义），下层完善节点（节点定义、承接、分析）。

为什么骨架先行？回到案例进展③那条悬空叶节点。它如果放在骨架阶段，第一天就会暴露“方案树缺一个检索节点”，因为骨架阶段就要约定“方案有哪些节点、哪些叶节点落哪些枝”。罗建先长完叶节点再统一搬，悬空的叶节点到最后才浮出。先约骨架、再填内容，缺枝问题最早暴露，而不是最晚。

### 6.4.2 链上只有分配步在做承接

上一节说“上一份的叶节点就是下一份的需求”。这句话很顺手，但它不是对链上每一步都成立。把相邻两份之间的步分一分，得到的就是**链上三类步**，边界也就清楚了——这也是 §6.2.3 三条依据的第三层用处。

| 步型 | 判据 | 本链实例 | 做与不做 |
|---|---|---|---|
| **分配步** | 两侧的叶节点不是同一批，须按映射定落点 | 业务流程定义 → 系统需求框架；系统需求概要 → 产品构成方案框架 | 做承接：Y=F(X) 在这里求值；单承接与承接覆盖检查在这里适用 |
| **整段搬入步** | 两侧的叶节点就是同一批，叶节点由上游直接给出 | 利益相关方需求 → 业务流程概要；业务流程概要的两个视角之间 | 不做映射：整段搬入；这一步的活是长干枝（把上游节点的属性长成下游的各级节点），并逐级判定其落位 |
| **形态轴步** | 同一份产品数据在两个形态之间 | 业务流程概要 → 业务流程定义；系统需求框架 → 系统需求概要 | 不是承接：形态轴上的深浅，不换产品数据、不换叶节点 |

本方法只在分配步上做 Y=F(X) 的求值。这条区分，是与“逐级分解”式说法最要紧的界线：如果把三类步一律当成承接，就会得出“上游第 N 级对应下游第 N−1 级”这类数量关系，而落到哪个节点由语义定，不由位置约定（§6.3.3）。

顺带说清一个词的两种用法：承接有时指承接规则（定义域只取上游叶节点、基数可 N:1 不可 1:N），有时指承接记录（“某份上游文档由哪份下游文档收下”的逐对对应）。后者是一份备档，不含映射语义，不受本条规则约束。本章其余各处，一律用前一个意思。

“上一份的叶节点就是下一份的需求”因此要加一句限定语：它在分配步上是机制，在另两类步上只是形态与视角的搬入。写进“下一份的需求”，不等于写进“下一次承接”。

### 6.4.3 链尾交给构件设计方

先把一句常被说错的话立正。理论上，设计的终点是生产的起点，设计画到“可以照着造”，就该交给生产了。但它不能说成“设计的终点是开发的起点”：设计与开发是一个概念（第 3 章），没有“设计完了才开始开发”这回事。

真正发生的，是设计层级之间的换手。本章这条链，走的是系统设计，系统设计的终点，是构件设计的起点。

整条链走到最后一份，产品构成方案框架，它只到框架这个形态落纸。它的节点（审批引擎、前端、合同台账这些构件）就是构件设计方可直接消费的最小单元。系统设计的终点，是一份带分解源判据的节点清单。

每个节点都带着两样东西：**承接判据**（它承接了哪些需求）和**分解源判据**（它该展开成哪类规格）。

分解源判据就是移交出去的规则接口。构件设计方接手后的第一件功课，就是照着它把构件展开成规格。这仍然是一次设计开发，只

是执行者从系统设计换成了构件设计；第 3 章讲过这个递归，每一级都是一次完整的设计开发。规格落定，照着规格把构件造出来，生产的起点，就在这条链的终点上。

所以链尾的换手，不是概念的断裂，是执行的接力：整条链每一对都是一次完整的设计开发；链尾这一次设计开发（构件 → 规格），由构件设计方执行。因为最后一刀的切法由消费方（构件设计方的实现结构）锚定，自然由构件设计方来切（§6.3.1 分解规则的制定输入）。设计与开发是一个概念，贯穿全链；换的只是谁来执行这一级。

覆盖也在这一步核验：§6.3.6 检查的“接住的需求一定被实现”，在链尾不再是设计期预演，而是构件规格评审中的正式审查。追溯链相应停在“需求 → 方案框架 → 节点”。

还有一层边界要说清：这条链按派生关系生成，只承载明示的条目。所以“链路自洽”只保证验证这一侧。

确认的判据是“**针对**特定预期用途或应用的需求”（ISO 9000:2026 §3.11.14），主体常常是隐含需求；方针是承诺加框架的关系，与派生关系不同构。确认的判据与方针都不在这条链上，它们要由需求基线之上另一个与派生链不同构的参照层承载。这也解释了 §6.2.7 留下的那句话：树装不下的内容可以另开分支，链装不下的东西只能另设一层。

### 6.4.4 一对从接需求走到冻结基线

两阶段说的是时间怎么组织；真正把一对“需求-方案”从头走完，要走六步（本章叫它**结对六步**）：**接 · 验 · 想 · 对 · 裁 · 签**

| # | 步骤 | 白话 | 概念归属 | 三个动作的展开 |
|---|---|---|---|---|
| ① | 接需求 | 需求就位：链首把存量材料结构化成需求树；链上各对验收上一对移交的最深形态——不新拆 | 需求就位（做） | 上一对分解分析的产出，本对的输入 |
| ② | 验就位 | 检查移交来的叶节点能不能用：可转化（完整、无歧义、不冲突、隐含已识别）、可分配（原子性、全覆盖、单承接、候选声明） | 需求就位的检查关 | 验 · 做 |
| ③ | 出方案 | 照着需求，想出两个以上候选“怎么做”（方案框架骨架：节点怎么长） | 分配生成 · 目标（先造树） | 生成 · 想 |
| ④ | 对需求 | 把每条叶节点落到方案对应的部分上：复用 → 改进 → 新增 | 分配生成（做） | 分配 · 做 |
| ⑤ | 做裁决 | 按规则比较候选，挑一个（改进谁、新增谁），把理由记下来 | 分配生成 · 裁决 | 生成 · 裁 |
| ⑥ | 签定稿 | 分解分析：节点 → 方案叶节点，覆盖检查与符合性分析（语义与指标都过）→ 利益相关方确认 → 冻结成基线 | 分解分析 + 定稿 | 分解 · 做 + |

| # | 步骤 | 白话 | 概念归属 | 三个动作的展开 |
|---|---|---|---|---|
| | | | | 核 + 封存 |

六步不是六个并列的动作，而是三个动作各自的展开，每件都缺一样它特定的东西，于是各长成一对：

- 需求就位长成接与验（①②）：叶节点是上一对的产出，接手必须验收（可转化、可分配），不合格退回上一对或走分析迭代重切；
- 分配生成长成想、对与裁（③④⑤）：下一棵树不会自己出现，分配前必须先想出骨架，骨架不是第四样，而是分配的目标树。没有目标树，分配无处可落；候选多个，必须先裁一个才算落定；
- 分解分析长成签（⑥）：骨架落定后切叶节点、查覆盖、确认冻结，阶段二的活全在⑥里收口。

> 六步 = 三个动作 + 各自缺的那件东西（验 · 想 · 裁）。

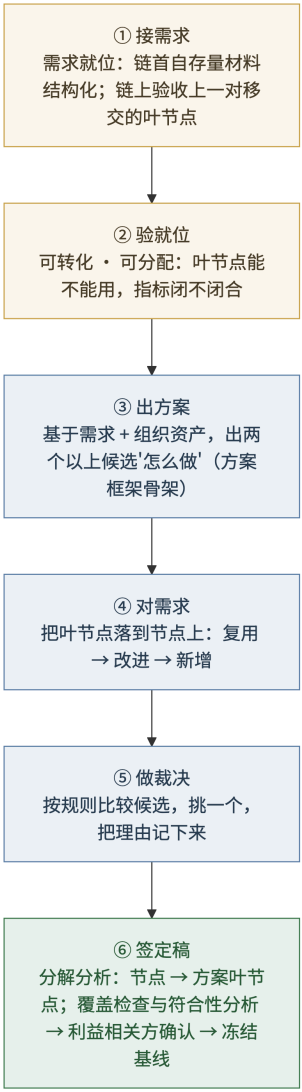



图 6-4

第②步的验收，用三把尺子加两把数值尺子：

| 尺子 | 检查什么 | 不过怎么办 |
| --- | --- | --- |
| 可转化（入口） | 移交来的框架定义本身够不够格用：完整、无歧义、不冲突、隐含已识别 | 退回上一对补识别——不是自己重切 |
| 可分配（出口） | 拆出来的叶节点能不能用：原子性、全覆盖、单承接、候选声明 | 不合格的叶节点登记待迭代——分析迭代重切（§6.3.4） |
| 边界不越位（深度） | 切到哪为止：止于“可被分配”，太细就侵入方案层 | 调粒度：太粗继续切，太细把细节剥回方案侧 |

三把尺子管“切得规不规范”，看形状、不动数值。另有两把数值尺子管“指标断没断”。一把是**指标闭合**：切出来的指标组合，能回到父指标吗？整机 ≤10W → 各模块之和 ≤10W，误差与可靠性同理。一把是分配预判：方案节点的指标，给需求留了满足空间吗？为什么这两查更要紧：切得再规整，指标闭合不了，系统照样不达标，指标闭合不过，全链返工；分配预判不过，方案白想、分配必然撞墙。

三个精化点，处处咬着本章既有概念：

1. ④ 先挂接、⑤ 后权衡：④ 是轻量覆盖映射（候选声明：只落不细），只给⑤ 的裁决提供覆盖率、均衡性、冲突性证据；正式细化留到⑤ 选定之后，被否掉的方案不白做工。
2. 派生需求回流：⑥ 定稿会派生对需求侧的新需求（接口约束、性能指标、可行性限制），必须回流需求基线、双向对账，方案侧不能单方面定义。这正是“结对双向”的一个落点：方案也在迭代需求。
3. ⑥“利益相关方确认”的双重身份：内部评审对质（决策理由、备选死因）＋ 利益相关方确认承诺，确认通过才冻结基线。这里的“确认”是利益相关方认可承诺，属理解确认与签认；第 8 章三把尺子里的确认要等真实使用场景取证。

> 与第 11 章划清：第 11 章讲的是**架构这一侧的进出口**——吃进五组、只把架构相关的那部分立成两样内容，而且**只走到“立成内容”**（选定与背书归第 12 章、表达成件归第 13 章，见第 11 章 §11.3.1 的四环）。本章的六步是每一对“需求-方案”的通用功课；层级不同、对象不同，别混为一谈。

### 6.4.5 设计开发的验收与追溯

做了这么多功课，设计开发怎么验收？答案是回到树本身，五个维度，每个都呼应着本章的一个概念。这五维是喂给第 8 章三道验收签字的检查内容：评审、验证、确认依次签下去，签的是这五维过了没有。

| 维度 | 检查什么 | 合格线 |
| --- | --- | --- |
| 干枝（框架定义） | 树完整（根到节点无中断、同层抽象一致）、每个节点四要素齐（节点定义／承接需求／符合性分析／映射关系） | 100% |
| 叶节点（详细定义） | 分解完成、叶节点原子（守住防切过碎的下界）、指标完整（数值＋单位＋测量条件） | 100% |
| 分配生成 | 全覆盖、单承接、N:1 合理 | 100% |
| 分解分析 | 每个节点语义与指标都过（覆盖检查＋符合性分析），无“满足”空话 | 100% |

| 维度 | 检查什么 | 合格线 |
| --- | --- | --- |
| 追溯 | 从原始需求到构件正向走通，从构件到原始需求反向走通 | 100% |

“干枝与叶节点”验收的是“一棵树”本身；“分配生成”是三自检；“分解分析”是覆盖检查与符合性分析。而追溯是把全链的树串起来的总验证——任取一条原始需求，看能否一路走到对应构件（正向）；任取一个构件，看能否说清它由哪些需求而来（反向）。走不通处，就是树与树之间断了。

最后回到第 3 章的一句话：设计开发止于可实现。现在你看到了“可实现”在树上的样子，叶节点切到可被单点落靠，节点具体到能被开发承接（带着分解源判据），每个节点都做过覆盖检查与符合性分析。设计开发的可靠性，不靠写得长，靠每一对“需求-方案”咬合得好。

### 6.4.6 认知反转

一个月后，恒信又开了一次方案评审会。还是那间会议室，还是那些人。

罗建在投影上打开的，不再是六十页文档，而是三页：

- 第一页：合同审批系统的框架树，一页纸，三个枝、十个节点；
- 第二页：需求叶节点与映射表，每条叶节点各有编号、描述、指标、验收标准、映射目标；
- 第三页：三张自检表，分配三自检、分解分析四查、每个节点的覆盖检查与符合性分析。

法务一秒钟定位到“审批流程”，找到驳回规则；财务翻到“合同校验”，看到对账的规则；业务部的同事指着“合同发起”说“这就是我要的”。

会议开了一小时，散会。罗建对顾言说：“我第一次觉得，这份方案是'活'的，每一条叶节点都能被找到，每个决定都有来路。”

回到章首那场会。你以为设计开发是“写方案”，质量取决于文笔和长度。本章的反转是：设计开发的对象是一对文档、不是一份，走的是三个动作、不是一支笔，方案不是写出来的，是种出来的。

四件事背后，是同一次方法时刻，第 1 章说的**织关系**：罗建把三百条需求排成一列（堆），顾言教他织成树（结构）。堆起来的方案找不到路，织起来的每条需求都有位置。第 10 章说架构是压缩，也是同一个原理：用结构消化复杂度。

用第 1 章的话收一次束：对“设计方案”“框架定义”“分配生成”“分析迭代”“分解分析”这些词，你原来只是“见过”。你知道设计开发对着“需求-方案”一对，每份文档是棵树，有三个动作。

本章把它们推到“拥有”：能答全四问，

- 它是什么：一对“需求-方案”、一次 Y=F(X) 求值、每份文档是棵树（框架／概要／定义三个形态）、三个动作（分配生成、分析迭代、分解分析）；
- 不是什么：不是单独一份文档、不是文章、不是简单分派、不是返工丢人、不是多列几条、不是空话、不是架构；
- 从哪来：从“可验证、可落实”的内涵推出来，从 Y=F(X) 的规则里长出来；
- 往哪去：链上每一对逐对接力，只在分配步求值；链尾交给构件设计方。

能答全四问才算拥有，你手里有了一对“需求-方案”，一个函数、一棵树一棵树往下走。

> 方案不是写出来的，是种出来的——一对“需求-方案”、一次 Y=F(X) 求值、三个动作，在整条链上逐对接力。

用第 1 章 §1.4.4 的尺子量一下：“需求-方案结对”是**第 ② 级（图式）**的功夫，你学会了把结构一块一块搭出来；至于“这么拆对不对”的判断，要到**第 ③ 级（心智模型）**。

## 案例进展⑤｜一条线从老总画到开发

恒信第一次把整条链走通，顾言和罗建坐在白板前，从下往上推了一遍。

最顶层，老总的一句话：“合同审批要数字化”。存量材料，还没被组织。第二层，三方需求：法务要条款校验、财务要对账的规则、业务要流程时效。第三层，业务流程：一份合同从发起、法务初审、财务校验、业务终审、签章、归档，运行概念里的故事，现在长成了树。

第四层，系统需求：SYS 叶节点，上一对分解分析切出来的那批（顺序审批、金额校验、期限校验……）。第五层，产品构成方案框架：审批引擎、前端、合同台账、签章服务、通知服务、检索服务。

顾言在白板上画了一条线，从“金额 ⩾ 500 万要重点审”这条要求，一路画到“审批引擎”那个节点：

“一条需求，从老总嘴里到开发手里，过了四对结对——每一对都是一次分配生成、分析迭代、分解分析。中间任何一道断了，这条需求就丢了。”

罗建盯着那条线看了很久，说：“所以方案不是写出来的，是每一对都走一遍三个动作、接力接出来的。”

顾言笑了：“你现在才懂。那六十页文章，就是四层功课都没做，只写了一篇感想。”

## 小结

本章回答的是“这份需求怎么被用来造东西”：一份需求与一份方案结成一对，一次 Y=F(X) 的求值，由需求开发出方案，并由方案迭代需求。

一对——设计开发的单元是一份需求文档配一份方案文档。一棵树——每份文档都是一棵树，干枝（框架定义）管结构、叶节点（详细定义）管内容，按定义到多深分框架、概要、定义三个形态，结构不轻易变、内容随时长。

**先定规则**——分解、构建、承接三套规则由人类主导制定，分解规则带着消费方的语义范围来定，单承接由分解规则直接保证。**三个动作**——分配生成（复用 → 改进 → 新增，长出骨架）、分析迭代（守住函数的单值性，违规就迭代需求分解）、分解分析（节点长出叶节点，接住的需求一定被实现）。

一条链——从存量材料经入口加工到第一份需求，再一份一份往下接：上一份的叶节点就是下一份的需求，追溯边自然生成。只有分配步在做承接，整段搬入步与形态轴步不做映射。链尾那一份只到框架，节点带着分解源判据移交构件设计方，执行换手，概念不分。

方案是结构，不是堆，第 1 章说的织关系，本章在“需求-方案”这一对上长成了两棵树。

下一章，我们先看这棵树长成什么，产品数据：开发产品就是开发数据。一棵树的干枝、叶节点、根，全是产品数据。再下一章，评审、验证与确认，三个看门人上场，本章的“覆盖检查与符合性分析”在更完整的验证框架里落定。

苏婷的工作夹里，多了一个“金额校验”的叶节点——她把这批叶节点的编号、指标、验收标准，一条条抄了进去。以前她得追着罗建问“做到什么程度算完”；本章之后，编号、指标、验收标准都写在叶节点上，不用问人了。

---

# 附录

## A 本章概念总图

一对（需求-方案结对 = 一次 Y=F(X) 求值）：由需求开发出方案、并由方案迭代需求——正反交替、逐对收敛

Y=F(X)：X=需求、Y=方案、F=三套规则（分解 / 构建 / 承接）——每一对先定规则，规则由人类主导制定（AI 辅助）

分解规则以消费方的语义范围为制定输入——单承接由分解规则直接保证，迭代试错只是兜底

单承接约束 = 函数的单值性：可 N:1、不可 1:N

落点以语义定级：下游能负责这条语义的节点——干枝或枝梢，不预设落点在枝梢

一棵树（对一份文档而言）：设计文档是树，不是文章

树要能定位、能承接、能追溯（文章三样都缺）

干枝 = 框架定义（管结构：根到节点 + 映射表；节点双向可用：承接判据 + 分解源判据）

叶节点 = 详细定义（管内容：五件套，单承接搬走）

三个形态：框架（根→枝梢）/ 概要（根→叶，只到说明）/ 定义（概要＋详细定义）；框架有阶、叶有三种身份

三条依据：可条目化 / 可树形化（成树三条件：判据单维、条目单归属、分组完备互斥）/ 可按 N:1 分配

树形假设有边界：横切 / 共享 / 多维 / 循环四类纯树放不下——树是存储骨架，引用与追溯是关系载体

框架定义 ≠ 架构定义：文档层面不等于；描述层面可作架构的一种描述

三个动作（叶节点驱动的循环：每轮处理相关联的一组，直至处理完）：

准备：定规则 • 结构化存量材料 • 只做直接承接（复用）

分配生成（阶段一）—— 叶节点落到能负责它的节点上：复用 → 改进 → 新增

自下而上调树：收敛 = 覆盖完整 + 兄弟互斥；父语义三查：独立可定义、非兜底、最小覆盖

三自检：全覆盖 / 单承接 / N:1 合理

分析迭代 —— 检查分配与生成：单承接成立吗？承载量合适吗？改进波及谁？

违规两分类：程序违规回炉重定分解规则再重切需求；切不动的整体走节点拆分、横切式承接

改进必做影响面复查；修正方向单向：节点是基准，需求是适配方

分解分析（阶段二）—— 节点 → 方案叶节点：切点沿消费方语义范围；语义四查 + 指标三情形

七种反模式：过分解 / 欠分解 / 影子需求 / 约束丢失 / 指标错配 / 指标落空 / 指标超预算

承接覆盖检查：该节点承接的所有需求的语义，包含于其叶节点语义之并（语义＋指标；满足 / 降级 / 丢失）

一条链：入口加工 → 利益相关方需求 → 业务流程 → 系统需求 → 产品构成方案框架 → 构件设计方

需求与方案相对：中间层是双面人（对上是方案、对下是需求）；上一份的叶节点就是下一份的需求

链上三类步：分配步（做承接，Y=F(X) 在此求值）/ 整段搬入步（不做映射，长干枝）/ 形态轴步（不是承接）

文档结对：有需求必有方案；骨架先行、填充并行；六步：接 • 验 • 想 • 对 • 裁 • 签

追溯边自然生成；链尾节点带分解源判据移交构件设计方——执行换手、概念不分

链的覆盖边界：按派生关系生成、只承载明示条目；确认的判据与方针不在链上，另设参照层

验收五维：干枝 / 叶节点 / 分配生成 / 分解分析 / 追溯；设计开发止于可实现

## B 本章的能力级

### 第 ② 级为主

- **表征**——把“需求-方案结对”“框架定义”“详细定义”“分配生成”“分析迭代”“分解分析”“符合性分析”“单承接”几个词说准：结对是一对文档、树有两个部分三个形态、落点以语义定级、单承接是硬约束；
- **图式**——一次 Y=F(X) 的求值这条结构；每份文档一棵树（根 → 枝梢 → 叶节点）；框架、概要、定义三个形态与“最深形态”；两阶段（阶段一长骨架、阶段二填叶节点）；一条链上的逐对接力；
- 心智模型——为什么必须是树（可条目化、可树形化、可按 N:1 分配，以及成树三条件）；单承接为什么是函数的单值性、违规两分类各回哪里修；指标三情形怎么判；改进的涟漪为什么要做影响面复查；
- 解释框架——本章不建，由第 17、18 章承担。

> 能答全四问才算拥有——你手里有了一对“需求-方案”，一个函数、一棵树一棵树往下走。

## C 自查 · 这些说法对吗？

1. “设计方案写得越详细越好，六十页比一页树强。”——错。文章不可定位、不可承接、不可追溯；没有结构的详细只是堆砌。
2. “结对就是由需求开发出方案，需求侧等着就行。”——错。结对是双向的：由需求开发出方案，并由方案迭代需求（重切需求、调整承接、修订规则），正反交替、逐对收敛。
3. “一条需求分给两个模块做，双保险。”——错。违反单承接（函数的单值性），重复实现、验证无对象、变更难定位。
4. “叶节点切得越细越好，切到不可再分才负责。”——错。切点由消费方的语义范围定；语义原子性只是防止切得过碎的下界，为迁就节点把语义整体切碎，下游消费不了。

5. “分解规则应该全项目统一，一套切法拆所有结对。”——错。切法必须业务化，每个结对按自己的领域经验定；全链统一的是判据。
6. “符合性分析写'本节点满足上述需求'就够了。”——错。那是空话；要对每条承接的叶节点说清“怎么满足、指标多少”。
7. “指标拆不动的就先不管。”——错。要标注“向下一层传递”，指标不消失，留到方案阶段再算。
8. “先把需求全部写完、切完，再开始设计。”——错。结对设计是骨架先行、并行填充；叶节点驱动的循环逐轮推进，否则悬空叶节点、方案缺枝要到最后才暴露。
9. “第 4 章的需求分析和本章的分析迭代是一回事。”——错。需求分析是加工（往前看，需求清不清楚），分析迭代是检查分配与生成并迭代需求分解（往回修，叶节点合不合用）。
10. “分配生成、分析迭代、分解分析是三个先后大阶段，走一遍就完。”——错。它们是叶节点驱动的循环：每轮处理相关联的一组叶节点，循环推进直到全部处理完。
11. “框架定义和详细定义焊在一起维护，省得两处对不上。”——错。变更频率与原因不同：结构不轻易变、内容随时长；焊死则每次内容变更都拖着结构动。
12. “设计文档交付一次就完了，画完即定型。”——错。需求会变；不可追溯就漏改；干枝和叶节点分开演进，文档是活的。
13. “规则定一次，终生不动。”——错。分解规则随消费方结构演进：改进或新增节点时，受影响的分解规则随影响面复查同步修订。
14. “链尾系统设计方把构件规格也写完，才算交付到位。”——错。执行换手：设计方交付到节点（带分解源判据），构件规格是构件设计方接手后的第一件功课，它仍是一次设计开发，只是换了执行者。

15. “上一份的叶节点就是下一份的需求，所以链上每一步都在做承接。”——错。只有分配步在做承接；整段搬入步不做映射，形态轴步根本不是承接。落点是判断，判据是语义。

## D 练习

1. 找一份你们团队最近的设计文档，判断它是树还是文章。如果是文章，找出它“不能定位”“不能承接”“不能追溯”各一个具体例子。
2. 把你们系统的一个功能域画成一棵框架树（根→枝梢节点），标出分类节点和枝梢节点，再给一个节点写出它的承接判据和分解源判据（双向可用）。
3. 用语义四查检验一批需求叶节点：覆盖完整吗？不超出吗？一致吗？约束继承了吗？每查都给出你检查到的证据。
4. 取一条带指标的需求，用指标分解三情形走一遍：先判断是“可拆分／下放／暂不分解”，写出你的判断过程和结论。
5. 给你们的方案节点做一次分配三自检：全覆盖吗？单承接吗？N:1 合理吗？有没有悬空的叶节点？
6. 找一条“不知道该归谁”的叶节点，判断它是“叶节点切粗了（迭代重切）”还是“方案树缺枝（生成新节点）”，并说明你的判断依据。
7. 用承接覆盖检查核一个方案节点：它承接的所有需求的语义，包含于它的叶节点语义之并吗？试按“针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足／降级／丢失”写一句。
8. 解释“框架定义”和“架构”为什么不能混，并举一个你们团队（或你听过的项目）“架构＝一份文档”的词汇病例子。
9. 一棵树的干枝和叶节点为什么能独立演进？举一个“修剪枝条不动叶节点、长新叶节点不动干枝”的例子，说明这对变更控制的好处。

10. 把一个文档结对走一遍：取上层三条叶节点，分配到下层两个节点（复用、改进、新增各至少试一次），再对每个节点做覆盖检查，记录每一步的自检结果。
11. 用第 1 章的四问法量“分配生成”（或“分解分析”）：它是什么、不是什么、从哪来、往哪去？哪一问最薄？最薄处往往正是最易混用处。
12. 取一个文档结对，走一遍六步：①接需求 → ②验就位（三把尺子加两把数值尺子）→ ③出方案（至少两个候选）→ ④对需求（复用→改进→新增）→ ⑤做裁决（写下选择的理由）→ ⑥签定稿（分解分析＋利益相关方确认＋冻结基线），并说明每一步对应本章的哪个动作或概念。
13. 给一段相邻产品数据之间的步判类型：它是分配步、整段搬入步，还是形态轴步？判完再说这一步该做什么、不该做什么。

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**


- [1] **R-02** ISO 9000:2015 —— §3.4.8 设计和开发〔已被 R-38 取代；本章只作版本对照〕
- [2] **R-03** ISO 9001:2015 —— §8.3.3 设计开发输入
- [3] **R-04** ISO/IEC/IEEE 42010:2022 —— §3.2 架构；§3.3 架构描述（“框架定义 vs 架构”的界分参照）
- [4] **R-12** ISO/IEC/IEEE 29148:2018 —— 单条需求的质量特性

- [5] **R-37** ISO/IEC/IEEE 15288:2023 —— §6.4.5 设计定义过程；§3.13 设计（设计是系统元素及其关系的规定，**其**完整程度足以支撑架构的合规实现）〔全书号已并到 2023 版条目〕
- [6] **R-38** ISO 9000:2026 —— §3.3.11 设计和开发；§3.5.1 需求（注 2:“规定的**要求**是明示的要求”——正本中译作“要求”，本书术语口径 requirement→需求，见附录 A T-26）

**经典文献**

- [7] **R-87** SEBoK—— 分割／对齐／分配三动词，与本章“分配生成”两级同名不同物；三动词的库内一手位＝ISO/IEC/IEEE 15288:2023 **§6.4.5.1 注 2（§5.9 不含此三动词，旧注有误**——§5.9 只给架构与设计的分工句：架构管适宜性·存续性·合意性，设计管兼容性与可行性）

> 对照阅读：本章“需求与方案结对”呼应第 3 章“需求、设计开发、质量”、第 4 章“需求工程”、第 5 章“需求的完整性”、第 8 章“评审、验证与确认”。

# 第 7 章 产品数据：开发产品就是开发数据

## 7.1 引言

第 6 章把一份需求做成了方案：一份需求配一份方案，结成一对；每份文档都是一棵树，干枝管结构、叶节点管内容，一对一对往下接。本章接着那棵树往下问一句：**这棵树到底是什么？**

它是产品数据。开发阶段你亲手造的不是实物，而是产品数据；实物只是这份数据复制出来的产物。本章要立的就是这句话：**开发产品就是开发产品数据**。

先把本章的位置说清。它在中篇的第二站，接第 6 章的设计开发落地，补上“造的是什么”这一问。它也是全书暗线的立论章：一个人的能力，最终体现在他交出的产品数据上，而这句话的根扎在本章。

先看产品数据的定义与它该有的样子，再磨开“开发产品就是开发产品数据”这句话的成立范围，最后看它在组织里怎么被用起来、归到哪里去。

### 7.1.1 误解现场

冰链科技 M3 二代的开发会，会议室墙上贴满了图纸。

嵌入式硬件工程师石军站在最前面，指着投影上的固件架构图，语气笃定：

“二代加了边缘断链预判——传感器异常时，箱体自己在本地判一次，不依赖云端。这一块是我最得意的地方。硬件已经改了两版，固件这周就能跑。”

会议室里有人点头，产品经理林涛却没动；他翻着手里的本子，问了一句：

“二代的功能基线呢？'边缘断链预判'这条需求，验收标准定了吗？拆到哪些模块？谁负责验证？”

石军愣了一下：“需求……不是写完了吗？在共享盘那个文件夹里。”

“哪个文件夹？”林涛追问，“我搜'二代'，搜出三个版本的需求文档，还有一个叫'M3 二代最终版'——哪个是准的？你刚才说硬件改了两版，改了什么、为什么改，有没有记录？”

测试季昭跟了一句：“追溯那段数据，我到时候按哪一版验？”

会议室安静了。石军这才意识到，他一直在说“把产品做出来”，可产品到底长什么样、为什么长这样、怎么证明做对了——这些他一件都说不全。

林涛合上本子，轻声说：

“石军，你把'开发产品'想成'把实物做出来'了。可我们开发一个产品，真正在开发的，是一整套**数据**——需求数据、设计数据、质量数据。实物，只是这套数据被'复制'出来的结果。”

石军还是没转过弯：“数据不就是文档吗？开发完补一补不就行了？”

“数据不是文档。”林涛说，“文档是容器，数据是内容。你刚才说'需求在共享盘'，那是文档的位置，不是需求本身。”

石军又问了一句：“那实现方案呢？画图、BOM、工艺这些，算产品数据吗？”

“算，而且是最重的一类。”林涛说，“八个方案，从技术方案一路排到运维方案。第 3 章说设计开发止于可实现，止的就是这里；方

案齐备，才算可实现的终点。制造全照着它造箱子，错一处，一万台箱子错一万处。”

石军盯着那面墙，第一次开始想这个问题。**产品是实物，数据是文档**——他干了几十年硬件，这个直觉从来没被问过。

一直没开口的产品开发经理赵刚接了一句：“第 3 章那张参数表，是我签的'通过'。当时我们比的是谁做得高，没问一句——它是不是 M3 真正要的东西。”

散会前，小张把自己那份清单按日期排好，推给旁边的小周。小周抽走最上面一页，两人都没说话。

本章要推翻的，正是这个直觉：你造的是数据，不是箱子。接下来的三块，把“产品数据”这棵树的根扎下去，再看它怎么被造出来，为什么决定产品质量，又怎么变成企业资产。

### 7.1.2 本章要回答什么

读完本章，你应该能回答：

1. 产品有几个“存在”？开发阶段你亲手造的是哪一个？
2. 什么是产品数据？它和文档、和配置项、和产品本身，分别差在哪？
3. 一份设计方案为什么必须能定位、能承接、能追溯？
4. 为什么说“开发产品就是开发产品数据”？这句话在什么范围内成立、什么范围不成立？
5. 产品的三种存在状态是什么？设计态为什么特殊？
6. 数据质量能决定产品质量吗？能决定到什么程度？
7. 用户和客户在产品数据里扮演什么角色？谁是“出题人”，谁是“判卷人”？
8. 职责该锚定在流程上，还是锚定在产品数据上？为什么？
9. 实物产品的实现方案包括哪些？为什么它最重？
10. 产品开发的流水线上，流动的是什么？缺陷为什么会沿流动放大？

11. 怎么判断一个活动值不值?
12. “数字化”和“转型”是一回事吗？差别在哪?

### 7.1.3 本章的骨架

本章要立的是“产品数据”这棵树。先把骨架立起来，后面三块都是给它添血肉。

> **开发产品就是开发产品数据**：产品有一个信息存在（产品数据）；设计态里开发产品 ≡ 开发产品数据；数据质量决定产品质量的上限；产品数据是组织协作的对象，也是企业最重要的资产。

本章的骨架是**一个判断、一横一纵、一个收口**。

**一个判断**——产品有两个存在，物理存在和信息存在。开发阶段你亲手造的，是信息存在（产品数据）；实物，是信息存在被“复制”出来的结果。第 6 章你学会把方案种成树，本章告诉你：那棵树，就是产品数据。

换个说法，这也是外化地图：每个人脑子里都有一张地图（第 1 章 §1.4.4 的四层：表征、图式、心智模型、解释框架）。产品数据就是那张图从个人脑子里搬出来、变成组织能共用的那一份。开发产品，说到底是把心里的图，绘成大家都读得懂的图。

一横，说的是产品的三种状态：设计态（as-designed，纯数据）、制造态（as-built，数据＋实物）、使用态（as-maintained，实物＋实况）。设计态里没有实物，它的全部存在就是数据，所以开发产品，开发的就是产品数据。

一纵——产品数据往纵深走，最终落到企业的账本上。它是协作的对象、评价的锚点、问责时要指的那一份，也是最重要的资产。产品会死，数据体系活着。

本章的方法底座时刻，是第 1 章概念能力里的织关系：把“产品”“产品开发”“产品数据”“两域三态”“质量”“资产”织成一张网，让每个概念各就其位，而不是把它们当成一堆词堆在一起。

本章还有四样值得先立起来：实现方案（八个方案，最重的一类）、流水线（流动的是产品数据，缺陷沿流动放大）、增值判据（活动值不值看数据是否增值）、数字化与转型（存起来 vs 用起来）。它们会依次立起来，最后收进企业的账本。

第一块把“产品数据”看清（它是什么、该是什么样子、含实现方案）。第二块把“开发产品＝开发产品数据”的边磨开（它成立在哪、质量机理如何、流动的是什么）。第三块看它怎么用（协作、价值、职责、增值、资产、数字化）。

---

## 7.2 产品不只是实物

> 这一块把“产品数据”看清：先看产品有两个存在，再拆产品数据的定义与六类，然后把四个最容易混的边界磨开——它不等于文档、不等于配置项、不等于产品、也不等于架构本身。

### 7.2.1 产品：物理存在与信息存在

石军盯着墙上的图纸，其实他盯的正是这个问题：M3 温控箱，到底“是”什么？

在制造车间里，M3 是一台台能插电、能制冷、能报警的实物箱子。可在开发部的电脑里，M3 是另一副样子：一份需求规格、一张

张图纸、一版版固件源码、一堆测试报告、一张 BOM 表。这副样子没有重量、不能制冷，但它决定了那台能制冷的箱子长什么样。

这就是产品的两个存在。

**物理存在**——你摸得到、用得了的那台实物；它在制造、使用中发生，会老化、会报废。

**信息存在**——描述这产品“是什么、为什么、怎么做、怎么证明对”的全部数据。它在开发、验证中发生，可以被复制、被传递、被留存。

石军几十年只跟物理存在打交道：图纸画完给车间，固件编完烧进芯片，剩下的事与他无关。所以他天然以为“产品 = 实物，数据 = 文档”。

可问题在于：物理存在的每一件，都是从信息存在里“复制”出来的。车间照着图纸造箱子，生产线照着 BOM 采物料，固件照着源码编译烧录。同一份图纸，可以造一万台箱子。

所以从开发的角度看，你真正“造”的东西只有一个：信息存在。实物只是信息存在的复制品。

> 产品 = 需求的物化 + 价值的载体 + 过程的输出；而这三样，全部以数据为形态存在。
>
> 术语注：这个“三视角一句话”是本书对工程界产品视角的凝练（三视角的出处见 §7.2.2：ISO 9000:2026 §3.7.9 产品；又，ISO/IEC/IEEE 15288:2023 §3.32 的产品定义与 §3.7.9 同文，其源注记为据 9000:2015 改），不是任何标准的原文；“以数据为形态存在”是本章的主张，它把三视角收束到“开发造的是信息存在”。

石军的直觉错在哪？他以为“开发”是对着物理存在工作。可事实上，开发全程没有一件实物可摸——你写需求、画图纸、编源码、跑测试、攒证据，全部是在数据上工作。直到移交制造，数据才第一次被“复制”成实物。

信息存在还有一个实物没有的特性：它不磨损、可复制、可传递。实物用一次旧一次、报废一件少一件；产品数据复制一万份不损耗、传递一百年不衰减。这也是为什么企业真正的资产是数据不是实物：实物会折旧，数据会增值。

### 7.2.2 产品：需求、价值与过程

在进入“产品数据”之前，先把“产品”这个词也钉一钉，否则“产品数据”会失去底座。

工程界看“产品”，有三个视角，合起来是一句话：

| 视角 | 产品是什么 | 一句话 |
| --- | --- | --- |
| 对需求 | 规定需求的实现物 | 需求是“应然”，产品是“实然” |
| 对顾客 | 价值交换的介质 | 价值从组织流向顾客的载体 |
| 对过程 | 开发过程的输出 | 产品是过程的结果物，不是过程本身 |

石军的 M3，三个视角都对得上：它是需求规格的实现物，需求说制冷到 -20℃，它就要能制冷；是给冷链客户的载体，价值靠它兑现；是设计开发过程的输出，图纸、固件、验证证据都是它的结果。

这里还有一条容易混的边界：**产品与服务的分界**。ISO 9000:2026 §3.7.9 给产品的定义是“组织的一种输出，这种输出可在组织与顾客之间**不发生交易**的情况下产生”（英文：output of an organization that can be produced without any transaction taking place between the organization and the customer）。§3.7.10 给服务的定义是“**组织的**、至少有一项活动必须在组织与

顾客之间进行的输出”。分界的关键词是 transaction（交易）。而按 §3.7.8 注 1，一项输出算产品还是算服务，“取决于所涉特性的主导性”；管理口语里两者才并列。

把“产品”钉住之后，“产品数据”就清楚了：它是产品的信息形态，三个视角的每一份记录。

再顺带把“产品”和它最容易混的几个词分开：

| 概念 | 判据要点 | 与产品的关系 |
| --- | --- | --- |
| 服务 | 至少一项活动必须在组织与顾客之间进行（ISO 9000:2026 §3.7.10） | 与产品由“所涉特性的主导性”分开（§3.7.8 注 1） |
| 系统 | 由部分组成或元素的安排，它们合起来呈现出各构成部分所不具有的某种既定行为或含义（ISO/IEC/IEEE 15288:2023 §3.46） | 产品通常是系统，但系统不必然可交付 |
| 项目 | 创造独特产品、服务或成果的临时性工作 | 创造产品的过程载体 |
| 结果 | 某事的后果；可取输出（output）的形式，也可取成果（outcome）的形式（ISO 9000:2026 §3.7.1） | 产品的“目的端” |

石军嘴里的“M3”，同时是产品，可交付；是系统，有元素与既定行为；是项目，一个开发它的临时组织；是结果，冷链运输能力。四个词四个层面，别混成一团。

还有一层经营视角值得点一句：在集成产品开发（IPD）里，“产品”常指产品包（offering），即硬件、软件、服务与解决方案的组合交付，多了一个商业维度。技术视角回答“产品是什么物”，经营视角回答“顾客愿意为什么付钱”，两个视角不冲突。

### 7.2.3 产品数据：形式化的信息表示

“信息存在”这个词太文绉绉。工程界有更精确的名字：**产品数据（product data）**。

ISO 10303-1（STEP，产品数据表达与交换标准）给的定义是：

> **product data**: representation of information about a product or a family of products in a formal manner suitable for communication, interpretation, or processing by human beings or by computers——关于产品或产品族的信息的、以形式化方式所作的表示，适合由人或计算机进行通信、解释或处理。
>
> 版本与出处说明：定义句引自 ISO 10303-1:2021 §3.1.29。早期版本的定义还强调一个特征——“与产品整个生存周期相关”：不只开发期，从概念到退役，凡是描述产品的信息都是产品数据。

拆开看，四个要点：

第一，“关于产品的信息”。产品数据是产品的信息形态，是描述被描述者的那一层，不是产品本身。

第二，“一种表示（representation）”。数据是内容，不是载体，载体可换。

第三，“形式化”。表示是形式化的，有确定的结构与语义，才能被一致地解释：同一棵设计树，你读、我读、机器读，读出的是同一个意思。这是第 6 章那套机械判据（单承接、覆盖公式）能成立的前提：比对对象必须先是形式化表示。

第四，“由人或计算机处理”。产品数据既给人读，也给机器处理，这一项是第 6 章人机分工的根据：人类据表示制定规则，算法据规则执行设计动作与机械核对。

产品数据到底包括哪些？工程实践里大致分六类：

| 类别 | 典型内容 |
| --- | --- |
| 需求数据 | 需求规格、利益相关方需要、验收准则（含操作目标清单——系统操作目标的可判定集合，需求侧编制、业务方确认，见第 4 章） |
| 设计数据 | 图纸、CAD 模型、架构描述、设计规范 |
| 制造数据 | 工艺文件、BOM、NC 程序 |
| 质量数据 | 测试报告、验证证据、缺陷记录 |
| 服务数据 | 维护手册、备件清单、故障数据 |
| 管理数据 | 变更记录、评审记录、基线标识 |

六类数据，覆盖产品从“该做什么”到“做对了吗”到“用起来怎么样”的全程。石军的“M3”，在这张表里占了一整行，可他从没把它们当成同一个东西。

> 术语注：六类数据里的“架构描述”是设计数据的一类，其定义承 ISO/IEC/IEEE 42010:2022 ——本书把**架构**凝练为“系统不可还原的涵义”（第 10 章展开）；**架构描述（AD）是它落纸的工作产品**（§3.3：An AD is an expression of an architecture）。

六类数据不是平起平坐的。其中设计数据 + 制造数据合起来，就是常说的实现方案：实物产品按“止于可实现”的说法具体化为八个方案，§7.2.6 专门展开。这里先辨析一句：八个方案里含测试、操作、

运维方案，其产出归质量数据与服务数据。八方案是“实现过程要交的方案”，六类是“沉淀下来的数据”。两把尺子按“方案 vs 数据”分开，不重复。

顺着 ISO 10303 的定义，把底层的词也拆一层：

- 对象（object）：任何可感知或可想象的事物（ISO 9000:2026 §3.5.3）——那台 M3，或它的图纸；
- 数据（data）：关于对象的事实（§3.8.1）——“制冷 −20℃”“固件 v2.4”；
- 信息（information）：有意义的数据（§3.8.4）——把“固件 v2.4 在 3 月 15 日烧录”组织起来，才有意义。

三层一叠，“产品数据”就精确了：它是有意义地组织起来的、关于产品的事实，也就是产品信息的形态。

再往下一层，术语学还分了“属性”与“特性”两支，别混：**property（属性）**是对象的特征（ISO 1087:2019 §3.1.3），**characteristic（特性）**是属性的抽象（§3.2.1）；而 ISO 9000:2026 §3.10.1 的 characteristic 译作“区别性特征”，是质量管理语境下的另一套用法（distinguishing feature，可固有、可赋予）。attribute（属性）是属性的命名化形式（ISO/IEC 2382-17:1999 §17.02.12，实体的命名属性）。两套语境不可互推：一个走术语学，一个走质量管理。本书凡说“固有特性满足需求”，用的是后者；而“特性的抽象／属性的命名”这一支，本章只在辨析时点到。

六类数据再展开一层，每一类都有“从哪来、到哪去”：

- 需求数据：从利益相关方需要转译而来，流向设计——它的质量决定“做的东西对不对”；
- 设计数据：从需求数据物化而来，流向制造——它的质量决定“方案好不好”；

- 制造数据：从设计数据复制而来，流向实物——它的质量决定“实物准不准”；
- 质量数据：从验证活动取证而来，流回需求与设计——它的质量决定“怎么证明对”；
- 服务数据：从使用现场回流，流回下一代设计——它的质量决定“产品能不能变好”；
- 管理数据：从变更与评审记录而来，贯穿全程——它的质量决定“怎么说得清”。

### 7.2.4 产品数据不等于什么

把六类数据放上桌，第一件要磨的边，是“产品数据不等于文档”。

**数据是内容，文档是容器。**同一份需求数据，今天在 Word 里、明天在数据库里、后天在墙上——容器换来换去，数据还是那份。反过来，一个 Word 文件里可能装着需求数据、设计说明、评审记录三样东西：一个容器，装着多份数据。

石军说“数据不就是文档吗”，错就错在把内容当成了容器。文档只是产品数据最常用的载体，不是产品数据本身。

**产品数据不等于配置项。**配置项是“被纳入配置管理范围的实体”（第 13 章会细讲）。产品数据是全部信息，配置项只是其中受控的那部分——被基线化、被变更控制的才是，草稿不是。六类数据里，只有被“管起来”的才算配置项。

产品数据不等于产品。描述与被描述者，本来就是两个东西。但验证要求两者一致——配置审计查的就是“数据说的”和“实物做的”对不对得上。石军说“图是图、箱是箱”，对；但他没意识到，箱子的好坏，先由数据的好坏决定，这一点第二块专门讲。

产品数据也不等于架构本身。**架构是那份内容，不是交付物**——它是系统“少了它就不是它”的那组基本概念或属性与管控原则（第

10 章专讲）；能进清单、能冻结基线的，是架构描述（AD）。它是设计数据的一类，与设计描述（DD）平级。

把架构当成交付物，是把没有形态的东西当成了可交付的**工作产品**。ISO/IEC/IEEE 15288:2023 §3.6 说，人工制品是“在项目期间产生并用于捕获和传递信息的工作产品”。

第 6 章“框架定义不是架构定义”，讲的是同一条线的另一半。

先把这句话钉在这：产品数据，是产品的信息形态，也是开发工作的全部对象。石军说的“把产品做出来”，正确翻译是“把产品数据造全、造准、造到可制造可验证”。

把几个词排一排，关系就清楚了：

- 产品是“物”；
- 产品数据是“物的信息”（全部）；
- 配置项是“被纳入配置管理范围的实体”（受控的那部分）；
- 文档是“装信息的盒子”（一个容器）；
- 架构描述是“架构的落纸件”（设计数据的一类，可冻结基线）。

几个词几层，石军以前全混成“产品 + 文档”两个词；第 15 章配置管理会用上这套区分。

### 7.2.5 产品数据该有的样子

第 6 章你刚学会一件事：设计方案不是一篇散文，是一棵树，干枝（框架定义）管结构，叶节点（详细定义）管内容。

现在把两章接起来：**那棵树里的每一样，都是产品数据**。

一棵树里有什么?干枝上有“结构是什么、承接了谁”（框架定义），叶节点上有“每个模块具体做什么、指标多少、验收标准是什么”（详细定义）。分配表把叶节点搬去下一棵树；符合性分析记录每片叶子接

没接住上游。这些东西，全部落在 §7.2.3 的六类数据里。设计数据、需求数据、质量数据，没有一样是“实物”。

所以第 6 章的树，不是“文档结构”，是**产品数据的结构**。系统架构师罗建把方案写成六十页散文，问题不止在“不好查”，更在“那不是产品数据该有的样子”。产品数据必须可定位、可承接、可追溯，树形正是为这三样而生。

可定位——每条内容有编号、有位置，说得清“它在哪”。可承接——上一份的叶节点落到明确的下游节点，说得清“谁接住了它”。可追溯——从任意一片叶节点往回走，走得到它是从哪条需求来的。散文式的写法，一条也做不到：你没法在六十页里指认“这条需求落在哪一段”。

这一层接上之后，后面的话就好说了。既然设计开发造的就是产品数据这棵树，那么评审、验证、确认（下一章）查的就是这棵树。模块工程师（第 9 章）种叶节点，模块架构师种干枝，通常就是模块工程师自己；系统架构师（第 18 章）种架构描述与决定。配置管理（第 15 章）管的，是让这棵树“一直在变还说得清”。

这棵树往下长，就是下一节的实现方案：树是产品数据的骨架，实现方案是树落到车间里的样子。

### 7.2.6 实现方案：最重的产品数据

§7.2.3 埋下的**设计数据 + 制造数据 = 实现方案**，这一节展开。它是产品数据里分量最重的一类——其他数据定义“该做什么、怎么证明对”，实现方案定义“怎么做”。

实物产品的实现方案，按设计开发“止于可实现”这条界，是**八个方案**，每个方案 = 给一个下游环节的“可实现需求”：

| 方案 | 给谁 | 内容 |
| --- | --- | --- |
| 技术方案 | 客体本身 | 结构、模块划分、接口关系、关键技术指标 |
| 物料采购方案 | 采购 | 物料清单（BOM）、规格、合格准则 |
| 制造方案（工艺） | 生产 | 工艺路线、工序卡、工装、NC 程序 |
| 测试方案 | 检验 | 测试计划、接收准则、验证用例 |
| 交付方案 | 交付 | 包装、运输、交接要求 |
| 部署方案 | 实施 | 安装、调试、上线要求 |
| 操作方案 | 用户 | 操作手册、使用场景 |
| 运维方案 | 维护 | 维护规程、备件清单、故障处理 |

八个方案的本质，是一个东西的八个面向，不是八件独立的事。“规定需求”按下游环节分配下去，采购照着买、生产照着做、检验照着测、实施照着装、用户照着用、维护照着修。第 6 章那棵树的叶节点，落到制造环节就是这一整套方案；**方案齐备 = 可实现 = 设计开发的终点**（第 3 章讲过）。

把“分配”落到物料这一层，物料的需求就现形了：它是产品需求经设计开发转化到物料层的“规定需求”，落纸形态就是物料采购方案里的规格 + 合格准则。这份需求不是采购自己定的，是设计开发分配给采购的。

设计开发止于“可实现”（第 3 章）。物料需求落在信息域（产品数据），物料实物落在物理域。来料检验（IQC）正是两域之间的对照动作：拿信息域的规定需求，量物理域的实物特性。

定制件还多一层。物料需求往下游走，是采购与验证的输入，它是产品设计开发的输出；往上游走，又是供应商那一级设计开发的输

入。供应商拿它再转化一轮，输出更细的物料规格与工艺。每一级元素都走一遍完整的设计开发过程。这就是设计开发的递归。

> 术语注：六类数据与“八方案”是本章按第 3 章“止于可实现”口径对“实现方案”的归纳与操作化，不是任何标准的清单。八方案的名目与“方案齐备即终点”的判据，依设计开发概念研究报告（01 号）§4.3；其条款落点在 ISO 9001:2015 §8.3.5、§8.4、§8.5.1。标准层面只有 ISO 10303-1 的“产品数据”与 ISO 9000:2026 §3.3.11 的“设计开发”定义。八个方案可按你们产品的下游环节增删（退役与回收属 ISO/IEC/IEEE 15288:2023 生存周期末段，增列即九个方案，其需求侧见 §5.2.5）。

拿石军的 M3 二代落进这张表：技术方案里是结构布局和接口契约，物料采购方案里是 BOM 和压缩机选型，制造方案里是注塑工艺和固件烧录，测试方案里是制冷性能试验大纲。实现方案不是一张图纸，是八份方案合起来的整套产品数据。

为什么它最重？因为制造环节完全照着它复制——错一处，一万台箱子错一万处。第 6 章说“方案不是凭空想出来的，是骨架分解出来、候选构思出来的”，本章补一句：实现方案，是产品数据里最重的一类。

软件系统（比如恒信）的实现方案呢？一样是产品数据，只是它没有独立的制造环节，实现方案几乎就是产品本身（§7.3.4 展开）。

**概念卡片 · 产品数据**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 关于产品或产品族的信息的、形式化的表示，适合由人或计算机处理——产品开发每一条输出都是产品数据；对立面：产品本身（物理存在），描述与被描述者 |
| **它不是什么** | ⑦⑧ | 辨析：≠文档（数据是内容、文档是容器）、≠配置项（受控的那部分才是）、≠产品（实物）、≠架构本身（可交付与可冻结的是架构描述 AD）；误区："数据是开发完再补的文档"——数据是开发工作的全部对象 |
| **它从哪来** | ②③⑨ | ISO 10303（STEP）为数据交换而定义；product data ← product（产品）+ data（数据）；与 ISO 9000:2026 §3.10.10"配置信息"同族 |
| **它往哪去** | ④⑤ | 设计态里开发产品 ≡ 开发产品数据；协作的对象、评价的锚点、问责的落点、企业最重要的资产 |

## 案例进展①｜"BOM 和图纸，对不上"

M3 二代的实现方案出来两周，制造的小周来找石军。

"石军，你这份 BOM 和装配图对不上。"小周把两张表摊开，"BOM 里压缩机写的是 A 型号，装配图上画的是 B 型号——底座孔位都不一样。我按图纸备了料，车间一装就装不上。"

石军翻了半天："图纸是设计那边出的，BOM 是工艺那边配的……两边各干各的。"

“所以实现方案不是一份，是好几份拼起来的。”林涛在旁边说，“技术方案、图纸、BOM、工艺、固件、测试，它们是一个整体——同一台 M3 的实现方案。任何一层对不上，制造就照着错。”

石军第一次意识到，他以为“图纸”就是实现方案，其实实现方案是一整套产品数据：技术、物料采购、制造、测试、交付、部署、操作、运维，八个方案，图纸只是其中一层。BOM 错、工艺错、固件错，跟图纸错一样致命——因为制造全照着它来。

---

## 7.3 产品从定义到复制

> 这一块把“开发产品 = 开发产品数据”磨开：先看产品开发这件事本身（定义与复制），再看产品在域与态里的存在方式，划出那句话的成立范围，看软件为什么特殊，讲数据质量与产品质量的关系，最后把流水线放进来——流动的正是产品数据。

### 7.3.1 开发止于可实现的需求定义

进这一块之前，先看“开发”这个动词本身，不然“开发产品就是开发产品数据”就立不住。

ISO 9000:2026 §3.3.11 给设计开发（design and development）的定义是：

> **design and development**: set of processes that transform requirements for an object into more detailed requirements for that object——把某一客体的要求转换为该客体更详细的要求的一组过程。

这里要紧的是：引文里的“要求”就是本书的**需求**——开发交出来的仍是需求，比输进来的更详细，**细到可实现为止**；不是“实物”。产品开发的直接产物是产品的定义与基线，生产环节才把它复制成实物。

顺着这个定义，产品开发有三个特征，正好解释石军的 M3：

第一，一次性定义，多次复制。开发定义基线，生产复制基线。开发高风险、高成本、低重复：定义错了，后面全错；生产高重复、低成本：复制对了，可以无限造。石军花两年定义 M3 二代，车间用两周造出第一批：定义和复制的代价，差着两个数量级。

第二，跨职能协同：市场、研发、制造、采购、财务同时参与。不是研发画完图丢给车间，而是需求、设计、制造、验证的数据在多个职能之间接力。

第三，以评审为门：每个阶段出口用评审把关，防止错误向后传递放大；第 8 章的三个看门人，就是站在这些门上的。

一句话收住：产品开发是把“应然（规定需求）”变成“实然（产品）”的系统化过程；这个过程的每一步，都落在数据上。

开发的输入是市场需求与已有基线，输出是产品工件与产品规范、基线（baseline）。开发在“不断逼近并冻结一组基线”，而基线就是一组受控的产品数据。ISO 9000:2026 §3.10.12 说，基线是经批准的配置信息，确立**产品或服务**在某一时点的特征、作为产品或服务整个生存周期各项活动参照；第 15 章配置管理会接上。

这条链上有一处容易漏的接驳：从“需求定义”到“设计定义”之间，还夹着一个架构定义过程（ISO/IEC/IEEE 15288:2023 §6.4.4 system architecture definition process 与 §6.4.5 design definition process）。架构定义的输出，正是设计定义的输入——它的身份从“成果”转为“规定需求”。省略这一环，链条就不完整。（第 10、11、16 章分别展开。）

开发产品的全过程，需要的技能正好对应数据六类：

- 需求工程 → 需求数据；
- 架构与设计 → 设计数据；
- 工艺与制造工程 → 制造数据；
- 评审与验证 → 质量数据；
- 跨职能协同与市场洞察 → 服务与需求数据；
- 项目管理与配置管理 → 管理数据。

一个团队能不能开发好产品，就看成这六项能力对应的数据，造得全不全、准不准、可不可追溯。石军的团队，硬件和固件是强项，需求工程和配置管理是弱项，所以 M3 二代在“功能基线”和“数据断层”上翻车。

### 7.3.2 两域三态：产品的存在方式

产品数据不是抽象概念，工程实践给它记了两笔账：它在哪个域，又在哪个态。

两个域是信息域和物理域。

信息域里发生的是开发与验证：需求被写下来、设计被画出来、验证被记录下来。物理域里发生的是制造与使用：实物被造出来、被用起来、被修起来。两个域之间靠映射连着，工程上把它叫数字孪生。

> 术语注：本章“数字孪生”只指“信息域的模型跟上物理域的实况”这一层意思，不作展开。该词**无通行的英文对应写法**；本书只用它指这一层意思，**不引作论据**。

三种状态（PLM 视角）是产品在生存周期三个阶段的不同形态：

| 态 | 数据内容 | 产品 = |
| --- | --- | --- |
| 设计态 as-designed | 需求、模型、规范、**EBOM**（工程 BOM） | 数据（还没有实物） |
| 制造态 as-built | **MBOM**（制造 BOM）、序列号、批次、制造记录 | 数据 + 实物 |
| 使用态 as-maintained | 维护记录、配置实况、现场变更 | 实物 + 实况 |

注意三种状态里的产品越来越重。设计态里产品 = 纯数据。一台 M3 二代还没诞生任何实物，全部存在就是那套需求、图纸、源码、验证计划。制造态里产品 = 数据 + 实物。车间照图纸造出第一台，箱子还连着 MBOM、序列号、制造记录。使用态里产品 = 实物 + 实况。维护记录、配置实况、现场变更反过来成为新数据。

这三态是行业通行的 PLM 视角。

石军的直觉错在只认制造态和使用态：他眼里产品就是实物，但开发工作全部发生在设计态，那里只有数据。

再把“开发产品 = 开发产品数据”用三个视角检验一遍，产品数据 = 产品的信息本质，成立；开发的直接产物 = 产品规范与定义，成立；产品 ≠ 数据，这一条也成立：产品还有实物、体验、价值实现。合起来就是本章反复说的那句话：开发产品开发的是产品的“定义”，制造产品制造的是产品的“实物”；“开发 = 定义生成”成立，“产品就是数据”不成立。

石军在冰链看到的“云端追溯平台”，本质就是两个域之间映射的一个雏形。但只有使用数据回流、设计数据实时更新，它才真正跟得上现实，否则只是一个存数据的仓库。

管这套追溯平台的吴浩在旁边补了一句：“追溯数据就是产品用起来之后往回吐的那一环——箱子把温度实况吐回云端，平台收进

来、存成可查的记录。它要作数，先得采集可靠：现场断一秒，追溯就缺一块，缺了块，整条链都不完整。”石军听到这里愣了一下：连使用期的实况，也是产品数据。

### 7.3.3 设计态里，开发就是定义生成

现在可以给本章标题那句话划线了——那句话的全称是**开发产品就是开发产品数据**；本节就给它划成立的边界。

> 术语注：这里的“同一性”（**设计态同一性**，开发 ≡ 定义生成）与“同一性”（**同一性目的论**，系统是谁由目的构成、演化中仍是它自己；见附录 A T-144）是同一个词的两个概念。前者讲“开发这个动作和生成定义是同一件事”，后者讲“系统是谁跟着目的走”，别混。

**在设计态，开发产品 ≡ 开发产品数据，没有歧义。**因为设计态里产品没有物理实体，它的全部存在就是那组受控的产品数据。需求数据定义“该做什么”，设计数据定义“怎么做”，质量数据定义“怎么证明对”。数据完备到“可制造、可验证”，开发才算完成。

这一句为什么重要？因为它把“开发”的实质钉死了：开发不是“做东西”，是“生成定义”。但这句话有两个边界，必须划清：

第一，只在设计态、对纯信息产品严格成立。制造态里有实物环节：制造偏差、材料、工艺、体验，不成立；物理域的一切，数据管不着。严格的说法是“开发 = 定义生成”。

第二，“开发产品 = 开发产品数据”不等于“产品就是数据”。产品还有实物、还有体验、还有价值实现。数据是产品的信息形态，不是产品本身——§ 7.2.4 刚划过的边界，这里不收回。

一句话：石军以为自己是在“做东西”，其实他是在“把定义造全”。

这条同一性不是一句口号，它有实操意义：它决定你把力气花在哪。既然开发 = 定义生成，开发的管理重心就该放在数据上：评审数据、积累质量数据、管住数据基线，而不是盯着“样品有没有做出来”。石军团队以前考核看“样机出来没”，现在该看“数据齐不齐、准不准、可不可追溯”。样机只是数据完备性的一个旁证。

### 7.3.4 软件没有独立的制造环节

有一类产品，开发与制造之间的那道界几乎不存在：软件。

对硬件，开发和制造是两件事。开发产出的是产品的定义：需求、图纸、BOM、工艺。制造照这份定义把实物做出来，定义写得全不全、对不对，直接决定做出来的箱子对不对。这就是“数据质量决定产品质量”的机理。

软件没有独立的制造环节：你写的源码、编译出的程序，本身就是最终交付物。开发就是生产，“定义”与“实物”合成一步。这就是软件迭代能快到按天算、硬件要按版本爬坡的原因：硬件每改一次都要重新打样、试产、验证，软件直接改定义本身。

软件和硬件的数据特征还能再对比一列：

| | 软件 | 硬件 |
|---|---|---|
| 开发与制造 | 合成一步（编译即交付） | 两件事（定义与复制） |
| 制造环节 | 无（编译即交付） | 有（打样、试产、量产） |
| 修改成本 | 改定义（低成本） | 改定义 + 重做实物（高成本） |
| 版本节奏 | 按天迭代 | 按版本爬坡 |

石军做硬件，最深的体会是“改一次固件，要重烧、重测、重验证”。软件没这个负担，但反过来也更容易“改着改着忘了改了什么”。所以软件的配置管理，比硬件更依赖产品数据的严谨。

### 7.3.5 产品质量的上限由数据质量决定

石军还有个朴素信念：数据是文档，质量在车间里，这句话对了一半。

对的一半：**实物环节确实不在数据里**；制造偏差、材料批次、工艺水平，数据管不着。

错的一半：**开发阶段可评价的质量，就是数据质量**。设计态里没有实物，你能拿什么评判“产品好不好”？只能拿产品数据——需求全不全、设计对不对、验证有没有证据。石军说“质量在车间里”，可车间开动之前，产品的“质”已经由数据定下一大半了。

把两半拼起来，是一个**多环相乘**的模型：

> 最终质量 = 需求正确性 × 数据质量 × 制造质量 × 使用质量
>
> 术语注：这个“多环相乘”模型是本书对质量成因的凝练。它把第 3 章的质量定义（ISO 9000:2026 §3.5.2：对象的一组固有特性满足要求的程度）落到开发环节，把“质量是设计出来的”变成可相乘的式子，不是任何标准的公式，欢迎检验、欢迎校准。

每一环都是乘法，任何一环为零，全盘皆输；这个模型回答了石军的困惑：

数据质量决定产品质量的上限，不决定达成值。数据完美，最多把“能到的高度”抬到极限；但需求方向错了，验证全绿、确认全红，数据再好也没用；制造环节不达标，数据再准也复制不出好实物。

所以石军说“质量在车间里”，对制造环节成立，对开发环节不成立：开发环节的质量，就是数据质量。这就是“质量是设计出来的，不

是检验出来的”的真正含义——检验只能发现数据里的缺陷，缺陷早在数据里了。

把乘法链落到最具体的一环，一个常见误会就现形了：“合格物料是合格产品的前提”。严格逻辑上，它既不是充分条件，也不是对每一条要求的必要条件。

好料未必出好产品：装配错、工艺偏、设备磨，任何一环出错，好料照样出次品。反过来，个别物料超差，经让步接收（评审无害后放行）、规格过严（偏差无害）、或产品验收恰好测不到，产品仍可能合格。

必要的不是“每颗料都合格”，是“物料合格性被受控管理”：按接收准则验收、不合格不放行、让步接收留痕。这才是外部提供控制要管物料的理由（其条款见 ISO 9001:2015 §8.4）。合格物料，是合格产品的基础，不是保证。

数据质量本身怎么评？ISO/IEC 25012 的数据质量模型分十五项特性（据 42030:2019 表 A.3 转述）。本章取最要紧的五把尺子：准确性、完整性、一致性、现时性、可信度。五把尺子全是拿“数据”当对象，再一次印证：开发在数据上工作。

> 出处说明：该数据质量模型的载体是 ISO/IEC/IEEE 42030:2019 表 A.3（Table A.3 — ISO/IEC 25012 Data quality model），另见 ISO/IEC/IEEE 42010:2022 正文引用（data qualities as described in ISO/IEC 25012:2008, Clause 4 …）。第五把尺的中译按 42030 中英文对照版作“**现时性**”（Currentness）；产品数据研究报告（14 号）旧译作“时效性”，本章从定名。此处不写成“标准说……”。

五把尺子各举一个冰链的例子，就知道怎么用了：

| 特性 | 含义 | 冰链的例子 |
| --- | --- | --- |
| 准确性 | 数据反映真实对象 | 固件 v2.4 的记录，跟实际烧录的版本一致吗 |
| 完整性 | 数据覆盖无遗漏 | “边缘断链预判”的需求、设计、验证，都齐吗 |
| 一致性 | 数据内部不矛盾 | BOM 里的压缩机和装配图上的，是同一个型号吗 |
| 现时性 | 数据跟上现实 | 需求改了，共享盘里的文档更新了吗 |
| 可信度 | 数据经得起验证 | 验证报告是实测跑出来的，还是纸面写出来的 |

五把尺子一量，M3 二代的病就现形了：准确性和一致性，正是案例进展①里 BOM 与图纸对不上那件事。数据质量是能当场量的：这五把尺子一量一个准。

数据质量这五把尺子，正好呼应第 3 章的质量定义。数据质量，就是拿产品数据这组“固有特性”满足要求的程度；五把尺子是这个程度的具体化。第 3 章把质量钉在“程度”，本章补一句：开发阶段，这个程度全靠产品数据来量。

还有一个机理必须钉死，缺陷传播链：需求错 → 设计错 → 制造错 → 使用错，越往下游修复成本越高（早期修是 1，使用期修是 30~70 倍；此为业界经验数字，转引自 02 号报告《需求工程概念研究报告》§六，一手来源未核，读者可按自家数据校准）。这是乘法模型的必然，与惩罚无关。

### 概念卡片 · 两域三态

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 产品在信息域、物理域与设计态、制造态、使用态中存在的模型；对立面：物理存在（实物）——数据管不着的那一半 |
| **它不是什么** | ⑦⑧ | 辨析：三态是同一产品的三种存在方式，不是三个产品；设计态同一性 ≠ 产品就是数据；误区：“质量在车间里”——开发环节的质量就是数据质量 |
| **它从哪来** | ②③⑨ | PLM 与 STEP 的数据管理实践；as-designed ← 设计、as-built ← 制造、as-maintained ← 维护；数字孪生即信息域对物理域的实时映射 |
| **它往哪去** | ④⑤ | 划定“开发产品 = 开发产品数据”的成立范围；多环相乘模型决定质量上限；流水线视角（流动的是产品数据） |

### 7.3.6 沿工位流动的是产品数据

把产品开发看成一整条流水线，本章的话就全串起来了。

流水线上有三样东西：**工件**，被加工的物；**工位**，加工动作；**增值**，加工效果。制造流水线上，工件是钢铁零件，工位是机床，增值是零件越来越接近成品。产品开发的流水线上呢?

| 流水线要素 | 产品开发中的对应 |
| --- | --- |
| 工件 | 产品数据（需求规格、设计模型、验证证据……） |
| 工位 | 需求定义（转译）、设计（物化）、实现（实例化）、验证（取证）、移交（冻结） |
| 增值 | 每个工位往数据上添加价值，也可能添加缺陷 |

石军在制造流水线上干了一辈子，一眼就懂这个模型。但他从没意识到：他的开发流水线上，流动的从来不是实物，是产品数据。需求数据流进设计，设计数据流进验证，验证证据流进基线，一路变成受控配置。

流水线视角立刻显出一个关键差异：制造流水线和开发流水线有三处本质区别。

第一，工件性质：制造是实物，靠复制；开发是信息，靠原创。信息可以被复制一万次而不磨损，实物不行。

把第一点里的“工件”再往细里看：产品开发流水线的工件是产品数据，而设计开发这一站加工的具体对象，是规定需求。需求从利益相关方那里“进厂”，经分解分配加工成更详细的需求，一路传到验证当尺子。

它和制造流水线的物料最像，但有一个关键区别：物料进了成品就被消耗，需求在链条里始终有效。它既是每级加工的输入，又是逐级核对的标尺，更像图纸、不像原料。

第二，加工方式：制造是确定性复制，照图纸做；开发是创造性转换，把需求译成设计。所以开发每个工位都有判断、有取舍、有人的专业在里面。

第三，缺陷行为，这是最要命的一条：制造缺陷在单件上，开发缺陷沿流动传播并放大。需求里的一个错误，流进设计、流进实现、流进验证，越往下游修起来越贵；这就是上一节讲过的缺陷传播链，在这里沿流水线放大。

制造流水线的次品可以挑出来扔掉；开发流水线的缺陷已经长进产品数据里，跟着数据一路传到每个下游。

这一节把整块串成一条线：开发产品 = 开发产品数据（定义生成），数据沿流水线流动增值，缺陷沿流动放大。第 8 章的三关在关键工位检查数据，第 13 章的基线在关键节点冻结数据——两条防线，都是站在流水线上、对产品数据设的关卡。

流水线视角还给第 15 章埋了个位置：配置管理，就是这条流水线上的“标识 + 基线 + 记实”，给每个工件发身份（标识）、在关键节点冻结（基线）、记录流动轨迹（记实）。没有它，数据沿流水线流到哪、哪一版是准的，全都说不清。本章把“流动的是什么”讲清了，第 15 章把“流动的怎么管”接上。

这里的“流水线”和常说的“瀑布流程”不是一回事：瀑布是流程组织方式（阶段串行），流水线是数据流动视角（工件沿工位增值）。不管跑瀑布还是敏捷，产品数据都沿工位流动。流水线视角看的是“数据在不在流动、有没有增值”，不是“流程怎么排”。

## 案例进展②｜“二代功能基线，还是对不上”

开完会一周，冰链开了第一次 M3 二代设计评审。

林涛把六类数据铺在投影上，问石军：“我们一件件过。需求数据——'边缘断链预判'的验收标准，定了吗？”

石军翻出需求文档，指着一行：“定了。传感器异常时，箱体本地判定并报警，30 秒内完成。”

“这条需求拆到哪几个模块了？每个模块的接口契约呢？”

“拆到固件和传感器两级。”石军愣了一下，“接口契约……我们以前直接干，没写过。按第 6 章那套判据，需求数据的叶节点得在设计数据里接得上，可设计数据里没有这一条。”

系统工程师小张翻开桌上那份清单：“这一版的数据，下一段接得上吗？判据只有一条：需求里写定了本地判定 30 秒，设计里得有一条接口契约接住它，验证里得有实测数据证明它真做到了。”

季昭接过话：“那验证就做不了。验收标准在需求数据里，接口契约在设计数据里，这两层对不上，我的测试用例就没法定——我测'本地判定 30 秒'，可我不知道是固件负责还是传感器负责，也不知道哪个接口给我触发信号。”

吴浩也接了一句，他的云端追溯平台正好要存这条告警：“我这边一样断——箱子本地判完，告警还得回传云端落记录，追溯平台才知道那一刻出了什么。接口契约没定，我不知道存哪个字段、由谁触发，这条记录就写不全。”

会议室安静了三秒。石军第一次看到，他做了一辈子的事，原来是**一层一层的数据接力**：需求数据接住客户需要，设计数据接住需求数据，质量数据接住设计数据。任何一层断，产品就断。

“所以你说开发产品就是开发产品数据……”石军慢慢说，“我明白了，不是补文档，是**把每一层的定义造全、造准，让下一层接得住**。”

“对，这就像一条流水线。”林涛点头，“需求数据流进设计，变成设计数据；设计数据流进验证，变成验证证据。每一站都往数据上

加价值，也可能加缺陷。第 15 章的基线就冻结在这条线上。你以前只盯着最后的实物，其实你该盯的是这条线上流动的数据。它哪一站断了，产品就在哪一站断。”

石军看了看投影上那六类数据，忽然觉得，自己干了几十年的东西，第一次有了一张完整的图。

---

## 7.4 产品数据能装下什么

> 这一块看产品数据往哪去：先看它是组织协作的对象，再看它能装下多少价值，然后把职责挂到工件上，最后看它变成企业资产——产品会死，数据体系活着。

### 7.4.1 利益相关方围绕产品数据协作

产品数据不是开发部自己的事：协作落到工程里，就是一张名单，谁读什么、谁写什么。

说“协作”之前，先把参与者钉一钉。ISO/IEC/IEEE 15288:2023 §3.44 给利益相关方（stakeholder）的定义是：

> **stakeholder**: individual or organization having a right, share, claim, or interest in a system or in its possession of characteristics that meet their needs and expectations——对系统、或对其具备的满足自身需要和期望的特性享有权利、份额、主张或利益的个人或组织。

用户和客户都在其中，他们是 needs and expectations 的所有者。

内部利益相关方（市场、研发、制造、质量、采购、项目），是围绕产品数据**读写协作**的人。大家不围绕“目标”协作，也不围绕“计划”协作，而是围绕**同一份产品数据**协作。

把内部利益相关方的读写列细一点，就是一张协作分工表：

| 利益相关方 | 读 | 写 |
| --- | --- | --- |
| 市场 | 需求数据、使用数据 | 洞察、需求 |
| 研发 | 需求数据、接口规范 | 设计数据、实现产物 |
| 制造 | 设计数据、工艺 | 制造数据、BOM |
| 质量 | 设计数据、验证计划 | 验证证据、缺陷 |
| 采购 | BOM | 供应商、物料数据 |
| 项目 | 全部 | 进度、管理数据 |

每一格都是一份产品数据的读写。协作的真相就藏在这张表的每个格子里：石军以前觉得“协作 = 开会”，其实协作是数据的读写，会议是数据对齐的形式，数据才是协作的实质。

外部利益相关方（用户、客户），是产品数据的源与宿：数据从这里进来，价值在这里兑现。

用户是使用产品的人（使用价值、验收裁判）；客户是接收产品或服务的**个人或组织**（交易价值、需求源头，ISO 9000:2026 §3.9.1；该条 EXAMPLE 里的 purchaser 即“购买者”一例）。买奶粉的是客户，喝奶粉的是用户——一句口诀，两个角色。

> 术语注：“客户”是本书行文用词；ISO 9000:2026 §3.9.1 的中译定名为“**顾客**”，两词所指同一（该定义句也含“个人**或组织**”，不止自然人）。本书各章统一用“客户”，读者读标准时按“顾客”对接。

用户与客户通过四条路径贡献并提升产品数据：

| 路径 | 机制 | 产物 |
| --- | --- | --- |
| 需要（源头） | 痛点与期望经代理转译 | 需求数据（规定需求） |
| 使用（新数据） | 遥测、日志、行为 | 使用数据（使用态的一部分） |
| 反馈（修正） | 缺陷报告、变更请求 | 变更与缺陷数据 |
| 验收（把关） | 确认“是否满足需要” | 校正需求数据 |

有一条精确的表述：产品数据的内容由用户与客户给出，形式由内部利益相关方写出。除了“使用数据”是用户直接产生的，其余三条都是间接但根本的贡献。

石军现在能看懂冰链的业务了：疾控中心（客户）的审计需求，是需求数据的源头；冷链车上的箱子（使用），每小时吐回温度实况，是使用数据；现场反馈的故障，是缺陷数据。用户与客户不写答卷，但出题和判卷都是他们。

不过这条闭环有两处必须标清的边界。

第一，需求数据链只承载“明示”的部分。从需要到规定需求，是一条按派生关系生成的链，它只承载 stated 的部分（ISO 9000:2026 §3.5.1 注 2，规定的需求是明示的需求）；而确认的判据与方针不在链上：它们不是从需要派生出来的，须另设一层与派生链异构的参照。（第 5 章、第 8 章分别展开。）

第二，转译有损耗。期望的集合严格大于需求的集合：利益相关方说的、写下来的、下游读到的，逐级都会掉东西。所以“需要进来了”不等于“需求就对了”。

把三个角色放进去收个尾：产品数据是大家对齐的那一份东西，评审是当场对表，配置管理是版本把关。评审的人不自己动手改数据，但所有人对着同一份数据对齐。配置管理不参与内容判定，但确保没

人看错版本、改错条目。人可以换，那份数据不变，这就是产品数据作为协作对象的意义：它让组织不依赖任何一个人的记忆。

把四条路径和内部利益相关方的读写合起来，就是一个完整的产品数据闭环：需要进来（需求数据）→ 开发转译（设计数据）→ 验证取证（质量数据）→ 交付使用（实物 + 实况）→ 反馈回流（使用与缺陷数据）→ 反哺下一代。闭环每转一圈，产品数据就厚一层。“数据体系活着”，正因为它不是静态仓库，而是一直在转的闭环。

### 7.4.2 价值三层：哪一层落进了数据

上一节说产品数据是协作的对象，有人会接一句：“那产品数据的价值，就是全部价值？”不是。这里要磨一道更细的边。

利益相关方对产品的贡献，沉淀下来分三层：

| 层 | 内容 | 是否在产品数据中 | 如何评价 |
| --- | --- | --- | --- |
| 物化层 | 需求、架构描述、实现工件 | 完全在 | 可证伪、可度量 |
| 决策层 | 取舍、转译、选型判断 | 部分沉淀（ADR） | 评审（事中）+ 结果回溯（事后） |
| 过程层 | 沟通、反馈、评审、影响力 | 结果记入 | 下游指标 + 复盘 |

前两层好懂。**决策层**是重点：架构师拍板“用微内核”，这个判断本身不在产品数据里，它的结果（架构描述、ADR）沉淀下来了，但判断的“为什么”只留了一半。**过程层**更飘：一次评审会上谁说服了谁、谁的影响力推着方向走，这些根本没有落成数据，只留下“评审记录”这一行。

决策层里有一条值得单独看：**架构决策记录**。它跟其余产品数据一样，是设计态的纯数据——不产实物，只产两样：一是**约束数据**，指下游必须依据的内容；二是**理由数据**，记当初为什么这么约束。它在本章那张数据图谱上还充当一**跳**：把关切与需求连到架构元素上。少了这一跳，需求与元素之间的关系只能靠人脑补。

用数据质量五尺量它，最容易坏在两处：**只写结论、不写依据**，坏在**可信度**；**把关切与需求的原文抄进来，而不是连过去**，坏在**一致性**与**现时性**——上游一改，记录就对不上了。

所以三层模型给出一个重要的修正：**产品数据是价值的“投影与锚”，不是价值本体**。

工件是判断的投影：一份完美的设计文档，可能是错误决策的产物，比如验证全绿、确认全红，第 8 章会细讲。

这个修正不是否定产品数据，而是给它定位：数据是锚，让人能评价、能追溯、能问责；但价值真正的源头，是人的判断与协作。石军的价值不在他画的那张图里，而在画图时的判断里；可没有那张图，他的判断就无处锚定。

顺着这个定位，评价一个角色就有了完整式子：角色价值 = 工件质量 × 判断质量 × 下游影响（乘数）。前两项好理解；第三项最容易被忽略——架构师的一个好决策，让下游一百个工程师少走弯路，他的价值体现在别人的工件质量上。

> 术语注：这个式子（工件质量 × 判断质量 × 下游影响）是本书给出的评价口径，不是任何绩效标准的公式。它是“价值三层”的操作化，作用是把“角色价值”从印象变成可核对的量纲。

顺着这三层，有一句话值得记住：隐性知识写不进数据。决策层与过程层的价值，能沉淀进数据的只有一小半。

所以评价一个角色，要三维一起量：只量物化层（工件），会漏掉决策与过程的价值；只夸判断，又没有锚点可考；而三维都落在产品数据上（§7.4.3 那张表）。

### 7.4.3 职责为什么挂在工件上

职责可以挂在不止一个地方，但能被问责的只有一个。

两种候选：挂在**流程**上（角色 × 活动），还是挂在**产品数据**上（角色 × 工件）。答案是后者，三条理由：

**数据职责可证伪**——工件可以对照规定需求检查，对不对一目了然；活动无法证实也无法证伪。

数据职责可映射——权限、签字、审批链、考核指标，四样硬机制都能挂到工件上；流程没有硬机制。

数据职责可度量：缺陷率、覆盖率、追溯完整率，都拿数据说话；流程只能评执行率（形式化）。

反证更锋利：缺了流程，数据职责还在，只是降级可运行；缺了数据，流程职责空转，数据无人负责。工程佐证随手可得：GitHub 的 CODEOWNERS，就是把“每份代码谁负责”写进数据。

所以职责的完整定义是：角色 ×（活动 × 工件），而锚点在工件，不在活动。落地三步法：

1. 识别工件——列出配置项清单：需求规格、架构描述、设计模型、验证证据、基线；
2. 指定 owner——每个配置项唯一 owner（一件只一人，一人可多件）；
3. 挂机制——工具权限 + 评审签字 + 变更审批链 + 考核指标，全部挂到 owner 上。

职责还有三种类型，别都压成“负责”：

| 职责类型 | 依据 | RACI 对应 |
| --- | --- | --- |
| 所有权职责（主） | 产品数据 / 配置项 | A（Accountable） |
| 执行职责（辅） | 流程 / 活动 | R（Responsible） |
| 决策职责（辅） | 评审门 / 基线 | Approve |

三类职责里，所有权是主——挂在产品数据上，可问责、可考核；执行和决策是辅，挂在流程和门上。

一句口诀分清主次：以产品数据为主（操作），它是唯一可问责、可考核、可改进的通道；以判断与协作为本（价值），那是价值真正的来源。主是管道，本是源头。

把“以产品数据为主”落到冰链的三个角色上，职责就活了，系统架构师、系统工程师、模块工程师，各按工件所有权分层（其责任矩阵的原始出处是工程角色谱系研究报告，13 号 §7）：

| 角色 | 拥有（工件） | 决策 |
| --- | --- | --- |
| 系统架构师 | 系统目的或任务理解确认单、系统基本概念和基本属性清单（入场券，§18.4.6）+ 架构描述、决定清单（ADR）、管控原则、系统接口契约、系统级验证方案 | 架构基线、技术路线 |
| 系统工程师 | ConOps、规定需求与追溯矩阵、八方案、系统集成与缝合清单、验证与确认报告、变更记录 | 需求基线、验证放行 |
| 模块工程师 | 模块详细定义、实现物、单元测试记录、版本与基线记录 | 组件变更 |

模块工程师在其模块内还兼模块架构师（后记四角色）。**架构不止系统有——模块也有**。42010:2022 把被架对象由 system 扩为 **entity**，模块既是一个系统元素，也可以是一个被架实体。对应的角

色就是模块架构师，通常即模块工程师自己或他的团队：模块内部那组定义（模块框架定义、内部结构、内接口、模块级准则）归它，模块级实现归模块工程师。

三条协作规则跟着工件走。约束向下：架构师定**本质与管控原则**、系统工程师定义“语义”，模块工程师在约束里实现。反馈向上：模块工程师发现偏差反馈系统工程师、再反馈架构师。追溯贯穿：追溯矩阵是三方共同维护的唯一工件。

一句话：架构师管“**立得住**”（本质与管控原则），系统工程师管“要得对”（语义），模块工程师管“做得对”（细节）；各管一段产品数据，各负其责。第 9 章（模块工程师）和第 18 章（系统架构师）会把这套分工各自展开。

再各钉一句正确履职：架构师定义边界与接口，用 ADR 记录决策理由；系统工程师转译不丢原意，维护追溯矩阵，需求可测试才放行；模块工程师遵循架构约束，实现与设计一致，偏差必走变更，单元验证自证。

划责规则一句话，一个缺陷只定位一个责任工件——需求缺陷找系统工程师，**本质、管控原则或接口契约的缺陷找架构师**，实现缺陷找模块工程师。数据在哪，责任就在哪；数据有 owner，缺陷就有归处。

顺着三维的尺子，把三个角色的价值各量一遍，就是下表：

| 角色 | 工件质量（直接） | 判断质量 | 下游影响（乘数） |
| --- | --- | --- | --- |
| 架构师 | 架构评审缺陷率 | 评审质疑解决率、路线复盘 | 下游架构偏差率 |
| 系统工程师 | 需求缺陷密度、追溯完整率 | 转译正确性复盘 | 下游返工率 |

| 角色 | 工件质量（直接） | 判断质量 | 下游影响（乘数） |
| --- | --- | --- | --- |
| 工程师 | 实现缺陷率、单元验证通过率 | 设计评审复盘 | 集成缺陷率 |

缺陷率、追溯完整率、返工率，一查数据就知道；数据是评价的锚点，不靠印象、不靠人情。

**概念卡片 · 价值三层**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ①⑥ | 利益相关方对产品贡献价值的沉淀层（物化、决策、过程三层）；数据是价值的“投影与锚”；对立面：价值本体（人的判断与协作）——数据管不到的源头 |
| **它不是什么** | ⑦⑧ | 辨析：三层不等于全部价值（决策层与过程层只部分沉淀）；误区：“数据全 = 价值全”——完美工件可能是错误决策的产物 |
| **它从哪来** | ②③⑨ | 源于“工件是判断的投影”；Polanyi《个人知识》“我们知道的多于我们能说出的”；ADR 即决策层的部分沉淀 |
| **它往哪去** | ④⑤ | 让产品数据可评价、可追溯、可问责；把职责挂到工件上；增值判据（活动值不值看数据）；成为企业资产（产品会死，数据体系活着）；数字化是存起来、转型是用起来 |

## 案例进展③｜恒信的“上线了，但数据没人管”

冰链在磨数据，恒信那边也没闲着，只是病不同。

恒信的合同审批系统上线半年，罗建志得意满：系统跑起来了，UAT 也过了；可这天，需求分析师顾言拉他开了个会。

“罗建，你回答我三个问题。”顾言摊开一张纸，“三个问题都不问系统跑没跑起来，只问数据在不在、归谁管。第一，'25 号之后签的合同算哪个月'——这条规则，现在在系统里，还是在谁的脑子里？第二，上线后改过四次审批流，每一次谁改的、为什么改，有没有记录？第三，下个月王默调走了，这套系统的设计逻辑，他从哪交接给新人？”

罗建答不上来。系统“上线了”，可系统的**数据**——规则、变更、设计逻辑——散在开发部每个人的脑子里和各自的文件里。

“这就是产品数据。”顾言说，“你以为开发完了，其实数据没归位。软件的产品数据就是产品本身——你的需求、设计、规则、变更，都在这套数据里。它没人管，等于系统没人管，只是'跑起来'骗过了所有人。”

恒信的问题，和冰链隔着一整个行业，却撞在同一棵树上：**产品数据的完整性**，才是产品站不站得住的根本。

罗建沉默了一会儿：“我一直以为'上线'是终点，代码跑起来就成功了。现在才知道，上线只是把数据从开发部交出去，数据有没有归位、有没有人接，才是系统活不活得下去的根。”

顾言点头：“这就是本章说的——开发产品就是开发产品数据；产品会死，数据体系活着。恒信的合同审批系统会迭代、会退役，但那套规则、决策、变更记录的数据体系，才是恒信真正留下的东西。”

---

### 7.4.4 活动值不值：产品数据增值判据

组织里每天一大堆活动，有些忙完一圈，产品数据一动没动。

有一个可操作的办法：**看这个活动有没有让产品数据增值**。评审让需求数据更准，增值；改设计让图纸更对，增值。可有些活动忙了一圈，产品数据没动，就要追问一句“值不值”。

但要给“增值判据”划三条边，否则会误伤：

第一，增值的内容必须顾客需要。过度加工也是“往数据上加东西”：加了一堆顾客不要的额外特性，数据是多了，价值没涨。贡献了数据 ≠ 有价值，还得顾客需要（呼应第 5 章“可实现性需求客户不关心”）。

第二，不增值不等于浪费，状态改变也是增值。增值有两种。内容增值，是数据本身更准更对：评审让需求更准，改设计让图纸更对；状态增值，是内容没变、状态变了：审批让草稿变受控，合规打上标注使数据满足规则且证据可追溯。只走流程、数据既没变好也没变状态的活动，才是必要非增值，要压缩保留，不能一刀砍。

第三，判据要双维：既要看工件增量，也就是数据有没有变好；也要看流动效率，也就是数据有没有卡住。等待、交接这些不直接动数据的环节，恰恰是要打击的核心。

把这个判据画成一张判定树，一个活动落下来就能落进一格：

| 活动 | 让数据增值? | 顾客需要? | 结论 |
| --- | --- | --- | --- |
| 完善需求 | 是 · 内容 | 是 | 增值 · 优化 |
| 加顾客不要的特性 | 是 · 内容 | 否 | 过度加工 · 浪费 |
| 审批草稿转受控 | 是 · 状态 | — | 状态增值 · 必要 |
| 合规打标注满足规则 | 是 · 状态 | — | 状态增值 · 必要 |
| 重复造轮子 | 否 | — | 纯浪费 · 消除 |

用这张表看冰链：石军花三个月优化一个没人用的参数（过度加工）、季昭重复测已测过的用例（重新学习）、审批卡了两周（等待）。三件事全是“产品数据没增值”的活动。产品数据增值判据，是把精益落到研发里最可操作的检视点。

精益研发把这套办法落成一张表，研发语境下的七大浪费：

| 浪费 | 研发语境含义 |
| --- | --- |
| 部分完成的工作 | 半成品工件堆积（没冻结的需求、写了一半的设计） |
| 额外特性 | 过度加工——贡献了数据但顾客不要 |
| 重新学习 | 同样的知识重复获取（没有沉淀为产品数据） |
| 交接 | 信息在角色间传递的损耗 |
| 任务切换 | 上下文切换的成本 |
| 等待 | 信息阻塞（等评审、等决策、等数据） |
| 缺陷 | 错误数据流入下游并放大 |

注意看，这张表里的“额外特性”和“重新学习”两行，正是本节增值判据在精益里的两个反面教材。

增值还有一个要点：数据增值不只是变好，更是流动起来。一条需求写完躺在文档里，不算增值。被设计接住、被验证测过、被基线冻结，才算活了；等审批、等决策、等数据，都是流动的停滞，也是价值流失。

### 7.4.5 产品会死，数据体系活着

产品数据的最后一站，是企业的**账本**。

什么是资产？至少满足四条，**可积累、可复用、可转移、可评价**；拿这四条检一遍候选：

| 候选资产 | 可积累 | 可复用 | 可转移 | 可评价 |
| --- | --- | --- | --- | --- |
| 产品数据 | √ | √ | √ | √ |
| 人才（判断、协作） | ✗ | ✗ | ✗ | △ |
| 客户关系 | △ | △ | ✗ | △ |
| 流程文档 | △ | √ | √ | △ |

**能同时满足这四条的，只有产品数据**。人才是价值的源泉，但不是资产的形态：人是本，数据是产；资产形态只能是数据。

> 术语注：这里的四条（可积累、可复用、可转移、可评价）是本书判断“价值载体能否沉淀为资产”的操作化口径，不是会计或标准对“资产”的定义（会计口径强调“拥有或控制 + 预期经济利益”）。本书用它比的是产品数据与人才、客户关系的资产性差异。

更扎心的一句话：产品会死，数据体系活着。单个产品会过时、会退市，它是资产的输出，不是资产；真正的资产，是能跨产品、跨代际复用的产品数据体系：需求知识、架构模式、验证基线、配置基线。

石军做了几十年冷链温控，真正留下来的不是那些报废的箱子，而是几十年攒下的那套数据：哪一版固件采的、那个坑怎么趟过去的、那条需求为什么这么定。

四类数据资产，是研发企业真正攒下来的东西：需求资产，知道该做什么；设计架构资产，知道怎么做得好；验证资产，知道怎么证明对；管理资产，知道为什么这么做。这才是配置管理、PLM 存在的终极理由。

为什么抄不走？因为追溯链与决策史是一点点攒出来的，不是一次能补的。竞争对手可以买你的图纸，买不到你十年里“为什么这么

定”的决策史；可以挖你的工程师，挖不走“这批数据哪一版是对的”的追溯链。产品数据是一年年攒出来的：你今天攒下的每一份决策记录，别人明天都补不上。

这也是“产品会死，数据体系活着”的另一半：这四类数据资产，是跨产品、跨代际复用的。石军做 M3 二代，用的正是 M3 一代攒下的需求知识、架构模式、验证基线；没有那套数据体系，二代等于从零开始。

从架构的视角再看一眼，这一半可以更尖：产品数据凭什么跨代际复用？因为产品数据，是产品架构的表达。架构本身没有别的持久化载体，只能活在产品数据里。“全程可追溯、断链必告警”这条不变量，靠追溯数据结构与告警接口这些产品数据把它写死。

所以单个产品会死，产品架构却靠产品数据跨代际活着。

这正是 §7.2.5 说的那棵树：干枝叶节点是数据，“为什么这么定”的根，也是数据；也正是架构师这份角色的日常——把架构写下来、落成产品数据（第 18 章 §18.4.6 的文档族）。同一件事的三种说法。

## 案例进展④｜十年后，谁还记得怎么做箱子

石军的师傅退休那年，把一本手写的本子交给石军，里面是他干了一辈子温控箱积累的“为什么”：这条工艺为什么这么定、那个故障当年怎么解的、哪个供应商的料出过事。

石军接过来，翻了几页，问：“师傅，这本子传给谁？”

“传给你，你以后再传给季昭。”

“本子会被弄丢的。”石军说，“上个月小周的工艺卡就找不回来。那上面的校准参数本来就是产品数据，只是它没进系统，也就没进资产。”

小周在产线那头接话：“那是我最疼的一张卡——断链检测的校准参数、老化试验的判定线，全在上面。丢的不是一张纸，是几条工艺线怎么校准的'为什么'。老师傅走了，新来的按剩下的卡试，差半度都试不出来。”

旁边的林涛听到，补了一句：“所以产品数据要进系统、进资产，不是靠师傅的本子，是靠组织的数据库。需求、设计、验证、配置，每一件都归档、都追溯、都可查。十年后师傅不在了，做 M3 的方法还在；产品退市了，做下一代的底子还在。”

“这就是你说的'产品会死，数据体系活着'?”石军问。

“对。人会把记忆带走，数据不会。只要你把它当成资产，年年往里存、年年往外用。”

---

### 7.4.6 数字化不等于转型

最后一站，看产品数据在组织里怎么被“用起来”。这常常被叫成数字化转型。

先拆词。**数字化（digitization）**是把纸变成电：模拟变数字，结果是被**存档**（档案化）。**转型（transformation）**是让数据变成价值：从“被管理”变成“被使用”，改变工作方式。

一句口诀：数字化是把数据存起来，转型是让数据用起来。

很多企业把第一步当成全部：数据存进系统就宣布“转完了”。其实那只是存起来，数据没被用起来。真正的转型，是让产品数据从“被管理的档案”变成“驱动决策的依据”：需求数据驱动下一个产品，缺陷数据驱动质量改进，配置数据驱动每一次变更决策。

数字化转型不止“产品数据数字化”一条，它通常有五条战线，但主战场是产品数据：

| 维度 | 内容 | 与产品数据的关系 |
| --- | --- | --- |
| 产品数据数字化 | 最重要资产的数字化 | 主战场 |
| 流程数字化 | 评审、变更线上化 | 让产品数据流动起来 |
| 决策数字化 | 数据驱动的决策 | 让产品数据被用起来 |
| 产品智能化 | 智能硬件、SaaS | 产生新的使用数据 |
| 商业模式数字化 | 订阅、在线服务 | 让产品数据变成服务 |

五条线都往产品数据汇聚，所以它是主战场：数字化要让它活起来。

还有一个常见的坑：把“买平台”当“做转型”。上 ERP、上 PLM、上数据中台，买的是工具，不是转型。工具存数据，转型用数据。石军如果听到“我们上了 PLM 就转完型了”，可以直接反问一句：“那数据里跑出来什么决策了吗?”答不上来，就是只存起来了、没被用起来。

### 7.4.7 认知反转

一个月后，冰链 M3 二代的第一轮验证会。

石军打开的，不再是墙上那堆图纸，而是那套数据：需求基线、接口契约、验证证据、追溯矩阵。每一片叶节点都有编号、有来路、有验证记录。季昭指着“边缘断链预判”的验证报告：“本地判定 30 秒，实测 28 秒，两条边界用例覆盖。”

石军对林涛说：“我第一次觉得，产品是我'造'出来的——每一层数据，我都接得住。”

回到章首那场会。你以为开发产品是把实物做出来，数据是开发完再补的文档；其实开发全程没有一件实物可摸——你造的是产品数据，实物只是它的复制品。

本章的反转，是数据质量决定产品质量的上限、不决定达成；产品数据是协作的对象，是职责落到实处的那一份，也是企业最重要的资产——产品会死，数据体系活着。

这些事背后，是同一次方法时刻，第 1 章说的织关系。石军原来把需求、图纸、报告当一堆散件，那是堆；现在把它们织成树、织成链，那是结构：数据不是文档堆，是产品唯一的信息形态；第 6 章把方案织成树，本章告诉你那棵树就是产品数据。

用第 1 章的话收一次束：对“产品数据”“两域三态”“数据质量”“价值三层”这些词，你原来只是“见过”，知道它们重要；本章把它们推到“拥有”：能答全四问，也就是它是什么、不是什么、从哪来、往哪去。能答全四问才算拥有——你会说出产品有两个存在，开发的就是数据。

> 开发产品就是开发产品数据：产品是价值的载体，产品数据是价值沉淀的唯一形态。

用第 1 章 §1.4.4 的尺子量一下：本章给的不只是第 ② 级（图式），还落了“外化地图”这一步，产品数据就是那张从个人脑子里搬出来、变成组织共用的图。

## 小结

本章讲的是“开发的产品到底是什么”：不是实物，是**产品数据**，也就是产品的信息存在。

我们从三个角度把它看透。**它是什么**：产品有两个存在，开发造的是信息存在；六类数据覆盖全程；实现方案是最重的一类；数据不是文档、不是配置项、不是产品，也不是架构本身。

**开发产品 = 开发产品数据**：一次定义多次复制、两域三态划出设计态；数据质量决定上限不决定达成；流水线上流动的是产品数据，缺陷沿流动放大。它怎么用：协作的对象、价值的投影与锚、职责按工件所有权分工挂在产品数据上、增值判据、企业的资产。产品会死，数据体系活着；数字化是存起来，转型是用起来。

第 6 章你种下了一棵树，方案就是树；本章告诉你，那棵树就是产品数据。现在这棵树站起来了，可它还要被人检查、被人守护、被人一路送到上线运维。

下一章，三个看门人上场——评审、验证、确认，查的正是这棵树。评审对目标、验证对需求、确认对用途。再往后，工程师（第 9 章）种下半个棵树，架构师（第 18 章）种上半个；配置管理（第 15 章）让它一直在变还说得清；整合（第 17 章）让它沿变更闭环流到上线运维。

散会时，林涛想起自己宣布“产品成功”那天，手里连一半这样的数据都没有。他今天点破了别人，也认了自己那一笔欠账。

# 附录

## A 本章概念总图

产品数据：产品的信息存在（开发工作的全部对象）

两个存在：物理存在（实物）/ 信息存在（产品数据）——实物是数据的复制品

产品 = 需求的物化 + 价值的载体 + 过程的输出；≠ 服务 / 系统 / 项目 / 结果

六类数据：需求 / 设计 / 制造 / 质量 / 服务 / 管理

边界：≠ 文档（数据是内容，文档是容器）；≠ 配置项（受控的才是）；≠ 产品；≠ 架构本身（可冻结的是架构描述 AD）

实现方案 = 八方案（技术/物料采购/制造/测试/交付/部署/操作/运维）——最重的一类

该有的样子：可定位、可承接、可追溯

开发产品 = 开发产品数据（设计态同一性）

产品开发：定义与复制；跨职能协同；以评审为门；基线 = 一组受控的产品数据

两域三态：信息域 / 物理域；设计态（纯数据）/ 制造态（数据+实物）/ 使用态（实物+实况）

严格说：开发 = 定义生成；软件：开发与制造合成一步

数据质量决定上限，不决定达成：最终质量 = 需求正确性 × 数据质量 × 制造质量 × 使用质量

流水线：流动的是产品数据；缺陷沿流动传播并放大（越下游修复越贵）

产品数据怎么用

协作：内部利益相关方读写协作；用户/客户是源与宿（出题人判卷人）；需求链只承载 stated，转译有损耗

价值三层：物化 / 决策 / 过程——数据是投影与锚，不是本体

职责：挂在工件上（工件所有权 + 系统架构师 / 系统工程师 / 模块工程师），以判断与协作为本

增值判据：活动是否让数据增值——内容增值（顾客需要 × 流动效率）+ 状态增值（草稿转受控、合规标注）；不增值 ≠ 浪费

企业资产：可积累 / 可复用 / 可转移 / 可评价四条兼备；产品会死，数据体系活着（需求 / 设计架构 / 验证 / 管理四类资产）

数字化 vs 转型：存起来 vs 用起来

前向：配置管理（第15章）= 操作；整合（第17章）= 流动；工程师/架构师（第9/18章）= 角色侧

## B 本章的能力级

### 第 ② 级为主

- **表征**:产品数据、两域三态、价值三层、设计态同一性、实现方案、数据质量的准确性 / 完整性 / 一致性 / 现时性 / 可信度、流水线——这些词你现在能答全四问；并分得开产品 / 产品数据 / 配置项 / 文档 / 架构描述。
- **图式**：产品的两个存在与“实物是信息存在的复制品”；两域三态（设计态 = 纯数据）；六类数据覆盖全程；“开发 = 定义生成”的成

立范围；多环相乘（需求正确性 × 数据质量 × 制造质量 × 使用质量）。

- 心智模型：数据质量决定上限、不决定达成；缺陷沿流水线传播放大；价值三层与“数据是投影与锚、不是本体”；职责为什么挂在工件上；资产检验（可积累、可复用、可转移、可评价）与“产品会死、数据体系活着”。
- 解释框架：本章不建此级，见第 17、18 章。

> 能答全四问，才算拥有；你会说出产品有两个存在，开发的就是数据。

## C 自查 · 这些说法对吗？

1. “产品就是实物，数据是文档，开发完再补。”——**错**。产品有两个存在；开发全程在信息存在上工作，数据不是事后补的文档。
2. “开发产品，开发的是产品数据。”——**对**。设计态同一性；严格说是“开发 = 定义生成”。
3. “配置项就是全部产品数据。”——**错**。配置项是受控的那部分（被基线化、被变更控制），草稿不是。
4. “产品数据就是文档换个说法。”——错。数据是内容，文档是容器；一个容器可以装多份数据。
5. “产品数据就是架构。”——错。**架构是那份内容，不是交付物**；可交付、可冻结基线的是架构描述（AD）。
6. “数据质量决定最终产品质量。”——错。决定上限，不决定达成；质量 = 需求 × 数据 × 制造 × 使用，多环相乘。
7. “需求方向错了，数据再好产品也错。”——对。需求正确性决定方向；验证全绿、确认全红。

8. “用户与客户只负责提需求，不碰产品数据。”——错。他们是源与宿：需要进来、使用产出、反馈修正、验收把关。
9. “职责挂在流程上，比挂在产品数据上更清楚。”——错。数据职责可证伪、可映射、可度量；缺数据流程空转。
10. “产品数据是企业最重要的资产。”——对。能同时满足这四条的只有产品数据；人是源泉（本），数据是资产（产）。
11. “软件没有独立的制造环节，开发就是生产。”——对。定义与实物合成一步；软件迭代快，因为直接改定义本身。
12. “质量是检验出来的，不是设计出来的。”——错。开发环节的质量就是数据质量；检验只能发现数据里的缺陷。
13. “实现方案只是产品数据里普通的一份。”——错。它是产品数据里最重的一类——制造完全照着它复制，错一处，一万台箱子错一万处。
14. “开发流水线上流动的是实物零件。”——错。流动的是产品数据（需求数据 → 设计数据 → 验证证据 → 基线）；实物只是设计态数据被复制的产物。
15. “往产品数据上加东西就是增值。”——错。增值还要顾客需要；加顾客不要的特性是过度加工（浪费）。
16. “数字化就是把产品数据存进系统，就算转完型了。”——错。数字化是存起来，转型是用起来、驱动决策。

## D 练习

1. 列出你们产品的六类数据（需求 / 设计 / 制造 / 质量 / 服务 / 管理），每类举两个具体工件。
2. 找出你们团队一个“把产品数据当文档、开发完再补”的例子，说说它造成了什么后果。

3. 用两域三态画你们产品的信息域与物理域：三种状态各有什么数据，两域靠什么映射？
4. 用多环相乘模型分析一次你们的质量事故：哪一环断了？数据质量在中间起了什么作用？
5. 用 ISO/IEC 25012 的五个特性（准确性 / 完整性 / 一致性 / 现时性 / 可信度）检查你们的一份产品数据，逐项给证据。
6. 你们产品的“出题人”和“判卷人”是谁？他们通过哪条路径（需要 / 使用 / 反馈 / 验收）影响产品数据？
7. 用价值三层分析一个你熟悉的产品决策：哪些沉淀进了产品数据，哪些没有？漏掉的那层，后来怎么补？
8. 用职责落地三步法（识别工件 → 指定 owner → 挂机制）给你们的核心配置项各指定一个唯一 owner。
9. 回答“产品会死，数据体系活着”：你们企业最值钱的数据资产是哪几类？能不能跨产品复用？
10. 列出你们实物产品的实现方案八个方案（技术 / 物料采购 / 制造 / 测试 / 交付 / 部署 / 操作 / 运维），标出哪一个最容易漏、为什么。
11. 画你们开发流水线的工位与工件：数据在哪些工位流动？找一条“缺陷沿流动放大”的真实例子，说清它从哪一环开始、最后贵了多少倍。
12. 用产品数据增值判据评价你上周做的三件事：哪件增值、哪件过度加工、哪件必要非增值、哪件纯浪费？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

### 标准


- [1] **R-02** ISO 9000:2015 —— 历史版本行（已被 **R-38** 的 2026 版取代）；本章 §7.2.2 术语注点到 15288:2023 §3.32 的源注「据 9000:2015 改」。
- [2] **R-03** ISO 9001:2015 —— §8.4 外部提供（物料合格性受控管理的条款落点：按接收准则验收、不合格不放行、让步接收留痕）
- [3] **R-04** ISO/IEC/IEEE 42010:2022 —— 架构描述（设计数据的一类）
- [4] **R-22** ISO 10303-1:2021（STEP）—— §3.1.29 产品数据的定义
- [5] **R-23** ISO/IEC 25012 —— 数据质量模型（本章取五把尺子：准确性 / 完整性 / 一致性 / 现时性 / 可信度）〔经 ISO/IEC/IEEE 42030:2019 表 A.3 可达〕
- [6] **R-24** PMBOK® Guide —— 项目 = 创造独特产品、服务或成果的临时性工作（产品与项目辨析）
- [7] **R-37** ISO/IEC/IEEE 15288:2023 —— §3.6 人工制品、§3.32 产品、§3.44 利益相关方、§6.4.4 系统架构定义过程、§6.4.5 设计定义过程（系统生存周期过程）
- [8] **R-38** ISO 9000:2026（第 5 版）—— §3.3.11 设计开发、§3.5.1 需求、§3.5.2 质量、§3.5.3 对象、§3.7.1 结果、§3.7.9 产品、§3.7.10 服务、§3.8.1 数据、§3.8.4 信息、§3.9.1 顾客、

§3.10.1 特性、§3.10.10 配置信息、§3.10.12 配置基线（产品三视角 / 数据—信息—对象链 / 用户与客户 / 开发＝定义与复制）

**经典文献**

- [9] **R-88** Polanyi《个人知识》/ 隐性知识 —— “我们知道的多于我们能说出的”（价值三层·决策与过程层的不完整沉淀）

> **术语注**：ISO/IEC/IEEE 24765 的「产品 = 由过程产出的工件」本章**不以它为据**；该主张由 15288:2023 §3.6 人工制品与 §3.32 产品两条一手承住（前者与 ISO 9000:2026 §3.7.9 同文）。 对照阅读：本章“产品数据”呼应第 6 章“需求与方案结对”（树 = 产品数据的结构）、第 8 章“评审、验证与确认”（查这棵树）、第 15 章“配置管理”（操作这棵树）、第 17 章“整合”（流动的工件）、第 9 章与第 18 章（工程师、架构师 = 角色侧）。

# 第 8 章 评审、验证与确认：各有一个参照

## 8.1 引言

第 7 章把那棵树立了起来：开发产品就是开发产品数据，你亲手造的是那棵树，不是实物。树立起来了，接下来是：谁来查它，拿什么当尺子，查完放不放行。

本章回答这一问。它做三件事：把评审、验证、确认这三个天天被混着用的词分开；把它们的边界一条条画开；再看它们在一份需求从生到用的一生里，各站在哪儿。

### 8.1.1 误解现场

恒信集团合同审批系统上线前的评审会，周三下午两点，会议室坐满了人。

**系统架构师**罗建在投影上打开 UAT（用户验收测试）报告，语气笃定：

“UAT 全部通过。用例通过率 96%，P0/P1 缺陷清零，三条业务主流程全部走通。我们验收合格，可以上线。”

产品开发经理曹峰已经把报告翻到最后一页的签字栏，笔尖悬在上面：“UAT 全绿，业务方也点了测试环境——验收合格，这个字我签了。”

罗建在旁边低声接了一句：“架构上确实没给确认留接口——这页报告里没有那一栏。”

会议室里有人开始收东西了。财务对接的**郑亮**却没动，他盯着报告翻了两页，问了一句：

“你们让我们点的那个测试环境，我们月底对账的场景——25 号之后签的合同算哪个月——测了吗？”

罗建愣了一下：“测试用例是照着需求清单写的，需求清单里……没有这条。这是上一轮口头补的需求，还没排进用例。”

“没排进用例？”**郑亮**皱起眉，“那我们月底对账靠什么？”

法务的人也放下手机：“等一下。上线前那条'电子签章合同也走审批流'的变更，你们说做了影响分析。可我们法务从头到尾没被拉去评审过这条变更——影响谁、谁审批、出了问题谁兜底，你们自己定的？”

会议室安静下来。需求分析师顾言坐在角落，把 UAT 报告合上，轻声说：

“罗建，你把三件事当成一件事办了。”

“哪三件事？”

“评审、验证、确认。”顾言竖起三根手指，“你说业务方点了测试环境，所以确认过了——那不是确认，那是让业务方当测试员，替你跑用例。你说测试全绿，所以验证过了——是过了，但你验证的是'按需求做对了'，不是'真实场景能用'。你说上线前评审过了——评审对的是目标，你们只对了需求清单。”

罗建不服：“这三样……不都是把关吗？方向对、做得对、能用，验收不就是看这个？”

“正因为三样都叫把关，才必须分清。”顾言看着他，“你这一句话里混了三个词，三个词各对着一处东西——平时混，没事；混到验收签字那一刻，就是合同纠纷。”

如果你坐在这间会议室里，大概会站在罗建那边。因为大多数人对这三个词的直觉，和他一模一样：都是“检查一下”，都是把关，差不多——中文里都带一个“评/验/认”字，听起来像近亲。

本章要推翻的，正是这个直觉。

### 8.1.2 本章要回答什么

读完本章，你应该能回答：

1. 评审、验证、确认这三个词，差在哪？为什么中文看起来像近亲？
2. 它们各自对着什么——评审对什么、验证对什么、确认对什么？
3. 一件事该走哪一道，凭什么判？各自的判据是什么？
4. 三句话——“评审就是过一遍”“测试全绿就是验证过了”“业务方点了就是确认过了”——各错在哪？
5. 这三道关在一条链上怎么排？缺了哪一道，会漏掉什么？
6. 它们只对做出来的实物做吗？一份需求文档、一份设计描述，算不算它们的对象？
7. 为什么“只做验证不做确认”，最后会得到“测试全过、没人用”？

### 8.1.3 本章的骨架

先把三个词各自是什么立起来，再把它们最常被认错的地方画开，最后看它们怎么用。

本章的骨架，是一句话加一条因果。

> **评审对目标、验证对规定需求、确认对预期用途。这三样判定不挑节点，也不挑对象——实现可以查，产品数据也可以查。缺哪一样，就漏一类东西；而最贵的一课是：只做验证不做确认，隐含需求系统性漏失，测试全过、没人用。**

三块：

- **是什么**——三个词为什么被混用、各自参照什么、判据各是什么；
- **不是什么**——三个最贵的误解、两处射程、方向由谁看；
- **怎么用**——一条需求怎么走完这几道关、这些关怎么排满全程、三套手法各怎么做。

---

## 8.2 判定：对什么，凭什么

> 这一块把三个词分开看清：先讲病根——它们为什么被混用；再立三个参照对象；用一张表把它们摆开；最后逐个钉死各自的判据。

### 8.2.1 病根：译名看着像近亲

罗建的问题不是他笨，是中文给了他一个陷阱。

把三个词摆到英文面前，陷阱当场现形：**review、verification、validation**。三个词源不同、构词不同、归属也完全不同，译成中文是“评审 / 验证 / 确认”，都带一个“评/验/认”字。

这是第 1 章说的第三种词汇病“翻译伪朋友”的教科书案例：英文一眼能看出的不同，中文因为都是“一个动词加一个形近字”，看起来像三个近亲。三个词不是“检查一下”的几种说法，而是三件不同的事。

更狠的是，标准在定义里就把它们分了家——靠的不是分组，而是“确定”与“证实”的分野。

- **评审（review，§3.11.2）**的动作是“**确定**”：它的定义直接建立在 **determination（§3.11.1）**之上——查明特性、得出结论，是一次主动调查，**不要求客观证据**；

- **验证（verification，§ 3.11.12）**和**确认（validation，§ 3.11.14）**的动作是“**证实**”：定义用 **confirmation** 配 **objective evidence**——结果已经在那儿了，去证实它，**必须有客观证据**。

一个是判，一个是证。判，靠判断力；证，靠客观证据。中文的形近字把这层区别糊上了，糊了四十年。

这里要把一个同形的词分开。第 3 章 § 3.4.1 的节题叫“程度怎么认定”，那里判的是质量满足程度（差、好、优秀）；**本章的动作词统一用“证实”**。同一个动作，两章各叫各的名：第 3 章要“把程度定出来”，本章要“拿出客观证据证明它”。往后见到“认定”，先看它判的是程度，还是规定需求。

一个版本细节值得留意：ISO 9000 的历版编排里，这三个词曾分属两个不同的术语组——评审在“确定”组，验证与确认在“数据、信息与文件”组。到 2026 版，三者并入同一个“**有关确定的术语**”组，组内次第仍留痕：评审紧跟确定（3.11.1 → 3.11.2），验证与确认在组尾（3.11.12 / 3.11.14）。

分组是编排，动作词才是本质。标准把三个词收进同一组，却仍用“确定”与“证实”分别定义它们。这恰恰说明判与证的区别不是编排出来的，是概念本身的。

第 1 章点过的那场翻译伪朋友病，最重的代价是一场合同纠纷。一家医疗器械公司和一家医院在合同里写了“系统通过验收后付清尾款”，但谁也没定义“验收”是什么：公司说验收是功能测试通过，医院说验收是临床科室用得顺。双方扯皮四个月。

那四个月，就是把验证和确认糊在一个词“验收”里，等纠纷发生时一次性结算的代价。本章把三个词分家，就是那场纠纷的预防针。

### 8.2.2 参照落在三个不同的世界

为什么各参照一处，而不是共用一个参照？因为它们看向三处不同的东西。把开发的世界画成四层：预期用途挂在旁边的场景层，另有两条旁路通道从侧边汇入需求。一图看清。

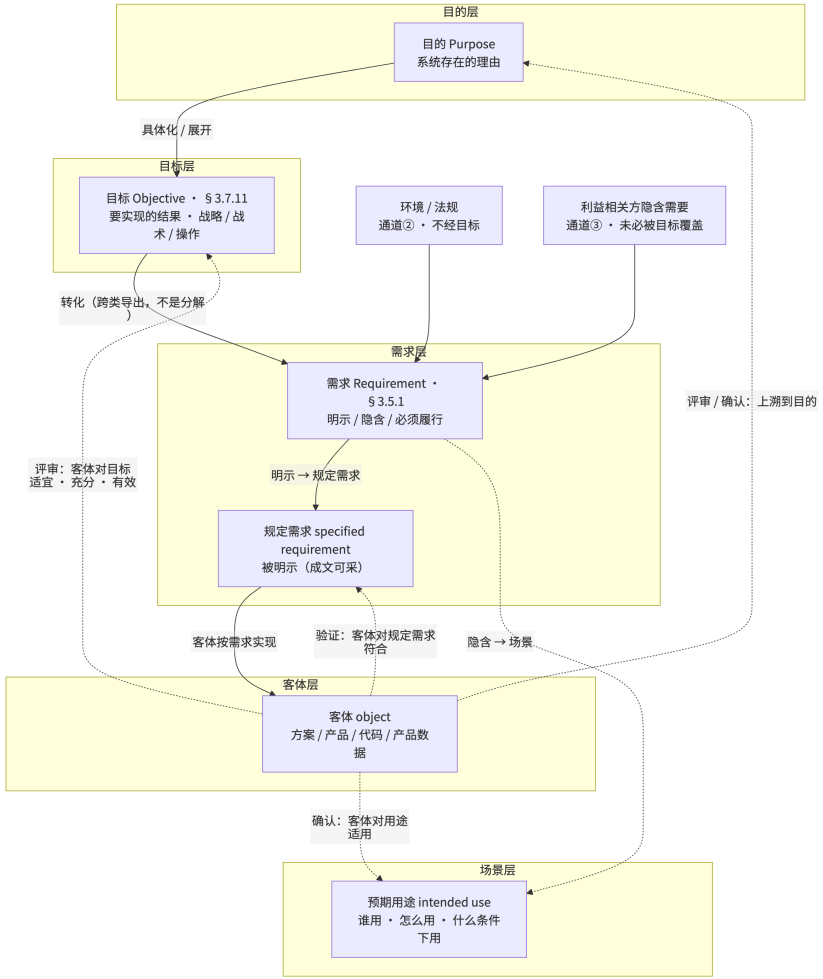



图 8-1

- **目的（purpose）**：系统存在的理由，回答为什么存在；目标是它在操作层的展开；
- **目标（objective，§3.7.11）**：要实现的结果，可分战略、战术或操作层面。其中**操作目标**是运行层可判定的那一层，也是链接需求的那层。“我们要让月底对账不再靠手工”是目标；
- **需求（requirement，§3.5.1）**：要满足的需要或期望，分明示、隐含、必须履行三种呈现方式，不都是规定需求；
- **规定需求（specified requirement，§3.5.1 注 2）**：被明示的需求，是“明示”这一形态的子集。“合同必须经财务、法务、业务三方审批”是规定需求（第 3 章展开过）；
- **预期用途（intended use）**：这个客体被设计出来，到底要在什么场景、给什么人、干什么用。

**先把两个系统的目的钉在这里**。冰链 M3 的目的，是“让疫苗在冷链运输中一路不失活、安全送达”；恒信合同系统的目的，是“让合同审批合规、月底账目清楚对得上”。两个系统的目标、需求与案例，都从各自的目的上长出来：恒信的“月底对账不再靠手工”、M3 的“断链即报废时还能撑到接入冷库”，分别是这两个目的在操作层的展开。

这张图里最容易被忽略的有几点。目的在最顶层，目标次之，规定需求居中，客体在最下面；预期用途不在这一列，它在旁边的场景层。需求还挂着两条不经目标的旁路通道。向下的建造是这条链：目的具体化成目标，目标转化出需求，需求明示成规定需求，再约束客体。三个判定则全部从客体出发，分别向三处不同的参照看去。

- 评审看客体对**目标**——方向对不对；
- 验证看客体对**规定需求**——做对了吗；
- 确认看客体对**预期用途**——做对的东西能用吗。

这里有一处最容易被漏掉的前提。图里画的“客体”是最笼统的说法：它既指实现出来的实物与系统，也指一纸规定需求、一份架构描述、一份设计描述——那些都是产品数据。第 7 章说过，开发全程你亲手造的是数据，实物是数据复制出来的结果。所以下面这几道判定，量的从来不只是车间里的东西；**数据本身也在射程内**（§8.3.2、§8.4.1 说透）。

把三个参照放回闭环，接缝才看清：目标是系统目的在操作层的展开，需求是目标转化（跨类导出，不是分解）的结果——但需求不只有目标一个来源。环境法规与利益相关方隐含需要这两条通道不经目标，直接进需求。目标的完备性靠评审的充分性把关，旁路通道靠需求工程兜底。

需求再往下，三种呈现方式在这里分叉。明示的那一种**写下来、定下来**（成文与基线是可采条件），就是规定需求；验证的参照在文档里。隐含的那一种写不出来，只能靠场景兜；确认把它放进真实用途，让它现形。三段的接缝，恰好是需求三种形态的分叉口。

评审与确认，都沿链上溯到目的：评审以目标为参照，根子上看的是系统目的；确认在真实场景里终审，方向照样对到目的。方向上的把关因此不是一处，是前后两处（§8.3.6）。

三种参照落在三个不同的世界里：目标属于“想要的”，需求属于“写下来的”，预期用途属于“真实的”。三者由此各守一种错。评审守方向错，验证守转写错，确认守匹配错。

这也解释了一处结构差：目标可以推导出需求，向下建造；唯独预期用途不能推导。它不是被想出来的，是真实场景里本来就有的事实，只能放进场景里发现。

一句话定案，各就各位：

> **评审对目标，验证对规定需求，确认对预期用途。**

换成三个可当场问的问句，按“哪个错最贵”排：

> **产品的方向对不对？（评审）→ 在真实场景里立不立得住？（确认）→ 是不是满足规定需求？（验证）**

方向错，整个产品白做；用途错，做对了也没人用；实现错，可以返工。若按站岗先后排（开工前 → 做完了 → 交给用户），就是评审 → 验证 → 确认，本章定案句与接力顺序都照此排。但要点在这里：这几处站位是典型站位，不是排他的位置——一份数据在它的每一个交付口上，都可以过这几道判定（§8.4.1）。

> 术语注：这句“评审对目标，验证对规定需求，确认对预期用途”是本书对 ISO 9000 三个定义（§3.11.2 / §3.11.12 / §3.11.14）的**凝练**，不是任何标准的原文——三个定义的原文见 §8.2.4~§8.2.6；本书的命名与判据拆法同样是作者主张，欢迎检验、欢迎校准。

### 8.2.3 差别不只在参照

光说参照不同还不够，因为参照不同可以只是修辞。把三个词摆成一张对比表，你会看到几处差别一起把它们分开：验证与确认的动作相同、证据要求也相同，只在参照上不同；评审则与前两者分了家。

| 维度 | 评审 review §3.11.2 | 验证 verification §3.11.12 | 确认 validation §3.11.14 |
|---|---|---|---|
| **定义**（汇总） | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定 | 通过提供客观证据，证实规定需求已得到满足 | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足 |
| **参照** | 目标 | 规定需求 | 预期用途 / 应用 |
| **动作** | 确定（判断） | 证实（证明） | 证实（证明） |
| **证据** | 不要求客观证据 | 必须有客观证据 | 必须有客观证据 |
| **对象** | 工作产品（纸面即可） | **实现物，或人工制品**（产品数据） | **不限**（须在使用中兑现） |
| **时机** | 事前 / 事中做判断 | 对结果做证实 | 在真实或模拟场景中做证实 |
| **软件落点** | 需求评审、设计评审、代码评审 | 单元 / 集成 / 系统测试、压测 | UAT、Beta、灰度、试运行 |

> 注：表中“定义”一行是三个词各自权威定义的总览，算**汇总行**；真正的对比维度是下面几行。**“对象”一行是本章 2026-10-08 新增的**：评审桌上摆的是工作产品；验证与确认的对象并不限于实现出来的东西，人工制品也就是产品数据，同样在内。依据见 §8.2.5 的术语注。

逐处看，三处差别最要紧：

动作不同，是最根本的分家。评审的动作是确定，验证与确认的动作是证实。确定是一次主动调查：从“没有结论”到“有结论”，结论是造出来的；证实的前提是结果已经在那儿，动作只是去证实它。这就是为什么评审不要求客观证据，而验证与确认必须有。

证据要求不同，是最容易踩的坑。“测试通过了”是验证的证据；这条证据不是评审本身。评审的结论是“这方案对目标合不合适”，不是证据，是判断。很多团队把评审开成读文档会，把证据念一遍，没有判断，那不算评审。

参照不同，是决定性的差异。评审对目标，验证对规定需求，确认对预期用途。这一条是后面所有部分的骨架。

评审、验证与确认并排立在这里，判据主义也在这里登场：第 1 章说过，不给没有判据的工具。评审三问（适宜、充分、有效）、验证与确认的三要素（对照、证据、证实），就是各自的判据。下面逐个钉死。

本章分的只有这三个词。工程上另有两个动作常被混进来，这里只划出去、不展开：评估（42020:2019 §9 的架构评估），是对架构本身值不值、够不够所作的分析与价值评定；认证，是判断实现符不符合架构（42010:2022 §3.1 把它列进架构活动），结论落成一份面向客户或用户的正式声明。它们各自的对象都不是评审桌上那版方案，四者的分野与依据，第 18 章分开讲。

### 8.2.4 评审：适宜、充分、有效

先看第一道。评审的权威定义（ISO 9000:2026 §3.11.2）：

> **评审 review：为达成既定目标，对客体的适宜性、充分性或有效性所作的确定。** determination of the

> suitability, adequacy or effectiveness of an object to achieve established objectives.
>
> 示例：管理评审、设计和开发评审、顾客要求评审、纠正措施评审和同行评审。（注：本书按统一口径写作“设计开发评审”——设计开发不拆开。）

拆开定义，四个关键词：客体（被评的东西）、既定目标（尺子）、确定（动作）、**适宜性 / 充分性 / 有效性**（三个问句）。前三个都好懂，关键是最后这个三连问——评审不是问“对不对”，是问三件事。

| 问句 | 英文 | 通俗 | 恒信的例子（目标：月底对账不再靠手工） |
|---|---|---|---|
| **适宜性** | suitability | 对症吗？ | 想解决月底对账，方案却只做了审批提速，不适宜 |
| **充分性** | adequacy | 够全吗？ | 对账口径只覆盖了当月，25 号之后的归属没算，不充分 |
| **有效性** | effectiveness | 管用吗？ | 功能全上线，月底对账还是要手工算三个小时，无效 |

三个问句层层递进，一个都不能跳过。

> **方向不对，做得再全也是白费（适宜性）；覆盖不全，方向对了也会翻车（充分性）；前两个都过了，还要看结果说话（有效性）。**

ISO 9000 还补了一句注：评审也可以包括确定“效率”（efficiency）——不仅看管不管用，还要看投入产出比。管理评审（**据** ISO 9001:2015 § 9.3，**转述**）同样落在这三问上——适宜性、充分性、有效性，三问是同一套，从产品评审一路用到公司管理。

拿冰链的 M3 走一遍三问，面目会更清楚。目标：断链即报废时，还能撑到接入冷库。

第一问是适宜性。方案选了双压缩机冗余，断一台还有一台，与“撑到接入冷库”相称。第二问是充分性。方案覆盖了压缩机故障，却没覆盖“运输车抛锚两个小时”：只做了设备级冗余，没做旅程级冗余。

第三问有效性——就算设备级、旅程级都齐了，真断链时能不能撑到冷库，要等真实运输的结果说话（§8.3.4 那辆会颠的车，会给出答案）。

三问不是同一个会里一次问完：第一问挡掉方向错的方案，第二问在方案细化时不断追问还漏了什么场景，第三问要等真实结果。很多人觉得评审是开一次会。错了：评审是一个动作，这个动作从方案出生那天起，就一直没停过。

**概念卡片 · 评审**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（ISO 9000:2026 §3.11.2）；动作是确定（判断），不要求客观证据 |
| **它不是什么** | ⑥⑦ | 与验证构成最接近的辨析——评审对目标（方向对不对，判断），验证对规定需求（做对了吗，证明）；评审不是核对需求——核对需求是验证的姿势，评审是唯一在纸面阶段抬头看目标的环节（确认是建成后的实证终审，见 §8.3.6） |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它从哪来** | ②③⑨ | 源自工程里的把关实践——设计评审、代码评审让多双眼睛在缺陷进门前拦一道；没有它，错误只能等验证与确认阶段才被证伪，返工贵数倍。ISO 9000 用确定（determination，§3.11.1）定义评审，与验证、确认的证实（confirmation）分属不同的动作（§8.2.1）——中文把它们译成“评/验/认”近亲，糊了四十年；review 由 re-（再）加 view（看）构成，第一遍是做，第二遍是看 |
| **它往哪去** | ④⑤⑧ | 它回答方向对不对；在需求评审、设计评审、代码评审、发布评审这些关口上用（§8.4.2）；误区 ✗ 评审就是开会过一遍——没结论、没记录，最多算聊天 ✗ 评审目标太抽象，评不了——目标具体化配方见 §8.4.4 |
| **判据** | 三问 | 适宜、充分、有效三问过了吗？通过标准会前定了吗？团队、结论、记录三样齐吗？ |

### 8.2.5 验证：对照、证据、证实

第二道，也是大家最熟悉的——验证。它的权威定义（ISO 9000:2026 §3.11.12）：

> **验证 verification：通过提供客观证据，证实规定需求已得到满足。** confirmation, through the provision of objective evidence, that specified requirements have been fulfilled.

定义里有三个要素，**缺一个都不算验证**。

| 要素 | 问什么 | 反例 |
| --- | --- | --- |
| **① 对照：规定需求** | 有白纸黑字的尺子吗? | 口头“大概要快一点”，无法验证 |
| **② 证据：客观证据** | 有可复核的证据吗? | “我认为满足”，不算验证 |
| **③ 证实：明确结论** | 有明确结论吗? | “差不多吧”，不是验证 |

第一个要素呼应第 4 章的规定需求：验证的参照必须是**可采**的规定需求——成文与基线是可采的条件，不是构成条件。“口头补的”需求（章首那条“25 号之后归属”）没进文档，就永远不在验证的射程内。

第二个要素最容易被低估。“客观证据”是验证和评审的分水岭：评审可以不依赖证据，验证必须拿证据说话。第三个要素决定验证的输出是结论：“已验证 / 未验证”，不是“跑完了”。

三要素背后还藏着一条推论，值得单独看一眼，它是第 4 章“需求质量里那条可验证”的真正出处。ISO 9000 定义规定需求（§3.5.1 注 2）时，只说它是“被明示的需求”，**并没有说“可验证”**。

“可验证”是从哪来的? 从验证的定义（§3.11.12）推出来的。验证要求提供客观证据，而验证的参照是规定需求；那么能被当参照的规定需求，就必须能提供客观证据，也就是应当可验证。这是一条逻辑推导，不是定义原话。

> **“规定需求可验证”不是标准直接写的，是从“验证要求客观证据，验证以规定需求为参照”推出来的——可验证，是规定需求当验证参照的资格证。**

这条推论解释了第 4 章为什么把“可验证”列进需求质量：它不是一个可选的加分项，而是“这条需求能被验证”的入场券。口头那

句“大概要快一点”，连被验证的资格都没有。没有资格，就没有证据；没有证据，就永远停在感觉上，到不了证实。

> 术语注（**对象不限，是本轮新立的一条口径**）：标准说验证的对象时用的是 a system, system element, or artefact——系统、系统元素**或人工制品**（ISO/IEC/IEEE 15288:2023 §6.4.9），并举例说人工制品包括系统需求、架构描述、设计描述。所以**验证并不是只对做出来的实物做**：一份规定需求、一份设计描述，都可以被验证——验证它满足“这份文档自身应满足的规定需求”。第 6 章的“符合性分析”就是这种验证（§8.3.2）。至于**确认，它的现场在使用态，必须在使用中兑现**；工作产品层拿不到确认结论，能给的最重也只是**条件性合规**（§8.3.6）。
>
> 同族的两个动作也要分清。**检验（inspection，§3.11.11）**是对规定要求符合性的确定；**表明合格时**，它的结果可以作验证的证据。**试验（test，§3.11.13）**是按针对特定预期用途或应用的要求所作的确定；**表明合格时**，它的结果可以作确认的证据。这里说的是“哪一族的证据”，不是“哪个动作更高级”。
>
> 另有一处正本内部的张力：两册互为属概念、方向相反——42010 把工作产品说成人工制品，15288 把人工制品说成工作产品（15288 §3.6 与附录 B.1 的 NOTE）。本章取 15288 的父子序。

> **概念卡片 · 验证**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 通过提供客观证据，证实规定需求已得到满足（ISO 9000:2026 §3.11.12）；三要素＝对照规定需求＋客观证据＋做出证实 |
| **它不是什么** | ⑥⑦ | 与评审构成最接近的辨析——验证对规定需求（做对了吗，证明），评审对目标（方向对不对，判断）；验证不是测试——测试是提供证据的手段之一，验证要对全部规定需求做证实（§8.3.2） |
| **它从哪来** | ②③⑨ | 随“按规格建造”的工程实践而生——产品对规格的符合性必须能被客观证明，做对了才算定案；ISO 9000 用证实（confirmation）配客观证据定义验证（§3.11.12），与确认共享同一动作、唯参照不同，一个在文档、一个在场景（§8.2.1）；verification 出自 verus（真），求真——对照规定需求，证明为真 |
| **它往哪去** | ④⑤⑧ | 它回答做对了吗；在交付时逐级回收（V 模型的右半边，§8.4.5）；误区 ✗ 测试全绿就是验证过了——测试只是证据之一 ✗ 隐含需求靠验证兜住——验证的参照在文档里，够不着没写下来的需求（§8.3.4） |
| **判据** | 三要素 | 三要素齐吗（对照、证据、证实）？逐级回收（单元、集成、系统）了吗？ |

引：第 6 章的“符合性分析”在这里归位——它和测试一样满足验证三要素，只是证据形态是文档评审（§8.3.2）。

### 8.2.6 确认：换参照，不换判法

第三道，也是全书最被低估的一个。它的权威定义（ISO 9000:2026 §3.11.14）：

> **确认 validation：通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足**。confirmation, through the provision of objective evidence, that the requirements for a specific intended use or application have been fulfilled.

注意，确认和验证**共享同一套三要素**：对照、证据、证实。它们唯一的区别在第一个要素——验证对照“规定需求”（文档），确认对照“特定的预期用途”（场景）。就这一处参照之别，决定了二者完全不同的射程和命运。

“预期用途”不是一句空话，它由三样东西构成。

| 构成 | 问什么 | 恒信的例子 |
| --- | --- | --- |
| **目标用户** | 给谁用? | 法务、财务、业务三方审批人，不是 IT 测试员 |
| **使用场景 / 任务** | 用来干什么? | 月底对账、电子签章合同审批、审批人出差时手机处理 |
| **环境与条件** | 什么情况下用? | 月末高峰并发、弱网、审批人休假交接 |

> **这套三构成，和 UAT 那句“真实用户加真实任务加真实场景”一一对应：目标用户对真实用户，使用场景对真实任务，环境与条件对真实场景**。后文 §8.3.3、§8.4.6 会反复用它。

确认的尺子上，还有一处更精确的写法值得钉死：ISO 9000 对照的是「特定的预期用途或应用的要求」——注意是“**要求**”（requirement，§3.5.1）。按标准，要求＝明示的、通常隐含的、必须履行的需要或期望，**隐含形态天然就在里面**。这把尺子拆开是两半：

- **明示的那半**：操作目标——运行层确认的显性判分线，§8.4.4 那句“月底对账时间不超过两个小时”就挂在这里；
- **隐含的那半**：未明示的使用需要与期望——“审批人休假交接也能顺”“月末高峰不卡”，靠场景确认兜住（§8.3.5 展开）。

操作目标不是预期用途要求的全部，只是它的显性子集。它当“所确立目标”进评审，是全身进入；进确认，只是参照的一部分。这也预告一句要紧话：操作目标全绿，确认未必通过（§8.3.5）。

确认的证据，ISO 9000 注 1 给了：试验结果，或替代计算、文件评审。注 3 更重要：使用条件可以是真实的，也可以是模拟的——这一句给了灰度发布、仿真测试的合法性。

M3 温控箱的“真实运输车跑全程”是真实条件；恒信“用测试环境模拟出月底高峰的并发”是模拟条件。都算确认，都满足“放进场景里看结果”。

镜照一眼冰链，把 M3 的预期用途填满三构成。目标用户是疾控中心和运输司机，不是实验室的测试员。使用场景是疫苗出库到接种的全程运输，不是恒温台架模拟。环境与条件是低温车厢、路面颠簸、信号弱的山区，不是平稳的测试环境。

M3 的预期用途，浓缩在“全程温度不断链”这几个字里。后面案例进展里那辆会颠的车，就是在用真环境填这三个格子。

还有一层要说清：确认的对象也可以是一份产品数据。数据的预期用途，就是接收方拿它干什么：制造数据给产线，场景记录给运维，需求数据给设计。这一层是本书的演绎。

标准的确认定义不限定对象，但另一册把确认锚在“系统在使用时”（15288:2023 §6.4.11）；本书的桥是第 7 章那句话，开发产品就是开发产品数据，实物是数据复制出来的结果。演绎归演绎，底线的两样不动：确认的证据必须是客观证据，确认必须在使用中兑现。

还要当场划清一类“确认”：本书也会在一般汉语意义上用这个词——《系统目的或任务理解确认单》、“业务方确认”“同行确认”。这类确认是理解确认、是签认：对目的或任务的理解达成一致、靠签字生效，不要求客观证据，也不对照预期用途——它不是这一节说的那个确认。

往后见到“确认单”，别把它当成第三道关。这一节的确认只认一件事：真实或模拟的场景里，那份为特定预期用途的要求，满足没有。

**概念卡片 · 确认**

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **它是什么** | ① | 通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（ISO 9000:2026 §3.11.14）；三构成＝目标用户＋使用场景／任务＋环境与条件 |
| **它不是什么** | ⑥⑦ | 不是验证的加强版——参照不同（场景对文档）、射程不同（确认兜得住隐含，验证够不着）；也不是签认——《系统目的或任务理解确认单》那一类是理解确认，不要求客观证据、也不对照预期用途（本节末） |
| **它从** | ②③⑨ | 随“产品要在真实用途里兑现”的工程实践而生；validation 出自 validus（强、有效）——问的是“这东西 |

| 四问 | 格 | 内容 |
| --- | --- | --- |
| **哪来** | | 真能用吗”；ISO 9000 用证实（confirmation）配客观证据定义确认（§3.11.14），与验证共享同一动作、唯参照不同（§8.2.1）；15288:2023 §6.4.11 把它的现场锚在“系统在使用时” |
| **它往哪去** | ④⑤⑧ | 它回答“做对的东西能用吗”；参照在场景里、不在文档里（§8.3.5）；使用条件可以真实也可以模拟（注 3）——灰度发布、仿真测试因此合法；✗ 业务方点了不等于确认过了——让业务方点测试环境，是把确认做成了验证（§8.3.3） |
| **判据** | 三要素 | 三要素齐吗（对照预期用途、客观证据、做出证实）？真实用户＋真实任务＋真实场景齐了吗？隐含需求在真实场景里显现并处理了吗？ |

引：确认的对象也可以是一份产品数据——数据的预期用途，就是接收方拿它干什么（本节末，**本书演绎**）；但确认必须在使用中兑现，工作产品层只出条件性合规（§8.3.6）。

## 案例进展①｜恒信重开上线评审会

章首那场会散会后，顾言提议第二天重开。这次他先做的，不是讲方案，是定目标。

“这次评审的目标是什么？”顾言问。

罗建：“上线前评审，目标就是……系统能上线。”

“'能上线'评不了。目标要有对象、有时限、有可衡量结果。”顾言在白板上写下三行：

- **月底对账**：上线后第一个月底，对账单与台账逐笔核对一致率 100%，对账时间不超过两个小时；
- **法务合规**：电子签章合同的审批链完整、每一步可追溯，法务可随时抽查；
- **业务可用**：业务人员三个工作日内完成一次合同发起到归档，不用求助外援。

“今天这场会，就按这三条评审。每条都有通过标准，会前定的，会后逐条打勾。”

罗建对着这三条，忽然卡住了：“……法务合规这条，我们的方案里，电子签章合同的审批链，到底谁审、谁签？这条变更当初是口头定的，需求清单里没有，方案里也没有。”

会议室安静了。

顾言没有意外：“这就是评审对目标的意义。如果今天只核对需求清单，你会说全部覆盖、通过。可你的尺子不是需求清单，是法务合规这个目标——目标一摆出来，你立刻发现：这条变更连需求都不是，是口头的一句承诺。”

罗建后来对顾言说：“我以为评审是核对需求，原来评审是先把目标摆出来，再拿目标照需求。”

## 8.3 误用在哪，射程到哪

> 这一块先拆三句最顺口的话——“评审就照需求核”、“测试过了就算验证”、“业务方点了就是确认”；再往两头看：这几道判定够不着的地方在哪儿；最后看清方向上的把关为什么是前后两处。

### 8.3.1 评审不是核对需求

这是评审三问背后那个最容易被跳过的问题。如果评审以需求为参照，不是更省事吗？需求是现成的文档，一条条核对就是——很多团队就是这么做的（冰链团队的病，案例进展②会看到）。

四条理由，每一条都足以让“以需求为参照”破产。

理由一：需求本身可能是错的。需求是人写的——理解错、互相矛盾、不完整，都是常态。如果评审拿需求当尺子，就等于默认需求一定对。那谁来审需求本身？

需求评审的本质，恰恰是审需求对业务目标是否适宜、充分、有效。恒信那条电子签章变更没被评审：不是没核对需求，是需求背后那个目标（法务审批合规）没人去看。

这里有个对称值得记下：规定需求是**评审的对象**（需求评审审的就是它），也是**验证的参照**（验证对照的就是它），还是**验收与责任的依据**（争议时以它为准）。第 4 章把它立为全章枢纽、定成验证的参照；到本章，前两样正好各归一道：评审审它、验证对它；确认不参照它——确认的参照在场景里（预期用途），兜住的是验证够不着的东西。

同一个词，评审和验证都绕不开它；它没成文，这两道就都闭门。这正是章首那条口头补的需求的下场。

理由二：否则评审和验证就重复了。验证（§3.11.12）本来就是“以规定需求为参照、确认符合性”。如果评审也以需求为参照，两个术语就变成同一件事——标准用“确定”定义评审（§3.11.2）、用“证实”定义验证（§3.11.12），这套区分就塌了。两个词必须是两件事，否则标准不会用“确定”与“证实”分别定义。

理由三：满足所有需求不等于达成目标。需求全做完，做出来的可能还是没人要的东西：目标错了，需求再对也白做。评审是整条链上唯一在纸面阶段抬头看目标的环节：验证低头对规格、不沾方向；确

认走进场景当实证终审。所以说“早拦方向，只有评审拿着目标这把尺子”；确认也看方向，但它要到使用态才登场（§8.3.6）。

理由三值得再往前推一步：它是最容易被“需求管理”掩盖的一种失败。恒信的目标是“月底对账不再靠手工”，需求清单里却全是审批提速、流程优化。需求一条条满足了，目标一个都没碰到。需求满足得越彻底，目标的偏离走得越远。

评审不是要否定需求，是要每隔一段就问一句：我们做的这些需求，还朝那个目标吗？

理由四：目标本身会漂移，漂的是系统目的。前三条理由默认目标是对的、静止的，这一条戳破它：目标不是钉死的靶子。

先把链点破。目标是系统目的在操作层的展开：“月底对账不再靠手工”是目标，“让合同审批合规、月底账目清楚对得上”才是系统目的。评审抬头看目标，根子上看的是系统目的：《系统目的或任务理解确认单》锚定的就是它。

真正会漂移、漂了最危险的，正是系统目的。产品定位从专业工具滑向大众应用，业务方在评审里一点一点改规则，没有任何一个环节宣告过目的变了，需求却还钉在旧目的上。这就是第 16 章说的目的漂移：无症状、事后才发现，只能靠定期评审、以系统目的为参照检出。评审因此是捕捉缓慢漂移的传感装置。

反过来，系统目的显式变更（战略调整、产品定位改变）就是第 16 章说的目的变更：显式发生的强制扳机，必须靠变更流程挂钩、触发架构重审。

评审以目标而非需求为参照的深层理由就在这里：需求是静止的文档，目标与目的是活的判据——只有评审，能在目的悄悄漂走时把船拉回来。

把四条收成一句话：

> **要求是路，目标是目的地。评审的价值，就是在低头检查路况之外，抬头看看目的地对不对。**

最失败的评审不是没查出问题，而是把评审做成了验证：只核对满足需求没有，从不问这需求本身对吗。章首罗建那条“UAT 全绿”，就是这个病的晚期。

## 案例进展②｜冰链镜照：评审做成了核对

冰链科技的需求评审会，开了整整一下午。

流程很标准：产品经理林涛拿着需求清单，逐条念，逐条打勾——“有编号 √、有来源 √、可测吗 √、验收标准有 √”。三十条念完，清单上划满了对勾，林涛宣布：“评审通过。”

测试季昭没有跟着打勾。他翻到第十一条，问了一句：

“这条是'断电保温不少于七十二小时'。我们的目标是什么？疫苗断链即报废，断电后要撑到接入冷库。那这套需求里，有没有写'运输车抛锚两个小时'这种场景？断电是从哪儿断的、断多久、什么时候算撑不住？”

会议室安静了。林涛翻了翻：“需求清单里……没写。但这些是应急场景，要不要写进需求，可以后面再说。”

季昭：“后面再说，就是评审没过。你刚才的评审，只核对清单有没有写——那是验证的姿势，对的是需求清单。ISO 9000 §3.11.2 那条，参照写的是既定目标：既定目标，不是需求清单。这套需求，能不能撑起'断链即报废时还能撑到接入冷库'？这才是评审要回答的问题。”

林涛的病，和章首罗建是同一个病的镜像。罗建把确认做成验证（让业务方当测试员）；林涛把评审做成验证（逐条核对需求清单）。两

边的病根一模一样：这三个词里，只认识中间那道。评审对目标、验证对需求、确认对用途——把中间那道当成了全部。

林涛后来在需求清单顶部加了一行字：“评审第一问：这套需求，对目标够不够？”逐条核对，是评审的最后一小时，不是评审本身。

### 8.3.2 测试通过不等于验证通过

这是验证这一节最要紧的一个辨析。测试只是提供客观证据的手段之一。验证的证据形态不少，本书统一列举为五种：测试、检验、审查、计算、文档评审。这是本书的归纳，不是标准的清单。标准在验证的注里举的是检验的结果、替代计算、评审文件（§3.11.12 注 1）；在确认的注里举的是试验结果、替代计算、文件评审（§3.11.14 注 1）。

把“测试过了”等同于“验证过了”，是把手段当成了目的。

拿恒信的例子说透：UAT 报告写“用例通过率 96%”——这是一份**测试结果**，是证据的一种。但验证要回答的是“规定需求是否得到满足”，证据要覆盖全部规定需求——96% 意味着 4% 的用例没过或没写，这 4% 对应的需求是否满足，测试没有给出证据。

测试报告是验证的输入，不是验证本身；验证是拿着全部证据、对全部规定需求做出证实。

顺带把“检验”也请出来。它有双重身份：在 ISO 9000 里归 §3.11 这一组、和评审同组（§3.11.11）；但在证据链上又服务于验证——和测试一样，是提供客观证据的手段之一。检验是“对符合规定要求之合格状况的确定”，听起来和验证几乎一样，区别在粒度。检验盯着特性（温度波动不超过半个摄氏度这种单条）；验证盯着规定需求（一条需求可能含多条特性）。

第 6 章的符合性分析展示了另一种证据形态：文档评审。它没动一行代码，把“方案枝梢对需求的满足”当证据，也是验证，分两档。设

计期的覆盖检查是满足度的**预演**；链尾构件规格评审上的那一遍，才是**正式审查**。

第 6 章说“符合性分析不等于评审，分析是评审的输入”，现在补全另一句：符合性分析是评审的输入，也是验证的一种。同一个动作，在“看它对不对”时是评审的输入，在“证明它满足需求”时是验证的证据。关键在于你在哪道门里用它。

> **符合性分析是设计层的验证（文档评审式）；测试是实现层的验证（试验式）。两者都满足验证三要素，只是证据不同。**

这里正好接上 §8.2.5 那条口径。验证一份规定需求、一份设计描述、一份架构描述，都是名副其实的验证——标准把人工制品明明白白写进了验证的对象里（15288:2023 §6.4.9）。所以“这只是一份文档，没法验证”这句话是错的：能验证，只是它的证据形态是文档评审、替代计算这类，不是跑一遍程序。

验证的证据有很多种：测试、检验、审查、计算、文档评审。别把验证窄化成跑测试，也别把跑测试升级成验证过了。

### 8.3.3 业务方点了不等于确认过了

确认最容易被误解的形态，是 UAT。拿它说透：

> **UAT 不是让业务方点一遍测试环境，而是真实用户、真实任务、真实场景三样齐了的确认。**

章首罗建的 UAT 错在哪？他让法务、财务、业务在测试环境点了一圈——那是让业务方当测试员，替测试团队跑用例，对照的还是需求清单。

真正的 UAT 是：财务用真实的对账数据、在真实的月底时间点、跑真实的月底对账任务——对照的是“月底对账能用”这个预期用途。前一种 UAT，测的还是需求；后一种 UAT，测的是场景。

让业务方点测试环境，是把确认做成了验证。业务方替你打勾的，还是需求清单上的格子；真实场景里会不会对不上账、审批人休了假怎么办，测试环境替不了真环境回答。

## 案例进展③｜恒信把确认做成了验证

上线前两周，顾言把那次 UAT 的整个过程调出来，把罗建、王默、苏婷叫到会议室复盘。

“你说业务方点了测试环境，通过了。我问你三个问题。”

“第一，月底对账的数据，用的是真实数据还是造的数据?”——造的数据，二十条，没有一条是 25 号之后签的。

“第二，月底对账，是在真实月底的时间点、真实并发下做的吗?”——不是，周二下午，三个人轮流点。

运维王默说：“测试环境那三台机器是我部署的。周二下午三个人轮流点，是人在点；月底是几十个人同时冲进去对账，测试环境替不了真环境——那种并发，它扛不住，也测不出来。”

开发组的苏婷接了一句：“用例是我们照着测试环境写的——环境什么样，用例就长什么样。”

“第三，法务的电子签章审批链，业务方验证过谁审谁签吗?”——没有，那条变更压根没排进 UAT。

苏婷说：“UAT 的数据和用例都是我准备的，全照着需求清单写——清单里没有 25 号后归属，用例里就没有；清单里没有电子签章，用例里也没有。我以为跑到 96% 就是确认过了，现在才明白：我验证的只是需求，不是它好不好用。”

顾言合上文件夹：“所以那次不是确认，是让业务方当测试员。你把确认做成了验证，让业务方对着需求清单替你打勾。ISO 9000 §3.11.14 那条，对照的是预期用途或应用的要求，不是需求清单。真正的确认要回答的问题只有一个：真实业务场景里，这套系统转得起来吗？你没问。”

罗建：“那现在怎么办？”

顾言：“补。确认不能后补到上线后——上线后补确认，就叫上线事故。把月底对账、电子签章、审批人休假交接三个真实场景，排进下一轮 UAT，用真实数据、真实任务、真实时间点。验证证明做对了，确认证明能用——两个都要，缺一个，另一个就是自我安慰。”

### 8.3.4 验证够不着没写下来的需求

验证有一处致命的边界，这一节把它看清。验证的参照是**规定需求**——规定，意味着**明示**；成文与基线是它的**可采条件**。那么：

> **没写进文档的需求，验证够不着。因为验证的尺子在文档里，没文档就没有参照。**

第 3、4 章讲过隐含需求——“箱体要能扛住装卸磕碰”“25 号之后签的合同归属下个统计月”，这些没人写，但漏了用户会骂。它们不是规定需求，于是不在验证的射程内。

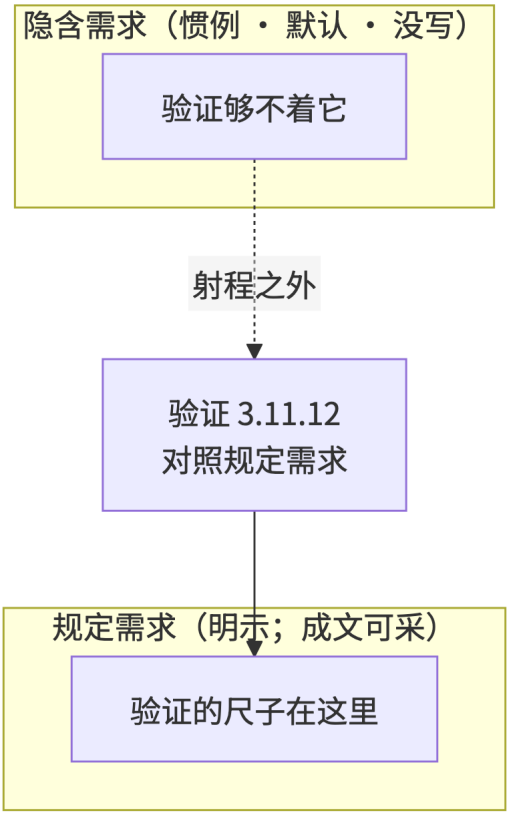



图 8-2

这就是“只做验证不做确认，测试全过没人用”的前半截：如果团队只做验证，那么所有没写进文档的隐含需求，全部在体系外漏着。测试全绿，恰恰是漏光的那批需求绿不了，因为根本没被测。后半截，由确认来补。

### 8.3.5 确认的参照在场景里

把验证和确认摆在一起，一句话划界。

**验证的参照在文档里，确认的参照在场景里。**

规定需求**通常以文档形态出现**：明示，成文与基线是可采条件。预期用途不是一份文档，它是一种**真实使用情境**：把产品放进情境里，好用不好用是自然显现的事实。这个区别带来一处决定性的推论——**确认能兜住验证够不着的东西**。

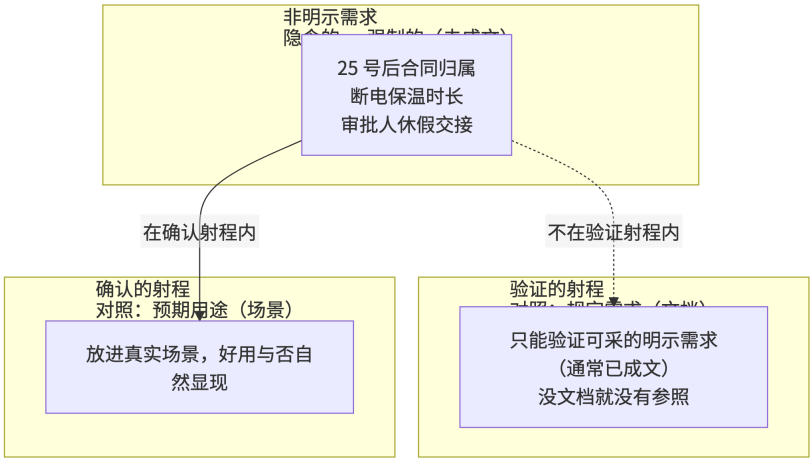



图 8-3

为什么场景兜得住、文档够不着？因为真实场景自带答案。月底对账、运输车颠簸、审批人休假——这些场景不会因为你没写文档就不存在。把产品放进场景，隐含需求漏没漏，当场现形：对不上账、断档半分钟、待办卡在离职审批人手里，全是看得见的缺陷。

顺着这把尺子再往前一步：操作目标全绿，不等于确认通过。操作目标只是预期用途要求的显性子集（§ 8.2.6）——它达标是确认的必要条件，不是充分条件：指标全绿、真实场景却用着别扭，确认照样失败。

确认的独特价值，恰恰在兜住操作目标没有覆盖的隐含要求：验证够不着的是文档外的隐含（§8.3.4），确认还要兜住操作目标外的使用期望——两层“外”，都只在场景里现形。

这里有个比喻，值得放进记忆——隐含需求像冰山，水面下的部分占了大半。传递到下游的只能是浮出水面的部分；但没有“水下还有冰山”这个认知，船就直接撞上去了。

验证是量吃水线的尺子，它只管浮在水面上的部分；确认是把船开进那片水域的试航，水下撞没撞到，试航时才知道。第 4 章说过隐含需求不会自己开口；它不开口，就靠把船开进真实水域，让水下部分撞出声来。

冰山下面不只有隐含，还有强制。法规与安全的强制要求（第 3 章 M3 的断链即报废、恒信的合同数据留档十年），梳理成清单之前同样是未成文的。它们同样不在验证的射程内：要么靠确认看真实场景里法规是否满足，要么先梳理成合规清单再走验证。

## 案例进展④｜冰链镜照：测试全过，路上不行

M3 温控箱交付前，季昭组织了一次真实场景测试。不是测试用例，是一场真的运输：疫苗装箱，上真实的疫苗运输车，跑出库、运输、到达全程。

第一天晚上，问题就出来了。车厢过减速带时，温度记录仪因振动断电重启，断档三十秒。温度数据不是连续的。这箱要是真疫苗，按断链处理，整批报废。

季昭回看数据，发现断档发生在每次颠簸之后。他试着在测试台架复现——台架没有颠簸，复现不了。翻用例：里面只有“温度记录连续”，没有“振动下温度记录连续”。

“这不是测试用例没写好。”嵌入式硬件工程师石军看完数据，说了句，“是测试环境里没有路。测试台架稳得像桌面，真实车厢会颠。用例全过，说明不了上路能过。”

他们把 M3 装到真实车上连跑三天，又抓出三个问题：电池在低温下掉电快于预期、箱门在振动下轻微漏温、云端断网重连要四十秒。

季昭把四个问题抄在清单背面：“验证那条 §3.11.12 对照的是规定需求，确认那条 §3.11.14 对照的是预期用途。我们拿前一句量了一路，后一句一次也没念过：带上路抓出来的四条问题，全落在后一句管的地方。”

三天，四个问题，没有一个能被测试用例提前发现——因为用例的参照是需求文档，而真实场景的参照是“疫苗能不能安全送到”。

把这四个问题翻回需求清单，一条都对不上：颠簸断电、低温掉电、振动漏温、断网重连，没有一条被写进过需求文档——它们是典型的隐含需求，真实存在、没人写过、验证够不着。确认的价值正在这里：它把没写下来的需求显影成看得见的缺陷——那次颠簸断档三十秒，就是需求文档里缺“振动下温度记录连续”这一条的证据。

这就是确认和验证的差别：验证在文档的尺子上跑，确认在真实世界的尺子上跑。冰链的测试台架做得再好，也替代不了那辆会颠的车。

### 8.3.6 评审早拦，确认终审

这三个词里，验证是唯一不看方向的——需求本身错了，它照样放行。抬头看方向的是评审和确认两双眼睛，不是并列的两处，是一前一后。

| | 评审（前一处，早拦） | 确认（后一处，终审） |
|---|---|---|
| **时机** | 事前 / 事中，系统未建成 | 事后，系统投入使用 |
| **方式** | 预测性把关：判断与论证推演“方向应该对” | 实证性把关：真实或仿真使用证实“方向确实对” |
| **对象** | 纸面**工作产品**（目标、需求、**架构描述**、方案） | 系统本身，或载体为产品数据的人工制品 |
| **覆盖** | 显性部分（写在目标里、可逐条论证的） | 连隐含部分一起兜（真实使用暴露一切） |

> **术语注**：评审桌面上摆的是**工作产品**，不是“架构”本身——**架构是那份内容，不是交付物**，表达它的是**架构描述（AD）**。两个英文词互为属概念、方向相反：artefact（**人工制品**，ISO/IEC/IEEE 15288:2023 §3.6 即 work product (3.32) that is produced and used during a project to capture and convey information）；work product（**工作产品**，ISO/IEC/IEEE 42010:2022 §3.3 注 1 即 A work product is an artifact produced by a process）。15288 侧 artefact 是 work product 的下位，42010 侧 work product 是 artifact 的下位。本项目按所属标准不同、属概念层级不同分流译名：指 15288 侧写“人工制品”，指 42010 侧写“工作产品”。

评审能前置，这是它最便宜的地方。一份需求文档没法被使用，也就做不了确认；但它可以评审，也可以验证。评审不要求客观证据，只要产品一定型就能做；验证则要等到有证据可举。这也是为什么评审能洒在整个流程的每一个节点上（§8.4.2）。

评审通过，只代表“基于当时信息的最佳判断”——它可能被确认推翻：纸面上再顺的方案，放进真实场景站不住，终审照样打回。所以两处关缺一不可：只靠评审，隐含需要无法被论证覆盖，评审对隐性盲区无能为力；只靠确认，方向把关全推到建成之后，纠偏成本爆炸。

但这里有一条不能越的界：评审替代不了确认。评审判的是“按既定目标够不够”，而既定目标本身可能从一开始就定错了。一屋子专家对着错误的目标评审，只会一致同意那个错目标。这正是确认必须在链外、且必须用起来的原因。

同理，也不能靠工作产品层拿确认结论。工作产品不能被使用，该层能给的最重结论是**条件性合规**：若实现如承诺则合规。**本书口径**：工作产品不能被使用，只能出条件性结论。真正的合规要由实现举证——ISO/IEC/IEEE 15288:2023 §3.13 `design` 的定义句要求的，正是“足够完整以支撑**架构的合规实现**”（a compliant implementation of the architecture）。这一点第 2 章已经点过一次，本章把它落到判定动作上。

> **一句话：方向对不对，评审先拦、确认终审；验证只管按写下的做对了吗，从头到尾不抬头。**

## 8.4 一份需求走完全程

> 这一块把这三个词用起来：先看一份需求怎么走完全程，再看这些关怎么排满整条流程，然后看清那条最硬的因果，最后是三套手法各怎么做。

### 8.4.1 先把三关串成一条链

这三个词不是三座孤立的门。它们串起来，就是第 3 章“程度三步得出法”的第三步落地。回顾第 3 章：质量是特性满足需求的程度，程度怎么得来？三步——确定特性值（测量、检验、试验、评审、计算），与需求比对（**接收准则**——到评审那一关就叫**通过标准**），认定满足程度。

第三步的“认定”，动作上正是验证和确认：验证证实特性满足规定需求，确认证实客体满足预期用途。

所以这几道判定不是质量体系的外挂，是质量判定那个“程度”从感觉变成认定的必经动作。第 3 章说“没有接收准则，程度永远只能感觉”，本章补全后半句：“没有验证与确认，连感觉都算不上，因为没人证实。”

先说清两处口径，后面才不会读歪。

第一处，这几道判定不挑节点。下面按开工前、交付时、交给用户排，是因为这三个站位最好记，也是它们最常出现的地方；但它们不是排他的位置。评审从方案出生那天起就能做，一直到发布之后还能做；验证在数据交到哪一级就在哪一级做；确认在任何一个已经用起来的场景里都能做。

第二处，它们也不挑对象。查的可以是实现物，也可以是一份规定需求、一份设计描述、一份架构描述——都是产品数据。下面这张全景图画的虽然是一份需求从生到用，它同样适用于树上的每一份数据。数据交到哪一级，验证就在哪一级回收；数据连同它的实现放进真实场景，确认才做得了终审。

把三部分合起来，让一份需求从生到用走一遍。这是本章最重要的一张全景图：

场景：恒信口头补了一条变更——“电子签章合同也走审批流”，它连需求都不是，只是口头承诺。

**第一关 · 评审（对目标）**：这条变更的目标是什么？法务审批合规。评审问：电子签章合同走审批流，谁发起、谁审批、谁签章、谁归档？法务认可这个审批链吗？这条变更对月底对账、业务可用两个目标有没有副作用？

评审在开工前，把目标摆出来照方案。案例进展①里恒信就是在这一关发现这条变更连需求都不是。

第二关 · 验证（对规定需求）。开发完成，这条需求被规定成文：“电子签章合同提交后进入审批流，审批通过后自动进入归档。”

验证问：这条需求满足了吗？单元测试过（签章触发逻辑）、集成测试过（审批引擎与签章服务）、系统测试过（全流程走通）、符合性分析过（审批引擎这个枝梢的语义与指标）。这一关的对象不只是跑起来的系统：那条需求本身、承载它的设计描述，同样在射程内。

验证在交付时，拿规定需求当尺子，逐条证实。

第三关 · 确认（对预期用途）。上线前，法务、财务、业务用真实场景跑一份电子签章的采购合同。从发起、法务初审、财务校验、业务终审、签章到归档，全程能不能走通？月底对账时，电子签章合同的统计规则对不对？

确认在交给用户时，把产品放进真实场景，问能用吗。

缺哪一关，故事都不一样。每缺一关，缺的是一道具体判定，后果完全不同：

| 缺哪一关 | 后果 | 恒信的例子 |
| --- | --- | --- |
| **缺评审** | 需求错了没人发现，白做 | 电子签章变更没有目标可对照，它连需求都不是，是口头承诺 |
| **缺验证** | 做错了没人知道，返工 | UAT 通过率 96%，剩下 4% 没有证据，却照样宣布验证通过 |
| **缺确认** | 做对了也没人用，白做 | 真实对账场景从没测过，测试全过，月底对账对不上 |

这三个词各看一处，谁也替不了谁。

反过来，这张表还能做反向诊断：系统上线运行了，业务问题却还在。需求做对了、真实场景也能跑，说明转写（需求译成设计那一步）与匹配（放进真实场景那一步）都没错，根子只可能在最上游。评审没把“这系统对达成目标有效吗”这一关。目标是可度量结果，上线只是系统在跑，不等于目标达成。

建外场排故知识库是为解排故慢，需求却写成支持库存查询，上线了排故依旧慢：缺的正是评审这一关。

这三个词的产出——评审记录、验证证实、确认结论——合起来，就是后记四角色清单里系统工程师团队的《验证与确认报告》：怎么过的，都有证据落纸。这份报告本身就是产品数据，也要被管起来、被引用、被追溯。

同一套判定，在冰链长得一模一样，这是本书最喜欢的双域对照。评审：M3 的目标是“断链即报废时还能撑到接入冷库”，评审拿它照需求清单（案例进展②的教训）。验证：零部件试验、系统联调、整机低温试验，对照的是各项规定需求。确认：真实运输车跑全程（案例进展④的教训），对照的是“疫苗能不能安全送到”。一个软件，一个硬件；一条审批流，一辆运输车。同一个概念，两扇世界的门。

### 8.4.2 评审贯穿整条流程

把关要卡在每一次交接之后——从需求到发布，每个关键节点站一班。

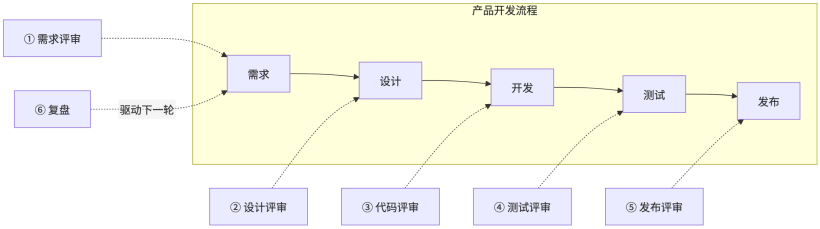



图 8-4

| 把关节点 / 复盘节点 | 目标陈述 | 判定内容 |
| --- | --- | --- |
| **① 需求评审** | 需求达到可转化状态（可作设计开发输入） | 符合业务目标（适宜）/ 无遗漏（充分）/ 可落地（有效） |
| **② 设计评审** | 方案可行且满足全部需求 | 对症（适宜）/ 周全（充分）/ 能落地（有效） |
| **③ 代码评审** | 代码正确、可维护 | 正确性 / 可维护性 / 规范性 |
| **④ 测试评审** | 质量达到准出 | 覆盖充分 / 缺陷收敛 / 结果可信 |
| **⑤ 发布评审** | 可安全上线、风险可控 | 就绪度 / 风险 / 兜底 |
| **⑥ 复盘** | 经验沉淀、流程改进 | 归因准确 / 行动闭环 / 记录归档（复盘是改进而非把关，见下） |

评审类的关口，都是评审三问（适宜、充分、有效）这一判据精神在具体关口的一次落地。上表各关口的判定词不同（如代码评审看正确性、可维护性，测试评审看覆盖、收敛、可信），追问的仍是同一件事：方向对吗、够全吗、管用吗。复盘除外。

而且注意时机：问题越早发现，修复成本越低——需求阶段的缺陷修复成本是一，发布后是三十到七十倍（第 4 章讲过）。这些节点排下来的本质，是把把关从终点挪到全程：让返工成本最高的那一段堵在源头。

为什么偏是这些节点？因为它们每个都是一次**转写**：需求译成设计、设计译成代码、代码译成测试、测试译成发布。每译一次，想要的就可能被带偏一层：方向带歪、内容漏掉、意思歧义。

这些关口恰好卡在每一次转写之后，对照上一层的源检查转没转偏：设计评审对兑现需求且站得住、代码评审对兑现设计且站得住、发布评审对可上线。评的都是本层产物自己够不够格，不是验证那种逐条对照需求。

评审点的密度，就是转写层的密度：不是评审点选得巧，是转写层本来就有这么多。

评审类关口的通过标准，都该是一组**可勾选的硬条件**。拿需求评审和发布评审举例，两组清单的样子：

| 需求评审通过标准 | 发布评审通过标准 |
| --- | --- |
| ☐ 每条需求有唯一 ID 加验收标准（可测） | ☐ 发布清单逐条通过 |
| ☐ 业务场景覆盖 100% | ☐ 回滚预案就绪（已演练） |
| ☐ 无未决 TBD 项、无需求冲突 | ☐ 无未决 P0/P1 缺陷 |
| ☐ 每条需求可测 | ☐ 已知缺陷有缓解方案 |

| 需求评审通过标准 | 发布评审通过标准 |
| --- | --- |
| □ 资源、排期认可 | □ 运营、合规、安全准备到位 |

两组清单的共同点：全部由数字加动作构成，没有“质量要好”这种形容词。会前发出去，会后逐条打勾——打勾齐了，才叫通过。

这里有个连锁推论：通过标准会前定，意味着评审的尺子不是评审会当天造出来的，是方案出生时就该有的。第 3 章季昭那句“测试判据是什么”问的就是这个：测试方案里没有接收准则，评审就没有打勾的资格。

镜照一眼冰链，这些节点在硬件侧一个不少：需求评审、设计评审、试产评审、测试评审（出厂检验标准）、上市评审、复盘——名字换成试产、出厂，把关的骨架不变。硬件团队尤其容易犯“评审只在试产时做一次”的错。这组节点不分行业，分的是你有没有把它铺到全程。

再说复盘。它是这排节点里最容易和评审划到一起的一个：两者都有回顾审视的直觉，但它和评审是两件事。

| | 评审（review） | 复盘（retrospective） |
| --- | --- | --- |
| **时机** | 干活之前或之中把关 | 干完了，事后总结 |
| **归属** | IEEE 1028-2008 五类 / ISO 9000 术语 | 敏捷实践（Scrum Guide）/ 组织过程改进 |
| **目的** | 把关：判断适宜、充分、有效 | 改进：沉淀经验、驱动下一轮 |
| **位置** | 在流程链条上 | 绕回起点 |

一句话：评审管把关，复盘管改进。复盘不在 IEEE 1028-2008 的五类里，也不是 ISO 9000 的评审。它绕回起点，把上一轮的教训变成下一轮评审的目标。

复盘还有一层身份，容易和治理混。**复盘是检测器**——它像传感装置一样例行运行：每个迭代、每轮上线后定期复盘，把偏差显影出来（对不上账、漏了场景、口头变更没记录）。

但复盘发现偏差，不等于裁决：把偏差显影出来的是检测，对偏差做裁决（要不要改规则、谁有资格批准、边界要不要挪）才是治理——那是第 16 章的事，本章只把检测器立稳。

同一条界，还有一对要分开：**评审通过 ≠ 决策作出**。评审产出的是判断与建议（这一版够不够、该不该放行），而**怎么处置，在评审的范围之外**——那是拍板那一侧的事（谁有权、拍完记在哪，第 12 章讲）。两者混起来的典型症状是“会开完了就等于定了”，而会上真正通过的那条，谁批准的、依据什么，事后查不到。

复盘的通过标准同样要会前定：每个行动项有负责人和截止日期，上轮行动项关闭率不低于八成，问题按根因归类而不是甩锅，记录归档可追溯。

复盘不产出“我们很努力”，产出“下一轮改什么、谁改、何时改完”。案例进展⑤里恒信那次对账事故复盘，产出的就是两条行动项——变更必须先过评审对目标这一关、UAT 改真实场景跑，各配负责人和期限。

复盘的价值，一半在诊断，一半在闭环；只诊断不闭环的复盘，是把同样的错再复一遍。

### 8.4.3 只做验证，会漏掉什么

现在把第 4 章的种子、第 6 章的动作、本章的两道判定串起来——这是全章最硬的一条推论。

> **只做验证不做确认，隐含需求系统性漏失，测试全过、上线没人用。**

拆成三段：

1. **隐含需求没进文档**（第 3、4 章）——它真实存在，但验证够不着；
2. **只做验证的团队**——所有没写进文档的需求，全部漏在体系外，连被检验的机会都没有；
3. **上线后**——功能全对、测试全绿，但用户用起来处处别扭：对账对不上、要的操作找不到、离职审批人把流程卡死。用户嘴上不说，行动上不用——**这就是“测试全过、没人用”的完整因果**。

反过来推一句更狠的，这是确认的第一层意义：确认的存在价值，恰恰建立在非明示需求上。如果所有需求都能写下来被验证，确认这个活动就不需要存在了——验证就够了。正因为永远有写不下来的需求（顺手、直觉、动效、节奏、惯例），确认才不可替代。

而确认兜住的，不止漏掉的隐含需求，还有一类**文字说不清的需求**。这是确认的第二层意义：冰链 M3 的手感要高级、恒信审批系统的用起来要顺，这类体验类需求显性化写不出来，只有真实使用能裁决。确认是这类需求的唯一裁判。

还有第三层意义，最容易被忽略：确认把漏掉的隐含需求变成可发现的缺陷。隐含需求没被写下来，所以它不是文档里的空洞：它是用户用不出来、卡住、放弃、抱怨。确认的价值，是把隐性缺失显影成缺陷证据：用户卡住了是证据，对不上了也是证据。

隐性缺失看不见，显影出来的缺陷看得见。章首罗建的 UAT 报告里没有一条缺陷——那不是因为他没有缺陷，是因为他把产品放进了一个造不出缺陷的场景（测试环境），缺陷无处显影。

## 案例进展⑤｜月底对账，对不上了

上线两周后，恒信第一个月底对账日。

郑亮打开系统，导出对账单，按合同编号逐笔对着台账核对。两个小时，越对脸色越差。下午三点，他把顾言、罗建和曹峰叫到了财务办公室：

“对不上。25 号之后签的合同，编号全落在当月，系统把它们统计进了当月。我们月底对账的规则是——25 号之后算下个月。”

罗建：“这条……当初财务提过，后来口头说的，一直没进需求清单。合同编号的归属规则，谁也没定过。”

郑亮：“我知道没进清单。但现在是真金白银的对不上，不是清单对不上。”

王默在边上查完一圈回来：“系统本身没坏——月底并发我盯着，没崩、没卡、日志干净。坏的从来不是机器，是'25 号后算哪个月'这条规则，从没被写进系统。”

顾言让罗建把这次事故从头捋一遍，捋到最后，他在白板上写了八个字：验证做了，其余没做。

- 验证：UAT 全绿，测试证明按需求做对了——做了；
- 评审：这条变更从没评审过，目标（月底对账规则）没人对照过——没做；
- 确认：真实对账场景从没测过——当初排进下一轮 UAT 的三个真实场景，上线前被赶进度挤回了测试环境，25 号后的合同编号是真实对账时第一次出现——没做。

顾言放下笔：“测试全过、没人用的下一句，是测试全过、月底对账对不上。你们以为它是运气问题，其实它是因果问题——该来的判定只来了一道，剩下两道，在等一场事故。”

曹峰把笔放下：“我签的那个字，签的是'按需求做对了'，不是'用起来行不行'。”

这场复盘，顾言让罗建把两条行动项记下来——变更必须先过评审对目标一关、UAT 改真实场景跑。复盘显影了偏差，但偏差的裁决（合同编号归属规则要不要定、归谁定）复盘给不了，那是第 16 章治理的事。

罗建这次没有反驳。他后来在变更流程里加了一条：任何需求变更，必须先过评审对目标这一关，再谈开发；UAT 改成真实场景跑。第二个月底，对账在一个小时内对完：不是系统变聪明，是这三个词都站到了该站的地方。

合同编号这条线，从这次对不上开始，后面还会在第 12 章被当成字段想改就改、在第 16 章三方编号对不上、在第 17 章上缝合清单。

### 8.4.4 评审怎么评

评审对目标，有一处操作性难题：目标是抽象的，怎么评？答案是先把目标具体化，再评审。抽象的目标评不了，“做出高质量的产品”无法评审；具体的目标评得了，“月底对账不再靠手工、对账时间不超过两个小时”可以评审。

目标具体化的配方，三个成分缺一不可：

> **目标＝对象（谁 / 什么）＋时限（何时）＋可衡量结果（多少）**

| 抽象（评不了） | 具体（可评审） |
| --- | --- |
| “做出高质量的产品” | “上线后三个月内，月底对账时间从三个小时降到两个小时以内” |
| “提升用户体验” | “审批人在手机端完成一次审批平均不超过一分钟” |

| 抽象（评不了） | 具体（可评审） |
| --- | --- |
| “做好电子签章” | “本季度完成电子签章与审批流的打通，签章合同可全程追溯” |

评审会开场前，先做一遍**会前三检**——任何一个答不上来，就是目标还没定好，先别开会：

1. 评审结束后，**通过的标准**是什么（怎么算达标）？
2. 这个达标有没有**可观察、可测量的判定**（数字、行为、状态）？
3. 如果两个人理解不一致，**看什么能裁决**（哪份文档、哪个数字）？

三个都答得上，这个评审有尺子；答不上来，会开了也是白开——因为大家连过没过都没共识。而通过标准的正确用法，是**随评审邀请一起发出**：“本评审通过标准：……全部打勾等于通过；任一不满足等于打回修改。”不是开会时现想，是会前定好、会后逐条打勾。

还有一个容易踩的点：目标本身还**分层级**。ISO 9000 说目标可以是战略的、战术的或操作层面的——注意这里的操作是运行层面，不是动手操作：它指管理层级里最贴近执行的那一层，不是让人去操作机器。同一份方案挂不同层级的目标，结论可能完全不同。

“我们要成为行业内合同数字化的标杆”是战略目标，激励人用；“月底对账时间不超过两个小时”是操作目标，评审用。战略目标让人相信值得做，操作目标让评审开得下去。评审要挂可判定那一层的目标：拿战略目标当尺子，等于没有尺子。

还要看清目标的来历：它不是供方自定的，是需方与供方就成功判据达成的约定。业务方提方向（要达成什么可度量结果），供方译成可评审的操作目标，再请业务方确认。确认过，它才成为评审的尺子。第 4 章链头说“立得对不对要业务方点头”，说的就是这一笔。

操作目标还有第二身份：它是确认参照里明示的那一半——进评审全身进入，进确认只是显性子集，所以操作目标全绿不等于确认通过（§8.2.6、§8.3.5）。

还记得第 3 章林涛那句“符合性没有问题”吗？那是评审不了的——因为没问题不是标准。把它翻译成“温度波动不超过半个摄氏度、断电保温不少于七十二小时、交付前百分之百通过出厂检验”，才评得起来。评审对目标，但目标必须是能打勾的目标。

讲到这里，有人会问：评审这么好，凭什么信它是评审，不是聊天？IEEE 1028-2008（软件评审与审计标准）给系统性评审钉了三个必备属性：

> **团队参与、评审结果文档化、评审过程文档化**。三个属性缺一个，按标准的话说，就是非系统性评审——说人话：**没记录的评审不算评审，最多算聊天**。

这一条值得停下来多看一眼，因为它是评审重形式的正当理由。为什么非要记录？因为评审的职责是主动暴露问题，不是汇报成果——而问题要被人看见、被追踪、被验证已解决，就必须落纸。第 6 章讲过覆盖完整性：漏一片叶子，下层方案就不知道要做；评审记录同理，不记录的问题，下次评审没人记得它存在过。

IEEE 1028-2008 把评审做法分成五类，各管一段：

| 类型 | 英文 | 干什么 | 落点 |
|---|---|---|---|
| **管理评审** | management review | 评估项目状态，支持管理层决策 | 项目级定期评审（进度、风险、资源） |
| **技术评审** | technical review | 对照规格，识别技术偏差 | 需求评审、设计评审、发布评审 |

| 类型 | 英文 | 干什么 | 落点 |
| --- | --- | --- | --- |
| **审查** | inspection | 最正式，逐行检测，缺陷检出率最高 | 代码评审（最正式时） |
| **走查** | walk-through | 作者引导讲解，同行熟悉产品 | 代码评审（非正式时） |
| **审计** | audit | 独立于开发方，核实符合合同与标准 | 上线前独立核查 |

这里有一处术语分工要留意：IEEE 1028-2008 里的 inspection 本书译审查，让给 ISO 9000 §3.11.11 的 inspection（检验）——同一个英文词在两册里指的不是一件事。一册看规定需求，一册看软件工作产品。

五类从轻到重：走查最轻（作者带看一遍），审查最重（逐行挑刺）。但无论轻重，三个属性不变——有团队、有结论、有记录。

ISO/IEC/IEEE 12207:2017（软件生存周期过程）把评审定为软件生存周期里的支持过程，并强调评审方与被评方角色分离：评的人不能只评自己写的东西。这正是“有团队”的深层含义。

那评审到底图什么？把 IEEE 1028-2008 和工程实践里的评审目的收拢一下，是五件事，每一件都独立成立。

| 目的 | 说明 | 恒信的例子 |
| --- | --- | --- |
| **质量把关** | 多双眼睛盯同一份产出，把缺陷拦在上线之前而不是出事之后 | 设计评审挡掉“电子签章变更没有审批链” |
| **风险识别** | 不只看对不对，更看会不会出事，给项目做体检 | 发布评审发现“月底对账场景没人验证” |
| **对齐共识** | 把每个人脑中的想象差异摆上桌面当场敲定 | 三方审批人对谁审谁签当场对齐 |

| 目的 | 说明 | 恒信的例子 |
| --- | --- | --- |
| **决策推进** | 给出明确结论：通过、有条件通过、不通过 | 上线评审给出“暂缓上线，补确认” |
| **经验沉淀** | 新人学团队标准和坑，老手梳理思路 | 复盘沉淀“口头变更必炸”的教训 |

五条里，最容易被轻视的是决策推进——因为大多数评审会开完就散，没人敢说不过。开完会没下文的评审，前三件事全白做。决策推进的落地是三个动作：明确结论（通过、有条件通过、不通过）加有条件通过必须写明条件、复核人、期限，加记录留档。

这三条与 ISO 9001:2015 §8.3.4 的方向一致（转述）：评审发现的问题要采取措施，并保留成文信息。评审的终点不是散会，是有一个字面结论、有一份记录、有下一步谁负责。

## 8.4.5 验证怎么验

验证不是最后做一次就完——它和设计开发的分解一样，是**逐级**的。第 3 章讲过 V 模型：需求的左半边（需求、设计、详细设计）逐级分解下去，验证的右半边（部件验证、系统验证、确认）逐级回收上来。左右是镜像。

需求从哪一级分解下去，验证就从哪一级收回来。这就是 V 模型的形状。

一个具体的样子（恒信合同审批系统）：

| 验证层级 | 验证对象 | 证据 |
| --- | --- | --- |
| **单元验证** | 单个函数、模块（金额校验逻辑） | 单元测试、代码走查 |
| **集成验证** | 模块之间（审批引擎与签章服务） | 集成测试、接口契约检查 |

| 验证层级 | 验证对象 | 证据 |
| --- | --- | --- |
| **系统验证** | 整个系统（合同审批全流程） | 系统测试、压测、合规检查 |

关键在左右镜像：“金额不低于五百万触发重点审”这条需求从系统需求层一路分解下去，它对应的验证就必须在每一层收回来。不是在最后一次性验，是分解到哪一级，验证跟到哪一级。

这里的镜像还有另一半，正好接上 §8.4.1 那条口径。分解下去的是需求数据，回收上来的是验证数据，而这两样都是产品数据。所以“分解到哪一级、验证就在哪一级回收”换句话说：数据交到哪一级，验证就在哪一级做。数据停在纸面，验证就停在纸面（文档评审、替代计算）；数据变成实物，验证就跟到实物。

这个镜像可以推到底，推演一遍你就明白它为什么不是教条：需求分解到哪一级，说明需求在哪一级被规定；被规定的需求在哪里，验证就在哪里回收。

恒信把“金额不低于五百万触发重点审”分解到系统需求层，系统验证就在这一层回收（系统测试覆盖重点审触发）。它再分配到审批引擎，集成验证就在该引擎与签章服务的接口处回收。它落进金额校验这个函数，单元验证就在这一层回收。

每一级分解，都对应一级回收——V 模型的形状不是画出来的，是“需求在哪一级被规定，验证就在哪一级证明”这条逻辑逼出来的。

只做最后一次性验证，等于让所有分解白做了：需求分解到每一层，验证却只在最后一层收，中间断掉的层级，恰恰是缺陷藏身的地方。

镜照一眼冰链，硬件侧的验证长得不一样，但同一个骨架。单元验证是零部件试验（传感器精度校准、压缩机耐久台架）；集成验证是

系统联调（温控单元、云端、应用软件协同）；系统验证是整机试验（低温环境箱里跑全温域）。

软件叫测试，硬件叫试验，V 模型的左右镜像纹丝不动。

你下次听到“我们要加强测试”，先问一句：是加强哪一级的验证？只加强系统级，等于只验最后一层，前面几层的分解没人接。

### 8.4.6 确认怎么确认

确认怎么落地？关键就在 §8.2.6 提到的那句 ISO 9000 注 3 —— 使用条件可以是真实的，也可以是模拟的。这一句给了确认一整个手段谱系，按成本从轻到重：

| 手段 | 场景 | 谁当裁判 |
| --- | --- | --- |
| **仿真测试** | 模拟场景（测试环境造一个月底高峰让系统跑） | 造出来的场景 |
| **灰度发布** | 一小部分真实用户加真实流量先跑 | 真实场景的小样 |
| **试运行** | 全部真实场景 | 真实场景全量 |

灰度发布的合法性就在这里——它是确认，不是缩小版的验证。放一成真实流量进去，就是用真实场景当裁判。别把“灰度没出问题”读成“验证过了”，要读成“确认初步通过了，剩下九成的真实场景还没当裁判”。

确认还有一类照不到的强制需求：法规与安全的强制要求（第 3 章 M3 的断链即报废、恒信的合同数据留档十年），梳理成清单之前同样是未成文的，一样不在验证的射程内，见 §8.3.5。对付它有两手：靠确认在真实场景里看法规满没满足，或者先梳理成合规清单，再走验证。

数据侧的确认，前面说过一句，这里补完。数据的预期用途，是它的接收方拿它干什么：制造数据交给产线，产线能不能照着造；场

景记录交给运维，运维能不能照着判。这一层是本书的演绎，不是标准的原话；做法与实物侧一致——把数据连同它的实现放进真实使用里，看它立不立得住。演绎归演绎，标准那条底线不动：必须有客观证据，必须在使用中兑现。

时机上，确认常在设计开发形式结束之后（第 3 章说过，确认锚定产品，不锚定需求）。它站在工程系统的门口（第 2 章）：技术系统上线，不等于工程系统成功。确认问的正是：整个社会技术系统能转起来吗？确认不能后补到上线后：补得再认真，也只能补成复盘，补不成确认。

### 8.4.7 认知反转

回到章首那场会。罗建说“UAT 全绿，验收合格，可以上线”：他说错的不止一处。

第一处，他把评审做成了核对需求。评审对的是目标，不是需求清单。恒信那条电子签章变更没被评审，不是没核对，是没有人拿“法务合规”这个目标去照它。核对需求是验证的姿势；评审是唯一在纸面阶段抬头看目标的环节，方向这事，确认还会在真实场景里终审一次（§8.3.6）。

第二处，他把测试当成了验证。96% 的通过率是一份证据，验证是对全部规定需求做出证实。测试报告是验证的输入，不是验证本身。

第三处，也是最深的一处，他把确认做成了验证。让业务方点测试环境，是让业务方当测试员；真正的确认是真实用户、真实任务、真实场景。只做验证不做确认，隐含需求就系统性漏失——测试全过、没人用，不是运气问题，是因果问题。

本章的反转，是那层被翻译糊了四十年的动作词区别被重新撕开。

> **评审对目标，验证对规定需求，确认对预期用途——方向对不对、做对了吗、真实场景好用吗，谁也替不了谁。**

而它最后指向的，是第 2 章那个没算完的账：恒信的系统上线了，但工程系统没成功——因为确认那道门没开。本章让你看见的是：工程系统的成功，不靠上线这个词，靠这三个词各自站住。

回到章首顾言那句警告——混到验收签字那一刻，就是合同纠纷。本章看完，你应该说得出那句警告的全部含义：验收不是一句话，是这三个词依次签字。评审签方向对，验证签做对了，确认签能用——三个字都签齐，才轮到验收通过四个字。

少了任何一个，验收就是把几个词糊成一个的代价——§8.2.1 那场四个月扯皮的预演。

用第 1 章的话收一次束：对评审、验证、确认这三个词，你原来只是见过——背得出定义、叫得出名字，却分不清它们各自参照什么。本章把它们推到拥有。

能答全评审、验证与确认各自的四问（各是什么、不是什么、从哪来、往哪去），能用三问和三要素量一张 UAT 报告、一场评审会、一次月底对账。

能答全四问才算拥有——评审、验证、确认，现在装进你手里了。

用第 1 章 §1.4.4 的尺子量一下：这几道判定站在**第②级与第③级之间**——它们是把看得出结构校准成判断得住的那道质检关。

## 小结

本章讲的是评审、验证与确认。**评审对目标**——站在开工前，拿目标这把尺，问方向对不对（适宜、充分、有效三问，目标具体化才评得起来，没记录的评审最多算聊天）。

**验证对规定需求**——站在交付时，拿规定需求这把尺，提供客观证据做证实（测试只是证据之一，第 6 章的符合性分析在这里对上，V 模型逐级回收）。

确认对预期用途——站在交给用户时，拿预期用途这把尺，把产品放进真实场景（真实用户、真实任务、真实场景，UAT 是确认不是验证）。三者接力，构成一份需求从生到用的几道关。方向上的把关是前后两处：评审在纸面早拦，确认在真实或模拟的场景里终审——验证不沾方向（§8.3.6）。

本章还立了一条常被忽略的口径：这几道判定不挑节点，也不挑对象。开工前、交付时、交给用户只是三处最好记的站位；而它们查的从来不只有实现出来的实物——产品数据同样在射程内。数据交到哪一级，验证就在哪一级回收。

最要紧的那句因果，值得贴在墙上：只做验证不做确认，隐含需求系统性漏失——测试全过、没人用，不是运气问题，是因果问题。月底对账那天合同编号对不上的教训，就是这句因果的活例证：该来的判定只来了一道，剩下两道在等一场事故。

复盘站在另一头：它是检测器，例行地显影偏差，但复盘本身不裁决——复盘触发的裁决，才是第 16 章治理的事。

下一章，我们先去看把关背后那个人——工程师：头衔不定义他，六项能力才定义他，判断换个人接得住才算数。再下一章（第 10 章），才进入全书概念的重心——架构；到第 11 章，看“这个实体是什么”这句话是怎么被造出来的；**评审那一侧（选定与背书）在第 12 章**。

你会看到，第 6 章那棵树的枝条走向里藏着的，正是评审最该抬头看的那两样：系统的不可还原涵义（基本概念或属性），以及管住它实现与演化的管控原则。

运维的场景表里，添上了月底对账一格。这些规则以前只在业务方脑子里、不在纸上；本章之后王默知道：场景不写下来，确认就够不着。

---

# 附录

## A 本章概念总图

评审、验证、确认（判据主义：每道判定各带可检验判据）：

评审 review 3.11.2 —— 对目标 • 确定（判断）• 不要求客观证据

- 三问：适宜性（对症）→ 充分性（够全）→ 有效性（管用）
- 系统性三属性：团队参与 • 结果文档化 • 过程文档化（没记录＝聊天）

验证 verification 3.11.12 —— 对规定需求 • 证实（证明）• 必须客观证据

- 三要素：对照规定需求 + 客观证据 + 做出证实
- 测试只是证据之一；逐级回收（V 模型右半边）；射程够不着隐含

确认 validation 3.11.14 —— 对预期用途 • 证实 • 必须客观证据

- 参照在场景里：真实用户 + 真实任务 + 真实场景（UAT/Beta/灰度/试运行）
- 兜隐含需求（冰山水下）：只做验证不做确认 → 测试全过没人用

对象不限（本章 2026-10-08 立）：评审桌上摆的是工作产品；验证与确认的对象，实现物与人工制品（产品数据）都在内

—— 依据：15288:2023 §6.4.9「a system, system element, or artefact」；确认侧为本书演绎（桥＝第7章实物是数据的复制品）

节点不限：开工前 / 交付时 / 交给用户，是三处典型站位，不是排他位置

ISO 9000 的动作分野：评审＝确定（判），验证与确认＝证实（证）——同组而动作词不同，中文糊了四十年

接力链：评审（开工前 • 方向）→ 验证（交付时 • 需求）→ 确认（交给用户 • 用途）

把关铺在流程全程：需求 → 设计 → 代码 → 测试 → 发布 → 复盘

辨析：评审 ≠ 核对需求 | 验证 ≠ 测试（证据 vs 手段） | 确认 ≠ 让业务方点

环境 | 评审 ≠ 复盘 | **评审 ≠ 决策**（评审出判断与建议，处置在评审之外）
方向的前后两处：评审（早拦 • 纸面 • 预测）→ 确认（终审 • 真实 • 实证）——验证不沾方向
评审替代不了确认：一屋子专家对着错目标评审，只会一致同意那个错目标
工作产品层只出条件性合规（合规的承担者是实现）
复盘 = 检测器（例行显影偏差），复盘触发的裁决才是治理（第16章）
合同编号对不上（月底对账）：该来的判定只来了一道，剩下两道在等一场事故

## B 本章的能力级

> **第 ② 级为主**，并落在 ② 与 ③ 之间。

- **表征**——把评审、验证、确认三个词说准，并把挨着它们的词分开：检验、试验、审核、核对需求、复盘、UAT、签认（确认单那一类）。
- **图式**——三处参照（目标、规定需求、预期用途）加一条接力链（开工前、交付时、交给用户，是典型站位不是排他位置）加一条因果链（只做验证不做确认，隐含需求系统性漏失，测试全过没人用）。
- **心智模型**——从症状反推缺了哪一道；方向由谁看（评审早拦、确认终审，验证不沾方向）；评审能前置；工作产品层只出条件性合规。
- **解释框架**——本章不建（归第 17、18 章）。

> 能答全四问，才算拥有——评审、验证、确认，现在装进你手里了。

## C 自查 • 这些说法对吗？

1. “评审就是开会过一遍需求。”——**错**。评审对目标，不是核对需求；没结论、没记录，最多算聊天。

2. “UAT 跑了就是确认过了。”——**错**。让业务方点测试环境是当测试员，不是确认；确认要真实用户、真实任务、真实场景。
3. “测试全绿就是验证过了。”——**错**。测试是提供客观证据的手段之一；验证要对照全部规定需求做出证实。
4. “验证和确认差不多，都是证明没问题。”——**错**。翻译伪朋友：验证对规定需求（做对了吗），确认对预期用途（做对的东西能用吗）。
5. “满足所有需求就是达成目标。”——**错**。需求本身可能是错的；评审是唯一在纸面阶段抬头看目标的环节——方向还有一处实证终审（确认，§8.3.6）。
6. “确认是验证的加强版，范围更大而已。”——**错**。参照不同（文档对场景），射程不同（验证够不着隐含，确认兜得住隐含）。
7. “这一份需求文档、这一份设计描述，谈不上验证，只能评审。”——**错**。标准把人工制品明写进了验证的对象（15288:2023 §6.4.9）；对文档能做的是评审与验证，做不了的是确认——确认要在使用中兑现。
8. “把评审通过标准写出来太形式化。”——**错**。目标不具体就评不了；对象、时限、可衡量结果，会前三检都答得上才开会。
9. “复盘也是评审，因为都有回顾审视的直觉。”——**错**。评审管把关，复盘管改进；复盘是检测器，复盘发现的偏差要等一道裁决才成为治理。
10. “测试全过、没人用是运气问题，多用心就能避免。”——**错**。只做验证不做确认，隐含需求系统性漏失，是因果问题不是运气问题。
11. “验证最后做一次、系统级全量跑一遍就够了。”——**错**。V 模型左右镜像：需求从哪一级分解，验证就从哪一级回收；中间断掉的层级恰是缺陷藏身的地方。

12. “评审目标太抽象，评不了，先看方案再定标准。”——**错**。目标具体化配方（对象、时限、可衡量结果）加会前三检；通过标准会前定、会后逐条打勾。
13. “评审过了就等于确认过了，两道门做一道就行。”——**错**。评审只代表基于当时信息的最佳判断，可能被确认推翻；而一屋子专家对着错误的目标评审，只会一致同意那个错误的目标。

## D 练习

1. 用一张表列出评审、验证、确认的参照、动作、证据、对象，再用你自己最近一个项目的具体例子各填一行。
2. 找出你们团队最近一次把评审做成逐条核对需求的例子——把这次评审重做一遍：先写出目标（对象、时限、可衡量结果），再问三问。
3. 你上一次说测试全绿的时候，验证三要素（对照、证据、证实）齐了吗？缺的是哪一条？
4. 拿你们一份需求文档或设计描述，试着做一次验证：它自身应满足的规定需求是什么？证据拿出来了吗？再说一遍：它做不了的是确认，为什么？
5. 用评审对目标检查你们一条正在做的需求：这条需求对应的目标是什么？拿目标照需求，需求够吗？
6. 测试全过、没人用的病根是什么？用验证射程与确认射程，解释隐含需求为什么只有确认兜得住。
7. 找出你们一条口头改的需求（没进文档、没评审、没验证的）——它现在在哪一步漏着？
8. 设计一次真正的确认：选一个你们系统的核心场景，列出真实用户、真实任务、真实场景、真实数据，写一份确认方案。

9. 用 V 模型解释：为什么“金额不低于五百万触发重点审”这条需求要逐级验证，而不是最后一次性验？这条链上流动的是什么，是实物还是产品数据？
10. 评审和复盘的区别是什么？你们团队最近一次复盘，是改进还是把关？复盘发现的偏差，后来由谁做了裁决？
11. 你们一次上线前评审，通过标准是会后现想的吗？如果是，把它改成会前定好、会后逐条打勾。
12. 用三关走一遍你们最近一次上线：评审对目标过了吗？验证对需求过了吗？确认对用途过了吗？缺的那一关，代价是什么？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-03** ISO 9001:2015 —— §8.3.4（评审、验证、确认三分：问题须采取措施加保留成文信息）、§9.3 管理评审（适宜性、充分性、有效性）
- [2] **R-11** IEEE 1028-2008 —— 五种评审类型（管理、技术、审查、走查、审计）加系统性评审三属性（团队、结果文档化、过程文档化）加评审产出（异常清单加行动项）
- [3] **R-16** ISO/IEC/IEEE 12207:2017 —— 联合评审是支持过程（评审方与被评方角色分离）

- [4] **R-37** ISO/IEC/IEEE 15288:2023 —— §6.4.9 验证过程（对象含 a system, system element, or artefact）、§6.4.11 确认过程（系统在使用时）、§3.6 artefact、§3.13 design
- [5] **R-38** ISO 9000:2026（第 5 版）—— 评审（§3.11.2）/ 验证（§3.11.12）/ 确认（§3.11.14）/ 确定（§3.11.1）/ 检验（§3.11.11）/ 试验（§3.11.13）/ 目标（§3.7.11）/ 需求与规定需求（§3.5.1 及注 2）/ 客观证据（§3.8.6）；三者同属 §3.11“有关确定的术语”组，靠“确定”与“证实”分野——中文糊了四十年

**经典文献**

- [6] **R-71** Scrum Guide（Schwaber & Sutherland）—— 敏捷实践：复盘归属对照

> 对照阅读：本章的目标、需求与用途呼应第 3 章的“需求、设计开发、质量”、第 4 章的规定需求、第 6 章的符合性分析、第 7 章的产品数据就是那棵树。架构评审（评审四看、对质三问、背书分工）不在本章展开——它是架构活动收尾的那一段，归**选定与背书**那一侧，见第 12 章。

# 第 9 章 工程师：一份能力清单

## 9.1 引言

### 9.1.1 误解现场

恒信 IT 部要招一名“高级工程师”，顾言主持面试。

候选人甲是算法高手，笔试几乎满分，口答流畅得像背书：“会写代码的，不就是工程师吗？”

候选人乙带着软考高级证书，职称材料叠了一沓：“有职称、有证书，还不算工程师？”

候选人丙干了二十年，项目数不清。顾言问他：“你的判断，换个人接得住吗？”他答不上来：“干了这么多年，经验还不够吗？”

一个把载体当判据，一个把名义当判据，一个把年资当判据。顾言没有反驳他们，他只问了一句：

> **换个人，用同样的需求，交付的东西能和你质量相当吗？**

三个人都沉默了。旁听席上的罗建，名片上印着“架构师”。这个问题，他这些年也没有答上来过。

第 2 章问过一件事：工程师头衔为什么不等于在做工程。第 8 章把三道门讲完，末了留了一句：下一章去看把关背后那个人——头衔不定义他，能力才定义他，判断换个人接得住才算数。本章就是那句话的兑现，换个问法就是——**如果不看头衔，凭什么说一个人是工程师**。

### 9.1.2 本章要回答什么

读完本章，你应该能回答：

1. 为什么头衔、证书、年限都不定义工程师？定义他的是什么？

2. 工程师的能力是一份什么样的清单？它从哪来，为什么是这几项？
3. 这份清单里，哪些管“能做事”、哪些管“定方向”、哪些管“传下去”？
4. 判断一个人是不是工程师，最少要问几句话？
5. 工程师和技术员、科学家、手艺人、会写代码的人，分界各在哪一条？
6. 同样是工程师，为什么背的账不一样？专责和终责差在哪？
7. 从一个新手到一个工程师，路上有哪几道关口？先过哪一道？
8. 工程师最常失败的方式有哪几种？每一种缺的是哪一项？
9. 为什么能力最后要到产品数据上去看？
10. 冰链的石军和恒信的罗建，谁是工程师？

### 9.1.3 本章的骨架

本章只做一件事：把“工程师”这个人，放到第 2 章那把量工程的尺子下面量一遍，交给你一份可以逐项打勾的清单，和一条能走的路线。

先说工程师是什么：立清单，一项一项立。这份清单不是本章凭空立的，它长在前面的章上。能力来自第 2 章那几层；活来自第 3 章到第 8 章，按第 3 章立的那条路径依次走一遍：需求、设计开发、产品数据、评审验证确认。每一项按同一套问题走：它是什么、它不是什么、凭什么说你有、怎么练、缺了会怎样。

再说工程师不是什么。先和外面分开：不是技术员，不是科学家，不是手艺人，不是会写代码的人，不是头衔、证书、年限；再往里划一道：工程师这个名字底下，不止一种人。

最后说工程师怎么做成：从一个新手到一个工程师，路上有几道关口，每道关口先过什么、怎么算过了，最后落到一件看得见的东西上。

全章只问一句话：换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果。清单里的每一项都是这句话下的一个检查点：每一项的物证是产品数据，侧面是粒度、熟练度、几种死法。

这条判据不是本章新立的——第 2 章 §2.2.4 立过它，本章只是把它从“量一件事”用到“量一个人”身上。本章也不再发明新概念：工程科学、方法论、工具与实践、价值与伦理这几层，第 2 章已经立好了，这里只用它们一次。

往前追，能力与活都不是本章的发明：那几层是第 2 章立的，那条路径是第 3 章立的，需求、设计开发、产品数据、评审验证确认都各有出处。往后落，第 15 章会把⑥可复现意识长成配置与基线，第 16 章会把④职业伦理长成治理；到第 18 章，再加上系统级的那几项增量，合成架构师的一份清单。

---

## 9.2 一份清单：能力与活

先说工程师是什么：把第 2 章那套分层，落到一个人身上。

### 9.2.1 清单不是拍脑袋列的

先把清单立起来。

第 2 章立过：工程科学、方法论、工具与实践、价值与伦理，这几层同时在场，工程才成立。那里讲的是学科的样子；这里要看的是人的样子。同样的几层，长在一个人身上，会变成什么。

答案不复杂：每一层，都要在一个人身上长成一项能力。工程科学长出科学素养，方法论长出方法论素养，工具与实践长出实践能力，价值与伦理长出职业伦理。学科缺一层，工程不完整；人缺一项，工程师不完整。

还有不占任何一层的能力。

**工程判断**横切在这几层之上。原理用到哪里为止、方法要留多少判断空间、工具选哪一套、底线划在哪——每一层都有取舍，取舍的本事不归任何一层独有。

**可复现意识**管着全部。它问的不是“你会不会”，而是“你会的这些，换个人能不能接住”。

清单如下。从层里长出来的那几项排在前面，工程判断与可复现意识另算；④职业伦理放在最后讲，它管方向与底线，把整份清单收住。

| 能力 | 长在哪一层／与这几层的关系 | 缺了会怎样 |
| --- | --- | --- |
| ① 科学素养 | 工程科学 | 换个场景就抓瞎 |
| ② 方法论素养 | 方法论 | 靠人传人，人走方法走 |
| ③ 实践能力 | 工具与实践 | 有想法，落不了地 |
| ④ 职业伦理 | 价值与伦理 | 会出事 |
| ⑤ 工程判断 | 横切在这几层之上 | 有流程也选不对，约束一变就懵 |
| ⑥ 可复现意识 | 不占哪一层（元） | 手艺——你在质量在，你走质量走 |

这份清单不是拍脑袋列的。

> **术语注**：本清单以第 2 章那套分层为骨架，对照 ABET 工程师成果七条凝练而成（能力清单及其分层映射是本书的整理，ABET 七条见 R-15）。⑤横切、⑥元、“职业伦理定方向”这些构造与判据，是本书主张，不是任何标准的条文；欢迎检验，欢迎校准。

清单立到这儿，只立了一半。上面那几样是能力，长在人身上；另一半，是他干出来的活。

工程科学、方法论、工具与实践、价值与伦理这几层，第 2 章讲的是学科的样子；落到一个工程师手上，它们除了长成能力，还要落到需求、设计开发、产品数据、评审验证确认这几件事上。

这几件事不是本章另立的，它们来自第 3 章到第 8 章：第 3 章给了它们的次序，定需求 → 出规格 → 判满足；第 4、5 章把“需求”这件事做透；第 6 章给“设计开发”，也给出产品数据那棵树；第 7 章给“产品数据”；第 8 章给“评审验证确认”。§9.2.8 一件一件说。

能力问“我有没有”，活问“我会不会干”；两样都答得上来，才叫是个工程师。

这份清单两组都立起来了；它的用法在 §9.4 讲，先记一句：**它不能背**。这不是知识点清单，是自问清单。

按 ISO 9000:2026 §3.10.6，competence 是“应用知识和技能以取得预期结果的能力”，它是干成事的本领，不是背下来的知识。这份清单属于这一类，所以它只能拿来逐项自问。可复现意识之所以也要写进清单，正是为了让清单自己受检验：一份只能背下来的清单，本身就过不了它。

还有一件事要在这里说清：这份清单的每一项，最后都指向同一句话——就是章首顾言问的那句。往后每一节的“凭什么说你有”，问的都是它的一个侧面。

### 9.2.2 ①科学素养：凭什么能行

“科学素养”在科普里指“懂科学、不被忽悠”。那句话的对象是公众，判据是不被忽悠。在工程里它是另一回事：对象是工程师，判据是一句更硬的话——我做的东西，凭什么能行？

拆开来，是这几件事。**理解原理**：知道它成立的逻辑，不是背下公式。**判断边界**：知道它在什么条件下成立、什么条件下失效。**应用建模**：把现场简化成模型，先估出量级，再谈精度。

第 2 章说过，没有力学定律，土木工程师只是个会搭积木的人。这句话反过来读也成立：一个只会套模板的人，换一个场景就会露馅，他的依据不在原理上，在“上次这么做成了”上。

所以科学素养不是懂很多公式。公式可以查；“依据成不成立”这件事，不能外包。

怎么练？第一句动作是写下预测，事后对账。

改动之前先把预测写下来：改了之后会怎样、哪个数会变、变多少。事后拿实际结果对账，找出偏差。偏差回来，先问一句“是哪条原理越了界”——是原理用错了范围，还是模型简化掉了什么。问清楚了，再决定是修原理还是修模型。

这一条和第 2 章那句“错误可见”是同一件事：能看见偏差，才谈得上修正。没有预测的实践不产生科学素养，只有“调到对为止”，那是手艺。

日常还有几个短练习可以用：反向追问（拿一个顺手的工具，问一句“它凭什么能行”）、数量级估算（不查资料先推个量级）、讲给外行（讲不通就是没懂）。

> **判据**：我做的东西，凭什么能行？换个上下文，还成立吗？

缺了这一项会怎样？换个场景就抓瞎，只会套模板。恒信的罗建砍掉容量评估那天，没有人问他“凭什么能行”，他自己也没有问（案例进展②）。

### 9.2.3 ②方法论素养：把工作组织成方法

方法论素养不是会背方法论名词：敏捷、瀑布、CMMI 张口就来，那是方法论词典，不是素养。

方法论是“关于怎么组织工作、怎么选方法”的学问；素养是把工作组织成方法的能力。它的做法也落在几件事上。

**过程设计**：每一步有输入、有输出、有检查点。**显性化**：把个人做法写成文档、规则、标准，可教可学。**过程治理**：检查过程有没有在跑、偏差出在哪、下次怎么改。

本章把设计开发当作工程里最要紧的那套方法，也就是由需求到方案的逐对生成。第 6 章给了它全套：三套规则、三个动作、结对六步。

它不是一句“最佳实践”，是一组有判据的过程：每一步有输入、有输出；承接有单值性，一条需求只落到唯一一个节点上；覆盖有公式，一个节点承接的所有需求，语义包含于它叶节点语义之并。一个过程有判据，才算方法；没有判据的，只是习惯。

ISO 9000:2026 §3.3.11 给设计和开发的定义是“将某一客体的需求转换为该客体更详细需求的一组过程”。本书读法：一组有判据的过程，就是一套方法。所以②要问的不是“你会不会跑那套设计开发”，那是第 6 章的功课；它要问的是：第 6 章那套章法，你能不能照你这摊活组织出来、写下来、交给别人跑。

第 3 章说过这句话的另一半：工程＝概念这一侧的方法（需求、设计开发、质量，管“要什么、怎么证明做对”）×技术本体。

从“我会做”到“这个方法谁都能用”，中间是一条转化链：做法在心里，是隐性判断；写成文档与规则，是显性化；定下步骤、输入输出与检查点，是标准化；换个人也能交付，才成了组织能力。

这条链上每一段都有风险。写不出来的，死于“人走方法走”；写得太细的，死于“算法化”：判断空间被写没了，方法退化成一张照单

执行的清单。第 2 章那条界线在这里最要紧：方法要有章法，但每一步都得给判断留位置。

怎么练?第一句动作是把你这摊活的规则写出来, 交给同事照做, 看他卡在哪。

写你最近一次做对的事，步骤、依据、坑都写上，交给一个没参与过的人照着做一遍。他卡住的地方, 就是你还没显性化到位的地方。拿着现有流程再过三问: 每一步有输入输出吗?有可检验的检查点吗?换个人做，结果稳不稳？答不上来的环节，通常就是事故多发处。

> **判据**：我的方法能被别人学会吗？还是只有我会?

缺了这一项会怎样？工程靠人传人，人走了方法也走了。冰链的石军就卡在这里：他的手艺是团队最值钱的资产，但那份判断说不成体系（§9.2 案例进展①）。

### 9.2.4 ③实践能力：亲手把方案变成现实

实践能力不是会写代码、会画图，那是载体。实践能力是亲手把方案变成现实、东西坏了能修好。

拆开也是这几件事。**动手实现**：想法变成代码、图纸、样机。工具驾驭：编辑器、集成流水线、测试环境；方法论必须落成工具和动作，才产生价值。排错集成：不工作时能定位，把部件接起来跑通——缝里的问题，常常只有上手才发现。

这一项只能从“做”里长出来。读和听几乎不产生它。它有一条强度光谱：

阅读（看不会）→ 看演示（眼睛会了）→ 跟做（手还没会）→ 仿做（照猫画虎）→ **独立做**（判断进场）→ **救火**（排错定力）。

到“独立做”那一级，判断才开始进场；到“救火”，才练出“坏了能修好”的定力。

怎么练?第一句动作是把强度光谱往外推一格:这个月在“跟做”，下个月就找一件能“独立做”的事；已经在“独立做”的，主动接一次救火。

它的边界也要划清。不是埋头苦干——做完要显性化，配合方法论素养，否则只是手艺。不是工具集——换个工具就抓瞎，说明缺的是科学素养。自动化时代它反而更值钱——机器接走执行，人接住的是判断密度最高的那部分执行。

> **判据**：我最近一次亲手落地，是什么时候?

缺了这一项会怎样?有想法落不了地。这一项缺了，其他几项都悬空——石军之所以是本团队里最值钱的人，首先因为他这一项顶配。

### 9.2.5 ⑤工程判断：在约束里取舍

工程判断，是在约束下做对的取舍，而且担得住。它不是“会做决定”。罗建那句“先上再说”也是个决定，问题在于那是个坏的取舍:他做了取舍，却没让任何利益相关方知情，也没走评审。

第 2 章那句话可以搬到这里用：约束越多，判断越值钱。

好的工程判断是一个闭环。

**识别约束**：先看清冲突在哪——成本、工期、安全、法规，哪几个在打架。**权衡取舍**：排序、找平衡点，trade-off 要有依据，不能凭手感。**担责闭环**：让人知情、走评审、留记录。

选得对只完成了一半，另一半是能不能负责。这也正是工程判断和职业伦理的交界：判断负责选，伦理负责划不可越的线。

判断力是清单里唯一“必须亲自做错”的能力。它不能教，只能长；生长点不是“做决定”，是复盘。做错的决定加上认真复盘，胜过做对的决定加上不复盘。

怎么练？第一句动作是把每一个重要取舍记下来：当时哪些约束在打架、选了哪个、后果如何、重来会怎么选。低风险的补充练法是案例裁决——拿恒信“先上再说”、挑战者号发射决策这类真实案例，先自己裁，再对照书里的判断；判错了不会出事，但会留下痕迹。

> **判据**：规则没有覆盖的地方，我能做对的取舍吗？

缺了这一项会怎样？有流程也选不对，约束一变就懵。这也解释了本章开头那场面试的第三个人：二十年经验如果没有变成判断，那只是把同一年重复了二十次。

### 9.2.6 ⑥可复现意识：让判断被接住

可复现意识，管的是你不在场的那一刻。手艺人的全部本事长在本人身上；工程师的要求是“换个人也能交付”——第 2 章的试金石，落到个人身上就是这一项（这就是第 2 章那把试金石落到个人身上的版本）。

它和方法论素养最容易混。方法论素养是“写成什么样”的本事；可复现意识是“必须写成那样”的自觉。前者会写，后者主动写。很多人前者有、后者缺：文档写得出来，就是不写。那不是能力问题，是意识问题。

它从近到远要做几件事：**判断显性化**（把心法写成规则、文档、标准）、**方法可传授**（换个人照着做，结果质量相当）、**组织化**（变成组织能力，人走判断不走）。

怎么练？第一句动作是做一次交接演练。

假设明天你就把手上这摊活交出去，写一份交接包：步骤、依据、坑。写不出来的地方，就是你还没想清楚的地方。

它也有边界。不是复制粘贴模板——可复现的是方法，不是结果。不是消灭个人价值——石军的经验仍然最值钱，这一项是把它写出来，不是把它废掉。不是把判断写成规则——§9.2.3 说过，判断不能写全成规则；这里写下来的是方法和依据，最后的判断仍然留给人。

> **判据**：我的判断换个人能接住吗？我把它写成规则、文档、标准了吗？

缺了这一项会怎样？手艺：你在，质量在；你走，质量走。恒信那套流水线就栽在这里：脚本在罗建那台机器上跑得顺，换台机器就缺那三条隐藏命令和两个环境变量（案例进展②）。

### 9.2.7 ④职业伦理：给能力定方向

先正名：职业伦理不是道德说教，也不是规章制度。

它是“工程师”这个身份的定义性特征：工程是一种职业（ABET 用 the profession，见 R-15）。NSPE 伦理准则的序言把这件事说得更直白：工程师的服务必须致力于保护公众的健康、安全与福祉（见 R-62）。

这一项之所以要单独说，是因为能力不是中性的。

清单里前几项都在回答“能不能做成”；这一项回答“往哪个方向做”。一个科学素养、实践能力、工程判断都顶配、而职业伦理为零的人，破坏比“不会做”的人大得多：他会算、能做、判断准，只差守住底线。737 MAX 里不缺会算的工程师，缺的是守住底线的工程师。

恒信那次月底急上、没做容量评估的接口也是这个形状：赶进度时，“写一半顺手改审批状态机”，能顺手改，说明能力不差；没评估影响就改，说明这一项缺席。

一次妥协算不算“客观原因”？第 2 章 §2.3.3 那三问，到这里就是这几句话：

利益相关方知不知情？有没有评审？有没有记录？

客观原因不留记录；价值与伦理失守，恰恰发生在“没有记录”的地方。这三问也正是第 8 章那三道门禁在压力下的样子：评审要利益相关方知情、结论要记录在案，验证确认要客观证据。

它在这份清单里管的是方向与底线：**定义方向**——能力是选择的规模，伦理是选择的方向；**守底线**——压力面前不越线。所以它和前几项不是一类：前几项是增量型，做多做少都算数；它不能靠“多做练习”增值，只能靠在压力下不越线来证明。

但“守”这件事可以练：写下自己的不可越线清单（安全、合规、隐私、数据真实性），先划线才谈得上守线；每次取舍主动留痕；再靠机制兜底——把评审和记录制度化，不靠个人自觉。

它的边界也要划清：**不是规章清单**（规则背不完，压力时刻判断要在场）；不是风险管理（风险管理算概率，伦理定立场——概率再低，红线也不能越）；不是服从组织（对公众负责，不是对老板负责）。

> **判据**：压力面前，我会不会为进度悄悄牺牲可靠性？利益相关方知情吗？有记录吗？

缺了这一项会怎样？会出事。而且它缺位时，能力越强、破坏越大。

### 9.2.8 工程师干的活：从需求到判定

前面几章各交出一件事。它们不是并列的几件，是第 3 章那条路径按次序走一遍：定需求 → 出规格 → 判满足。这条路的起点不是一份现成的规定需求，是利益相关方的需要与期望（ISO/IEC/IEEE 15288:2023 §6.4.2）。本章把“评审、验证与确认”简称**判定**。

| 他干的活 | 它是什么 | 来自哪几章 | 用得上哪几项能力 |
| --- | --- | --- | --- |
| 需求 | 从利益相关方的需要与期望出发，把它们开发成能当依据用的规定需求，再管住它一直是那一份 | 第 3、4、5 章 | ②方法论素养、①科学素养、⑤工程判断 |
| 设计开发 | 由需求到方案的逐对生成，把依据做成能实现的结构 | 第 3、6 章 | ②方法论素养、①科学素养、⑤工程判断、③实践能力 |
| 产品数据 | 这条路径上造出来、又被查着的东西，你亲手造的就是它 | 第 6、7 章 | ③实践能力、⑥可复现意识 |
| 评审、验证与确认 | 拿证据判“对没对”，三次判定各对自己的对象 | 第 8 章 | ⑤工程判断、④职业伦理 |

（评审、验证、确认三条同属 ISO 9000:2026 §3.11 一组，组内根术语是 §3.11.1“确定”；本章要用的就是这三条。）

这几件事之间的交接，各章都立好了，这里只点名：

- 需求这一段内部，是批准 → 基线：开发负责做出来，管理负责让它一直是那一份（第 4 章）；
- 需求交到设计开发手上：那份规定需求就是设计开发的输入（第 3 章）；
- 设计开发交到产品数据手上，是“上一份的叶节点，就是下一份的需求”（第 6 章）；
- 产品数据交到判定手上，是三次判定各对自己的对象：目标、规定需求、针对特定预期用途或应用的需求（第 8 章）。

> **术语注**：“评审对目标、验证对规定需求、确认对预期用途”这一句，是本书对 ISO 9000:2026 §3.11.2、§3.11.12、§3.11.14 三个定义的凝练，不是任何标准的原文。

到这儿，第 2 章给了能力，第 3 到第 8 章给了活，都点到人身上了。它们最后都落到同一件东西上：产品数据。§9.4.7 会用它把这份清单量一遍。

### 9.2.9 清单不挑领域：变的只是权重

同样一份清单，换一个领域还成立吗？

系统工程、软件工程、硬件工程，领域的工程特征不一样。系统工程是元工程：整合部件、管接口、做系统级验证，自己不造部件。软件工程把工程化落在过程上：版本控制、持续集成、评审、测试链，每一步要求可重复、受控、可验证。硬件工程的物理约束最硬：力学、材料、制造、安全裕量，任何一处算错都要重新开模。

而清单是同一份，哪个领域都用它，一项不少。活也一样：设计开发在软件工程里落在过程上，在硬件工程里落在图纸与工艺上；需求、设计开发、产品数据、判定，哪一样都不缺。

区别只在权重。系统工程最吃科学素养与工程判断——接口管理、涌现意识，“部件全对不等于系统对”，靠的是这两项。软件工程的主战场是方法论素养与实践能力——过程不健全，代码写得再好也难以为继。硬件工程的红线是科学素养与职业伦理——安全裕量不是参数，是红线。

一句话：系统工程管“缝”，软件工程管“过程”，硬件工程管“物理”；清单一样，只是配比不同。本书的两个案例团队也正落在这条线上：冰链造有形的箱子，恒信做无形的系统。

> **判据**：换个领域，这份清单还成立吗？

### 9.2.10 沟通与团队，是能力的执行手段

清单有一个看起来的缺口：它没有单列“沟通能力”和“团队协作”。

这个缺口是真的。ABET 把沟通与团队列为独立的工程师成果（见 R-15）。工程教育界几十年补不上的，也正是这一块：雇主抱怨最多的，常常不是技术，是“说不清、合不来”。

但本书不把它们另立一项。原因是：它们不是另一项能力，是已有几项能力的执行手段。

- 职业伦理要让利益相关方知情——靠沟通；
- 工程判断要把取舍讲清楚、让判断被记录——靠沟通；
- 可复现意识要让方法被接住——还是靠沟通。

一个判断做得再对，说不清楚，等于没做；一套方法写得再好，不主动沟通，没人会用。团队协作同理：工程师几乎从不独自交付，他的能力要通过接口、评审、交接与他人接上。

怎么练？第一句动作是把你的一次取舍讲给别人听，看他能不能复述出你的依据。他复述得出的部分，是已经传出去的能力；复述不出的部分，还留在你身上。

> **判据**：我的判断讲出去，别人接得住吗？

对模块工程师和系统工程师都一样：沟通是让能力落到别人手上的方式。把它当可选项，前几项就永远只长在一个人身上。

## 案例进展①｜石军：手艺的极致

冰链的石军，手艺是第 2 章量过的那一份：判断全在他脑子里，不在纸上。本章看另一面：这份手艺在团队里留下了什么缺口。

这几个人，都绕不开石军那句“心里有数”。季昭要做验证，翻遍台账找不到成文的放行判定口径，只能拉着石军现场口述“哪几项测、容差多少”；小周排产，放行单卡了三天。

规程写着“依放行判定口径判定”，可口径在他脑子里，不在台账里；产线照转，却没人敢替他签这个字。

他只好拨通石军电话。

“这批 M3 周二出货，放行的判定口径还没进文档，你不在，谁敢签？”

电话那头，石军一句句报：哪个批次降额、哪家供应商得多留三天老化。报完，他自己接了一段：

“我报得出，可我说不清换个人怎么接。哪批降额、哪家多留三天老化，我能给你数，凭哪条原理、边界划在哪，我也说得上来；可要问换个人照这套判断做得出做不出——我答不上来。这些年我是试出来的，没记过。按判据说，这不是可复现的判断，是手感。”

小周一边记一边想：换个人连依据都问不出来；他要是走了，这条产线也跟着停。

林涛主持评审，最后总要追问一句“石军，你确认?”；吴浩盯实时数据，读数一不对，第一个电话就打给石军。

评审结论压在赵刚那儿。他签字前问一句：

“这一条谁验的？验证记录上哪一行写着'验过'?”

没人答得上来。赵刚签了字，照例在背面记一行：“判定依据待补。”

按这份清单量：石军的科学素养顶配、实践能力顶配、工程判断顶配（他懂原理，每个取舍都准）。但他缺方法论素养——方法说不成体系；缺可复现意识——判断换个人做不出来。

石军是①③⑤顶配、缺②⑥的人：手艺的极致，不是工程师的画像。清单第一次落到人身上，量出的是“有里子、缺面子”——里子是做对与守住的那几项（①③④⑤），面子是别人接不接得住的那两项（②⑥）。

把他放进粒度谱系看（§9.3.6），石军的岗位是**模块工程师**——嵌入式硬件，负责 M3 温控箱的**温控硬件与固件**这一个元素。而他干的是系统级的事：管 M3 整机追溯、拦下破坏数据所有权的直连、把老规矩写下来。**模块工程师的岗位，系统工程师的活**——先记下，§9.4 再展开。

---

## 9.3 往外划边界，往里分角色

上一节把清单立起来了。这一节办另一件事：把“工程师”和挨着它的东西分开。

先往外分：技术员、科学家、手艺人、会写代码的人。再看名义：头衔、证书、年限。最后往里划一道：工程师这个名字底下，不止一种人。清单里的能力，要划的是对外的边界；那几件活，要划的是对内的边界——谁在哪一级干、账归谁。

先说边界的分法。磨边界靠反例：正着说“工程师是什么”，容易各说各的；拿一个挨着的东西来比，界线就出来了。下面的每一条，都是同一个问法：它和工程师差在哪一条上。

### 9.3.1 不是技术员

技术员照单执行：操作规程怎么写，就怎么做；判据是什么，就按什么判。

工程师也执行。但工程师在规则没有覆盖的地方做判断。

两者的分界不是学历，是规则留白时谁拍板。技术员遇到表里没有的工况，会停下来问；工程师遇到表里没有的工况，要自己判断、自己负责。

这不是高下之分。一条成熟的产线上，多数工况本来就该被规则覆盖：那正是第 2 章说的“把交付从人身上挪到方法上”。要留神的是另一件事：规则覆盖不到的地方，如果也没有人判断，那里就是空的。

> **判据**：我做的事里，哪些是照单执行，哪些是规则没覆盖、要我拍板的？

### 9.3.2 不是科学家

科学家研究现存的世界，工程师创造从未存在的世界。第 2 章引过冯·卡门那句话，这里把它放到人身上。

科学家问“世界是什么”，工程师问“应该造出什么”。同一个问题，科学家要的是“说清”，工程师要的是“能用”。

失败的样子也不同。科学家的失败常常是可贵的——推翻一个假说，认识也往前走了；工程师的失败是灾难——桥垮了就垮了。

求真和求有用，是两种目标，也是两种物种。

但这不等于工程师可以不求真。上一节的①科学素养，要的正是求真的那一面；它只是服务于“能用”，不服务于“说清”。这两件事在一个人身上可以同时长着，判据却不一样：说得清不等于造得出。

> **判据**：我手上这件事，是在把世界的道理说清楚，还是在造一个还没有的东西？

### 9.3.3 不是手艺人

这一对最容易混，因为两边都“很厉害”。

区别只有一条：**交付依不依赖具体的人**。

手艺人靠个人经验与技巧，结果绑在人身上——石军在，质量在；石军不在，质量跟着走。工程师靠可复现的方法，结果能脱离具体的人——换个人，用同样的需求和约束，能交付质量相当的成果。

第 2 章那条试金石，到这里落成一条角色判据：

> **如果你不在，交付还能不能维持？能，你是工程师；不能，你是手艺人——哪怕你也会写代码、画图纸**。

手艺不是贬义词。石军几十年的经验是团队最值钱的资产。但“个人厉害”和“工程厉害”是两回事：前者依赖人，后者可复制。

> **判据**：我这一摊活，我不在的时候，还转不转？

### 9.3.4 不是“会写代码的人”

这是最流行的一对混淆，区别只有一条：**写代码是工程师的载体，不是工程师的判据**。

一个会写代码的人，可能只是把需求翻译成程序的手艺人：判断留在规则里，代码写成什么样全凭个人手感。工程师也写代码，但他在写之前问“为什么这么设计”，写完问“换个人能维护吗”，上线前问“这一改会不会破坏别的”。

载体人人可以有，判据不是人人有。把“会写代码”当成“工程师”，就像把“会用刀”当成“厨师”——刀人人会用，火候不是人人有。

> **判据**：我做的事里，有多少是载体，有多少是判断？

### 9.3.5 不是头衔、证书、年限

回到章首那场面试。头衔是组织发给你的位置：可以被印上，也可以被收走。证书是社会给你的认证：它证明你通过了一次考试，不证明你天天在做工程。年资是时间的累积：干了二十年，也可能二十年都停在同一套手艺里。

**名义不定义能力**。头衔、证书、年限合起来，都回答不了章首那个问题。

判断一个人是不是工程师，可检验的不在名片上，在这份清单上。而且不是看他能不能背出清单，而是看他的判断换个人接不接得住、压力面前守不守得住底线。

> **判据**：我认一个人是工程师，凭的是他的名义，还是他的清单？

### 9.3.6 不止一种角色：专责与终责

最后一道边界在内部。“工程师”这个名字底下，不止一种人。工程实践横跨不同的尺度：有人负责系统的一个元素，有人负责整个系统，有人负责系统的**架构**（基本概念或属性与管控原则）与它的守持。它们的大小差着量级，责任也不一样。这条尺度，本书叫**粒度**。

| 角色 | 负责什么 | 责任性质 |
| --- | --- | --- |
| **模块工程师** | 系统元素（system element，15288:2023 §3.47）或模块的实现 | 专责——划得清的账 |
| **系统工程师** | 系统全生存周期的技术事务 | **终责——甩不掉的账，不可委托** |
| **系统架构师** | 架构（基本概念或属性与管控原则）的成立与守持 | 专责——可委托、可追责 |

先说责任性质，这是最硬的一条分界。

系统工程师对“系统成败”负的是**终责**（accountability）：出了事他兜底，这本账不可委托给任何人。模块工程师和系统架构师对“自己那一块”负的是**专责**（responsibility）：这本账可以委托出去，也可以被追责。一句话：专责≠终责——终责是甩不掉的账，专责是划得清的账。

架构师把架构搞砸了，追责追到架构师；但对外向客户、向监管扛雷的，是系统工程师。

再说分形。模块工程师不是“小一号的系统工程师”那么粗：他是系统工程师的分形：在模块这一级，他跑的是与系统级同构的完整闭环。系统级走的是“需要与期望→规定需求→架构→设计→实现与集成→验证与确认”，模块级走的是“模块需求→模块设计→模块实现

→模块验证”。站在自己那一级，他什么都要管，一样不少。本书因此把模块工程师叫“迷你系统工程师”。

但分形有一处不对称，恰恰是最要紧的地方。

模块级比系统级少的是这几样，不是遗漏，是结构：**对外的那部分架构**、确认、部署、集成方案。对外的那部分架构说的是，模块的边界、接口与职责来自上层架构的分配，模块工程师不能改；确认要落到真实或模拟的使用条件下，由系统级来兜；部署与集成方案都发生在系统级。

少掉的这几样在别处；归他自己的那一块也很清楚：模块**自身的架构**——它的基本概念或属性与管控原则（内部结构、内部依赖、内部接口、实现策略是实现它们的载体）——由模块工程师自己定义。在自己的边界内，他就是模块这一级的架构师。边界内自治，边界外听架构的。

这里补一句，免得和另一种说法相撞。另有一种描述把工程师说成“在单一学科内把分配到的需求做成部件/子系统的执行者”（本书引作对照）。从学科分工那一面看，工程师是把分配到的需求做成部件的人；从闭环那一面看，模块这一级又是完整的一圈。两种说法说的是同一件事的两面：模块工程师不必跨学科整合，但在他自己那一级上，需求、设计、实现、验证一样不缺。

系统工程师是把系统工程落到项目里的那个人。他不是头衔，是角色。按 24765 的角色词条模式，系统工程师指实践系统工程学科的人，与其正式头衔无关。这一句是类推：所本是 24765 同书 systems security engineer 词条的句式（见 R-20）。15288:2023 §3.50 给的是“系统工程”这一途径的定义，正本逐字 transdisciplinary and integrative approach。你叫架构师还是 CTO，只要在实践系统工程，你就是系统工程师。

他的职责可以收成这样几个动作：转化（把需要、期望与约束翻成规定需求，再导向方案）、统筹（跨学科整合，让各专业域协同成一个系统）、把关（证明“做对了”且“做了对的东西”）、守护（让系统在生存周期内持续正确、可控地演化）。这几个动作是本书的凝练，不是标准的原文。

同一副骨架，粒度不同。这份清单，哪一种粒度都要用，一项不能少：模块工程师要科学素养，系统工程师要，架构师也要。区别不在能力种类，在粒度与责任——模块工程师把它用在“我这个元素”上，系统工程师用在“整个系统”上，架构师用在“**架构的成立与守持**”上。

系统架构师是这条谱系上的另一个角色——工程师角色体系中专责“架构定义”的那一个。本章先把他放进谱系里点个名，第 18 章再专门把他从头到尾量一遍。

> **判据**：我背的账，是专责还是终责？我清楚自己站在哪一级吗？

## 案例进展②｜罗建：伪工程

恒信的罗建，团队上了敏捷、上了持续集成、上了测试，方法论和工具都在。

但按这份清单量：他有②方法论素养、③实践能力，缺①科学素养——容量评估被砍，他不追问“凭什么能行”；缺④职业伦理——为进度牺牲可靠性，没让任何人知情。更致命的是，持续集成靠的是三个人的私藏，⑥同样站不住。

这些缺项，最先落到他身边每个人头上。

容量评估被砍那天，苏婷问过一句。

“不做容量评估，月末高峰顶得住吗？验收清单上没有这一项，到时候谁验？”

罗建摆摆手：“先上再说，容量评估下次再补。”

曹峰那边也一样：日期是他压的，评估被砍，他到宕机那天才知道。

“按第 2 章 §2.3.3 那三问，头一问是'利益相关方知不知情'。这一问，我这儿也是空的。”

郑亮直到上个月那场事故——数据库连接池耗尽、合同审批系统宕机四十分钟、法务和财务几百个审批卡住——才知道评估早被砍了。他把这句话补完：

“我不管你们的工具链怎么搭。我就问一句：卡死那天，谁提前告诉过我评估被砍了？容量评估问的是'凭什么能行'，脚本只在他那台机器上跑得顺，问的是'换个人接得住吗'——按判据算，一笔缺的是科学素养，一笔缺的是可复现意识；没让利益相关方知情，缺的是职业伦理。”

王默的委屈在发版。上线那晚，部署脚本换台机器就跑不动——缺的还是那三条命令和两个环境变量；环境是罗建亲手搭的，可跑通它要什么，分记在三个人手里，没进任何文档。罗建一出差，发版就只能等他回来补一句口令。王默嘀咕：“这哪是持续集成，是他一个人的发版。”

罗建是有②③、缺①④⑥的人——方法论和工具都在，人这一侧却站不稳；这就是伪工程：形状在，底座不在。

他说的和做的对不上。判词从“初挂”推进到“为什么缺”：缺的正是关键时刻护不住的①与④。

把他放进粒度谱系看，罗建的名片上印着**架构师**——谱系里的系统架构师。而架构师恰恰最不能缺底座：系统级判断全长在这份清单上，底座缺一块，增量全是空转。**他缺的不是画图，是底座**——这一笔，第 18 章接着算。

## 9.4 从会做，到能判

边界划清了，剩下的问题只有一个：从哪一步开始，按什么次序走。

这一节给出一条路线：路上有几道关口，先过哪一道，每道关口上做什么、怎么算过了；走岔了会是什么样子；最后落到一件看得见的东西上。

路线是这样：先能亲手做出来；再过“从我会做到别人会做”；再过“从照做到会取舍”；最后过“知道什么不能牺牲”。这几道关口过完，一个工程师就立起来了。再往前走，还有一条横向的路：把责任范围从模块扩到系统。

### 9.4.1 亲手做出来

清单上的每一项，都要从“做”里长出来：科学素养要在做里被验证，方法论素养要从做过的做法里提炼，工程判断要在真实的取舍里练。不上手，前几项都长不出来。

所以起点不是读书，也不是听课，是上手。而且起点不必高——照猫画虎做出第一件东西，也算站在起点上了。第③项实践能力那条强度光谱（阅读、看演示、跟做、仿做、独立做、救火）说的就是这一站的深浅：先站上去，再往里走。

这一站最常见的错，是拿“学过”当“做过”。学过的人会说，做过的人能拿出来。

> **判据**：我最近一次亲手把东西做出来，是什么时候？

### 9.4.2 从我会做到别人会做

这道关口上，人的分岔最明显。有人停在这里，成了手艺的极致；有人过去了，成了工程师。石军就停在这里——不是他不会做，是他的做没法被人接住。

过关靠的是同一件事的两面。

**写下来**：把你做对的那件事写成方法——步骤、依据、坑。这是②方法论素养。**交出去**：写完主动交给别人用，看他能不能照做。这是⑥可复现意识。

只写不交，方法躺在文档里；只交不写，接的人还得靠问。两样都做完，你的能力第一次离开你本人——它不再只长在你身上。

这也是石军那句“心里有数”的分水岭。判断在心里，是手艺；判断在纸上、别人照着能做出质量相当的成果，才是工程。

> **判据**：如果我明天走人，团队还能交付同等质量吗？

### 9.4.3 从照做到会取舍

新手问“这一步怎么做”——照单执行；熟手问“这一步有几种做法”——开始会选；资深问“这里要不要再选一次”——会权衡；工程师问“这个取舍守不守得住”——知道什么不能牺牲。判断的依据从“怎么做”升到“守什么”，物种才从执行者变成工程师。

这条上升的线，底下垫的是①科学素养和⑤工程判断。**①科学素养**——会取舍的前提是知道依据成不成立，不然取舍凭的是手感。**⑤工程判断**——在约束里做对的取舍，还要能担责。

只升尺度、不补依据的人，会变成“很有主见但说不清依据”的那种人；只补依据、不升尺度的人，会一直停在“照做”上。两样都得动。

> **判据**：规则没有覆盖的地方，我能做对的取舍吗？

### 9.4.4 知道什么不能牺牲

前面几道关口练的是“能做事”和“会取舍”，这一道练的是方向。判断越准、能力越强，方向错了破坏越大，这就是能力不中性的意思。

这一站的标志很朴素：**第一次为一件“做了也没人反对”的事说不**。那个“不”的第一个对象，常常是自己的冲动——“这个功能月底要上，容量评估下次再补”，说不；“先上再说，反正没人查”，说不。

说不是动作，护住是结果：利益相关方知情，有记录，有机制兜底。一个人是不是真的过了这一站，不看他说得多硬气，看他护住的那件事有没有留下痕迹。

> **判据**：我上一次为“没人反对的事”说不，是什么时候？利益相关方知道吗？

### 9.4.5 从模块工程师走到系统工程师

几道关口过完，一个人已经是一个工程师了。再往前走，有两条方向不同的路。

一条是把责任范围往外扩——从模块工程师走到系统工程师；另一条是往管理走——技术经理、项目总监。两条没有高低，只有方向不同。本章讲前一条，因为它是本书的行程；后一条也正当，只是不在这本书的路线里。

拓宽不是技能叠加，是视角换轨加责任接管：从对“我的模块”负责，切到对“整个系统”负责；优化目标从“单点最优”变成“系统最优”；而系统最优，常常要求某个模块让利。

把视角摆在一起看：

| 维度 | 模块视角 | 系统视角 |
| --- | --- | --- |
| 关注对象 | 我的模块内部：内聚、性能、实现 | 模块之间、系统与环境 |
| 完成标准 | 模块验证通过，任务完成 | 验证加确认，看整个生存周期 |
| 协作方式 | 接口按文档走，其余不管 | 涌现行为、系统级权衡 |
| 优化目标 | 单点最优 | 系统最优（可以牺牲单点） |
| 责任 | 专责，元素实现 | 终责，系统成败 |

换轨有几个低成本的入口：主动认领跨模块任务（集成测试支持、跨模块缺陷的根因分析——被迫同时看多个模块和接口）；去需求评审现场坐着，哪怕旁听，看“我的模块需求”怎么从“系统需求”分配下来。还有一个入口：把“验证”升级成“验证加确认”，主动问一句“模块验证通过，但系统真的做对了吗”。

换轨有一个标志，很朴素：第一次因为系统牺牲了自己的模块，还觉得值。反过来，如果永远把“我的模块”护在最前，那只是模块工程师做得久，不是系统工程师长得成。

要分清两件事：**升职是组织给你换位置，拓宽是你自己换轨道**。被任命的粒度，也不等于实际承担的粒度：石军的名片是模块工程师，干的是系统级的活；反过来，挂着系统工程师头衔只做单点开发的人，也不是系统工程师。**粒度看的是你背的账，不是名片**。

> **判据**：我愿意为系统牺牲自己的模块吗？我是在单点最优，还是系统最优？

### 9.4.6 从失败看缺项

路线不只往长成的方向走，也可能走岔。工程师走岔的样子，常见的是这几种。

只会做，不会说。活干得漂亮，判断全在心里，从不成文。交付依赖他一个人，他一休假，产线就停转。缺的是⑥可复现意识。

只会说，不会做。方法论讲得头头是道，流程、规范、最佳实践张口就来，但落不了地。会做的人判断在手上，这种人的判断在嘴上。缺的是③实践能力。

守不住底线。症状不是“不会做”，是“做的时候没守住”。泰坦尼克号按“不会沉”配的救生艇，挑战者号在工程师的反对声中升空，737 MAX 的安全需求在进度与市场的压力下没有被认真对待，一路让了位；这些事故的共同点不是不会做，是守不住（事故经过见第 2 章 §2.3.3）。缺的是④职业伦理。

能力失控。最危险的一种，因为他看起来最优秀：能力越强、这一项越缺，方向错了破坏越大。缺的仍是④职业伦理那一项，只是败法不同：不是守不住，是方向偏了。

每一种都有可观察的标记：看只会做的，看他休假时产线停不停；看只会说的，看他说的能不能换成任何一个具体的决定；看守不住底线的，看他上次为进度砍了什么、利益相关方知不知道；看能力失控的，看他最得意的那个决定方向对不对。

这几种死法死于不同的地方——说不清、做不了、没底线、方向偏——但缺的是同一份清单上的空格。

> **判据**：我最近一次失手，缺的是哪一项？

### 9.4.7 拿产品数据检验

路线走完了，最后一个问题：凭什么说一个人真的长成了。

清单上的能力长在人身上，看不见摸不着。面试会看走眼，领导会凭印象打分，连他自己也容易高估。能力要能被检验，就必须落到东西上。

落到什么上？产品数据。第 7 章立过：开发产品就是开发产品数据；第 6 章那棵树的叶子，就是产品数据。看一个工程师，不看嘴上说得多满，看他能不能开发出高质量的产品数据——换个人拿着它，做得出、接着种得动。

每一项能力，都有一条别人看得见的到达证据：

| 能力 | 到达证据（谁能看见、看见什么） |
| --- | --- |
| ① 科学素养 | 你留下的预测对账记录：改动前写的预测、实际偏差、以及“哪条原理越了界”的结论 |
| ② 方法论素养 | 别人照你写的步骤做，结果稳定；他卡住的地方，就是你还没显性化到位的地方 |
| ③ 实践能力 | 最近一次你亲手落地的东西在跑——不是你说过，也不是你评审过 |
| ④ 职业伦理 | 你为一件“做了也没人反对的事”说不，且利益相关方知情、有记录 |
| ⑤ 工程判断 | 一个规则没覆盖的取舍：选了、写了依据、有人知道 |
| ⑥ 可复现意识 | 你休假两周，产线不停 |

他干的活也有：

| | |
|---|---|
| 他干的活 | 到达证据（谁能看见、看见什么） |
| 需求 | 一份规定需求：明示、可采（能当依据用）、三类内容齐全，而且进了基线 |
| 设计开发 | 一棵树：每条需求落到一个能负责它的节点上 |
| 产品数据 | 一份可验证、如实规定、换个人能接着种的产品数据 |
| 评审、验证与确认 | 三次判定各对其对象（本书凝练，见 §9.2.8），有证据、有记录 |

有一件事要说清，免得看起来数了两遍：产品数据既是这条路上的第三段，也是整份清单的物证。它既在路上被造出来，又被拿在手上量这份清单；两重身份，同一件东西。

按粒度分账，各人负责的那一套产品数据也不一样。这里的粒度分账是本书的工作清单，不是标准的条文（这张表按工件分账，角色在生存周期各过程里的参与形态是另一张表，见第 18 章）：

| 粒度 | 负责的产品数据 | 与其他粒度的关系 |
|---|---|---|
| **系统架构师** | 架构描述（两样内容：基本概念或属性与管控原则） | 主责；模块级边界由它分配 |
| **系统工程师** | 利益相关方需求、运行概念、系统需求、系统详细定义、技术方案、实现方案、集成方案、集成 | 主责系统级；**批准**模块级 |

| 粒度 | 负责的产品数据 | 与其他粒度的关系 |
| --- | --- | --- |
| | 测试、验证、确认、部署、运维等系统级工件（不含架构描述） | |
| **模块工程师** | 模块需求、模块详细定义、模块实现方案与实现报告、模块测试、模块验证、模块运维 | 主责模块级；对外边界听架构的，系统工程师批准 |

系统工程师是唯一对系统成败负终责的角色：系统级那一套几乎全由他主责，模块级那一套全由他批准；模块工程师干活，他验收。这就是终责落在谁身上：谁负责的东西最多、又要给别人的东西签字，谁就是那个兜底的。

**粒度隔离就是模块化的意义**：系统工程师不需要知道模块内部的细节，只需要模块的接口和验证结果；模块工程师在自己的边界内自治，把成果交上去。

还有一件事要分清：**产品数据以规定为主，不是记录**。它写在实现之前，声明“产品应该是这样”，被采购、制造、测试、运维照着执行；记录写在实现之后，只在被审计、复盘时查阅。两者都在产品数据里，但工程师的看家本领是前者：把“应该是这样”规定清楚，让别人照做，让验证有标尺。

所以，看一个工程师，不看他说得多满，只看他能不能开发出高质量的产品数据。能力再强，开发不出高质量的产品数据，这个工程师也不成立。高质量不是写得多，是叶子站得住：可验证、如实规定、换个人能接着种。

> **判据**：换个人拿着我负责的那套产品数据，能不能把产品做出来、继续演进？

## 案例进展③｜石军与罗建：都还在长成

那么，谁是工程师？

**石军**：判断是顶配，但不可复制。他有工程师的里子——做对与守住的那几项都在场；缺“换个人能接住”的面子——表达与复现都不在场。他是模块工程师的岗位，做着系统工程师的活，缺的是把一切写成方法的②⑥。

**罗建**：方法能说清，但守不住底线。他有工程师的面子——方法能说清；复现那一半却站不住；缺“关键时刻护住”的里子——①科学素养与④职业伦理不在场（③实践能力、⑤工程判断在场）。他的名片是架构师，缺的却是架构师最不能缺的底座。

两个人都不是完整的工程师。但两个人都值得被叫一句“在成为工程师的路上”，因为他们都在用这份清单量自己，都在补自己缺的那一项。

面试散场，罗建没有走。

“按这份清单，我算不算？”

顾言没有直接答：“第 2 章 §2.2.4 那条试金石，量的是一个动作：你的判断离开你以后，还在不在。清单两组，一组问能力，一组问活；活那四段从第 3 章排到第 8 章，每一段都要拿产品数据去对。今天那三个人，只答了前一半。”

**判词和上一节量的是同一句话**。石军缺的那一半，正是“换个人拿你的产品数据做得出、接着种得动”那一半；罗建缺的那一半，是“产品数据在，却不可信”那一半。一句话落到两个人身上，量出两种缺口。

### 9.4.8 认知反转

回到章首那场面试。会写代码的、有证书的、干了二十年的，都答不上顾言那个问题。

现在我们知道，那个问题问的是**可复现**：换个人能交付吗？三个人都答不上来，因为他们都把“工程师”定义在了自己身上：会写代码的人、有证书的人、有经验的人。

定义在自己身上的，都不稳；定义在能力上的，才稳。而能力最终只被一件事检验：你开发出的产品数据；换个人接着种得动，才叫稳。

本章还多打碎了一个定义：把“工程师”定义在一个名词上，同样不稳。工程师不是某一个人的称呼，是一种统称；同一种统称底下，站着好几种粒度。

把模块工程师、系统工程师、系统架构师放进同一句话里叫“工程师”，就像把树叶、树干、树根都叫“树”——都是树，各管一段，账不是同一本。理解了粒度，你才不会拿模块工程师的尺子去量系统工程师，也不会拿系统工程师的账去问架构师。

人们把一个本该由能力定义的东西，交给了载体、头衔和年资去定义——这就是“工程师”这个词的全部误解。

这本书要带你走的，是从工程师到架构师的路。这条路的起点，就是本章量的这份清单。

---

## 小结

本章把前面七章收拢到一个人身上：工程师首先是一份能力清单。能力来自第 2 章那几层，各长一项，另加横切的工程判断与元的可复现意识；活来自第 3 到第 8 章，需求、设计开发、产品数据、评

审验证确认，按第 3 章那条路径依次走一遍。这份清单是模块工程师、系统工程师、系统架构师共用的底座。

它划清了外面的边界：技术员照单执行，科学家求真，手艺人依赖个人，会写代码只是载体，头衔、证书、年限都不是判据。它也划清了内部那道边界：同一个统称底下，账不是同一本，模块工程师负元素实现的专责，系统工程师负系统成败的终责（不可委托），系统架构师负**架构的成立与守持**的专责；模块工程师是系统工程师的分形，而分形不对称。

它给出一条路线：先亲手做出来，再过“从我会做到别人会做”，再过“从照做到会取舍”，最后过“知道什么不能牺牲”；再往前走，是从模块工程师到系统工程师的拓宽，而拓宽是换轨，不是升职。走岔的样子也给了，每一步的到达证据也给了。

最后，它把能力从“人身上”挪到“纸面上”：能力最终体现在能不能开发出高质量的产品数据——换个人拿着它，做得出、接着种得动，能力才成立。

下一章，我们看全书概念的重心——架构：先把这个词说准——给它一个能对账的定义，并清掉几处最贵的误解。架构师，就是在这个底座上长出系统级判断的人（第 18 章会给全那份清单）。

散会时，郑亮说了句实在话。

“招工程师，我就认一条判据——换个人，这活也接得住。石军的判断换个人接不住，罗建的'先上再说'换个人也守不住：一个缺'别人也会做'，一个缺'守住底线'。这两笔账，最后都在产品数据上看得见。”

罗建没接话。他知道，这句自己也答不上来。

# 附录

## A 本章概念总图

工程师：一份能力清单（不是头衔、证书、年限）

一条判据（全章唯一）：

换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果

├── 清单里的能力（问"我有没有"；源＝第2章那几层）：

│ ① 科学素养（凭什么能行） ② 方法论素养（把工作组织成方法）

│ ③ 实践能力（亲手把方案变成现实） ④ 职业伦理（能力不中性，它定义方向）

│ ⑤ 工程判断（横切在上面） ⑥ 可复现意识（元能力）

├── 清单里的活（问"我会不会干"；源＝第3到第8章，按第3章那条路径依次走一遍）：

│ 需求（从利益相关方的需要与期望，开发成能当依据用的规定需求；第3、4、5章）

│ → 设计开发（由需求到方案的逐对生成；第3、6章）

│ → 产品数据（这条路径上造出来的东西；第6、7章）

│ → 评审、验证与确认（拿证据判"对没对"；第8章）

├── 侧面：粒度（在哪一级被检验）/ 熟练度（照做→会选→会取舍→守底线）/

│ 走岔的样子（只会做 / 只会说 / 守不住底线 / 能力失控）

└── 物证：产品数据（换个人拿着它，做得出、接着种得动）——

它既是这条路上的第三段，也是量整份清单的那件物证

一份清单从哪来：能力＝第2章那套分层（工程科学 / 方法论 / 工具与实践 / 价值与伦理）各长一项，

另加横切的⑤与元的⑥（以 ABET 工程师成果七条对照凝练，本书主张）；

活＝第3到第8章，次序由第3章那条路径给定

清单不是什么：不是技术员 / 不是科学家 / 不是手艺人 / 不是会写代码的人 /

不是头衔、证书、年限

同一个统称，不同的账：模块工程师（专责）/ 系统工程师（终责 • 不可委托）/
系统架构师（专责）；模块工程师是系统工程师的分形，
分形不对称（模块级没有对外的那部分架构 / 确认 / 部署 / 集成方案，
模块自身的架构归自己）

怎么长成（一条路线）：
亲手做出来 → 从我会做到别人会做 → 从照做到会取舍 → 知道什么不能牺牲
→ 横向的路：从模块工程师走到系统工程师（换轨标志：第一次为系统牺牲自己的模块还觉得值）
→ 证据：产品数据（按粒度分账）

往前追：能力来自第2章，活来自第3到第8章。
往后落：可复现意识长成第15章的基线，职业伦理长成第16章的治理；
第18章再加上系统级的增量，合成架构师的一份清单。

判词：石军①③⑤顶配、缺②⑥（接不住）| 罗建有②③、缺①④⑥（守不住）：都还在长成
下一章：说准"架构"——定义与误解

## B 本章的能力级

**第 ③ 级为主。**

- **表征**：把“工程师”和挨着它的词说准、分开——技术员／科学家／手艺人／头衔、证书、年限／模块工程师、系统工程师、系统架构师／终责与专责。
- **图式**：一份能力清单的构造——能力（分层各长一项，另加横切的工程判断与元的可复现意识）＋活（需求、设计开发、产品数据、评审验证确认，按第 3 章那条路径依次走一遍）；同一个统称底下的粒度谱系；模块工程师是系统工程师的分形，且分形不对称。

- **心智模型**：能推演“缺哪一项会走岔成什么样”；能说出这份清单从哪来（能力来自第 2 章那几层，活来自第 3 到第 8 章）；能力不中性、职业伦理定方向；能力最终被产品数据检验；粒度决定账本。
- **解释框架**：本章不建（归第 17、18 章）。

用第 1 章 §1.4.4 的尺子量一下：本章把工程师这个词从第 ① 级（表征）推到第 ② 级（图式）——清单的构造、统称底下的粒度谱系、模块与系统的分形关系，都成了认得出的结构；再往前一档是第 ③ 级（心智模型）：能推演缺哪一项会走岔成什么样。本章以第 ③ 级为主；第 ④ 级（解释框架）不建，归第 17、18 章。

能答全四问才算拥有——你能拿着这份清单，量别人，也量自己。

## C 自查 · 这些说法对吗？

1. “会写代码的人，就是工程师。”——**错**。写代码是载体，不是判据；工程师在规则留白处判断。
2. “有职称、有证书，就是工程师。”——**错**。证书只证明考过一次试；名义不定义能力。
3. “干了二十年，一定是好工程师。”——**错**。二十年也可能都停在同一套手艺里；可复现才是工程。
4. “工程师就是照着流程做的人。”——**错**。那是技术员；工程师在流程没有覆盖的地方判断。
5. “工程师只要技术好，伦理是管理者的事。”——**错**。职业伦理是清单里的一项，缺了会出事；而且能力越强，破坏越大。
6. “能力越强的工程师越值得信赖。”——**错**。能力不中性，职业伦理定方向。
7. “把经验写成文档，就算工程化了。”——**对了一半**。显性化只是第一步，还要可检验：换个人照着做，质量相当。

8. “我的判断，不需要别人接得住。”——**错**。可复现意识就是让判断被接住；接不住的只是手艺。
9. “工程师的追求是零错误。”——**错**。工程要的是错误可见，不是零错误（第 2 章）。
10. “拿到头衔就是工程师了，一辈子都是。”——**错**。头衔只是别人给你的位置；清单一项项亮起来，才算成为。
11. “从新手到工程师，能力慢慢堆出来就行。”——**错**。判断的依据要换尺度，从“怎么做”到“守什么”。
12. “模块工程师就是小一号的系统工程师。”——**错**。他是同构的分形，但模块级没有**对外的那部分架构**、确认、部署与集成方案——而**模块自身的架构**恰恰归他自己。
13. “系统工程师是比模块工程师更高的职级。”——**错**。粒度不是职级，是责任性质；终责与专责不是高低。
14. “挂着头衔、只做单点开发，也是系统工程师。”——**错**。粒度看的是你背的账，不是名片。
15. “一个工程师好不好，聊一聊就能判断。”——**错**。能力长在人身上，不可直接检验；最终看产品数据——换个人拿着它，做得出、接着种得动。
16. “产品数据就是记录。”——**错**。产品数据以规定为主，记录也在其中；写在实现之前的规定才是它的主干。
17. “清单上有能力就够了，干没干过不重要。”——**错**。能力问“我有没有”，活问“我会不会干”；两样都答得上来，才叫是个工程师。

## D 练习

1. **体检**：用这份清单给“你们公司最像工程师的人”做一次体检——逐项打勾，指出缺项，并说出缺这一项会走岔成哪一种。

2. 粒度定位：把你们团队的核心成员放进谱系（模块工程师／系统工程师／系统架构师），说出每个人背的是专责还是终责——有没有“名片一个粒度、实际另一个粒度”的人？
3. **起点**：你最近一次亲手把东西做出来，是什么时候？如果答不上来，先做这一件事。
4. 第一道关口：把你最近一次做对的事写成一份可交接的记录（步骤、依据、坑），交给一个没参与过的同事照做一次——他卡在哪一步？
5. 第二道关口：写下你最近一次重要取舍——当时哪些约束在打架、选了哪个、依据是什么、后果如何、重来会怎么选。
6. 第三道关口：写下你的不可越线清单（安全、合规、隐私、数据真实性各一条），再找出最近一次“为进度让路”的现场——利益相关方知情吗？有记录吗？
7. **证据**：拿你负责的那套产品数据，问一句“换个人拿着它，能不能做出来、继续演进”——答不出的部分，就是你的缺口。
8. 拓宽自测：你现在的视角是模块视角还是系统视角？你最近一次“为系统牺牲自己的模块”，是什么时候？
9. **清单对表**：把你手上这摊活按“需求 → 设计开发 → 产品数据 → 判定”摆一遍。看哪一段断了，就回头读哪一章。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022 —— §3.2 架构定义（基本概念或属性，以及为实现和演化而设的管控原则）
- [2] **R-09** ISO/IEC/IEEE 15288:2015 —— 历史版本行（本章引用一律走 2023，即 R-37）
- [3] **R-15** ECPD 1941／ABET 1979／INCOSE 2019 —— ABET 工程师成果七条；ABET 把工程定义为“运用判断力加以应用”（applied with judgment）；INCOSE 2019 的工程系统定义
- [4] **R-18** CMMI —— 工程方法论举例
- [5] **R-20** ISO/IEC/IEEE 24765 —— module 与系统工程师（本章“系统工程师”一句系按 24765 的角色词条模式类推）
- [6] **R-37** ISO/IEC/IEEE 15288:2023 —— §3.47 system element（系统元素：系统的离散部分，能加以实现以满足规定的需求）；§3.50 systems engineering（系统工程——一种跨学科、综合性的途径）；**§6.4.2 利益相关方需要与需求定义过程** —— §9.2.8“把需要与期望开发成规定需求”的依据
- [7] **R-38** ISO 9000:2026（第 5 版）—— §3.10.6 competence（ability to apply knowledge and skills to achieve intended results，应用知识和技能以取得预期结果的能力）：能力是干成事的本领，不是背下来的知识点，所以这份清单只能逐项自问；**§3.3.11 设计和开发**（set of processes that transform requirements for an object into more detailed requirements for that object）—— §9.2.3 引它说“一组过程，就是一套方法”（**本书读法**）；§3.11.2／§3.11.12／§3.11.14 评审、验证、确认—— §9.2.8 三条判定的出处；**§3.5.1＋注 2**（规定的需求＝明示的需求）、**§3.11.1 确定**（三判定的根术语）—— §9.2.8“规定需求”与“判定”的锚

**经典文献**

- [8] **R-62** NSPE Code of Ethics —— 工程师的服务必须致力于保护公众的健康、安全与福祉
- [9] **R-86** INCOSE Systems Engineering Vision 2035 —— 系统工程师的能力分组与“从模块走向系统”的能力缺口


> **术语注（跨标准用词）**：本书引 ISO 9000 原文处**照正本用词**；本章自撰正文一律作“利益相关方”（与全书一致）。本章能力清单的回扣链：§2.2.4（试金石）、§2.3.3（分层与价值失守的事故）、§2.3.4（自动化）；第 6 章（那棵树）、第 7 章（产品数据）、第 8 章（三道门禁）；第 10 章（架构）、第 18 章（架构师）；附录 A T-25（工程师）、T-113~T-117（系统工程师／模块工程师／系统架构师／粒度／终责专责）、附录 C 第 9 章行、附录 E 第 9 章块。

# 第 10 章 架构概念：定义、价值与误解

## 10.1 引言

架构是这个行业里最重的词之一，也是被用得最松的词之一。评审会上有人说“我们的架构是微服务”，共享盘里躺着一份“M3-架构 v1.0”，有人把拓扑图、部署图、ER 图统统叫架构，还有人把“这次改个字段”说成“架构变了”。

这些说法里，最要紧的是两个误解。一个把**架构描述**当成了架构——那是架构被写下来、画出来、存起来之后的样子。按 ISO/IEC/IEEE 42010:2022 的区分，架构描述是有形的工作产品，架构本身无形、抽象（该标准 Introduction 逐字：Whereas an AD is a tangible work product, an architecture is intangible and abstract, understood through its concepts, properties and principles.）。能签收、能入库、能版本化的，是架构描述；架构是它所承载的那份内容。

另一个把系统的结构、以及设计定义的高层级（概要设计）当成了架构。这一族更难认，因为它们跟架构挨得很近：都能画出来，都能开会讨论，都能往下传。可差别不在粗细，在种类——能被设计违反的那一层才是架构，系统的结构、概要设计本身不是架构。

同一个词罩着几样东西，误读就从这里长出来。

本章把架构的定义说准，把这个定义意味着什么讲透，再把贴着这个词长出来的误读逐个清掉。

### 10.1.1 误解现场

架构评审会，冰链科技，周四上午。嵌入式硬件工程师石军把 U 盘插上投影。一张图，四层方块，从下到上依次是硬件、固件、云端、App；一根箭头从传感器一路画到 App，旁边标着“温度数据”。

“这就是我们 M3 的架构，也是这一版的概要设计。”石军讲了十分钟。“硬件和固件是我这边，云端是吴浩他们三个，测试的季昭，验证与确认归他。服务边界和接口，等详细设计再细化。有问题的现在提。”

产品开发经理赵刚指了指那张图：“先记一个问题——客户问'M3 是什么系统'，我们能拿这张图答吗?”

制造侧的小周举手：“我有个问题。这图里没有'我们怎么造它'——哪道工序测什么、哪个批次装哪个固件?”

石军：“架构图讲的是 M3 的结构——四层、一根主数据流。系统长什么样，不是怎么造的。”

小周点点头，没再问。散会前，石军把这张图存进共享盘，文件名“M3-架构 v1.0”。它是立项报告里的方案框图，他只把标题从“方案”改成了“架构”。此后三个月，没人打开过它。

产品经理林涛想调需求：“温度数据上传间隔从 1 分钟改成 5 分钟，影响什么?”没人答得上来。图上只有一根叫“温度数据”的箭头，没说间隔定在哪一层。

石军想加“断电补传”，问云端团队：“重连逻辑在哪一端?”云端说这是固件的事。可固件就是石军自己写的，他知道里面没有重连逻辑。两边在图里找依据，图里没有。

还有一个代价更大的问题。林涛带 M3 谈一笔冷链订单，客户问：“你们 M3 到底是什么系统?”石军答“温控箱”，吴浩答“追溯平台”，

云端那边做 App 的又答“冷链管理 App”。三个答案，没有一个对得上。饭桌上安静了一会儿，那笔订单后来没了下文。

石军把图翻出来想维护一版新的，对着它看了很久，忽然意识到一件令人后背发凉的事：**这张图从来没有描述过真正的 M3**。

坐在那间会议室里的人可能觉得他们挺冤——图画了，评审开了，还要怎样？本章要推翻的正是这个直觉。那张四层方块图在架构领域有个准确的名字：**架构描述**，而且只是其中极粗糙的一小块。石军把它当成了架构本身，于是图烂掉的时候，他以为架构也烂了——可 M3 的架构一直活着，只是从没有人把它写下来。

### 10.1.2 本章要回答什么

读完本章，你应该能回答：

1. 架构到底是什么？为什么“画一张框图”不算做了架构？
2. 权威定义里的两半各管什么？“或”与“和”有什么不同？
3. 架构与架构描述，哪一份才是能签收、能入库、能版本化的？
4. “基本”这两个字凭什么？什么算基本，什么不算？
5. 元素与关系在架构里住在哪？定义句里为什么没有它们？
6. “改起来贵的东西就算架构”，这句话错在哪？
7. 架构为什么值得要？不补这一层，会缺什么？
8. 它凭什么管得住后面的设计，又管到哪儿为止？
9. 业务架构、数据架构、应用架构、技术架构，是几个架构？它们本身是什么？用了某个框架，算不算有了架构？

### 10.1.3 本章的骨架

先把骨架立起来，后面三块都是给它添血肉。

一句定义：架构是一个实体在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。

这句话里并列着两样性质不同的东西：一头回答“它是什么”，一头回答“它必须怎么造、怎么变”。前半是描述性的，后半是规范性的——规范性是说它可以被违反。不理解这个并列，就理解不了这个定义。

第二句：架构是那份内容，就是定义句里那两样：基本概念或属性，加上管控原则。它不是承载它的那份东西：这两样内容可以精确、可以落成清单；能签收、能入库、能版本化的，是架构描述。日常说“架构在这台服务器上”，说的其实是描述。

第三句：“基本”不是形容词，也不是“改起来贵”。它的判据只有目的——把这一条内容拿掉，这个实体还能达成它的目的吗。

第四句：这一层非有不可。两头都推不过去，只能先定一个并有人负责；它管得住下游，靠的是收窄设计空间，不是指定答案。

骨架立完，本章分三块走：架构的定义（两样内容与它的两侧）、架构的价值（它站在哪、值在哪、管到哪儿、凭什么管得住）、架构的误解（两族误读逐个清掉：一族把描述当架构，一族把设计当架构）。

---

## 10.2 架构的定义

### 10.2.1 架构这个词的来路

架构不是软件和系统领域的原住民。它最初是建筑学的词，指承重结构、空间布局。软件工程在 1970 年代借用了它：结构化设计把模块划分、层次关系叫“架构”。

本章题面里的“架构概念”，说的是**“架构”这个词所指的那件事本身**——这一节交代它的来路，后面两块交代它的价值与误解。要留心，标准里另有一个同形说法，指**架构里的那些概念与属性**（42020:2019 §8.4.8 Capture architecture concepts and properties，中译：捕获架构概念与属性）；那是第 11 章的产出、第 13 章要表达的东西，跟题面这句不是一个意思。附录 A 另收“候选架构概念”一词（T-129 构思），指发散阶段的一个个可能解。三样别混。

1990 年代系统越做越大，“软件架构”成为一门显学，词义也一路漂移。把四站摆出来，能看清漂的是哪一头：

| 时点 | 表述 | 主体 | 界定方式 |
|---|---|---|---|
| 1996（Hilliard） | 系统在其环境中的**最高层级的构想**（the highest level conception） | 系统 | 偏主观的“构想” |
| 2000（IEEE 1471） | 系统的**根本组织**，体现于其**组件**之中 | 系统 | 把主观构思压成客观结构 |
| 2011（42010 第 1 版） | 系统在其环境中的基本概念或属性，**体现于其元素、关系以及设计和演化的原则之中** | 系统 | 把架构从具体结构上提到概念层 |
| 2022（42010 第 2 版） | **实体**在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的**管控原则** | **实体** | 删去“体现于元素与关系” |

（前三站的表述经库内 22 号，06 号等报告转引）

2022 版这一站有两个动作值得记住，它们直接决定本章后面几节怎么讲。

一是主体扩域。从“系统”换成“实体”（entity of interest，所关注实体）：企业、组织、产品、过程、数据、应用都可以被架构。系统只是实体最常见的一种形态——本书讲的主要是系统。

二是删去了界定方式。2011 版用“体现于其元素、关系以及设计和演化的原则之中”给架构划界；2022 版把这一句从定义句里删去。删得干净：现行定义句里不再有它。那一层含义只保留在 42020:2019 §3.3 的注 1 里，措辞由“元素、关系”改成“组件、组件之间的关系、实体与环境的关系”，强度由定义句里的“是”降为注释里的“通常意指”（usually intended to be embodied in）。注释级，没有术语地位。

对读者来说，这一删一降带来一个必须当面交代的后果：元素与关系不再是架构的定义要素。它们有它们的位置，但那不是定义句——§10.2.5 专讲这件事。

> 术语注（本书读法）：本书把这条定义读作“**系统不可还原的涵义”——关于系统根本性质的那组基本概念或属性（不是“一组决定”**：决定只是认出并记下它的抓手，正位在 §5.2.12 ／§6.10）。这个读法是本书的凝练，不是任何标准的原文（42010:2022 正本全文没有“不可还原”的对应表述）。它落的是定义句里“基本概念或属性”那一半，管控原则是与它并列的第二样内容，不在“涵义”之内。欢迎检验、欢迎校准。

## 10.2.2 架构是基本概念或属性，加管控原则

把定义句逐词读一遍。原文是这样的（ISO/IEC/IEEE 42010:2022 §3.2）：

> **Architecture: fundamental concepts or properties of an entity in its environment and governing principles for the realization and evolution of this entity and its related life cycle processes.**〔正本条目号为 §3.2；原文将 environment 标为参见条 environment (3.13)〕
>
> 中文（依中英对照版 §3.2 条，语序从中文习惯）：架构——一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则。

这句话读起来像一串名词堆在一起，其实每个词都压着分量。

“基本概念或属性”（fundamental concepts or properties）是前半句。它回答“这个实体是什么”。架构不关心全部细节，只关心那些“如果改变，就会让这个实体变成另一种实体”的东西，判据在 §10.2.4。

原文用的是“或”（or），不是“和”（and）。中文常译成“概念和属性”，读起来像是必须同时具备的两半。“或”比“和”宽松：可以从“核心概念是什么”切入，比如订单是一种交易意图的记录；也可以从“核心属性是什么”切入，比如数据全程可追溯；两条路都能走到架构。

这个析取式不是随手的并列，它是 42010 工作组有意识的选择。用意是不带偏见地兼容两种哲学（42010:2022 附录 A.2 逐字）：Architecture as concept，架构是存在于人脑中的系统构想；Architecture as property，架构是实体的属性或特性。标准对两派不表态（without prejudice）。

“属于一个实体、在它的环境中”（of an entity in its environment）交代了架构住在哪。现行版的主体是**实体**，不限于系

统；而“在它的环境中”这半句常被读丢。同一个温控设备，放在实验室台架上，架构可以是“温度控制单元”；放在冷链运输里，必须包括“断链检测与补传”。环境变了，什么算基本就变了。

“及管控原则”（and governing principles）是后半句，也是这次改版升格上来的一块。2011 版把原则放在“设计和演化的原则之中”，是界定方式的一部分；2022 版把它提出来，与“概念或属性”并列，还给它加了一个限定词 governing。这个词的分量在下一段。

> 术语注（管控原则，三处同源不同位，混用会同时污染术语表与治理讨论）：
>
> - **定语位** governing principles＝**管控原则**，就是定义句里这一块：有约束力、可据以判定违规；
> - **关系位** a viewpoint **governs** a view＝**治理／管辖**，说的是架构视角与架构视图之间的关系（该关系归第 13 章）；
> - **名词位** governance＝**治理**，是组织治理（第 16 章）；它的架构侧义项见附录 A T-118「架构治理」。
>
> 三处只共用“治理”二字，不是一回事。

“实现与演化”（realization and evolution）带的是两个宾语：该实体，以及它的相关生存周期过程。定义句写的是“该实体及其相关生存周期过程”的实现和演化，架构原则同时约束两件事：东西怎么做成，以及做它的那些过程怎么走。这一处常被读丢。它是“架构管到退役与替换”的出处，第 15、17 章接的就是这条线。

两半的性质不同，这是本节最要紧的一句话。前半（概念或属性）是描述性的，说的是这个实体是什么；后半（管控原则）是规范性的，说的是它必须怎么造、怎么变。规范性有一个可检验的标志：它可以

被违反。第 14 章会拿“该内容能否被设计违反”当筛选判据；本章只记一句：把两半合成一个名词去叫，就会丢掉“能被违反”这一半的锋芒。

> 术语注（**本书判断**：正本内部一处口径不一，引用时须一并交代）：定义句（§3.2）是**两项并列**；而同一标准 §5.2.2 写的是 The architecture of an entity of interest **comprises the fundamental concepts or properties** of that entity considered in its environment——“comprises”只提前半，“管控原则”被排在紧随其后的 can pertain to any or all of the entity's: … principles governing its design, use, operation and evolution 清单里；附录 A.2 谈两种哲学时同样只说“concepts or properties”。**本书一律照定义句 §3.2 写（两项并列）**，此处交代差异，不作调和。

这两半落到纸上，自然就是两份东西。前半是基本概念或属性，写成一份清单；后半是管控原则，写成一份管控原则清单。后一份清单天然分两栏：管“怎么造对”的实现侧，与管“怎么变不散架”的演化侧。这与前面那句“实现与演化带两个宾语”是同一件事的两种说法：在定义句里它是一句话的两个宾语，落到纸上就是两栏。

三处要说清。其一，清单是“表达”，不是“架构”。按 §3.1，架构工作是一串动作（构想、定义、表达、编制文档、沟通、证实其正确实施、维护和改进）；清单是“定义”这一步的产物，不等于定义句所讲的那个架构本身（§10.2.3 划这条界）。其二，“或”不是“和”：原文 concepts or properties 兼容两种哲学，概念与属性不必各列一张，合成一份更贴正本。

其三，清单可以按“方面体”分块：§3.9 的 aspect 是“实体性格或性质的一部分”，标准示例是功能／结构／信息，按它分块是 2022 版明确承认的清单化结构。方面体是标准给的分块维度，与 §10.4.4 那组“关注域”不是一回事：前者按实体的性格或性质切，后者按“管哪一摊”（管理对象）切。

清单的用处是对账，不是罗列。列完要能回答三件事：哪些关注点没有条目承接，哪些条目只服务单一关注点且不承重，哪些内容其实可以被设计自由处置。答不出这三问，清单越长，越说明“什么算基本”这条判据已经丢了。（这几份清单怎么长出来，是第 11 章的事。）

### 10.2.3 架构与架构描述：内容与载体

先分清哪一份是内容、哪一份是载体。标准用一组对照把这件事说得很清楚（42010:2022 Introduction 逐字）：

> Whereas an AD is a tangible work product, an architecture is intangible and abstract, understood through its concepts, properties and principles.
>
> 架构描述是有形的工作产品；架构则是无形、抽象的，要通过它的概念、属性和原则来理解。

先说清这里的“抽象”是哪个义。“抽象”在中文里至少有三个所指（三分是本书的整理）：

| 用法 | 说的是什么 | 例子 |
| --- | --- | --- |
| **含义层**：概念／共相 | 词的意思是一般的、不指向某个时空个体 | “服务器”“三角形”“需求”这些概念都抽象 |

| 用法 | 说的是什么 | 例子 |
| --- | --- | --- |
| **本体层**：抽象对象与具体对象 | 指称物有没有时空位置、有没有因果力 | 数、命题、类型是抽象对象；这张纸、这台机器、这个团队是具体对象 |
| **粒度层**：省略细节 | 工程口语里的“抽象层次” | 概览图比详图抽象，任何层次都能再抽象一层 |

三层混用会得出一个空洞的结论：“一切术语的指代都是抽象的”。那不是发现，那是把“含义总是抽象的”这个平凡事实当成了本体论结论。更要紧的是，粒度层的“抽象”（概略、高层级）恰是本章要清掉的误读之一（§10.4.6）。所以本书不用“抽象物”这个词：那句话在标准里的职能是给 AD 作对照，说的是**不是人工制品、没有时空位置**那个义，不是含糊，也不是高层级。（该句在引言，不在 §3.2 定义句内。）

标准自己把这件事的动词也分开了。architecting 一整套动作是 conceiving, defining, expressing, documenting, communicating…（§3.1）；**定义**与**表达、记录**分属两个动作。AD 的定义则是 work product used to express an architecture（§3.3）：它是那份表达。（另见 §5.2.2 L461：An AD is an expression of an architecture. 该句在 §5.2.2，不在 §3.3；§3.3 全文只有“AD＝用来表达架构的工作产品”这一句）

于是位置就定了：清单、模型、视图都落在“表达与记录”这一侧，合起来是架构描述。由此得到本书的一条判据：清单可以是清单，甚至可以很具体，但清单是表达，不是架构。具体不是缺点。内容要具体到能判“属不属于这一类”“违没违反”，它才排得掉做法。能签收、能入库、能版本化的，是那份描述。

内容与载体总有两侧，架构这一对是最常被混的。这里说的“内容”不是虚指，就是定义句那两样：

| 概念 | 内容 | 载体 |
| --- | --- | --- |
| **架构** | **基本概念或属性，加上管控原则**（定义句那两样） | 描述它的**架构描述** |
| 关注点 | 利害、立场 | 记录在案的关注点条目 |
| 架构视角 | 那套惯例 | 视角规格（可入库、可版本化） |
| 架构视图 | 所表达的内容 | 架构描述中的信息部件 |

后三对（关注点、视角、视图）归第 13 章。本章只用第一对。分不清内容和载体，就会把“改了一份文档”当成“改了架构”。

这一对有三条当场可检验的判据（三条判据见 R-04 §3.3 与 §3.2）：

- 可交割性：系统能交付、验收、上线、停机；架构描述能签收、入库、版本化；架构不能。给客户签的那份东西，是架构描述。
- 同一性：同一个架构可以有几份不同的架构描述，也可以有两个实现。多个载体，一个内容。描述这一侧恰好相反：每一份都是个别的工作产品、个别可版本化的信息部件（§3.3、§3.14）。
- 因果力：架构不导致任何事。它只能靠被理解、被援引、被遵守起作用；系统能算、能发指令、能故障。

日常说“我们的架构在这台服务器上”“架构在文档库第三版”，说的其实都是描述。这句话不错，但它换了一个指称——把内容说成了载体。

内容与载体这一对，本章只走到“能签收的是描述”为止。可“签收得下来”还只是第一步——那份描述凭什么真管得住下游的设计、评

审与变更？答案在下一层：视图把内容写成了被定义的语言，让不同的人读出同一个意思（**同解**）。那是第 13 章的事，本章点到载体为止。

### 10.2.4 “基本”是对目的必要且充分

定义里那个“基本”（fundamental）是整句最容易被一带而过、分量却最重的词。它来自拉丁语 fundamentum（地基）：地基在最底层，承托一切，一动则整栋楼动。

地基仍是比喻，判据得自己立。而“什么算基本”这条判据，标准没有给。

两部标准都没有回答它：42010:2022 只说架构可以关乎什么（§5.2.2），42030 只说评估对的是规定的评估目标（§1／§3.4）。SEBoK 直接写明 fundamental 这个词“没有普适定义”，详细程度“取决于使用的语境和设计目的”。它还要求架构描述“说明何为纳入、何为排除”（均系报告转引）。这是一处权威缺口，本书给出的判据是形式化，不是标准条文。

判据的形式是两条，参照系都是**目的**：

| 判据 | 问法 | 管什么 | 失效形态 |
| --- | --- | --- | --- |
| **必要性**（membership，入会） | 去掉 x，目的是否落空？ | 成员资格：什么必须进 | **架构缺失**（under-specification） |
| **充分性**（closure，闭包） | x 齐备之后，剩余问题是否都不影响目的的可达？ | 闭包：架构到哪儿停 | **架构越界写成设计**（over-specification） |

一处精确化不可省：这一对判据不是数学意义的充要条件。架构不构成目的达成的充分条件，达成还要靠实现、验证、运行；充分性只指一件事：目的所要求的结构条件已被完整表达。

必要性一侧可以落成三句更顺口的问法，三句各问一面，问的还是同一件事：

- 不可再分：去掉它，就不再是它了？
- 不可缺失：缺少它，这个实体就垮掉？
- 不可替代：换掉它，性质就变了？

三句共有一个锚：那条内容是否还服务于这个实体在其环境中的目的。目的变了，“基本”就变了。同一个温控算法，在心脏起搏器里是基本，在计算器应用里只是实现细节。

目的判据要跑两遍。一遍是兑现侧，问实现这个目的需要它吗；一遍是演化侧，问目的变了它守不守得住。定义句里“and evolution”就是第二遍的出处，只跑一遍必漏演进类的基本条目，比如“新增传感器类型不得破坏已有追溯链路”这类，只在演化侧才现形。

判据之上还有一层顺序，不能倒。拿目的筛完之后，可违反性、可引用性、可判定性这几条才依次上场。它们的位次在必要性之后，作用是排序与检查，不是准入：

> ① **目的必要性**（总闸，参照目的做承重测试）→ ② **可违反性**（这条内容能不能被设计违反）→ ③ **可引用性** → ④ **可判定性** → **关注点闸门**（这次写哪些、写多细、给谁看）

这条判据从哪来？从治理推。治理的核心动作是裁量：某条内容合规还是不合规、准还是不准。既然是裁量，就得有一把不随人变的尺。这把尺不能是关注点：关注点互相冲突，而治理存在的理由恰恰是仲裁冲突的关注点。于是尺只能落在**目的**上。一句话说清：“基本”由目的界定，不由关注点界定。

由此得到两道闸门与一类来源。目的与关注点是闸门，环境是来源，各管一段：

| 闸门／来源 | 管什么 | 判据 |
| --- | --- | --- |
| **目的（承重性）** | 条目够不够格进“基本”这一层 | 拿掉它，这个实体还能达成它的目的吗 |
| **关注点（取舍与呈现）** | 够格的候选里，这次写哪些、写多细、给谁看 | 哪些关注点没有条目承接（缺口）；哪些条目只服务单一关注点且不承重（降级） |
| **环境（语境与影响）** | 一类来源：实体周围的**语境与影响** | 定义中 in its environment——法规、组织与业务环境、物理条件、外部接口、资源 |

这也解释了为什么“监管合规”能进架构，而不是可选项：不是因为它有人在乎（那是关注点层的理由），而是因为它构成实体在环境中达成目的的条件。

一处方向性提醒，本章只记结论：目的的定义权在治理机构，不在架构师（42020:2019 §6.4.3／§6.4.4／§8.4.4 是它的正本位置）。架构师对目的有质询权与守护责，可以问“这条目的还算数吗”，不能自己改写。目的被悄悄换掉，架构就悄悄失效。这是架构失效的第一病因，第 16 章展开。

这条判据还有一个常被忽略的后果：约束即解放。判据筛完，够格进架构的只有“去掉它目的就落空”的那几条；剩下的一大片（某个接口用 REST 还是 gRPC、某个界面怎么排）被判据主动放还给做的人。所以好的架构不是“管得最多”，而是“管得最准”：约束只落在要命的地方，别处留给执行者自己拿主意。

### 10.2.5 元素与关系不在定义里

很多讲架构的材料，第一句就给一张图：几个方块，几根连线。方块是元素，连线是关系，看起来顺理成章。可翻开现行定义句，元素与关系根本不在里面。这一节把它们的去向交代清楚。

先说清事实。2011 版定义句里有“体现于其元素、关系以及设计和演化的原则之中”；2022 版删去了这一句。同时删去的还有 42020 的注：42010:2022 §3.2 的 SOURCE 行逐字写着 ISO/IEC/IEEE 42020:2019, 3.3, modified — The notes to entry have been removed.（42020:2019 §3.3 的注已被移除）。

再说它们的去向。那一层含义没消失，它搬到了 42020:2019 §3.3 的注 1，并且降了调。注 1 逐字：

> The fundamental concepts or properties of the architecture entity are **usually intended to be embodied in** the entity's **components**, the relationships between components, and the relationships between the entity and its environment.

三点变化：措辞由 elements／relationships 改成 **components**；强度由定义句里的“是”降为“通常意指”；地位由定义句降为**注释**（注释级，没有术语地位）。

在 42010 这一侧，元素与关系另有一个位置。§5.2.2 逐字：The architecture of an entity of interest **can pertain to** any or all of the entity's：组成元素；元素之间的交互或相互关系；与环境（含环境中的其他实体）的交互或相互关系；行为与结构；管住其设计、使用、运行与演化的原则。语气是“可以关乎”，不是“体现于”。附录 A.2 说得更直白：In some cases, the fundamental elements are physical

or structural components of the entity and their relationships.（在某些情况下，根本要素就是实体的物理或结构组件及其关系。）

三句话合起来就是这一节的结论：元素与关系不在定义句内，最多是“通常意指体现于”（42020:2019 §3.3 注 1，注释级）。它们是架构可以关乎的对象（§5.2.2），也是架构描述要交代的内容，但不是定义要素。

把它们当架构本身，是误读的第一步，日常的说法很顺口：“架构就是系统的结构”。这句话一出口，错的不止一处：元素与关系从“可以关乎的对象”被升格成定义要素，而定义句里真正承重的那两样（基本概念或属性、管控原则）反倒没人问了。

元素这个词还要再分三层，否则一定混。同一份材料里出现“元素”，可能指三样不同的东西：

| 术语 | 出处 | 在哪一层 | 是什么 |
| --- | --- | --- | --- |
| **架构描述元素**（AD element） | 42010:2022 §3.4 | 描述层 | 架构描述的零件：利益相关方、关注点、方面体、架构视角、架构视图、视图组件、模型种类、对应关系…… |
| **架构落点**（**本书工作名**；旧名“架构构成物”已废止） | components／relationships 见 42020:2019 §3.3 注 1（附录 AT-168） | **实体侧**（不在定义 | 实体内部的构造（**注释级，没有术语地位**）——是架构“**可以关乎**”的对象，不是架构的构成物 |

| 术语 | 出处 | 在哪一层 | 是什么 |
| --- | --- | --- | --- |
| | | 句内） | |
| **系统元素**（system element） | 15288:2023 §3.47 | 实现层 | 可被实现的离散部件 |

还有第四个词要分清：架构实体（architecture entity，42020:2019 §3.6；附录 A 作“被架对象”，同物异名）指“被架构的那个事物”，≈ 42010 的所关注实体，不是“架构的构成物”。中文圈常把它误读成构成物，此处钉住。15288:2023 §5.9 另有一句可作旁证：架构实体与系统元素“代表两种不同的概念”（依中英文对照版；英文原文作 notions）。

为什么这一节的最后要强调“关系”？因为系统的涌现属性来自关系，不来自单个元素。第 2 章讲过：每个发动机都正常、每块机翼都达标，飞机未必能飞。放到架构这一层，它有一个很具体的推论：可靠性的瓶颈通常不在单个组件，而在组件之间的交互模式。单个传感器精度再高，也拦不住“传感器→固件→云端”这条链上某个环节的重传超时。

### 10.2.6 假判据冒充真判据

真判据只有一条：目的。“什么算基本”这个问题太要紧，市面上就长出几个代理来冒充它。

| 假判据 | 为什么会冒充判据 | 失真在哪 |
| --- | --- | --- |
| **不可逆性**（改起来贵） | 可观测、直观、易沟通 | **漏判**：不贵但承重（“日志须可被外部审计员独立读取”）；**误纳**：很贵但与目的无关（框架版本、组织惯例技术栈） |
| **可判定性／可引用性／高层级／概要** | 都是“看起来像判据”的代理 | 同一形状的错误换了外衣——**代理冒充真判据** |

“改了它还是不是这个实体”这句问法看着像不可逆性，其实是承重测试的另一半。可逆的改动也可能承重：不贵，但去掉目的就不成立。不可逆的改动也可能无关：很贵，但与目的无关。两类失真都真实发生。

代理不是没用，是不能当总闸。它们的正当用途有四样：**排序**（先定哪几件）、**定决策时机**（什么时候必须拍板）、**配治理强度**（多严的评审与留痕）、做回代检查（事后核这条决定是否真承重）。

这里有一处必须替标准说清的边界。42010:2022 §6.10.1 确实给了一张“哪些决策值得记录”的选择准则清单，里面有一条正是 that are likely to be costly to change（改起来可能很贵的）。那是决策记录的取舍准则，不是架构的准入判据。两者都用到“贵”，但一个管“记不记”，一个管“算不算架构”。混起来，就会把很贵但无关的东西记进架构，同时把不贵但承重的东西漏在架构之外。

第 12 章会专门讲决策的时机与记录；本章只用它的反面，把“贵即架构”这条误读钉住。

## 10.3 架构的价值

### 10.3.1 架构站在“要什么”与“怎么做”之间

需求说的是“要什么”：可测，而且标准明令它不许指向实现。设计说的是“怎么做”。这两头之间必须有一层，把“要什么”变成“什么不许违反”。

落差就在这里：需求侧被明令不许说“怎么做”，说了就不叫需求，叫设计；设计侧必须挑一个做法落地。中间那一段，谁都不管就会被填上。

拿 M3 说：客户要的是“全程温度可追溯”，这是“要什么”。产线要知道的是“这批货能不能出厂”，这是“怎么做”里的一环。中间那一步，比如“新增传感器类型不得破坏已有追溯链路”。石军在案例进展①里开始找的，正是这类东西。

标准给这一层的正式名字是**架构即约束**：The constraint is on the design space available to the design definition process.（42020:2019 附录 E.2.1 逐字）。约束的对象是**设计定义过程可用的设计空间**，不是实现者，也不是产品。

为什么非得由它来承？因为两头都推不过去。

上向推不过去。系统在标准里的定义本身就写着涌现：arrangement of parts or elements that together exhibit a stated behaviour or meaning that the individual constituents do not（15288:2023 §3.46）。整体性质不在部件里，把部件查得再细也加不出来。

下向推不过去。分解从来不唯一，它是选择，不是推导。42020:2019 §8.4.7 给的是任务（Decompose, when necessary, and allocate…；原文含“when necessary”，引时不可省）；而在

它之前，§8.4.5 那一步的名字是 Synthesize potential solution(s) in the solution space：综合出潜在解，不是演绎出唯一解。

两头都推不过去，剩下的路只有一条：**先定一个，再做出来验**。可“先定的那一个”如果不是某个人明确定下的，它就会被默认——被最早那份方案默认，被某个人的习惯默认，被一次没人记录的会议默认。

架构被默认了、却查无此人，这是最贵的一种状态。42010:2022 §6.10 要求把决策与理据记下来，防的正是这件事（§6.10.1 管决策记录，§6.10.2 管理据记录；记什么、怎么记，归第 12 章）。这里还有一处反向误读要防住：**记下来不等于定死了**——决策可以改，改要留痕；标准要求的三个时间戳里就有“修改”那一枚。

还有一条同向的理由：选定之前，“合规”没有对象。15288:2023 §3.13 给 design 的名词定义是 specification of system elements and their relationships, that is sufficiently complete to support a compliant implementation of the architecture：合规的基准是架构。没有那份“它是什么”，下游连“合不合规”都无从问起。

所以这一层不是锦上添花。它是“要什么”落到“怎么做”的必经一段。

### 10.3.2 不补架构，连“好不好”都无从判

先看反面。15288:2023 §6.4.4.2 列了架构定义过程的成果八条（a)~h)），反着读，就是“不补这一层会缺什么”：

| 落空 | 缺的是什么 |
| --- | --- |
| **判据落空** | 合规没有对象；质量属性没有判定通道；评审对不上目标 |

| 落空 | 缺的是什么 |
| --- | --- |
| **责任落空** | 重大概念、属性与约束没有分配到架构实体上；利益相关方的关注点没被回应；接缝没有出处 |
| **留痕落空** | 视图与模型不存在；可追溯性缺失；决策不被记录〔出处分别对应 15288:2023 §6.4.4.2 a)~h) 与 42010:2022 §6.10 ——“决策不被记录”一项来自 §6.10，不在 §6.4.4.2 八条之内〕 |

三组指向同一件事：不是做得不好，是没有可以判“好不好”的东西。

正面看，标准给这件事起了四个名字，四个名字就是四样价值：

| 名字 | 出处 | 换来的东西 |
| --- | --- | --- |
| architecture as constraint | 42020:2019 E.2.1 | 下游有一片被判明的设计空间 |
| compliant implementation of the architecture | 15288:2023 §3.13 | **合规有基准**——评审与验证有靶子 |
| convey the architectural intent | 42020:2019 §5.6 | **意图可传达**：接口只定到“足以传达架构意图”即可 |
| as design-agnostic as possible | 15288:2023 §5.9 | **不指定答案，反而留出设计自由** |

再加两条由本章判据直接推得的：

- **约束即解放**：判据筛完，够格进架构的只有“拿掉它目的就落空”的那几条；剩下的（某个接口用 REST 还是 gRPC、某个界面怎么排）被主动放还给做的人。
- **复杂度可以拿住**：一个真实的企业系统可能有几百个服务、几十个接口、几十个数据对象，远超任何单个人的脑子；架构把它们压成一组装得进脑子的基本概念或属性与管控原则。于是处理任何一

个具体问题时，你只需要关心系统的一小部分（42010:2022 讲架构工作的动机，用的正是“帮利益相关方管理复杂度”这句）。

一条边界要说清：**架构的必要性是逻辑的，不是文书的**。不成文的架构仍然存在：第一版方案已经把这件事办了；它只是没有被评审、没有被记录、没有人对它负责。规模与可丢弃性决定它的形式（写三页纸，还是开一天会），不决定它有没有。

有它和没它，差别落在三个日常场合。

评审：有它，评审有一个可对照的靶子。合规的基准是架构，不是需求。没它，只剩逐条对功能清单，对完也不知道系统该不该长这样。

接缝：有它，接口的分歧有出处可查（“这是哪条原则管的事”）；没它，两个团队各让一步，让出来的东西没人认账。

交接：有它，新来的人拿到的是“这个实体作为哪一类东西存在”；没它，他拿到的是几十万行代码和几位老人的记性。

这三处正是三组落空在日常里的样子。

### 10.3.3 架构管到哪儿为止

它的边界由目的裁，不由别的裁。这条判据在前一块立过（拿掉它，这个实体还能达成它的目的吗）；放到这一层看，它同时回答了“管到哪儿停”：概念对应可达的机制，属性对应可判的度量面，管控原则对应可持续的守持。

需求为什么替不了它？需求也可判——15288:2023 §6.4.3.1 逐字写它 creates a set of measurable system requirements。但同一节紧接一句：As far as constraints permit, the requirements should not imply any specific implementation.

**需求的判定宾语不是做法**，就一个做法也排不掉。一个东西再可判，只要判的是“达标不达标”，它管的就还是结果，不是做法；**正因为需求不许说“怎么做”**，架构那层才有位置。

收窄常被说成另外三件事，三件都不是它：

| 不是 | 那是 |
| --- | --- |
| 写得更详细 | 细化（42020:2019 **§ 8.4.8 注 1** 说明：概念化只需描述到 suitable for its intended users 的特定程度与粒度；该语是注，不是规范句） |
| 规定更多“必须做” | 把设计搬进架构 |
| 写得更“高层”更抽象 | 与“基本”正交的粒度问题 |

所以“管得准”不是“管得多”：只划那几条“不划就会丢掉目的”的界，其余留给做的人。

这句话有两面，都要认。**管得太多**，就是上文表格第二行说的“把设计搬进架构”，它会带来最坏的一种后果：架构与设计两本账，谁都不知道以哪本为准。管得太少，就是目的那几条界没划出来，落空的是三组东西（判据、责任、留痕）。

判“到哪儿停”用的是同一条判据，只是反过来问：留着它，目的还需要它吗？不需要，就该放还给设计。所以这一节和上一块那条承重测试是同一把尺子，量的是两个方向。

### 10.3.4 架构凭什么管得住下游

说它有约束力，不等于说清了它凭什么有。先撤两条坏论证。

一条是“架构是一个定义，定义就是划界，所以它自动收窄空间”：多数定义与设计空间根本无关。“钉钉子的工具”、数字 7、三角形，都不落在“做法”这个集合上。另一条是“标准指派架构去约束设计空

间，所以它能约束”：指派是授权，说的是准不准它管；不是能力，答不了它凭什么管得住。

标准把这件事分别指了出来：

| 问题 | 正本事实 |
| --- | --- |
| 约束谁 | **设计定义过程可用的设计空间**（42020:2019 附录 E.2.1） |
| 怎么起作用 | **引导设计活动，同时留出最大灵活性**（42020:2019 §5.6） |
| 分界在哪 | **尽可能与设计无关**（15288:2023 §5.9） |

它起作用的机制不是指挥，是排除。下游对架构的判据是相容性：15288:2023 §6.4.5.1 逐字要求设计实现 in accordance with the system requirements and architecture，并 consistent with architectural entities defined in models and views（与**架构模型与视图里定义的架构实体**保持一致）。**靠相容性排除**，才能既不说技术，又照样把不相容的做法排掉。这也是“尽可能与设计无关”能成立的唯一理由。

指挥与排除的差别，落在两句话上。“接口一律用 MQTT”是指挥：它替下游挑了一个做法，换掉它就等于改架构。“新增传感器类型不得破坏已有追溯链路”是排除：它没说用什么协议、什么型号，但任何会破坏追溯链路的做法，从它落纸那一刻起就不成立。前者越界，后者才是架构的活。

排除的力气还有一处来源：**违背它的后果是“这不再是这个实体”**，不是“某人不高兴”。这就是为什么架构能靠一份不多的内容，管住后面所有的选择。

这份能力来自架构内容自身的四条性质：

| 性质 | 说的是什么 |
| --- | --- |
| 一 · 异类自排 | 它说的是“这个实体作为哪一类东西存在”，**异类被它自己排掉** |
| 二 · 判定宾语都是做法 | 两样内容的判定宾语都是做法：分类判定（属不属于这一类）／度量判定（能不能兑现这些特性）／合规判定（违不违规） |
| 三 · 先于下游 | 它先于下游的设计选择：架构定义过程排在设计定义之前 |
| 四 · 可陈述可追溯 | 它可陈述、可追溯：约束的不是一时想法，是这个实体往后全部参与者的选择 |

还有一条要单说：架构的效力不是宣布出来的，是靠被确立、被依赖长出来的。内容凭目的裁，候选凭检验筛，效力凭治理背书，忠实凭追溯证（后两件各归第 16 章与第 13、14 章，本章只点位置）。

一句话收住这一整块。架构工作，是把“这个实体是什么”从几种可能，立成一句可判的定案，写成能签收的形式，接通下游的引用与追溯。收窄是这个动作的后果。判一份架构做没做出来，看两件事：那句“是什么”说清了没有；它排掉了什么。

## 案例进展①｜石军要的是一条能判的界

章首那场会之后，石军被“三个人三个答案”刺痛了。他把林涛、季昭、小张拉进会议室，想弄清一件事：我们到底守着一个什么样的系统。

林涛先开口：“M3 就是个温度记录仪加个箱子，能调温、能记录、能上传。你说的'系统'，不就是这些功能的集合吗?”

石军摇头：“功能能加，架构不能。按承重测试问一遍就分开了——去掉它，M3 还能达成它的目的吗。加个摄像头记录装卸，M3 还

是 M3，目的是同一个；可要是把'全程可追溯'拿掉，它就从医疗冷链设备退成一只会调温的箱子。所以我不数功能，我数'少了什么它就不是它'。数目对不上不要紧，方向对得上就行。”

测试季昭盯着纸上那行字：“我测了两年，最难受的不是测，是说不清凭什么说'测过了'。判据得能追问到出处——你刚说的'少了什么不是它'，就是我要的那个靶子：M3 的架构在哪，我该验的东西就在哪，不用再对着功能清单一条条猜。”

石军把笔放下：“还有一层得说清：我们守的不是那张图。图是别人画的，明天换个人也能画；我们守的是它背后那几句话——'断链即报废'还算不算底线、温度数据归谁管、新增传感器类型能不能破坏追溯链。图丢了可以再画，那几句话要是说不清，M3 就不是 M3 了。”

石军在纸上写下这一句，随即划掉。他要的不是一句口号，是一条能判的界：“断链即报废的疫苗，能从出库一路撑到接入冷库，全程温度可追溯。去掉它，M3 就是一只能调温的箱子。”

系统工程师小张把纸上那行字读了两遍：“这句话我认，可它还判不了。'全程可追溯'要能判，依据只有三样：从哪一段起算、每一段由谁签字、断在哪一节点算断。三样不写，明天换个人读这句话，读出来就是另一个界。”

四个人都不说话了。他们第一次意识到，自己守着一个系统快两年，却从没讨论过它“不可还原”的部分是什么。

## 10.4 架构的误解

前面两块把架构和它的描述都立起来了。这一块从反面走一遍：把贴着“架构”这个词长出来的误读逐个清掉。它们分两族：一族把描述当架构，一族把设计当架构。清完，前面那些定义才算真正站住。

### 10.4.1 不是一张图

石军的病不是“没画好图”，是把图当成了架构。这两种病看着像，治法完全不同：前者去改图，后者去改对“架构”的理解。

“架构”在日常里被用得非常泛滥。这些用法有一个共同的毛病：把架构当成了一样能画出来、能保存、能过期的东西。

标准知道这个毛病。它在附录里记了一件有意思的事（42010:2022 附录 A.2 逐字）：

> Empirical studies have discovered four metaphors for architecture found in organizations: architecture as blueprint; architecture as literature; architecture as language; architecture as decision.

中译：组织里实际存在四种关于架构的隐喻，架构即蓝图、即文献、即语言、即决定。这是实证研究的发现，而标准紧接着说：本文档的概念基础**不预设其中任何一种**，与四种都兼容。

这一句值得停一下。它说明两件事：第一，“架构是一张图”不是谁在胡闹，它是有实据的组织习惯；第二，**假的是把它当成全部**。蓝图只是四种隐喻之一，而且是最容易过时的那一种。图会过时，被图投影的那组基本概念或属性可以一直有效。一张图可以过时，架构可以一直有效——石军团队就是被这个区分困住的典型。

### 10.4.2 不是一组决定

另一种常见的算式是“架构就是那组改起来贵的决定”。这个说法听起来比“一张图”高级，方向也对了一半：它至少指向了“内容”而不是“载体”。它仍然不成立，原因与上一节同源：标准把“架构即决

定”同样列为四种隐喻之一，可见它是组织里真实存在的说法，但标准不预设它。

“决定”在标准里另有正位，三处都要一起看：

- **§ 5.2.12 给了它定义**：An architecture decision is a collection of choices made in the overall context of an architecture.（架构决策是在某个架构的总体语境中作出的一组选择。）它在标准里是一个独立的架构概念；
- **§ 6.10 是 shall 级的要求**：架构描述**应当记录**被认为对该实体架构至关重要的架构决策及其理据。注意这里的“至关重要”由组织与项目自定准则（§ 6.10.1 给了一张取舍清单）；
- § 5.2.9 把它列为架构描述元素所承载的概念之一（架构决策与理据在内）；而 § 3.4 注 1 的清单里没有它——这是正本自身的两处不一致，引用时把两处都交代出来，不要只引一处。

三处合起来说清一件事：“决定”是认出架构、并把架构记下来的抓手，不是架构本身。把抓手当本体，代价很具体：决定清单会越写越长，凡是贵的都往里加，而“这个系统是什么”始终没有正面回答。M3 那次评审会上，石军的图里没有任何决定；等他把老规矩写下来的时候，写下的也不是“决定的集合”，而是一句涵义加一组原则（第 11 章会把它写下来）。

### 10.4.3 不是更粗的设计

架构与设计最容易混，日常一句话就把两者并成一个：“架构就是概要设计”。两者确实都落在“系统里需要决定的事”上，分界不在粗细，在种类（据 R-37 § 5.9 的分工句：架构聚焦适宜性、存续性与合意性，设计聚焦兼容性与构建集成的可行性）：

> **架构与设计的差别是种类（kind），不是粒度（granularity）。**

这条结论有一个很硬的证据：合规关系的存在本身就是证明。若架构只是“更粗的设计”，那么更细的那一层不可能对更粗的那一层“不合规”：自己对自己无所谓合规。

那条分界可以拧成一句可判定的判据：这份内容能不能被设计违反？能被违反的（设计边界、接口约定、数据所有权、向后兼容的承诺），属架构这一侧。不能被违反的（某个接口用 REST 还是 gRPC、某张表的索引怎么建），属设计这一侧。

顺着这条界看设计那一头的定义，会更清楚（ISO/IEC/IEEE 15288:2023 §3.13）：设计是“系统元素及其关系的规格，足够完整到足以支撑对架构的一次合规实现”。注意承担合规的是实现，不是设计本身。设计只是把架构与需求转成可实施的规格，它给出的结论只能是条件性合规（第 15、16 章展开）。

两个过程不是两个阶段。15288 允许迭代与并发，“先把架构定义做完、再开始设计定义”是瀑布式误读；真实形态是同一时刻在相邻两个种类层上的两类工作（正本侧见 R-37 §6.4 并发的表述、R-05 中英对照版）。

### 10.4.4 不是一堆架构

市面上有业务架构、数据架构、应用架构、技术架构，还有需求架构、功能架构、逻辑架构、物理架构，甚至微服务架构、事件驱动架构，这些到底是不是多个架构？

一个所关注实体只有一个架构。那些“XX 架构”分属几套不同的维度，叠在一起才生出了混乱：

| 说法 | 实际是什么 | 判断维度 |
| --- | --- | --- |
| 业务／数据／应用／技术 | TOGAF 4A 的四个名字，对应企业运营体系的四个子系统（业务流 | 按“管哪一摊”切 |

| 说法 | 实际是什么 | 判断维度 |
| --- | --- | --- |
| | 程体系／企业应用系统／业务数据体系／信息基础设施）；四个子系统各有自己的架构与各自的多种视图，而 TOGAF 给它们配的内容落在设计侧、是各自的方案——借“架构”命名属术语误用（详见下） | |
| 概念／逻辑／物理 | 同一个架构从抽象到具体的**描述层次** | 按“多详细”切 |
| 微服务／事件驱动／分层 | **架构风格**——架构决策的套路 | 按“用哪套套路”切 |
| “用了 Spring Cloud” | 框架——把套路物化成可复用的制品 | 按“借了谁的骨架”切 |
| 企业架构／系统架构／参考架构／总体（战略）架构／SoS 架构 | **不同所关注实体（或不同范围）的架构**——换的是实体，不是同一实体的几块 | 按“谁的架构”切 |
| 基线架构／目标架构 | 同一个架构在**两个时点**上的样子 | 按“什么时候”切 |

第一组是四个关注域：业务、数据、应用、技术。TOGAF 把这四个分区称作 architecture domains（逐字 the architectural area being considered，正在被考虑的架构区域）。它的官方中文版把这个词定名为“架构域”（第 4.12 条）；而本书的“架构域”另有所指（＝架构领域），故本书一律写“关注域”。中文征本里没有 architecture domain 这个词；42020:2019 §E.4.1.5 另有“领域架构”（domain architecture，“领域中各系统的一种通用的组织性结构或设计”），两者不是一回事。

这四个名字要分两件事。它们的实指是四个领域各自的设计方案（下一段说它装的是什么）。若取一个系统（比如企业运营体系）为所关注实体、按关注域组织它的架构描述，这四个名字才对应**四块架构视图**：按 42010 §3.7，视图＝架构描述的一部分信息部件。§5.2.7 注 1 说关注点、利益相关方角度与方面体正是视角的组织基础。

而 TOGAF 那四块装的内容落在设计那一侧，是四个领域系统各自的设计方案：业务那段是业务战略、治理、组织与关键业务流程；数据那段是逻辑与物理数据资产和数据管理资源的结构；应用那段是待部署应用系统及其交互的蓝图；技术那段是支撑部署所需的软硬件能力。

TOGAF 拿“架构”给它们命名，是**误用**：它自报的“架构”两义本就停在“计划／结构”那一层（其一义逐字 a formal description of a system, or a detailed plan of the system at component level to guide its implementation），且自报依 ISO/IEC 42010:2007 并“采纳但不严格遵守”其术语。

TOGAF 把它们叫 business／data／application／technology architecture，名字里都带 architecture，这就是“看起来有四个架构”的字面来源。

那么，“四个架构”这个说法什么时候才成立？要等到你换掉所关注实体：那时得到的是那四样东西各自的架构，仍不是 TOGAF 那四块内容。42010:2022 引言里列的实体示例，本来就有这几个：“企业、组织、解决方案、系统（包括软件系统）、分系统、过程、业务、数据（作为数据项或数据结构）、应用、信息技术（作为集合）、使命、产品、服务……”。

取企业为所关注实体，你得到一个架构，业务／数据／应用／技术是它的四个关注域。若取业务、数据、应用、信息技术各自为所关

注实体，它们各自的架构也该是那两样内容（基本概念或属性＋管控原则）。

分界不在名字，在你此刻拿谁当所关注实体；而无论取谁当实体，那四块给的都是结构、蓝图与能力清单，落在设计那一侧。

而选哪一层，正本不替你定。15288:2023 §6.4 的注写着：“当企业或 SoS 为 SoI 时，企业架构或 SoS 架构即为系统架构”。同一套架构过程，落在哪个所关注实体上，就得到哪个层级的架构。

两层同时要时，42020:2019 §E.4.1.10 另给了“系统的系统架构”：满足 SoS 准则的系统（组成部分在运行上与管理上的独立性、演进本性、涌现行为、交互限于信息交换）的架构。

把这一节收成一张层级图。

- 企业架构是「企业运营体系架构」的简称。42020:2019 §E.4.1.2 给了两种读法，本书取第二种：企业架构「或者是一个企业的架构，或者是从企业角度考察的某个实体的架构」；那个实体就是企业运营体系。企业运营体系是一个系统的系统（SoS）（42020 §E.4.1.10），有它自己的一个架构，该架构承接企业战略。
- 业务流程体系、企业应用系统、业务数据体系、信息基础设施是它的四个子系统；四个子系统各有自己的架构与各自的多种架构视图，各自承接企业在这个领域的子战略。
- TOGAF 的 4A（业务／数据／应用／技术架构）对应的就是这四个子系统——它给子系统配的是“方案”，却借用了“架构”这个词，这就是前面那句“术语误用”的落点。

42010 把实体这件事交回问题空间：The identification of an EoI generally results from the definition of the problem space.（EoI 即所关注实体；意为：所关注实体的标识一般由问题空间的定义得出。）TOGAF 之所以把四者当作一个架构的四个域，是因为它取企

业为实体，而且它的跨域对齐（业务—应用映射、数据—应用映射）以“共享同一个实体”为前提。若真是四个不同实体的架构，“对齐”这件事根本无从谈起。

还有一条常被当成常识的说法：业务架构“推导出”数据架构与应用架构，这两者再“推导出”技术架构。这不是本书的立场，也不是标准的立场。四个域之间是对齐（相容）关系，不是推导关系：共同的上游是目的与需求，共同的下游是设计，域与域之间传递的是约束，而约束只排除做法、不指定做法（§10.3.4）。

“对齐”落在哪一层，取决于所关注实体。四个名字是**同一个系统的四块视图**时，要对的是视图之间：一条真实场景横穿四张图。它们各自是四个系统的架构时，要对的是架构之间：接口、边界与（不）重叠相容（本节 SoS 一段）。两件事不能互相顶替：拿视图对齐去要求四份独立架构，会逼出“它们必须合并”；拿接口相容去要求四块视图，对齐就落空。

一个可判定的反证：若是推导，结果应当唯一。而同一套业务与数据要求，在不同预算、不同合规环境、不同团队手里会得到不同的技术架构。这是常态，不是失败；推导不许这种多解，约束允许。TOGAF 的定义句也站在约束一侧：技术架构描述的是“支持业务、数据和应用服务部署所需的逻辑软硬件能力”，“支持……所需的”说明它响应要求，不是从应用架构推出来。

正本自己也承认这件事。42020:2019 §E.4.1 把“架构种类”列了十二种，总则里有一条注说得更直白：“用于表达某个视角、某个方面体、某个抽象层级或某个角度的描述，有时称为架构种类”，示例给的正是安全架构、逻辑架构、物理架构。所以“XX 架构”这类名字，多半只是一份描述的命名，不是“一个实体有好几个架构”。

第一个名字要当心的是“数据架构”：42020 §E.4.1.12 把它单列成一种架构种类（“由模型、方针、规则或标准组成，它们管控收集哪些数据，以及数据在数据系统和组织中如何存储、安排、集成和使用”），而 TOGAF 把“数据”列为四个关注域之一。同一个词，一边是种类、一边是域，它是这四个里唯一身兼两职的。

混乱的根源在日常用语：把“架构描述”简称为“架构”，把“架构视图”也简称为“架构”，简称来简称去，听起来就好像系统有多个架构了。就像只有一副骨骼的人，医生正面拍 X 光、侧面拍 X 光、横切面拍 CT：这些是同一副骨骼的影像，不是多副骨骼。同一个架构的若干视图也是这个关系。

> 术语注（这条断言的证据边界）：**正本支持的是“一个架构可以有多个架构描述”**——§5.2.2 逐字 The architecture of an entity of interest can be understood through one or more distinct ADs, each created for a purpose relative to the architecture and stakeholder needs.（一个实体的架构可以透过一份或多份不同的架构描述来理解。）**“一个所关注实体只有一个架构”这句更强的话，是本书的读法**（本书的读法）。它的用处是把“XX 架构”归位，不是拿来否认可选方案——分析阶段可以有多套候选架构，被采纳的只有一个。

> 术语注（“架构种类”这个筐里装着两种东西）：42020:2019 §E.4.1 的十二种种类，其实分甲、乙两路。**甲种按“谁的、什么时候、干什么用”分**——企业架构、系统架构、参考架构、总体（战略）架构、SoS 架构、产品线架构、产品服务系统架构；基线架构与目标架构按时点分。总则给的准绳是“架构的**目的、应用领域**以及在**实体与架构生存周期中的角色**”。**乙种其**

**实是描述的命名**——总则那条注说的安全架构、逻辑架构、物理架构，以及长得最像甲种的“数据架构”。**甲、乙不能并成同级**，否则“企业架构”和“逻辑架构”听起来就像同一个层级的两样东西。

术语注（两处同词不同义）：① 42020:2019 把**架构种类**（§E.4.1，十二种）与**架构视图**（§E.4.2，如语境视图）分成**两份清单**——本书 §10.4.4 给的定性是**四个领域各自的设计方案**（TOGAF 那四块的实指）；正本那份种类清单里与它同名的只有“数据架构”一种，其余三种（业务／应用／技术）在正本里根本没有条款。

② TOGAF 的 **architecture domain**（如“业务域”）≠ 42020 §E.4.1.5 的 **domain architecture 领域架构**（逐字：“领域中各系统的一种通用的组织性结构或设计”），后者属参考架构那一族；**而 TOGAF 官方中文版把 architecture domain 也译作“架构域”**（第 4.12 条），与本书“架构域＝架构领域”同名不同义，引用时按“谁的”分开。TOGAF 另一处自述“四个域本身就是视角”，紧接着自限“不是完整的视角定义”——故四域可作**视角组织的分类参考**，不等于 42010 意义上的视角规格（§8.1＋附录 B）。**而 TOGAF 的“视图”另有所指**：它把“企业安全视图”“可管理性视图”“标准合规视图”这类横切表达叫 view（逐字 The Enterprise Security view is concerned with the security aspects of the system.）。**故三套叫法各在一个坐标系里**：**域**＝分区名（TOGAF 定义句“正在被考虑的架构区域”）、**视角**＝归组惯例（TOGAF 另一处自述“四个域本身就是视角”）、**视图**＝架构描述的信息部件（42010 §3.7）。**而把**

> **TOGAF 的四域说成“四个视图”同样不成立**——它们的内容是**四个领域系统各自的设计方案**（见上）；TOGAF 的 domains／viewpoints／views 三套词彼此也不一致。

> 术语注（“架构域”＝**架构领域**，与 TOGAF 同名不同义）：本书的“架构域”是**架构领域**的简称——**以架构为专业的那一个领域**，与“数据域”“流程域”并列（第 16 章“治理落到架构域”、同章“架构域出架构决策＋理由”即是此义）。**它与 TOGAF 的 architecture domain 同名不同义**：TOGAF 官方中文版把后者也译作“架构域”（第 4.12 条），指的是业务／数据／应用／技术四个域；**本书把那四域叫“关注域”**，两处不可互换。至于“域／视角／视图”三者的完整立户与规格要求，归**第 13 章**（描述侧三词；概念依据＝30 号《架构描述核心概念研究报告》）。

### 10.4.5 不是框架与风格

有一种说法比“微服务架构”更常见，也更唬人：框架。“我们技术栈全是 Spring 全家桶”“M3 用的某某嵌入式框架”，紧接着一句：“用了 Spring Cloud，所以我们系统是微服务架构。”

这句话错在哪？Spring Cloud 是一套开源的框架：服务注册、配置中心、熔断降级，现成的轮子一堆。你引入它、按它的规则填进业务代码，一个微服务项目的骨架就搭起来了。但骨架搭起来，不等于架构有了。服务边界划在哪、数据归哪个域、接口契约怎么定，这三样框架一个都没替你决定。你下载的是工具。

一句话分清：架构是“这一个”实体在其环境中的**基本概念或属性与管控原则**，框架是“这一类”的组织方式。架构属于实体，框架属于领域。

试金石也简单：删掉它，系统还在吗？换掉 Spring Cloud，你的系统还是那个系统，对外功能可以一个字节不变。改动它的基本概念或属性，比如“数据所有权归哪个域”这条根本约定，这个实体就不是原来那个了。同一性问的是“**目的还在吗**”，不是“零件换了没”（忒修斯之船·T-144）。

风格也照这条尺子量：微服务、事件驱动、分层都是套路，换一套，系统还是那个系统。

顺带把挨着的几个词分开。构架是名词的动词化，本义“搭建承重骨架”；说“把 XX 构架出来”，口语里指把主干结构从无到有立起来。架构是本文讲的那组基本概念或属性与管控原则。框架是可复用的半成品。

三者的验收标准不同。“构架出来”的默认验收是骨架可见，而架构定型的灵魂是基本概念或属性与管控原则。图上有了框和线就说“构架好了”，问“为什么这么分、备选死在哪”却答不上来，这正是画图匠与架构师的分界。

分完三个词，还得说一句**字面**。“架”与“构”都从“木”：架是架起，构是搭盖，两字合起来就是架木造屋——搭出一个承重的骨架。这两个字天然把人往“结构”上带，而结构只是架构**可以关乎**的对象（42010:2022 §5.2.2 的五类之一），不是架构的内容。

要紧的是，字面最强的那一支联想，恰好是定义排除掉的那一支：图上有了框和线，读起来就像架构已经到手。这不是汉语独有的落差——英文的 architecture 出自拉丁语 architectura（建造术），字面同样停在建筑那一层。术语的意思由定义给，不由字面给；后文见“结构”“骨架”“蓝图”，先问一句：它说的是架构，还是架构可以关乎的对象？

### 10.4.6 不是“高层级”

最后一片误读最隐蔽，**以为“基本”就等于“高层级”**：画得越顶层、越概览，越像架构。

这是两个轴被叠成了一个轴。本书否掉了两个说法，这是其中之一：

> 说法：架构描述中的元素是系统的高层级元素。 否掉的理由：类型错误（“高”是**呈现粒度**，不是**基本性**）；即使指架构落点（components／relationships），“是”这个动词仍然错；深层是**把 fundamental 偷换成了 high-level**。

**“基本”与粒度正交**。共识算法很复杂，但可以是基本的；变量命名很简单，但永远不是。随手画的模块框图，再高层级也不是架构；起搏器的定时算法这类关键细节，再细也可能是架构。判别只看一条：**去掉它，这个实体还能达成它的目的吗**。

另一个被否掉的说法是“架构描述＝设计描述的概要”，“架构就是概要设计”是它的近亲。它们与本节同源：都是把种类层的东西搬到粒度层去说；分界见 §10.4.3 与 §10.3.3。

### 10.4.7 认知反转

回到章首那场评审会：石军放图、讲十分钟、散会。那场会连错三次，一次比一次深。

第一处，他把“架构描述”当成了“架构”。那张图只是架构描述里一个粗糙的切片：画了元素，没画关系，更没画原则；真正的架构，是那组不可还原的涵义（基本概念或属性）与管住它的管控原则。图会过时，那组涵义一直活着。

第二处，他把“画了图”当成了“做了架构”。做架构不是画一张漂亮的框图，是识别出“去掉就不是这个系统”的那组基本概念或属性，并立起管住它怎么造、怎么变的原则。图三个月没更新，M3 却被正确造出来，因为那组东西一直有人在脑子里守着。图是投影，投影后面那个东西才是实体。

第三处，也是最深的一处：他以为“改起来贵的”才配叫架构。于是他既漏掉了一些不贵但要命的东西：那几条老规矩没有一条是因为“贵”才被立起来的；也差点把一些很贵却与目的无关的选择（某个框架版本）请进架构。

三处错，其实是同一处错的三种说法：

> **架构不是一张图，也不是一堆贵的决定；它是实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则。它是那份内容，不是承载它的那份东西——能签收、能入库、能版本化的是架构描述；而“基本”的判据只有一条：目的。**

用第 1 章的一个词概括：石军对“架构”，完成了从见过到拥有的跨越：“见过”是背得出“架构”、画得出四层方块图；“拥有”是能答全四问（它是什么、不是什么、从哪来、往哪去）。四问答全，才算拥有。

回看第 8 章。那一章的判据是“评审对目标”：评审对着系统的目标审，审的恰恰包括架构描述是否对得起系统的那组基本概念或属性。而“不可还原”给下一章埋下最重要的种子：这组东西不是画出来的，是设计出来的。

下一章讲架构定义：什么算输入、什么算产出——不只是一份需求的一组输入，怎么被变换成两样内容，说清“这个实体是什么”的基本概念或属性，加上“它必须怎么造、怎么变”的管控原则。元素与关系是架构“可以关乎”的对象，其规格归下游设计定义过程；把它

们表达成架构描述，是再往后一章的事。“怎么在承重处做选择”，是再下一章的事。

## 案例进展②｜四张图，四个对不上的“架构”

恒信的架构师罗建，最近也在为“架构”头疼。他把合同审批系统画成四张图：业务、数据、应用、技术，挂满一面墙。

运维王默先看了一遍，指着技术架构图问：“部署在哪、坏了从哪查、监控看什么——这三样在里面吗？运维要的不是服务清单，是出事以后我照着能不能查下去。这一面图上没有，我这条线就是空的，出了问题只能挨个问人。”

罗建：“技术架构图我按模板画的，服务边界、依赖关系都在。”

需求分析师顾言路过，站住看了五分钟：“罗建，合同审批里'一份合同'，在这四张图里分别是什么？”

罗建指着四张图：“业务架构里，是从发起、法务初审、财务校验、业务终审到归档的一条流程；数据架构里，是'合同主表＋审批记录＋附件'三个实体；应用架构里，是审批引擎里的一个流程实例；技术架构里……呃，我没画合同。可这四个说的都是同一份合同，怎么会指成四样东西。”

顾言说：“就凭这最后一句——你的四张图是四个系统。ISO/IEC/IEEE 42010:2022 §5.2.2 写的是：同一个所关注实体的架构，可以透过一份或多份架构描述来理解。业务、数据、应用、技术是对同一个架构的四个关注域，必须能对得上。”

顾言把四张图并排推到罗建面前：“不是让你把它们合成一张。是要说清一句：这份系统作为哪一类东西存在——它是一套'合同审批能力'，还是三张表加一个引擎。前者能排掉做法：审批留痕、编号唯

一、跨部门可查，这些都从它推得出来；后者只是你此刻的实现，换一套照样能用。”

顾言接着说：“检验的办法是拿一条真实场景横着穿：'法务在 App 上审批一份五百万的合同', 业务图里走哪条流程、数据图里动哪些表、应用图里调哪个服务、技术图里跑在哪台机器上。一条场景走不完四张图，四张图就是四个系统。”

罗建那天把四张图撤下来重画。他后来说，他画的不是“四个架构”，是一个架构的四个切面：让它们对得上的不是模板，是一条真实场景。

## 小结

本章把“架构”这个词说成三件事：定义、价值、误解。

架构是一个实体在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。这两半并列，一头回答“它是什么”，一头回答“它必须怎么造、怎么变”；后一半是规范性的，可以被违反。

架构是那份内容，不是承载它的那份东西。两样内容可以精确、可以落成清单；标准把“定义”与“表达、记录”分成两个动作（§3.1），能签收、能入库、能版本化的是**架构描述**（§3.3）。元素与关系不是定义要素：它们搬到 42020:2019 §3.3 的注 1 里，降成了“通常意指体现于组件及其关系”，注释级、没有术语地位；它们在 42010 里的位置是 §5.2.2 的“架构可以关乎”那五类，以及附录 A.2。

“基本”不是形容词，也不是“改起来贵”。它的判据是目的：把这一条内容拿掉，这个实体还能达成它的目的吗；跑两遍，兑现侧与演化侧。可违反性、可引用性、可判定性排在它后面，是排序与检查，不是准入。

这一层非有不可，因为两头都推不过去：上向有涌现（15288 §3.46），下向分解是选择（42020 §8.4.7、§8.4.5）。它站在“要什么”与“怎么做”之间：标准给它的名字是**架构即约束**，约束落在设计定义过程可用的设计空间上（42020 附录 E.2.1）。它管得住下游，靠的是**相容性**（15288 §6.4.5.1）：说清“这个实体作为哪一类东西存在”，不相容的做法自己就出局。判一份架构做没做出来，看两件事：那句“是什么”说清了没有；它排掉了什么。

全章要清掉的误读分两族。一族把描述当架构：一张图、一份文档、立项报告里的方案框图。能签收、能入库、能版本化的是架构描述。另一族把设计当架构：系统的结构、概要设计、设计定义的高层级。差别在种类，不在粗细。

两族之外还有三样：一组决定（决定是抓手，不是架构）、一堆架构（一个所关注实体只有一个架构）、框架与风格（用框架不等于有架构）。它们的共同点只有一个：都在回答“架构是什么”的时候，把别的东西推了上去。

下一章讲架构定义：这两样内容是怎么从一组输入里被立出来的；再往后是架构决策（凭什么定下来的是这一个）与架构描述（一份架构描述里有什么）。

---

# 附录

## A 本章概念总图

架构，是一个实体在其环境中的基本概念或属性，加上管住它怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）

两半并列：前半描述性（它是什么）| 后半规范性（它必须怎么造、怎么变，可以被违反）

本书读法：系统不可还原的涵义（落前半句；管控原则是并列的第二样内容；**

不是"一组决定"**——决定是抓手）

架构是那份内容（基本概念或属性，加上管控原则），架构描述是承载它的载体：能签收、能入库、能版本化的是架构描述（§3.1 动词链 defining / expressing, documenting；§3.3）

三条判据：可交割性 / 同一性（多个载体，一个内容）/ 因果力

正本 *intangible and abstract*（Introduction L177）——义项＝不是人工制品、没有时空位置；不是"含糊"，不是"高层级"

"基本"（fundamental）：对目的必要且充分——必要性管入会（把这一条内容拿掉，这个实体还能达成它的目的吗），充分性管闭包（到这层可以停了吗）

判据栈：目的必要性（总闸）→ 可违反性 → 可引用性 → 可判定性 → 关注点闸门

两道闸门：目的（承重性）• 关注点（取舍与呈现）｜另有一类来源：环境（语境与影响）

两遍跑：兑现侧＋演化侧｜假判据：不可逆性 • 可判定性 • 可引用性 • 高层级（详见 §10.2.6）

价值

它站在"要什么"与"怎么做"之间——**架构即约束**：约束落在**设计定义过程可用的设计空间**上（42020:2019 附录 E.2.1）

非有不可：上向涌现（15288 §3.46）/ 下向分解是选择（42020 §8.4.7 • §8.4.5）⇒ 只能先定一个，且必须有人负责（42010 §6.10）

｜选定之前"合规"没有对象（15288 §3.13 *compliant implementation of the architecture*）

价值是哪几样：合规有基准（§3.13）• 意图可传达（42020 §5.6）• 不指定答案反而留出自由（15288 §5.9）• 约束即解放 • 复杂度可拿住

反面无它：三组落空——判据 / 责任 / 留痕（反读 15288 §6.4.4.2 a)~h)）｜必要性是逻辑的、不是文书的

管到哪儿：由目的裁（概念→可达的机制 / 属性→可判的度量面 / 原则→可持续的守持）

｜正因需求不许说"怎么做"，架构那层才有位置（15288 §6.4.3.1：measurable，却 shall not imply any specific implementation）

凭什么管得住下游：靠排除、不靠指挥——相容性（15288 §6.4.5.1 *in

accordance with* / *consistent with*）

四性质：异类自排 / 判定宾语都是做法（分类 • 度量 • 合规）/ 先于下游的设计选择 / 可陈述、可追溯

两句判据：那句"是什么"说清了没有；它排掉了什么

元素与关系：不是定义要素（2022 已从定义句删去）——搬到 42020:2019 §3.3 注 1（降为"通常意指体现于组件及其关系"，注释级）

42010 的位置：§5.2.2「架构可以关乎……」五类；附录 A.2「某些情况下根本要素就是组件及其关系」

元素三层：架构描述元素（描述层）| 架构落点＝components / relationships（实体侧，注释级）| 系统元素（实现层）

另有一词要分清：架构实体（附录 A 作"被架对象"）＝被架构的那个事物，不是架构的构成物

不是什么：不是一张图（蓝图只是四种隐喻之一）| 不是一组决定（决定是抓手，正位在 §5.2.12 / §6.10）| 不是更粗的设计（种类不是粒度；判据＝能否被设计违反）

| 不是一堆架构（关注域 / 描述层次 / 架构种类与范围 / 风格 / 框架五件事）| 不是框架与风格（用框架≠有架构）| 不是"高层级"（基本与粒度正交）

## B 本章的能力级

**第 ③ 级为主**

用第 1 章 §1.4.4 的尺子量一下：本章以第 ③ 级为主。架构不再是图上的一条线，而是能推演“改它会发生什么”的那组东西。

- **表征**——本章把哪几个词说准：架构（是内容，不是载体）／架构描述（工作产品：能签收、能入库、能版本化）／管控原则（定语位 governing principles）／一个所关注实体一个架构；并把四个近邻词分开：架构描述元素／架构落点（components／relationships）／系统元素／架构实体。（架构视角／架构视图／关注点／信息部件的立户归第 13 章。）

- **图式**——本章给出哪个可识别的结构：定义的两半；内容与载体这一对；判据栈与两道闸门；元素三层；价值的四节（站在“要什么”与“怎么做”之间／价值是哪几样／管到哪儿为止／凭什么管得住下游）
- 心智模型——本章给出哪个能推演的机制：承重测试（把这一条内容拿掉，这个实体还能达成它的目的吗）＋两遍跑（兑现侧／演化侧）；两头都推不过去 ⇒ 只能先定一个并有人负责；三组落空作反面；代理的两类失真
- 解释框架——本章不建此级；由第 17、18 章承担

> 收束句：**四问答全，才算拥有**。

## C 自查 · 这些说法对吗？

1. “我们画了架构图，架构工作就完成了。”——**错**。画出来的是架构描述；架构是那组基本概念或属性加管控原则。图会过时，架构可以一直有效。
2. “架构就是那张框图，立项报告里的方案框图也是，图过时了架构就过时了。”——**错**。过时的只是描述。M3 的图三个月没更新，那几条老规矩一直活着。
3. “我们有业务架构、数据架构、应用架构、技术架构四个架构。”——**错**。一个所关注实体只有一个架构；TOGAF 那四块的实指是四个领域各自的设计方案；若取一个系统为所关注实体、按关注域组织它的架构描述，那才是四块架构视图，必须能对得上。一条场景走不完，就是四个系统。
4. “微服务架构是架构。”——不准确。微服务是架构风格，是决策的套路；你的系统是什么架构，取决于它守着哪些不可还原的涵义、立了哪些管控原则。

5. “架构就是技术选型。”——错。技术选型是可以换的实现；换掉框架，系统还是那个系统，改动服务边界就不是了。
6. “”基本“就是高层级、就是概览，就是概要设计。”——错。“基本”与粒度正交：共识算法很复杂但可以是基本，变量命名很简单但永远不是；概要设计不是架构的别名。
7. “改起来贵、改不动的，就是架构。”——错。不可逆性只是代理，会漏判也会误纳；判据是目的——去掉它，这个实体还能达成它的目的吗。
8. “架构和架构描述是一回事，只是叫法不同。”——错。架构是那份内容，架构描述是承载它的工作产品：能产出、存储、交付、签收、版本化的是描述。
9. “把基本概念、基本属性、管控原则列成一张清单写完，就算有架构了。”——错。清单是“表达”，不是架构（§3.1 动词链：定义与表达、记录是两个动作）。清单可以很具体，但能签收的是那份描述，架构是它承载的内容。
10. “需求也能判达标，它不就等于把设计空间收窄了吗?”——错。需求可判，但判定宾语不是做法——15288:2023 §6.4.3.1 同节要求它 should not imply any specific implementation；判的是“达标不达标”，就一个做法也排不掉。
11. “标准指派架构去约束设计空间，这不就等于它管得住吗?”——不成立。指派是授权（准不准它管），不是能力；能力落在内容形态上——两样内容的判定宾语都是做法，不相容的做法自己出局。
12. “架构就是系统的结构，定义里说了它体现在元素与关系之中。”——错。那是 2011 版口径；2022 版已删去该层，它降为 42020:2019 §3.3 的注 1，注释级、没有术语地位。

13. “架构决策就是架构。”——错。决定是认出架构、并把架构记下来的抓手；架构决策在标准里另有定义（§5.2.12），是架构描述必须记录的内容（§6.10）。
14. “不补这一层，最多是文档少几页。”——错。会落空三组：判据（合规无对象、质量属性无判定通道、评审对不上目标）／责任（重大概念与约束没落到架构实体上、关注点没被回应、接缝无出处）／留痕（视图与模型不存在、可追溯性缺失、决策不被记录）。

## D 练习

1. 挑一个你正在做的系统，答“承重测试”：哪些东西“拿掉它，这个实体就不再是它”？写下至少三条。
2. 对上一题的三条，逐条跑两遍：兑现侧（实现目的需要它吗）与演化侧（目的变了它守不守得住）。
3. 找一条你们团队常说“很重要”的决定，用假判据检验一遍：它是不是因为“贵”才被叫架构？拿掉它，目的还达得成吗？
4. 找出你们团队所有叫“XX 架构”的图与文档，按五类归位：关注域、描述层次、架构种类与范围、架构风格、框架。有没有被当成四个架构的？
5. 挑一条你们团队说“必须遵守”的规矩，问它凭什么：是因为有人这么定，还是因为不合它就不叫这个系统了？前者是命令，后者才是架构在起作用。
6. 检查一次架构评审：评审的标的是一张图，还是“那组去掉就不是它的基本概念或属性”？把不该评的剔出去，把漏掉的请进来。
7. 你们用过的框架替你们定了什么、没替你们定什么？把“没替你们定的”列出来——那些就是架构决策。
8. 你们团队有没有一份“从没写下来、但大家都在守”的规矩？写下来，标出哪些是“去掉就不是它”的，哪些只是习惯。

9. 用四问的“它从哪来”量一次“架构”：从建筑图纸到基本概念或属性，这个词经历哪几个阶段？每阶段指什么？
10. 拿一份你们的架构描述，只问两个问题：那句“它是什么”说清了没有；它排掉了什么。（答不上第二个，就还没有架构，只有主张。）

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

### 标准


- [1] **R-04** ISO/IEC/IEEE 42010:2022 —— 架构定义（§3.2：实体的基本概念或属性＋实现与演化的管控原则）；架构描述＝工作产品（§3.3）；架构工作动词串（§3.1）；架构描述元素（§3.4）；架构视角＝一组惯例、定义句无属概念（§3.8）；方面体（§3.9）；架构视图＝信息部件（§3.7）；关注点（§3.10）；信息部件（§3.14）；规格（§3.16）；架构可以关乎的五类（§5.2.2）；关注点往往主观、有些与实体成功相抵触（§5.2.3／§5.2.4）；视角治理视图（§5.2.7）；架构描述元素所含概念（§5.2.9）；架构决策与理据（§5.2.12／§6.10，含 §6.10.1 记录取舍准则）；架构描述要交代的内容（§6.2~§6.10）；视角规格的 shall 清单（§8.1）；架构描述框架（§3.5／§7.1）；四种隐喻与两种哲学（附录 A.2）
- [2] **R-05** ISO/IEC/IEEE 42020:2019 —— 架构定义来源（§3.3，其注 1 承接 2011 版的“体现于元素与关系”并降调）；架构实体（§3.6）；关注点（§3.8）；治理与管理的分工（§5.3~§5.5）；治理的裁决动作（§6.4.3／§6.4.4／§8.4.4）

· [3] **R-19** TOGAF（The Open Group）—— 企业架构的四域（业务／数据／应用／技术），本书按“一种常见的关注域划分（分类参考）”引用；**中文征本无 architecture domain 一词**。**TOGAF 自报的“架构”有两义**（据 TOGAF 9 公开草案 chap01，页脚自述 unapproved draft）：① a formal description of a system, or a detailed plan of the system at component level to guide its implementation（系统的正式描述，或构件一级、用来指导实现的详细计划）；② the structure of components, their inter-relationships, and the principles and guidelines governing their design and evolution over time（组件结构、相互关系，以及管住其设计与演化的原则与指南）。两义都停在“计划／结构”这一层，且自报依 ISO/IEC 42010:2007 并“采纳但不严格遵守”其术语——其口径落在本书 §10.4.3 判归设计的那一侧（即所谓 4A＝四个领域系统各自的设计方案，不是四种架构，也不是一个架构的四块视图），引用时按另一坐标系处置。>>- **引用边界（ADM 那类流程图）**：TOGAF 的 ADM 是**迭代的**（iterative, over the whole process, between phases, and within phases）、**次序可变**（the Business Architecture (or at least the completion of it) may well follow completion of the Information Systems Architecture or the Technology Architecture）、**四域可各自独立立项**（each architecture domain … is developed and approved as a major project independently of the others, possibly by different personnel）；且 ADM 把 **system design／system engineering／system development** 明确列为**架构工作之外的相邻活动**。故“需求→业务→应用→数据→技术→实现→运行”这类**直线流水线读法不成立**——ADM 是循环，也没有“运行”这一阶段。（据 TOGAF 9 公开草案 chap05）

- [4] **R-28** ISO/IEC/IEEE 42030:2019 —— 架构评估以规定的评估目标为基准（§3.4）、对照架构的预期目的评估其质量与价值（§1 a)~d)、§4.5）；评估的对象是架构、不是架构描述的适宜性（§1 注）；三层评估（§4.3）
- [5] **R-37** ISO/IEC/IEEE 15288:2023 —— 架构定义（§3.5，主体为系统）；设计定义“足够完整到足以支撑对架构的一次合规实现”（§3.13）；系统需求（§6.4.3.1）；系统（§3.46）；系统元素（§3.47）；架构定义过程及其成果（§6.4.4.1／§6.4.4.2）；设计定义与相容性（§6.4.5.1）；架构实体与系统元素是两种不同的概念（§5.9，依中英文对照版，英文原文作 notions）

**经典文献**

- [6] **R-52** Kruchten，“The 4+1 View Model of Architecture”（IEEE Software, 1995；经典文献，非标准）—— 软件领域最经典的视角集合，本书按“一种常见的视角组织方式”引用
- [7] **R-53** Simon Brown，“C4 model”（c4model.com；非标准，具体版本号未核）—— 四层缩放
- [8] **R-54** John Zachman，“A Framework for Information Systems Architecture”（IBM Systems Journal, 1987；经典文献，非标准）—— 企业架构分类矩阵
- [9] **R-79** IEEE Std 1471-2000（架构描述推荐实践）—— ISO 42010 的前身（2000s IEEE 1471 → 并入 ISO/IEC/IEEE 42010）；本章 §10.2.1 的来路表引它

# 第 11 章 架构定义：把判断变成判据

## 11.1 引言

「架构」这个词说准了，下一个问题立刻就到：它是怎么来的。

上一章把架构说成了两样内容——实体在其环境中的基本概念或属性，加上管住这个实体怎么实现、怎么演化的管控原则。这两样内容不会自己长出来。有人得把它们立出来，还得让后面的人非照它办不可。本章讲的就是这件事：**架构定义**。

行业里这件事常被说成“开始做架构设计”。这句话里有两处偏移。一处把定义说成了设计——设计是另一样工作，在换手点的另一侧，归第 14 章。另一处把定义说成了一次落笔：好像架构师坐下来写一份文档，写完，架构就有了。真正发生的事比这具体：有一组东西送进来，有一组东西留下来，中间有几步要走。

先把入口和出口看清。

### 11.1.1 误解现场

冰链科技，同一间会议室，隔了一周。这回嵌入式硬件工程师石军带了东西来：一张图、一页选型对比、一页手写的老规矩。

图是上一章那张四层方块图，他新加了两处标注。选型对比是“边缘预判”这件事放在哪里：箱内算，还是上报之后云端算，各写了一栏利弊。那页老规矩是散会之后他一个人补的，字迹很挤——温度数据归谁管、断链就算失效、固件升级不许搞坏已经出厂的箱子。

“这次准备得比上次齐。”石军说，“图有了，选型有了，规矩也写了。这次总该算是把架构定下来了吧。”

产品经理林涛先开口，他关心的是能不能往下走：“预判这件事跑在硬件上还是云端上？”

石军：“两个方案都列了，还没定。”

林涛：“那你们打算什么时候定?”

石军：“看情况。”

林涛：“还有一个问题——这么改完，M3 还是 M3 吗?”

三个问题，一个都答不上来。会议室里没人觉得意外：上一周也是这么散的。

测试季昭补了一句：“我该验的东西在哪?”

云端软件的吴浩那边也接了一句，是复读那条老规矩：“温度数据归追溯平台管——这条我记着。可这条线，是谁划的?”

石军把那张图、那页选型、那页老规矩收起来的时候说：“我是从需求出发的。”

这句话听着没问题。问题就在这句话里：他收到的东西不止一份需求，他能交出去的东西也不该是一张图。

### 11.1.2 本章要回答什么

读完本章，你应该能回答：

1. 架构定义吃进去的到底是什么？为什么说“收到一份需求”把入口说小了?
2. 目的与目标算输入吗？如果不算，它们站在哪、以什么身份说话?
3. 送进来的东西，是不是都会变成架构内容？没变成的那些去了哪?
4. 需求与架构，究竟谁规定谁？“双向的必要且充分”是什么意思?
5. 这一步的动词为什么是“变换”，而不是“汇总”或者“整理”?
6. 留下来的是哪几样内容？它们在标准里的位置由哪一条定?
7. 清单和架构描述是一回事吗?
8. 元素与关系在这条链上住在哪？产出里为什么没有它们?
9. 一句判断凭什么变成别人非依不可的依据？中间少哪一步会塌?

10. 这一步走到哪里为止？选定和背书归谁，表达成件归谁？
11. 候选是从哪来的？“我想到了”和“这是一个候选”差在哪？
12. 手上没有资产、没有经验的时候，架构还做不做得出？哪几条线不许踩？

### 11.1.3 本章的骨架

先把骨架立起来，后面几节都挂在它上面。

一句主张：架构定义不是“给架构下定义”，是一次**看得见进出口**的变换。

入口那一句：送进来的是相关架构、组织与项目的方针与指令、生存周期概念与约束、利益相关方关切与需求、系统需求与约束。这句话出自 ISO/IEC/IEEE 15288:2023 §6.4.4.1 的目的句。

还有一样不在其中：目的是准绳，也就是上游给的业务或任务目标，它不是输入。而架构目标与关键成功准则是这一步要立的产物。

出口那一句：留下来的是基本概念或属性，加上管控原则。它们会落成清单，不是一份描述。

中间那一句：这一步靠**取材与综合**，不是发明。同一个问题，可以立成“箱内算”，也可以立成“云端算”，两条都成立。

还有一句是收尾的。这两样内容要成为下游必须依据的东西，四个环节缺一不可：先判断“这个实体是什么”，再把它立成内容，再经选定与背书，最后下游必须依据它。本章只走到第二环。

骨架立完，本章接着往下走。先看吃进什么、吐出什么、中间加什么，这是入口、出口与中间那一步。再看一句判断凭什么让人非依不可，那是中间那条链，也是本章停步的地方。最后看素材的来路：资产、知识、经验各占一位，以及一样都没有的时候怎么办。

## 11.2 吃进什么，吐出什么，中间加什么

先把这一进一出两件事说准。本章后面所有判断都压在这一进一出上：入口不清，“把范围收窄”就没有对象；出口不明，“依据”就立不起来。

### 11.2.1 吃进去的不只是一份需求

“我是从需求出发的”——这句话对了一半。需求确实在输入里，但输入不止需求。

标准把入口写得很具体。ISO/IEC/IEEE 15288:2023 §6.4.4.1 的目的句逐字：

> This process transforms related architectures (e.g. strategic, enterprise, reference, and SoS architectures), organizational and project policies and directives, life cycle concepts and constraints, stakeholder concerns and requirements, and system requirements and constraints into the fundamental concepts and properties…

（中译：这一过程把相关架构、组织与项目的方针与指令、生存周期概念与约束、利益相关方关切与需求，以及系统需求与约束，变换为……基本概念与属性……；其中相关架构指战略、企业、参考与体系之体系架构这类。）

动词的宾语是五样东西，需求只占其中两样。另外三样根本不是需求：

- 相关架构：战略架构、企业架构、参考架构、体系之体系架构。它们是上一轮架构工作的产物，这一轮直接拿来当起点。

- 组织与项目的方针与指令：治理方向、管理指南、适用政策与标准。它们规定的是“这件事得按什么规矩做”。
- 生存周期概念与约束：运营概念、保障概念这一类东西。它们给出场景与边界。

石军那句“我是从需求出发的”，把入口说小了三样。这三样他当天其实都碰过：公司的固件升级规程就是方针与指令；客户那句“冷链运输全程不许断”背后是运营概念；隔壁产线跑了两年的追溯做法就是一份参考架构。他一件都没当成输入。

第二样要单独说一句。组织与项目的方针与指令，给的是规矩；这是治理在这一步给出的头一样东西。它还有一样是效力，那一侧不走输入这一路，见 11.3.1。

**还有一样不在那五样里：目的与目标。**

它们不是被“送进来”的，它们是准绳。标准把这件事放在另一处动作里。那处动作先刻画问题空间，再定架构目标与关键成功准则（15288:2023 §6.4.4.3 b)1)、b)2)）。换句话说，**架构目标是产物，不是输入**。42020:2019 §8.4.4 f) 的动作是“识别并定义架构目标，使其在利益相关方关注点、需求或质量属性方面应对问题和机会”。

这条区分有实际后果。把目的当输入，等于承认“目的早就写好了，我只是照着执行”；把它当准绳，才是“这一次要做到哪一步，得由我定”。

最后立一条边界，它防的是最容易混的一件事：**架构内容不是需求的一个分支**。需求按需求的判据验，架构内容按架构的准入判据验——就是上一章那条承重测试。两样东西各有各的合格条件，不能互相收编。

### 11.2.2 架构相关的留下，其余各有去向

入口看清了，接着看留下来的那一部分是哪些。送进来的东西不会原样变成架构内容。标准在这里用了两个字：相关。

15288:2023 §6.4.4.2 e) 要的是一条可追溯的线，从系统架构元素追到关键架构相关（key architecturally-relevant）的利益相关方需求与系统需求。这句话的重心在那个限定词上：不是所有需求都架构相关。

于是在这一步，每一条需求都要过一道问：

> 这条需求，是不是只有在“这个实体是什么”定下来之后才判得了？

是，它就进架构侧；不是，它就不进。这条问法是本书对标准那句话的操作化——标准只要求建立“关键架构相关”那条可追溯的线，怎么判出“相关”是本书补的。

判完之后，需求有三条去向，哪一条都不许是“消失”：

- **落成架构内容**：它变成了某一则基本概念、某一条基本属性，或者某一条管控原则。
- **被既有架构覆盖**：上一轮架构工作已经把它管住了（参考架构、企业架构里就是这么定的），这一次不必重立。
- **不在架构层承接**：它归设计定义或者实现去管。比如“这个按钮按下后 200 毫秒内出结果”，它不改变“这个实体是什么”，归下游。

石军那次会上遇到的一个问题正好是个样本：林涛问“温度数据上传间隔从 1 分钟改成 5 分钟，影响什么”。这个间隔不进架构内容——改它，M3 还是 M3。但它也不是“没关系”：它落在第三类，要去设计层承接。上一章那张图之所以答不上来，恰恰是因为图上没有说清哪一层管这件事。

去向还有一个硬要求：**可追**。每条需求最后停在哪一格，要能倒查回去。42010:2022 §6.2 要求架构描述里写一份未处理项声明，把没回应的利益相关方与关切、资源限制、不符合项连同理由一并记下——这正是“不许消失”在标准里的着落。

同一个判据在恒信那边也出现过。合同审批系统里，法务提了一条：“单笔超过五十万的合同要自动加一道复核。”这条进不进架构内容？

按那句问法过一遍：它是不是只有在“这个实体是什么”定下来之后才判得了？不是。五十万这个数、加一道复核这个动作，都不改变“这个系统是什么”；改这个阈值，系统还是这个系统。它落在第三条去向：不在架构层承接，去设计层与流程配置里管。

紧接着法务又提了另一条：“同一份合同在任何一张表里，金额的折算结果必须一致。”这条不一样。它指向的不是某个动作，而是“这个系统是什么”——一个多事实源的系统和一个单事实源的系统是两回事。它进架构内容，而且很可能长成一条管控原则。

两条需求挨着提出来，一条进、一条不进，靠的不是谁更贵、谁听起来更严重，靠的是同一句问法。

### 11.2.3 需求与架构：双向的必要且充分

第 11.2.1 节说输入里需求只占两样，这一节说这两样与架构之间的关系有多紧。标准给的是一个双向的判据。

15288:2023 §6.4.3.3 NOTE 18 逐字：

> Feedback helps ensure that the specified system requirements have been adequately **captured and expressed**. **Confirmation is made that** they are a necessary and sufficient response to stakeholder

> requirements and a necessary and sufficient input to other processes, in particular architecture and design.

（中译：反馈有助于确保规定系统需求已被充分捕获并表达；并确认它们既是对利益相关方需求的必要且充分的回应，也是对其他过程——特别是架构与设计——的必要且充分的输入。）

两头都要“必要且充分”，这句话的分量在两个方向上不一样。

一头是“对它上游”：系统需求是不是对利益相关方需求的必要且充分回应。不必要，就是多做了——要求里写进了没人要的东西；不充分，就是漏了——该覆盖的没覆盖。这一头失效，后果是**做的不是该做的事**。

另一头是“对它下游”：系统需求是不是架构的必要且充分输入。不充分，架构就答不全，没法据它立内容；不必要，架构里就会多出没有需求来源的东西。这一头失效，后果是**架构答不全、设计里多出东西**。

还有一句限定，管的是输入长什么样。15288:2023 §6.4.3.1 说需求定义过程“产生一组可测的系统需求”，而且这些需求“**不应蕴含任何特定的实现**”（should not imply any specific implementation）。

这两条合起来，正是下一节“变换”的前提。如果需求不许指定实现，那么“怎么实现”就必须有人来定——定它的人就是架构定义这一步。反过来说，如果需求已经把实现写死了，这一步就无事可做：它只剩抄一遍。

### 11.2.4 变换：同一批输入，能立出不同架构

现在看中间那个动词。标准给的词是 transforms——**变换**。不是 collect（汇总），也不是 summarize（整理）。

为什么是变换？因为变换必然做选择。同一句“要能提前告警”，可以变成“箱内算”，也可以变成“上报之后云端算”，两条都成立。标准不管选哪一条，它只要求把选出来的那一条立成内容。

先看留下来的是什么。

定义句给的是两样。实体在其环境中的基本概念或属性，加上管住这个实体怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。42020:2019 §8.2 c) 把概念化的成果说成同一件事：key concepts and properties, and the principles guiding its formulation, application and evolution。

定义句里那个连接词是“或”——**概念或属性，取其一即可**。把概念与属性分列成两份清单，是实践里的做法：逐条准入方便，逐条分配也方便。这一点要说清楚，因为“分列”是本书与多数团队的整理习惯，不是标准的要求。

再看它们后面去哪。

标准把去向写得很清楚。42020:2019 §10.4.4 e) 的动作是：

> Allocate relevant concepts, properties, characteristics, behaviors, functions, features or constraints to the entities and relationships to be modeled.

（中译：把相关概念、属性、特征、行为、功能、特性或约束，分配给将要建模的实体与关系。）

注意它分配的对象是**实体与关系**。这一步在本书的链上属于后文——本章只把去向交代清楚，不在本章执行分配。同一节的 NOTE 2 还补了一句重要的话：这个分配可能已经在概念化过程里做过了；若是，细化侧只核查它是否仍然相关、细节是否够用。所以“分配归后

文”这句话要说得准：标准没有强制这次分配必须留到后文，概念化过程里就可以做。

管控原则管什么，标准另有一句。§ 10.4.4 f) 的动作是：

> Identify relevant principles and precepts that will guide evolution of the architecture entity.

（中译：识别将指导架构实体演进的相关原则与准则。）

这一句只管演化那一半。“实现”那一半在定义句里：§ 3.2 说的是 for the realization and evolution。两处合起来，才是“管住它怎么实现、怎么演化”。引用时分清这一点，不然会把半句当成整句。

最后是一条诚实边界。

> **标准没有给“什么算基本”的判据。**

定义句里那个“基本”（fundamental）是整句里分量最重，也最容易被一带而过的词。上一章已经说明：标准确实没给这条判据，本书那条承重测试是形式化，不是标准条文。本章承上，不重立、不重证。

## 案例进展①｜冰链：石军把判断写成老规矩

散会那天下午，石军没画图。他先问了自己一句：我到底要把什么写下来。

他想起上一章那句话：写下的不是“决定的集合”，而是一句涵义加一组原则。于是他打开一个空白文档，写下标题“M3 老规矩”，然后一行一行往下写。

第一行他写的是涵义的正面回答：

> M3 是一台**从装箱到交付全程可追溯**的医疗冷链温控箱；温控是它的手段，可追溯是它的目的。

下面接了三段，每段一个规矩，都从上面那句话里读得出来。

温度数据归谁管。采集端归固件，上传端归云端；两边各自记录时间戳，对不上时以采集端为准。他补了一句为什么：“数据归属不定，追溯链就有两个源头。”

断链就是失效。他写这条的时候停了一下，把它念出声：“从装货到交付，链条上任何一段缺失，这次运输就算失效——不管箱内温度多漂亮。凭什么？凭上面那句涵义：M3 卖的是可追溯，温控只是手段。拿掉可追溯，它就不是它了。”

升级不许毁掉已出厂的箱子。新增传感器类型不得破坏已有的追溯链路，新固件必须能读旧批次的记录。

写完三段，他停下来看了一遍，发现自己做了一件跟以前不同的事：这一次，他没有先画图，也没有列选型；他是先把“它是什么”写下来，再从这句话里推出规矩。图可以后画，选型后选——但这两样以前都是起点，今天成了后来的事。

那份文档他存了，一共一页。文件名不叫“架构”，叫“M3 老规矩”。

下班前，系统工程师小张过来看他写的那一页，读了两遍。她挑的不是哪一句写错了，是第三条缺的那一句：“前两条都说清了管到哪儿：哪一端记、整条链算不算断。第三条只说'升级不许毁掉已出厂的箱子'，说的是不许，没说管到哪一批、哪一版固件。一条管控原则说不出自己管到哪儿，就没法当判据；旁人照着它判，判出来就是另一个范围。”

石军没答上来。他把那一页折起来，收进了抽屉。

### 11.2.5 架构师写目的陈述，立概念与原则

到这里可以回答一个常被问的问题：这一步既然只是“变换”，那架构师到底做了什么？是不是把上游给的东西整理一遍？

不是。整理不产生内容。这一步真正往里加的东西有四样。

给每个候选写一句目的陈述。42020:2019 §8.4.5 e) 的动作是 Formulate purpose statement(s) for each potential solution，其 NOTE 5 补了一句：每个解可能有不同的目的陈述。也就是说，“这个方案是为了解决什么”不是从需求里抄来的，是写出来的。

把手段立成概念与原则。解空间里找到的做法要提升成内容：它是什么，这是概念；它必须守什么，这是原则。42020:2019 §8.4.7 与 E.3.3.4 把它写作 enshrine these characteristics in architecture principles——把特征供进原则里。

适配与调和。既有做法不能照搬。42020:2019 E.3.3.2 的动词是 instantiated and specialized，也就是实例化并特化；§8.4.6 NOTE 3 又补了 harmonization，说的是几件模式或战术一起用时，得调和到眼前这个局面上。

“什么都不改”也列成一条候选。42020:2019 §9.4.7 NOTE 2 明写：The status quo is sometimes one of the alternatives that could be considered。“什么都不改”本身就是一条候选，它得跟别的候选一起被看见、一起被比较。

这四样里，前三样是往里加内容，第四样是把“现状”摆上桌。四样都不是靠整理能得到的。

这四样在冰链那边各有一个画面。

目的陈述。石军给“边缘预判”和“追溯链新输入”各写了一句，两句不一样，这是关键；写成一样，说明他其实只有一个方案（例见案例进展②）。

把手段立成概念与原则。他写下的“断链就是失效”不是一句需求，而是一条原则：它管的是演化，将来新增什么功能，都不许让链路有空档，不只是这一次的实现。

适配与调和：他引用产线给的那套写法时，把槽位的粒度从行级提到版本级。这一步不是抄，是调。

“什么都不改”那一条是他最后补的，逼着另外两个方案回答一句——你比现在好在哪。

### 11.2.6 产出到哪儿为止

这一节的出口要点清：留下来的只有基本概念或属性，加上管控原则，再没有第三样。

它们的去向也说到了。概念与属性后面会被分配给将要建模的实体与关系，管控原则管演化那一面。两处动作分别见 42020:2019 §10.4.4 e) 与 §10.4.4 f)。那一步在本书的链上归后文。本章把清单立出来，就到这里。

再划两条界，它们防的是两种最常见的越界。

第一条：元素与关系不在产出里。

它们的含义写在 42020:2019 §3.3 的注 1 里，措辞是“通常意指体现于”（usually intended to be embodied in）。这是注释级措辞，没有术语地位。另一半也得说：架构可以关乎结构。42010:2022 §5.2.2 逐字列出架构可涉及的诸项，正本共列五项，里面就有行为与结构（behaviour and structure）。这是外延，不是产出；本章不切分元素、不连接关系，也不论哪些连接断不得。

另有一个词要钉住：**架构实体**。它指被架构的那个事物，不是架构的组成部分。15288:2023 §5.9 有一段写明，架构实体与系统元素“代表两种不同的概念”（two different notions）。这一句术语注不能省——省掉，读法就退回到把架构当成元素与关系的那个年代。

第二条：清单不是描述。

清单属内容侧，视图属描述侧。本章交出去的是清单；把内容按视角落成视图、写成一份架构描述，是第 13 章的活。标准在这里留了余地：那几份清单可以不属于任何视图。出处是 42010:2022 §6.7 EXAMPLE 2，那是示例，不是要求，但它足以说明清单与视图不是一回事。所以有一条分寸必须记住——**“清单写完”不等于“描述有了”**。清单是内容的表达，不是架构；描述是把这份内容交给别人的那种说法。

还有一笔账要在这里落下：这几份清单在第 7 章的账上也找得着。产品数据有设计态、制造态、使用态三种存在（第 7 章 §7.3.2），设计态里没有实物，**开发产品 ≡ 开发产品数据**；本章立出来的基本概念或属性、管控原则这几份清单，就是**设计态产品数据**里内容侧的那一支——是数据，不是实物，也还不是一份描述。

这一笔交代过，题面里“产品数据”四个字才算落地。至于它怎么按视角落成视图、写成一份交得出去的描述，仍归第 13 章。

还有一句留在这里，它属管控原则对下游的约束，不属架构内容：**关系是成本**。两个部分之间要是全互联，牵动就成片；“能不加的关系就不加”这一条可以立成一条管控原则，交给下游去守。至于关系怎么分类、怎么降耦、怎么契约化，那是设计定义过程的活，归第 14 章。

## 11.3 一句判断，凭什么让人非依不可

入口和出口看清了，看中间这一段路。一句判断停在一个人脑子里的时候，谁也不必依它；要让别人非依它不可，中间有几步不能省。这一块讲这几步，也讲清本章走到哪一步为止。

### 11.3.1 判断怎么才不只是意见

标准给这一步的动作是生成备选、选定、表达三件事（15288:2023 §6.4.4.1 目的句）。本书把它拆成一条更细的线，看一句判断怎么一步步变成依据：

① 判断“这个实体是什么” → ② 把它立成内容 → ③ 选定并背书 → ④ 下游必须依据它。

四个环节，缺哪一个都走不通：

| 缺哪一环 | 会怎样 |
| --- | --- |
| 缺① | 没有可划的界 |
| 缺② | 别人没法引用、没法评审、没法判定 |
| 缺③ | 它只是“架构师的个人意见” |
| 缺④ | 各做各的 |

这四个“缺”不是假设，冰链那间会议室里各有一个样本。

缺①的样本是石军画的那个框：“边缘预判模块”悬在固件层和云端层之间，他自己都答不出它算哪一端。界没划清，后面没有一处判得了。

缺②的样本是季昭那句问话：“我该验的东西在哪？”老规矩只在几个人脑子里，没有一处可以引用、可以评审、可以判定。

缺③的样本是“我们打算做成那样”——一句话没被选定、没人背书，下一个人接手就能改。

第三环要再说细一层。它是两件事挤在一个环里，不是一个人做两下。

**选定**是判断：在几条走法里，哪一条最合目的。它归架构过程，依据就是本章立的那一条。

背书是另一回事。它承诺的不是“这条更优”，是“这条就这么办了，后果我担”。所以它能依靠的凭据不是判据，是职权。职权不能自授——架构师自己给不出这一环。标准把这一步的动词写成“取得一致意见”（obtain agreement）：15288:2023 §6.4.4.3 e) 2)，说明它是去向别人要一个答复，不是自己写一个结论。

判定与授权分家，原因就在这：算得清优劣的人不承担后果，承担后果的人才有资格点头。这一步的展开归第 12 章；至于那位有职权者是谁、按什么程序点头，是组织职能的事，归第 16 章。

缺④的样本是图里那根“温度数据”的箭头：吴浩那边以为重连逻辑在固件，而固件是石军自己写的，他知道里面没有。两边在图里各找各的依据，接缝就断在这里。

这里有一处必须替标准说清。这条链是**本书自己归的读法**。正本给的是另外三样。四个活动是准备、概念化、评估、细化（15288:2023 §6.4.4.3 a)~d)）；三个核心过程是概念化、评估、细化（42020:2019 §8、§9、§10）；另有架构治理与架构管理两个并行过程（42020 §6、§7），**它们与本章是并行关系，不是本章的下一步**——两者的展开在下篇，治理见第 16 章。读法与正本不冲突，但它是读法，不是条文。

**本章只走到第二环**。①与②合起来，就是标准里的前三步：定要支撑哪些决策、把关切变成判据、取材并综合候选。③里的评估与筛选归第 12 章；④连同表达成件归第 13 章。

为什么不一次走完？因为这一步的边界不在“做完了没有”，在它交给下一站的是什么：本章交出去的是摆好的候选和它们各自的目的陈述，不是选定的那一个。

另一面也说一句。标准把“项目那份视图清单”的产生写成一条反推的路。关切由视角框定，每个视角至少有一张视图，每张视图要应对它那个视角框定的全部关切并记下已知问题。

这条路最后一步是反向查一遍，看是不是每个关切都被至少一个视角框定、被至少一张视图应对了。这条路的出处是 42010:2022 §6.6 与 §6.7。那几步落在描述侧，归第 13 章。本章这条链跟它方向相反，用的是同一批货：关切进来，内容出去。

### 11.3.2 先定要支撑哪些决策

这一步在标准里叫准备（Prepare）。它产出的不是架构内容，是“这一次怎么做”。15288:2023 §6.4.4.3 a) 列了几件事：

| 标准动作 | 白话 |
| --- | --- |
| 识别关键里程碑与决策 | 这次定出来的东西，要供哪几个决策用 |
| 给出架构定义的策略 | 走法与做到多深 |
| 为架构治理与架构管理工作做好准备并作出规划 | 谁来批、谁来守，什么时候过闸 |
| 识别并规划所需的使能系统或服务 | 建模工具、配置库、测试台这些支撑 |
| 取得对这些使能系统或服务的访问权 | 上面那些东西要真的能用 |

第一件最容易被跳过，也最要紧：**这次的定义是给谁用的**。同一份需求，供“要不要立项”用和供“这一版能不能投产”用，做到多深完全不一样。不先问这一句，后面所有“细化到什么程度”的争论都没有准绳可依。

第二件是策略——走法与做到多深。走法不止一条。从问题空间往前推、从既有架构往回接、从中间往外扩，都在标准的可选里（42020:2019 E.3.2 共列七种，此处只举三种）。做法不同，取材也不同。

剩下三件是支撑：谁批、谁守、手上有没有工具。它们不产出内容，但没有它们，前两件落不了地。

第三件那一行要补一句边界：它的展开不归本章。谁批、什么时候过闸，一半归第 12 章的决策权与时机，一半归第 16 章的治理机构与程序。

石军那天缺的正是第一件，而且缺得很典型。他带了三样东西进会议室，没一样是“这次要支撑哪些决策”的产物：他不知道这一版是给产线看的还是给客户看的，也就不知道该细到什么程度，于是图、选型、老规矩三样都只做了一半。

### 11.3.3 关切怎么变成判据

这一步在标准里是概念化的第一件事，但它的第一句是个否定句：不要重做问题空间。

15288:2023 §6.4.4.3 b)1) 的动作是“刻画问题空间”，其 NOTE 3 立刻补了一句：这件事是与业务或任务分析过程一起做的，那个过程负责识别并定义问题或机会的空间。也就是说，问题空间在那里已经刻画过一遍，这一步不重做，只在架构这一侧把它细化。

细化的产物是两样：**可判的质量度量**，和**架构目标与关键成功准则**。

42020:2019 §8.4.3 i) 的动作是“识别并排序（正本原词 prioritize）与这些利益相关方关切相关的质量度量，把它们用作架构开发与评估的基础”。注意“排序”两个字：关切进了清单不是就完了，它们要按重要程度排出来。

42020:2019 §8.4.4 接着给出目标的写法。识别并定义架构目标，使其在利益相关方关切、需求或质量属性方面应对问题与机会（f））；定义关键成功准则，用来评定问题在多大程度上得到解决，并

为备选方案的探索与选择提供依据（h)）；必要时把成功准则转成评估准则（j)）。

这里有一处最容易混的地方，值得单独说：**关切本身不是判据，判据是目的**。

关切说的是“谁在乎什么”——客户在乎链条不断，产线在乎造得出来，运维在乎修得起。它给出的是判据生效的现场，不是判据本身。真正拿来筛东西的判据仍然只有一个：把这一条内容拿掉，这个实体还能达成它的目的吗。关切的作用是让这条判据有地方用——没有关切，你连“目的有没有达成”都没法量。

分清这一层，两个常见的失效就有了名字：

- **查漏**：某个关切没有任何条目承接。客户在乎断链，可老规矩里没有一条管它。
- **查滥**：某条目只服务一个关切，且不承重。为了某位领导的一句话，专门立一条谁都不会违反的原则。

石军那份老规矩正好一漏一滥。漏的是“客户怎么知道链条没断”——那份一页纸里没有一条管告知；滥的是他原本想加的“显示界面必须用蓝色”，既不影响目的，也没人会违反。

### 11.3.4 候选从哪来

目标和准则有了，接下来是找解。这一步的动词最容易被说错。

标准用的词是 Synthesize potential solution(s) in the solution space（15288:2023 §6.4.4.3 b)3)），也就是在解空间里综合出潜在解。42020:2019 §8.4.5 的动作清单是三件：识别既有解、识别问题缓解策略、为每个潜在解写出目的陈述。正本里前两件同用 Identify，第三件用 Formulate。

**动词是综合、识别、写出，不是发明**。这句话有两层意思：一层是“你不需要天才”；另一层更要紧，解不是凭空来的，它有来路。

来路有三条。

一条是上游给的偏好解类。15288:2023 §6.4.4.3 b)3) NOTE 4 写明：业务或任务分析过程已经定下了偏好解类，潜在解是在类内被识别与分析的。类的边界不在架构师手里——他在类内找解。

一条是既有解与既有架构。42020:2019 §8.4.5 b) 的动作是识别既有解；同节的 NOTE 1 补了一句：已经跑过的做法是天然候选，“什么都不改”也在其中。

一条是模式、战术与启发式。42020:2019 §8.4.5 c) 6) 的动作是识别相关的架构模式、启发式与战术。它们是组合规则与基本手段，可实例化、可特化。

三条来路各给一样东西：类给边界，既有解给素材，模式战术给手段。找完还要做一件标准明确要求的事，就是**为每个候选写一句目的陈述**（§8.4.5 e)）。它的 NOTE 5 说得很清楚——每个解可能有**不同的**目的陈述。

这句话的分量在“写”字上。候选之间真正的差别，往往不在技术而在目的：这一个是为了少花钱，那一个是为了少出事。目的陈述写不出来，说明这个候选还没想清楚，它只是“我觉得可以”。

有一处数量上的误会值得先说清。标准确实要求候选得“够”。数量、种类、跨度三者都要够（42020:2019 §9.4.7 NOTE 3）；评估时若判定候选的数目、种类或跨度不够，可以要求再造。但“够”从来不是“多”。

真正管住这件事的，是下一节要立的四条准入要求：过得了的就是候选，过不了的多摆一条只是多花时间。冰链那边三条候选，恒信那边也三条，都不是因为三这个数吉利，是因为各自的来路不同——一条来自上游给的偏好解类，一条来自既有做法，一条是“什么都不

改”。来路不重复，权衡空间才铺得开；来路重复，摆五条也只是同一条候选的五个说法。

### 11.3.5 什么样的想法才算候选

列出来的想法不会都算候选。标准给了两组要求，合起来可以落成四项检查。

辨得开。权衡完全相同的两条，标准说不予考察（42020:2019 §9.4.8 NOTE 1）；彼此相似、提供不了区分的那一条，剔除（§9.4.7 b)）。两条候选在权衡空间上分辨不出差别，多摆一条只是多花时间。

可追溯。用了哪件既有解、哪个模式，要能指出来；更要紧的是**适配理由写没写**。42010:2022 §6.10 要求架构决策连同理据一起记录：§6.10.1 管决策记录，§6.10.2 管理据记录。用了别人的东西而不写为什么适用，等于把一件没验证过的做法直接搬进来。

把这两条展开，就是四项检查：

| 检查 | 问什么 |
| --- | --- |
| 有类可归 | 它落在上游给的偏好解类里吗? |
| 有物可依 | 它用了哪件既有解、参考架构、模式或战术？适配理由写了吗? |
| 有径可循 | 它是既有做法的实例化与特化，还是“我以前都这么干”? |
| 有辨可评 | 它跟别的候选在权衡空间上辨得开吗? |

**四项全过＝候选；任何一项不过，它就是一个“想法”**。这个区分不是文字游戏：候选要进后面的评估，想法要退回去接着想。

看一个过不去的样本。恒信那边有人提过一条：“把审批流的每一步都做成可配置的规则。”听着很像个方案。四项检查一条条过：

有类可归：上游给的偏好解类里没有“规则引擎”这一类，它悬着。有物可依：用了哪个既有解？答“没有，我们自己设计”，于是适

配理由也无从谈起。有径可循：它既不是既有做法的实例化，也不是特化，它是从零发明。有辨可评：它跟“现状照旧”分辨得开吗？分辨得开，这一项过了。

四项过了一项，它就是一个想法，不是候选。想法不是没用——它是往后找候选的线索。把它当候选摆进评估，会占掉一条权衡空间的位置，还让评审把时间花在“这个要不要做”上，而不是“这几个哪个更好”上。

### 11.3.6 概念、属性和原则从哪来

那几样内容不是从上游现成搬过来的。三样的来历各不相同，各有一句话。

**概念是幸存下来的**。一个实体身上可以说的概念很多，能进架构的只是极少数——它们经历了一轮又一轮的取舍，最后活下来。活下来的理由只有一个：去掉它，这个实体就不再是它自己。

**属性是涌现出来的**。这一点有正本支撑。15288:2023 §3.46 给“系统”的定义逐字是：

> arrangement of parts or elements that together exhibit a stated behaviour or meaning that the individual constituents do not.

（中译：各部分或各元素的安排，它们合在一起呈现出单个构成部分所不具备的既定行为或意义。）

整体呈现的东西，零件单独不呈现——这就是涌现。架构要管的那几条属性，恰恰是这一类：它们不由任何一件部件保证，只能由这个实体整体保证。所以它们必须在架构这一层被立出来，往下分是分不出来的。

**管控原则是自加的**。它不是从任何一条需求里读出来的。42020:2019 §8.4.7 与 E.3.3.4 的动词是 enshrine——把特征供进原则里。需求说“要可追溯”，原则说“新增传感器类型不得破坏已有的追溯链路”；后一句是前一句在这一轮的具体化，它是加进去的，不是抄出来的。

三句话合起来，正好回答“这一步到底加了什么”：它让一批概念幸存、让一批属性现形、再亲手补上几条原则。

## 案例进展②｜冰链：先写目的陈述，再摆候选

石军最后还是把选型那一页重新做了一遍。这回他先写目的陈述，再摆候选。

他的目的陈述只有一句：在箱子落地的第一时间就知道这次运输能不能签收。

第一版他写了三行：一句目的陈述，两条候选。

**候选甲：边缘预判**。箱内算，实时判定，异常当场报警。目的陈述写的是“把判断留在现场，不依赖网络”。

**候选乙：追溯机制的新输入**。把预判结果当成追溯链上的一条记录，判定仍然在云端做。目的陈述写的是“复用已有的追溯链，不新增一条并行机制”。

写到这里他卡了一下——因为两条都成立，而且第一条更“高级”。他差一点就顺手把甲留下、把乙删掉。

季昭正好路过，看了一眼他摊在桌上的三行，指出一处：“甲这条，你写的是'边缘预判'——它落在哪个偏好解类里？用到了哪件既有解？”

石军答不上来。他查了一圈：公司内部没人做过，也没有现成的模式或战术可依。

季昭说：“那按四项检查过一遍，它四项里只过一项。过不了的，只能叫想法，不叫候选；标准对权衡完全一样的两条根本不予考察，§9.4.8 的注里写着。想法是往后找候选的线索，不能直接摆到评估桌上。”

乙那条不一样：追溯链是现有的，往里面加一类输入是既有做法的实例化，可追溯。

于是他没删甲，改了甲的写法，把它写成一条候选，并注明它的来路与代价：新硬件、新固件、产线要重排测试。然后他补了第三条。

**候选丙：现状照旧**。什么都不改，继续用现在这套。他一开始觉得这条不用写——“照旧”算什么方案。但标准里写着它是候选之一，他还是写了，目的陈述留空，后面用准则来判。

三条候选摆出来，他自己看了一分钟，说了一句：“原来我不是在选一个方案，我是在写三个不同的打算。”

## 11.4 素材：资产、知识与经验

入口、出口和中间那条链都讲完了，还剩一个问题：那几样内容是拿什么做出来的。架构师手上不可能什么都没有——他总有些旧解可参、有些做法可循、有些经验可用。这一块讲这三样各占什么位置，以及一样都没有的时候怎么办。

### 11.4.1 资产：起点与素材

组织里有形的、可以拿来就用的东西，在标准里各有一个名字：相关架构、架构仓库、参考架构、模式、战术、既有解。它们是这一步的起点与素材。

起点与素材这两个词的分量不一样。起点是说，很多架构工作不是从白纸开始的：上一轮的结论、隔壁型号的做法、行业里公开的参

考架构，都可以直接拿来当第一版候选的底子。素材是说，拿来之后不能照抄。

标准在这里的动词是 instantiated and specialized，也就是实例化并特化（42020:2019 E.3.3.2）。把它落到你这个实体上，再按你的局面改。照抄的后果很具体：参考架构是给一类实体定的，它管的是那一类共有的东西；你的实体独有的那部分，它一个字都没说。

用了就要说明为什么适用。42010:2022 §6.10 要求架构决策连同理据一起记录（§6.10.1 决策记录／§6.10.2 理据记录）。在这里它多一层分量：把一件既有做法搬进你的架构，正是这样一项要记的决策。**适配理由写没写**，是“用了资产”与“被资产用了”的分界。

资产还有两种偏差要防。一种是可能缺失。新领域的项目手上真没有可参的东西，42020:2019 E.3.1.2 讲的就是这种极端情形，正本里写作 no prior system。另一种是不该沿用。旧资产在那儿，但这次的意图是彻底离开它；E.3.1.5 给的名字是 legacy revolution，说的是遗产还在，可从赞助方到团队都打算跟它划清界限。有资产不等于要用资产。

参考架构怎么特化，看一个具体的动作。行业里常见的一份参考架构会写“每个数据源要有唯一的归属方”——这是一条原则。

搬到 M3 上，它变不成原话。M3 的数据源不止一个，箱内传感器、云端记录、产线的批次表各算一路；“唯一归属方”要落到“哪一段数据归哪一方”上，最后写成“采集端归固件，上传端归云端，对不上时以采集端为准”。同一条原则，在行业那份参考架构和 M3 上长出两句不同的话——中间那一步就是特化。

“不该沿用”也有样本。石军上一代产品留下的那份固件升级规矩是：“新固件必须兼容上一代硬件。”这一代 M3 换了主控，旧硬件的兼容要求不但用不上，还挡路——它会把选型限制在一个已经停产

的芯片上。这条资产不是坏的，它只是不属于这一轮。分清“这条规矩错了”和“这条规矩不合这一轮”，正是重判资产可用性那一步要做的事。

### 11.4.2 知识：能不能交给别人用

第二样不是实物，是想事情的形式。标准把这一类叫 enablers，也就是使能项。它们是视角、模型种类、建模方法、模板、元模型、框架（42020:2019 §8.4.8 f)）。

它们的用处不在内容本身，在让内容长成可复用、可交接的样子。同一个判断，用散文写下来和用一套视角规格写下来，区别不在这份文件本身，在下一个团队能不能接着用。

所以这类东西的评价标准只有一条：能不能交给别人用。标准把这件事写得很直白。这些使能项可以放进组织的仓库，供其他架构工作使用（§8.4.8 f) NOTE 2）。放不进仓库、只能留在某个人电脑里的东西，不算这一类。

这条标准很实用。石军那页老规矩里，“温度数据归谁管”是知识；“这块硬件我熟”不是知识，它是石军的经验，交不出去。

可交接与交不出去，差别落在一个具体动作上：别人拿到它，能不能不问你就往下做。

石军那页老规矩里，“温度数据归谁管”这一条可交接。新来的人读到它，知道去哪一端找数据、以谁的时间戳为准，可以照着往下做，不必再把石军叫来。“断链就是失效”也可交接，因为它说的是判据（链条断没断），不是做法（用什么字段判）。

交不出去的样子也常见：一张画得极漂亮的时序图，上面只标着“上报”，没写谁发、发什么、什么条件下发。作者在场时人人都懂；作者一走，它就成了一幅画。**判据不是“写没写下来”，是“别人能不能接着用”**。

还有一类东西也归这里，而且最容易被当成私产：**视角规格**。一次架构工作里立起来的视角，是可以写下来、可以直接给别人用的：这个视角框定哪些关切、用什么模型种类表达、按什么惯例读。写下来它就成了组织的知识；只留在某个人脑子里，它就只是那个人的习惯。

所以这一节真正要问的问题是：这一次做完，你给组织留下了什么可以拿走的东西？一页老规矩、一份视角规格、一套命名与分层的约定，都算。**留下的东西越多，下一次的起点就越高**。

### 11.4.3 经验：只帮着找，不帮着选

第三样是经验。标准给的正式名字是启发式（heuristics），定义是“架构工作的指导方针”，并明说它们来自经验（derived from experience，42020:2019 E.3.3.5）。

启发式在标准里跟参考架构、模式、战术是平级的（E.3.3.1 把它们并列），各有各的用处。它们在生成那一环最有用：想不出候选的时候，经验告诉你“这类问题以前是这么解开的”，一下就能起个头。

但它们有一处界限必须钉住：**不充当选定判据**。

同一条启发式前面还有一句限定。不是所有启发式在所有情形下都适用（Not all heuristics apply in all circumstances）。选定阶段的判据是评估准则、强制项与关切，不是“我以前都这么干”。

**经验最容易出错的位置，恰好在“选”那一环**。想候选的时候用经验，是本事；挑候选的时候用经验，是把偏好偷渡成理由。这两种用法看起来都是“凭经验”，位置差一环，性质完全不同。

石军那句“我一直这么干”，用在找候选上是资历，用在选定上就是话术。它回答的是“我以前怎么做的”，而不是“这次为什么该这么做”。

经验在选定环出错的完整样本，恒信那边有一个。系统架构师罗建上一轮做过一个审批系统，用的是“流程引擎加表单配置”这条路，

做成了。这一次他一看需求，也是审批，也要留痕，当场就说：“还是流程引擎，我熟。”

这句话的问题不在判断本身（流程引擎也许真的合适），而在它出现的位置。它出现在候选还没摆完的时候，于是后面所有的评估都成了给这个结论找理由：选它，再补两行权衡，再写一句“符合上一轮经验”。他省下的不是时间，是把别的候选摆上桌的机会。

正确的用法是：把他那句“我熟”放在生成那一环。它帮着快速写出第一版候选，还帮着预判哪里有坑。到了选定那一环，判据换成评估准则、强制项与关切；他还是可以发言，但发言的内容得是“这个方案在准则 X 上得几分”，不是“我熟”。

### 11.4.4 资产稀缺时怎么办

有一类项目叫新领域：手上没有可用资产，团队也没做过同类的事。这时候架构还做不做？

先回答一句：**资产多少不改变这件事有没有，只改变怎么做**。标准没有把“资产不足”列为可以不做架构的条件，它把不同情形分成几种，各给不同的走法。分流的依据写在 42020:2019 E.3.1.1 的引入句里：

> The approach which should be adopted depends upon the complexity of the system of interest, its novelty, realization mechanisms and/or the uncertainty in the stakeholder needs.

（中译：应当采用的走法取决于所关注系统的复杂性、它的新颖程度、实现机制，以及利益相关方需要的不确定性。）

按这几个因素分，常见的有四种情形：

| 情形 | 标准里的样子 | 资产状况 | 架构工作的要点 |
| --- | --- | --- | --- |
| 全新开发 | no prior system，一切全新（E.3.1.2） | 几乎为零 | 从问题空间往前推 |
| 产品线启动 | 要建的是一条产品线，不是单个系统（E.3.1.7） | 几乎没有，但要一次抽共性 | 架构一次为多次服务 |
| 遗产革新 | 旧系统在，但意图是彻底离开（E.3.1.5） | 有资产，但不打算沿用 | 重判资产可用性 |
| 增量启动 | 不确定“什么最合适”，且不确定性很大（E.3.1.6） | 少且不确定 | 产出后续步骤的选项 |

> **正本全数与本书取法**：42020:2019 **E.3.1 共六种情形**——E.3.1.2 全新开发／E.3.1.3 产品线中的新产品／E.3.1.4 遗留系统演进／E.3.1.5 遗留系统革命／E.3.1.6 增量式启动／E.3.1.7 产品线式启动。本节取其中四种；另两种（**产品线中的新产品、遗留系统演进**）**手上有资产**——前者“已有能力与技术相近的既有系统”，后者“遗留系统存在，且不能被弃置”——不在“资产稀缺”这个题目下。

第三种和第四种最容易混为一谈。遗产革新是有资产不用，要防的是被旧做法拖回去；增量启动是没把握，标准给的处方是“拥抱这种不确定性，先给出初始步骤，同时给出后续步骤的选项，把已知与未知的不确定性都算进去”（E.3.1.6）。

情形不同，适配的地方也不同，可以归成五处：

- 走法从“资产优先”改成“前向为主”。
- 取材换来源：专家、技术扫描、模式扫描、风险评估。

- 输出加两件：决策与理据从可选变承重；“后续步骤的选项”本身成为架构内容的一部分。
- 迭代策略换：评估前置、细化工期推后，并把“现状照旧”列进候选。
- 收窄的分寸也要调。

收窄的分寸这一条再往下引，就是三条红线，都落在这一节。

- **不许拿“没资产”当不写架构的理由。**缺资产改变的是取材方式与细化深浅，不改变有无。标准明许架构里的元素初始可以是概念性的（notional），接口只定义到“足以传达架构意图”为止（42020:2019 §5.6）。
- **不许把参考架构当答案。**正本的动作是实例化并特化；用了就必须写适配理由。
- **不许把“不熟”写成“不该”。**一条界划下去的理由，得是“它威胁目的”，不能是“我们不熟悉它”。

所以有一句话要写下来，每次划界的时候问一遍：这一条界，是因为它威胁目的，还是因为我们不熟？说不出理由的界，就不该划。

还有一条连带也值得记：**资产稀缺时的架构工作，本身就在补资产。**这一轮立起来的视角规格、模型种类、接口契约范本、模式与战术，都可以回流到组织仓库里供下一次用（42020:2019 §8.4.8 f) NOTE 2）。第一次做的人多花的时间，有一部分是在给后来的人铺路。

### 11.4.5 认知反转

本章要反转的直觉只有一个：架构是“画”出来的。

在第一间会议室里，他做的每件事都符合这个直觉。他画了一张图，图上有四层方块和一根箭头；又在图上新加了两处标注；还把这件事叫作“做架构设计”。整套动作里，唯一没有发生的事是：没有人问过“这个实体是什么”，也没有人把答案写下来。

于是没有可划的界（他答不出“边缘预判模块”算哪一端），没有可引用的内容（季昭找不到该验的东西），没有可背书的定案（“我们打算做成那样”），也没有可以立得住的依据（吴浩和石军在图里各找各的）。四个环节全空。

第二间会议室里，他做的事少得多：一页纸，一句涵义，三条规矩。可这一次，四个环节里的前两个活了——有一句“它是什么”，有三条从这句话里读得出来的规矩。他没有画图，架构反而第一次有了形状。

这就是反转：**架构不是画出来的，是定义出来的**。画图属于后面的表达，归第 13 章；定义这一步做的是另一件事——把送进来的东西变换成几样内容，再把它们立成能被别人依据的样子。

反转还有一半，藏得更深：**这一步不是整理，是加东西**。整理不产生内容，而这一步必须产生内容：它要把概念、属性、原则这三样立起来；它要给每个候选写一句目的陈述，要把既有做法适配到眼前这个局面上。上游给的东西再全，也换不出这几样。

所以“架构定义”这四个字读起来像个语义动作，做起来却是一次有输入、有产出、有次序、有归属的变换。它吃进的东西比“需求”多，吐出来的东西比“清单”少——只有那几样内容，而且它们还得有人定、有人背书，下游才非依不可。

章末那句话是产品开发经理赵刚说的。他把工牌从脖子上摘下来放在桌上，说：“咱们做了这么多年 M3，怎么被问到‘它是什么’，一句话都说不上来。”

## 案例进展③｜恒信：罗建先列关切，图一笔没画

恒信那边，罗建把上一轮的“四张图，四个对不上”贴在工位上，贴了两个月。

这次合同审批系统要改一处：法务要求所有金额字段加上“本币折算口径”这一栏。上一轮他是直接开画——四张图同时铺开，谁先画完谁算数。这一次他没动笔。

他先做的事是列关切。他在一张纸上写了三行。

- 法务：口径必须唯一，同一个合同在任何一张表里折算结果一致。
- 财务：折算率有生效日期，跨期合同要能看到当时的口径。
- 法务：录入时不能要求录入的人记住口径代码。

三行写完他看了一眼，发现第二行跟第一行挨得很近，第三行其实是一句需求而不是关切，它说的是“界面该怎么做”，那是设计的事。他把第三行划掉，改成“录入的人不需要懂折算规则”。

然后他去找素材。他没有去翻架构文档，那里没有可用的东西。他去翻了两样：上一轮项目留下的追溯表模板，一份同类的既有解；集团层面那份数据口径管理规定，一份方针与指令。

两样东西他都用了，各写了一句适配理由。

“用追溯表模板，是因为它已经把'金额、口径、时间'这条链的槽位留好了；特化点是把生效日期从行级提到版本级——这是对模板的改动，得记下来。”

“用集团口径规定，是因为它定的折算顺序与本系统一致；特化点是它只管年度报表，本系统要管到单笔合同——这一条差异也得记。”

他把这两句念给了会上的需求分析师顾言听。

顾言问了那句他每次都会问的话：“你用的这件既有做法，适配理由写在哪？”罗建指了指纸上那两段：“写在这儿了——用了别人的东西，就得把理据一起记下来；用了别人的东西不写清楚它为什么适用，等于把一件没验证过的做法直接搬进来，下一轮谁都不知道它凭什么留在这儿。”

写完这两句，他才开始摆候选。三个候选里有一个是“现状照旧”——继续用现在这套人工核对。他没删它，理由写得很直白：另外两个方案都要改数据模型，而这个方案的成本最低，得摆在一起比。

那天他一笔没画。他自己也没想到，“不画图”这件事，对他来说才是这一轮最大的变化。

## 小结

本章讲的是架构定义这个动作本身。

它的入口比“一份需求”要宽：送进来的是相关架构、组织与项目的方针与指令、生存周期概念与约束、利益相关方关切与需求、系统需求与约束；目的与目标不在其中，它们是准绳。它的出口比“一份描述”要窄：留下来的只有基本概念或属性，加上管控原则；元素与关系不在产出里，描述也不是。

中间那一段的动词是变换，不是汇总，也不是整理。变换必然做选择，而这个动作真正往里加的有四样：给每个候选写一句目的陈述、把手段立成概念与原则、把既有做法适配到眼前、把“现状”也摆到桌上。

那几样内容要成为下游必须依据的东西，中间有四个环节：判断“这个实体是什么”、把它立成内容、选定并被背书、下游必须依据它。本章只走到第二环——选定与筛选归第 12 章，表达成件归第 13 章。

供给这一步的三样东西位置不同：资产是起点与素材，知识是能交给别人用的形式，经验只帮着找、不帮着选。三样都没有的时候，架构照样要做——只是取材的方式和细化的深浅要换，还有三条线不许踩。

一句话收口：架构定义不是给架构下定义，是把输入立成基本概念与属性、管控原则这几样内容，并且立到别人非依它不可。

# 附录

## A 本章概念总图

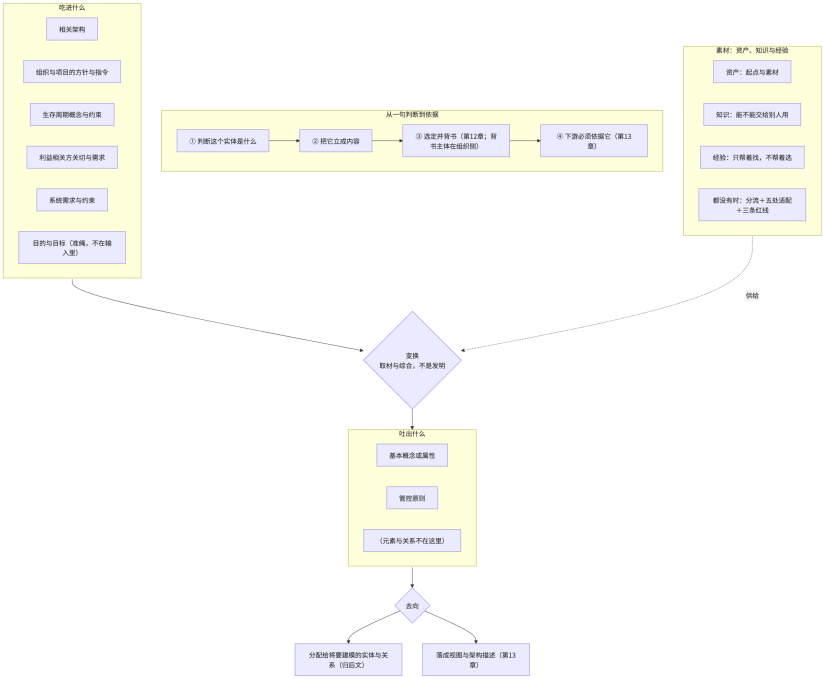



图 11-1

## B 本章的能力级

### 第 ③ 级为主

用第 1 章 §1.4.4 那把尺子量一下：本章以第 ③ 级为主。架构定义不再是一次落笔或一次整理，而是能推演“少哪一环会塌”的那条链。

- **表征**——本章把哪几个词说准：架构定义（是一个动作，不是给架构下定义）／输入（不止需求；目的与目标不在其中）／关键架构相关（不是所有需求都架构相关）／变换（不是汇总、不是整理）／

清单与描述（一个是内容侧，一个是描述侧）／资产·知识·经验（三样位置不同）／资产稀缺（本书取四种情形，正本 E.3.1 共六种）。

- **图式**——本章给出哪个可识别的结构：吃进什么 → 变换 → 吐出什么 → 下游必须依据它这条链；需求进架构的三条去向；候选的四项检查（有类可归／有物可依／有径可循／有辨可评）；三样素材各占一位、都不构成最终根据。
- **心智模型**——本章给出哪个能推演的机制：同一份输入可以变换出不同架构（变换必然做选择）；一条需求该往哪去，看它是否只有在“这个实体是什么”定下来之后才判得了；缺资产改变取材方式与细化深浅，不改变有无；经验偏差正好发生在“选”那一环。
- **解释框架**——本章不建此级；由第 17、18 章承担。

> 收束句：**能答全四问，才算拥有**——这一次变换吃进什么、吐出什么，现在你能自己判了。

## C 自查 · 这些说法对吗？

1. “架构定义就是给架构下一个定义。”——**错**。那是第 10 章做完的事。架构定义是一个动作：有输入、有产出、有次序、有归属。
2. “我是从需求出发的，所以我的输入就是需求。”——**不准确**。需求只占输入里的两样；另有相关架构、方针与指令、生存周期概念与约束三样。缺了这三样，入口就小了。
3. “目的和目标当然也是输入，谁做架构不看目的?”——**错**。它们不在输入里，它们是**准绳**。架构目标与关键成功准则本身就是这一步的产物。
4. “送进来的需求最后都会变成架构内容。”——**错**。只有关键架构相关的那一部分会。其余的各有去向：被既有架构覆盖，或者不在架构层承接，但它们都不许消失。

5. “架构定义就是把需求整理一遍、汇总一下。”——**错**。动词是变换：同一批输入可以立成不同的架构，变换必然做选择。整理不产生内容。
6. “先有需求，需求决定架构，所以是单向的。”——**不准确**。这一段是双向的：系统需求既要对利益相关方需求作出必要且充分的回应，又要成为对架构的必要且充分输入。两个方向都有失效形态。
7. “基本概念、基本属性、管控原则这三样，是标准要求分开列的。”——**错**。定义句的连接词是“或”——概念或属性，取其一即可。分列两张是实践做法，便于逐条准入与分配。
8. “标准规定了什么叫'基本'。”——**错**。标准没有给这条判据，本书那条承重测试是形式化，不是条文。
9. “清单写完了，架构描述就有了。”——**错**。清单属内容侧，描述属描述侧。把内容按视角落成视图、写成一份描述，是第 13 章的事。
10. “元素与关系是架构的第三样内容。”——**错**。架构可以关乎结构，但那是外延不是产出；它们的规格归设计定义过程。另外要分清：**架构实体**指被架构的那个事物，与系统元素是两回事。
11. “这一步既是变换，那架构师只是把上游的东西整理一下。”——**错**。这一步往里加四样：给每个候选写目的陈述、把手段立成概念与原则、把既有做法适配到眼前、把“现状”也摆上桌。
12. “有一句判断就够了。”——**不成立**。判断要成为依据，中间有四个环节：判断、立成内容、选定并背书、下游必须依据它。缺任何一个都走不通。
13. “候选就是我能想到的方案。”——**不准确**。候选要过四项检查：有类可归、有物可依、有径可循、有辨可评。任何一项不过，它只是一个“想法”。
14. “参考架构拿过来就能用。”——**错**。正本的动作是实例化并特化；用了不写适配理由，等于把没验证过的做法直接搬进来。

15. “经验丰富就是优势，选的时候也该听老手的。”——**错**。经验（启发式）在生成那一环最有用，**不充当选定判据**；经验偏差恰好发生在“选”那一环。它回答的是“我以前怎么做的”，不是“这次为什么该这么做”。
16. “手上没有资产，这个架构就没法做。”——**错**。缺资产改变取材方式与细化深浅，不改变有无。三条红线：不许拿“没资产”当不写架构的理由、不许把参考架构当答案、不许把“不熟”写成“不该”。

## D 练习

1. 拿一份你手上的输入清单，逐项放到它该在的位置：哪些是相关架构，哪些是方针与指令，哪些是生存周期概念，哪些是利益相关方关切与需求，哪些是系统需求。**目的与目标在不在清单里**？在的话，把它挪出去——它是准绳，不是输入。
2. 挑十条你们项目的需求，逐条判去向：落成架构内容、被既有架构覆盖、还是不在架构层承接。判据只有一句：这条需求是不是只有在“这个实体是什么”定下来之后才判得了。
3. 对上面归到“不在架构层承接”的那几条，各写出它该去哪一层。写不出来的，说明它其实可能没人管。
4. 拿一次你们做完的架构活动体检：判断、立成内容、选定并背书、下游必须依据它。四个环节缺了哪一个？缺的那个，在会上是怎么被绕过去的？
5. 找三条你们团队说“必须遵守”的规矩，各问一句：它是从哪条需求或目的读出来的？读不出来的，它可能是一条该立的管控原则，也可能只是一句习惯。

6. 把你正在做的系统里能想到的候选列出来，逐条过四项检查。哪几条过不了“有物可依”（用没用什么、适配理由写没写）？哪几条过不了“有辨可评”（跟别的候选分辨得开吗）？
7. 把“现状照旧”写成一条正式候选，也给它写一句目的陈述。写不出来，说明你还没认清现状在保什么。
8. 给一个候选写三句话：它是什么，即概念；它必须守什么，即原则；它为什么适用，即适配理由。三句里哪一句最难写？
9. 想一想你们团队最靠经验的那位同事：他的经验在找候选时帮上了什么忙，又在挑候选时帮了什么倒忙？举一个具体的例子。
10. 如果你的项目属于“资产稀缺”那一类，先给自己分流：是全新开发、产品线启动、遗产革新，还是增量启动？按这个情形，五处适配里哪几处要动？
11. 每划一条界，写一句理由：这一条界，是因为它威胁目的，还是因为我们不熟？把你最近划的三条界拿来试一遍。
12. 挑一句你们团队常说的话，比如“我们是微服务架构”“这个必须用云原生”，问它四个问题：它说清“这个实体是什么”了吗？它是一条概念、一条属性还是一条原则？它去掉之后，这个实体还能达成目的吗？它凭什么管住下游？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022 —— 架构定义（§3.2：实体的基本概念或属性＋实现与演化的管控原则；连接词为 or）；架构可以关乎的五类（§5.2.2，含 behaviour and structure）；未处理项声明（§6.2）；关切由视角框定（§6.6）；每个视角至少一张视图、视图应对其视角框定的全部关切并记已知问题、反向覆盖（§6.7，含 EXAMPLE 2 “不在任何视图之内的信息部件”）；架构决策与理据的记录（§6.10，含 §6.10.1 记录取舍准则）
- [2] **R-05** ISO/IEC/IEEE 42020:2019 —— 概念化过程的动作（§8.2 c) 成果句；§8.4.3 i) 识别并排序质量度量；§8.4.4 f) h) j) 架构目标、关键成功准则、评估准则；§8.4.5 b) c)6) e) 识别既有解／模式战术启发式／为每个候选写目的陈述；§8.4.6 NOTE 3 调和；§8.4.7 立概念与原则；§8.4.8 f) NOTE 2 使能项入组织仓库）；评估侧判据（§9.4.7 b)、§9.4.8 NOTE 1）；细化侧去向（§10.4.4 e) 把相关概念、属性等分配给将要建模的实体与关系——其 NOTE 2 明许该分配可在概念化过程完成；§10.4.4 f) 识别将指导架构实体演进的原则与准则）；架构与设计的关系（§5.6）；架构即约束（E.2.1）；走法与资产稀缺（E.3.1.1$_{\text{E.3.1.7、E.3.2、E.3.3.1}}$E.3.3.5，含 instantiated and specialized 与 derived from experience、“Not all heuristics apply in all circumstances”）
- [3] **R-28** ISO/IEC/IEEE 42030:2019 —— 架构评估以规定的评估目标为基准（§3.4）；本章只用它划“评估与筛选取第 12 章”的边界，不入论据
- [4] **R-37** ISO/IEC/IEEE 15288:2023 —— 系统架构定义过程的目的句与输入（§6.4.4.1：transforms 与相关架构、组织与项目的方针与指令、生存周期概念与约束、利益相关方关切与需求、系统需求与约束）；成果里的关键架构相关可追溯性（§6.4.4.2 e)）；四个活动与其任务（§6.4.4.3 a)1)~5) 准备、b)1) 刻画问题空间

＋NOTE 3、b)2) 定架构目标与关键成功准则、b)3) 综合潜在解＋NOTE 4 偏好解类）；系统需求侧形态（§6.4.3.1 可测且不蕴含特定实现）；双向的必要且充分（§6.4.3.3 NOTE 18）；系统的定义（§3.46, 涌现的出处）；设计（名词）的定义（§3.13, 元素与关系的规格归设计）；架构实体与系统元素是两种不同的概念（§5.9）

- [5] **R-38** ISO 9000:2026 —— 规定的需求是明示的需求（§3.5.1 注 2：A specified requirement is one that is stated）

# 第 12 章 架构决策：在承重处做选择

## 12.1 引言

上一章把“几种可能”摆了出来：几个候选，各自带着一句目的陈述，也各自带着来路。可摆出来只是摆出来。桌上放着几条路，谁也没说走哪条。本章回答的就是这一句：**凭什么定下来的是这一个**。

### 12.1.1 误解现场

恒信合同审批系统，业务方的郑亮提了一个“小需求”。

他把手机屏转过来：“合同编号得改。电子签章平台那边的规则是'合同类型-年份-流水号'，跟咱们库里对不上。帮我们把编号规则改了？”

系统架构师罗建瞄了一眼：“改个编号格式的事，今天就能上。这算什么架构决策。”

路过的产品开发经理曹峰听了半句，没停下脚步：“改个字段而已，测试跑一遍就上，别拖了。”

当天下午，罗建改了合同主表的编号字段，顺手给合同状态枚举加了两项。他在测试环境里跑了一遍，正常，随即上了线。

三个月后，账从三个地方同时找上来。财务对账对不上：编号按新规则重排之后，与已经归档的纸质合同对不上了。台账系统按旧规则解析，把同一份合同认成了三条。法务要查半年前那份作废合同当初是谁批的，历史审批记录的关联键断了，审批链断在中间。

会议室里，罗建还觉得冤：“我就是改了个字段。”

需求分析师顾言说了一句让整间会议室安静下来的话：“你改的不是一个字段。你改的是这套系统的数据模型。”

这一间会议室里坐着四位，他们把“决策”认成了四样别的东西。

管理层那位想的是“这个要不要上会定”，在他那里，决策是一场会。架构师那位想的是“先把图画出来，决策后面再说”，在他那里，决策是图之外的附带物。

开发那位想的是“这个我改个字段就行”，在他那里，决策是一次代码变更。流程与工具那位想的是“决策记录写了吗，这个季度写了几篇”，在他那里，决策是一份文档产出。

四样都不是标准说的那件事：一组选择，加一份说明它的理据。

今天这件事，问题不在“罗建该不该改”。改编号规则这个需求是真的，电子签章平台的规则也是真的。问题在另一个地方：**凭什么定下来的是这一个改法**。桌上只有一条路的时候，“定下来”根本不需要理由，把编号一改就完了；一旦把它摆到别的走法旁边，理由就非给不可：历史数据为什么不动，双轨为什么不走，眼下为什么非动不可。

### 12.1.2 本章要回答什么

读完本章，下面这些问题你应当答得出来：

1. “权衡”在标准里有没有定义句？如果没有，它的定义性表述落在哪个词上？
2. 几条候选摆在一起，“此消彼长”发生在哪几个方向上？
3. 权衡为什么不能承诺“最优解”？权重是不是非有不可？
4. 拿什么判？为什么判据不在解的技术优劣上？
5. 强制项是什么？它跟“可选项”差在哪？
6. 把哪几条列为候选，为什么本身也是一条决策？
7. 筛的标准是什么？看不出差别的两条为什么不必考察？
8. 选定跟拍板是不是同一件事？架构过程能不能替谁拍板？
9. “改起来贵”能用来做什么，不能用来做什么？
10. 排序与时机看哪几样？信息不足的高风险决策该怎么办？
11. 一份记录里必须写什么、应该写什么？“暂不决定”记在哪里？

12. 记下来之后还能不能改？改了要留下什么？

### 12.1.3 本章的骨架

本章要钉的只有两件：权衡与决策。权衡在标准里没有定义句，可用的定义性表述落在 **tradespace**（中译：权衡空间）上，它说的是“与满足架构目标与利益相关方关注点相关的参数、性质与特征的范围与程度”。决策有定义句，在 42010:2022 §5.2.12：a collection of choices made in the overall context of an architecture（中译：在一个架构的整体语境中作出的一组选择）。两件的关系一句话说清：权衡管**能不能把东西分开**，决策管**谁把它定下来**。

本章的骨架是一条链：把候选摆进同一个空间，按问题侧的判据筛，选定并记下来。链上最容易断的地方是这几下：判据拿错，用技术优劣当了判据；筛法用错，要求候选先满足全部关切；记法写错，只留结论、不留理由与署名。

判据只有一条，与上一章同源：判据是目的，不是解的技术优劣，更不是“改起来贵”。

由这条判据，推出本章要守的几件事。第一件关于候选：桌上不能只有一条，只有一条的时候，权衡不存在，决策也退化成确认；摆上桌的几条还要辨得开，在权衡空间上分不出高下的两条，占了位置却不带来信息。

第二件关于代价：代价只管排序、不管准入，“改起来贵”决定什么时候定、定多重，不决定它算不算架构。第三件关于记录：记录不等于拍板（授权出处在治理一侧），也不是一次性的，决策可以改，改要留痕。

四个块各管一段。12.2 管“摆”：把候选摆进同一个空间，拿定判据，认清哪几条不是选项。12.3 管“筛与选”：先筛掉看不出差别的，再处置强制项，最后说清选定与拍板不是一件事。

12.4 管“排序与时机”：什么时候定，信息不足怎么办，划界的理由要不要写下来。12.5 管“记”：记什么，谁署名，能不能改。

## 案例进展①｜恒信：“改个字段”的代价

那晚的回滚只做了一半。

运维王默那晚执行回滚，只回了一半就停了。他把运维记录摊在桌上：“代码能回滚。台账和财务不跟代码一起回滚——它们已经按新编号解析过了，要全回滚，得先把数据迁回去，账重对一遍。这条决策已经生效，回滚是在付第二遍代价。”

“我到现在也没想明白，”罗建说，“我改编号规则之前，把引用这个字段的地方全查了一遍，十二处，都改了。测试也过了。为什么还会炸？按 §12.5.1 那条界线看，我当时把它当代码变更了。”

顾言没直接答，反问：“你改之前，问过一句没有——'合同编号'这四个字，除了代码里那十二处引用，还牵着哪几条关切？”

罗建：“还有谁？台账、财务、电子签章……它们都是对接方，改完通知一声就是了，这不还是一条代码变更？”

“对。它们不在你的代码里，可它们是这份数据模型的下游。”顾言说，“按第 10 章的判据量一遍：拿掉它，台账、财务、法务三家的关联键同时断，系统就达不成”账要能对上“这个目的。承重与否看的是目的，不是代码里改了几处——你只排查了字段层面的十二处引用，把一条架构决策当成了字段级的操作。”

开发组的苏婷负责对接方接口，她插了一句：“合同编号是接口契约里定义得最死的一个字段，台账、财务都拿它当关联键。改它之前，得先看数据归属、下游几家关切挂在这条线上——这是强制项，不是可选项。”

“可那三问里的第二问——下游几家关切挂在哪——我答不上来。”苏婷说，“改之前我只会看接口字段。”

罗建沉默了一会儿，问：“那我怎么知道，一个改动是字段级还是数据模型级？这一档的识别判据在哪？”

顾言说：“数据模型这一层的东西，别的地方改十次也换不来它一次的分量。判断的方法第 12.4 块会讲全，这里先给你一句够用的：**它是架构决策，不是代码变更**——先论证，先评审，先记录，而不是'今天就能上'。”

罗建把这三句抄在本子上。第二天一早，他先去把台账、财务、法务三家下游逐个问了一遍。

这回他先问的是苏婷。苏婷把那张表翻出来：“六问里'影响'那一栏，我填过一次，填完就知道漏了两家。”

## 12.2 权衡：把候选摆进同一个空间

上一章交下来的时候，几个候选是一条一条分开摆的：各有各的目的陈述，各有各的来路。分开摆着，不等于可以比。这一块做的事只有一件——把它们摆进同一个空间，让它们彼此照面。摆到一起才看得见的是“此消彼长”：这一条多要了什么，那一条就得少给什么。能不能比、拿什么比、哪几条根本不该算选项，都在这一节里。

### 12.2.1 候选摆到一起，才看得见此消彼长

上一章摆出来的那几条，各自都过了四项检查，都算候选。可候选是候选，比较是比较。冰链那三条，箱内算、云端算加一条追溯记录、什么都不改，分开看每一条都成立；摆到一起才看得出来，前两条争的其实是同一件事：判断留在现场，还是留在云端。

标准给这件事留了位置，也留了一个词。不过要说明白：**权衡在四份标准里都没有定义句**。检索的结果是：`trade-off` 与 `tradeoff` 在

42010、42020、15288、42030 的第 3 章都不成词条；42020 的关键词表里列了它，可关键词不是定义。标准能给的，是另一个词的句子。tradespace，中译作权衡空间，在 42020:2019 §8.4.6 注 2：

> Tradespace is the range and extent of parameters, properties and characteristics that are relevant in satisfying architecture objectives and stakeholder concerns.（中译：权衡空间是在满足架构目标与利益相关方关注点方面相关的参数、性质与特征的范围与程度。）

同一条注（即 §8.4.6 注 2）往下还有一句，把“常常没有好答案”这件事写死了：The tradespace often does not include a solution that will completely satisfy the architecture objectives and stakeholder concerns.（中译：权衡空间往往并不包含能完全满足架构目标与利益相关方关注点的解。）

若问它由什么构成，同节的注 5 给的是另一句：注 2 交代范围与程度，注 5 交代构成。那句是：The collection of tradeoff considerations, along with the influences and interactions between solutions, is called in this document the “tradespace”.（中译：权衡考虑因素的集合，连同各解之间的影响与相互作用，在本文件中称为“权衡空间”。）

> **术语注**：“权衡”一词没有标准定义句；本书所用它的定义性表述落在 tradespace 上（42020:2019 §8.4.6 注 2 与 §9.4.8 注 3 两处，措辞略有差别，引时照所引条款照录，两处不互相替换）。“权衡空间（tradespace）”本轮入附录 A（T-209）；配套入附录 A 的还有“架构理由（理据）”（T-208）、“强制项（强制项与

> 要求）”（T-210）、“影响半径／信息成本／修改风险”（T-211）、“决策权属方”（T-212）、“未决议题”（T-213）；“架构决策”沿用既有词条 T-74（已按 42010:2022 §5.2.12 改正释文）。

摆到一起之后，此消彼长出现在哪里？标准点了几处：解内与解间，指同一个解的两种做法之间、以及各个解之间；与现状之间；以及与还没被提出来的解之间（42020:2019 §8.4.6 e) 1)~3)）。

最后那一条最容易被漏掉：候选里摆着的那几条都不好，可能只是没人想到另一条。

标准还顺手给了一张典型清单，说“宜考虑”的是这些（42020:2019 §8.4.6 注 4）：

- cost vs performance（成本对性能）
- cost vs schedule（成本对进度）
- weight vs speed（重量对速度）
- accuracy vs timeliness（精度对时效）
- acquisition cost vs operating cost（购置成本对运行成本）
- ease of use vs security（易用性对安全性）
- flexibility vs predictability（灵活性对可预测性）
- agility vs robustness（敏捷性对鲁棒性）
- risk vs reward（风险对回报）

它列完还加了一句“等等”，意思是这张单子并不穷举。

“与现状之间”这一句，标准在别处又单独说过一次（42020:2019 §9.4.7 注 2）。两处都在提醒同一件事：什么都不改，也是一条备选，不能靠“我们没做决策”把它悄悄放过去。至于“怎么比出高下”，标准的态度比很多人以为的要松：**权重不是非有不可**。

42020:2019 §9.4.5 注 2 的原话是 Weights are not absolutely required since this is nearly always a multi-objective problem, and the goal will usually be to present to the stakeholders the tradeoffs between those objectives, not necessarily to mathematically determine the “best” solution.（中译：权重并非绝对必需，因为这几乎总是一个多目标问题，而目标通常是把这些目标之间的权衡摆给利益相关方，而不必在数学上定出“最佳”解。）同节的注 1 另留了一句余地：这件事不必总是定量，也不必总是套用测量标度。

把这几句放在一起，能读出权衡的一条性质：它的目的不是算出最优解，是把“要什么就得少要什么”摆出来让人看见。算不出最优，不等于这件事没做对。

### 12.2.2 判据只有一个：目的

拿什么判?标准把评估准则分成两套：一套叫价值评估准则，一套叫架构分析准则（42020:2019 §9.4.3 注 1）。分两套不是为了好看，是这两套管的事不一样：一套量“值多少”，一套量“是什么、有多大”。

更要紧的分野在目标那一侧。42020 在两处注里把目标分成 ends objectives（中译：目的性目标）与 means objectives（中译：手段性目标）：前者在问题那一侧，说的是“要什么”；后者在解那一侧，说的是“用哪种做法去达成”（42020:2019 §8.4.4 注 2、§8.4.6 h) 注 7）。这两套混在一起比，比出来的就是“哪个方案技术更漂亮”。

判据落在哪一侧，上一章已经定了：判据是目的。关切本身不是判据，它给出的是判据生效的现场；真正拿来筛候选的，仍然是那一句：把这一条内容拿掉，这个实体还能不能达成它的目的。

本章接着用这句，只是对象换了。上一章拿它筛“哪些内容够格进架构”，本章拿它筛“哪一条候选站得住”。

架构目标与关键成功准则，上一章已经立好了（42020:2019 §8.4.4）。架构目标要就利益相关方关切、需求或质量属性，应对对应的问题与机会；关键成功准则则用来评定问题在多大程度上得到解决。本章不重立，只用。

只补一句层标注：**架构目标不是系统目标**。系统目标的主语是系统或产品，写的是用户要的结果，比如对账不过夜、断链必告警；架构目标的主语是这份架构，写的是它要守住什么。两者不同层，不能互抄。

怎么知道自己这一轮的目标立住了？有一个反过来的问法好用：**去掉它，会发生什么？**这句话写得出来，说明它是目标；写不出来，写在纸上的多半只是一句愿望。

本书给的四套示例里，每一条基本概念、基本属性、管控原则都带着“去掉它／不共享的后果”一栏。那一栏就是这条反面表述的位置。

上一章讲过罗建那句“还是流程引擎，我熟”。那句话的毛病不在判断，在位置：它出现在候选还没摆完的时候，后面的评估全成了给这个结论找理由。在找候选那一环用经验，是本事；在挑候选那一环用经验，是把偏好偷渡成理由。

判据在问题侧，还有一条实用的推论：技术上的先进不能当架构上的正确。一项技术选得新、指标好看，回答的是“做得到吗、代价多大”；它回答不了“值不值得”——值不值得，只能回到目的上去问。

### 12.2.3 强制项：不是可选项

桌上摆着候选，还摆着另一类条目。它不是候选。

标准管它叫 mandates and imperatives（中译：强制项与要求）。它从需求那一侧来：42020:2019 §8.3 f) 的原文里写着，这些需求可能包含 relevant mandates and imperatives, including

relevant policies, regulations and standards（中译：相关的强制项与要求，含相关的方针、法规与标准）。

而评估活动的第一件事，就是把它挑出来。§9.4.3 a) 的动作是 Identify relevant mandates and imperatives, including relevant policies and standards.（中译：识别相关的强制项与要求，含相关的方针与标准。）

它和候选的分别只有一条，可这条最要紧：**候选是选项，强制项是给定**。一条候选不满足某个关切，那叫差一截，还可以比；一条候选不满足强制项，那就不是差一截，是要另走一条路。

标准给的处置有几件，摆在一起看很清楚。先是认出谁没满足，§9.4.7 c) 的动作是 Identify where alternatives fail to meet identified mandates and imperatives and propose changes to the architectures (or new alternatives to consider) that would meet these mandates and imperatives.（中译：识别哪些备选没有满足已识别的强制项与要求，并提出能满足它们的架构改动，或新的备选。）

再是减免。42020:2019 §9.4.8 d) 注 4 写得客气但明确：Sometimes the architecture meets the objectives and addresses stakeholder concerns, but does not meet a mandate or imperative. It might be possible to get relief from meeting such mandates or imperatives, so it would be good to examine this possibility.（中译：有时架构满足了目标、也应对了利益相关方关切，却没有满足某项强制项或要求。取得这种强制项或要求的减免或许是可能的，因此宜考察这种可能性。）

最后是早筛，但要看得见差距。同节的注 5 一句话给了两种态度：It is sometimes better to eliminate architecture

alternatives that fail to meet mandates and imperatives earlier in the process to avoid expending considerable resources in the further evaluation of these items. However, it could also be helpful to see how far off these are from meeting the mandates and imperatives.（中译：有时，宜在过程早期就剔掉那些不满足强制项与要求的架构备选，以免在后续评估中耗费大量资源；然而，看看它们离满足这些强制项与要求还差多远，也可能有助益。）

把这几件摆在一起，能读出一条纪律：**“能不能留”和“能不能选”是两件事**。留不留，看它还有没有话要说；选不选，得先把减免、豁免或者把需求放宽的路走一遍。迈不过强制项的那一条可以留在桌上，但不能靠“我会想办法”混过去。

> **术语注**：**强制项与要求（mandates and imperatives）** 本轮入附录 A。它与“关切”不是一回事：关切说“谁在乎什么”，强制项说“这一条不能不做”；一条强制项背后通常站着一个关切，可关切的范围比它宽得多。

### 12.2.4 把谁列为候选，本身是一条决策

摆上桌的几条从哪来，上一章讲过了：上游给的偏好解类、既有解与既有架构、模式战术与启发式。这一节说的不是来路，是另一个动作：**这一轮把谁摆上来，本身就是一条决策**。

它有正本出处。42010:2022 §5.2.12 给“架构决策”举例时，列的选项里有一项是 choice of option sets to be considered（中译：所要考虑的选项集的选择）。选择的范围怎么定，本身就是一次选择；把它定下来，比在后面给三条候选打分要紧得多。范围定窄了，后面评得再仔细，评的也只是同一件事的几种说法。

摆少了怎么办？标准留了一个明写的口子。42020:2019 §9.4.7 a) 注 3 说：sometimes the evaluation activity determines that there is an insufficient number, variety or extent of alternatives that have been predefined and that additional alternatives need to be generated.（中译：有时评估活动会判定，预定义的备选在数量、种类或跨度上不足，需要再生成一些备选。）判定“不够”是评估活动的正当结论，不是失职。

有一类候选必须上桌，标准单独说过：现状照旧。The status quo is sometimes one of the alternatives that could be considered.（中译：现状有时是可被考虑的各备选之一。）（42020:2019 §9.4.7 注 2）

把它放上去不是客气。现状天然是一条不满足关切的候选，否则根本不必改它；正因为它天然带着这一身毛病，它才最能当反证。连它都比不过，这一轮的改动就还没想清楚。

反过来说，也别拿凑数的候选占位置。一条跟别的候选比不出区别、被否的理由也是编出来的候选，将来理据失效、决策必须重作的时候，它拿不回来用。摆放候选是为了让这一轮的选择经得起重作，凑数的候选只会把这件事做假。42010:2022 §6.10.1 注 3 说的正是这件事：被否的备选连同被否的理据，在将来可能很有用。

上一章那四项检查管的正是这一段，过不了检查的叫想法，不叫候选；本章接着往下走一步——过了检查的那几条，怎么摆、摆几条、留不留现状，是这一轮的决策，该写下来。

## 12.3 筛与选：先筛掉，再选定

候选摆进来了，判据也定了，接下来是筛。筛不是打分排序：先剔掉不带来信息的，再处置强制项那一类，最后才谈选定。而“选定”这个词要小心用——它跟拍板不是一件事，这一块最后要把它说清。

### 12.3.1 筛掉看不出差别的候选

筛掉一条候选，最常见的理由是“它比较差”。标准给的第一个理由不是这个。

42020:2019 §9.4.7 b) 的动作是 Eliminate alternatives that are similar to each other and that do not provide a discriminating case with respect to the evaluation objectives and criteria.（中译：剔除彼此相似、且就评估目标与准则而言不能提供区分性情形的备选方案）。同标准 §9.4.8 b) 注 1 说得更硬：Two alternatives that present the exact same tradeoffs are not examined.（中译：呈现完全相同权衡的两个备选，不予考察。）同一个注还补了要做的事：A set of alternatives that “span” the tradespace is also examined.（中译：还会考察一组“跨越”权衡空间的备选。）

把这几句放在一起，筛的界线就清楚了：**筛掉的不是“差的”，是“看不出差别的”**。两条候选在权衡空间上分不出高下，留在桌上不带来信息，还会把评审的时间从“这几条哪个更好”挪到“这条要不要做”上去。

恒信那次桌上摆着三条，彼此辨得开：一条走事实源，一条以审批流为中心，一条什么都不改。三条都留着。

反过来说，如果当时有人把“以合同台账为事实源”写成两条，一条叫“台账为准”，一条叫“台账为准但要能导出”，那两条在权衡空间上分不出高下，就该合成一条，再补一条真不一样的上来。

淘汰的理由也分两类，写下来的时候要分清。“与目的相抵”是一类，某条候选跟这一轮的目的正面冲突，留着它没有意义。“看不出差别”是另一类，它跟别的候选不是好坏之别，而是同一条的两种说法。

两类理由都要落进淘汰理由表。标准要求评估产出里有筛选痕迹与淘汰理由，本书把这两类理由分别记作筛选痕迹与淘汰理由表。

### 12.3.2 强制项：留得下，未必选得中

上一节讲的是强制项是什么。这一节往前走一步：标准把“留”和“选”放在不同的动作里办。

留，是允许的。不满足强制项的候选可以留在桌上，理由在 42020:2019 §9.4.8 d) 注 5 的后半句里：看看它们离满足强制项与要求还差多远，也可能有助益。差距本身是信息——它告诉你这条路的代价具体长在哪里，也告诉你这一轮的强制项卡得有多紧。

选，就另说了。要选一条不满足强制项的候选，得先把“给定”改掉。标准给了三条路。第一条是**减免**，42020:2019 §9.4.8 d) 注 4 说 It might be possible to get relief from meeting such mandates or imperatives, so it would be good to examine this possibility.（中译：获得对满足此类指令或强制要求的豁免或许是可能的，因此宜考察这种可能性。）

第二、三条在 §9.4.10 f) 的注里：If a promising architecture with high stakeholder value does not close (i.e. it does not meet all the requirements) due to a constraint imposed by a requirement, then discuss with relevant stakeholders to determine whether the requirement can be relaxed or whether some other compromise can be reached that allows the promising, high-value architecture to close. This could involve securing a waiver on a requirement or changing the

requirement.（中译：如果一个利益相关方价值很高、很有前途的架构，因某项要求施加的约束而收不了口，也就是没有满足全部要求，那么宜与相关方讨论，确定该项要求能否放宽，或者能否达成某种其他折衷，让这个高价值架构能够收口。这可能涉及就某项要求取得豁免，或更改该要求。）

三条路的共同点是：**它们都不在架构师手里**。放宽需求要利益相关方点头，豁免要有权的人签，改需求要回到需求那一环去走变更。架构师能做的是把差距量清楚、把话说到位，然后把这件事交出去。

由此得一条纪律：**“我会想办法”不是一条路**。把“能不能选”挂在一句“以后再解决”上，等于把强制项从桌上拿走，却不告诉任何人。

两处常见的误用，一个在这条纪律的两边。一边是把不满足强制项的候选直接删掉，连“差多远”都不看，于是连议减免的机会一起丢了；另一边是把不可选的当成可选的比，把还没批的减免默认成已经批了。两边都会让桌上少一条真实的信息。

恒信那边有一条硬约束可以验一验：“留档十年”是法规强制项。要是有条候选把归档压到三年，那它不是“差一点”，它是要走豁免；而豁免这件事，得法务点头，不是评审会上举手表决。

### 12.3.3 选定有时在架构过程之外

筛完了，剩下的是“定哪一条”。这一步最容易看错的地方是：很多人以为它发生在架构活动里面。标准说，不一定。

42020:2019 §5.7 的原话是 Architecture-related decisions could be made in conjunction with the architecture processes, but this is usually the responsibility of some other process where the decision is most relevant.（中译：架构相关决策可以与架构过程一并作出，但这通常是“决策最相关”的那个其他过程的职责。）

同一标准 §8.4.7 注 7 说得更具体：Sometimes the selection of the architecture(s) going forward is made through decisions outside the scope of the architecture processes. In that case, the candidate architectures are presented to decision makers along with an assessment of these architectures. This might entail involvement of the Decision Management Process or the Portfolio Management Process described in ISO/IEC/IEEE 15288.（中译：有时，往哪个架构走的选择是通过架构过程范围之外的决策作出的。此时，候选架构连同对这些架构的评估一起呈给决策者。这可能涉及 ISO/IEC/IEEE 15288 所述的决策管理过程或组合管理过程。）

评估那一侧也是同一个分工。42030:2019 §3.4 注 1 写着：A decision regarding disposition of the architecture is usually outside the scope of an AE effort, although it could be done in conjunction with the AE effort. The AE results are often reported to a decision maker who makes the actual determination of disposition based on those results and sometimes also on other factors (3.8) not considered by the AE effort.（中译：关于架构处置的决策通常不在 AE 即架构评估工作的范围之内，尽管它可以与评估工作一并作出。评估结果常报给决策者，由其依据这些结果，有时还依据评估未考虑的其他因素，作出实际的处置确定。）

那么“定”这件事归谁管？标准把它交给决策管理过程。15288:2023 §6.3.3.1 给了这个过程的目的句：provide a structured, analytical framework for objectively identifying, characterizing, and evaluating a set of alternatives for a decision at any point in the life cycle and select the most

beneficial course of action.（中译：提供一个结构化的分析框架，以便在生存周期任一时点，客观地识别、刻画并评估某个决策的一组备选，并选出最有益的路线。）

同一个过程的注 1 还交代了那些关键数据留下来干什么：Key study data (e.g. assumptions and decision rationale) are typically managed to inform decision-makers, re-justify the decision in the future, and support future decision-making.（中译：关键研究数据，如假设与决策理据，通常被管理起来，用来告知决策者、在将来重新证成该决策，并支撑未来的决策。）

谁有权，本章只点一句，展开归后面两章。42020:2019 §6.2 c) 要求有理由支撑的架构决策 made with appropriate authorities and communicated to stakeholders（中译：由适当的权威作出并传达给相关方）；§6.4.4 e) 给的是升级路径，Elevate other decisions to the appropriate enterprise decision forum（中译：把其他决策上报至适当的企业决策论坛）；而 15288:2023 **§6.3.3.3 a) 1) 及其注 1** 要求决策管理策略里含 roles, responsibilities, accountabilities, and authorities（中译：角色、职责、问责与权限——列这四项的是该项下的**注 1**，条款本体只写“规定决策管理策略”）。

把这些放在一起，能读出一句要紧的话：**评估出“最佳”是判断，处置是决定**。判断可以由架构活动给出，决定得由有权的人作出。两件事分开，才不会出现“评审通过就等于决策生效”这种账。

恒信编号那次，账就是这么错位的。罗建说“今天就能上”的时候，他其实替财务、法务、台账把板拍了。可这条决策的后果落在他们身上，不在他身上：拍板的人不在现场，后果在现场。

### 12.3.4 选定不是没缺点，是认下缺点

选定不是终点。定完还有两样必须留下来：为什么不是别的，和这一条还背着什么。

第一样是淘汰理由。42010:2022 §6.10.1 里有一句是 should 而不是 shall，分量却一点不轻：The AD should record alternative decisions considered and rejected and the rationale for those choices.（中译：架构描述宜记录考虑过并被否掉的备选决策，以及作此取舍的理据。）同一条的注 3 说明了它为什么值：被否的备选连同被否的理据，在将来理据不再适用、决策必须重作的时候，很有用。

这条要求对得上第 8 章评审那三问的第三问：为什么不是它。答不上来，说明淘汰是随手做的，不是判过的。

反过来说，一份写着“为什么不是它”的记录，也是将来重开这条决策时唯一的起点：理据变了，重新摆上桌的还是原来那几条，而不是从头再想一遍。

第二样是残余风险。本书把评估筛选那一步的这份产出记作“残余风险与未决项”。选定不是拿到一个没有缺点的方案，是接受一组已知的缺点。这句话听着消极，其实是这一步最实的收获：知道缺点在哪，才知道后面该盯哪里。

恒信那次要是当时就记下来了，“影响”栏里会写着两句话：一段时间内两套编号并存，报表要兼容；三个月后评估是否并轨。这两句话就是残余风险。缺少它们，后面的人只看到“我们定了双轨”，看不到“为什么要评估并轨”。

还有一类条目不属于这里：那些“这次不决定”的事项。它们不进决策记录，另有去处，后面讲。

## 案例进展②｜冰链：一个“直连”被拦下

冰链那边出了一档子事，正好把这一节的话照了一遍。

一家大型连锁药店订了两千台 M3，提了个要求：我们要自己导出原始温度数据，接进我们的质管系统。云端软件的吴浩第一反应很顺：“简单。给他们的质管系统开个直连接口，把原始数据推过去。”

嵌入式硬件工程师石军听到这句话，拦住了他：“吴浩，你知道这个'简单'改的是 M3 的什么吗？”

“不就是加个数据导出接口吗？不动架构。”

“数据从哪来？”石军问。“追溯平台。数据所有权和接口契约都在那个域里。”

“那这条直连，绕过谁？”

吴浩顿了一下：“绕过……追溯域。传感器到云端再到客户系统，不经过追溯平台。”

“对。你这一'简单'，一条架构决策就做完了，还是高不可逆的那一档。它改的是数据所有权：传感器的原始数据开始直连客户系统，追溯平台对这批数据失去控制。三年后追溯出问题，谁负责？追溯平台看不见、管不着、审计对不上。”

吴浩后背一凉。他忽然意识到，自己刚才准备动手的“加个接口”，是这套系统里最不能反悔的那一档。

“所以这不是接口问题，是一条架构决策；按 §6.10.1 它得记下来，连理据一起。”他把“直连”方案划掉，“客户要导出数据，可以。但要走追溯域的受控导出通道，每次导出留痕。这么一来，导出还是能导出，数据所有权这条线，谁都没动。”

石军点头：“不可逆的决策不是不能做，是要在看清代价之后、用对的方式做。代价这一档，三把尺已经量完了；接下来是记——这条决策的权属方是谁、和哪几条决策有牵连，都要写进记录。”

这一档“最不能反悔”是怎么量出来的，下一块给全。这里先留一句：吴浩改掉的那条路，走的正是“换一条不碰给定项的做法”，而不是“先直连、以后再补管控”。

## 12.4 排序与时机：什么时候做

前三块把候选摆好、筛过，也定了。这一节回答另一个问题：什么时候定。

这个问题要单独讲，是因为它最容易被一个词带偏。贵的东西早定，不贵的东西晚定，听上去天经地义。可“贵”只能决定次序，决定不了资格。

### 12.4.1 “改起来贵”只用来排序

先看这个词在标准里的位置。架构决策的定义句里没有“贵”；“贵”出现在另一个地方：哪些决策要记下来。

42010:2022 §6.10.1 要求架构描述记录必不可少的决策，并且给了一张选择准则的单子，用来判“哪些决策算必不可少（essential），要在 AD 里记录并以理据支撑”。单子上写的是这十三条（主语都还是“决策”）：

- regarding architecturally significant requirements（关于架构显著需求的）
- addressing fundamental concerns (such as performance, evolvability, safety)（应对基本关切的，如性能、可演化性、安全）
- linked to requirement compliance（与需求符合性挂钩的）

- linked to technical standard selection（与技术标准选择挂钩的）
- linked to system vulnerability mitigation（与系统脆弱性缓解挂钩的）
- needing a major investment of effort or time to make, implement or enforce（需要投入大量精力或时间才能作出、实施或执行的）
- that are highly sensitive to changes（对变更高度敏感的）
- that are likely to be costly to change（改起来很可能代价高昂的）
- that result in significant capital expenditures or indirect costs（导致重大资本支出或间接成本的）
- affecting key stakeholders or a number of stakeholders（影响关键利益相关方或一批利益相关方的）
- necessitating intricate or non-obvious reasoning（需要繁复或非显然推理的）
- that form a base for project planning and management (such as work breakdown structure creation, critical chain identification and management quality gate tracking)（构成项目计划与管理基础的，如工作分解结构编制、关键链识别与管理、质量门判定跟踪）
- that result in the replacement of assumptions with known information（导致以已知信息替换假设的）

这张单子判的是“要不要记”，不判“算不算架构”。这是本章要钉死的第一处边界。“贵”在单子上占四行，只说明改起来贵的东西通常值得记下来；它不说明改起来贵的东西就一定算架构。

两件事混起来，会出两种错。一种叫漏判：不贵，但承重的东西被放过。比如“日志必须能被外部审计员独立读取”，改它不难，可它一旦不成立，目的就缺一角。

另一种叫误纳：很贵，但跟目的无关的东西被请进架构。某个框架版本、某套组织惯例技术栈，换起来代价不小，可换不换，这个实体还是不是它？

一句结论：**“贵”能排序，不能当准入判据**。准入这件事第 10 章已经裁过，判据是目的，不是代价。这一节接着用“贵”，只把它用在该用的位置上：排序、时机，以及配多少审查资源。

### 12.4.2 影响半径、信息成本、修改风险

排次序要有依据。这一节给三样。

先交代来历。这三个名字是本书的凝练，不是标准的条目。标准给的是上一节那张单子里关于代价与不可逆的那几行；把那几行展开成可操作的三个量，是本书做的事。所以下面每一条都要标明，这是本书的读法。

> **术语注**：影响半径／信息成本／风险三名为本书凝练（旧稿即用此名，四份正本无对应词条）；本轮入附录 A，逐条标“本书读法”。三把尺量的都是代价，落在排序与时机之内，不作准入判据。

第一把是**影响半径**：改了它，得跟着改多少东西。半径有三个来路。结构上的，看数据归谁、接口给谁用，改一处要不要动别处；时间上的，看能不能渐进迁移，还是必须一次性同步上线；组织上的，看一个团队内部能不能拍，还是要跨团队对齐排期。

第二把是**信息成本**：改它的代价，常常不在动手，而在重新搞清楚应该改成什么样。这一把也有两个来路。一个是当初这个决策靠一条长推演链做出来，改它就得重走一遍；另一个是当初为什么这么定只留在当事人脑子里，人一走，改它先要把它重新发现一遍。

所以说，改一条决策的真实代价，等于重新发现当初为什么这么做，加上实际动手改。多数时候，贵的在前一项。

第三把是**修改风险**：改错了，后果落在哪儿。两个来路：这条决策是不是落在关键路径的交汇点上，比如对账、安全、合规；以及改动涉不涉及已经存在的数据。动历史数据比动新数据贵一档，动多个系统共享的核心数据再贵一档。

三把尺合起来，能给一条决策定一个档：高、中、低。定档的用处不是“决定要不要做”，是“决定什么时候做、做多重”：高的早做、做足分析；中的留出调整空间；低的晚做，错了再改。

拿恒信编号那次量一遍。影响半径大：财务、台账、法务三处下游都要跟，还要三个部门同步排期，而历史数据不能渐进迁移。信息成本高：当初编号为什么这么编，是法务、财务、业务三方谈出来的，没人写过；哪几处引用各承载什么语义，也没人一次说得清。

风险也高：编号压在对账与审计两条关键路径的交汇点上，还要动已归档的历史数据。三把尺全指高。

再拿冰链的温度数据归属量一遍，结论一样。数据被多个系统依赖，半径大；当初为什么分成采集端与上传端，只有石军和吴浩说得清，信息成本高。

改动本身也不轻：要动跑了几年的历史记录，还要说服客户与疾控审计，风险高。

数据归属与数据模型这一类，为什么几乎总是最难反悔的？因为它在这三把尺上同时占满。这也解释了上一章那条“老规矩”为什么值得当众立下来。

定了档，接着就是“什么时候做”。

### 12.4.3 定节奏：早做、晚做与显式推迟

定完档，接着是节奏：这一档决定什么时候做、做多重。

“早做”三个字要先说清，免得被读成“早写厚文档”。标准给的做法恰恰相反。42020:2019 §8.3 b) 里写着：During early stages, it is sometimes important to be agile and quick in conceptualizing many alternative architectures. Some of these early architecture descriptions can be little more than sketches. After doing several quick rounds of evaluation and conceptualization, there can then be a smaller number of viable architectures that are worth capturing in a more complete form and storing in the repository for later use. A more complete form of architecture description would be developed in the Architecture Elaboration process.（中译：在早期阶段，迅速敏捷地概念化许多备选架构有时很重要。其中一些早期架构描述可能只是草图。在完成若干轮快速评估和概念化之后，会剩下数量较少、值得以更完整形式记录并存入库中以备后用的可行架构。更完整的架构描述形式将在架构细化过程中开发。）

这段话说明的是节奏的两头：早期要多、要快、可以糙；越往后越少、越实、越该留档。把两头弄反，就成了开头慢、后面赶。

高不可逆为什么要早做，第 10 章末留下过一个理由，这里接着用：收窄设计空间的收益是即时的，改起来的代价是延后的，而且往

往高得多。收益当场兑现、代价推到以后，于是“再等等”看起来总是划算的，直到不划算的那天到来。

再等等也不是不行，前提是把它写成一件正经事。标准给这类情况留了一个合法去处，不是“算了不管了”，而是未决议题。42010:2022 §6.7 d) 注 2 的原话是 Open issues can lead to decisions to be made.（中译：未决议题可以导致待作的决策。）

这里有一处必须写死（把“未决议题”用作“暂不决定的合法去处”是本书用法；正本只说过“未决问题可以引致待作的决策”）：**“暂不决定”不是决策**。它不进决策记录，它进未决议题。它不占一条决策的位置，也不能拿它当“这件事已经处理过了”。把它记成一条待作决策，与把它当成一句“以后再说”，是两件完全不同的事。

三条一并讲清（推迟要回答的这几句，是本书的做法，不是标准条文）：现在为什么不做，等什么信息，什么时候再评估。三句里少一句，推迟就变成了拖延。

收成一句：**推迟是写下来，拖延是忘掉**。写下来的那三句，就是“还欠一个决定”的凭据。

### 12.4.4 威胁目的，还是我们不熟

定档这一步，最容易被两种情绪污染，而它们都伪装成量出来的结果。

一种是把“不熟”读成“风险高”。这块经验我们没有，于是把它记成高风险、高不可逆，然后要求最重的分析。可“不熟”说明的是取材方式要换，不是它的代价变高了。另一种是把“熟”读成“改起来不难”。我们做过好几回，手顺，于是记成低档、晚做，直到换了人、换了环境，代价才露出来。

上一章末尾给过一句问法，用在划界上；本章把它挪到定档上，问的还是同一句：**这一档，是因为它威胁目的，还是因为我们不熟？**

问完要有动作。前面三样量出来的那个档，后面要跟一句理由；这句理由只可能落在两类里：它威胁的是哪条目的，或者我们缺的是哪块经验。第二类要写成“取材方式要换”，不能写成“所以它更不可逆”。**写不出理由的那一档不算数**，它只是感觉。

恒信编号那次，没有人写过“为什么必须现在定”。于是“今天就能上”成了默认答案，而那条决策的代价是三个月后从三个地方一起找上来的。冰链吴浩那句“简单”，其实是因为他头一回遇到“导出外给”这件事，不熟；而这件事本身确实难反悔。两个理由都成立，但要分开写：一个说取材，一个说代价。混成一句，就分不清该换做法，还是该早做。

## 12.5 记录：一组选择与它的理据

前三块走完，桌上剩一条被选中的候选，档也定了，节奏也定了。这一节讲最后一步：把这条选择写下来，写成别人能接着用的样子。

写下来这件事，标准给了明确的位置。42010:2022 §5.2.12 的第一句是 An architecture decision is a collection of choices made in the overall context of an architecture.（中译：架构决策是在架构总体语境中所做选择的一个集合。）同一条接着说，这些选择通常涉及各类架构描述元素、实体要求，或对架构的环境影响。紧接着第二句给的是**架构理由**（本书行文简称“理据”）：Architecture rationale records explanation, justification or reasoning about architecture decisions.（中译：架构理由记录关于架构决策的解释、论证或推理。）

> **术语注**：架构决策（architecture decision）与架构理由（architecture rationale）的定义句在 42010:2022 **§5.2.12**，属概念章；术语

> 章 §3.1~§3.19 里没有这两个词条。二者是**架构描述元素**（§5.2.9），不是另立的一份文档。

两句并排看，一件事就清楚了：**决策是选择，理据是解释**。一份记录里，这两样各占一半。

这里有一处容易读偏。定义句的前半说的是“在架构的总体语境中”，**语境是架构，宾语另算**；宾语是三类：**架构描述元素、实体要求、环境影响**。

所以决策握住的不是“架构”这个整体，也不只是做法那一类：分层怎么分、归属写成什么规则，是描述元素上的选择；把哪条关切立成强制项、哪条需求放宽或豁免，是实体要求上的选择；法规变了、产线节拍定死了、供应商只有一家，是环境影响上的选择。12.2 那张候选单子上的每一条，都落在这三类里。

一句必须写死的分界：**“必不可少”这个判据标准没给**。§6.10.1 只说记录“被认为必不可少”的决策，把“哪些算必不可少”交给组织和项目自己定准则；标准给的是那张选择准则的单子，它不代替你判断。

### 12.5.1 决策是一组选择，不是一个设计

先说这一节要立的靶子。很多人把“做架构决策”理解成“把方案定下来”，于是决策记录写成了方案说明：接口长什么样、模块怎么分、数据怎么流。写完之后，读者知道了系统长什么样，却不知道当初为什么长成这样、还有哪条路没走。

标准的定义句恰好相反。决策是**一个集合**，一个集合是若干选择攒起来的，不是一锤子。以恒信编号那次为例，事后可以拆出好几个选择：编号由谁生成、状态枚举怎么划分、历史数据怎么迁移、对外接口给不给导出。四件事各有各的选项，也各有各的理据。当初它们

被打包成“改个字段”一句话，事后追的时候，正是这四件分开找回来的。

第二句给的是另一半。架构理由记录的是解释、论证或推理。§5.2.12 顺手列了理据六项，可以拿它当一张检查表：

- 做出决策的依据（the basis for making a decision）
- 对质量属性的影响（impact on quality attributes）
- 所考虑的备选方案与权衡（alternatives and trade-offs considered）
- 决策的潜在后果（potential consequences of the decision）
- 架构原则（architectural principles）
- 附加信息来源的引用（citations to sources of additional information）

这六项里，前四项是“为什么是它”的不同侧面，第五项是它跟哪条规矩不打架，第六项是别人上哪儿去核。合起来一件事：让一个不知道当时情况的人，读完知道为什么这么定。

这里要防一个常见的走形：把决策记录写成设计文档的删节版，只留“是什么”，不留“为什么”。这种记录看着完整，其实信息量极低，因为方案本身在代码和图纸里都有；记录独有的信息，恰恰是那些看不见的部分。

那看不见的部分由谁写、写在哪，标准也有交代。“定”归有权的人，“记”归架构描述：决策通常在架构过程之外作出，一旦进了架构描述，就要连同理据一起记下来。这一条是本章要守住的一条，12.5.4 还要接着说。

再顺手辨一件容易认错的事：**决策不在视图里确立，也不靠视图生效**。视图是架构内容的一个显影面，它能让你看见某条决策落在哪

儿；不在图上的决策照样成立，恒信那四条选择，一条都不在图里。至于这些内容与这些决定在描述里各占哪一处，是第 13 章的活。

### 12.5.2 必须记什么，应该记什么

落笔之前先分清两档，因为标准用的词不一样：shall 是必须，should 是宜。写记录的人分不清这两档，就会把有限的笔力用错地方。

必须记的有两件。第一件来自 §6.10.1 首句：An AD shall record architecture decisions considered essential to the architecture of the entity of interest within the scope and intended purpose of the AD.（中译：架构描述应记录在架构描述的范围和预期目的内、被认为对所关注实体的架构必不可少的架构决策。）第二件来自 §6.10.2：An AD shall include or reference rationale for each essential architecture decision (per 6.10.1).（中译：架构描述应包含或引用每项必不可少的架构决策的理由，按 §6.10.1。）决策要记，理由也要记，两件都是必须。

宜记的有四件，都在 §6.10.2 里：为使用而选择的每个架构视角的理由；每个架构描述框架与架构描述语言的理由；考虑与选择备选方案的证据和理由；架构描述局限性的理由，比如资源不够、时间不够，或已知有别的描述涵盖而省去的部分。

被否的备选另有一句，分量不轻。§6.10.1 第二句说的是：The AD should record alternative decisions considered and rejected and the rationale for those choices.（中译：架构描述宜记录已考虑并否决的备选决策，以及作出这些选择的理由。）这条是 should，可它是本章第 12.3.4 节留下的那个扣：被否的备选连同被否的理由，在将来理据失效、决策非重作不可的时候，是能拿回来用的东西。

同一处的注 3 把这件事说得更直白：Sometimes recording rejected alternatives and their rationale for rejection is useful, e.g. when in the future the rationale no longer applies and the decisions are necessary.（中译：有时记录被否决的备选方案及其否决理由是有用的，例如当该理由在将来不再适用而相关决策成为必要时。）

四件宜记的之外，标准还有一句是要求组织和项目做制度动作的，不是要求某一份记录：The organization and/or project should establish a decision recording and sharing strategy and criteria for selecting important decisions to record and support with rationale in the AD.（中译：组织和／或项目宜建立决策记录与共享策略，以及用于选择在架构描述中记录并以理由支持的重要决策的准则。）

这句话把 12.4.1 那张十三条单子的用处交代清楚了：它管的是“哪些重要决策该记”，不是“一条记录里写什么”。前者由组织定准则，后者才是本节要讲的。两件事在标准文本里是相邻两段，很容易被读成一段。

写什么另有一段。§ 6.10.1 用 When recording decisions, the inclusion of the following information should be considered（中译：记录决策时，宜考虑包含下列信息）引出一张清单，逐步展开时再逐条对照：

- 决策的唯一标识（unique decision identification）
- 对决策的清晰陈述（clear statement of decision）
- 决策权属方或决策责任人的标识（identification of decision authority or owner）

- 影响该决策的约束和假设的标识（identification of constraints and assumptions that influence the decision）
- 将该决策链接到其所涉实体的关注点或方面（link decision to the concerns or aspects of an entity to which it pertains）
- 将该决策链接到受该决策影响的架构描述元素（link decision to AD elements affected by the decision）
- 链接到决策理由（link to decision rationale）
- 与其他决策的关系（relationships to other decisions）
- 该决策的后果（与其他决策相关）得到记录（consequences of the decision (relating to other decisions) are recorded）
- 决策作出、获批和修改的时间戳（timestamps for when the decision occurred, when approved and when modified）
- 附加信息的来源引用（source citations for additional information）

十一项里有三样是旧写法整块缺的：**决策权属方或责任人**、**与其他决策的关系**、**三个时间戳**。这三样恰好各治一条病。第一条治“记下来就算拍板了”——记录不产生授权，谁有权定要单独写。第二条治“一条决策是一个孤立条目”——决策之间有约束、有触发、有细化，那些关系本身就是信息。第三条治“写了就冻结了”——修改的时间戳摆在旁边，意思很明白，它可以改，改了要留痕。

本节的最后一处交代：标准通篇没有给“栏位”，也没给模板。它给的是内容项，说的是“宜考虑包含”。把十一项压成几个问题来问，是本书的做法，不是标准条文。这个压法留到下一节，连同那份记录到底长什么样一起说。

### 12.5.3 一份记录该有的栏位

十一项是标准的内容项，不是栏位表。落到一张纸上，要先把它们压成几个能问出口的问题。

先说压的原则：每一项内容都要有着落，一个都不许丢；压的是问法，不是内容。标准给的是“宜考虑包含什么”，压法由使用者定。

本书压成六问，理由是这样问得出口、答得上来，一份记录一页纸能写完。六问依次是：这条决策叫什么；为什么会有这条决策；当时考虑过哪些候选；定了哪一个，谁定的，什么时候；定了它，影响什么；它有多难反悔，什么情况下要重审。

| 问 | 落点（标准 §6.10.1 十一项里的哪几项） |
| --- | --- |
| 一、这条决策叫什么 | 决策的唯一标识；对决策的清晰陈述 |
| 二、为什么会有这条决策 | 影响该决策的约束和假设；连到其所涉实体的关注点或方面 |
| 三、当时考虑过哪些候选 | 被否的备选与作此取舍的理由（§6.10.1 第二句）；连到决策理由 |
| 四、定了哪一个，谁定的，什么时候 | 决策权属方或决策责任人；决策作出、获批、修改三个时间戳 |
| 五、定了它，影响什么 | 连到受影响的架构描述元素；该决策的后果；与其他决策的关系 |
| 六、它有多难反悔，什么情况下要重审 | 决策理由（§6.10.2）；附加信息的来源引用 |

六问里，第一问与第四问最容易被省。第一问看着多余，其实它是**索引**：没有唯一标识，别的决策就没法指它、项目计划也没法引它。第四问最容易漏掉的是“谁定的”，记录本身不产生授权，这一栏空的决策，等于没人认领。

第四问还牵着另一头：**一条记录写下来，不等于它已经生效**。标准的做法是先定这份描述给谁看、按什么形式写，再捕获决策，最后才把整份描述开发出来（42020 §8.4.8）；也就是说，记录往往比完整的描述先出生。可它要等到进了描述、进了基线，“谁定的”那一栏有人署名，才对下游作数（42010 §6.10.1）。

第三问与第五问是一对。第三问答的是“当初摆在桌上的是什么”，第五问答的是“它牵连了什么”。两条都写下来，这条决策才不是孤零零一条：往前接得上被否的路，往后接得上受它影响的东西。

第六问要把两样东西并排写。一样是不可逆性的档（第 12.4 块量出来的高、中、低），一样是理据落在哪几处：依据、对质量属性的影响、考虑过的权衡、潜在后果、它守住哪条架构原则、出处。§6.10.2 要求的是“每项必不可少的决策都要有理由”，这一栏就是理由的家。

关于理据，还有一句要说清。42010:2022 §5.2.12 与 §5.2.9 把决策和理据都算作架构描述元素，对一条决策来说，理据与决策本身是两样并排的内容。§6.10.1 的注 1 把这件事讲得很短：An AD expresses decisions about an architecture. Architecture rationale expresses why decisions have been chosen.（中译：架构描述表述关于某一架构的决策；架构理由表述决策为何被选定。）

所以记录该长什么样，答案是一页纸、六问、一行一条，不是一份模板，也不是一本册子。写厚了没人读，写薄了下一任读不懂。它的目标只有一个：让一个没参与的人，读完能接着往下做决定。

最后回到价值本身。王默那句话点在这里：**代码能回滚，台账、财务不跟代码一起回滚**。决策记录降的是信息恢复成本，也就是将来要重新发现“当初为什么这么定”的那笔开销。这笔开销省下来，后面的人才接得上手。

### 12.5.4 认知反转

回到章首那间工位。郑亮提的是“小需求”，曹峰路过撂下一句“改个字段而已”，罗建当天改完当天上线。那句话里错的不只是一个判断，是他对“做决策”这三个字的理解。

他理解的决策是一个动作：这件事当场拍定，拍在提交代码那一刻。至于桌上摆过什么、为什么否掉那几条、谁有权定，都不算决策的一部分。

本章从头到尾在做的事，是把这三个字重新拆开。拆到最后，他错过的是四样东西。

第一样，编号怎么编、状态怎么分、历史数据怎么迁、接口给不给导出，是四条独立的选择，各有各的候选与理由。他打包成“改个字段”一句，四个决定一起做掉，还一个都没写。

第二样，他记了选了什么，没记为什么是它。于是系统长什么样他说得清，当初为什么长成这样他说不清。

第三样，“定下来”跟“被批准”是两件事，中间隔着决策权。他当天就上了，因为没人告诉他这件事要谁点头。

第四样最深：**记录不是一次定死**——这条决策往后会被修改、被复审、会牵动别的决策，所以标准在记录里放了三个时间戳，其中一个专记修改。

这四样是同一个判断的四个侧面：**决策不是拍板那一下**；它是“凭什么把它定下来”的一整套，而这一整套写下来，才算存在过。没写下来，它也发生过，只是没人能再找到它。

对“架构决策”，你原来只是见过：背得出定义、知道“要提前投资”这个说法。本章把它推到拥有：能答全四问——它是什么、不是什么、从哪来、往哪去——**能答全四问才算拥有**。

用第 1 章 §1.4.4 那把尺子量一下：本章把“架构决策”推到了**第③级（心智模型）**——判断一次改动时，先问它是不是一组选择，而不是一个动作；再问它有多难反悔、什么时候该重审。

第 10 章用承重测试回答“什么算架构”；本章回答“架构凭什么被定下来”——判据是目的，代价只管排序。

决定做出来，也写下来了，可系统还在变。谁来保证当初那条决定不被悄悄改掉、不被版本搞乱、不会在换人之后失传？

本章说“凭什么定下来的是这一个”。定下来之后还没完：先得把它表达成一份交得出去的描述（第 13 章），再把这份描述交出去、接住下游设计的依从（第 14 章）；至于怎么让它一直说得清——基线把已批准的决定固定下来作为参照，版本记下每个决策的演进，变更控制守住基线的门——那是第 15 章配置管理的事。

## 案例进展③｜恒信：决策被写下来之后

三个月后那次复盘会上，王默没有先看事故本身，先看的是那次改动的记录。

记录里只有一行：合同编号规则调整，当天上线。没有唯一标识，没有约束和假设，没有被否的备选，没有谁批的。

王默把财务和台账的两位负责人也叫来了。五个人对着同一份代码，一句一句往回追。

编号当初为什么这么编，是法务、财务、业务三方谈出来的。状态枚举为什么这几个值，是签章流程倒推出来的。历史数据为什么不能渐进迁移，因为财务台账按编号归档。

追到第三个小时，罗建停下来说了一句：“我一直以为我们做的是一个决定。现在看，是四件，而且一件都没写。标准说架构决策是一个

集合——编号由谁生成、状态枚举怎么划分、历史数据怎么迁移、对外接口给不给导出，我当成一个动作做掉了。”

顾言把这句话接了过去：“你现在把'改字段'当成'做决策'了。改字段是一个动作，做决策是一串选择加一份理据，还得说得出为什么是这几个。”

王默补了一句更实际的：“这些选择当初要是按六问写在一页纸上，今天的三个小时就不用五个人一起花。标准要求每项必不可少的决策都连理由一起记（§6.10.2）。理由在，将来重开这条决策的人才不用从头再想一遍。改一条决策的代价，一半在动手改，一半在重新搞清楚当初为什么这么定，记录省的就是后一半。”

会上定了一条：以后凡是被判定为架构决策的改动，先在恒信自己的架构决策台账上建一条记录；六问写不全的，不进评审。

这条新规矩有一头它够不着。它管的是**记录写没写全**，不管**这条规则谁有权改**——所以这道关，恒信自己也能撤。让规则不能一个人说了算，是另一件事，归第 16 章。

## 小结

本章做了一件事，把“架构决策”这四个字说清楚，并让它能落地：认得出来、排得出先后、记得下来。

第一块讲凭什么比。几条候选要摆进同一个权衡空间，才谈得上比。比较的判据只有一条：目的。谁承重、谁只是贵，由目的说了算，代价不参与准入。强制项是给定，不是可选项——它不许被绕过，但“留得下”与“选得中”是两件事。把谁列为候选，本身也是一条决策。

第二块讲怎么定。先筛掉看不出差别的，再在剩下的里选定。淘汰的理由要写进记录，因为它将来还要用。选定也可能发生在架构过程之外：评估出“最佳”是判断，处置是决定，拍板的人往往不在现场。

第三块讲什么时候做。三样量不可逆性：影响半径、信息成本、修改风险。这一档只决定次序，不决定谁有资格进架构。高不可逆的早做，低的晚做，信息不足的显式推迟——“暂不决定”不是决策，它进未决议题。

第四块讲怎么记。标准要求为每项必不可少的决策写下理由；决策记录另有一张内容清单，其中三样最容易漏：谁有权定、它跟别的决策什么关系、以及作出、获批、修改三个时间戳。理据六项、被否备选的取舍理由，都在其中。记录可以改、改了要留痕，它是这条决策在变化里说得清的依据（12.5.4 收这一句）。

拿恒信编号那次往回看一遍，这四块各拦得住一次：摆候选时就能看出数据模型动了四处；筛与选时那条“改个字段”根本排不进去；量不可逆性时三把尺全指高；写记录时那一栏“谁定的”会当场问住他。一次改字段，四道关都能拦住它，前提是这四道关在场。

对“架构决策”这个词，你从见过走到了拥有：手上有辨认的办法、有排次序的尺子、有写下一条决策的六个问题，叫什么、为什么会有它、当时考虑过哪些候选、谁定的、影响了什么、什么时候要重审。这里有一句要留在最后——**架构决策的终点不是“定了”，是“说得清”**：说得清为什么是这一个，说得清谁定的、什么时候定的，也说得清将来什么情况下要重审。

吴浩的追溯平台接口清单上，多了一条：受控导出。数据所有权归哪个域、能不能对外给，以前没人当决策看；本章之后它进清单了——**接口不是想开就开，它是一条要写下来的决定**。

# 附录

## A 本章概念总图

架构决策 = 一个集合，不是一次拍板

定义句：42010:2022 §5.2.12（在架构总体语境中所做选择的一个集合）

宾语三类：架构描述元素 • 实体要求 • 环境影响（语境是架构，宾语另算）

理据 = 与决策并排的一条：解释、论证或推理（同为架构描述元素，§5.2.9）

比（12.2）：

把候选摆进同一个权衡空间（tradespace）

判据只有一条：目的——承重由目的裁，代价不参与准入

强制项是给定，不是可选项（留得下 ≠ 选得中）

把谁列为候选，本身是一条决策

定（12.3）：

先筛掉看不出差别的，再在剩下的里选定

淘汰的理由要留下（将来理据失效时还要用）

评估出"最佳"是判断，处置是决定——拍板常在架构过程之外

什么时候做（12.4）：

三把尺：影响半径 • 信息成本 • 修改风险（本书凝练，只作排序不作准入）

按不可逆性定节奏：高的早做 • 中的留缓冲 • 低的晚做

显式推迟 = 未决议题，不是决策；"暂不决定"不占决策的位置

怎么记（12.5）：

必须记：决策本身 + 每项决策的理由（shall）

宜记：被否的备选与理由 • 视角与框架的选择理由 • 描述局限

十一项内容 → 六问压法（本书做法）：名称 • 缘由 • 候选 • 定者与时间 • 影响 • 反悔难度

最容易漏的三样：谁有权定 • 与其他决策的关系 • 三个时间戳（含修改）

记录不是冻结——它可以改，改了要留痕

## B 本章的能力级

① 表征：能说出架构决策是一个集合、理据与它并排；能报出三把尺的名字；知道记录里有“决策权属方”与三个时间戳两栏。

② 图式：能把几条候选摆进同一个权衡空间并看出谁跟谁不可比；能按三把尺给一条决策定档；能把十一项内容填进六问。

③ 心智模型：判断一次改动时，先问它是不是一组选择而不是一个动作；承认“没写下来”不等于“没发生”，只是没人能再找到它；把记录当成资产，而不是流程的尾巴。——这一步是本章的终点，也是“拥有”与“见过”的分界。

## C 自查 · 这些说法对吗?

1. “架构决策就是拍板那一下，会上定了就完了。”——**错**。决策是一个集合，是若干选择加它们的理据；拍板只是其中一步，而且常常在架构过程之外。
2. “我们没做架构决策，直接改代码。”——**错**。改写数据模型、编号规则、状态枚举，就是在做架构决策；不写下来，它照样发生，只是没人能再找到它。
3. “这个决策贵，所以它一定算架构。”——**错**。贵只能用来排序，不能发身份证。准入由目的裁。
4. “强制项可以留着，将来再说。”——**半对**。不满足强制项的候选**可以留在桌上**，但想选它，得先改掉“给定”——减免、豁免、放宽或改需求，三条路都不在架构师手里。
5. “候选里不该出现满足不了要求的那几条。”——**错**。看看它们差多远本身是信息，它告诉你这条路的代价长在哪里。
6. “越早定越好。”——**不准确**。按不可逆性定节奏：高的早做，低的晚做，一刀切会把分析资源均匀撒掉。

7. “推迟决策就是拖延。”——**错**。显式推迟是把它记成未决议题，并写明等什么信息、何时再评估；拖延是把它忘掉。
8. “记录写全六问就够了。”——**不准确**。六问是本书把标准十一项压出来的问法；决策理由（§6.10.2）另有要求，被否备选的取舍理由也宜留下。
9. “记录写完就冻住了，谁也不能改。”——**错**。记录里就有“修改”的时间戳，意思正是它可以改、改了要留痕。
10. “写记录是为了留个凭证，给审计看的。”——**半对**。它的作用是降低信息恢复成本：让一个没参与的人，读完能接着往下做决定。

## D 练习

1. 挑你最近做过的一次改动，把它拆成“一组选择”：这次改动里其实包含几条独立的选择？各有几个选项？
2. 找出你们系统里一条“数据模型级”的决定，用三把尺（影响半径、信息成本、修改风险）各打一个高、中、低。
3. 找一条“当初不知道为什么要这么定、现在也不敢改”的决定——它大概率是信息成本极高的那一条。把它写下来，先补“背景”。
4. 用六问格式，为你们最近一次重要决策写一页记录。如果你找不到“背景”和“候选”——这本身就是发现。
5. 列出你们系统里五条决策，按不可逆性排序，再看你们投入的审查资源是不是跟着这个排序走。
6. 找一条“信息不足但不可逆性高”的决策，写三句话：现在为什么不做？等什么信息？什么时候再评估？
7. 把你们桌上的候选过一遍：哪几条实际上在权衡空间里分不出差别？把它们合并或删掉，理由写下来。
8. 复盘一次“被动做架构决策”的经历（被事故逼着改的那种）：如果能回到当时，哪一道关拦得住它？

（答案要点见书末附录，与训练教材题卡同源）

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**


- [1] **R-04** 42010:2022 §5.2.9／§5.2.12（决策与理据的定义句；二者是架构描述元素）｜§6.10.1（shall 记录必不可少的决策；被否备选；选择准则十三条；记录内容十一项）｜§6.10.2（理据 shall 一处与 should 四处，共五处）｜§6.7 d) 注 2（未决议题可导致待作的决策）
- [2] **R-05** 42020:2019 §5.7／§8.4.7 注 7（拍板通常在架构过程之外）｜§6.2 c)／§6.4.4 e)（谁有权、何时升级）｜**§6.4.2 k)（建立配置控制管理机制，以管控架构集合中的变更——那条“不让一个人说了算”的机制，它的正本落点在这一条）**｜§8.3（早期快速轮与“值得完整记下来”）｜§8.3 f)（强制项从需求来）｜§8.4.6 注 2 与注 5（tradespace 定义句与构成句；此消彼长）｜§8.4.6 注 4（典型权衡清单）｜§8.4.8 a)l)（先定表达形式与视角，再捕获决策与理据，最后开发描述；“记录先出生”那一句的落点）｜§9.4.3 §9.4.5（两套准则与权重非必需）｜§9.4.3 a)（识别强制项）｜§9.4.5 注 1 与注 2（不必定量；权重并非绝对必需）｜§9.4.7 a) 注 3／b)／c)／注 2（候选不足须再造；剔相似与不辨者；谁没满足；现状是备选之一）｜§9.4.8 b) 注 1／d) 注 4 与注 5（权衡完全相同不予考察；减免；早筛与“差多远”）｜§9.4.10 f) 注（放宽、豁免、改需求）

- [3] **R-28** 42030:2019 §3.4 注 1（评估产出与决策处置的分界）—— 库内件为 CSA 采标转换件，条款号系从原文编号定义还原，**引前宜对一次正版**
- [4] **R-37** ISO/IEC/IEEE 15288:2023 §6.3.3.1（决策管理过程的目的句）与注 1（关键研究数据的三种用途）｜**§6.3.3.3 a) 1) 及其注 1**（决策管理策略含角色、职责、问责与权限；决策类别与优先次序方案——列这四项的是**注 1**）｜§6.4.4.3 e) 4)（记录关键架构决策与理据）｜§6.4.5.3 注 8（跨架构与需求作 trade-off）

**经典文献**

- [5] **R-56** Nygard, “Documenting Architecture Decisions”（2011）—— 架构决策记录（ADR）的实践来源

# 第 13 章 架构描述：把架构定义交出去

## 13.1 引言

上一章把“凭什么定下来的是这一个”办完了：几条候选摆进同一个空间，按目的筛，选定并记下来。可定下来还不等于交得出去。定下来的东西如果只在一摞图里、只在几个人的脑子里，后面的人接不住，评审也无从下手。

本章办的就是这件事：把这份内容与这些决定，做成一件别人能照着做，又查得下去的东西。标准给这件东西起了个名字：**架构描述**。

### 13.1.1 误解现场

恒信合同审批平台的架构评审会，系统架构师罗建是主讲。他准备了一摞图，一共十五张：业务流程、系统分层、部署拓扑、数据流向这几路各若干幅，另有几张临时补的界面草图。

产品开发经理曹峰先问：“数据归属唯一在哪张图上？”

罗建翻了翻，指着数据流向那一幅：“这张。”

运维王默接着问：“一份合同从发起到归档，跑在哪几张图上？中间哪些环节要盯着？”

罗建把三张图并排摊开，讲了两分钟，讲到第三张时自己停了一下。三张图上的泳道名各不相同。

业务方的郑亮问的是第三句：“这版能不能保证月底对账？”

罗建说：“这个得看数据那边怎么落。”

会议室安静了几秒。需求分析师顾言把手里那张纸推过去，问了三个问题：这摞图，每一张是按哪个视角画的？它框定的是谁的什么关切？这条关切又是从哪一条目的来的？

三个问题，一个也没答上。

罗建不是没画。每一张图他都下了功夫。问题在于，他画的是一摞图，交出去的也还是一摞图。

这一天，同一间屋子里，“描述”被认成了四样别的东西。

有人想的是“图上那个数对不对”，描述在他那里就是一张能看的图。也有人按平时的习惯画图，画法在他那里是一套个人习惯。还有人想的是“关切先放放，版本得上”，关切在他那里是可以跳过的一步。更有人想的是“交上来就算落了地”，描述在他那里是一件交差的材料。

四样都不是标准说的那件东西：一份把内容按约定写全、让人读得出同一个意思的信息部件。

今天这件事，问题不在“罗建该不该画图”。图画得不差，需求也是真的。问题在另一个地方：**凭什么说这一份交得出去**。一摞图交出去，评审只能猜；一份描述交出去，评审能查。查的是承诺过的那几样东西有没有到位。

### 13.1.2 本章要回答什么

读完本章，下面这些问题你应当答得出来：

1. 一摞图和一份描述，差在哪一样东西上？
2. 架构描述是什么？它凭什么能签收、能入库、能版本化？
3. 为什么说“架构本身无形”？有形的是哪一份？
4. 视角和视图，一个是规则一个是产物，这条界怎么划？
5. 内容都写全了，为什么还要视图？“同解”是什么意思？
6. 一份合格的视角规格，要写全哪几样？

7. 视角从哪来？现成的用不上时，怎么自己定一个？
8. 图与视图是什么关系？
9. 关切是被谁框定的，又被谁应对？
10. 一份视图自己该带什么？视图之间对不上，该改哪里？
11. 常用的视图有哪几类？每一类写到哪儿为止？
12. 交出去之前，拿什么对账？缺了哪一条不算齐？

上面这些问题合起来答的是一件事，就是架构定义交出去的那一份是什么、凭什么算交得出去。题面问“架构定义的可交付物”，答法分两半。一半是把里面有什么逐件点清（视角、视图、组件、账，加不在视图里的一档）；另一半是逐件给出判据，说清每一样凭什么算到位。清点不是跑题，清点正是“可交付物”这四个字的内容。

题面里“架构定义”四个字取的是宽义，把“定内容”（第 11 章那一步）与“把它表达、记录下来”算作一次工作的两半——42010:2022 §3.1 那一串动词里就有这两个：defining 与 expressing, documenting（中译：定义；表达、成文）。按严格义，本章干的是后半段那件事：把已经立成的内容表达、记录成一份能签收的描述。所以本章交出去的“可交付物”是**架构描述**，不是架构本身（术语见附录 A T-215）。

### 13.1.3 本章的骨架

本章要钉的只有两件：架构描述，以及它里面的规则与产物。描述是工作产品（42010:2022 §3.3：work product used to express an architecture，中译：用来表达某个架构的工作产品），它是唯一能被评审、被验证、被基线化、被交出去的那一样东西。

描述里面的规则叫架构视角（§3.8：一组惯例，定义句见 13.3.1），产物叫架构视图（§3.7：信息部件，定义句见 13.4.1）。两件的关系一句话说清：**视角是规则，视图是产物**。规则先订，产物后造。

本章的骨架是一条链。关切由视角框定，视图按它那个视角的约定把关切应对掉；视图由视图组件组成，组件要么标明模型种类、要么给出图例。链的末端挂着一份账：对应关系、对应方法与已知问题，以及不在任何视图里的那些信息。

链上最容易断的地方是这几下：**把图当描述**（图只是视图组件的一种形态）；先画图、再回头找视角挂上去（生成顺序颠倒）；**把清单当描述**（内容写全了，“怎么读”还没写下来）。

判据只有一条，与前两章同源：描述交得出去，不在它把内容写全了，在它把“怎么读”写成了公开规则，让不同的人、不同的时间读出同一个意思。这条要求不高，但它是硬要求。没有写下来的读法，换个人读就换一个意思。

由这条判据，推出本章要守的几件事。第一件关于规则：视角不是画图习惯，它是一组能写全、能版本化、能复用的约定；正本给它开了一串该写的条目，缺了其中几样，产出的东西就不成其为视图。

第二件关于顺序：先订规则，再造产物。视角可以在任何详细架构工作开始之前先订，这正是它的用处；反过来，先出图、再找某个视角把图挂上去，不合格。

第三件关于账：一份描述交出去之前要能对账。动笔之前手上该有什么，交件之前该交出什么，出现分歧时改哪里。这三样在 13.6 各有一处交代（逐条出处见各行），缺一条不算齐。

## 案例进展①｜恒信：贴不上视角的图

评审会第二天，顾言没开会。他给罗建递了三张白纸，只在页眉各写了一个词：关切、视角、对账。

“你自己填。第一栏写关切：谁要什么，这条关切从哪条目的来。第二栏写视角：图按哪套约定画的，那套约定管什么、不管什么。第三

栏写账：每条关切落到哪张视图上、拿什么去对。填得出来，说明你的图有主；填不出来，说明它们各自为政。这三栏不是表格，是判据。”

罗建坐了一下午。

第一张纸好办。曹峰要月底对账不出岔，郑亮要审批链查得到人，法务要合同档案留得住，审计要出事追得到人。他写了六行，写到第七行停住了——那一行写的是“平台运维可诊断”，可他说不出这条关切从哪条目的来。

第二张纸不好办。他得给每一行关切配一个视角，还要写下这个视角管什么、不管什么。写到一半他忽然发现，自己那摞图里有一张贴不上去。

那张是流程总览。它画得最早：几条泳道横贯全页，每个节点都带编号。可它不对应任何一份约定，只是他为了讲清楚画的一张目录页。

“这张怎么办？”他把纸推给顾言。

顾言看了一眼：“它不假。判它是不是视图，就问三句：它在一份描述里吗，在；它受某个视角治理吗，不受；它的组件标了模型种类、或者给了图例吗，都没有。三句过不去，它是一张目录页。目录页可以留着，但它进不了账。”

第三张纸最难。对账表要一行行对下去：关切对到目标，目标对到准则，准则对到内容，内容对到落在哪张视图，再对到依据的决策。写到第七行，他停住了：“平台运维可诊断”那条关切，落到视图上没有着落。

王默那天正好路过。他指着空着的那一格：“这条我以前提过三次，没人记——我以为说过了就算数。现在它有行了：关切对到目标、目标对到准则、内容对到视图，中间断在哪一环，表上看得见。”

罗建把那张流程总览从“视图”那一摞里抽出来，放到“附录”那一摞。“我以为写全了就算交出去了。”他说，“原来交出去的不是一摞

图，是一份读法。”他心里数出来了：十五张图，能上账的十一张。贴不上的四张里，一张是流程总览，另三张是他凭习惯画的示意图。

## 13.2 一份工作产品，不是架构

上一节那摞图，缺的不是功夫，是身份。它没有说清自己是哪一件东西：是内容，还是承载内容的那份东西；是规则，还是产物；是给自己看的草稿，还是给别人读的件。这一节先把身份立准，再看这份件由什么构成。

### 13.2.1 架构无形，描述有形

架构本身没有形态。它是一份内容。实体在其环境中的基本概念或属性，加上管住这个实体的实现与演化的管控原则。第 10 章把这一层说完了，本章不重讲。要紧的是另一句：这份内容交不出去。它有形态的那一面，叫架构描述。

标准给描述的定义句很短：work product used to express an architecture（中译：用来表达某个架构的工作产品）。三个词都算数。“工作产品”说的是身份。它是一件做出来的东西，可以签收，可以入库，可以版本化，可以被人引用。“用来表达”说的是它与架构的关系。它是表达者，不是被表达的那份内容。“架构”说的是它表达的是**什么**。架构本身无形、描述有形，这两样不许混为一谈。

这条界有一处最实用：能签收的、能入库的、能版本化的，永远是描述，不是架构。评审的对象是它，基线的对象是它，交接的也是它。所以本章后面讲的每一样东西，视角、视图、组件、账，都是在讲这份件里有什么，不是在讲架构本身有什么。

### 13.2.2 一份描述由什么构成

标准给的结构是分层的：一个架构描述包含一个或多个架构视图，而一个架构视角管控其中一个或多个视图。往下一层，视图由视图组件组成，组件受模型种类或图例管辖。

把它拆开，是规则层与产物层。规则层是架构视角：一组惯例，用来创建、解释与使用某个视图。产物层是架构视图，也就是从一组关注点的角度表达某实体之架构的信息部件。视图再往下一层是视图组件：视图的可分离构成部分，可以跨视图复用。

在这条链的最前面，还站着一个词：关切。标准中译作“关注点”（§3.10），也就是对利益相关方而言具有相关性或重要性的事项。它不在这两层里，却是这两层的宾语：视角框定它，视图应对它。前面的章节两次预告过这个词的立户归本章，这里给出。

规则层与产物层之间不是修辞关系。“视角治理视图”在标准里是一条可核的要求：Each architecture view shall adhere to the conventions of its governing architecture viewpoint（中译：每个架构视图应遵循其管控架构视角的约定）。读到“治理”两个字，可以当场问一句：治理谁、按什么治。答案都在这条句子里。

除了规则层与产物层，这份件里还有一样容易被漏掉的东西：账。这笔账分三处记：对应关系清单、对应方法与一致性；标准对这三样各有要求，13.4.4 一样一样说。视图内部还要记下已知问题：未决项、风险、例外，以及偏离模型种类或图例的地方。

最后还有一档不在这两层里：描述可以包含不属于任何视图的信息，标准举了六样例子。这一档留给 13.2.5，那里一样一样说。

### 13.2.3 规则先订，产物后造

标准把这个先后写下来了，不是流程习惯。视角可以先订：An architecture viewpoint can serve as a contract between

architect and other stakeholders… This contract agreement can be made before any detailed architecting is undertaken（中译：架构视角能充当架构师与其他利益相关方之间的契约……此项契约约定能在开展任何详细架构工作之前达成）。这句话的分量在“契约”二字：规则先谈好，后面才不必反复返工。

顺序的另一头是义务：视图必须遵循其管控视角的约定。所以生成的方向是从视角到视图；反过来，先把图画出来、再找一个视角挂上去，不合格。

这里要拦住一个容易走偏的读法：规则先订，不等于“决策做完才准动笔”。架构细化的活动，标准明说“可以按任何被认为适当的顺序执行”；而且架构描述里的视图“能在任一架构过程中创建”，只是更完整、更精细的那一套一般在细化过程里造。更直接的一条是：视图里显现出来的风险与机遇，回头要改架构（标准写的两个动作，见 13.4.5 那条链的末段）。

最直白的一条在生存期那一段：架构描述可以先于或后于架构的建立、更新或变更（42010 §5.3）。这句话的意思是，描述不是等架构定了再补的档案，它也可以先立起来、回头管着架构怎么定。本章就有一个现成的现场：视角是先订下来的规则，架构还没细做，它已经在管事了。

所以第 12 章讲决策、本章讲描述这个次序，是本书的**换手次序**，不是施工令；同一件架构工作里，两章的活来回穿插。

还有一处要并排说清：决策与理据不在视图里，也不需要先在视图里变成图。标准把架构决策与架构理由列为架构描述元素，它们与视图并排待在描述里；而不属于任何视图的那一档（见 13.2.5），明确包含架构理由。第 12 章把决策是什么说完了，本章只给它们在描述里占的位置。

再补一句时间的先后：标准的做法是先定这份描述给谁看、按什么形式写，再捕获决策，最后才把整份描述开发出来（42020 §8.4.8）；也就是说，**决策记录往往比完整的描述先出生**。出生早不等于算数早：它要等到写进这份描述、再进了组织收件的库、“谁定的”那一栏有人署名，下游才必须依据它。

所以顺序上是三句话：**规则先订，产物后造**；订完不是定死，视图里显现的问题回头改架构；决策记录与视图并排，各占一处。

### 13.2.4 为什么非有视图不可

看到这里，有一个质疑该冒出来了：内容不是都写全了吗？如果基本概念、属性、原则都有了，视角与视图不是重复劳动吗？

答案不在“视图更直观”。答案在于内容与读法是两件事。内容可以写全，但“怎么读”还在各人脑子里。同一段文字，五个人读出来五个意思。这不是读者不认真，是自然语言的性质。

标准把这一层点得很准：一个视角的规格，就是为创建、解释、呈现和分析视图而确立约定；它把造视图要用的那套词汇与使用指南，交到创建、解释或使用视图的那些人手上。

换句话说，视角把“怎么读”从各人脑子里搬到了公开规则上。搬上去以后，不同的人、不同的时间读出来是同一个意思，这就是本章说的同解。

有一处替代说法要挡住：把描述的价值说成“使架构可判定”。可判定是同解的结果，不是价值本身。先把读法写成公开规则，判定才谈得上。另一处是“分面加同一”：那说的是描述的组织方式，不是它为什么值得要。

也要说清这条论证的边界：同解管的是**读法**能不能对齐，不管内容对不对。内容对不对，判据在目的那一侧，前两章已经立过。描述这一侧能提供的，是“读得出同一个意思”。

### 13.2.5 不属于任何视图的信息

标准给这一类留了位置：所关注实体的概览、架构原则、跨视图适用的架构模式与架构风格、引用的架构依据（例如领域架构或参考架构）、视图之间的对应关系、以及架构理由。这是一个示例，不是要求，但它指出了一件事：一份合格的描述，是“视图”加上“视图之外那一档”合起来的。

除了上面那一档，还有两样最常被漏掉。这两样都是描述自身的必写项，上面那六样例子里并没有它们。先是**标识与概述**（§6.1）：这份描述是哪一份、什么状态、谁写的、谁评审的、谁批准的、改过几轮。没有它，一份描述进不了库，也说不清是哪一版。

再是**未处理项与不符合项的声明**（§6.2 末两句）：本该处理而没能处理的关注点与相关方，要写明原因，也就是已知的资源限制或其他约束；不符合项要识别出来，并给出理由说明。这一条最容易被当成客套话，其实它决定一份描述能不能被信任：它交代的不是做了什么，是没做什么、以及为什么。

还有一格不在上面那一档里，也要单独说：记录决策时要写下“决策权属方或决策责任人的标识”。这一格不是一个格式要求，它是**治理在描述里的落款位**（本书读法）。当时若根本没有一个有权者存在，这一格就是空的，而且补不回来：事后补上去的名字，不是当时拍板的人。

所以描述里有什么，答案一句话：有规则，有产物，有账，另有一档不在视图里。四样齐了，这份件才交得出去，也才查得下去。

## 13.3 视角：从惯例到规格

上一节把规则层与产物层立住了：规则层是视角，产物层是视图。这一节专门看规则那一层。一组惯例是什么，它管什么用，写到什么程度算写全，从哪来，怎么定出来，又怎么用起来。

### 13.3.1 视角是一组约定

标准给视角的定义句里没有“工作产品”，也没有“信息部件”，给的是一句关于约定的话：set of conventions for the creation, interpretation and use of an architecture view to frame one or more concerns（中译：为创建、解释与使用某个架构视图以框定一项或多项关注点而设立的一组惯例）。这句话里没有属概念，本书照写，不替它补一个标准没写的身份。

它容易与三样东西混。

一是**立场**。标准另有一个词叫利益相关方角度（§3.18）：way of thinking about an entity of interest, especially as it relates to concerns（中译：思考所关注实体的方式，特别是就其与关注点相关而言）。角度是关切的来源，视角是框定关切的规则：一个是“谁怎么看”，一个是“按什么读”。

二是**视图**。视角是规则，视图是产物，上一节已经分开。

三是**画图习惯**。习惯是各人一套、说不全的；视角是一组能写全、能版本化、能复用的约定。二者最大的差别就在“写不写得全”。

视角的惯例记在它自己的规格里。这一层很实用：惯例一旦写进规格，视角就可以有版本、可以入库、可以被别的工作复用。所以这一块后面说的“写全”，说的都是把规格写全。

视角不是凭空划出来的。标准给了一条线索（§5.2.7 注 1）：关切、利益相关方角度与方面体，可以作为视角的组织基础。三样各有

分工：关切就是 13.2.2 立过的那一项；角度是思考实体的方式；方面体是“实体特性的一部分”（§3.9），标准给的例子是功能、结构、信息，它是关切的细化，而这些细化出来的方面体，正是用来确立视角约定的东西。

这里顺带说清一处常见的混用：有些框架把企业架构分成业务、数据、应用、技术几块，那几块是描述分区（本书命名）的维度，不是视角。按标准，视角要有规格；拿一个域名去写规格，写不出来。这一处只点一句，展开在 13.5。

### 13.3.2 视角管什么：框定关切，治理视图

视角要办两件事，两件都是标准写死的。

第一件是**框定关切**：依据标准（§6.6），每条已识别的关切，都应至少由一个架构视角来框定。换句话说，一条关切如果没有任何视角管它，它在这份描述里就没有位置。

第二件是**治理视图**：一个架构视角管控一个或多个架构视图。视角是规则，规则得有人听。它管的就是将要被造出来的那些视图。

由此有三层意义。

一是分离关切，同时保住整体。标准在附录里把这件事说得很直白：把架构描述组织为受视角管控的视图，提供了一种基于利益相关方分离关注点的机制，同时提供对整个实体的整体视图。既要分开看，又要合起来是一件事。这是视角这个装置的全部用意。

二是把读法写进公开规则。这一条 13.2.4 已经立过，不重复。

三是**可复用**：视角不是一次性的，它可以在别的工作里再用。

视角有五个用处（本书归拢；五处出处分见 §6.6 注 3、§5.2.7、§B.2.6 注、§B.2.4、§B.2.8）。

其一，它可以充当契约：架构师与其他利益相关方就记法与表示约定达成一致，而且这件事可以在任何详细架构工作开始之前做，以减少或避免意外。

其二，它给出造视图的词汇。视角规格（记录这套惯例的那份件）提供的关键建模资源，确立的正是构建各视图所用的词汇；用标准的话说，它交出 AD 元素、语法与语义、使用指南与方向。

其三，它让人判断这个视角对当前这个实体合不合用：规格里写清关切、角度与方面体，架构师与其他人凭这几样就能判。

其四，它事先说明自己不适合什么。规格里还要列出这个视角不适用的问题种类。对某些被过度使用的视角来说，这一条最有用。

其五，它给出对应方法：视角或它的模型种类所定义的那一套，能跨视图组件、跨一份描述内的视图，也能跨几份描述使用。

### 13.3.3 一份规格该写什么

标准对视角规格给了一串要求，写全了才算合格。该写的是这几样：与本视角相关的利益相关方角度；由本视角框定的一个或多个关注点；与这些关注点相关的一个或多个方面体；持有这些关注点的已知典型利益相关方；构造视图时使用的模型种类与图例；捕获视图内及视图组件之间关系的对应方法；以及对本视角任何信息来源的引用。

在这串要求之外，还有一条“宜”：规格宜标识用于创建、解释或分析受本视角管控的视图的**视图方法**，以及一个或多个模型种类。另外，每个图例都要为它所记载的视图组件的解释给出指南。

模型种类自己也有规格要求：它所依据的约定（例如语言、记法或建模技术）、相关的视图方法与对应方法、版本标识、信息来源。记法可以有三种走法：给一个元模型、给一个供人填写的模板、或者援引一门已有的建模语言。

标准还附了一份模板，照着填就满足上面的要求：名称、概述、关注点（含不适用的问题种类）、角度、方面体、典型相关方、对应方法、模型种类规格、视图方法、示例、注、来源。这份模板的价值在于它把“写全”变成了一件可照着做的事。

标准对规格还给了三个词，过不了就不算合格：完整、精确、可验证。三个词一起看，意思是这份规格换个人拿去用，应当产得出同一个东西。

这里有一处本书的归并要标明：上面那串要求，标准写的是七项，外加一条“宜”。本书把它们连同名称、出处、版本并作一张表来教；表末那一行“必需的模型、元素、关系、约束与属性”是本书加的，正本里没有，内容出自附录 B.2.9.2（实体、属性、关系与约束）。标注在此，表里不删。

### 13.3.4 视角从哪来：先复用，后自建

先说一句要紧的：正本不规定任何特定的视角。标准明确写了“本文件不要求使用任何特定的架构视角”，也不要求任何特定的视角规格。所以每个项目的那份视角清单，都得自己给出来。

给出来有四条路。

一是标准自带的框架视角集。开放分布式处理的参考模型给了五个视角：企业、信息、计算、工程、技术，而且每个视角都有自己的视角语言；标准附录里还以举例方式列了若干框架的视角集。

二是业界已有的视角目录。标准附录点了名：有的按视图类型分，有的给出四加一的视图模型，有的给出功能、信息、并发、开发、部署、运行这样一套目录，还有社区维护的视角库。这些都可以拿来用。

三是架构描述框架与架构描述语言。一个描述框架的核心内容，本身就是一套预定义的视角库；用框架的人，视角的来路就在框架里。

四是按组织基础自定：回到关切、角度与方面体这三样上去切。标准给的方面体例子是结构、行为、功能、计划（§6.5 EXAMPLE）；角度例子是战略、组织、运行、逻辑、物理、技术（§6.3 EXAMPLE）。

四条路里，前三条是复用，第四条才是自建。所以纪律是：先穷尽复用，再谈自建。

还有一处要挡住：标准附录里“常用的视图”那一份清单，列的是视图，不是视角清单。两者不许混。这些视图各自的治理视角，是另一件事。

### 13.3.5 视角怎么定出来

标准把“识别或开发架构视角”单列为一项活动，下面列了该做的事：基于关注点选择、适配或开发视角与模型种类；识别架构细化信息的预期用户；

建立或识别拟用的架构框架；捕获选择框架、视角、模板、元模型与模型种类的理由；定义拟开发的每个模型与视图的目的和范围；

选择、修改或开发支撑性的建模方法与工具；选择、开发或修改架构框架、建模模板与元模型；以及选择、修改或开发相关的元模型规格与模板。

这一项活动还带三条提醒。其一，相关视角可能已在概念化过程里识别或开发过，所以先检查能不能原样用，还是需要改一改。其二，视角管控将要创建的视图，最好由最熟悉问题性质与那些关注点的人来开发，而且尽可能把它泛化，好用在别处。其三，如果开发规模较大，交给架构使能过程去做，并把它放进架构库，作为组织标准。

把模板的次序用起来，可以照着走：先写关切与“不适用”那一栏；再挂上角度与方面体；再点出典型相关方；再配模型种类与图例；再写视图方法与对应方法；最后补名称、概述、版本与来源。

这一轮做完，交出来的是四样（**本书归拢**）：视角规格（§10.4.3 a)）、模型种类规格与图例、选择理据（§10.4.3 d)）、以及用途范围（§10.4.3 e)）。

最后一句要紧：这一步交出来的是一件**规则**，不是这份描述的产物。规则能先订，这正是 13.2.3 那个次序的用处。

### 13.3.6 视角怎么用

用起来的第一步是纳入：描述应当包含或引用其中所使用的每个视角，每个视角都要有版本标识，而且规格要合乎前面的要求。

第二步是拿它造视图：视图按视角由一个或多个视图组件组成；组件标明其管控的模型种类（如果有），没有模型种类时给出图例。

第三步是查：每个视图都要遵循它那个视角的约定；评估时还要问一句“视图是否与所选视角一致”。这一条是视角这条规则真正落地的地方。规则定了没人查，等于没定。

第四步是记：当视角被选用并应用时，把实际的相关方与这个视角所框定的关注点之间的关联记下来。

第五步是往外走：把视角泛化，交给架构使能过程，放进架构库，作为组织标准。

还有一句边界：视角规格可以单独使用，可以作为架构描述的一部分，也可以作为描述框架或描述语言规格的一部分。

所以“从惯例到规格”这条路，走出来是五步：纳入、造视图、查、记、入库。前三步管这一次，后两步管下一次。

## 13.4 视图：关切怎么被应对

规则那一层讲完了。这一节看产物那一层：视图是什么，它由什么组成，关切是怎么被它一条条应对掉的，一份视图自己该带什么，怎么造出来，造完又怎么用。

### 13.4.1 视图是一个信息部件

标准给视图的定义句是：information part comprising portion of an architecture description（中译：包含架构描述的一部分的信息部件）。属概念是信息部件，不是工作产品。工作产品是描述，视图是描述的一部分。

这句短定义有三处要分开（**本书分法**）。

一是视图不是架构。架构是那份内容，视图是它在描述里的一个零件。标准给的例子最省事：一个信息视图或数据视图，里面装着概念数据模型、数据管理模型、数据访问模型，以及把这些零件连起来的对应关系。

二是视图不是图，下一节专说。

三是视图不是视角。视角是规则，视图是产物；一张视图恰有一个治理它的视角，而一个视角可以管一张以上的视图。

数量上有三条：一份描述包含一个或多个视图；所使用的每个视角至少有一张视图；一个视角管着多张视图时，这些视图描述的是同一个架构。

### 13.4.2 图只是组件的一种形态

视图不是一个整块，它由一个或多个视图组件组成，组件按它的管控视角来组织。组件是视图里可分离的那一部分，而且可以属于不止一张视图。

组件分两路。一路是基于模型的：它标明自己的模型种类，模型种类给出的是这类模型的约定。另一路不基于模型：那就得配一份图例。图例是对约定的非正式记录，用来说明这个组件里的记号怎么读。

这里最要紧的一句是标准明写的：本文件不规定视图组件的形式化程度；有形式化语义与语法的模型式组件能少一些歧义，但非模型式组件同样能有效使用。

标准给的例子很实在：一张功能视图里可以同时放数据流图、控制流图，再放一段说明怎么读这两张图的叙述；流图基于模型，叙述不基于模型，它们都是这个视图的组件。而那张数据流图还能同时是另一个信息安全视图的一部分。

所以有一句话要记住：图是视图组件里最显眼的一种形态，不是视图本身。清单、表格、一段决策记录，同样可以是组件。

### 13.4.3 框定与应对：一次分工

标准用一对动词把这两层的关系说清了：It is used to distinguish the stages of framing concerns by a viewpoint from addressing those concerns in a resulting view. This is analogous to the distinction between “framing a problem” and “solving that problem”.（中译：用它区分“由视角框定关注点”这一阶段与“在所得架构视图中应对那些关注点”这一阶段；这类似于“框定问题”与“解决那个问题”的区分。）

这一对词把责任分干净了：框定是视角的事，应对是视图的事。

接下来是义务，全是标准写死的要求：每个已识别的关注点，应由至少一个视角框定；每个已识别的利益相关方角度、关注点与方面体，应由至少一个视图按其管控视角予以应对；每个视图都要遵循它那个视角的约定。

应对到什么程度，标准也写明了：一个或多个视图组件，要应对其管控视角所框定的全部关注点，并就该视角而言覆盖所关注实体的部分或全部。后一半同样重要：标准允许一个视图有意只覆盖实体的某一部分，有时是依指令，有时是时间与资源有限，有时是这次架构工作的范围本来就窄。

应对得怎么样，要留下可查的痕迹。视图要记录它内部的已知问题：未决项、风险、例外，以及对管控模型种类或图例的偏离。标准还附了一句提醒：未决的问题会引致需要作出的决策，而例外与偏离可以记载为决策的结果与理由。除此之外，还有对应关系、对应方法与一致性这几笔账，下一节一样一样说。

还有一条要记住：一个视图可以应对一条以上的关切。

反过来说，什么样的情形叫“写下来了但没应对”？三种：视图没有治理它的视角；组件不服务于该视角框定的任何关切；视图之间对不上，却没有一条对应关系。三种都是账上的漏洞。

### 13.4.4 一份视图该带什么

标准对每个视图列了该包含或引用什么：标识与补充信息（按组织和项目的规定）、它的管控架构视角的标识、一个或多个视图组件（应对该视角框定的全部关注点，并覆盖所关注实体的部分或全部）、以及该视图内已知问题的记录。另有一条单独的要求：每个视图要有版本标识。

组件那层也有要求：一个视图按它的管控视角由一个或多个组件组成；每个组件要有版本标识；组件要标明它的管控模型种类（如果有），并遵循管控视角的约定；没有管控模型种类时，要给出图例。

模型种类与图例各自要交代的，是同一组内容（§5.2.8）：预期用途、术语、记法及其语法与语义、符号体系。换句话说，一个组件要能被人读懂，先得说清这套记号是干什么用的、每个符号什么意思。

在此之外，描述里还要带齐这几样：一份架构描述元素对应关系清单（每条要标识参与其中的元素）；一份对应方法清单（并记下每一条成立还是被违反）；一份一致性分析（以及任何已知的不一致）。

### 13.4.5 视图怎么造出来

标准把“开发架构的模型与视图”列成一项活动，里面的动作连成一条链。对每个视角，先按它与外部实体的接口和交互定义语境与边界，以此确定这个视角的目的与范围。再识别将要建模的实体种类及其关系。然后修改与所选框架关联的元模型与视图模板。

再识别将要建模的具体实体与它们之间的关系。接着把相关的概念、属性、特征、行为、功能、特性或约束分配到这些实体与关系上。这一步标准注明，它可能已经在概念化过程里做过，这里只是核查并补足细节。再识别指导架构实体演进的原则与准则。再选择、适配或开发模型。最后按已识别的视角，由模型组成视图。

后半段是收尾：识别在视图中已显现的风险与机遇，并酌情修改架构；使各模型与视图相互协调一致；基于它们生成或修改描述；按开发生存周期各阶段的反馈修改描述；最后把描述放进架构库。

这条链（正本 42020 §10.4.4 a)~m) 十三项）在本章归成四步（**本书归拢**）：定型、建模、对齐、成件，与附录 B 的“视图设计四步”同指。其中定型一步的产出物是视角规格，13.3 已经讲透，这里只留一句回指。

造视图还有一样工具：视图方法。标准把它分成三类：构建方法（怎么开始、下一步做什么，或者给出这类视图的模板）、解释方法（利益相关方与其他使用者怎么读懂它）、分析方法（检查、推理、变换、预测、应用和评估）。视图方法要在模型种类、视角、描述语言或描述框架的规格里写清楚。

最后是一条次序纪律（**本书归拢**，正本无此单句）：先订视角规格，再定模型种类与图例，再造模型、组件与视图，最后核对覆盖与对应关系。这个次序不可颠倒。先把图画出来、再回头找视角挂上去，不合格。

### 13.4.6 视图怎么用

视图造出来是给人读的。标准在讲视角那一处已经点明：视角规格交出的约定，是供创建、解释或使用视图的人用的。所以读，是它的第一个用场。

第二个用场是查。四条可以照着做：查视图与所选视角是否一致；查覆盖，看该视角框定的关切有没有全部落到组件上、实体有没有覆盖到；查一致性与对应关系；再拿 13.6 那份自检表逐条过一遍。

第三个用场是拿它办事。

标准附录把架构描述的用途列了一长串，与视图直接相关的有：作为设计与开发活动的依据。作为分析与评估架构备选实现的依据。作为开发与维护文档；支持技术与投资方面的决策并降低风险。作为仿真、系统生成与分析工具的输入；在开发、生产、部署、运行与维护各方之间沟通。

作为采办文件与合同谈判的依据。作为运行与基础设施支持以及配置管理的指南；确立实现对架构符合性的认证准则。作为对外部与组织特定方针的符合性机制；以及全生存周期评审、分析与评估的依据。

第四个用场是改。按反馈修改描述，并让它处于适当的变更控制之下。描述、视图与模型的变化，按配置管理过程来管（42020:2019 §10.4.2 n)）。哪一版被选定并冻结，那是基线的事，归第 15 章。

第五个用场是交：描述放进架构库，协调预期用户的使用，并留下交付与使用记录。

最后一句边界：这份描述交出去，接住它的是另一份描述——设计描述；两者是什么关系、架构对设计起什么作用，下一章说。

## 案例进展②｜冰链：老规矩缺一份规格

嵌入式硬件工程师石军那页“M3 老规矩”，第 11 章写过，抬头一句“从装箱到交付全程可追溯”，底下三条：温度数据归谁管、断链就是失效、升级不许毁掉已出厂的箱子。

硬件组的新人问：“水师傅，这三条我照着做没问题。可'断链'怎么算？探头报一次超限就算，还是要持续多久？”

石军说：“这还用问？断链是事件，不是读数——探头报一次超限只是一个读数，得持续了才算。”

那新人说：“那我按十秒算一次断链，行不行？这页上没写，我只能自己猜。”

石军一愣——心里那条线是三十秒，这页上没写。他写下三条判断，没写过刻度。

产品经理林涛路过，听了半句：“你这不是三条规矩，是一份没写规格的视角。约定得写进规格。标准 §8.1 给规格写了七项，关切、角度、方面体、模型种类与图例、对应方法都在里头；缺一样，别人就复现不出你那三条判断。”

石军没听过“视角规格”，但听懂了。照模板填：名称“接口视角”；关切三条：断链要判得准、出事追得到、升级不弄坏已出厂的箱子；角度照运行、逻辑、物理，方面体填信息。

填到“模型种类与图例”栏，云端软件的吴浩替他补：温度时间序列的记法与符号表，他在用，从没起过名。填到“对应方法”他停住了。制造侧的小周过来补一句：“扩展规则也得让产线查得到，这一条要写进规格。”石军加了进去。

测试季昭从后面经过：“这栏怎么了?”

“对应方法这一栏，我从来没写过。”石军说。

季昭说：“你说给我听就行——温度记录跟装箱单怎么对上，对应方法就写这个。”

石军说：“一批货一个箱号：温度记录按箱号挂到装箱单上。中途换了承运方，箱号不变、记录不断——对应关系就挂在这个号上。”

季昭说：“这些规矩，除了你还有谁按同一套办法查得出来?”他停了一下：“这就是对应方法。规格里专门有这一栏，写的是组件之间、视图之间怎么对上；不写下来，换个人就对不成一样。写下来。”

他写下第一条对应方法，这一栏填完，规格成形了——它是**接口视角规格**，下一步挂进 ADF（架构描述框架）。他抬头说：“原来我一直用的是自己的一套读法——脑子里那条线是三十秒；写进规格，它跟探头上那条线才是同一条。”

## 13.5 常用的几类视图

标准不规定该出哪几类视图，但也不让这件事没有着落。ISO/IEC/IEEE 42020:2019 自带了一份“常用视图”的清单（附录 E.4.2），另有一件更要紧的东西：一个项目该出哪几类，可以从关切反推。这一节先把这两样说清，再一类一类看过去。

### 13.5.1 清单是例子，不是要求

42010 在这件事上很克制，三处都写明了：不要求任何特定的架构视角（§6.6 注 1）、不要求任何特定的视角规格（§8.1 注 2）、不规定视图组件的形式化程度（§6.8 注 2）。前两条 13.3.4 已引，第三条 13.4.2 已引。42020 那边同样克制：附录 E 在正本里标为资料性，那份清单的原话也只是“下面各分条款给出了一些常用的视图”（§E.4.2.1）。

那份清单列了七类：语境、概念、功能、逻辑、物理、技术、组织；给的是常见做法，不是要求。

它们之间不是并列的一排，而是沿两条线摆开：语境、概念、功能、逻辑、物理、技术这一路沿着抽象层级推移，从外部条件，到外部思想，到要做什么，到问题域里那些数据与变换是什么，到落在哪些元素和接口上，再到由哪些技术门类实现。

组织视图不在这条线上，它问的是谁负责、谁有权。并起来看，前面那一路说的是“说得多细”，组织说的是“算谁的”。这两句都是**本书的归拢**，标准只给了那几条的名称与说明。

每一类各问一句话：语境问外面给了什么条件；概念问架构立在哪些思想上；功能问要做什么；逻辑问从用户与业务的角度看，那些数据与变换是什么；物理问落在哪些元素和接口上；技术问由哪些技术门类与接口实现；组织问谁负责、谁有权。

至于一个项目到底该出哪几类，做法是反推：先识别相关方与关切；再让每条关切至少被一个视角框定；每个视角至少有一张视图；每张视图要应对它那个视角框定的全部关切，并记下已知问题；最后反向核一遍：每个角度、每条关切、每个方面体，都要至少被一张视图应对。走完这几步，那份清单是从关切里推出来的，不是抄来的。

最后一句：这几类各自的治理视角，要按 13.3 的办法去定；13.3.4 挡过的那一处混用在这里同样管用。它是视图清单，不是视角清单。

下面几节各管一类。想知道某一类写到哪儿为止、或者它容易怎么错，去它自己那一节。还要交代一句：以下各节里的“定义句”照引正本，“应对的关切、典型组件与模型种类、写到哪儿为止、常见误用”四项都是**本节归拢**，正本 §E.4.2.2~§E.4.2.8 只给了每条的名称与说明（PESTEL、DOTMLPFI 两组例子除外）。

### 13.5.2 语境视图：外部给了什么条件

语境视图处理的是语境因素与驱动因素，标准举的例子是 PESTEL 那一组（政治、经济、社会、技术、环境、法律）方面体，或者 DOTMLPFI 那一组（条令、组织、训练、物资、领导、人员、设施、互操作性）方面体的描述。

它应对的关切是：这个实体处在什么外部条件之下，要看的是法规、市场、技术生态、运行与自然环境、既有系统格局、组织与人的条件。

典型的组件与模型种类有：PESTEL 表、DOTMLPFI 矩阵、外部实体与约束清单、语境图，后者画的是这个实体与它环境中的外部实体及接口。

它写到“外部因素、驱动与被排除的路径”为止，不写外部系统的内部设计。

常见误用有三种：把它写成项目背景介绍，通篇罗列，没有因素与后果的对应；把外部系统的内部构造搬进来；用 DOTMLPFI 却不改读。那一组原本是防务取向，用在民品上，条令要读成运行政策、组织要读成运营与维修单位、物资要读成备件与保障，改读要做，还要把改读写进视角规格。

恒信那边，语境视图上真正管用的不是“互联网加”“数字化”这类词，而是三条能追后果的条件：电子签章平台只认它自己那套编号规则；合同归档要满足档案年限；月底对账要在两小时内对完。

### 13.5.3 概念视图：架构立在哪些思想上

概念视图提供来自架构之外的主要思想或概念；标准说它给出的是 the fundamental view（中译“基本视图”），可以用作进一步架构工作的起点。这里的“基本”说的是“可作起点”，与定义句里量架构内容的那个“基本”不是一回事（见 13.4.6 末那句边界）。

它应对的关切是：驱动这次设计的那些外部思想与概念，包括运行构想、人因概念、领域基本原理、能力概念、任务概念，以及这个产品到底是什么、不是什么这一定位。

典型组件与模型种类：运行概念图、能力清单、任务线程图、概念草图，还可以有叙述型组件。

有一条判据可以拿来定它写到哪儿为止：说“虚像在准无穷远”，属概念；说“用哪一组透镜”，已经落到物理与技术视图。

常见误用三种：把它当成架构总览图，把后面几张的内容先画一遍，结果是同一信息两处画、两处不一致；把广告词当概念，先进、可靠、智能都不是概念；缺定位的二择，例如唯一手段还是辅助手段、必须自建还是可以外购，缺了这一条，下游的保证等级与验收判据无处生根。

冰链 M3 的概念视图上只有两句话：“温度数据归属追溯域，对外只走受控导出”；“断链是事件，不是读数”。后面几张图都是从这两句里推出来的。至于“智能、可靠”那类话，石军一句没写。

### 13.5.4 功能视图：要做什么

功能视图捕获所关注系统的功能。标准在定义句旁边带了一句要紧的限定：功能视图不是规范性要求，但几乎总会被包含在内。功能架构可以说是本质性的、起组织作用的功能结构，也就是系统为达成使命而把抽象化的输入转换为输出的方式。

它应对的关切是：要做什么、由哪几项功能合成、彼此怎么衔接。

典型组件与模型种类：功能分解树、功能流图（数据流与控制流可以各作一个组件）、运行场景时序图、功能与输入输出对照表。

它写到功能组织与变换为止；用哪种器件实现符号生成，不进这一层。

常见误用三种：拿它当符合性判据，而标准明说它不是规范性要求；写成模块清单，按实现件而不是按功能划分；功能树只到第二层就停，下游无从把需求分配到叶节点。

恒信的功能树上，“合同发起—审批—签署—归档—对账”是一条，权限校验是横切在上面的一条。当时有人提议按模块画，曹峰挡了：“我不管它落在哪个模块，我只问一件事从哪进、从哪出。”

### 13.5.5 逻辑视图：业务上，数据与变换是什么

逻辑视图通常是系统功能与留存数据两者的集成模型，它的抽象层级选在“与用户直接相关，而不是与实施相关”；它按问题域中有意义的方式定义数据，而把数据如何表示的定义推迟到其他视图，例如按位置、金额或用户所关注的对象来定义，而不按数据库字段来定义。

它应对的关切是问题域语义下的数据与变换：数据是什么、由谁权威提供、谁能改、谁与谁对齐。

同一件事，在逻辑层与技术层的写法不同。逻辑层写“25 号之后签的合同归下个月”，技术层才写数据库事务与报文格式。这个对照可以当成一条判据（**本书给的一条界**）：写到业务语义为止是逻辑层，写到表示方式就是技术层。

本书把这一层要写的东西压成三问，这三问是本书的操作化提法，不是标准原句。一，是什么：说出输入对象、输出对象，以及这项变换在业务上叫什么；说不出输入输出就等于没定义，用调用某个服务、写某张表来说，已经滑到表示方式。二，在什么条件下成立：写出当什么时做什么、不做什么，以及始终必须如何这两组句子。

三，结果归谁。回答三层归：归属，事实源是谁、谁能改；责任，谁有权定义与变更这条规则；去向，结果给谁用、以什么形态出去。三问缺一，后果各不相同：缺第一问，下游无法分配需求；缺第二问，下

游判不了合规、测试无处挂判据；缺第三问，下游必然各写一份，出事无人认领。

它不写报文格式、数据库字段、接口协议、前端框架。写进去就滑到技术视图了。另有一条限定注要带上：构成逻辑架构或物理架构的内容，往往带有一定主观性，这一谱系上的各类做法都能用。

常见误用三种：把逻辑当成粗一点的物理，这是把粗细换了名字；数据归属不明，出现多个事实源，下游各写一份；把该推迟的东西提前定了，例如在逻辑层写死接口报文，使物理与技术视图只剩下抄写。

恒信那条最典型：“25 号后签的合同归下月”，它是判断，不是算法；谁定的、能不能改、改了谁受影响，都比“用什么表达式写”更要紧。至于它最后落成数据库里哪个字段，那是技术视图的事。

### 13.5.6 物理视图：落在哪些元素上

物理视图是系统元素与物理接口的一种安排，标准说它为产品、服务或企业提供设计解，并旨在满足逻辑架构元素与系统需求。“提供设计解”那几个字是标准原文，照引。但这里要加一条限定：它出自资料性附录，而本书把设计解的归属放在设计定义那一侧。

它应对的关切是：功能与属性落在哪些系统元素上、这些元素之间的物理接口是什么、安装与承载怎么安排。

典型组件与模型种类：物理分解与安装图、接口框图、部署拓扑图、光路与管路与链路的布置图。

界线划在一个地方：型号、尺寸、材料、数量、实例数一出现，就属于设计定义。物理视图写到这道线之前为止。

另有一处要钉住：这里说的系统元素，按标准是硬件、软件、数据、人员、过程、规程、设施、材料以及自然存在的实体，不只有实物件。

常见误用三种：把物理视图读成构件清单，丢掉了分配与接口，也丢了治理它的视角；把技术选型写进来；元素性质认错，只当它是实物。

冰链的物理视图上，箱体、探头、通信模块、平台四样各有位置与接口。写到这儿够了；再往下写选哪一款探头、箱壁多厚，那就是设计定义。

### 13.5.7 技术视图：由哪些技术实现

技术视图按构成系统实施的技术上可识别的对象及它们之间的接口来定义；标准说它能通过技术加以实现。

它应对的关切是：由哪些技术门类的对象构成实施、它们之间的接口约定是什么、哪些技术门类被允许或被排除。

典型组件与模型种类：技术分层图、接口协议表、软硬件分配表、技术约束与禁用清单、环境与保证等级清单。

它只写门类与接口约定；选哪一款器件、哪个版本，属设计定义。

常见误用三种：把技术视图变成选型报告，管到了设计侧；只列技术栈不写接口约定，下游无从集成；与逻辑视图重复，把语义内容又抄一遍。

恒信的技术视图上写的是门类与接口约定：审批走哪一类消息，对账数据走哪一类通道，哪些外部服务允许接入。用哪一家的消息中间件，他没写，那是设计。

### 13.5.8 组织视图：谁负责，谁有权

组织视图表示其他企业视图（过程、信息与资源）中所识别的全部实体上的职责与权限；它把所识别的组织单元归并为部门、分部、科室等更大单元，以此满足企业组织的结构需要。同一处还说：与组织相关的方面体涉及决策层级、职责与权限。

它应对的关切是：谁负责、谁有权；其中包含变更权属，也就是谁定基线、谁批变更、谁签符合性证据。

典型组件与模型种类：职责与权限表、变更权属表、审定与采办路线图、跨主体契约图。

标准这一条的措辞是企业取向的。用到产品上必须改读成全生存周期上的责任主体，覆盖研制、使用与监督三侧。这一层改读要写进视角规格；规格里不交代它，这份规格就不算合规。

常见误用三种：把它当组织架构图，只画汇报关系不写权属；漏掉沉默方（维修、监管、审计、法务、二次开发方，两条线各不同）；不写变更权属，后面的治理与配置管理便无据可依。

冰链这张表上，最容易被漏的是那几个沉默方：审计部门、监管部门、维修方。他们不在会议室里，却是“出了事找谁”的答案。

一类一类走完，回到这一块开头那句话：这份清单是例子，不是要求。真正决定出哪几类的，是你手上那份关切清单。

## 13.6 对账：缺哪一条都不算齐

前面几节把描述里有什么说完了。这一节换一个角度：交出去之前，拿什么对账。两份单子、一张自检表，加一次反转。三样都是本书的归拢。标准给的是散在各处的条款；三样各自的出处，落在各自那一节末尾的一行里。

### 13.6.1 动笔之前，手上该有什么

入口这一侧，要摆齐十样东西。

一，一份关切清单，含相关方角度与方面体。二，相关方清单与他们的持有关系，关注点与相关方之间是多对多的。三，实体及其环

境、它的预期用途、这次工作的范围与深度。四，待表达的架构内容——基本概念或属性与管控原则，这时还没有分配到视图上。

五，可复用的视角与模型种类、图例、视图方法。六，架构描述框架与它的元模型。七，建模模板、元模型规范与工具。八，要建模的实体与关系的种类。九，对账与验证的需要：谁看、看什么。十，上一轮留下的约束与未处理项。

这十样的出处，按上面的序号：一 §6.4／§6.3／§6.5；二 §6.2＋§6.4 注 1；三 §6.1（含“预期用途”那句）／42020 §10.4.3 e)；四 42020 §10.4.4 e)、f)／§3.2；五 §3.8／§8.1（含第二段的“宜”）／§8.2／§8.3；六 §3.5／§7.1＋42020 §10.4.3 c)；七 42020 §10.4.3 f)~h)；八 42020 §10.4.4 b)；九 42020 §10.4.3 b)＋§10.4.1 a)、b)；十 §6.2＋§6.1 注 1。

其中两处要说明白。第二条里，相关方清单出自 §6.2，而“多对多”那一句出自 §6.4 注 1，两句分挂。第十条里，“上一轮留下的约束”对得上 §6.2 的资源限制与约束那一句，而“未处理项”标准没有单立，是本书归拢进来的。这一条既是本单的输入，也是本单要盯的东西。

这十样是入口条件：缺了前四样，动笔就是凭印象；缺了第五到第七样，得先造工具；缺了第十样，上一轮的坑会原样再踩一遍。

### 13.6.2 交件之前，该交出什么

出口这一侧，是十二样。

一，视角规格，写全标准那串条目，外加视图方法。二，模型种类规格与图例。三，架构视图，含标识与补充信息、治理它的视角、视图组件、已知问题，以及版本标识。四，视图组件，含版本与治理它的模型种类；组件可以跨视图复用。五，架构描述元素对应关系清单。

六，对应方法清单，以及每一条成立还是违反的记录。七，一致性分析，以及已知的不一致。八，视角、框架与模板的选择理据。九，视图与模型的用途、范围与深浅。十，视图中的已知问题：未决项、风险、例外、对偏离的说明。十一，描述里不属于任何视图的那些信息。十二，交付与使用记录、反馈回写。

这十二样的出处，按上面的序号：一 §8.1；二 §8.2；三 §6.7（含 a)）；四 §6.8；五 §6.9.2；六 §6.9.3 注 1；七 §6.9.1；八 42020 §10.4.3 d)；九 42020 §10.4.3 e)；十 §6.7 d)；十一 §6.7 EXAMPLE 2；十二 42020 §10.5＋§8.4.10。

另有两样也必须一并交，它们不在上面十二样里，却是描述自身的必填项：一是**标识与概述**（§6.1），二是**未处理项与不符合项的声明**（§6.2 末两句）。后者要写清哪些关切与相关方没能处理、为什么，不符合项还要连理由一起列出来。13.2.5 已经把这两样说全；放在这里，是作为交件账的两行说明。

还有一格要单独提：记录决策时的“决策权属方或决策责任人的标识”（§6.10.1）。它在正本里列的是“宜考虑”的那一组，不是硬要求；可它是账上唯一连着人的一格。治理在描述里落款的地方，定义与后果都见 13.2.5。

这十二样是出口条件：对不上的，不算交件。

### 13.6.3 自检：逐条问一遍

这张表是照着打的：一行一问，过不去就回去补。

1. 每条关切，有没有至少一个视角框定它？
2. 每个相关方角度、每条关切、每个方面体，有没有至少一张视图应对它？
3. 每个视角有没有至少一张视图？这些视图是否遵守该视角的约定？

4. 每个视图写明治理它的视角了吗？有版本标识吗？
5. 每个视图组件写明治理它的模型种类了吗？不基于模型的组件带图例了吗？
6. 对应关系清单在不在？每条参与的元素写明了吗？
7. 对应方法列了吗？每条是否成立、或者把已知的违反都记下来了吗？
8. 已知的不一致与视图内的已知问题记了吗？
9. 视角、框架与模板的选择理据记了吗？
10. 视图的用途、范围与深浅写了吗？有没有说明它是有意只覆盖一部分？
11. 未处理的关切与相关方，写明资源限制或约束了吗？

这张表要跟章末那份自查分开：这十一条是对描述的自检，拿一份现成的东西来量；章末那份是对你自己的自查，问的是概念有没有拿准。两者不许混。

这十一问的出处，按上面的序号：一 §6.6；二 §6.7；三 §6.7 注 1；四 §6.7 a)、b) 与版本句；五 §6.8；六 §6.9.2；七 §6.9.3 注 1；八 §6.9.1 与 §6.7 注 2；九 42020 §10.4.3 d)；十 §6.7 c) 注 3；十一 §6.2 末两句。

十一条全过，描述站得住；有哪一条过不去，回去补哪一条，不必重来。

### 13.6.4 认知反转

本章开头那间会议室里的困惑，到这儿可以点名了。它由四层错位叠成。

第一层，把图当描述。图是视图组件里最显眼的一种形态，交图不等于交描述。

第二层，把清单当描述。清单是内容的表达；描述要把内容按公开规则写成可共同读的件。内容写全了，读法还没写下来。

第三层，把“写全了”当“读得一样”。写全是对自己说的，读得一样是对别人说的，一份描述的价值正在后者。

第四层，把“写下来”当“定死了”。写下来是为了查得下去、改得留痕，不是把话钉死。

把这四层错位翻过来，就是本章立的那几句：规则先订、产物后造；关切由视角框定、由视图应对；一份描述交得出去，靠的是把读法写成公开规则，不是把内容写全。再往前一步：那些账不只是让人查得下去，也是治理落下名字的地方。

回到全书那条线上。第 10 章问“架构是什么”，本章问“架构定义交出去的是哪一份、凭什么算交得出去”。两问合起来，后面那几章要办的事才谈得上：设计依从的是哪一版、基线冻的是哪一份、交接用的是哪一件。

这个反转真正要留下的，不在“学会了几个词”，而在换一副眼光看同一件事：一摞图与一份描述，差的不是精细程度，是**有没有把读法写在纸上**。写下来，别人才接得住；接得住，那些账才查得下去；查得下去，名字才落得下。

## 案例进展③｜恒信：交件前对一遍账

罗建的三张纸填完了，视角清单也补齐了。顾言说：“别急着交。动笔之前手上该有什么、交件之前该交出什么，各有一张单子；一条一条对过去，对不上的不算交件。”

第一张单子对的是手上该有的东西。十条里有两条他手上没有：一条是“上一轮留下的约束与未处理项”，也就是上一版平台留下的两

个未处理接口，他从没在描述里提过；一条是“对账与验证的需要”：谁来查、查什么。他把这两样补进了开头。

第二张单子对的是该交出的东西。十二样里他缺两样：一是标识与概述，一个字没写；二是未处理项声明，那两个接口刚补进上一栏，还没写清“为什么不处理”。

写到这一步，他在记录里翻出一处卡住的：四件决策里有一件是“历史数据怎么迁移”，当时拍板的人不在评审现场。记录里“决策权属方”那一格，是空的。

他问顾言：“决策权属方这一格，我填谁？”

顾言说：“当时是谁拍的板？那一格是决策权属方，写的得是拍板的人。”

罗建想了半天：“当时是曹峰在会上说的……可那不算批，决策记录里没有他的署名。”

“那就空着。”顾言说，“记录里那一栏是决策权属方，写的是谁拍的板——它是治理在描述里落款的地方。当时没有有权者，它就空着；事后补一个名字上去，那不是当时拍板的人，反倒把记录做假了。”

十一问那条自检，罗建逐条过了一遍。第一问卡住了：有两条关切没有落到任何视角上，一条是“平台运维可诊断”，一条是“月底对账怎么算”。前一条的目的他填上了：出了事运维查不出原因，对账与留档的承诺都落不了地。他给它配了运维视角，后一条并进逻辑视图的对账语义里。

开发组的苏婷来拿这份描述，只问一句：“哪几页是我真要照着做的？”顾言说：“标了模型种类的照着做，标不出来的照图例读。标准§6.7 那几条就是为这一问写的。”

交件那天，他把两份单子和自检表一起放进架构库。曹峰问：“怎么还有一条空着的？”

罗建说：“那不是漏。未处理项声明里写的就是这个——本该处理而没能处理的，连原因一起写下来。这一格空着，是当时确实没人定，不是今天没人填。”

## 小结

本章只办了一件事：把交出去的那一份说清楚。

它是工作产品：能签收、能入库、能版本化的，是它，不是架构本身。它的构成是：规则层是视角，产物层是视图与它下面的组件，外加一份账，也就是对应关系、对应方法与已知问题；另有一档不在任何视图里，像标识与概述、未处理项声明，以及决策的记录。

视角那一层讲的是“从惯例到规格”。视角是一组约定，它框定关切、治理视图；一份合格的规格要写全，写全了才能版本化、能入库、能复用。视角从四条路来：标准自带的框架视角集，业界的视角目录，描述框架与描述语言，以及按关切、角度与方面体自定。先穷尽复用，再谈自建。

视图那一层讲的是“关切怎么被应对”。视角框定，视图应对；视图由一个或多个组件组成，组件要么标明模型种类，要么给出图例。图是最显眼的那一种组件，但它不是视图。

常用的视图有七类：语境、概念、功能、逻辑、物理、技术、组织。从语境到技术沿抽象层级推移，组织另立一维。每一类都给了定义、典型组件、写到哪儿为止，以及常见误用。最容易滑出去的两处，是逻辑视图滑到技术层、物理视图滑到设计定义。

交件之前，拿两份单子与一张自检表对账：动笔之前手上该有什么，交件之前该交出什么，十一条哪一条过不去就回去补哪一条。

本章开头那四层错位，翻过来只剩两句要紧的：规则先订、产物后造；关切由视角框定、由视图应对。再往上说一层，描述交得出去，

靠的是把读法写成公开规则，那正是 13.1.3 立的判据；而那些账，既是查得下去的凭据，也是治理落下名字的地方。

恒信那边，罗建最后放进架构库的不是一摞图，是三张纸加一份描述：关切台账、视角清单、对账表。记录里“决策权属方”那一格空着，他没有补：那一格是治理落下名字的地方，当时确实没人定。

后来有人问他，这份描述跟那摞图差在哪。他说：“图是画给自己看的。这一份，是拿来对账的。”

# 附录

## A 本章概念总图

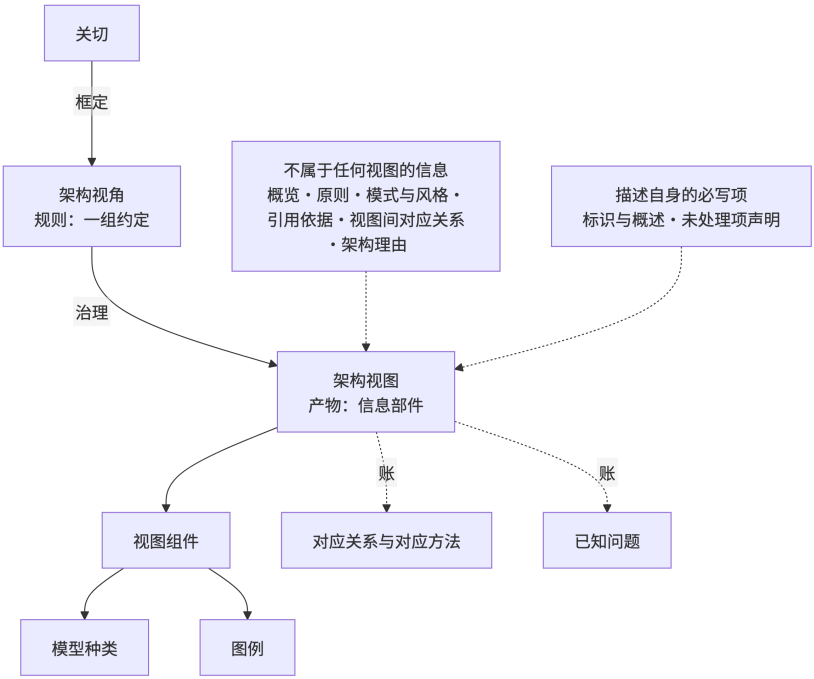



图 13-1

一句话读这幅图：关切由视角框定，视图按视角应对关切；视图由组件组成，组件要么有模型种类、要么有图例；账挂在一旁，视图之外还有一档。

## B 本章的能力级

用第 1 章 §1.4.4 那把尺子量一下：本章以第 ③ 级为主。描述不再是一摞图，而是能推演“读法不写下来，换个人就读出另一个意思”的那份件。

- **表征**：描述侧那八个词——架构描述＝工作产品，架构视角＝一组约定，架构视图＝信息部件，视角规格、视图组件、模型种类、图例、架构描述框架；并把三对近邻分开：架构与架构描述、视角与视图、视图与图。
- **图式**：规则层与产物层的组织链（外加一份账）；“框定到应对”这条机制链；常用视图的分工与推移；视图设计四步。
- **心智模型**：描述的价值在同解——把“怎么读”从各人脑子里写进公开规则；框定与应对是一次分工；规则先订、产物后造，而且这不是瀑布；有账可查，账是治理落下名字的地方。
- **解释框架**：本章不建（归第 17、18 章）。

能答全四问，才算拥有。一份描述交出去，别人读得出同一个意思、查得下去，也追究得到人。

## C 自查 · 这些说法对吗？

1. “架构描述就是那摞图。”——错。图只是视图组件里最显眼的一种形态；描述是信息部件，图进不了账就不算。
2. “架构本身有形态，能签收的是架构。”——错。能签收、能入库、能版本化的是描述。

3. “视角是一种立场。”——错。立场是利益相关方角度；视角是一组惯例。
4. “内容都写全了，视图是重复劳动。”——错。内容与读法是两件事；读法没写下来，五个人读出五个意思。
5. “视角规格写个大意就行。”——错。写全了才能版本化、能复用；缺项产出的东西不成其为视图。
6. “一条关切可以谁也不管。”——错。每条关切至少由一个视角框定，再由至少一张视图应对。
7. “先画图，再找视角挂上去也行。”——错。生成顺序不可颠倒。
8. “视图里的组件必须是模型。”——错。非模型式组件带一份图例即可，标准不规定形式化程度。
9. “功能视图是标准强制要求的。”——错。标准明说它不是规范性要求，但几乎总会被包含。
10. “决策权属方那一格空着，事后补个名字就行。”——错。补上去的名字，不是当时拍板的人。

## D 练习

1. 拿你们最近一份叫“架构”的文档，问三个问题：它在描述里吗？它受某个视角治理吗？它的组件标了模型种类或给了图例吗？
2. 找一份现成的视角规格，逐项数标准要求的那串条目，指出缺哪几项，并说出缺了会怎样。
3. 把一份文档里的图挑出来，逐张判它是视图还是视图组件；再看它能不能同时进另一张视图。
4. 按模板的次序写一份视角规格，只写一页；写完自查一句：换个人拿着它，能不能产出同一张图。
5. 取一张你手上的视图，把它的组件分成“基于模型”与“不基于模型”两堆，给不基于模型的那一堆写图例。

6. 用本章十一条自检量一份现成的描述，把过不去的条目列出来，并标出每一条归哪一步补。
7. 挑一个具体系统，用反推那几步列出它该出哪几类视图；再指出哪一类你们从来没用过，为什么。
8. 翻一翻你们的决策记录，“决策权属方”那一格填的是谁？如果空着，说清为什么空。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022（架构描述）：§3.2、§3.3、§3.4、§3.7（含 EXAMPLE）、§3.8（含注 1~注 3）、§3.9（含 EXAMPLE）、§3.10、§3.14、§3.15、§3.16、§3.18、§3.19（含注 1 与 EXAMPLE）；§5.2.2、§5.2.7（含注 1）、§5.2.8（含 EXAMPLE 2）、§5.2.9、§5.2.10、§5.2.11、§5.2.12；§6.1（含注 1）、§6.2、§6.3、§6.4（含注 1）、§6.5（含 EXAMPLE）、§6.6（含注 1、注 3）、§6.7（含 a)~d)、注 1~注 3 与 EXAMPLE 2）、§6.8（含注 2）、§6.9.1~§6.9.3（含注 1）、§6.10.1、§6.10.2；§7.1、§7.2；§8.1（含注 2 与第二段）、§8.2、§8.3；附录 A（A.1）、附录 B（B.2、B.3）、附录 C.7、附录 D（D.2）。
- [2] **R-05** ISO/IEC/IEEE 42020:2019（架构过程）：§8.4.8~§8.4.10；§10.3（含 e)）、§10.4.1~§10.4.4、§10.4.6（含 c) 5)）、§10.4.7、§10.5；附录 E.4.2（§E.4.2.1~§E.4.2.8）。

- [3] **R-37** ISO/IEC/IEEE 15288:2023（系统生存周期过程）：§3.13、§3.47、§6.4.4.1、§6.4.5。

**经典文献**

- [4] **R-56** Nygard, M. Documenting Architecture Decisions（2011）——决策记录的业界起源，只作线索。

# 第 14 章 架构与设计：不是概要和详细

## 14.1 引言

上一章把“架构描述是什么、怎么交得出去”办完了。视角是规则，视图是产物，两本账加一次对账，缺哪一条都不算齐。本章接的是它留在桌上的最后一句话。这份描述交出去，接住它的是另一份描述，也就是设计描述。这两份描述之间是什么关系？架构对设计，到底起了什么作用？

工程现场对这个问题有一个现成的答案。而且人人都会说：架构是粗一点的设计，我们只是把它做细。本章要办的，就是这一句话。它听着像常识，其实把两件不同的事压成了一件。压错了地方，后面每一步都跟着错。图里塞满大词，设计里长出没人认领的东西，两边都以为对方在管。

### 14.1.1 误解现场

恒信合同平台的系统设计评审，开在架构基线冻结之后。

系统架构师罗建交出了他的架构描述。十五张图那一关已经过去了。这一版是真正对过账的：视角有规格、关切有台账、元素有清单。签字那天他说了一句：“架构这一站，走完了。”

接手的是设计组。开发组的苏婷坐在投影前，讲合同签署链的系统设计描述：配置引擎怎么配、签署链怎么编排、扩展点怎么留。讲到第二十分钟，运维王默抬起头。

“我问一个运维的问题。”王默说，“你的设计里，哪些失效必须被监控、多久检出——这句话在哪儿？”

苏婷翻到监控那一节：“我们设计了监控组件。”

“组件我看见了。”王默说，“我问的是**判据**：漏了什么算漏，多久算迟。你这条设计特性，取值和依据在哪一栏？”

业务方的郑亮接着问：“业务方那句话——'月底账目对得上'。你的设计里，这句话落在哪个元素上？”

苏婷指着事件总线：“编排层保证顺序——**设计元素**层面保证。”

“那是**设计元素**在保证顺序。”郑亮说，“我问的是业务方那句'月底账目对得上'，落在哪个元素上，出了偏差谁发现、按哪条判据发现？”

需求分析师顾言最后问的是罗建：“你这版设计里，新增了一条'与外部对账平台直连'，这条在架构描述的哪张视图上？”

苏婷愣了一下：“这个……是设计细化出来的。”

“细化出来的，”顾言重复了一遍，“还是新长出来的？”

罗建和苏婷对视了一眼，两个人都想到了同一句话。那句话在两家公司都有人说过，而且说得理直气壮：架构不就是粗一点的设计吗？我们只是把它做细。

顾言把笔放下：“这句话我听过十年，它错在两处——第一，架构和设计不是粗细之别，是两类工作；第二，做细的时候长出来的新东西，如果没人认领，它就不在任何人的账上。”

会议室安静了几秒。今天这件事，问题不在苏婷答得对不对，那三问，她每一问都答了一半。问题在她脑子里的那张图上：她把架构和设计摆在同一条线上，架构是上半段、设计是下半段，中间只隔着一个“细”字。只要这条线还在，那三问就都答不上。

### 14.1.2 本章要回答什么

读完本章，下面这些问题你应当答得出来：

1. 架构定义与设计定义，凭什么说是两类工作，而不是同一件事的粗细两版？2. 同一个 design，为什么在标准里有两个意思？这个双义与“架构＝粗一点的设计”是什么关系？
2. 有效的架构为什么要“尽可能不依赖于设计”？这条要求保护的到底是什么？4. 需求能不能推出架构？“必要且充分的输入”与“原料”差在哪儿？
3. 架构交到设计手上，第一步的动作是什么？为什么它不是“把架构发下去”？6. 两条转化加一条分配，转的是哪几样东西？转不动的时候，是设计错了还是架构错了？
4. 设计里长出来的新元素，什么算正常、什么算出事？ 8. 一个视图上的元素，住在哪一层？它表达的是谁？
5. 架构实体只能是实物吗？一个实体只能对一个设计元素吗？10. 视图上的每个图标，都要满足“基本”吗？不必基本，是不是就等于不必相关？
6. 能不能进视图、它表达的是谁、它够不够格当架构内容，这三问，为什么谁也替不了谁？12. 系统分解结构是不是架构的产物？物理视图写到哪儿为止？

这十二问合的是一件事。架构交到设计手上，交出去的是什么、接住的是什么，两者凭什么不是同一件事的两档精度。题面把答案的一半写进了副题，种类不是粒度。剩下的一半在过程里：交出去的是约束，接住的是依从。

### 14.1.3 本章的骨架

**先交代两个反复出现的说法**：本章说“**正本**”，指的是库内那几份标准的原文（ISO/IEC/IEEE 15288:2023、42010:2022、42020:2019）。说“**同版同年旁证**”，指的是 INCOSE 系统工程手册

5.0 中文版——它与 15288:2023 同版同年，可以对着看，但不是标准条文，凡用它都当场标明。

第 10 章说架构是什么。第 11 章说架构怎么被造出来。第 12 章说架构决策凭什么定下来的是这一个。第 13 章说架构怎么表达成一份交得出去的描述。四章合起来，把“架构”这一侧说完了。但架构不是自己活着的，它必须被交给下一段工作。有人拿着它，去定出元素、接口、特性与取值，一直到能拿去采购或实现。这一段工作叫设计定义。

本章按下面五块走：

- 先把这个分界说准，先把名字分清，再看架构要什么、设计要什么，看 design 一词的宽义与窄义；然后回答“凭什么是两类工作”：深浅说不通，独立性要求就是判据；最后落到位置：设计在链上，架构不在链上。
- 交出去什么，接住什么，入口是检查，不是接收；中间是转化与分配；出口是“能拿去采购或实现”；而交接从来是双向的。
- 视图上的元素是谁家的，元素分四层、隔两道界；架构实体不必是实物，也不必逐一对上；系统分解结构不是架构的产物。
- 视图上的元素都要“基本”吗，“基本”管的是谁；“基本”在视角上定，不在图标上；不必基本，不等于不必相关。
- 能不能进，是谁的，够不够格，先问能不能进视图，再问图上的元素属于哪一层，最后问拿掉一条目的还达得成吗；三关各管一段，谁也替不了谁。

走完这五块，本章要立住三句反直觉的话：架构与设计是两类工作，不是同一件事的两档精度。架构管适宜·存续·合意，设计管兼容·可造·可集。两把尺不同轴，所以不能问“哪一张更粗”，只能问“这条信息属于哪一类”。（viability；正本中译作“可行性”，本章取“存续”以与设计侧的“可造”分开）

架构不是基于需求设计出来的。需求是架构的必要且充分的输入，不是它的原料。架构是基于原则、概念与属性定义出来的解，设计才是把架构与需求转换成可实现的系统设计。

架构交出去的那一段，不是接力，是换手。交出去的是约束，接住的是依从。这条通道还是双向的：设计能打回架构，接口可以在设计里新增，基线候选物要交出去。

## 14.2 架构和设计，到底差在哪儿

> 这一块先把名字分清，再看标准给两个过程写的两句目的，最后回答本节的问题：**它们差在哪儿**。答案不是“哪个更细”，而是“分属哪一类”。

### 14.2.1 先把名字分清楚

本章要用的名字有五组，先把它们摆正，后面才不会串。第一组是架构与架构描述。架构是那份内容本身，实体在其环境中的基本概念或属性，加上为实现和演化它的管控原则。架构描述（AD）是承载那份内容的工作产品：能签收、能入库、能版本化的，永远是描述，不是架构。这两样的界线，第 10 章与第 13 章各说了一遍。本章不重复，只借它们做一件事，把“设计”这一侧的名字也对上。

设计这一侧，第一个名字是设计定义（T-164）。它是与架构定义并列的一个过程名，不是“给设计下定义”。它的目的句写着：提供关于系统及其元素的足够详细的数据与信息，以便按照系统需求与架构实现解。

第二个名字是设计描述（design description；附录 A 记作 T-165 系统设计描述），设计定义过程的产物，也就是本章标题里“接住的那一份”。正文里简称“设计描述”，这两个名字一过程一产物，不

能互换。说“设计定义”的时候，说的是那件一直做下去的活动；说“设计描述”的时候，说的是它交出来的那份东西。

第三个名字是设计描述符（T-166，**SEH 表述**）。它是一组设计特性和它们的可能取值。“设计做到够不够细”，最后就落在描述符上：特性列出来了，取值定了，才算这一格填满。

第四个名字是原子元素（T-167），递归往下分到哪一层可以停的判据。它判的不是深度，是信息够不够；怎么判，放在 14.3.5。本章只点到它，因为它是设计定义过程内部的停点判据，不属“架构交出去的那一段”。

五组名字摆开，一件事就清楚了：“架构”与“设计”各自有一份内容、一份描述。把它们放在同一条线上的那个“粗细”，在名字里根本找不到落脚点。

### 14.2.2 “粗一点的设计”，错在哪

“架构就是粗一点的设计，我们只是把它做细。”这句话之所以最容易被信，是因为它有一半是真的。设计的产物确实比架构详细，这一点没错，谁也不必否认。一份系统设计描述比一份架构描述细得多，图上画的、表里列的，都更具体。错在把形状当成了判据。详细只是这件事的形状；真正要问的是：这条内容转过去了没有？把“细”当判据，会顺手推出三句更错的结论：

第一句，“设计方案是架构视图的细化”，标准里给的动词不是“细化”，是转化。把架构实体与关系转化为设计元素，把架构特性转化为设计特性。细化只是把同样一类东西写得更长。转化是把一类东西换成另一类东西，判据不是“细了没有”，是“转没转过去”。

第二句，“视图里画了元素与关系，所以视图就是设计”。视图里的元素是架构实体或被架构实体的表达，不是系统元素。架构只是“可

以关乎”结构，那说的是外延，不是产出。元素及其关系的规格归设计定义过程。

第三句，“设计做完了就不需要架构了”，反了。设计的完成判据本身就引用架构：一份设计够不够，问的是它能不能支撑对架构的一次合规实现。没有架构，“合规”没有对象。

三句话合起来，是同一个错的三种说法：它们都在拿“粗细”回答“种类”的问题。本章后面每一块，做的都是同一件事：换一把尺子问，不问“哪一张更粗”，只问“这条信息属于哪一类”。

### 14.2.3 架构要什么，设计要什么

把标准给两个过程写的两句目的并排放一次，差别比任何解释都直接。

> **系统架构定义过程**：The purpose of the system architecture definition process is to generate system architecture alternatives, select one or more alternative(s) that address stakeholder concerns and system requirements, and express this in consistent views and models.（中译：系统架构定义过程的目的是生成系统架构备选方案，选择一个或多个应对利益相关方关注点和系统需求的备选方案，并在一致的视图与模型中表达该架构。）（ISO/IEC/IEEE 15288:2023 §6.4.4.1）

> **设计定义过程**：The purpose of the design definition process is to provide sufficient detailed data and information about the system and its elements to realise the solution in accordance with the system

> requirements and architecture.（中译：设计定义过程的目的是提供关于系统及其元素的足够详细的数据与信息，以按照系统需求与架构实现解。）（ISO/IEC/IEEE 15288:2023 §6.4.5.1）

两句都在说“做方案”，要的东西却不一样。一个要的是“生成、选择、表达”，另一个要的是“足够详细的数据和信息”。

差别写在另一段里，这一段是本章最该背下来的：

> Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and on feasibility of construction and integration.（中译：架构聚焦于适宜性、可行性与合意性，而设计聚焦于与技术及其他设计元素的兼容性，以及构建与集成的可行性。）（ISO/IEC/IEEE 15288:2023 §5.9）

三处不同，外加一条质量要求。分工不同：架构管适宜·存续·合意，设计管兼容·可造·可集。这是两句不同轴的尺，不是同一把尺的两档刻度。驱动不同，设计定义 is driven by requirements that have been vetted through evaluation with the architecture（中译：由那些已通过结合架构的评估得到审查的需求所驱动；同段 §5.9）。也就是说，进到设计那一步的需求，已经先跟架构对过一遍。产物不同，一个产出架构描述，一个产出系统设计描述。

还有一条质量要求，An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space.（中译：有效的架构尽可能与设计无关，以便在

设计权衡空间中获得最大灵活性。）（同段 §5.9）最后这一条，值得单独看一节，它听着别扭：架构不依赖设计，那它依赖什么？

### 14.2.4 design（设计）：宽义与窄义要分开

有人会拿出另一句话来反驳：标准自己都说过，定义架构属于 design，这话不假，但那个 design 是另一个意思。

design 这个词在标准里有两义：

| 哪一义 | design 指什么 | 出处 |
| --- | --- | --- |
| 宽义 | 定义架构、系统元素、接口及其他特性的过程或结果 | ISO/TS 14812:2022 §3.1.15.1（页面自标源 ISO/IEC/IEEE 24765:2017） |
| 窄义 | 架构定义之外的那一段工作（本章用此义） | ISO/IEC/IEEE 15288:2023 §6.4.5 |

宽义那句定义里，确实明写着 define the architecture，所以“架构是设计的一部分”这句话，在宽义下完全成立。问题出在下一步：有人从这个成立的宽义句，推出了“架构是设计的粗粒度版本”。这一步是偷换，不是推论。

宽义说的是“design 这个词覆盖得很广”，不是“架构和设计是同一种工作的两档精度”。前一句是关于词的，后一句是关于事的。从词的大小，推不出事的关系。记住这个分岔，读标准时就有了一个固定动作，凡见 design，先问一句：这是宽义还是窄义。

### 14.2.5 深浅说不通：独立性要求就是判据

回到那条听着别扭的要求：

> An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space.（中译：有效的架构尽可能与设计无关，以便在设计权衡空间中获得最大灵活性。）（ISO/IEC/IEEE 15288:2023 §5.9）

它依赖目的、关切、需求与原则，唯独不依赖“将来会选哪个设计”。好的架构把约束立在该立的地方，把还没定的事留给设计去权衡。现在看这条要求最要紧的地方：它本身就是那条判据。

同一件事，不会对它自己提出“尽量不依赖自己”。**把这句话代进去念念**：一份草稿不会要求自己“尽量不依赖定稿”。一张图纸的上一版不会要求自己“尽量不依赖下一版”——**只有两样不同的东西，才谈得上“不要黏得太紧”**。

如果架构和设计真是同一件工作的两档精度，“有效的架构尽可能不依赖于设计”这句话就等于说“粗的那一版尽可能不依赖于细的那一版”。这句话没有意义。这条要求能立得住，恰恰因为它说的两样东西不是一回事。

所以，遇到“架构和设计差在粗细吗”这个问题，有一条最省事的答法。看标准有没有提这条要求：提了，就说明不是。还要说清它的边界，它不是“架构可以不管设计”；它是“约束立在该立的地方”。一个逼着设计只能有一种做法的架构，通常不是严谨，是越界。它把本该由设计权衡的事，提前替设计定了。

### 14.2.6 位置不同：设计在链上，架构不在链上

前面说清了“不是粗细”，那么它们各自是什么？正面答案在两段话里。这两段话用的动词不一样。

> The system architecture definition activities **define a solution based on principles, concepts, and properties** logically related to and consistent with each other. The solution architecture has features, properties, and characteristics which satisfy, as far as possible, the problem or opportunity expressed by a set of system requirements …（中译：系统架构定义活动基于彼此在逻辑上相关且一致的原则、概念和属性来定义解。解架构具有若干特征、属性与特性，它们**尽可能满足**……由一组系统需求所表达的问题或机会。）（ISO/IEC/IEEE 15288:2023 §6.4.4.1）

> This process transforms architecture and requirements into a design of the system that can be realised.（中译：本过程将架构与需求转换为可实现的系统设计。）（ISO/IEC/IEEE 15288:2023 §6.4.5.1）

两句对读，得到一组对句：架构是“定义”出来的解；设计是“转换”出来的设计。

- 架构：基于原则、概念和属性定义解；需求在这里扮演的是“问题或机会”的表达者，架构“尽可能满足”它。
- 设计：把架构与需求一起转换成可实现的系统设计。

一个“定义”，一个“转换”，从需求指向架构的那条线上，标准没有放“转换”这个动词。

那需求算什么？标准在别处说得很清楚：

> Confirmation is made that they are a necessary and sufficient response to stakeholder requirements and a necessary and sufficient input to other processes, in particular architecture and design.（中译：确认这些需求是对利益相关方需求的必要且充分的响应，并且是其他过程（尤其是架构过程与设计过程）的必要且充分的输入。）（ISO/IEC/IEEE 15288:2023 §6.4.3.3 注 18）

需求是架构必要且充分的输入，不是它的原料。输入说的是“该给的都给了、够用了”；原料说的是“把它加工成另一个东西”。前者是问题侧的一份完整表述，后者是被转换的对象。把这两样合成一样，就会得出“需求一细，架构就出来了”；而标准从没这么说过。

这一段可以不用引文就验一遍。假如架构真能从需求推出来，那么同一份需求就该推出同一个架构。可工程现场不是这样：同一份需求交给两家公司，做出来的架构可以完全不同，而且两家都合格。

工程里能在同一份输入上做出不同选择、还都合格的，那不是计算，是判断。标准把这一次判断叫“定义”，把“把架构与需求一起转换成可实现的系统设计”叫“转换”。两个词的分工，说的就是这件事。

于是本章要摆的，是同一页上的两种东西：

- 一条链：需求—方案结对，一份的叶节点就是下一份的需求，逐对往下走。这是设计这一侧的形态，第 6 章已经讲透（本章只回指一句，不重讲）。
- 另一件事：目的与关切在前面，架构是基于原则、概念和属性构造并选定的那一次判断。它不在那条链上，也不是那条链的上游一站。

两条之间传递的是约束与相容，不是接力，也不是转换。把它摆成一条，就会得到“架构是粗一点的设计”；摆成两条，这句话自然立不住。

## 14.3 交出去什么，接住什么

> 分界说清之后，看交接本身：从哪一步开始、中间转什么、到哪儿算完、以及这条路能不能往回走。

### 14.3.1 入口是检查，不是接收

标准把“准备设计定义”写成一组任务（ISO/IEC/IEEE 15288:2023 §6.4.5.3 a)）：定义设计定义的策略／确定构成系统的每个系统元素所需的技术／确定设计中表示的系统特性的必要类别／定义设计演进的原则／识别并规划支持设计定义所需的使能系统或服务／获取或取得这些使能系统或服务的使用权。

把这组任务概括成一句话，最到位的是 INCOSE 系统工程手册 5.0 中文版那句（同版同年旁证，它转述的正是这一活动）：

> **检查系统架构**，以确定适用于系统设计的基本属性和概念，以及应指导设计及其演化的原则。

注意这句话的动词：检查，不是“接收”，不是“分发”，更不是“照做”。从架构出发不是一句比喻。它是设计方主动去架构里取三样东西：适用于本层设计的基本概念或属性、指导设计及其演化的原则、以及适用于本层的约束。取不出来的地方，就是交接处的第一个缺口。

这一点，恒信那次评审上说得最直白。罗建交出去的是那份架构描述，厚厚一摞；可设计方真正该拿走的不是那摞纸，是约束。苏婷答

不上王默那三个问题。根子不在她读得不够仔细，在她从来没有“取”这个动作：她以为“发下来了”就等于“接手了”。交文档不算交接，取到约束才算。

### 14.3.2 转化什么，分配什么

取完之后是三件事：两条转化，一条分配。

> 2) Transform architectural entities and relationships into design elements.（中译：将架构实体与关系转换为设计元素。）（ISO/IEC/IEEE 15288:2023 §6.4.5.3 b) 2)）

> 3) Transform architectural characteristics into design characteristics.（中译：将架构特性转换为设计特性。）（同 §6.4.5.3 b) 3)）

两条转化旁边，标准还按了一条注，把“转过去”的样子说清了。each architectural entity (e.g. enterprise or project goals, capabilities and effects, operational activities, resource functions) and relationship is mapped into the appropriate system design elements（中译：每个架构实体和关系都映射到适当的系统设计元素，其中架构实体如企业或项目目标、能力与效果、运行活动、资源功能；同 §6.4.5.3 注 5）。转出来的那一侧，标准也给了例子：functionality, behaviour, dimensions, shapes, materials, critical quality characteristics, data processing structures（中译：功能、行为、尺寸、形状、材料、关键质量特性、数据处理结构等；同 §6.4.5.3 注 6）。

第三件是分配：

> 1) Allocate system requirements to system elements.（中译：将系统需求分配到系统元素。）（同 §6.4.5.3 b) 1)）

它有一条注，值得单独盯住：The purpose of this task is to complete the allocation to the extent necessary to address all system requirements and architecture objectives.（中译：本任务的目的是在应对所有系统需求与架构目标所需的程度上完成分配；同 §6.4.5.3 注 4）

“与架构目标”这几个字不能丢。只对着系统需求做分配，会分出一个“每条需求都有人认领”的设计，却可能整体偏离架构要守的那几件事。对着架构目标分，才会发现“某一条目标找不到承担者”这种病。恒信那次正好是这一种：郑亮问“月底账目对得上落在哪个元素上”，苏婷指了编排层，需求找到了主人。可那条架构目标的承担者是谁、要做到什么程度，谁也没说。

转不动的时候，先判在哪一侧，两种病因要对症：

- 架构侧没立住，那条特性本身不可判定，设计方怎么转都转不动。这时候要回的是架构，不是补设计。
- 设计侧没转，架构给了可判定的东西，设计里却有元素，没有特性落在它身上。这时候补的是设计那一侧。

判据不是“细了没有”，是“转没转过去”。

### 14.3.3 出口是能拿去采购或实现

设计定义到哪儿算做完？标准的答案落在一个动作上：

> 8) Capture the design.（中译：捕获设计。）（同 §6.4.5.3 b) 8)）

> This includes the design description in a form that can be used to either procure or realise the system elements that comprise the system.（中译：这包括以可用于采购或实现构成系统的系统元素的形式给出的设计描述。）（同 §6.4.5.3 注 12）

“以可用于采购或实现的形式”是这道出口的名字，一份设计描述如果停在“这里待定”“取值后续确认”，它就没过这道出口。它不叫“做得还不够细”，叫还没做完。

> 术语注：本节标题取正本原话（“以可用于采购或实现的形式”）；ISO 侧另有把这处出口叫“开发为规范”的表述，出自 **INCOSE SEH 5.0**（同版同年旁证）——**正本 15288:2023 §6.4.5 全节检索无此句**，本书不收进题面，判据仍以正本那四条为准（见下表）。

出口之前，正本在几处给了判据，另有一条出自同版同年旁证，合起来是四条：

| # | 判据 | 正本 |
|---|---|---|
| 1 | 完备 | 提供关于系统及其元素的 sufficient detailed data and information（中译：足够详细的数据与信息），并到 can be used to either procure or realise（中译：可用于采购或实现）为止（§6.4.5.1／§6.4.5.3 注 12） |
| 2 | 一致 | 与 architectural entities defined in models and views（中译：模型与视图中定义的架构实体）一致，并与组织或项目采用的设计指南和标准一致（§6.4.5.1） |

| # | 判据 | 正本 |
|---|---|---|
| 3 | 可追溯 | bi-directional traceability is maintained between the design characteristics and the architectural entities, identified interfaces, analysis results, verification methods or techniques, and system element requirements（中译：在设计特征与架构实体、已识别的接口、分析结果、验证方法或技术以及系统元素需求之间维护双向追溯性；§6.4.5.3 注 19） |
| 4 | 描述符匹配 | 元素及其设计描述符要与整体系统的设计描述符相匹配（INCOSE SEH 5.0 表述——同版同年旁证，此术语与此句均非正本条文） |

第 2 条有一个现成的别名，叫“一致落到哪”，它永远落在跨视图的对应关系上。设计跟某一张视图冲突，是冲突。设计的两个部分各自跟不同的视图对上了、彼此之间却对不上，也是不一致。

还有一样东西，旧稿把它列成第五条判据，现在要拆开说。接口，旧的说法是“接口只细化不新增”，这个说法已经废了。标准明写：It is possible that additional interfaces need to be identified and defined that were not addressed in the system architecture definition process.（中译：可能需要识别并定义系统架构定义过程中未予处理的附加接口；同 §6.4.5.3 注 10）

正确的说法要分两半：接口的“存在”归架构，“长相”归设计。架构识别过的接口，设计把它写细：时序、语义、错误处理、字表。架构没识别过、而两支元素确实要连的接口，不是就地补上，而是认领（怎么认领，下一节说）。

### 14.3.4 交接从来是双向的

如果架构与设计是接力，那么架构棒交出去、设计方跑完就算。标准不是这么写的。回流的正式通道，标准逐字给了一条。另有**两件**事走在这条路上，但去向不同——**一件回架构，一件往下交**。

这一条路：迭代反馈，设计逼着架构改。标准明写这条反馈：

> This process provides feedback to the system architecture definition process to consolidate or confirm the allocation, partitioning, and alignment of architectural entities to system elements that comprise the system.（中译：本过程向系统架构定义过程提供反馈，以整合或确认将架构实体分配、划分并对齐到构成系统的系统元素。）（同 §6.4.5.1 注 2）

第二件：添加接口与衍生需求，新长出来的东西要有人认领。

| 情况 | 处置 |
| --- | --- |
| 架构确实漏了（这本是它该交代的事） | 回架构补一次识别 |
| 需求变了（新出现了一条功能） | 把衍生需求纳入系统需求基线（**本书读法**：正本只给“衍生需求”与“系统需求基线”两处，合为一步是本书的处置口径） |

两条路都叫认领，认领这两个字是这一节的重点。设计会长出新东西，这是正常的；不正常的，是新长出来的东西没有主人。章首那条“与外部对账平台直连”，处置起来一点都不难，难的是先认出它无主。

无主比违规更危险：违规有人拦，无主没人拦。它不会在评审上被挡下来，因为它看上去只是一个“细化结果”。它会一直躺到集成那天，才和其他几十条无人认领的东西一起结账。

最后，设计定义还有一样东西要交出去：

> The configuration management process is used to establish and maintain configuration items and baselines. The design definition process identifies candidates for the baseline and then provides the artefacts to configuration management.（中译：配置管理过程用于建立并维护配置项与基线。设计定义过程识别基线的候选对象，随后将人工制品提供给配置管理。）（同 §6.4.5.3 注 20）

这一句是本章交给下一章的东西。架构交出去的那一版要算数、设计依从的到底是哪一版，答案是“架构基线”，那是下一章的事。本章只管把东西交到门口：设计定义识别候选，配置管理收下它们。

### 14.3.5 分到哪一层停，看信息够不够

设计定义会一层一层往下分，分到哪一层可以停？INCOSE 系统工程手册 5.0 把这条判据落在一个名字上：原子元素（T-167）。

如果一个系统元素只需要用黑盒（不透明箱）表示法来描述它的需求，并且能够自信而明确地定义它在现实世界中的解决方案。那么就可以在不进一步详细说明它的情况下，自信而明确地做出“购买、制造还是重用”的决策。（此条中文逐字取 INCOSE 系统工程手册 5.0，同版同年旁证；正本无此句。）

读这条判据要抓住两个词：黑盒、自信。它问的不是“分到第几层”，而是“关于这个东西，我手上的信息够不够我决定买、造还是重用”。它是信息充分性判据，不是深度判据。“一层分几块管得过来”那类经验规则是可管理性规则，不是停止规则。

这一段只点到为止。设计定义过程内部的其余动作，四组活动、几类可指名的方法、描述符法与可实现性关切，不属“架构交出去的那一段”，本章不展开。

## 案例进展①｜恒信：文档交出去了，约束没取到

散场之后，顾言把苏婷留下，摊开一张表。

“你交上来的那一版，我按判据挨条查过。”他说，“差的三条都在这儿，条条卡在判据上。”

| 查什么 | 查到什么 | 结论 |
| --- | --- | --- |
| 1 实体转化 | 设计里多了“与外部对账平台直连”，架构描述里没有这条 | 加戏一处 |
| 2 目标承担 | 架构目标“配置出来的孤岛必须可检出”——在谁身上是空的 | 缺承担者 |
| 3 描述符匹配 | 扩展点那条“什么算破坏性变更”，停在〈待定〉 | 未过出口 |

另外六条过了。顾言在结论栏里没有写“不合规”，他写的是条件性合规，三处补项各带责任方，期限空着。产品开发经理曹峰拿过去填了“月底”：“判据你们定，日期我定。”

“这三条都不合规，为什么不直接打回去?”苏婷问。

“打回去容易。可打回去之后，这三条判据谁来定?”顾言说，“这三条里有两条，根不在你这儿——一条是**架构**漏识别，一条是**判据**没定。”

他指着第一行：“这条接口，架构那边根本没识别过。你说'细化出来的'，可它不在任何人的账上。”罗建推门进来，听见这句，停了一下：“这条不是违规，是我的架构没识别。”他在补项栏签了自己的名字。

第三行他让苏婷自己填。苏婷卡了十分钟，抬头说：“我填不下去，因为判据没定。”

“记下来：判据没定，不是没做完，是没法判。”顾言说，“这两句不一样：前者要催进度，后者要回去把那条判据定下来。你们缺的不是工作量——**架构交出去的时候，约束没有被取走**。交文档不算交接，取到约束才算；取不到的那几条，回架构补识别，不要在设计里挑一个看着像的答案填上，那样它就成了一条没人认领的线。”

第二天，苏婷做了一件事：她把架构描述重读了一遍，把这一层真正要受的约束抄成两栏。左栏是“适用于本层的基本概念或属性”，右栏是“指导设计及其演化的原则”。抄完她写了一句：“上一次开会之前，我没有做这件事。”

苏婷后来讲给同组人听，用了一句很短的话：“我以为我们缺的是文档，其实我们缺的是那一次'取'。”她补的那两栏，后来成了设计组的固定动作。往后每接手一份新架构描述，先抄一遍约束；抄不出来的地方标出来，先问，再动手。

## 14.4 视图上的元素是谁家的

> 交接的路径说清了，看路上跑的东西。一块图上的方框、一根线、一个标签，它们各自是谁家的？这一块把元素分成四层、隔开两道界。

### 14.4.1 先给元素分层

答错的人多半不是不认得那些方框，而是把不同层的东西放进了同一条线。先把层分开：

| 层 | 是什么 | 拿什么量它 |
| --- | --- | --- |
| 描述层 | 架构描述元素与视图组件——图上那个框、那根线、那个标签 | 它遵守治理它的视角的约定吗 |
| 模型层 | 架构实体——模型里“以特定而离散的单位存在的东西” | 它回应关键关切、落实架构目标了吗 |
| 设计层 | 设计元素与设计特性 | 它够不够完整，支撑得了对架构的一次合规实现吗 |
| 实现层 | 系统元素——可被实现以满足规定要求的离散部分 | 它满足自己的规定要求了吗；能不能造、买或复用 |

描述层、模型层、设计层、实现层摆开，最先要看见的是：它们不是“同一个东西的四种粗细”。描述层是描述的零件，模型层是被描述的那个实体；设计层是实现方案的元素，实现层是真造出来的东西。

而两道界就藏在这四行之间，且经常被合成一道：

- 界一：描述自己的零件，与被描述的东西。
- 界二：架构实体，与系统元素。

### 14.4.2 描述里的零件，与被描述的对象

第一道界，问的是：你手上这个东西，是“描述的零件”，还是“被描述的对象”？标准在给架构描述元素下定义的时候，列过一张清单。先看它那句总纲——An AD element is an occurrence of one

or more architectural concepts in an AD.（中译：架构描述元素，是一个或多个架构概念在一份架构描述中的**出现**。）（42010:2022 §5.2.9）

然后它列了清单：AD elements include stakeholders, concerns, stakeholder perspectives, and aspects identified in an AD, ADLs, ADFs and correspondences and correspondence methods used in an AD, and architecture views, view components, architecture viewpoints, and model kinds included in an AD.（中译：架构描述元素包括 AD、ADL、ADF 中所标识的利益相关方、关注点、利益相关方角度和方面体，AD 中所用的对应关系与对应方法，以及 AD 中所含的架构视图、视图组件、架构视角和模型种类。）（ISO/IEC/IEEE 42010:2022 §3.4 注 1）

**先解释那个“出现”**：同一个概念（比如“关切”）可以在一份描述里出现很多次。**每一次出现，就是一个架构描述元素**。这张清单里全是描述这一侧的东西：相关方、关注点、角度、方面体、视角、视图、视图组件、模型种类、对应关系……它们都是这份描述自己的零件。没有一个是“被架构的那个实体”。

这里要把一处与前一节的分岔说清：14.2.2 说“视图里的元素是架构实体或被架构实体的表达”，说的是它表达的内容属哪一层。这一节说的是它自己是哪一类零件。同一个方框，作为零件是架构描述元素，作为表达指向架构实体——两句说的不是同一件事。

所以，“图上的元素就是系统元素”这句话，第一处就错了。图上的元素连“实体”都还不是，它是描述这个动作借用的零件。

这里有一条容易漏掉的边界。标准另有一处清单，列出的是“架构描述元素包括哪些架构概念的出现”，其中含架构决策与架构理由（architecture decision, architecture rationale；ISO/IEC/IEEE

42010:2022 §5.2.9）。两张清单都是十二项，但覆盖不同。这一点是本书的归拢，不是标准里的对照句（标准只是分别在两处给了两张清单）。归拢成一句就是：问“某某是不是架构描述元素”，要看的是 §5.2.9 那张。§3.4 注 1 那张不含决策与理由，不能拿它当全集。

### 14.4.3 架构实体不是系统元素

第二道界，标准用一句话钉死了，它先说设计定义过程要向架构定义过程提供反馈，然后紧接着写：

> Note that an architecture entity (an entity being architected or one subject to the system architecture definition process) and a system element (a discrete part of a system that fulfils specified requirements) **represent two different notions**.（中译：注意，架构实体（被架构的实体，或经受系统架构定义过程的实体）与系统元素（系统中满足规定需求的离散部分）**代表两种不同的概念**。）（ISO/IEC/IEEE 15288:2023 §5.9）

系统元素是什么，标准另给了一条定义：discrete part of a system that can be implemented to fulfil specified requirements（中译：系统的离散部分，能加以实现以满足规定的要求；同 §3.47）。两句放在一起，“两种不同的概念”落在哪就看得清了。架构实体是“被架构的那个事物”（企业、组织、系统、子系统、产品、服务、软件项、硬件项……）；系统元素是“构成系统的那个离散部分”。一个是“被架构的”那一头，一个是“构成系统的”那一头，同一个系统之上，两个词各指一头，不可互换。

而它们之间隔着的动作，正是上一块说的那条转化：Transform architectural entities and relationships into design elements.

（中译：将架构实体与关系转换为设计元素；同 §6.4.5.3 b) 2)）。注意终点是“设计元素”，还不是系统元素。从架构实体走到真造出来的系统元素，中间还有设计这一步。

### 14.4.4 架构实体可以是名义的

第二处最容易想歪的是：既然叫“实体”，那它总得是个实物吧？不必。标准在两处把这个误会堵掉了。

一处说元素在早期只是名义上的：

> Sometimes the elements of the architecture entity are **initially notional** until Design Definition process … has occurred since this depends on the actual design(s) to be done.（中译：有时，架构实体的元素在最初是名义性的，直至设计定义过程……完成，因为这取决于实际要完成的设计。）（ISO/IEC/IEEE 42020:2019 §5.6）

另一处说得更直接：Elements in the model of the architecture will represent things that exist as particular and discrete units. They are neither necessarily tangible objects nor visible items.（中译：架构模型中的元素将表示作为特定且离散单元而存在的事物，它们既不必是有形物体，也不必是可见条目。）（同 §10.4.4 b) 注 1）

> **译名说明**：notional 一词，库内 42020 中英对照版译作“概念性的”，本章用“**名义的**”。两者指同一件事：它现在有一个名字、有一个位置，但还没有确定由什么来实现。本章取“名

> 义的”，是为了与第 13 章的“概念”一族词（基本概念、架构概念）避开。

这一条的要害在时间上：架构实体与最终的系统元素之间，不存在一份现成的对应。对应是靠设计定义这一步建立起来的。所以“架构里画了五个块，实现时就一定是五个模块”这句话，在架构那一侧根本没有依据。

### 14.4.5 不必一一对齐

那么，一个架构实体对应几个设计元素？标准没有给 1:1：

> This task helps to ensure that each architectural entity … and relationship is **mapped into the appropriate system design elements** to help ensure that all the architectural objectives are addressed by the design.（中译：本任务有助于确保每个架构实体……和关系都**映射到**适当的系统设计元素，以帮助确保设计应对所有架构目标。）（ISO/IEC/IEEE 15288:2023 §6.4.5.3 注 5）

用的是 mapped into（映射到），没限定个数。**顺带把一个叫法对上**：正本有的地方写“系统设计元素”，说的就是“设计元素”。它指“构成这个系统的那些设计元素”，不是另立的东西。一个架构实体可以映射到几个设计元素，几个架构实体也可以落到同一个设计元素上。配着这条看，还有两件事要一起说：

一是追溯要双向。 bi-directional traceability is maintained between the design characteristics and the architectural entities, identified interfaces, analysis results, verification methods or techniques, and system element requirements.（中

译：在设计特征与架构实体、已识别的接口、分析结果、验证方法或技术以及系统元素需求之间维护双向追溯性。）（同 §6.4.5.3 注 19）往下能追到出处，往上能追到它落到哪儿，两头都要通。

二是可以递归。一个系统元素，本身还能再被当作一个实体架构一次。the appropriate system life cycle processes are applied recursively to an SoI to resolve its structure to the point where understandable and manageable system elements can be implemented (made, bought, or reused)（中译：将适宜的系统生存周期过程递归地应用于一个所关注系统，以把结构分解到可理解且可管理、可以制造、购买或复用的系统元素的程度；同 §5.2.2）。这正是“模块也有架构”的标准根据：模块在上一层是系统元素，在它自己这一层又是被架构的实体。

### 14.4.6 系统分解结构不是架构的产物

“是谁家的”这个问题，还有一样东西经常被算错，系统分解结构（system breakdown structure，SBS）。标准把“建立适当的分解结构”写进了项目规划过程的任务里，4) Establish appropriate breakdown structures.（中译：建立适当的分解结构。）（ISO/IEC/IEEE 15288:2023 §6.3.1.3）它旁边那条注把话说全了：Typical breakdown structures include work breakdown structure, functional breakdown structure, system breakdown structure, and organizational breakdown structure.（中译：典型的分解结构包括工作分解结构、功能分解结构、系统分解结构和组织分解结构；同 §6.3.1.3 注 4）

三点要记住：第一，SBS 的户口在项目规划过程，不在架构过程。它是一份规划台账，用来组织工作、分派责任、管理进度；不是架构的产物，也不是描述的一部分。

第二，系统结构本身不必是层级。分解结构之所以长成一棵树，是“台账要好查”这个用途逼出来的，不是系统本来就长这样。把“结构是树”当成系统的性质，就会看漏那些横切、共享、循环的关系。

第三，“产品分解结构”（product breakdown structure，PBS）不是标准术语，本书不用。讲分解结构的那条注里列的四种，也没有它。遇到这个词，一律改说 SBS，或者直接说“哪一份台账”。

这一节最后要划两条界，它们都在物理视图那一侧：

- 物理视图 ↔ 设计描述：写到哪儿为止？型号、尺寸、材料、数量、实例数一出现，就属设计定义。物理视图只到“系统元素与物理接口的一种安排”为止，再往下是设计解的活。
- 物理视图 ↔ 系统分解结构：物理视图上的块是系统元素（以及它们的安排），不是 SBS 上的节点。两者可以互相参照，但不是同一份东西，一份是架构的表达，一份是规划的台账。

### 14.4.7 描述层到实现层，全是关系，没有粗细

先看表。表里每一行，都是前面几节立下的一条关系：

| # | 关系 | 一句话 | 正本 |
|---|---|---|---|
| 1 | 表达 | 视图表达架构；架构描述是架构的表达 | 42010:2022 §3.7／§3.3 |
| 2 | 两种概念 | 架构实体与系统元素代表两种不同的概念 | 15288:2023 §5.9（该段） |
| 3 | 转化 | 架构实体与关系→设计元素；架构特性→设计特性（不是“细化”） | 15288:2023 §6.4.5.3 b) 2)3) |
| 4 | 一致 | 设计要与模型和视图里定义的架构实体一致 | 15288:2023 §6.4.5.1 |
| 5 | 双向追溯 | 设计特征与架构实体、接口、分析结果之间两头都追得到 | 15288:2023 §6.4.5.3 注 19 |

五条里没有一条是“粗细”，它们说的是表达、身份、转化、一致与追溯。把元素放对地方，靠的就是这五条。

## 案例进展②｜冰链：吴浩把取值与出处分开写

冰链那边，同样的缝，换了一个方向。嵌入式硬件工程师石军把接口视角的规格挂进项目描述框架之后没几天，吴浩找上门来。他在做子系统那一层的设计，卡在一个地方。

“按你那条视角规格，我这层的降级状态该分几级?”云端软件的吴浩问。

石军没直接答：“五级是你定的，还是我这条**视角规格**定的?”

吴浩愣了一下，回去翻那份规格。翻完回来说：“规格里只说'连续性不得中断'，没说分几级。”

“那它管不着你：**视角**管的是形状，不管你的取值。”石军说。

吴浩回去把设计描述改了：降级状态定五级，然后在旁边补了两栏。一栏写“本层取值：五级”，一栏写“依据：接口视角规格第 4 条（连续性要求）”。

产品经理林涛后来看到这两栏，说了一句收尾的话：“架构没管你的取值，但你的取值有了出处。”

这两栏正好落在两个不同的位置上：一栏是设计特性，那是设计这一侧的事，架构管不着。一栏是可追溯性映射，往上追到的是那条它必须守的约定。取值归设计，出处归依从，补上这两栏，吴浩那一格才算既自由，又对得上账。

这个场面里没有谁犯错。石军那条规矩本来就只该管到形状，吴浩那个取值本来就该由本层定。错只错在，两样东西原来没有被分开写。分开写之后，规矩守住了该守的，取值也留在了该留的地方。两边都松了，也两边都有着落。

## 14.5 视图上的元素都要“基本”吗

> 前一块把元素是谁家的分清了。紧接着就有一个必然被追问的问题：既然架构讲的是“基本概念或属性”，那视图上画出来的每一个东西，是不是都得“基本”？

答：不必。“基本”修饰的是架构的内容，不是描述上的每个元素。

### 14.5.1 “基本”管的是谁

先看那句定义，看“基本”这个词在句子里的位置：

> architecture — **fundamental** concepts or properties of an entity in its environment and **governing principles** for the realization and evolution of this entity and its related life cycle processes（中译：架构——实体在其环境中的**基本**概念或属性，以及为实现和演化该实体及其相关生存周期过程的管控原则。）（ISO/IEC/IEEE 42010:2022 §3.2）

注意语法位置： fundamental（基本的）在句子里修饰的是 concepts or properties（概念或属性）。它界定的是架构由哪些内容构成，管不到描述上画了什么。

往下看两处，更能说明问题：

- 视图的定义句里，没有“基本”：information part comprising portion of an architecture description（中译：包含架构描述的一部分的信息部件；同 §3.7）。判断一份东西是不是视图，问的是“它在不在描述里、受不受某个视角治理”，不问它“基本不基本”。

- 视图与视图组件的构成要求里，也没有“基本”这一项：标准对视图要求的是标识与辅助信息、治理它的视角、视图组件、已知问题；对组件要求的是有治理它的模型种类或图例（§6.7／§6.8）。

“基本”住在架构那一侧，不修饰视图元素。这一条有了，另一半问题才好答：既然不基本，那它们靠什么站得住？

### 14.5.2 “基本”在视角上定，不在图标上

答案在“关切—视角—视图”这条链上。标准把这条链写成了三句要求：

> Each concern identified in accordance with 6.4 **shall be framed by at least one architecture viewpoint**.（中译：依据 6.4 识别的每个关切应由至少一个架构视角来框定。）（ISO/IEC/IEEE 42010:2022 §6.6）

> 视角的规格须列出 one or more concerns framed by this architecture viewpoint（中译：由本架构视角框定的一项或多项关切；同 §8.1 b)）。

> 视图的组件须 address all of the concerns framed by its governing viewpoint（中译：应对其治理视角所框定的全部关切；同 §6.7 c)）。

三句合起来，是一条准入链：哪些关切够格被视角框定，是在“视角”这一环上被裁的。一旦某条关切被某个视角框定，这个视角下的视图元素就是“为这条关切服务的”，不再逐个去接受“基本”的检验。

所以“基本”不是消失，是换了发生的地方。它发生在“哪些关切够格成为架构要回应的关切”这一步，而不是发生在“图上每一个图标”那一层。要拿“基本”去量的，是架构的内容；不是视图的零件。

### 14.5.3 不必基本，不等于不必相关

话说到这儿，有一个很自然的滑坡要挡住：既然图上的元素不必“基本”，那是不是画什么都行？

不行。标准在同一个地方立着几条硬边界：

- 视图必须是“表达被架构实体之架构”的信息部件（§3.7），它是那份描述，不是背景资料。
- 组件必须应对其治理视角所框定的全部关切（§6.7 c），shall）。
- 必须遵守其视角的约定（§6.7，shall）；而 The governing viewpoint or model kind or legend determines the syntax and semantic conventions for these introduced AD elements.（中译：起管控作用的视角、模型种类或图例确定这些引入的架构描述元素的语法与语义约定；同 §5.2.9）。

而且，标准明许“非基本”的东西合法地留在架构描述里：

> Information parts not within any view could include overviews of the entity of interest, architecture principles, architecture patterns and architecture styles whose application spans more than one view; referenced bases for the architecture, such as domain or reference architectures; correspondences between views; and architecture rationale.（中译：不属于任何视图的信息部分可包括：所关注实体的概览；架构原则、架构模式以及适用跨越一个以上视图的架构风格；所引用的架构依据，如

> 领域架构或参考架构；视图之间的对应关系；以及架构理由。）（同 §6.7 示例 2）

这张单子里，没有一条是“基本概念或属性”，它们照样是这份描述正当的组成部分。准确的表述是：视图上的元素要“为被框定的关切服务、并在该视角的约定内被解释”；要“是架构的表达”。但不必是“基本的”。不必基本 ≠ 不必相关。前一句划掉的是“逐个图标都要过基本这一关”，后一句守住的是“每个元素都要有它服务的那条关切”。

### 14.5.4 换个位置就不基本了

“基本”还有一个容易被忽略的性质：它是相对所关注实体的判定，不是内容的固有标签。定义句写的是 of an entity（中译：属于某个实体的），每个被架构的实体，各有其“基本”。再叠上上一块说过的那条递归（一个系统元素自己还能再被架构一次），就得到一条很实用的判断。同一条内容，在上层可能是基本属性，在下层只是设计取值。

以平视显示器为例：

| 内容 | 在整机层 | 在显示源（子系统）层 |
| --- | --- | --- |
| “共形精度 ⩽ 毫弧度量级” | 基本属性——目的所依赖，去掉即不成立 | 设计取值——实现它的容差预算，是设计特性 |
| “组合器的安装公差” | 不是架构内容——下游取值 | 可能是该子系统架构的基本属性 |

所以问“这条够不够基本”，先要问清一句：对着哪个实体、它的目的是什么。同一句话，换个位置，答案就变了。

## 案例进展③｜冰链：一条新接口该去哪儿

吴浩那条接口，后来还是长出来了。子系统里两块元素确实要连，架构描述里没有这条线。他先按习惯在图上补了一笔，画完又停住。这一笔要是就这么留着，它就成了章首苏婷那条“细化出来的”，谁也不知道它是从哪儿冒出来的。

他列了一张小单子，问了三句：

- 这条接口，架构识别过吗？没有。
- 它是“架构漏了”，还是“需求变了”？两块元素本来就要连，是架构那一版没识别到。
- 那它该回哪儿？回架构补一次识别。

他把这条报给石军。石军补进架构描述的那一版，重新走了一次冻结。同一个月，恒信那边也长出来一样东西，但性质不同。业务方新增了一条“要能查到历史对账记录”：这是需求变了，不是架构漏了。苏婷按第二条路处置，把衍生需求纳入系统需求基线，走变更流程。两条路都叫认领。分岔只在那一句：它是架构漏了，还是需求变了。

他把这三句写在一张便签上，贴在了工位边上。后来冰链的设计组把这张便签抄成了三行，放在每一份设计描述的末页：

> **新长出来的东西，先问三句**：架构识别过吗？是架构漏了还是需求变了？该回哪儿？

三条都答得出来，它就有人认领了。

这三句后来被系统工程师小张收进她那份交接清单。清单上原有两栏，谁认、什么时候；她在后面又添了第三栏，专写依据。

林涛把那张单子要过去看了一遍：“这张我也照着答。”

她说：“认领不是报一声就算。15288 §6.4.5.3 注 19 要的是双向追溯：往下追到它落在哪个元素上，往上追到它是哪条依据带下来的；这两头都写得出来，换个人拿着这张单子问下去，才答得上来。”

## 14.6 能不能进，是谁的，够不够格

> 三块合起来，全章要问的其实是同一件事的三个问句。这一块把它们摆在一张桌上，说清为什么谁也替不了谁。

### 14.6.1 先问能不能进视图

第一问：这个东西，能不能进视图？这一问的对象是视图上的元素，量它的尺子是描述这一侧的那些“应”。

来源交代（一句话）：下面这六项是**本书的归并**——正本没有“六项”这份清单，它取的是 42010:2022 §6.6／§6.7／§6.8／§6.9.2 里的 shall 要求（共二十句），外加 §5.2.9／§3.7 的定义性约束。表里除第 ⑤ 项是定义性外，其余都有 shall 垫底

| # | 条件 | 正本 |
|---|---|---|
| ① | 它所属的视角已纳入本描述 | §6.6：An AD shall include or reference each architecture viewpoint used therein.（中译：架构描述应包含或引用其中所使用的每个架构视角。） |
| ② | 它落在该视角框定的关切之内 | §6.7 c)；§8.1 b) |
| ③ | 它由某个模型种类或图例治理 | §6.8：组件应标识其管控模型种类（如有）；无模型种类的，a view component legend shall be included（中译：应包含视图组件图例） |

| # | 条件 | 正本 |
|---|---|---|
| ④ | 它符合该视角／模型种类的语法语义约定 | §6.7：Each architecture view shall adhere to the conventions of its governing architecture viewpoint.（中译：每个架构视图应遵循其管控架构视角的约定。）；§5.2.9 |
| ⑤ | 它是架构描述元素，且表达的是被架构的实体 | §5.2.9；§3.7（定义性，非 shall） |
| ⑥ | 它与视图内其他元素的关系可被表达 | §6.9.2：An AD shall include or reference a list of AD element correspondences.（中译：架构描述应包含或引用一份架构描述元素对应关系清单。） |

六项之外，还有一条操作判据：“由谁引入”的追溯，每一样东西都该说得出来它是被哪个视角、模型种类或图例引进来的（As viewpoints, model kinds and legends are specified and applied, additional AD elements are introduced.／中译：随着视角、模型种类与图例被规定和应用，附加的架构描述元素被引入；同§5.2.9）。追不到引入者，又没有图例，它就只是“画在图上的东西”。

实际入场有三条路，每条都说得出来源：

- 由模型种类或图例引入，视角规定了用哪几种模型种类，模型种类自带一套构造（用例里的参与者、边界，活动模型里的输入、输出、控制、机制……），这些东西是被“约定”引进来的（§5.2.9）。
- 由另一份架构描述精化或引用，An AD element in an AD can be refined or elaborated by reference to another AD. An AD can utilize protocols introduced as AD elements of distinct ADs.（中译：一份架构描述中的架构描述元素，可以参照另一份

架构描述来精化或细化。一份架构描述可以利用作为不同架构描述的架构描述元素而引入的协议；同 §5.2.9）

- 由视图方法把信息源翻译过来，视图方法规定“架构描述元素将如何被呈现”“信息源如何被转换或翻译”（§8.3）。

三条路的意义在追溯：任何一个合法的视图元素，都该能说出自己走的是哪一条。说不出走哪条，它就还没进这个体系。

### 14.6.2 进不去的，去哪

进不去视图，不等于没地方去。标准给了一处正当的容身处，而且用的是 may：

> An AD **may** include other information which is not part of any architecture view.（中译：架构描述**可**包含不属于任何架构视图的其他信息。）（ISO/IEC/IEEE 42010:2022 §6.7）

| 进不去的 | 该归何处 |
| --- | --- |
| 视角规格、模型种类规格（它们是治理者，不是被表达物） | 描述内的其他信息 |
| 跨视图的东西：全景总览、架构原则与模式、所引用的架构依据、视图间的对应关系、架构理由 | 描述内不属于任何视图的信息（§6.7 示例 2） |
| 架构决策与理由 | 架构描述元素（§5.2.9 明确列入），但通常不塞进某一视图；它们另有记录要求（§6.10：必不可少的决策应记录，理由宜记录） |

| 进不去的 | 该归何处 |
| --- | --- |
| 纯粹的实现细节与设计取值 | 设计描述——架构阶段接口只定到 the level or extent needed for the architecture intent and understanding（中译：架构意图与理解所需的层级或程度；15288:2023 §6.4.5.3 注 10） |
| 本视角框定范围之外的关切 | 换个视角，或新立一个视角（并记下为什么） |

这张表的意思不是“进不去就丢掉”，是“每样东西都有自己的家”。混进去的代价很具体：描述里塞满了不该在上面的东西，读者就再也不知道哪一块是承诺。

### 14.6.3 再问：图上的元素属于哪一层

第二问：这个东西，表达的是谁？这一问的对象是它的身份。量它的尺子是上一块那四个层、两道界。

同一个方框，可能落在四个不同的位置上。它是描述自己的零件（视图组件、视角规格），还是被描述的架构实体，还是设计元素，还是系统元素？

三句最容易说错的话，都出在这一问上：

- “视图上的元素就是系统元素。”错。中间隔着两道界（§3.4 注 1；§5.9 末段）。
- “视图上的元素是系统元素的泛化版。”反了。架构实体不必是实物，也可以是名义的（42020 §10.4.4 b) 注 1／§5.6），而系统元素的标准是“可被实现以满足规定要求”（15288 §3.47），一个是理解单位，一个是实现单位。
- “架构已经定义了系统元素。”只有半句对。架构定义过程的成果里确有“系统元素及其相互接口被定义”这一项（§6.4.4.2 g)），

但“过程成果”不等于“架构”；而且接口在架构阶段只定到“足以传达架构意图”的程度，细化在设计定义过程（§6.4.5.3 注 10）。

### 14.6.4 最后问：拿掉一条，目的还达得成吗

第三问：拿掉它，这个实体还达得成它的目的吗？这一问的对象是架构的内容。量它的尺子只有一个：目的。

这一问在前一块已经量过一遍（§14.5）。它管的是“哪些关切够格成为架构要回应的关切”，发生在视角这一环上，不落在每一个图标上，这里只补一句：这把尺子不认别的判据。不认“粒度够不够细”，不认“层级够不够高”，也不认“改起来贵不贵”，那些都回答不了“拿掉它，目的还达得成吗”。

### 14.6.5 各管一段，谁也替不了谁

三问摆在一起，是一件东西的三个面，不是三次重复：

| 关 | 对象 | 量它的尺子 |
| --- | --- | --- |
| 描述关 | 视图上的元素 | 能不能进视图：六项＋“由谁引入”的追溯 |
| 实体关 | 它表达的是谁 | 描述层零件／模型层架构实体／设计层设计元素／实现层系统元素 |
| 目的关 | 架构的内容 | 拿掉它，这个实体还达得成目的吗 |

混起来会出两种典型错误：

1. 用“基本”去要求每一个图标，图里塞满大词，视图不再回答任何具体关切。这是把第三关的尺子拿到第一关上用。
2. 用“画得出”当“够基本”，把设计取值当成架构内容收进视图，于是“架构”变成“更粗的设计”。这是把第一关的通行证当成第三关的合格证。

落到手上，是一个四问的动作。看到视图里的一样东西，连问四句：

> ① 它服务于本视角框定的哪条关切？——答不出：它不是视图元素，或者该归“描述内其他信息”。② 它按哪个模型种类或图例的约定被解释？——答不出：必须补图例。③ 它表达的是被架构实体的架构吗（还是背景、参考、别人家的东西）？——后者可以留在描述里，但要标明是引用基线或外部上下文。④ 它排掉了什么？——排不出任何东西，它可能只是信息，不是架构内容。

**四句与三关的对应是**：①②属**描述关**（它服务于哪条关切、按哪个约定被解释）；③属**实体关**（它表达的是谁）；④属目的关（它排掉了什么，才配当架构内容）。三关各管一段，谁也替不了谁，这才是全章最后要立住的那句话。

### 14.6.6 认知反转

回到章首那间评审室。苏婷那三问，每一问都答了一半；罗建那句“交出去了”，说的时候底气很足。如果你读完了前面十三章，你大概会替他们说一句：设计确实比架构详细，说“做细了”不算错吧？这句话站不住，因为它把四件事混成了一件。

第一层，把“种类”当成了“粗细”。架构和设计不是同一支笔的两档笔尖：它们的目的句不同、作用面不同。还有一条独立性的要求：有效的架构尽可能不依赖于设计。同一件事不会要求“尽量不依赖自己”。这条要求的存在，本身就证明两者不是同一件事的两档精度。

第二层，把“输入”当成了“原料”。需求是架构必要且充分的输入，该给的都给了、够用了；可它不是被加工的原料。架构是基于原

则、概念和属性定义出来的解。设计才是把架构与需求转换成可实现的系统设计。把这两句合成一句，就会得出“需求写细一点，架构就出来了”，而标准从没这么说过。

第三层，把“接收”当成了“交接”。罗建交出去的是文档，设计方该拿走的是约束。交文档不算交接，取到约束才算。第四层，也是最容易被漏掉的一层：把“换手”当成了“接力”。设计会长出新元素，这是正常的；不正常的是没人认领。而且交接从来是双向的：设计能打回架构，接口可以在设计里新增，基线候选物要交出去。无主比违规更危险，违规有人拦，无主没人拦。

本章真正要立的，是那三句最短的话。架构与设计是两类工作，不是同一件事的两档精度；架构不是基于需求设计出来的，需求是它必要且充分的输入；架构交出去的那一段不是接力，是换手。

用第 1 章的话收一次束：对“架构与设计”，你原来只是“见过”，见过图、见过设计文档、见过“架构冻结”四个字。本章把它推到“拥有”，能说出分界在哪、入口取什么、转的是哪几样、元素住在哪一层、三关各量什么。能答全这四问，才算拥有。

## 小结

本章讲的是架构交出去的那一段路：从一份架构描述，到一份能拿去采购或实现的系统设计描述。它先把**分界**说准了：两边的目的句不同、作用面不同（一头管适宜·存续·合意，一头管兼容·可造·可集），design 一词还带着宽窄两义。深浅说不通，证据就是那条独立性要求。位置也不同：一边是“定义”出来的解，一边是“转换”出来的设计，而需求是前者必要且充分的输入、不是原料。

它立了交接的动作：入口是检查架构、取约束，不是接收文档。中间是转化与分配（架构实体与关系→设计元素；架构特性→设计特性；

系统需求→系统元素，须同时对着系统需求和架构目标）。出口是能拿去采购或实现，停在〈待定〉即未过出口。而交接从来是双向的：设计能打回架构，接口可以在设计里新增，基线候选物要交出去。长出来的新东西必须被认领，无主比违规更危险。

它把元素分了四层、隔开两道界：描述自己的零件与被描述的东西；架构实体与系统元素。架构实体不必是实物，也可以是名义的，一个实体不必只对一个设计元素，系统元素自己还能再被架构一次。系统分解结构不是架构的产物。

它回答了那个必然被追问的问题。视图上的元素不必“基本”，“基本”修饰的是架构的内容；“基本”只从视角这一环起作用；而不必基本，不等于不必相关。最后它把三个问句摆在桌上：能不能进视图（描述关）、它表达的是谁（实体关）、拿掉它还达得成目的吗（目的关）。三关各管一段，谁也替不了谁。

本章的三句话：架构与设计是两类工作，不是同一件事的两档精度；架构不是基于需求设计出来的，需求是它必要且充分的输入；架构交出去的那一段不是接力，是换手，交出去的是约束，接住的是依从。回头看章首那间评审室：苏婷那三问答不上，不是因为她不认真，是因为她把两样东西摆成了一条线。把线拆成两条，问题就都能问了。

# 附录

## A 本章概念总图

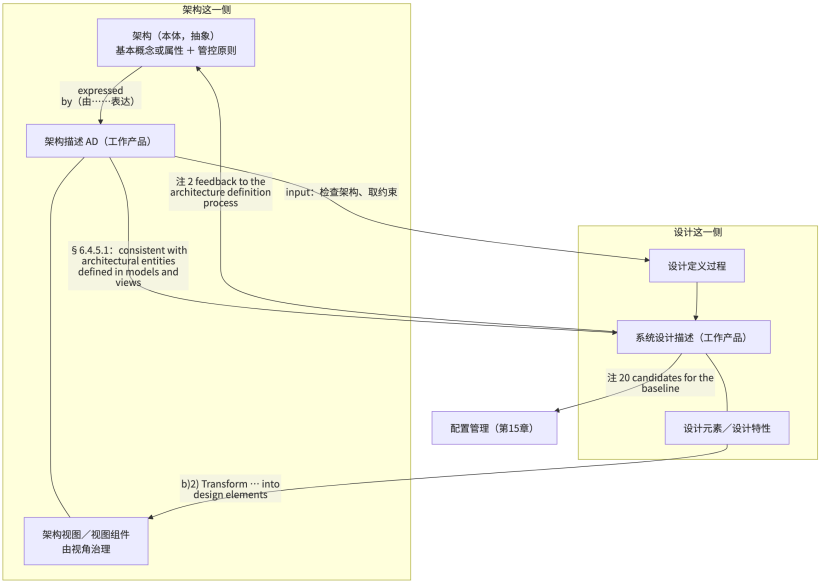



图 14-1

## B 本章的能力级

第 ③ 级为主

- 表征，本章把哪几个词说准：架构／架构描述（界线重述）｜设计定义 T-164（过程）／设计描述 T-165（产物；正本用名是 design description，本章首次给出其标准用名与全书简称的关系）／设计描述符 T-166／原子元素 T-167｜四个层名（架构描述元素与视图组件／架构实体／设计元素与设计特性／系统元素）｜系统分解结构 SBS 及其户口（项目规划过程，不是架构的产物）
- 图式，本章给出哪个可识别的结构：四层两界图（描述层／模型层／设计层／实现层；界一＝描述自己的零件与被描述者，界二＝架

构实体与系统元素）｜两条不同的东西并排图（需求—方案链，与目的→架构）｜视图元素的准入判定链（六项＋由谁引入的追溯）｜三关对应表（每关：对象／尺子／混起来会出什么错）

- 心智模型，本章给出哪个能推演的机制：种类不是粒度（两把尺不同轴；独立性要求即判据）｜转化不是细化（判据是“转没转过去”；设计会长出新东西，要管的是认领）｜“基本”只从视角这一环起作用（不必基本 ≠ 不必相关）｜三关不可互相替代（描述关／实体关／目的关）｜架构不是基于需求设计出来的（定义 vs 转换；需求是必要且充分的输入）
- 解释框架，本章不建此级，由第 17、18 章承担

再用第 1 章 §1.4.4 那把尺子量一下：本章站在第 ③ 级（心智模型）。它给你的是一张交接图，拿到任何一份系统设计描述，你都能顺着这张图走一遍，指出它在哪一步转丢了。

> **能答全四问，才算拥有**——架构和设计两样东西摆在你面前，你能说清这条内容属于哪一类、那个元素是谁家的、它够不够格当架构内容。

## C 自查 · 这些说法对吗？

1. “架构和设计就是一回事，只是粗细不同。”（✗） 2. “标准说了，定义架构也属于 design，所以架构是设计的一部分，也所以架构是粗的设计。”（◐ 前半对，后半是偷换）
2. “有效的架构要尽可能不依赖于设计——这句话说明架构比设计更基础。”（✗） 4. “需求是架构的输入，所以把需求写细一点，架构就出来了。”（✗）

3. “接手架构的第一步是接收文档，然后照着做。”（✗） 6. “分配就是把系统需求分到元素上，让每条需求都有主。”（◐ 漏了一半：还须对着架构目标）
4. “设计里长出来的新元素，就地补上就行。”（✗） 8. “接口只能细化，不能新增。”（✗ 已废；不合格的是追不到架构依据，又不回流）
5. “视图上的元素就是系统元素。”（✗） 10. “架构实体必须是实物。”（✗ 也可以是名义的）
6. “一个架构实体只能对应一个设计元素。”（✗ 不必逐一对上） 12. “系统分解结构是架构的产物。”（✗ 属项目规划过程）
7. “视图上每个图标都要满足'基本'。”（✗） 14. “既然不必基本，那画什么都行。”（✗ 不必基本 ≠ 不必相关）
8. “能不能进视图、它表达的是谁、它够不够格——这三件事是一件事。”（✗ 三关各管一段）

## D 练习

1. 拿你们最近一份系统设计描述，把“本次从架构取到了哪几条约束”抄成两栏，左栏适用于本层的基本概念或属性，右栏指导设计及其演化的原则。抄不出来的地方，就是交接的缺口。
2. 找出你们设计里“架构描述里没有、设计里却有”的东西，逐条判：是架构漏了，还是需求变了？两种处置各走什么流程？
3. 挑一条架构特性，追它这一路：它在架构那一侧叫什么，在设计这一侧变成了哪几个设计特性，落到哪个元素上，取值定在哪里。
4. 用四层两界给自己项目上的十个元素分一次层，看有多少个在第一遍分错了。 5. 拿一份物理视图，找出“型号、尺寸、材料、数量、实例数”出现的第一处，那就是该收笔的地方。

5. 找一条你们团队争论“要不要写进架构”的内容，连问第四问：拿掉它，这个实体还达得成目的吗？ 7. 把你们视图里的三个元素拿出来，各问四句（服务哪条关切／按哪个约定被解释／表达的是谁／排掉了什么）。
6. 写给下一层接手的人一段话：这一版架构交出去，你要他“取走”的是哪几样东西。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022《软件、系统与企业，架构描述》，§3.1 架构工作；§3.2 架构；§3.3 架构描述；§3.4 注 1；§3.7 架构视图；§3.8 架构视角；§5.2.2；§5.2.9；§5.3；§6.1／§6.2；§6.6／§6.7／§6.8／§6.9.2／§6.9.3；§6.10.1／§6.10.2
- [2] **R-05** ISO/IEC/IEEE 42020:2019《架构过程》，§5.6；§8.4.5／§8.4.7；§10.4.2 h)·n)；§10.4.4 b) 注 1／e)·f)·m)；§10.4.5 注 1；附录 E.2.1（架构即约束）
- [3] **R-28** ISO/IEC/IEEE 42030:2019《架构评估框架》，以目的检验架构
- [4] **R-37** ISO/IEC/IEEE 15288:2023《系统与软件工程，系统生存周期过程》，§5.2.2 系统结构；§5.9 概念与系统定义；§3.6 人工制品；§3.13 设计（名词）；§3.31 递归；§3.47 系统元素；§6.3.1.3 项目规划过程（任务 4) 与注 4）；§6.4.3.3 注 18；

§6.4.4.1 系统架构定义过程目的与输入；§6.4.5.1~§6.4.5.3 设计定义过程（含 §6.4.5.3 b) 1)~8) 与注 2·4·5·6·10·12·19·20）

- [5] **R-38** ISO 9000:2026《质量管理体系，基础与术语》，§3.3.11 设计和开发；§3.5.1 要求；§3.10.1 特性

**经典文献**

- [6] **R-63** INCOSE《系统工程手册》（SEH 5.0 中文版，与 15288:2023 同版同年）—— 本章“检查系统架构”一句与原子元素两条件的出处

# 第 15 章 配置管理：让架构判断受控、可复现

## 15.1 引言

上一章把“架构交出去、设计接住”办完了：架构与设计不是粗细两版，是两类工作；交出去的是约束，接住的是依从。本章接的是它留在桌上的最后一句话：设计依从的，到底是哪一版架构。桌上还压着一张更早的工单：第 9 章说，可复现意识（你不在场，判断还接得住）往后要在**配置与基线**里长成制度。

工程现场对“配置管理”另有一套根深蒂固的说法：它是行政事务，管参数、存档、走流程；版本号标上了，就等于配置管理做好了。本章要办的正是这件事。它把一项能力当成了一道手续：手续可以外包、可以跳过，能力跳不过去——你交出去的每一份判断，凭什么在换个人之后仍然算得出来。

### 15.1.1 误解现场

冰链 M3 二代的项目例会上，接口视角规格挂进项目架构描述框架（architecture description framework，ADF）之后的第三周。

云端软件的吴浩把子系统设计描述摊在桌上，第一页就卡住了：“我这层的降级状态分了五级。这个取值，是照石军那份接口视角规格定的——可规格里没写五级，只写'连续性不得中断'。那我这一版，到底算不算依从那版架构？”

嵌入式硬件工程师石军翻出他那份规格：“我这版是三月十四号定稿、挂进 ADF 的。你看的是哪一版？”

“我是按挂进去的那版抄的。”吴浩说，“不过上周我又问过你一次，你说降级状态那一条你改了个说法。”

会议室安静了两秒。产品经理林涛把电脑转过来：“ADF 里现在有两份：三月十四号那版，和上周那版。设计组手上是后者，架构组的评审纪要写的是前者。”

系统工程师小张翻的是两份的签页：“三月十四号这份有四个人的签字，上周那份只有一条修订记录，没有人签。”

制造侧的小周接了一句：“产线上还烧着按三月十四号那版做的样件。”

测试季昭问的是一句更窄的话：“那我们说'二代的设计依从架构'，依从的是哪一版？”

没人答得上来。不是没人知道答案，而是这件事从来没有被谁定下来过：规格有三版、描述有两版、评审纪要和样件各认一版，每一版都真实存在、都能拿得出手，可“哪一版算数”没有落款。

产品开发经理赵刚在后排低声说了一句：“我们说'设计依从架构'，还怎么站得住。”

### 15.1.2 本章要回答什么

读完本章，下面这些问题你应当答得出来：

1. “配置”是参数文件吗？为什么说它是被记录下来的功能与物理特性？配置项、配置信息又是谁？
2. 什么算配置、什么不算？一条特性凭什么进得来？
3. 版本是什么？它记的为什么只是“刻度”，而“最新版”为什么当不了基线？
4. 基线凭什么算基线？批准、冻结、参照这三样，缺哪一样就散架？

5. 配置基线、产品基线分别立在哪一步？功能基线与分配基线为什么不能当正本讲？
6. 变体与版本是同一件事吗？为什么一个是空间、一个是时间？
7. 草稿与“定下来的那一份”之间，隔着一个什么动作？这个动作为什么不在技术过程里？
8. 架构描述是“定稿之后才交出去管”，还是在过程中就受控？
9. 配置管理平时做哪几件事？“配置审计”与“审核”为什么不是一件事？
10. 改了东西，凭什么说得清“从什么变到什么”？
11. **挑出候选**与定成基线，为什么是两件事、分属两只手？谁有权批这一版，又归谁管？
12. 不做配置管理会坏在哪？“有版本管理、没有配置管理”是哪种坏法？
13. 架构基线到底指哪一版？设计依从的为什么是它？
14. 这件事替架构定义做了什么？它跟第 4 章的需求变更闭环，边界在哪？

这十四问合起来是一件事：你交出去的那一份判断，凭什么在换个人之后还站得住、还查得下去。答案的一半写在题面副题里：受控、可复现。另一半在过程里——配置管理不改你的架构，它只把“哪一版是定下来的那一份”固定下来，并且一直守得住。

### 15.1.3 本章的骨架

先交代两个反复出现的说法。本章说“**正本**”，指的是库内那几份标准的原文：ISO/IEC/IEEE 15288:2023、ISO 9000:2026、42010:2022、42020:2019。说“**同版同年旁证**”，指的是 INCOSE 系

统工程手册 5.0 中文版；它与 15288:2023 同版同年（此说为通行说法，本库不核）。可以对着看，但不是标准条文，凡用它都当场标明。

第 10 章到第 14 章把“架构”这一侧说完了：架构是什么、怎么被造出来、决策凭什么定、怎么表达成一份交得出去的描述、交出去之后设计怎么接住。可架构描述不是交出去一次就完事的——它会被改，会被迭代，会在多层级上同时被引用。于是“哪一版算数”这个问题，从交出那一刻起就悬在那里。本章办的就是它。

还有一层位置要说清：配置管理与前面各章是并行关系，不是它们的下一步。15288:2023 里，十四个技术过程，每一个都在干同一件事：“提供已选定用于基线的关键人工制品”（Provide key artefacts that have been selected for baselines；中译：提供已选定用于基线的关键人工制品）。每个过程底下都附一条注，写明“本过程识别基线的候选对象，随后把人工制品提供给配置管理”。

这十四个过程点得出名字：业务或任务分析、利益相关方需要与需求定义、系统需求定义、系统架构定义、设计定义、系统分析、实现、集成、验证、转换、确认、运行、维护、处置，一个不缺。

架构定义只是这十四个供料口里的一个，它自己能做的只到“提名候选”为止；把这些候选收下、正式化、维护成基线的，是配置管理。

与需求那一章的分工也在这里划清。需求侧的变更闭环归第 4 章：变更请求、影响分析、批准、**再验证**、关闭。本章把“冻结、变更、再冻结”这个循环推广到所有配置项，需求只是其中一项；它草稿阶段是产品数据，被批准纳入基线之后才成为配置信息。

最后一条要交代的：本章说“**能力**”，一律指第 9 章那份清单里的意思，并且具体落在那份清单的第 ⑥ 项（可复现意识）上。这里的“能力”不与第 1 章的“能力四级”（表征、图式、心智模型、解释框架）

混。这两处用的是同一个词，量的是两件事：一处量概念掌握到哪一级，一处量你手里长出了什么。

本章按下面三块走。先把三个词各管什么说清：配置、版本、基线，各就各位，近邻词当场分开（15.2）。再把最容易混的地方一条一条拆开：配置不是参数设置，版本不是基线，变体不是版本，没有基线改动就说不清（15.3）。最后进过程里看它怎么干：怎么收下判断、怎么守住基线、不做会坏在哪、它替架构定义做了什么（15.4）。

---

## 15.2 配置、版本、基线各管什么

三个词各说一件事：**配置**回答“它是什么样”，**版本**回答“走到哪一步”，**基线**回答“哪一版是定下来的那一份”。三句话不重叠，也不能互相顶替，现场九成的混乱，都是拿其中一个去答另一个的问题。

这一块要做的是：把这三个词放到各自的位置上，顺手把它们旁边最像的词分开。

### 15.2.1 配置：记下产品是什么样

现场说“改一下配置”，十有八九指的是改参数：阈值调一调、开关翻一翻、界面上的选项换一换。这个用法在日常里没错，但它不是标准里那个“配置”。

正本给的定义只有一句：配置是“在配置信息中规定的产品（或服务）的相互关联的功能特性和物理特性”（ISO 9000:2026 §3.10.8）。照这条定义，配置是**一套特性**，不是一套设置；它记的是“这东西是什么样”，不是“怎么把它调成别的样子”。阈值是可调的一个数，而“这条通道在 −20℃ 以下仍要保持连续性”是一条特性，前者随时能改，后者一旦定下来，产品就成了那个样子。

围着这句话还有两个词，三个词各有各的所指，不能混着用：

- 配置（§3.10.8），那套相互关联的功能特性和物理特性本身；
- **配置项**（§3.10.9），“配置之中满足某一最终使用功能的实体”，是配置这件东西被切出来的管理单位；
- **配置信息**（§3.10.10），“关于产品或服务的设计、实现、验证、运行和支持的**要求**”，也就是把上面那套特性写下来的那一份。

三个词在正本里是互相引用的：配置用配置信息定义，配置项用配置定义，配置信息又指回配置项。这个环形不是标准写错了，而是三样东西本来就连在一起：**特性、实体、记录**，一个产品被管起来的时候，这三样同时存在。把它们分开看，是为了回答三个不同的问题：这个东西是什么样（配置）、这个单位归谁管（配置项）、凭什么说它就是这样（配置信息）。

> 一条译名提醒：国家标准 GB/T 19000—2016 把 configuration 这一族词译作“**技术状态**”（技术状态项、技术状态信息、技术状态基线）；库内 ISO 9000:2026 中英对照本按本项目说法统一译作“配置”系。看到“技术状态”时，它说的是同一件事。

那么，架构这边到底有什么东西进得来？架构描述是工作产品（42010:2022 §3.3）。它连同决策记录、模型、视图和各份清单，都是可以被记录、被标识、被管起来的实体。要注意一句容易说错的话：进配置管理的是“描述”那一份东西，不是架构本身。架构无形而抽象，是被描述出来的；能被选定、被冻结、被引用的，永远是承载它的那份描述。

要判断一样东西算不算“配置”，就问它答的是“**它是什么样**”，还是“**它怎么调**”。答“怎么调”的，是设置；答“是什么样”的，才进得来。

### 15.2.2 什么算配置：不是所有特性都算

上一节说配置是“相互关联的功能特性和物理特性”。这句话有个反过来的读法，同样要紧：不是所有特性都算。

一个产品身上的特性多得很：外观的、成本的、交期的、供应商的、团队习惯的。它们都是事实，但绝大多数不进配置。进不来的理由不是它们不重要，而是“进配置”这个动作本身有代价。一旦进来，改它就要走门、就要留痕、就要通知拿着旧版干活的人。选择本身就是一次判断：哪一组特性值得被这样管起来。

现场拿不准的时候，可以连着问三句：

1. 它是产品自身的特性吗，不是过程的、不是组织的，也不只是某个人的做法？团队习惯换个人就变，它管不住产品。
2. 它在那一组相互关联的特性里吗，孤立的一项不构成配置；“这条通道在 −20°C 以下保持连续性”要跟供电、报警、记录一起看，才是配置。
3. 它被选定要管了吗？选定的动作必须有人做、有记录，否则“配置”只是心里的名单。

第三问最容易出岔子，因为它把**选择**这件事摆到了台面上：配置项不是从天上掉下来的，是被选出来的。选定之后要有一份配置项清单，清单上每一项都要答得出“它是什么、归谁管”。

架构这一侧的实指：架构描述（AD）连同架构决策记录（ADR）一起纳入配置管理（16 号报告 §6.2）。谁把它们交出去？按正本，架构定义过程只做一半，它“识别基线的候选对象，随后把人工制品提供给配置管理”（15288:2023 §6.4.4.3 e) 6) 与注 12）。也就是说，挑

候选的是架构侧，收下候选的是配置管理这一侧。这条分工是本章后半段的一个关键，15.4.5 会专门讲它。

一句话答不出“它在不在那份配置项清单上、谁定的”，那它多半还只是某个人的习惯，不是配置。

### 15.2.3 版本：给每一次改动编号

版本回答的是另一个问题：走到哪一步了。

正本没有给“版本”下定义——它把力气花在了一件更硬的事上：架构描述的各样构成物都得有版本标识。42010:2022 在架构视角、架构视图、视图组件这三处的条目下各写了一条：“Each architecture viewpoint shall include version identification as specified by the organization and/or project.”（中译：每一张架构视角应含由组织和／或项目规定的**版本标识**。）同一组要求另落在架构描述框架（§7.1）与模型种类（§8.2）的条目下——但那两处用的是 should（宜），与上面这三处的 shall 性质不同。

版本最朴素的样子就是编号：给每一次改动一个号，让人一眼看出先后。软件界常见的“主.次.修订”三段式（业界惯例，不是标准条文）就是这种编号法的一个例子。改了不兼容的东西进主号，加了功能进次号，修了缺陷进修订号。这套编号规则怎么定并不重要，重要的是它给出了一件东西：位置。

位置能回答的问题只有一类：“这批记录是哪一版采的？”“你看的是哪一版描述？”——它答得出先后，答不出资格。最新版不等于算数的那一份。最新版只是“到目前为止走得最远的一版”，它有没有被批准、有没有被冻结、能不能拿来当参照，版本号对这些一个字都不说。

先问“**这批是哪一版**”，再问“**凭什么算数**”。第一问答得上、第二问答不上，就是下节要讲的那种状态，有版本管理，没有配置管理。

### 15.2.4 基线：被批准、被冻结，能当参照

基线回答第三件事：哪一版是定下来的那一份。

正本给了两句话，一句从“版本”那边说，一句从“信息”那边说。15288:2023 §3.8：基线是“配置项不论介质如何、在配置项生存周期内的特定时刻被正式指定并固定的、经正式批准的版本”（该条出处为 IEEE Std 828-2012《系统与软件工程配置管理标准》）。

ISO 9000:2026 §3.10.12：配置基线是“经批准的配置信息，它确定产品或服务在某一时间点上的特性，并作为该产品或服务生存周期内各项活动的参照”（正本此句中译作“生命周期”，本书按定译作“生存周期”）。

两句话并排读，基线有三个动作，缺一个就散架：

- **批准**，它得有一个有权威的批准动作。批准给的是**资格**：从这一刻起，这一份可以当依据用。
- **冻结**，它得在某个时间点上被固定住。冻结给的是**状态**：此后任何改动都是“对这一版的偏离”，都得走门。没有冻结，批准只是一句好话。
- **参照**，它得真的被拿去当参照。参照给的是**用途**：别人引用、比对、追溯时指向它。没有参照，冻结也就守不住。

三个动作里最容易漏的是第三个。现场常见的场面是：会开了、字签了、文件归档了，然后大家接着用各自电脑上那份“顺手改过一点”的版本干活。批准和冻结都有，参照没有，基线于是名存实亡。

基线的对象也要说准：被冻结的永远是承载内容的那一份东西，不是内容背后的实体。架构这一侧最硬的一个例子，是把架构交出去时附的那套凭据：接口定义基线、约束条目表、追溯矩阵、基线版本号。这四项凭据合起来必须纳入配置管理，随架构描述那一版基线同版冻结（24 号报告 §7.7）。

道理很直白：架构描述一动，照着旧条目做的设计就成了“依从一个已经作废的版本”，双方都觉得自己没错，账却对不上。

还有一条边界必须说清：基线承载的是“明示的”那一部分。评审与验证的判据都长在这条链上，所以基线撑得住它们。确认（validation）撑不住：它问的是“这东西符合不符合它的预期用途”，判据是使用情境，不在链上。

所以需求基线之上还得另有一层参照来装确认的判据，连来源指纹（谁提的、谁批的、什么时候）也得挂在那层上（23 号报告 §9.1、§9.2）。这条边界不影响本章的做法，但它挡住了一个很容易犯的错：把基线当成“什么都能判”的东西。

看一样东西是不是基线，就问它有没有一个有权威的批准动作，此后改动是不是**必须过门**。

### 15.2.5 先分清配置基线与产品基线

正本讲基线时用了两个名字，各自对着一前一后两步。

**配置基线**出现在预期结果里：“配置基线（包括已批准的配置）得到维护”（15288:2023 §6.3.5.2 b)）。它管的是受控内容本身，配置信息和它的基线被建立起来、被维护住。

**产品基线**出现在活动的末尾：“执行配置及配置管理验证活动与审核，以建立产品基线”（§6.3.5.3 e) 4)）。它管的是“能不能定型”：审核做完了，产品这一版才可以建立起来，进入发布。这一条底下还附了一条注，把两种审核点了名，**功能配置审核**（functional configuration audit，FCA）看功能与性能能力，**物理配置审核**（physical configuration audit，PCA）看系统对运行信息部件与配置信息部件的符合性；执行审核的是验证过程（注 22）。

两个名字分别对着“内容受控”和“产品定型”，一步在前、一步在后，不要混着说。

这里还要防一个误读：FCA 与 PCA 不是两种新的验证。它们是在配置管理这一侧对“配置与配置信息”做的审核，执行审核的仍是验证过程（注 22）：审核这一动作归验证，看的对象与判据归配置管理。同一份标准里，验证问的是“它符不符合它的要求”，确认问的是“它符不符合预期用途”。两种配置审核都落在前一件事上，不越到确认那一侧去。

还有一套名字要交代清楚：“功能基线、分配基线、产品基线”这三分法是行业惯例，不是标准术语。这类三分法在标准里找不到落点：15288 附录仅把 SAE EIA-649 列为配置管理附加指南。本章因此不拿它当文本骨架讲。讲哪一套，都得先说自己讲的是哪一套的依据。

为什么这个区分对架构这一侧要紧？因为它决定了架构与设计交出去的那几份东西在哪一步被收进去。描述本身走的是配置基线这一步；产品定型那一步要审的，已经不只是描述，还有照它做出来的实物与信息部件。

听人讲“基线”，先问一句：他说的是内容受控的那一步，还是产品定型的那一步。

这一块落到你身上，是两句话。对**架构定义**，配置、版本、基线三样东西合起来回答了一个问题：交出去的那一份，凭什么一直被当作那一份。没有这三个位置，架构判断就只是一次发言，换个场合就要重说一遍。

对**你**，这一块量的是第 ⑥ 项可复现意识的第一层：你手上那几份东西，位置摆正了没有。它是什么样（配置）、走到哪一步（版本）、哪一版算数（基线），三问答得出，才谈得上“换个人也接得住”。三问里有一问答不出，能力就还停在你自己手里。

---

## 案例进展①｜冰链：版本标着，基线没有

季昭把一张参数清单拍在桌上：“上报间隔、告警阈值、断链判定，都在这张清单上。参数齐了，配置就算清楚了吗？”

林涛看了一眼那张纸：“参数是配置信息里的条目。你这一页答'怎么调'，配置答'这台设备是什么样'。”

三个月后，审计员来了，桌上摊着一份疫苗运输记录：四月运输，全程温度记录正常。

审计员说：“请你们证明一件事：这批记录，是在哪一版固件下采集的？从那批之后，采集逻辑、断链判定有没有升过级？”

石军答得快：“查一下这批设备当时跑的固件版本就行。”

查不清。升级记录上只有一句“已推送 v2.3”，没有“哪台设备、什么时间、从哪一版升到哪一版”的明细。采集算法还改过一次：v2.1 与 v2.3 的断链判定不一样，同一条温度记录算不算“断链”，取决于它是哪一版固件采出来的。

季昭把那本测试记录翻出来，一页一页对了一遍：“同一份记录，落在这三个版本上过。测的是哪一版，我这儿也答不上来。”

小周翻开产线台账：“出厂时每批设备烧的哪版固件，这儿都有账。出了厂，推送升过几回、哪批升了哪批没升，这边一概没有。批次要追的不是'设备没坏'，是'这一刻它在哪一版上'；中间那根线，两头没接上。”

石军觉得冤：“我们有版本号啊：v2.1、v2.2、v2.3 都标着，升级也走了流程，季昭也验证过。这怎么就不算配置管理了？”

季昭问的是另一句：“这批记录，是拿哪一版采的？”

“v2.1 到 v2.3 之间……”石军停住了。他答不出一个具体版本。

季昭说：“这就是审计员要的那句话。版本号给的是位置，基线给的是资格。你的版本号都在，可没有哪一版是被批准、被冻结、被拿来当参照的——查得清改到第几步，查不清哪一版算数。”

小张把两份规格推到中间：“我们每一段都说得出'我按哪一版做的'，没有一段说得出'那一版是谁批的、什么时候定的'。批准、冻结、参照，这三样一样也没有——不是技术没做完，是没人把它收下、定住。”

赵刚说了一句：“卖出去时我们讲'全程可追溯、断链必告警'。今天连自己都追溯不清——这句话还怎么站得住。”

石军在纸上画了一道门，门口写两个问题：谁批的？什么时候？

---

## 15.3 别把配置、版本、基线混着用

上一块把三个词放到了各自的位置上。这一块反过来看：它们各自最容易被谁顶替。四节，四句最容易脱口而出的话，一句一句拆。

### 15.3.1 配置不是参数设置

“改一下配置”——这句话在现场等于“改个参数”。要拆开这一句，只要问一句：你改的是那一个数，还是这台设备的样子。

参数是配置信息里的一个条目，它被记录在案，当然属于受控范围。但配置不是那一个条目，而是那套关联起来的特性。上报间隔、告警阈值、断链判定逻辑、采集算法，它们连在一起，定出这台设备“是什么样”。改一个参数，变的不是那个数，是这台设备的样子，连它采出来的温度记录该怎么读，都跟着变。

顺带说清参数的位置：参数是配置信息里的一条，记下来、控住了；可配置信息不止参数，还有关于设计、实现、验证、运行与支持的那些要求。把配置等于参数，等于只认了这本账里的一页。

所以审计员问“这批记录是哪一版固件采的”，问的不是参数，是**形态**：哪一套采集逻辑与判定逻辑组合出来的行为，采出了这批记录。参数是随时能翻的开关，配置是“它当时是哪副样子”。

这一处同名不同义，会在另一类系统上换个样子再出现一次。平台类产品的现场常说“配置出一堆孤岛”。某个业务域图省事，在**配置里**自建一份他方权威数据的副本，或者引入平台没注册的字段（32 号报告 §C.3.11）。

那里的“配置”指的是平台的**参数化配置**，也就是可配置性带来的那一套设置；它跟本章讲的“配置”不是同一个东西。同一份产品上，两个“配置”同时存在：一个答“它是什么样”，一个答“它怎么调”。混着说，就会推出“配置管理就是管平台参数”这种结论，而它恰好把最要紧的那一半漏掉了。

把“配置”当参数的人，改完只会说“我把阈值从 3 改成 5 了”；知道配置是什么的人，会补一句“这一版的样子，从今天起变了”。

## 15.3.2 版本不是基线

版本与基线都在时间轴上，这是它们最像的地方，也是这两个词最容易被混着用的原因。分清它们只要看一件事：它回答的是“走到哪儿”，还是“哪一版算数”。

版本顺着时间走，一个一个排下去，像尺子上的刻度；基线是竖直切一刀，把某一刻固定住。一横一竖：

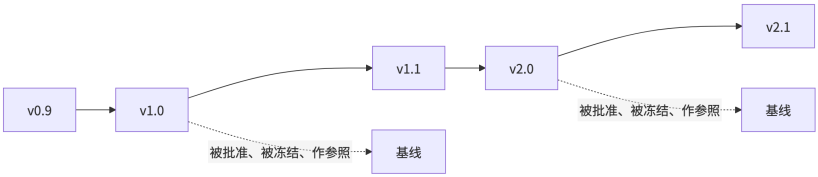



图 15-1

图里那两条虚线是全部差别所在：刻度是连续的、彼此无差别的；基线是从刻度上被挑出来的、带资格的那一个点。所以“最新版”顶不了基线：最新只说明它在最右边，不说明它被批准过。

这两样为什么总被合起来说？因为日常工具把它们摆在了同一个界面上。版本控制工具（Git 这一类）既能记下一次次改动，也能给某一版打个标签，于是很容易滑出一句“我会用版本控制工具，就等于我会配置管理”。

工具给的是记录位置的能力，这只是那必要的一半。从位置到资格还差三个动作：有人批准、在某个时刻固定住、此后谁都拿它当参照。工具替你记住走到哪儿，替不了你决定哪一版算数。

拿两个问题连着问。第一问“这批是哪一版”，第二问“哪一版算数”。只答得出第一问，就是有版本管理、没有配置管理。

### 15.3.3 变体不是版本

第三个容易混的词是“变体”。变体与版本的关系不是先后，是两个方向：变体是**空间**，同一时刻并存的好几种形态；版本是**时间**——同一条线上的一次次改动。

M3 有两种机型：一种跑 −20°C 的冷链，一种跑 2~8°C 的冷藏。它们是**变体**，同时存在，谁也不是谁的升级。而 −20°C 那一型从 v2.1 走到 v2.3，走的是版本。把变体当版本，最典型的一句话是“这台是 v2.3、那台是新版本”，其实那台可能是另一个变体的第一版。

混起来会有什么后果？两条线叠成一条，追溯的时候谁也说不清“这台是哪一支上的第几版”。变体各有自己的配置项，各自往下走自己的版本线；基线也各自立。变体管“哪一种”，版本管“哪一次”，基线管“哪一版算数”，三样东西各占一个位置；缺一个，另外两个就顶不住。

问那两样东西是“同时存在”还是“先后出现”。同时存在的，是变体；先后出现的，是版本。

### 15.3.4 没有基线，改动说不清

三个词的边界划完，最后说一件后果：没有基线，“变了什么”这句话就说不出口。

“不做”有两种，它们看起来一样，坏起来不一样。第一种是**全无**：没有版本、没有清单，改动随手就来。这种坏法一眼能看见，所以不常见。第二种更常见也更麻烦：有版本管理、没有配置管理。版本号标得整整齐齐，日志记得清清楚楚，可没有哪一版被批准、被冻结、被拿去当参照。上一节那句“查得清改到第几步，查不清哪一版算数”，说的正是这一种。

第二种为什么更危险？因为它看起来是做完的。查得到版本号、调得出升级记录，谁检查都能过关。可一旦要回答“这批东西是按哪一份做的”“改动是从哪一版开始的”，整条链就断在那里。冰链四月那批疫苗记录，坏的就是这一处。

还有一种说法要拆开：“我们把东西都存档了，这就算配置管理。”存档只做了一件事——留了一份副本。配置管理要做的是另外三件：建参照（这份副本从哪一版来、能指回哪一版）、设门（改它必须过哪一道手续）、能追（从任何一份追到“凭什么改成这样”）。存档不建参照，也不能设门，更留不下理由：副本越多反而越难找，谁都能往库里丢一份，追到最后只剩一句“就是这份”。

一句话判断：改它要不要过门？出了问题追不追得到“凭什么”？两个都答不出，那只是留了副本。

一眼看上去版本齐全的现场，再问一句“哪一版是**定下来的**那一份，谁批的、什么时候”，答不出，就是第二种。

这一块落到你身上：对架构定义，把三个词分开，等于把“哪一版算数”这句话的前提补齐——前提不齐，后面的正式化（见 15.4.1）、冻结、引用都无从谈起。对你，这一块量的是 ⑥ 的第二次机会：你分得清“走到哪一步”和“哪一版算数”吗。分不清的人，会把最新版当依据交出去；分得清的人，交出去的每一份都带着“它凭什么算数”的落款。

---

## 案例进展②｜恒信：另存为 v2

几乎同一段时间，恒信踩进了同一个坑，只是这一次，版本号连样子都没有。

合同审批系统上线半年，运维只有王默一个人。每次发布，他的做法是把构建产物扔进按日期命名的文件夹：“2026-05-12”“2026-05-18”。合同管理模块改一次字段校验，他把新包打进“2026-05-18”，旧的懒得删。

两个月后，一个合同状态流转的缺陷暴露了。系统架构师罗建：“回滚到上上个版本。”王默打开“2026-05-18”——里面躺着两个包：“合同系统_final.zip”和“合同系统_final_2.zip”，分不清哪个是哪个。

罗建：“我们有版本号啊。”

需求分析师顾言把两个包的名字并排看：“你这里有三个东西被混成了一个：文件名、版本、基线。文件名的先后不是版本——版本要能精确指到'哪一次完整构建'：这份产物由哪个提交编译出来、含哪几条变更。'final_2'既不知道是哪次构建，也不知道有没有经过批准，更不知道和上一版差在哪。它连版本都算不上，更谈不上基线。”

罗建看着那个文件夹：“所以出问题要回滚，我连回滚到哪儿都指不出来。”

“对。”顾言说，“版本是回滚的地址，基线是变更的参照。你只有日期文件夹，地址和参照都没有——两个月前还能说的'变了什么'，两个月以后就说不清了。”

后来恒信补上了发布记录：一个包对应一个版本号、一个提交号、一版需求基线和一份变更记录。从此“上上个版本”在恒信变成了一句可执行的指令。

罗建翻这套文件时自己都笑了，第 4 章合同需求靠“v2.doc”“最终版 v3.doc”管版本，如今系统部署靠“final_2.zip”管版本，同一个毛病换了个文件后缀，又犯了一遍。

---

## 15.4 收下判断，守住基线

前两块把三个词各管什么说清了。这一块进过程里看：一句话怎么从“某人说出来”变成“一份受控的东西”，又靠什么一直守得住。九节，从那个不在技术过程里的动作开始。

### 15.4.1 正式化：把草稿定成基线

正本把这件事说得很直白。15288:2023 §6.3.5.3 b) 4) 底下有一条注：

> 基线在指定时刻或规定情况下捕获系统元素不断演进的配置状态。基线的内容通过技术过程形成，但通过配置管理过程在某一时点予以正式化。基线构成下一次更改的基础。（注 8）

原文是：“The content for the baselines is developed through the technical processes, but is formalised at a point in time through the configuration management process.”（中译：基线的内容通过技术过程开发，但在某一时点经由配置管理过程予以正式化。）这句话里有一个转折：内容由技术过程开发，**正式化**不在技术过程里。

架构定义过程把内容做出来：概念、原则、属性、视图、决策记录，都是它的产出。它做不到的事只有一件：从哪一刻起，这一份算数。那个动作归配置管理。41 号报告给这个动作定了一个词，正式化（41 号报告 §6.6），它一次含三件事：获批、冻结、作参照。

三件事在 15.2.4 讲过，这里要说的是它们的位置：正式化不是技术过程里的一个内部步骤，它是技术过程与配置管理之间的那道交接动作。所以“我们把架构定稿了”和“架构基线建立了”不是同一件事，前者说的是内容做完了，后者说的是这一份被收了、被固定了、可以当依据引了。

同一个动作，宾语可以不同。对一份架构描述正式化，得到架构基线；对一份需求规格书正式化，得到需求基线；对治理指令正式化，得到治理那一层的基线。动作只有一个，收进去的东西各不相同。

顺带划一条界：正式化不等于技术评审。评审评的是内容够不够好，方案对不对、判据全不全、取值有没有依据。正式化管的是这一份东西的资格：从哪一刻起它算数。

评审合格是正式化的常见前提，但它不等于正式化——一份评审合格的草稿仍然是草稿，直到有人把它收进受控范围、固定下来。反过来也成立：一份已经被正式化的东西，内容上仍可能在下一轮评审里被推翻，那时走的是“变更”这条路（15.4.3 的更改管理），不是“重新正式化”。

两件事各有各的判定，混在一起，就会出现“评审过了怎么还不算数”这类反复出现的争论。

问一句“这一份是什么时候、通过哪个过程变成算数的那一份的”。答“写完就发下去了”，说明正式化这个动作从来没发生过。

### 15.4.2 定稿之前就受控

现场有个很常见的想法：先做、做完、定稿，定稿之后再“交给配置管理去管”。这个次序听着合理，但正本给的不是这个。

42020:2019 在架构描述的预期结果里写了一条：“架构描述处于配置控制之下，并提供给所有相关方”（§10.2 e)）。同一份标准在架构细化过程里要求：确保架构描述处于适当的变更控制之下（§10.4.2 h)），并以配置管理过程为指南管理架构描述、视图与模型的变更（§10.4.2 n)）。细化过程还要求把产出置入存储库（§10.4.4 m)），而存储库的条目“宜处于配置控制之下”（§3.19 注 1)。

这几条连起来是一个次序：受控与编写同时开始。道理不难想，架构描述在被写的过程中就已经被人引用了。设计那边等你这一版开工，采购那边照着上一版询价，评审那边按某一版出纪要。如果它到定稿才受控，中间那一段里所有的引用都是没有落款的引用。

由此还带出一条纪律：改动不是“定稿之后才需要通知”，而是边写边通知。谁在按这份描述干活，谁就要在改动发生时被通知到；等到定稿再统一发布，等于让所有人先按一个会变的版本干了几个月。

顺带说清一件事：**在过程中受控**不等于“每一处改动都要开会”。受控的对象是那些会被别人引用的东西，控制的方式可以是登记、通知与版本标识，不必事事上升到评审。反过来，凡是已经有人在照着它干活的东西，哪怕它还叫草稿，也得先纳入受控范围。

问一句“这份描述在定稿之前，受不受控”。答“定稿之后才管”，说明中间那一段在受控之外。

### 15.4.3 配置管理平时在做什么

正本给这门过程一句目的：“配置管理过程的目的是在系统与系统元素的生存周期内管理其配置”。管理包括“建立并保持一致性、完整性、可追溯性与控制”（15288:2023 §6.3.5.1）。

它成功执行的结果，标准列了六条：系统与系统元素的配置得到管理；配置基线（包括已批准的配置）得到维护；纳入配置管理的项的更改得到控制；配置状态信息可获取；所需的配置审核得以完成；系统发布得到批准（§6.3.5.2 a)~f)）。六条都是当场可以检查的状态，不是态度。

平时做的活，标准归成五项（§6.3.5.3）：

- **准备**：定角色、职责、问责与职权，定要建立哪些基线，定存储的位置与条件，定“何时启动配置控制、何时维护基线”（a) 项下各任务）；
- **配置标识**：把哪些东西选进来、怎么标识、怎么描述（b)）；
- **更改管理**：识别并记录更改请求与偏离请求，协调、评估并处置它们；偏离请求有时也叫偏差、豁免或让步（c) 项与注 13）；
- **配置状态记帐**：记录并报告配置信息、所提议更改的状态、已批准更改的实施状态（15288:2023 §6.3.5.3 d)；该句逐字引自 ISO 9000:2026 §3.10.11）；
- **验证与审核**：验证配置满足要求，监视已批准更改的纳入，并执行审核以建立产品基线（e)）。

这五项不是并排的五个工作包，它们串成一条线。

准备决定这门过程在这里怎么落地：谁有权批准、哪些东西要立基线、存在哪、什么时候开始控制。它最容易被跳过，因为它不直接产出内容；可它一缺，后面四项就都成了临时应付，临时选配置项、临时定门、临时补记录。

**标识**把东西从“大家心里知道”变成“清单上写着”：选进受控范围、给标识、写描述。做完这一步，才有“改的是哪一份”这种说法；没做这一步，讨论改动时连宾语都指不出来。

更改管理是这条线上唯一一门管住入口的活：它不产出内容，它决定内容能不能进来、怎么进来。常规更改与紧急更改走的是同一条门，区别只在时限，不在门槛，紧急请求可以快，但不能绕。

**状态记帐**让“现在是什么状态、曾经是什么状态”随时查得到。它不是给人看的台账，是给追溯用的底账：出了问题要精确召回哪几项，答案就在这一项里；这一项没做，就只剩全量召回。

验证与审核是这条线的收口：内容做完不等于产品能定型。验证配置满足要求、监视已批准更改的纳入，最后执行审核，这才把产品基线建立起来。

两处细节值得单独点出来。第一，更改请求与偏离请求常常处于配置控制委员会（CCB）的正式控制之下，评估包括“需要与影响”的分析（注 15）。准备活动里也写明，更改管理包括已计划的配置控制委员会、常规与紧急更改请求（a) 项 g)）。

第二，审核有两种典型：功能配置审核（FCA）聚焦功能与性能能力，物理配置审核（PCA）聚焦系统对运行信息部件与配置信息部件的符合性。执行审核的是验证过程（注 22）。

这里要拆一对最容易混的词：**配置审计**与**审核**不是一件事。前者的对象是配置项与产品，判据是基线与配置信息，问的是“实物与文档对得上吗”（15288:2023 §6.3.5.3 d) 注 19）。后者的对象是组织与体系，判据是审核准则（ISO 9000:2026 §3.12.1），问的是“体系按不按自己定的规矩在跑”。两个词只共用了中文一个“审”字。

最后记一句译名：同一件事，15288:2023 的中译作“配置状态记帐”，ISO 9000:2026 的中译作“配置状态纪实”；国家标准 GB/T

19000 系把 configuration 一族通译为“技术状态”。本章随 15288 用“记帐”。

把五项活动对着现场点一遍：哪一项没人做，或者只做了个形式，那里就是下一次事故的入口。

### 15.4.4 变化总得跟基线比

“改了”这个词，只有在有一个可指的上一版时才有内容。没有参照，“改了”能指的东西太多：改了什么、改到哪儿、改完之后跟谁对得上，每一问都落空。

41 号报告给了三个可核的问句，合起来就是参照存在的判据：上一版是哪一版？谁批的？什么时候？（41 号报告 §4.1）

三问里任何一问答不出，参照就不存在；后面所有关于“变化”的说法，都是空转。

基线在过程里的位置也正在这里：它是“下一次更改的基础”（15288:2023 §6.3.5.3 b) 4) 注 8）。这句话两头都管，变了什么要跟它比，变了之后又要以它为新起点。所以基线不是终点，是每一次循环的基准点：冻结、变更、再冻结，一轮一轮往前走。

现场有三种东西常被当成参照，其实都不是。第一种是“最新版”：它给位置，不给资格（15.3.2 已拆过）。第二种是“我桌上那份”：一份没批准、没标识的描述，在别人那里根本指不到。第三种最像参照——“会上说过的那版”：纪要写着“原则通过架构描述 v1.4”，可 v1.4 有没有正式化过、改动有没有纳入、纪要之后又改没有，纪要一概不管。

这三种有个共同点：它们都能被人指着说“就按这个”，但都不能被别人指回来。参照之所以是参照，靠的是**双向可指**——你能指着它说“按这个做”，别人也能从它指回来确认“就是这一版”。单向的指认，只是某个人的记忆。

听人说“我们改了”，紧接着问那三问。答得出，变化是事实；答不出，变化只是一个印象。

### 15.4.5 候选是谁挑的，基线是谁定的

这是本章最要紧的一条分工，也是“十四个供料口”那句话着地的地方。

按正本，架构定义过程在做完自己的活之后，还有一件收尾动作：识别基线的候选对象，随后把人工制品提供给配置管理（15288:2023 §6.4.4.3 e) 6) 与注 12）。设计定义过程也一样（§6.4.5.3 注 20）。也就是说：挑候选的是架构与设计，收下并正式化的是配置管理。

这条分工常被读成“架构交出去就不管了”。它其实说的是另一件事：判断与正式化是两件事，分属两只手。架构侧能做的是判断——哪一份够格当候选、哪一份还不够。它做不了的事也只有一件：从这一刻起，这一版算数；那要有批准的权威与受控的记录，属配置管理。

分清这一条，现场那句“我们评审通过了，怎么还说不算数”就有答案了。评审过的是它的内容够不够好，正式化管的是那一份东西的资格。内容合格但没被收进基线，它仍然不是别人可以引的依据。

具体递过去的是什么？正本的说法是“已选定用于基线的关键人工制品”（15288:2023 §6.4.4.3 e) 6)）。落到本书讲过的那些东西上，架构这一侧就是这几样：架构描述本身（含各张视图与模型）、架构决策记录与它的理据、各份清单（关切台账、视角清单、对账表这一类）。设计这一侧照同一句式（§6.4.5.3 注 20）：系统设计描述、设计侧的决策与理由、设计特性的取值与依据。

要留意这一递不是“交出去存档”，它是进入受控范围的第一步：递过去的每一份都要先答一句“它够不够格当候选”。不够格的留在过程里继续改；够格的交给配置管理，由那边完成正式化。

那么，谁有权批这一版？这一问本章不答，它与“谁有权建立配置控制机制、谁裁定架构集合中的变更”是同一个问题。这一问归架构治理（42020:2019 §6.4.2 k)），本书在第 16 章讲。本章只把候选挑出来、递过去：批不批、谁批，由治理那一侧定。

一份东西被拒了，先问它被拒在哪一环，**内容不合格**，还是**没被收进基线**。这两件事的补救办法完全不同。

## 15.4.6 不做配置管理会怎么样

把 15.4.3 那六条预期结果一条一条反过来读，就是“不做会怎样”的清单（41 号报告 §7.2 的做法）：

- 配置得不到管理，谁在改、改成什么，没人说得清；
- 基线得不到维护，“哪一版算数”没有答案，引用只能凭记忆；
- 更改得不到控制，改动绕过门口，事后才知道；
- 配置状态信息不可获取，查不到“现在是什么状态”，也查不到“曾经是什么状态”；
- 审核做不了，没有基线，审核没有比对的对象；
- 发布得不到批准，没有可批的那一份，只能是“先发出去再说”。

比“完全没做”更常见的是**做了一半**，而且每一半都能报出成绩。三种典型：

- **有标识、无控制**：清单立起来了、标识也给了，可改动不进这道门，清单上写的“当前版本”随时过期；
- **有控制、无记帐**：改动确实走门了，走完之后没人记录状态，问“现在是什么状态”要靠回忆；
- **有记帐、无审核**：日志记得很全，可没有人对着基线核一遍实物与文档，日志与事实之间缺一道校验。

这三种有个共同特征：对外都说得出一套流程，对内却答不出“当前算数的是哪一版、凭什么是它”。所以现场核对别问“有没有做配置管理”，问那几项活动里哪一项没有真实发生。

上面六条预期结果对应的风险不是并排的，它们有先后。41 号报告把缺位形态归成四类，每一类都指出最先坏掉的一处，并配了一句可以直接拿去问现场的自检（41 号报告 §7.5）：

| 缺位形态 | 最先坏掉的一处 | 一句自检 |
| --- | --- | --- |
| 无标识 | 引用，指不认某一条决策或某一版视图 | “上一轮讨论的是哪一版?” |
| 无基线 | 参照，评审与验证找不到对象 | “上一版谁批的？什么时候?” |
| 无变更控制 | **影响评定**，改动波及哪些下游，没人算 | “这次改动波及哪些下游?” |
| 无状态记帐与审核 | **追责与发现** | “它是怎么变成这样的？谁核过实物?” |

四句话当场问一遍，哪一句答不出来，缺的就是哪一类。

而四类缺位各有一处最先坏掉的地方。落在“无基线”这一类的，是**参照**：一旦“哪一版算数”没有答案，追溯就断了起点，评审没有比对对象，变更没有参照（41 号报告 §7.5）。所以现场核对不必六条全查，**先查起点**。

不做配置管理的现场，第一个症状不是“乱”，是“**说不清**”：每个人手里那份看起来都成立。

### 15.4.7 设计依从的是架构基线

现在可以回答第 14 章交下来的那一问了。

**架构基线**，就是架构描述被选定并冻结的那一版。这里要把对象说准：被冻结的是描述，不是架构本身——架构无形而抽象，能被选定和冻结的永远是承载它的那份东西。跟着一起进去的还有那份描述所依据的记录：架构描述与架构决策记录一起纳入配置管理（16 号报告 §6.2）。第 14 章已经说过，架构描述是“可基线化的工作产品”（35 号报告 §十七）。

正本对“基线架构”这个角色有一句很准的话。它“充当同步关键点，用于架构的评审以及将架构作为参考予以提供”（角色依据＝42020:2019 附录 E.4.1.6；“选定并冻结”的资格依据＝15288:2023 §3.8）。两个用途——评审的同步点、对外提供的参考——正好对着“设计依从的是哪一版”这一问题：设计依从的，是那一个同步点上的那一版。

所以第 14 章那句交接句的答案是：设计依从的是架构基线。这也解释了为什么“最新版”在这里格外危险。设计手里的版本如果比基线新，它可能在依从一个还没被批准的改动；如果比基线旧，它在依从一个已经作废的版本。两种错都不在内容上，而在资格上。

这句话在实践里有个很具体的用法：评审要写清“按哪一版评的”，引用要写清“引的是哪一版”。评审纪要上只写“架构评审通过”，等于把一个同步关键点丢掉了；三个月后有人问“当时是按哪一版评的”，只能靠回忆。基线架构的意义正在这里：它把“当时”固定成一个可以回指的时点；顺着这个时点，追溯与决策记录才有地方可挂。

设计说“我按架构做的”，补一句“按哪一版、那一版是不是基线”。

### 15.4.8 配置管理替架构定义做了什么

按 41 号报告的归纳（§四），配置管理替架构定义做的事是这几样：

- 给判断一个起点，把“某一时点的一组配置信息”变成经批准、被固定、可作参照的基线。没有它，所有涉及“变”的说法都没有参照系；可核的问句就是 15.4.4 那三问。
- 给迭代、递归与并发一个交点：技术过程在层级上层层递归、在时间上并发推进，多层级同时改的时候，“父架构是哪一版”必须有答案，否则分配与追溯失去共同对象。
- 补上“交得出去”的另一半：描述内容本身合格，只做到了其中一半；能不能让人确认“拿到的是算数的那一份”，得靠补齐动作，把描述置入存储库（42020:2019 §10.4.4 m)）。
- 给追溯与决策记录一个可核的挂点：架构实体与需求、接口、分析结果、验证方法之间的双向追溯要有地方可挂（15288:2023 §6.4.4.3 e) 5) 与注 11）；决策记录也一样，它得有一版可指的描述作依托。

第四件事值得多说一句，因为它把第 12 章与本章接上了。第 12 章讲过，决策必须记录、理据随行，记录里有一套内容项。本章加的是：那份记录本身也要被收进受控范围，跟着某一版描述一起定格。两章之间因此有了一张对照：

| 第 12 章（架构决策那一侧） | 本章（配置管理这一侧） |
| --- | --- |
| 决策记录的内容项 | 配置信息；记录本身就是候选物之一 |
| 关键决策的冻结 | 基线；把它固定下来的那个动作 |
| 未决议题与显式推迟 | 变与不变的分离：可变部分外置，留出调整空间 |
| 变更影响分析 | 更改管理；影响分析接的是下游约束那一层 |

表里左边这一列，第 12 章都讲过；右边这一列是本章在那一侧补上的位置。两章不是一回事，也不互为下一步，它们各管住判断的一面：一面管“凭什么这么定”，一面管“定了之后哪一版算数”。

最后是一笔账，也是这四件事里最实的一笔：精确召回与全量召回（04 号报告 §14.3）。有基线、有状态记帐，出问题时能精确召回受影响的那些项；没有它们，只能全量召回，把某一时段的所有东西一起退回去。两者成本差一个量级，而且全量召回会顺手把不该退的改动一起退掉，制造出新的问题。这笔账不需要说服力，只要算一次。

还有一层是时间尺度。41 号报告把这项作用沿生存周期排过一遍：早期阶段，它的重心偏“控制”，把判断固定住、把改动管住。到了运行与维护阶段，重心转到“取证”，出了问题要能证明“当时是哪一版、谁批的、凭什么”（41 号报告 §11.3）。

随之有两条规律。第一条，记录必须比产品存续得更久（15288:2023 §6.3.5.3 d) 注 18）：产品退役了，记录还要在，因为审计与追责往往发生在后头。第二条，同一件记录在不同阶段服务不同的读者：设计阶段服务设计者，运行阶段服务运维与审计（41 号报告 §11.5）。

这也是“存档”与配置管理看着像、却不一样的一个地方：存档的寿命通常跟着项目走，配置记录要跨过项目。

这一块落到你身上：对架构定义，这四件事合起来说的是，它把“判断”变成“可以一直被引用的东西”。对你，这是第 ⑥ 项的制度形态：你交出去的每一份判断，都有版本、有基线、改了说得清。第 9 章留的那个检查点在这里答得上了：你休假两周，这条线停不停？停，说明判断还在你脑子里；不停，说明它已经落在受控的东西上。

### 15.4.9 认知反转

本章要翻的直觉，其实是两个一起翻。

第一个是“配置管理是行政事务”——管参数、存档、走流程，谁做都一样，做不做都行。翻过来是：它是一项能力，具体说就是那道“把内容变成算数的那一份”的动作。

这个动作不在技术过程里，技术过程做不了它；它没做的时候，技术过程产出的一切都停在“草稿”这一档上：交不出去、引不了、追不了。把它当手续，赶工期时就会第一个砍掉它；把它当能力，就知道砍掉的是“别人还能不能接着用你的东西”。

第二个是“可复现就是把文档写全”。翻过来是：可复现说的不是文档写得全不全，而是一件东西的状态——它有版本、有基线、改了说得清。文档写得很全但版本对不上，出问题时照样召回不了；文档写得不多但位置摆正了，换个人照样接得住。本章量的从来不是“写得够不够多”，而是“位置摆得对不对”。

顺带翻掉一个更小的直觉：“这是配置管理员的事”。配置管理员这门岗确实存在，他管的是这门过程的运转。可本章讲的那些判断——哪一份够格当候选、哪一版该被正式化、这次改动该不该进门——做的人却是交出判断的那一个。把“摆正位置”这一步整个交出去，也就把“别人还能不能接着用你的东西”一起交出去了。

两个直觉一起翻过来，本章那句话才立得住。配置管理不改你的架构，它只把“哪一版是定下来的那一份”固定下来；而“固定下来”这件事，正是判断从一个人手里走到组织手里的那一步。

---

## 案例进展③｜冰链：季昭给测试记录加基线栏

审计过去一个月，季昭把测试记录的模板改了一处：在“结论”旁边加了一栏“基线”。

这一栏填三样：对哪一版测的（版本标识）、那一版是谁批的（批准）、测的时候这一版处于什么状态（冻结）。第一次填的时候，测试组有人笑：“这不就是多加一道手续。”季昭没解释，只把审计员那句话抄在栏头——“这批记录，是在哪一版固件下采集的?”

三个月后，M3 二代的一次内部评审上，有人问：“这批测试结论，是针对评审前那版描述做的，还是评审后改过的那版？”

会议室里安静了一下。上一次这样的问题，答案要翻半个小时的邮件。这一次季昭把测试记录翻开，指着那一栏：“三月十四号那版，评审前定的基线，测完没动过。”

“这一栏补的是什么？”有人问。

“第 14 章问的是'设计依从的是哪一版'，到了测试这儿，同一个问题换了个样子：'这份结论是对哪一版测的'。”季昭说，“基线这一栏补上，这个问题当场就答得出来——也就是第 9 章那句话：你不在场，它照样接得住。”

林涛在会后说了一句：“你这一栏补的不是手续，是别人敢不敢用你的结论。”

季昭后来说，他改模板时想的是审计员，改完才发现受益的是自己：“'测过了'这三个字，我以前当成结论；现在知道它还缺一半——对哪一版测的，也得写下来。”

那位当初笑他的同事，第二个月把这一栏抄进了自己那份检查表。

---

## 小结

本章讲的是一件不显眼、却决定别人能不能接着用你东西的事。

**配置、版本、基线**各占一个位置：配置回答“它是什么样”，版本回答“走到哪一步”，基线回答“哪一版算数”。位置分不清，现场就会出现两种最常见的坏法：把参数当配置，把最新版当基线。这三个词最容易混的是四处：配置不是参数设置、版本不是基线、变体不是版本，以及没有基线那句“改了”就说不出口。

配置管理这门过程，做的是把内容变成算数的那一份：技术过程开发内容，正式化在某一时点把它固定成基线（15288:2023 §6.3.5.3 b) 4) 注 8）。受控也不是定稿之后才开始，它和编写同时开始（42020:2019 §10.2 e)、§10.4.2 h) n)）。平时做的活是五项，准备、标识、更改管理、状态记帐、验证与审核。架构与设计这一侧只做一半：识别候选、把人工制品递给配置管理；收下、正式化、维护基线的是配置管理；而谁有权批这一版，是第 16 章治理的事。

它替架构定义做的是四件事：给判断一个起点、给迭代与并发一个交点、补上“交得出去”的另一半、给追溯与决策记录一个可核的挂点。第 14 章交下来的那一问，答案也落在这里：设计依从的是架构基线，也就是架构描述被选定并冻结的那一版（角色依据＝42020:2019 附录 E.4.1.6；“选定并冻结”的资格依据＝15288:2023 §3.8）。

本章留给你的，是三个问题加一句话。三个问题对着任何一份“v1/v2/final”都能用：它是哪一版？谁批的？什么时候？那一句话是本章的站句：

> 你能让交出去的每一份判断都有版本、有基线、改了说得清
> ——你不在场，它照样接得住。

还有一句要收进来：这件事没有“做完了”的那一天。基线会再被冻结，配置项会再被选进来，记录会一直往下写。你能做到的，是每一轮都让“哪一版算数”答得出来，而不是把一套体系一次性搭完就完事。

本章把判断变成了受控的东西，也把候选递了出去。接下来谁有权批、他凭什么说了算，是下一章的事：架构治理给的不是判断，是效力。你已经把“哪一版是定下来的那一份”固定下来了，接下来要有人能决定这一版能不能被引。

# 附录

## A 本章概念总图

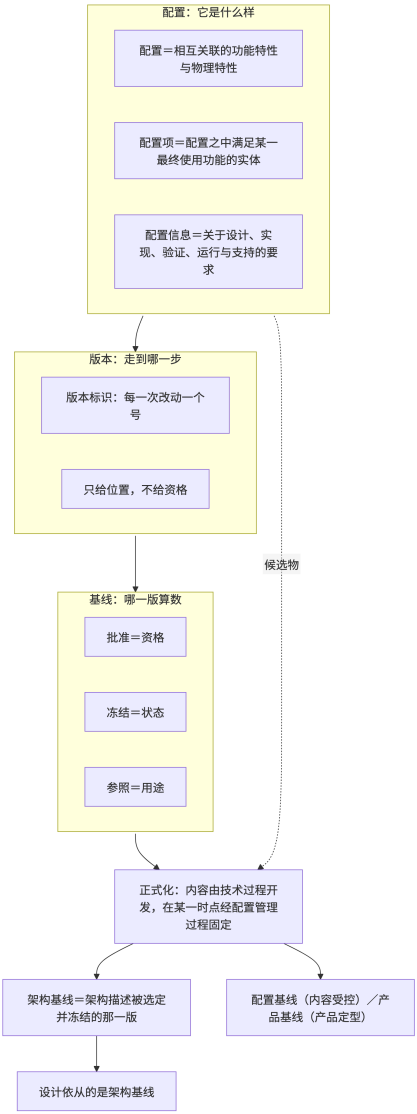



图 15-2

图注：横着从左到右，是三个词的位置顺序（配置、版本、基线）；竖着从上到下，是基线那三个动作的顺序（批准、冻结、参照）。虚线那一

笔是本章最要紧的一条分工：架构与设计这一侧做的是提名候选，正式化归配置管理。

## B 本章的能力级

主级：第 ③ 级（心智模型），读完能推演：为什么“最新版”不能当基线、为什么受控要和编写同时开始、为什么挑候选与定基线是两只手。底座铺到第 ② 级（图式）：一横一竖那张图、基线的三样、五项活动。第 ① 级（表征）是入场券：配置、配置项、配置信息、版本、版本标识、基线、配置基线、产品基线、正式化、配置状态记帐、功能配置审核、物理配置审核。解释框架不建（归第 17、18 章）。

本章补充定位：这一级量的是“你手上那几份东西的位置摆正了没有”，不是文档的厚薄。

能力落在两档上。工程能力那一侧，它是第 9 章第 ⑥ 项可复现意识的落地形态，管好自己交出去的那几份。架构能力那一侧，它是“让整套架构描述与决策记录受控、并让下游必须依据它”那一格。两档不是两件事，是尺度：同一套动作（有版本、有基线、改了说得清），落在一份产物上是习惯，落在一整套描述上就是过程与制度。

再用第 1 章 §1.4.4 那把尺子量一下：本章站在第 ③ 级（心智模型），它给你的是一套位置——配置回答“它是什么样”、版本回答“走到哪一步”、基线回答“哪一版是定下来的那一份”。拿到任何一份“v1/v2/final”，你都能说出它是哪一版、谁批的、什么时候。

> **能答全四问，才算拥有**——你能让交出去的每一份判断都有版本、有基线、改了说得清；你不在场，它照样接得住。

## C 自查 · 这些说法对吗？

1. “配置就是那个参数设置页。”——不对。配置是相互关联的功能特性与物理特性；参数只是配置信息里的一个条目。
2. “配置项就是我们想管的那几样东西。”——差一步。配置项是被选定、有清单、有归属的那些；“想管”不等于“选定”。
3. “有版本号，配置管理就做好了。”——不对。有版本管理、没有配置管理，正是最常见的坏法。
4. “版本号标到 v3.0，就说明它是最成熟的一版。”——不对。版本只说明位置，成熟与否要靠批准与冻结。
5. “最新版就是基线。”——不对。基线是“被批准、被冻结、作参照”的那一版，最新只说明它在最右边。
6. “基线建好以后就不许改了。”——不对。基线是“下一次更改的基础”：冻结、变更、再冻结，循环往前走。
7. “变体就是高一级的版本。”——不对。变体是空间（同时存在），版本是时间（先后出现）。
8. “配置管理是定稿之后的事。”——不对。架构描述在过程中就处于配置控制之下。
9. “我们评审通过了，它当然就算数了。”——不对。评审过的是内容；算不算数要有正式化这个动作。
10. “配置管理是配置管理员的事。”——不对。它是每一个交出判断的人的事——你交出去的那一份，位置摆没摆正。
11. “配置审计就是内审。”——不对。配置审计的对象是配置项与产品、判据是基线；审核的对象是组织与体系、判据是审核准则。
12. “基线能判的东西最多，什么都能拿它比。”——不对。基线承载的是“明示的”那一部分，确认（validation）那一侧它撑不住。
13. “功能基线、分配基线、产品基线，标准里就这么分的。”——要标依据。这三样是行业惯例。

14. “配置管理是架构之后的下一步。”——不对。十四个技术过程都在识别基线候选物并交给配置管理，它与前面各章并行。
15. “有了配置管理，就不会出变更事故了。”——不对。它保证的是“说得清、查得到、追得下去”，不是“不出事”。
16. “可复现就是把文档写全。”——不对。可复现是一件东西的状态：有版本、有基线、改了说得清。

## D 练习

1. 一份设计描述上写着 `V2.3`，评审纪要写着“通过”。它进基线了吗？还缺什么？
2. “这一版改了告警阈值”与“这一版改了断链判定逻辑”，哪一句在说配置变了？为什么？
3. M3 的 −20℃ 机型从 v2.1 升到 v2.3，同时 2~8℃ 机型一直在 v2.0 上。这里有几条版本线、几个变体？
4. 有人问“这批货是按哪份图做的”，现场答“按最新那版”。这个答案缺哪一样？
5. 架构侧评审通过了 AD v1.4，设计手里是 v1.3。设计算不算依从架构？补一句什么话就能判？
6. 恒信的 `final_2.zip` 属于哪一种“不做”？它和“完全没有版本号”坏在哪一处不一样？
7. 一份架构描述在定稿前被三个下游引用了。如果它到定稿才受控，中间那一段的风险是什么？
8. 谁有权批准一份架构基线？本章为什么不答？
9. 有人说“我们评审纪要有、归档也有”，这算配置管理吗？还缺什么？
10. 一份测试记录写着“结论：通过”。它缺哪一栏？为什么那一栏决定别人敢不敢用这个结论？

参考答案：

1. 没进。缺正式化——要有批准的权威、固定下来的时点、此后被当作参照；版本号与评审通过都只是它的前提。
2. 第二句。第一句改的是可调的一个数，第二句改的是设备的形态，形态一变，采出来的记录怎么读都跟着变。
3. 两个变体、两条版本线；−20℃ 这条走到了 v2.3，2~8℃ 那条停在 v2.0。
4. 缺“哪一版、谁批的、什么时候”——没有参照，“最新”不是答案。
5. 判不了，先要问 v1.3 是不是当时的基线、v1.4 有没有正式化。补一句“按哪一版、那一版是不是基线”就能判。
6. 属于“有版本管理、没有配置管理”（前一半还是假的：文件名不是版本）。它比“完全没有版本号”更危险，因为它看起来是做完的。
7. 那三个下游是在按一个会变的版本干活，改动发生时他们不会被告知；等定稿统一发布，等于让他们返工。
8. 因为它是治理的事——“谁有权建立配置控制机制、谁裁定架构集合中的变更”归架构治理，本书第 16 章讲；本章只把候选递出去。
9. 不算。纪要与归档都只留了副本：缺参照（指不回是哪一版）、缺门（改它不必过手续）、缺理由（追不到“凭什么”）。
10. 缺“基线”一栏——对哪一版测的、谁批的、测的时候是什么状态。没有这一栏，“通过”只对当时在场的人成立。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022《软件、系统与企业 — 架构描述》—— §3.3 工作产品（能被选定、冻结、引用的永远是承载它的那份描述）、§5.2.9 AD 元素、§6.6~§6.8 版本标识（三处 shall）、§6.10 决策与理由
- [2] **R-05** ISO/IEC/IEEE 42020:2019《软件、系统与企业 — 架构过程》—— §3.19 存储库、§6.4.2 治理任务、§7.4.4 管理任务、§10.2~§10.4 架构细化、附录 E.4.1.6 基线架构
- [3] **R-08** ISO 10007:2017《质量管理 — 配置管理指南》—— 配置家族与变更控制的行业出处；正文不引其条款
- [4] **R-21** SemVer（语义化版本）（**业界惯例，不是标准条文**）——「主.次.修订」三段式编号的出处；本章只在 §15.2.3 举例，并明记它给的是**位置**、答不出**资格**
- [5] **R-37** ISO/IEC/IEEE 15288:2023《系统与软件工程 — 系统生存周期过程》—— §3.8 基线、§3.11 配置项、§6.3.5 配置管理过程（五项活动 §6.3.5.3 a)~e)）、§6.4.4 系统架构定义过程、§6.4.5 设计定义过程
- [6] **R-38** ISO 9000:2026《质量管理 — 基础和术语》—— §3.10.8~§3.10.12 配置家族、§3.5.1 要求、§3.12.1 审核（与配置审计分家）
- [7] **R-41** IEEE Std 828-2012 —— 基线定义句的出处
- [8] **R-42** ISO/IEC/IEEE 24748-7／SAE EIA-649／MIL-STD-973 —— 「功能基线／分配基线」三分法的行业出处

**经典文献**

- [9] **R-63** INCOSE《系统工程手册》第五版中文版 —— 与 15288:2023 同版同年

# 第 16 章 架构治理：立规矩，对结果问责

## 16.1 引言

第 15 章把判断定成了受控的东西：哪一版是定下来的那一份，答得出来了。可定下来之后还有一问没答：那一份，谁有权改？改它要经过谁、按什么程序？答不上来，基线就只是一份没人签字的文件。本章讲的是那一层：**架构治理**。

### 16.1.1 误解现场

恒信合同审批平台上线八个月，架构基线冻结在去年冬天。这天下午，业务方把一张变更单送到项目组桌上：历史对账记录的查询年限，从两年放到五年。

小事吗？曹峰（产品）觉得是小事。他在系统里点过，那条查询本来就摆在那儿，把年限改大就行。开发那边也说，改一个配置项，两天的事。

苏婷（设计）不同意。上个月需求分析师顾言拿着九查表一项项问过她，其中一项就是“历史记录归谁管”。她在自己那两栏里抄过一句：数据归属唯一——这条是架构定下的，不是设计能自己改的。她说：“这得看架构怎么办。”

罗建（架构）把架构描述翻出来。那一版是去年冬天评审通过、签过字、冻结起来的。改了这条，那一版就不再是那一版了：下游按它做的设计、按它写的测试记录，都得跟着动。他说了一句：“我先看看。”

曹峰等了三天，没等到回话。他跑去问项目经理：“这事到底谁说了算?”项目经理说：“架构那边的意见。”曹峰再问：“架构那个人是谁?”——没有人答得上来。

这不是谁在推诿。真要往下问，问的是三件事：

- 谁有权改这一版架构——是罗建吗?他手里有**架构描述**，可他不是管审批的那个人。
- 按什么程序改——找谁评估、找谁点头、改完通知谁?组里没有这条规矩。
- 改了之后算不算数——开发照着改了，测试按新的测了，可要是有人问“你们凭什么这么改”，谁拿得出依据?

最后一件最要紧，也最容易被跳过。把一条判断算清楚（这条改动的代价、影响、方案，都能算）和让这条判断算数，是两件事；后者不归算的人管。

### 16.1.2 本章要回答什么

前面十五章里，这个问题被推到过四次，每次都说“后面再讲”。

第 11 章讲架构定义的过程，写到收尾那一环时留下一句：那位有**职权**者是谁、按什么程序点头，是组织职能的事，归第 16 章。

第 12 章讲架构决策，提到恒信那道“六问写不全的不进评审”的关——写不清的进不来。可这道关本身，有没有人能撤?那一句也推给了本章。第 13 章讲架构描述，说到“决策权属方”那一格，明写那是“治理落下名字的地方”。第 15 章把判断定成基线，交出来的最后一句话是：接下来谁有权批、他凭什么说了算，是下一章的事，架构治理给的不是判断，是效力。

四处都指向同一个位置。本章要回答的就是它：

你算出来的那些判断，谁给它效力、谁能改它、按什么程序改。

### 16.1.3 本章的骨架

答这一问，先要看清一个位置关系：架构治理不是架构做完之后的第三步。

按 42020:2019 的说法，各架构过程可以并发执行；治理指令与管理指令流向管理，运行计划与状态又流回治理。它是与**架构定义**并着走的那一层——在架构定义的一头一尾朝里伸手（起手为治理做准备，收尾为治理提供支持），同时管着一件架构自己管不了的事：谁能改架构本身。

看清位置之后，本章分三块走：

- 它站哪儿（16.2）——治理是哪种事：它和“管理”差在哪儿，和“架构定义”是什么关系，这个词又是怎么定下来的；
- 它为什么非有不可（16.3）——它保的是“架构还符不符合目的”；这件事一天都不停，所以它是持续的动作，不是一个审批环节；
- 架构判断怎么才算数（16.4）——效力从哪儿来、批完之后谁按着做、符合不符合谁来评、谁能改架构本身。

三块都指向同一句话：治理不替你算，它只决定你算出来的算不算数。

---

## 16.2 治理、管理、架构定义各站哪儿

> 块导读：三块里的第一块，先把位置站对。治理既是组织里那套制度系统，也是那套系统一直在做的动作；它和一个天天被混用的词——“管理”——差在哪儿；它和架构定义又是什么关系；最后，这个词是怎么落到今天这个意思上的。

### 16.2.1 治理：一套制度，一个动作

“治理”在中文里是个老词，到了工程语境里却常被当成新词。它在本书里要担两件事，而这两件事长在同一个词上。

第一件，是一套制度系统。42020:2019 附录 G.1（资料性）给的说法是：架构治理是在组织范围内对架构进行管理和控制所依循的规程与取向（the practice and orientation by which architectures are managed and controlled at an organization-wide level）。

> **先说两个词**：“正本”指标准原文。

下面这五样，是本书对“制度系统”的归拢。正本附录 G.1 的定义句只说它是什么；它自己列的是 a)~d) 四条要求。

五样各有出处。**机构与程序**，是组织结构（§6.4.1 d)、e)）加上治理论坛与程序（i)、k)）。后四样，是正本 §6.5 列出的工作产品：架构治理计划（architecture governance plan）、架构治理指令与指导（architecture governance directives and guidance）、架构治理符合性状态报告（architecture governance compliance status report）、架构集合目标（architecture collection objectives）。

一处名字要说清：§6.4.1 f) 里那条动作叫“建立指导原则、策略与指令”，而 §6.5 给它的产物名是“架构治理指令与指导”，一个说动作、一个说产物，指的是同一件事的两头。

这一套是看得见、查得着的东西：谁在治、拿什么治、治出了什么。它答的是“组织里的治理长什么样”。

这五样各管什么，值得逐一看一眼。

机构与程序是骨架：谁在治（治理机构里坐着谁）、按什么程序治（一条变更从提出到裁决走哪几步）。正本把这一件说得很直：要建立

角色、职责、问责、职权和组织结构（§6.4.1 d)、e)）；程序与治理论坛另有两处（i)、k)）。

治理计划是这份骨架一次性的产物：治理工作要覆盖哪些架构、需要什么资源、什么时候收口（内容是 §6.4.1 l) 的 1)~7)；m) 只规定要有这份计划）。它使治理这件事本身也被规划、被检查。

治理指令与指导是治理往外递的东西：指导原则、策略与指令（§6.4.1 f)）。它是下游唯一能拿到手的“规矩”——没有它，治理只存在于会议里。

符合性状态报告把“有没有照办”变成一份可看的记录：哪些指令被执行了、哪些没有、为什么没有（评定在 §6.4.5，未实施原因在 §6.4.6 b)，报告本身是 §6.5 的工作产品）。它使符合性从一个态度变成一个状态。

架构集合目标是治理盯着的那把尺：这一批架构要达成什么、达成到什么水平（§6.4.3）。

五样合起来，才是“制度系统”这个词的实体。只说“我们建了治理机制”，五样里缺一样，这套系统就有一处是空的。

第二件，是一个动作。同一份正本的第 6 章讲的是过程，而过程的日常是一个一个动作：准备并规划、监视、评定和控制、确立架构集合目标、作出治理决策、监视并评定符合性、评审指令与指导的实施。它答的是“这一刻谁拍板、谁裁例外、谁去看有没有照办”。

两件事不是两个概念，是一个词的两面，而且有先后：先有反复地做那个动作，才沉淀出那套制度。正因为一次一次地“作出治理决策”，才留下治理计划与指令；正因为一遍一遍地“评定符合性”，才有符合性状态报告这一份东西。名词义是动词义的沉淀——这一句，是本章后面所有内容的地基。

一处词形上的提醒：英文里有三个词长得像、意思不同，本书一律分开译。

governing principles 译管控原则——它是架构的内容之一，归第 11 章；governance 译治理——就是本章这两义；governs 译管控——第 13 章讲视角管视图用的就是它，一个架构视角管控一个或多个架构视图。

还要补两槽：guiding principles 按正本译文是指导原则（治理发出去的那一组，§6.4.1 f)），governing directives 是管控指令（因架构变更而需要修订的那一组，§6.4.2 b)·c)）。

“管控”这一个词用了两处：作定语时对应 governing（管控原则、管控指令——说的是“这套东西是管事的”），作谓语时对应 governs（视角管控视图——说的是“谁管谁”）。两者同源、位置不同。几个词读成一个，本章后面会全部跟着歪。

### 16.2.2 治理平时做什么

正本 §6.4 把治理展开成一组活动（准备并规划／监视、评定和控制／确立架构集合目标／作出架构决策／监视并评定符合性／评审指令与指导的实施）。它们平列着摆，读者不容易记住；本章按它们各管的那一段归成三组：给出指导、评定符合性、对结果问责。

这三组是本书的归拢，正本给的是六项平列，这句话先放在前面。同一套东西，本章统称“三抓手”（后面各节同此名）。

给出指导

- 准备并规划架构治理工作（§6.4.1）：识别应纳入治理的架构集合，建立角色、职责、问责、职权与组织结构，建立指导原则与策略，产出治理计划。
- 确立架构集合目标（§6.4.3）：审查当前与未来的业务需要和任务需要，确立架构战略与架构集合目标。

评定符合性

- 监视、评定和控制架构治理活动（§6.4.2）：盯业内与法规的变化，盯治理工作本身的度量与风险；建立**配置控制管理机制**以管控架构集合中的变更，就在这里（§6.4.2 k)）。
- 监视并评定对治理指令与指导的符合性（§6.4.5）：看架构有没有照着指令与指导走，并让有职权者能直接查证。

对结果问责

- 作出架构治理决策（§6.4.4）：发布治理指令、指派责任、把不属于治理决策的其他决策上报至适当的**企业决策论坛**（治理自认有权限边界，是本书的读法），并复核与传达决定。
- 评审治理指令与指导的实施（§6.4.6）：一轮收尾时评“做了什么、没做什么、为什么没做”，收集教训，并据此修订指令本身——指令自己也会变。

一项区别要先说清：这六项里，活动是“应”（shall）级，任务才是“宜”（should）级，而且活动可以按任何适当的顺序执行（§6.3）。所以治理不是一条流水线，是一组随时可以起用的动作：用哪几项，取决于你手上这件事动到了什么。

这三组在项目里长什么样。恒信那年买回“数据治理平台”，是想解决数据不干净，那是替管理做事。可同一批人从来没做过“给出指导”这一组：没有谁把“数据归属唯一”写成一条指令发下去。到了出事那天，第二组也没人干：工单对不上，没人去评“到底按哪一条才算符合”。第三组更空：登记表上“决策权属方”那一栏，一直写着是空的。三组都缺，治理就只剩一个名字。

冰链那边相反，缺的是第一组的落款。嵌入式硬件工程师石军把接口的老规矩一笔一笔写下来，又要求“除我之外还有谁按同一套办法查得出来”。这就是“给出指导”。

测试季昭在测试记录里补上“对哪一版测的、那一版谁批的”，使“评定符合性”第一次有了可看的依据。可那套规格自己按哪一版走、改完谁去评它有没有被照办，一直没人说得清；那年冬天在规格里补的那一行，只答了前半句。

它们之下另有共 49 条任务（本库回正本逐条核：15／11／5／6／5／7）。这一串数字不必记，要记的是这个：这几项活动里，有三项的名字是“监视……”“监视……”“评审……”，而“作出决策”只占一项；它名下 6 条任务里，没有一条叫“审批”。把治理等同于开会拍板，是对这份正本最大的误读。

这张表把三层串起来：左边是本书归拢的三抓手，中间是正本展开的那几项活动，右边是它在项目上长什么样。

| 三抓手 | 正本展开的活动 | 在项目上长什么样 |
| --- | --- | --- |
| 给出指导 | §6.4.1 准备并规划架构治理工作 | 把“数据归属唯一”这类架构红线写成指令发下去；把谁有职权、按什么程序定下来 |
|  | §6.4.3 确立架构集合目标 | 说清这一批架构要达成什么（“月底账目对得上”），以及达成到什么水平 |
| 评定符合性 | §6.4.2 监视、评定和控制架构治理活动 | 盯业内与法规的变化、盯治理工作本身的风险；把配置控制管理机制建起来 |

| | | |
|---|---|---|
| 三抓手 | 正本展开的活动 | 在项目上长什么样 |
| | §6.4.5 监视并评定对治理指令与指导的符合性 | 看合同系统与财务系统到底按哪一条办；让有职权者能直接查证 |
| 对结果问责 | §6.4.4 作出架构治理决策 | 发布指令、指派责任、把够不着的决定往上送，并复核与传达 |
| | §6.4.6 评审治理指令与指导的实施 | 一轮收尾时问“哪条没照办、为什么没照办”；把教训并回指令 |

同一项活动可以同时服务两个抓手。§6.4.6 既评实施（对结果问责），又据此修订指令（给出指导）。§6.4.1 里既有“建立指导原则、策略与指令”（给出指导），也有“建立角色、职责、问责、职权”（对结果问责）。分组看的是主要归到哪一组，不是排他归属。

本书看下来，这几项活动里最容易被跳过的是“确立架构集合目标”（这是本书在两案上的观察，不是正本的判断）。它不产出一条可执行的指令，产物是目标本身。在一个只关心进度的组里，它常常被当成务虚。可没有它，“符合”就没有可比的对象：目标不写下来，评的时候只能凭印象。

同样容易被跳过的还有“评审治理指令与指导的实施”（这也是本书的观察）。一轮做完了，人散会了，谁也不回头问“哪条没照办、为什么没照办”。可治理的指令要能活下去，靠的正是这一问——它是让规矩自己会变的那个动作。

这几项活动里，有几处值得再往里看一层。

§6.4.1 那 15 条任务，开头两条就把范围定死了：识别应纳入某一需要治理监督的架构集合的那些架构，以及识别可能直接适用的现有架构或参考架构。也就是说，治理的第一步不是定规矩，是划范围：哪些架构归它管。范围没划清，“这条要不要上治理”就永远吵不完。

§6.4.4 里有一条不显眼但很要紧：把超出治理权限的决定升级到企业决策论坛。它承认了一件事：治理不是最高层，它自己也有权限边界；够不着的事要往上送，而不是硬扛。

§6.4.1 里的 g)、h) 两条也值得记：识别可复用元素、识别知识资产。这两条把治理从“管住”往“攒下”推了一步：治理的过程本身在给组织积累东西。

§6.4.6 的收尾是整份正本里最反直觉的一条：一轮治理结束时要收集教训、识别并选择要改的地方、把改动并入指令与指导。指令本身是可以被改的，而且改动要有出处。规矩一旦定了就再也不动，那不叫治理，那只是把规矩挂在了墙上，再没人去碰它。

### 16.2.3 治理和管理差在哪儿

这两个词天天连着说，说的却常常是同一件事：开会、审批、协调。要把它们分开，最省事的办法是把正本里那两句目的句并排读。

- 架构治理（42020 §6）：建立并保持架构集合中的架构与企业目标、策略和战略以及与相关架构的一致性。
- 架构管理（42020 §7）：实施架构治理指令，以及时、高效和有效的方式达成架构集合的目标。

前一句动的是“对不对得上”，后一句动的是“做没做到”。再往下推一层，两边的分量落在不同的东西上。

治理守的是不变量：企业的目标、策略、战略，架构的原则与红线。这些东西一动，下游全部要重来，所以不轻易动；治理的活，就是看着它们别被悄悄改掉。

管理调的是框架内的变量（“**守不变量**、调变量”是本书对这两句目的句的读法）：资源怎么排、进度怎么走、哪一版先做、谁先上。这些可以在既定框架里来回调，调错了再调回来就是。

在组织里也查得到——这一条比画面更结实。同一套机制上，两者各占一端。

- 给出与实施：治理建立指导原则、策略与指令（§6.4.1 f)），管理实施它（§7.1 目的句）。
- 评定分两级：治理评的是对治理指令与指导的符合性（§6.4.5），管理在架构集合之内评。
- 问责的设立与承担：治理建立角色、职责、问责、职权（§6.4.1 d)），并让有适当职权者作出决策（§6.2 c) **预期结果**）；管理在架构集合内承担并执行。

一句话：区别在范围与权限，关系是同一条流的两端。

这三条对应还解释了一个常见现象：很多组织的“治理”其实是管理。因为第一端就是断的——指令不是治理发的，是某次管理会议上顺口定的；符合性不是治理评的，是项目经理自己看的；问责没有设立过，出了事只能追到做事的人。端与端之间的分工一旦错位，治理就退化成管理流程上的一个审批格。

恒信那一幕正好是反例。五百万买回来的“数据治理平台”，是拿来做数据清洗的——那是做事；他们缺的是“谁定规矩、谁担责”——那是掌舵。买回来的是一把桨，缺的是舵。

### 16.2.4 治理与架构定义并行

最容易走错的一步，是把治理排在架构之后：先做架构，做完交上去审。

正本把这条堵死了。42020 §5 里有一句逐字的：

> 各架构过程可并发执行，其间存在交互并随时间迭代。同时，治理指令与管理指令从架构治理过程流向架构管理过程，运行计划和状态则从架构管理过程流向架构治理过程。

注意那个流向。不是“架构做完了交给治理”，而是治理发指令给管理、管理把运行计划与状态回流给治理：两条线并着走，还来回走。

在架构定义那一侧，正本还留了两处对照。15288:2023 §6.4.4.3 讲系统架构定义过程。它起手 a) 3) 要求“为支持组织的架构治理与架构管理工作做好准备并作出规划”，收尾 e) 3) 要求“为组织的架构治理与架构管理工作提供支持”。**架构定义一头一尾都朝着治理与管理伸手**，中间那段才由它自己走。

可见两者不是上下两道工序，是并行的两条线。

还有一层位置关系值得记一笔：注意这里的“六”是六个过程，不是 16.2.2 列的那几项治理活动，两者不是一回事。42020 把六个架构过程分了组：架构治理与架构管理同在一组，架构概念化、评估、细化是三个核心过程，另有**架构使能过程**。

也就是说，治理不是架在核心过程之上的第四层，而是与核心过程并列的那一组（后半句是本书的读法，正本只给了分组）。本章后面讲“批完之后谁按着做”（16.4.2），靠的就是这一层位置。

### 16.2.5 治理这个词，从哪儿来

先说英文。governance 从 govern 来，而 govern 的老家是拉丁语 gubernare，再往上是希腊语 kybernan——本义是掌舵。后来它一路扩张：掌船、掌城邦、掌公司，凡是“定方向、划边界、担最终责任”的活都归了它。

中文的“治理”走的是另一条路。它原本就是治国理政的常用词：“治”从水，是疏导；“理”从玉，是顺着纹理剖开。两样都指向同一个动作——不是硬掰，是按它的纹理把它导到该去的地方。

这个词在现代中国的政策语境里被重新起用过一次，而且是一次有意的定译。1990 年代，西方发展研究里的 governance 进来时译法五花八门；到 2013 年“国家治理体系和治理能力现代化”写进决定，“治理”这个译名就固定下来了。它要跟“管理”拉开距离：管理是执行，治理是定方向、立规矩、担责任。本书用“治理”，用的是这一层。

它在标准里的户口。正本里 architecture governance 的定义在 42020:2019 附录 G.1：注意它是资料性的（informative），规范主干是第 6 章那个过程。三件事要说清。

- 架构治理的定义权就在这两处，不在行业框架那儿；TOGAF 只作行业参考（本书早期稿件把定义权挂错过，已改）。
- 讲组织治理的 ISO 37000:2021 与讲 IT 治理的 ISO 38500:2024，本章只在旁证处引其条款。
- 正本 §3 是术语条，里面没有 architecture governance 这个词条——它的定义在资料性的附录 G.1 里。这也是本章要专门把它说清的原因。

## 16.3 治理为什么非有不可

> 块导读：上一块把位置站对了；这一块答的是“它凭什么一直要做”。它保的是架构还符不符合目的——而目的会变、架构会偏，所以它不是一个审批环节，是一个持续的动作；这个动作

> 靠三样东西转起来，转起来之后，它对一个组织的意义也就落到了实处。

### 16.3.1 架构得一直符合目的

正本里第一个该留意的字眼，落在“保持”上。42020 §6.1 给架构治理的目的句是：

> 建立并保持架构集合中的架构与企业目标、策略和战略以及与相关架构的一致性。

“建立”是一次，“保持”是一直。为什么必须“保持”？因为两边都在动。

目的会变。42030:2019 讲实施架构评估的原因，第一条就是它：

> 确定所关注实体是否已经或正在以使其实现预期目的的方式被架构（或能按适合新目的的方式予以改变）。

这句正本的译文有点绕，意思其实直白：看这个实体是不是照着“它该达成的目的”架构出来的，或者它有没有可能被改造成适合一个新目的的样子。括号里那半句，就是“目的会变”：今天符合目的的架构，明天可能因为业务改了方向而不符合。

架构也会偏。42020 §10.2 的注（排在 h) 后）把这一层说得更清楚：

> 一致性是架构治理与管理之间、以及概念化与细化之间的共同责任。随着架构视图和模型在细化期间得到精化，这些其他过程早先建立的一致性有可能被削弱。当在细化期间发现不一致

> 时，重要的是将其告知这些其他过程，以确保在可能且适宜时能够采取适当的缓解措施。

注意那句共同责任：一致性不是哪一方单独维持得住的。细化一深、元素一多，早先对齐过的东西就会一点点磨掉，不是有人违规，是没有人看着。

正本把“治理做成了会看到什么”写成了六条预期结果（§6.2）：

- 目标与需要：架构集合目标在满足企业及其客户、供应商当前与预期的业务需求（a)）；关于架构集合的目的、目标与具体目标，关键利益相关方之间保持共识（f)）。
- 决策与职权：架构工作活动与架构决策同企业关注点及语境关注点保持一致（b)）；有理由支撑的架构决策，由具有适当职权者作出并传达给利益相关方（c)）。
- 架构之间：架构集合中的各个架构与组织的目标、具体目标、战略与愿景保持一致（d)）；各架构之间在必要时彼此对齐（e)）。

（六条是正本的平列；本章按“目标与需要／决策与职权／架构之间”归成三条线，这三条线是本书的归拢。）

六条分开看抽象，配上一处现场就清楚了。

- a) 目标与需要——架构集合目标在满足企业及其客户、供应商当前与预期的业务需求：恒信买平台是为了“数据干净”，可它没回答“月底账目对得上”这条业务需要落在架构的哪一条上。
- b) 与关注点一致——架构工作与决策同企业关注点及语境关注点保持一致：冰链断链之后处置那批货，谁也没去对监管那条留档要求。
- c) 决策与职权——有理由支撑的架构决策由具有适当职权者作出并传达：恒信那张纪要写着“会议决定”，没有名字、没有理由、没有传达对象。

- d) 与组织目标战略一致——架构集合中的架构与组织的目标、具体目标、战略与愿景保持一致：合同平台那条编号规则，和“账目对得上”是什么关系，没人说得清。
- e) 架构之间彼此对齐——各架构之间在必要时彼此对齐：合同系统与财务系统的编号规则，正是“该对齐而没对齐”的那一处。
- f) 共识保持——关于架构集合的目的、目标与具体目标，关键利益相关方之间保持共识：曹峰以为改个年限是小事，苏婷知道动的是“归属唯一”——两个人对同一件事的认识，从一开始就不在一个层面上。

一处用词：本书早期稿件里说“目的漂移”“架构偏移”。

这六条为什么放在一起看。单看每一条都平；六条排在一处，治理的位置就出来了。a) 与 f) 管的是**目标**：上面那条线要一直通着。b) 与 c) 管的是**决策**：谁作出、有没有理由、传达给谁。d) 与 e) 管的是架构之间：对外与组织的目标对齐，对内彼此对齐。

它与前一块的“三抓手”不是一回事：三抓手说的是治理的动作（给出指导／评定符合性／对结果问责），三条线说的是预期结果的三个面（目标／决策／架构之间）。

三条线里，a) 与 f)、d) 与 e) 说的是状态：目标通不通着、架构之间齐不齐；b) 与 c) 说的是动作：决策怎么作出、怎么传达。本章讲的正是第二组（本书的读法；正本一律写作“预期结果”）。

正本把“决策由具有适当职权者作出并传达”写成预期结果，等于承认：判断要算数，不是判断本身的性质，是有人作出并传达的结果。`drift` 与 `erosion` 在 42020 里一次都没出现；正本说的是“一致性被削弱”“不一致（misalignment）”。本章照正本说。

### 16.3.2 给出指导，评定，问责

治理把“保持”这件事做出来，靠的是三样动作——给出指导、评定符合性、对结果问责（合称三抓手；与 16.2.2 归的那三组是同一套东西）。三样都能在正本里找到条文。

给出指导。治理先得把规矩递出去：建立指导原则、策略与指令（§6.4.1 f)），由具有适当职权者作出、并传达给利益相关方（§6.2 c)），并落到一份看得见的工作产品上——架构治理指令与指导（§6.5）。递出去之后还不能不管：指令自身也要被评审（§6.4.6），因为它也会过时。

评定符合性。有了规矩，就得看有没有照办。§6.4.5 要求监视并评定对治理指令与指导的符合性，并把结果落到符合性状态报告上（§6.5）。§6.4.2 还要盯治理工作自身：它的度量、它的风险，以及配置控制管理机制有没有建起来。

对结果问责。评出结果之后，得有人担：治理建立角色、职责、问责与职权（§6.4.1 d)），作出架构治理决策（§6.4.4），并由具有适当职权者作出并传达（§6.2 c)）。

这三样凭什么转得起来？因为“符合不符合”这件事是可判定的。把 governs（视角—视图）的实质说成一句话：**可判定的符合性**；再往前一步：可判定的前提是**同解**（见 R-04 §5.2.7 与 §3.8）。读同一条规则的人，得能读出同一个意思。把这两句从“视角与视图”移到“治理指令与符合性”上来用，是**本书的移用**（30 号自己就提醒过：那个用法与架构治理不是同一回事）。

移过来之后补一句：同解的前提，是被定义的语言。

这一条听着抽象，落到实处就是本章前半块做的事——把“治理”“管理”“指令”“符合性”这些词各自钉住。词不钉住，评出来的“符合”就是各说各话。

### 16.3.3 治理对组织意味着什么

到这儿，治理对架构判断的意义已经说清了：它让判断算数。但它对一个组织的意义还要单说：这一层常被漏掉，漏掉之后，治理就显得像一道多余的手续。

它是一条通路。企业的目标、策略与战略，靠什么落到架构集合上？不是靠号召，是靠指令与指导（§6.4.1 f)）。治理把上面那一层话翻译成架构这一层能执行的规矩，再靠管理去实施。没有这条通路，战略与架构之间是断的。

它让问责落到人头上。 §6.4.1 d) 要求建立角色、职责、问责、职权与组织结构——这件事的组织含义是：“谁点头、谁担责”必须写在纸上有名字。技术权威位不等于治理位：一个人可以是这个领域里最懂的人，但点头的权力不在懂不懂上。正本把这一条写进了预期结果（§6.2 c)），说明它不是本书的发挥。

它让符合性可被评定，因而可审计。一份符合性状态报告（§6.5）使“有没有照办”变成可以查的状态，而不是一个态度。组织的审计、复盘与追责，都要拿这份东西当依据。

它还是组织资产的沉淀处。治理过程中长出来的东西——原则、指令，以及被识别出来可复用的元素（§6.4.1 g)），还有知识资产（§6.4.1 h)）——不会随项目和人员散掉。架构会换、人会走，这一层留下的是组织自己的东西。

一句现场。本仓库自己也有一份这样的东西——90-治理登记/01-例外与治理机构登记（登记表 01）。它的抬头就写着：本目录内没有具备否决权的治理机构，所以本表只做“登记、度量、留痕”，一切放行须由本项目治理角色作出。这正是“治理对组织意味着什么”在最小尺度上的样子：它不一定是一个委员会，它可以只是一处把名字、理由和状态写下来的地方。

还有一层要向回接。第 5 章说过，目的的所有权在业务方（§5.4.2）。治理这条通路的起点正是那条所有权：治理不生产目的，它把已经确立的目的落到架构集合上，并看着它一直被符合。

前三条各配一处现场，就落地了。

通路那一处：恒信那条真正的业务目标是“月底账目对得上”。它要落到架构上，得经过两次翻译：一次是把它写成架构目标（“数据归属唯一”“语义唯一”），一次是把架构目标写成设计约束。第一跳归需求，第二跳归架构；而这条通路要有人守，守的人就是治理：它保证每一次翻译之后，上头那句和底下那条还对得上。

问责那一处：曹峰在选型会上签了“同意”，那是好意，他想的是“数据干净了，合同系统用起来才省心”。可这句话落在管理那一边（买什么、什么时候买），不落在治理那一边（谁来定规矩、谁担后果）。好意不能顶替担责；真正的问责栏，那一年一直空着。

能一条条查的那一处：冰链的季昭在测试记录里补了“基线”那一栏，写的是对哪一版测的、那一版谁批的、测时处于什么状态。补完之后，疾控那边再问“你们这版凭什么是这一版”，答得出来了。这就是“能被评定、也就能一条条查”的样子：不是多写一栏字，是让一个问题第一次有了答案。

## 16.4 架构判断怎么才算数

> 块导读：前面两块把位置站对、把理由说清；这一块落到最实处——你算出来的判断，凭什么算数。效力从哪儿来、批完之后谁按着做、符合不符合谁来评、谁能改架构本身，最后把“假治理”认出来。

### 16.4.1 架构判断的效力：谁来给

先把这个词说准。架构判断的效力（本书读法；正本里没有这个词，它讲“有效性”用的是 effectiveness。下称“效力”），指的是你算出来的那些判断，算不算数。它是两句话：别人必须依据它；谁也不能单方面改它。

为什么“算不算数”要单说？因为把一条判断算清楚，和让这条判断算数，是两件事。前者靠本事：这条改动影响多大、有几个方案、哪个更划算，都算得出来。后者不靠本事：它靠治理那一侧作出。一个组里，最会算的那个人，往往不是说了算的那个人。

把它放进整条链上看，是三段。

1. 判断：架构定义产出判断——这条约束立不立、这个接口定不定、这个取舍选哪个（第 11、12 章）；
2. 正式化：配置管理把判断定成“哪一版是算数的那一份”（第 15 章）；
3. 效力：治理给这一份效力——它算不算数、谁能改它（本章）。

前两段做完、第三段没做，会是什么样？第 15 章已经交过一句话：“谁有权批这一版”不归**配置管理**答。那一份被冻结的架构描述，写着日期、写着版本、写着谁解的冻——可没有一处写着“谁有权动它”。

正本在架构定义那一侧还留着一处口子，说明这一步非有不可。15288:2023 §6.4.4.3 讲系统架构定义过程的收尾，e) 2) 那一条英文只有六个词——就架构定义取得一致意见（Obtain agreement on the architecture definition）。这个动作的位置值得琢磨：它是架构定义过程自己的一步，可“一致”要跟谁取得，过程里没有说。那正是治理那一侧要接的地方。（16.2.4 引过同一个过程的起手与收尾两句，那里讲的是并行；这里讲的是效力。）

所以本章要答的那一问，答案就落在这一段：**判断算不算数**，不由算的人定，得由治理那一侧作出。

### 16.4.2 架构管理管什么

治理给了效力，往下怎么走？中间还有一层，前面各章都没给它户口：架构管理。

42020 §7.1 给它一句目的：

> 实施架构治理指令，以及时、高效和有效的方式达成架构集合的目标。

一句话里三处值得拆开看：实施治理指令（它是治理的执行端）；达成架构集合的目标（它盯的是一批架构，不是一个项目）；及时、高效和有效（它管的是节奏与代价）。

它的活动有六项（§7.4）：准备并规划架构管理工作、监视、评定并控制架构管理活动、制定架构管理途径、执行架构集合的管理、监视架构工作的有效性、为完成架构管理计划做好准备。名字读起来平，位置却很关键——它站在治理与核心过程之间：

- 往上，它接治理的指令与指导；
- 往下，它给架构概念化、评估、细化这三个核心过程下指示；
- 再往回流：运行计划与状态回到治理那一侧（§5 那句“运行计划和状态则从架构管理过程流向架构治理过程”）。

正本的附录 G.2 把它的活说得更实：架构管理**建立、维护并使用一套连贯的指南、原则和管理制度**，为架构的设计与开发提供方向和指示。它还**识别潜在风险，并按照治理指令去缓解**。

于是链子清楚了：治理定规矩、给效力 → 管理把规矩变成对核心过程的指示 → 核心过程干活 → 计划与状态回流。

六项各做什么，逐一看：

- 准备并规划架构管理工作（§7.4.1）：按相关性、各自的目的与范围、时效、彼此是否一致，把架构归类成一个架构集合。
- 监视、评定并控制架构管理活动（§7.4.2）：看管理活动有没有跑偏，必要时纠正。
- 制定架构管理途径（§7.4.3）：依架构治理方针、指令与指南（正本中译此处作“指南”，与前文的“指导”同是 guidance；**本书统一写“指导”**），识别管理途径、约束、方法、工具与技术，并据此制定适用于架构集合的架构管理计划。
- 执行架构集合的管理（§7.4.4）：最实的一项——架构的变更在这里被管，而以 15288 的配置管理过程为指南（m）。
- 监视架构工作的有效性（§7.4.5）：看架构工作本身有没有效。
- 为完成架构管理计划做好准备（§7.4.6）：收口。

读这六项，要与治理那六项对着看——有两对活动同名，对象不同（这组对读是本书的读法，正本给的是两项活动名）。

治理的“准备并规划”（§6.4.1）与管理的“准备并规划”（§7.4.1）同名，可一个规划的是治理工作，一个规划的是管理工作；“监视、评定和控制”也同名（§6.4.2 与 §7.4.2），可一个管的是治理活动本身，一个管的是管理活动。

另有两处名字不像、却常被混起来：治理的“监视并评定对治理指令与指导的符合性”（§6.4.5）对的是整个组织，管理的“监视架构工作的有效性”（§7.4.5）对的是架构工作本身。

但别把这条对照读成“管理不评符合性”。管理自己就有一条：§7.4.2 b) 要求“监视并评定架构治理指令与指南是否得到遵循”；§7.2 a) 也把“与治理指令和指导保持一致”列为管理的预期结果。

所以两者的差别不在评不评符合性，而在评的对象与层级：治理评整个组织对指令与指导的符合性（§6.4.5），管理在架构集合之内评（§7.4.2 b)）；此外，§7.4.5 还评架构工作本身有没有效。

名字像，对象不同，这正是 16.2.3 那三条对应在条文上的样子。

这一层为什么非有不可？因为治理说的话是给组织的，不是给某个项目的。一条“配置控制管理机制要建起来”，落到合同平台这个项目上，是哪个环节建、什么时候建、谁来建，这是管理的活。没有这一层，治理的指令就停在半空；反过来，只讲管理不讲治理，规矩就没人定。

### 16.4.3 指令发下去，谁看有没有照办

这件事正本叫“监视并评定对治理指令与指导的符合性”（§6.4.5），它有五条任务，从 a) 到 e)，可归成三件：

- 监视：用适当的办法盯架构集合的治理状况（a)）；
- 评定：看架构有没有符合外部义务与内部工作惯例（b)），看管理计划与状态有没有符合治理指令与指导（c)），另外按 e) 再看两样：**将来会用到的外部义务**，以及**会影响架构治理的标准**，架构符不符合它们；
- 让人直接查：允许有适当职权者直接访问**按设计构建、而该设计基于架构描述**的实体，核对它符不符合治理指令所确立的义务（d)）。

d) 后面挂着的一个注，是这一节最该记住的一句：

> 仅检查架构的符合性可能不够充分，因为架构特征可能未能恰当转化为该实体的设计或运行。（§6.4.5 注 4）

架构写对了，东西不一定做对。所以符合性要一路查到实物上去。

这里有一个最容易混的地方：不是所有“评”都是同一个评。本章管的是其中一个，另外两个各归各章。

| 评的是什么 | 谁在评 | 出处 |
| --- | --- | --- |
| 符合性——有没有照治理指令与指导办 | 架构治理 | 42020 §6.4.5 |
| 架构本身——这个架构好不好、达没达目的 | 架构评估过程 | 42020 §9 |
| 实物与记录——造出来的东西与记录对不对得上 | 配置审核 | 15288:2023 §6.3.5.2 e)：“所需的配置审核得以完成” |

三样各评各的，一个都不能替另一个。把配置审核说成“治理的符合性评定”，或者反过来，都是把两把尺子当成了一把。三样都评过，才谈得上“这件事是受控的”。

三种评定各配一处现场，区别就明显了。

符合性评定：恒信那次工单对不上，谁也没有去评“按哪一条算符合”，因为没有指令可评。这是最典型的治理缺位：不是评错了，是根本没有可评的东西。

架构评估：冰链断链之后做的那件事，是评“这套架构还能不能达成目的”。它评的是架构本身，结论会进架构的下一次细化：这一步归 42020 §9 的架构评估过程（它与本章评的符合性、与配置审核之间的分野，第 18 章分开讲），不归本章。

配置审核：断链那天，现场那批货与记录对不对得上，是实物与记录的核对。它问的不是“架构对不对”，是“造出来的和记下来的，是不是一回事”。

三样合起来，才叫“这件事受控了”：有指令可评符合性、架构本身评过、实物与记录核过。少一样，说“受控”都是提前说的。

报告之外，还有一步。上面 d) 那条“让人直接查”，完整说法是：允许有适当职权者直接访问按设计构建的实体，而那个设计基于架构描述（§6.4.5 d)）。跟那份符合性状态报告（§6.5 明列为“应产生”的工作产品）比起来，这一步把符合性从“别人告诉我”补成“我自己看得到”；恒信那种“纪要写着会议决定、谁也查不了”的局面，缺的正是这一步。

### 16.4.4 谁能改架构自身

本章最硬的一处在这里。

正本里有两句话长得像，位置却分得很开；读混了，就会把治理读成“日常变更的审批岗”。

第一句在治理那一侧：§6.4.2 k) 要求“建立配置控制管理机制，以管控架构集合中的变更”。注意，它建的是机制——一套规矩：变更怎么识别、怎么评、谁有权定。

第二句在管理那一侧：§7.4.4 m) 要求“以 ISO/IEC/IEEE 15288 中的配置管理过程为指南，管理对架构的变更”。这里动的是过程——一件一件具体的变更，按配置管理那套办法走。

机制不等于过程（这组区别是本书的读法，正本给的是两条分置的任务）。治理建的是机制——这一类变更该由谁定、按什么程序定；执行过程的是管理——这一条变更具体怎么走流程。两句话一混，治理就变成替每一个变更签字——可正本里“作出治理决策”只是一项活动，“审批”这个词一次也没出现过。

那么什么必须由治理定？**判据本身**。目的、架构原则、治理红线的变更，只能由治理裁决：判据不能自己裁自己。这一条在本书里落在“三层基线”上：

- 治理基线：目的、原则、红线这一层；
- 协议基线：跨团队、跨系统约好的那一层；

- 产品 CM 基线：某个产品上被正式化的那一份（第 15 章讲的）。

三层里，往上动一层，下面全要跟着动，所以越靠上越难动、越要治理。

三层基线各举一例，位置就清楚了。

- **治理基线**：冰链那条“连续性不得中断”，与第 11 章讲的管控原则、第 12 章讲的架构红线是同一层——动它，整套架构的取舍前提都变了。所以这一层的变更只能由治理裁。
- **协议基线**：合同平台与外部对账平台之间的接口约定，是两边团队约好的那一份。它不是哪一方单独说了算的——改它要两边同时动，所以这类变更需要治理出面撮合。
- 产品 CM 基线：那一份架构描述，连同跟着它一起冻结的凭据包——这是第 15 章讲的“被正式化的那一份”。它是三层里最靠下的，动它的门槛最低（走配置管理的流程），但动它的记录最全。

一句话记住三层的差别：越靠上，改它需要的权力越大；越靠下，改它需要的记录越细。这里必须写清一句：“三层基线”是本书的复合词，不是 ISO 术语。正本的术语条只有单数的“基线”（15288:2023 §3.8）；而那一条，正本自注其来源是 IEEE Std 828-2012。“配置基线”“产品基线”只在任务条文里零散出现，不成其为一套三分法的术语条。

另有一套“功能基线／分配基线／产品基线”，作为一套三分法其出处不在四份标准里，与三层基线是两条不同的轴，不能并着讲。

还有一处容易漏：治理指令自己也是受控的。指令会过时，要评审、要修订（§6.4.6）；修订前后是哪一版，它也进配置管理。规矩本身有版本，“按哪一版规矩办”才答得出来。

还有一处名字值得记住：够得着“改架构”的那些人，正本附录 G.1 给了它一个制度形态的名字——设计权威机构（Design

Authority）。它划的是“谁有权按架构原则作判断”这一格；不够时就升级，附录 G.1 明写要有升级的途径。

最后点一句人事。恒信那年最该建却没建的，正是这一条机制。他们的变更走的是“谁发现问题谁提、项目经理拍板”，没有“这一类变更该由谁定”的规矩：所以同一件事，改编号在开发那儿定，改字段在数据那儿定，两边的判据不一样。这就是缺 k) 的样子：不是没有变更流程，是变更没有权威。

第 9 章把工程师的能力列成六项，其中第 ④ 项是职业伦理。它讲的是“能力会放大你选择的规模，而方向由这一项定”；它的检查点问的是“压力面前，你会不会为进度悄悄牺牲可靠性”。那一项落到制度上，就是本章这一层：目的、原则、红线写在纸上，改它要走程序、要留名字。没有人看着的红线，等于没有红线。

### 16.4.5 什么样的治理是假的

真治理与假治理，有时候只差一个名字。

第一种假，是名字上的。一件事本来是管理，开会、排期、协调资源，被叫作“治理”；一处日常审批格被叫作“治理委员会”（现场常这么叫；本书统一称治理机构）。判断不难：看它有没有“给出指导”那一半。一个机构只在事后审、从不立规矩，它是过滤器，不是治理机构。

第二种假，是缺位。缺位的三种样子并列在一起，正好当三面镜子：

- 缺配置管理：指令发了，可没有落款、没有版本、没有台账——公告没有牙；
- 缺治理：台账记得很全，条目一条不少，可没有一处写着谁裁的——记录成了档案；
- 缺架构定义：会开得很勤，可没有内容可管——治的是一个空集。

第三种假，是“越早越好”。治理不是把口子卡得越靠前越好。一条改动该不该进治理，看它动不动目的、原则、跨系统的约定；动不动就往上报，等于把治理的精力全耗在日常变更上。

一条能用的体检法。治理那一堆条款可以压成五个问题，问一次变更就够：

1. 它进范围了吗——这条变更属于受治理的架构吗？
2. 谁有权定——这一栏写得上名字吗？
3. 依据是哪一条——目的、原则、强制项，还是“我熟”？
4. 谁该知道——要传达给谁、记在哪里？
5. 改这条要什么手续——和“改那个字段”是一样的手续吗？

恒信那次对五问的回答，是五个空。

### 16.4.6 认知反转

到这里，本章最反直觉的一句可以说了。

一个组里最会算的人，往往不是说了算的人。这话听着不舒服，但它是分工，不是不信任。算得清是你的本事，算不算数得由另一处作出，因为“算”这件事有个天然的缺口：算的人总偏向自己的结论。让另一处来点头，不是为了压住谁，是为了让那条结论对全组都成立。

所以治理不是多出来的一道手续。真正多出来的那道手续是：会开了、字签了、流程走了，可没人说得清这一版谁有权改。治理要做的恰恰相反：把“算不算数”这件事交回给人，交给一个说得清名字的地方，由它决定这条判断算不算数，并且承担它。

这也是本章标题上那两个字的意思：效力不长在本事上，长在有人愿意担下来上。

判断与效力，从本章起分家。往后你再看到一份“已批准”的架构描述，会多问一句：谁批的？按什么程序？改它要走哪条路？依据是哪一条？

## 案例进展①｜恒信：买回来的是工具，不是规矩

那年恒信的数据出了事：月底对账，合同系统和财务系统对不上。是业务方的郑亮先对出来的，他把两张对账单并排推到桌上，一行一行地核。查了三天，原因是两边的产品编号规则不一样：合同系统按自己那套号编，财务系统按另一套编。

“同一份合同，合同系统里是 2026-0417，财务那边是 HT-0417。差在编号，不差在数据。”

王默翻着自己那本运维台账接了一句：“编号规则我也比过一遍——两边各有一套，谁也没写进配置项。”

“这规矩谁定的？”

没有人答得上来。

曹峰的方案很直接：买一套数据治理平台。“数据干净了，合同系统用起来才省心。”选型会上他签了“同意”——五百万，配一个季度开一次的委员会。

平台上线半年，效果确实有，数据清洗得比人工干净。可到第二年，王默把运维台账拿了出来：负载不到三成。

“我们买的是一套清洗工具，”他说，“它每天在跑，跑的是我们给它的活。可'谁定规矩'这件事，没人接。”顿了一下，他补了一句：

> “桨还是那把桨。”

问题不在平台。这套系统能清洗数据，却答不了“数据归属唯一”由谁定、“编号规则”改了谁批、“两边对不上”按哪一条裁。顾言在会上把这层点破了：

> “标准 §6.4.1 f) 那一项，要的是'建立指导原则、策略与指令'，落到 §6.5 是几份工作产品。你们那五百万买回的是清洗，不是指令。你们买的不是治理，是一把桨——治理要的是舵：谁定方向、谁担最终责任。这两样不在同一个地方买。”

第一种假，恒信有实物：季度开的那个机构，名字叫“数据治理委员会”，议程三条里两条是通报、一条是排期，没有一项是“立规矩”。它不产出指导原则、不产出指令、不产出符合性状态报告。按本章的说法，它是过滤器而不是治理机构：只在事后审，从不事前定。

第三种假，一个组里常见的样子是这样：凡改动都往上报，理由是“稳妥”。三个季度下来，治理会开了十几次，每次都过一堆日常调整——真正该它裁的那条（“归属唯一”要不要动）反而没人提，因为它淹没在琐事里。治理的精力是有限的；把日常变更送上去，等于把治理用废。

## 案例进展②｜冰链：断链不是技术问题

M3 冷链那次断链，第一反应是查技术：温度探头坏了吗？网关掉线了吗？时间戳是不是漂了？查了两周，技术上都对：探头合格、网关在跑、时间戳有校准记录。

报告初稿的结论是四个字：技术问题。

产品经理林涛把稿子退回来了。他问的不是技术，是三个问题：

> “这次断链，谁知道？”“知道之后，谁有权决定这批货怎么办？”“这个决定，记在哪一版上？”

三个问题，一个都没有答案。石军的接口老规矩里写着“连续性不得中断”，可那是给设备定的；断链之后谁拍板放行、谁签字担责，规矩里没有这一条。

于是结论改了。这不是技术问题，是框架级问题：该有人管的那一层，没有人在管。林涛在报告末尾添了一句：

> “设备没坏。缺的是那条'谁定、谁裁、谁担'的线。§6.2 c) 写的是'有理由支撑的架构决策，由具有适当职权者作出并传达给利益相关方'：这三样，那天一样也没有；我们只答了'技术问题'四个字。”

石军后来把这三问抄在自己那本老规矩的首页上。有人问他为什么抄这个，他说：

> “我这一辈子记的都是'怎么查得出来'。这三个问的是'谁说了算'——这两样不是一回事，可它们得在一处。”

系统工程师小张把这三问也抄了一份，抄到末页她停住：

> “这份规格自己呢？改了谁批，批完谁评它有没有被照办？管着'谁裁'的那份东西要是自己不受控，它裁过的事，下一版就能推回去。光定谁裁，只答了前半句。”

没有人接这话。

那年冬天，冰链的接口视角规格多了一行，是林涛加的：“本规格的变更，由架构评审组裁定，须记录理由与影响范围。”一行字，把断链那天缺的那条线补上了。

## 案例进展③｜恒信：会开了，谁定的没说

季度委员会照开。议程三条：数据质量问题通报、下季度清洗重点、遗留事项。

那一年恒信开了四次。到年底，王默翻会议纪要，翻出一个规律：每次都有决定，每次都没落款。

比如第三次会议定了一条：合同系统的产品编号以财务系统为准。纪要上写着“会议决定”。可这条决定是谁提的、谁反对过、依据是哪一条、从哪天起生效，都没有写。两个月后两边又对不上：开发按纪要改了，财务说不知道有这回事。

曹峰去问项目经理：“这条到底定了没有？”项目经理翻出纪要，指着“会议决定”四个字说：“定了啊。”

“定了”和“算数”，在这里第一次显出区别。会开了、话说了、纪要写了——可没有人有权说“这条从现在起算数”，也就没有人担这条决定的后果。

这件事后来成了那五个问题的原型：它进范围了吗、谁有权定、依据是哪一条、谁该知道、改这条要什么手续。恒信那一次，五个空。

第二年开春，恒信把委员会改了。议程从三条改成两条：一是审定架构原则与红线的变更，二是评审符合性状态报告。通报和排期挪出了会议室，回到周会上。

王默在改造后的第一次会上说了一句：

> “以前我们开的是进度会，现在开的才是治理会。区别不在开几次，在议程第一条：这一条算不算数、谁签的字。以前那三条议程，通报、排期、遗留事项，没有一条要人担责。”

曹峰散会时多问了一句：“那今天定的这条，谁写的?”项目经理把纪要递给他，第一行写着三个字：“**裁定人**”——后面是他的名字。

他愣了一下，签了。

## 小结

本章走完，回到那句话：治理不替你算，它只决定你算出来的算不算数。

本章做了三件事。

- 站位置：治理是组织里那套制度系统，也是那套系统一直在做的动作；它守不变量，管理调变量；它与架构定义并行，不在架构之后。
- 说理由：它保的是“架构还符不符合目的”——目的会变、架构会偏，所以它是持续的动作，靠给出指导、评定符合性、对结果问责转起来。
- 给效力：判断算不算数，不由算的人定；判据不能自己裁自己；机制不等于过程。

暗线那一笔：那天下班前，罗建在自己的架构文档扉页上添了一行字：“不可逆的决策，过治理”。他以前以为架构师说了就算；本章之后他明白：他掌舵，但船不是他的。

用第 1 章 §1.4.4 那把尺子量一下：本章把“判断”这件事从第 ② 级推到第 ③ 级，从“知道治理与架构定义并行”，推到“能推演出判断与效力分家，并说得出效力从哪来”。

# 附录

## A 本章概念总图

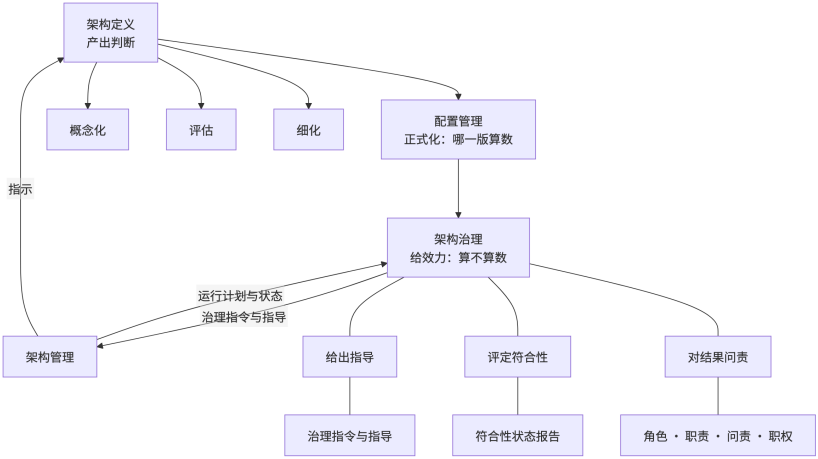



图 16-1

> **读这张图注意**：上面那条“架构定义 → 配置管理 → 架构治理”，说的是同一个判断走的三段——产出判断、正式化、给效力——不是三道工序。治理与管理那一层与架构定义并行：指令往下流，运行计划与状态往回走；架构管理再往架构定义（概念化、评估、细化）下指示。

## B 本章的能力级

① 表征：治理、管理、架构治理、架构管理四个词分得开；governing／governance／governs 三槽分得开；§6.5 列出的工作产品报得出来；“架构判断的效力”一句话说得清。

② 图式：画得出“判断 → 正式化 → 效力”三段；画得出治理与管理并行、指令与状态双向流动那两条线；说得出三层基线谁在上、谁在下。

③ 心智模型（本章主级）：推得出判断算不算数不归算的人定；判据不能自己裁自己；机制不等于过程；符合不是一次达成的。

**能答全四问**，才算拥有——拿到一条“已批准”的架构决定，答得出：谁批的／按什么程序／改它走哪条路／依据是哪一条。

本章的能力落在哪里。第 9 章把工程师的能力列成六项；本章主要长的是其中的 ④ 职业伦理（目的、原则、红线要有人守，改它要走程序）；⑥ 可复现意识已在第 15 章长成配置与基线，本章接住它的下半截（谁批的、按什么程序、改它走哪条路、依据是哪一条，都要留得下）。第 18 章会把这两项接着往上加，合成架构师那一份清单。

## C 自查 · 这些说法对吗？

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “治理就是把管理层升一级” | 把治理当管理 | 治理守不变量、管理调变量；组织侧看“给出与实施、评定两级、问责的设立与承担” | 这话里有没有“给出指导”那一半？没有，那就是管理 |
| “架构定完了，交给治理去审” | 把两条线读成串行 | 正本明写各过程可并发，指令与状态双向流动 | 能指出治理在**架构定义一头一尾**的两处伸手吗 |
| “治理委员会审批变更” | 把治理读成审批岗 | 机制不等于过程；正本那几项活动里没有“审批” | 这条变更按哪一版规矩办？答得出，就不必上治理 |

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “基线就是最新那一版” | 混淆基线与版本 | 基线＝被批准、被冻结、能当参照 | 谁批准的这一版 |
| “架构合格了，系统就合格了” | 漏了实物那一层 | 42020 §6.4.5 注 4：仅检查架构的符合性可能不够充分 | 三种评定各评过了吗 |
| “**三层基线**就是功能／分配／产品基线” | 两套三分法混讲 | 三层基线是本书复合词，正本只有单数“基线” | 说的是“谁有权改”那条轴，还是“成熟到哪一步”那条轴 |
| “配置管理是治理的一只手” | 层级错位 | 配置管理是过程（15288 §6.3.5）；治理建的是机制 | 治理建的机制和实际走的过程，分得开吗 |
| “治理就是多开几个会” | 把动作当成了机构 | 治理的那几项活动里“作出决策”只占一项；日常是看与立 | 上一次治理会产出了哪一条指令 |
| “有版本号就算是受控了” | 把版本当受控 | 受控要的是“谁批的、按什么程序、改它走哪条路、依据是哪一条”答得出来 | 这四句答得出几句 |
| “指令定了就不该再动” | 把规矩当碑 | 正本 §6.4.6 要求一轮收尾时评审并修订指令 | 这条指令上一次被评审是什么时候 |
| “治理越早介入越好” | 把门槛当政绩 | 该不该进治理，看它动不动目的、原则、跨系统约定 | 这条变更动到了哪一层 |

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “架构师既然最懂，就该他拍板” | 把技术权威当治理权 | 技术权威位不等于治理位；正本 §6.2 c) 要的是“具有适当职权者” | 这一栏写的是名字，还是“我们组” |

## D 练习

1. 找一个你们组里“已批准”的技术决定，写出四句：谁批的、按什么程序、改它要走哪条路、依据是哪一条。写不出的那一格，是谁的位置？
2. 翻出最近一次变更评审的纪要，用“五问”（进范围了吗／谁有权定／依据是哪一条／谁该知道／要什么手续）逐条对一遍。
3. “我们建了治理机制”——请为制度系统的五样各举一个实例；举不出的，写明缺哪一样。
4. 找一条“所有产品都要遵守”的规矩，问：这条规矩的判据是谁定的？如果它只是一个项目组自己定的，它算不算架构约束？
5. 你们组里有没有“开完会就定了”的事？判断它属于哪一类：管理决定，还是治理决定。
6. 用自己的话说清“治理”和“管理”的区别——不许用“掌舵”“划桨”这类说法。
7. 找一条你们组里“改了但没人通知”的规矩，把该通知的人列出来，并写明依据。
8. 把你们最近一次“升级上报”的决定找出来，写明：为什么要升级（超出谁的权限）、报到了哪里、结果谁传达。
9. 假设你们明天要建一个**治理机构**，写出它的第一条指令；再写出这条指令将来“怎么被改”。

10. 用本章说的那几类假给自己所在的组做一次体检，各举一个可能的例子，并写明判据。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。


**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022《架构描述》（§6.10 决策与理由）
- [2] **R-05** ISO/IEC/IEEE 42020:2019《软件、系统与企业 — 架构过程》（§5 总则与并发流向、§6 架构治理过程、§7 **架构管理**过程、§9 **架构评估**过程、§10.2 预期结果及其注、附录 G 资料性）
- [3] **R-28** ISO/IEC/IEEE 42030:2019《架构评估》（评估原因）
- [4] **R-37** ISO/IEC/IEEE 15288:2023《系统与软件工程 — 系统生存周期过程》（§6.3.5 配置管理过程、§6.4.4 系统架构定义过程）
- [5] **R-38** ISO 9000:2026《质量管理 — 基础和术语》（§3.11.2 评审）

**经典文献**

- [6] **R-57** Tricker, Corporate Governance（1984）—— 「掌舵／划桨」的分工比喻：§16.2.3 用它说「治理管舵、管理划桨」（本章练习 6 要求读者不用这套比喻也能说清区别）

# 第 17 章 一路贯通：从需求到上线运行

## 17.1 引言

前面十六章，一章讲一个词。每个词都讲清了：它是什么、不是什么、往哪用。可有一个问题一直没答：这些词放在一起，是什么关系？

工程现场从来不缺词。需求、设计、评审、架构、配置、治理，每一件都有人做，每一件都有流程、有模板、有责任人。可系统还是会在某个地方出问题，而且常常出在所有人都做对了的地方。

本章不引入新概念。它做另一件事：把本书讲过的那些词放回同一条时间线，看它们在哪一段起作用、在哪几处要连上、由谁负责连上。

### 17.1.1 误解现场

恒信集团，电子签章上线两周后。

郑亮（业务）拿着两张表找到王默（运维）：“同一个星期签的合同，台账里的编号和签章系统里的编号对不上。月底就要对账，财务那边过不去。”

王默先查系统：“是不是部署出了问题？”部署记录、版本基线、配置项一条条核过，都对得上。“不是部署的事。”罗建（架构）把编号标准翻出来：“规则也没变，上季度刚定的，法务归口，全集团统一。”

变的是编号的**来源**：以前编号在台账登记时生成；现在由第三方服务商在签章完成那一刻带回来，写进业务库，绕过了台账。

顾言（设计）把这条线追了一整天。需求文档里没有“签章编号要不要进台账”；设计树里，“CA 对接”和“台账登记”之间没有第

三片叶子；测试用例里没有“签章→台账”的场景；变更记录写的是“新增 CA 对接模块”，没有“编号来源变更”。

那条线哪儿都不在。

“我们不是哪一步做错了。”顾言合上电脑，“需求评审、设计评审、验证、确认、基线、变更控制，一样不少，每一步都做得挺像样。问题在这儿：这些步骤各管一段。需求管需求，设计管设计，测试管测试。'编号从哪来'这件事，正好落在两段之间，谁都不管。”

同一天，三千公里外，冰链的实验室里，嵌入式硬件工程师石军和产品经理林涛也在发愣。

M3 刚推完固件 v2.5，断链自动补采，固件、云端、App 都测过，也验收过。疾控的审计员复查批次追溯，只问一句：“这批补采记录的时间戳，是按哪一条规定需求定的？”

石军（固件）答：“设备本地时钟打的戳。”

吴浩（云端）答：“归档按服务器时间。”

一个箱子，两个时间。小周（批次）翻开记录：“批次上全是 v2.5 采的，没错。可记录里的时间，两种都有。”两边的时间差了几个小时；这一段算不算断链，追不下去。

“固件没问题。”石军说。

“云端也没问题。”林涛接上，“时间戳按谁的时间算，需求里没写成一条规定需求，设计里没落成一片接口的叶子，验收的时候也没有一条尺子照得到它。两边都没问题，有问题的是中间那段没人管的地方。”

### 17.1.2 本章要回答什么

两场事故，一南一北，一个跨部门的业务流程，一个硬件加软件的冷链温控箱。查到最后，两边说的都是同一句话：每一步都没错，错在两步之间没人管。

读完本章，你应该能回答：

1. 从一条需求到上线运行，本书讲过的那些概念，各自在时间线的哪一段起作用？
2. “所有概念是同一件事的不同切片”是什么意思？切片之间靠什么连上？
3. 为什么每一片单独看都合格，系统还是可能失败？
4. 顺着时间切和横着时间切，差在哪儿？为什么横着切的那几样不能当成时间线上的一站来做？
5. 一个跨部门的业务流程，一个硬件加软件的冷链温控箱，两个最不像的系统，为什么走的是同一条时间线？
6. 为什么说“缝合不是最后一步，是每一步”？
7. 一条变更上的接缝，怎么列成一张缝合清单？每条缝要写清哪几件事？
8. 为什么“不属于任何一片”等于“没人负责”？
9. 技术上线和工程成功，差在哪一步？
10. 换一个行业，这套概念还成立吗？

### 17.1.3 本章的骨架

本章的主线是一句话：每一片都合格，缝还是会断——一路贯通不是最后多出的一道工序，是每一站都在做的事。

围绕它，本章先讲怎么分、缝在哪儿、怎么合上，再一站一站走完全程，最后拿两个陌生行业证一次：

- 概念怎么分，缝在哪儿，怎么合上（17.2）：本书讲过的那些概念，是同一个系统的不同切片。切片只有两种切法：顺着时间切、横着时间切。片与片之间靠四种缝连上：承接、参照、记录、问责；连完之后，要能写成一张清单。
- **逐站问一句，一站不跳**（17.3），把恒信和冰链两条线放上同一条时间线，一站一站走：每一站接手什么、交出什么，它前后那道缝在哪儿，这一站的账落在哪份产品数据上。
- **换一个行业，这套概念还成立吗**（17.4）：拿两个本书之外的行业各走一遍，看这套东西挑不挑行业。

三块合起来回答一件事：那些词不是彼此不相干的东西，是同一件事的不同讲法；把它们放回系统上，才看得见它们之间的缝。而缝，是要有人管的。

> **术语注**：“切片”“顺次切片”“横切切片”“接缝”“缝合”“缝合清单”这几个名字与判据，是本书的主张，不是任何标准的条文。它们顺着第 15 章“版本顺着流、基线垂直切”长出来，欢迎检验，也欢迎校准。

---

## 17.2 概念怎么分，缝在哪儿，怎么合上

### 17.2.1 每一步都做对了，系统还是失败

把恒信和冰链两场事故摆在一起看，会发现它们不是两种病，是同一种病在两种行业里的两副面孔。这里说的“每一步”，就是后面要讲的“每一片”。

恒信这边，出问题的地方在部门和职责之间。“签章编号由第三方服务商生成”是一条依赖，可这条依赖归谁？需求分析师觉得这是设

计的事，架构师觉得这是集成测试的事，测试觉得这是需求没写的事，配置管理的人觉得这是变更记录的事。每个人在自己的那一段里查了一遍，都找不到“编号来源”这一项。因为它本来就不在任何一段里，它落在两段之间。

冰链这边，出问题的地方在两个部件之间。固件记设备本地时钟，云端按服务器时间归档，两段各自都正确，中间缺一条“时间戳按谁的时间算”的约定。这个约定，在需求里是一条没说出口的隐含需求，在设计里是跨叶子的接口，在测试里是集成用例，在配置管理里要定格成一条接口基线。它同样不属于任何一段。

前面十六章治的是一种病：同一个词各指各的，同一种东西叫了不同的名字。这两场事故是另一种病：**词都用对了，事还是漏了**。

病根在哪儿？把“需求”“设计”“评审”“架构”“配置”“治理”这些词，当成了彼此不相干的东西。每一件都做到位了，可系统不是几件东西拼起来，它是一件事的几种讲法：你换一个角度看，就换一套词。

这里要分清一件事：不是说每个人只看懂了一部分。恰恰相反，书里讲的每一个词都对，它准确地说出了同一件事的一个侧面。出问题的地方不在侧面里，而在侧面与侧面之间：两边的话都说了，中间那一句没人说。

三个最容易犯的错，就长在这处地方：

- 把缝断了当成部件坏了。恒信去改编号格式、冰链去查固件逻辑，都治不了病。部件是好的。查问题的时候先问一句：坏的是一个零件，还是两样东西之间那件事？
- 把“不属于任何一片”当成“不归我管”。凡是落在所有切片之间的依赖，必须有一个写得上名字的人；没有这个人，那里就是将来出问题的地方。判据很简单：这一处有没有一个能写下来的名字。

- 把技术上线当成工程成功。服务器部署了、接口通了、验签能验了，这说明技术这一侧通了；法务真能用它审合同、财务对账真能对上、业务方愿意用，工程系统才算在运转。第 2 章讲过：**工程系统包含技术系统**，它承诺的是整个系统能用起来，不是设备跑起来。判据也很简单：用的人接上了吗。

### 17.2.2 顺着时间切，还是横着时间切

要把一件事看明白，先得有个切法。本书讲过的那十几个词，是同一件事的十几种切法；切法只有两种。

一种是顺着时间切。一条需求从提出到上线，中间有几段路，一段接一段，前一段的产出是后一段的输入：

- **需求**：把诉求变成能当依据用的东西（第 3、4、5 章）；
- **设计**：把需求逐层分下去，变成能造、能集成的方案（第 6、7 章）；
- **把关**：拿三把尺子量一遍，目标对不对、需求满足没有、用途接得上没有（第 8 章）；
- 运维：上线、在役升级、直到退役（第 2 章的外框）。

这四段是顺次切片。它们的顺序不是习惯，是依赖：没有前一段交出来的东西，后一段无从下手。其中“设计”这一段交出来的形态，是产品数据（第 7 章）；数据不另占一段，就挂在设计那一段上。

另一种是横着时间切。有三样东西不在时间线的某一点上，而是穿过每一段：

- 架构：从需求到运维，每一段都有“这件事定下来之后还能不能改”的决定，都有要立住的概念与原则（第 10、11、12、13 章）；
- 配置：从需求到运维，每一段都有产出要标识、要定成哪一份、要能说清改了什么（第 15 章）；
- 治理：从需求到运维，每一段都有“谁有权定、谁担责”的问题（第 16 章）。

这三样是横切切片：都不占时间线上的一段。

把它们画在一张图上，是这样：




写成一句话：顺次切片是时间线上的那几段，横切切片是穿过每一段的那条线。顺着流看，你看到的是接力；横着看，你看到的是每一段上都要做的那几件事。

第 9 章讲的工程师不在这两种切法里：它讲的是人，不占任何一段。

### 17.2.3 架构、配置、治理，不能当一站做

最常见的那个错位，是把横切的那三样当成顺次的一站来做：先做需求，再做设计，再“做架构”，再“做配置”，最后“做治理”，一件一件排好。

这样排，看着整齐，实际会把最要紧的那几件事漏掉。

漏在哪儿？横切的那三样，每一段都要用上一次。

- 需求那一段就有架构问题：业务边界怎么划，这是一个决定；
- 设计那一段也有：模块接口怎么定；
- 运维那一段还有：升级策略怎么选。
- 配置同理：需求定稿要立一版，设计分配定稿要立一版，实现交付还要立一版；

· 治理同理：编号来源谁归口、采集算法能不能动，这类问题不分阶段，哪一段碰上都得裁。

把它们排成时间线上的一站，等于只在一站上做这件事，其他站放手。

恒信那场事故就是这么来的。“编号来源变更”被当成了一次普通的字段改动，在改的那一站上处理完就过了。可它是数据模型层面的东西，需求那一段、设计那一段、把关那一段、运维那一段都得跟着动。这个判断没人做，因为在这套排法里，“架构”是前面某一站的事，早就过去了。

冰链那次也一样：时间戳按谁的时间算，被当成固件那一站的技术选择，实际上它是一条**数据所有权**的决定，穿过所有段。

这里要防一个误会：下面两种说法一旦混用，意思就全拧了。“顺次”和“横切”这两条线，作为内容的分类是成立的，作为流程的分类不成立。换句话说，“架构、配置、治理穿过每一段”说的是它们的内容覆盖面，不是说要另开三条并行的流水线；它们在每一段里与那一段的活揉在一起做，不是各走各的。

第 14 章讲的“架构与设计的交接”，就是这道题的实例：架构横着切过来的那一份描述，被设计这一段接住，一级一级往下走。架构交出去的不是另一条流程，是一份要遵守的约束。

### 17.2.4 接缝：切开之后靠什么连上

切开是为了看清，不是为了分开。可这一刀下去，哪些地方会断开、要有人接住，前面还没说。

本书讲过的词，散在各章里，看上去各有各的用法；把它们放回一条时间线，会看到它们连的其实是同一批地方。这些连接处有四种：

第一种，承接：上一段交出来的东西，下一段能直接接住吗？规定需求被设计接住，设计出的方案被验证对照，架构描述交出去之后，

接住它的是另一份描述，也就是设计描述。相邻两段之间，总有一份东西是交出去的那一份、一份是接住的那一份；交出去的那一份说不清，接的那一段就无从下手。

恒信就断在这一条上。“编号从 CA 来”这个事实，需求那一段没写成条款，设计那一段就没有东西可接；两片叶子各管一头，中间那条过渡谁也没写。冰链的接口也断在这条上：固件和云端两边都写对了自己那半句，可“两端时间戳要对齐”这句话，谁也没有交出去。

第二种，参照：几段对同一件事的说法，对得上吗？关于同一个对象，有三问要在几段之间接上：应该是什么、实际是什么、怎么变成这样。需求回答“应该是什么”，验证回答“实际是什么”，配置记录回答“怎么变成这样”。三个答案都对得上，才是同一个系统；对不上，先别互相指责，把三个答案放回同一件事上核。

恒信那两周的现场，就是三个答案对不上的样子：台账说编号应该是这个，签章系统说编号实际是那个，变更记录里查不出它怎么变成这样。冰链也一样：一台箱子的时间，设备说一个、服务器说一个，谁也说不清哪个是“这件事的样子”。

第三种，记录：人走了，还查得到吗？决策记录、配置信息、追溯矩阵、评审记录，都是在往同一份组织记忆里写。这一种缝靠记录连上，不靠默契。口头约定的接口、没写进基线的变更，就是恒信和冰链两次事故的共同死因；人一走，缝就断了。

这一条最容易被低估。第 12 章的道理在这儿：一个决定当时说清了，也点头了，可要是没写下来，它就不存在组织里，只存在那个人的记忆里。第 15 章解决的也正是这件事：把判断定成“哪一版是定下来的那一份”，后来的人才查得到。

第四种，问责：这条缝有名字、有主人吗？有些缝不是手艺能连上的，得有人有资格为它负责：所有权定给谁、规则由谁定、由谁执

行。恒信“编号来源”断掉，本质是没人被授予裁决“编号从哪来”的资格；冰链“时间戳按谁的时间算”补上，本质是有人为这条接口担了责。

前三种缝合靠手艺：识别得出来、写得好、记得住；这一种靠资格。这也是为什么第 16 章要单独讲一次“谁有权定”。

四种连法，配四句自问：

- **承接**：这一段的产出，下一段能直接拿来用吗？
- **参照**：几段对同一件事的描述，对得上吗？
- **记录**：这条缝，人走了还查得到吗？
- 问责：这条缝，有名字、有主人吗？

四句都答得上，缝是连好的；任何一句答不上，那里就是将来的事故现场。恒信的编号、冰链的时间，都不是技术事故，是四句里有一句没人答。

### 17.2.5 缝合清单：写得出来，才缝得上

四句话是判断，判断落到纸上，才是活的。

一张缝合清单，每一行只写三件事：

- **缝在哪两片之间**：上一段交出来的是什么、下一段接住的是什么；
- **靠哪一种连上**：承接、参照、记录、问责；
- **责任人是谁**：一个能写字的名字，不是一个部门。

三样写全，这条缝才算有人管。写不出来的，就是埋在系统里的雷：不是它不存在，是没人为它负责。

恒信事后补的那张清单，列了六行：

| # | 缝在哪两片之间 | 靠哪一种连上 | 责任人 |
|---|---|---|---|
| 1 | 需求：场景里那句话 ↔ 需求规格里那条条款 | 承接 | 顾言 |
| 2 | 设计：“CA 对接” ↔ “台账登记” | 承接 | 罗建 |
| 3 | 把关：签章 ↔ 台账 | 记录 | 苏婷 |
| 4 | 架构：编号来源的决定 ↔ 后续各段的实现 | 记录 | 罗建 |
| 5 | 配置：编号来源的变更 | 记录 | 王默 |
| 6 | 治理：编号归谁归口 | 问责 | 委员会（归口到法务） |

六行逐条对着上面那三样写。

- 需求那一段：补一条“签章编号必须经台账登记、纳入法务归口的编号标准”；
- 设计那一段：“签章编号映射”作为一片叶子，补在两片叶子之间；
- 把关那一段：把“签章→台账→对账”补成一个集成验证场景；
- 架构那一段：“编号来源变更”按数据模型层面的决定重做一遍；
- 配置那一段：变更记录写进基线；
- 治理那一段：归口由治理裁一次。

六行，六处缝。

冰链补了三行。需求规格里补一条：“补采记录的时间戳＝设备本地时间，原样存储，展示时统一换算。”固件与云端两棵树之间，补一片“时间戳对齐”的接口叶子。再跑一趟真实隧道，把“补采时间戳和原始记录时间戳对齐”验一遍。

这六处，其实是同一个漏法漏了六次：只要缝上任何一处，都不会炸。

这句话里有清单的分量：清单不是把简单的事说复杂，是把散在几个人脑子里的默契，搬到一张谁都能看的纸上。一张清单在手，接缝有几个、在哪儿、谁管，一眼看得出；换个人来，也接得住。

---

## 17.3 逐站问一句，一站不跳

### 17.3.1 恒信和冰链，走同一条时间线

上一块把切法和接缝讲完了。现在把它们用起来：拿两条真实的线，走一遍全程。

一条是恒信的电子签章：一个跨部门的合同审批流程，靠软件加组织协同。一条是冰链的断链补采：一台冷链温控箱，硬件加固件加云端。一个是流程，一个是设备，看上去八竿子打不着。

可它们走的是同一条时间线。

为什么敢这么说？因为这条时间线不描述任何具体技术。它描述的是同一件事：一个系统，从“要成为什么”被确定的那天起，到它退役那天止，中间被一次次变更唤醒，每一次都要走的那几段路。

下面一站一站走。**七处＝顺次的四段＋横切的三样**；下面为省事，把这一站、这一样都叫“这一处”。每一处只答三问：接手什么、交出什么；它前后那道接缝在哪儿；这一处的账，落在哪份产品数据上。

| 这一处 | 接手什么 → 交出什么 | 前后那道缝 | 这一处的账落在 |
| --- | --- | --- | --- |
| 需求 | 一句话 → 能当依据用的需求 | 没说出口的，有没有被写下来 | 需求规格 |

| 这一处 | 接手什么 → 交出什么 | 前后那道缝 | 这一处的账落在 |
| --- | --- | --- | --- |
| 设计 | 需求 → 能造、能集成的方案 | 两片之间那条接口，有没有人接 | 设计描述 |
| 把关 | 设计与实现 → 三份能查的证据 | 跨两片、又没写进任何一片的那条依赖 | 测试记录、确认记录 |
| 架构（穿过全程） | 每一段的决定 → 架构描述、决策记录 | 目的关、代价关有没有带进每一段 | 架构描述、决策记录 |
| 配置（穿过全程） | 候选 → 定下来的那一份 | 片与片之间的约定，有没有留档 | 配置信息、基线 |
| 治理（穿过全程） | 算出来的结论 → 裁定与指令 | 这条缝有没有一个有资格的人负责 | 治理指令与裁定 |
| 运维 | 上线 → 在役、退役 | 所有接缝在这里总验一次 | 运维记录、退役档案 |

恒信这条线，接的是郑亮在走廊里的一句话：“新法规下来了，电子合同得有电子签章，要有 CA 验证。”冰链那条线，接的是疾控的一纸要求：“断链之后，数据要能补回来。”

两句话都很短。接下来的七处，就是这两句话怎么变成两个真在跑的系统。

### 17.3.2 需求：先弄清要成为什么

这一站接手一句话，交出能当依据用的需求。（第 3、4、5 章）

恒信这边，按三年前的做法，郑亮那句话会直接变成一张开发任务单。这次顾言没有急着写条款，他先问了一句：“先别管系统要做什么，先讲清楚它怎么被用。”

于是他写了一份运行概念，讲了一个场景：郑亮收到一份电子合同，系统验证签章真实有效；他审条款、改一处、重签；归档的时候，

这份电子合同和纸质合同进同一个台账，用同一套编号。下个月财务对账能对上。

故事讲完，才轮到条款。验签算法、防篡改、归档格式，一条条落下来。最后顾言在需求规格里单独划了一条：“电子签章的合同必须进台账，编号与既有编号标准一致。”

这条没人提过，它是从场景里挖出来的。需求这一站最要紧的手艺，就是把没说出口的东西挖出来，写成条款。藏在场景里的那句话不挖出来，后面每一站都够不着它。

冰链这边，林涛也是先写场景、再落条款：一辆 M3 在隧道里断了 30 秒链，出隧道后自动补采，把断档期间的温度补传回来，平台上能看到补采记录。写到“补采的时间，就是设备当时真实记录的时间”这一句时，他停了一下：设备的时间和服务器的时间，不是一个时间。他把“时间戳按设备本地时间算”写进需求规格，标注“需利益相关方确认”。

这一站前后那道缝，在“没说出口”和“写下来”之间。恒信的“编号进台账”、冰链的“时间戳按设备时间算”，本来都藏在场景里；写下来，它们才成为下一站能接住的东西。

**这一站的账，落在规定需求上**，也就是需求规格那份产品数据（第 4、5 章）。这是设计那一段要接住的那一份。

### 17.3.3 设计：需求怎么变成方案

这一站接手需求，交出能造、能集成的方案。（第 6、7 章）

第 6 章讲过：设计不是写文章，是把需求一层层分下去。恒信的罗建这次没写六十页散文，他列了一份框架定义：签约发起、CA 对接、验签、台账登记；每一块下面挂详细定义，每一条有编号、有描述、有指标、有验收标准。

分完之后，顾言来挑刺。他指着两片："这片是'CA 对接'，管签章验证；那片是'台账登记'，管编号落账。中间呢？CA 返回的签章编号，要不要进台账，怎么进？"

罗建没有急着答。他把需求规格翻回去，一条一条找依据。那条"电子签章的合同必须进台账"，他挂到了"台账登记"那一片上。可"编号从 CA 来"这个事实，落在两片之间。

他补了第三片："签章编号映射"，管"CA 返回的编号怎么变成台账的编号"。这一片不是从哪条需求搬下来的，是从两片之间的空处补出来的。

冰链的设计是三棵树：固件、云端、App，靠接口对接。时间戳那条需求拿给两端看，固件那边写"设备本地时钟打戳"，云端那边写"按服务器时间归档"。两棵树各自都对，中间没有一片说"两端的时间戳要对齐"。

石军问："这条谁负责？"没人答得上来。最后吴浩开口："固件负责打设备时间，云端原样存，展示的时候统一换算。"这句话是在会议上口头定下来的，后来也没写进任何一片叶子。

这一站前后那道缝，长在两片之间。需求里的约束有没有传下来，是可以查的；两片叶子之间那条接口，不在任何一片叶子上，得有人专门去看。恒信补了一片叶子，冰链那条留在了会议纪要里。

**这一站的账，落在设计描述上**（第 7 章的产品数据）。它就是下一站要对照的那一份。

这一站的入口，先要接住架构交过来的那一份（第 14 章）：设计开工前，先看架构描述、把约束取下来。架构横着切过来的那一份，是在这里被接住的。

### 17.3.4 把关：各对一个参照，各出一份证据

这一站接手设计与实现，交出三份能查的证据。（第 8 章）

三把尺子量三样东西：评审对目标，验证对规定需求，确认对预期用途。

恒信的电子签章走到验收。罗建打算出一份测试总结，顾言拦住：“三件事，分开办。”

评审抬头看目标：电子签章要达成的是法务能离线验证签章真实性、财务对账不出错、业务方在真实流程里签一份合同不卡壳。评审会上，郑亮问了一个需求里没有的问题：“客户发来的 PDF 是打印件，扫回系统，能验吗？”这是对目标的追问，不是对条款的核对。

验证低头对条款：验签用例、防篡改用例、归档用例，对着需求规格一条条给证据。测试全绿。

确认出门看用途：最后一步不是让测试员替业务方签字。郑亮带着法务的人，在真实的审批流程里审了一份电子合同，从收到、验真、审批到归档，走了一遍。确认的参照在场景里，不在文档里。

三把尺子都过了，才轮到“验收通过”四个字。可顾言心里记着一件事：那一片补出来的“签章编号映射”，在验证用例里没有。测了“CA 返回的编号”，也测了“台账登记编号”，没测“CA 返回的编号落进台账”。

冰链这边，测试季昭把确认做到了隧道里：一辆真的 M3 装上 v2.5，在真隧道里断了 30 秒，开回来，平台能对上补采记录。同样，确认验的是“补采功能能跑通”，没验“补采时间戳和原始记录时间戳对齐”。

这一站前后那道缝，在“写下来的”和“没人写下来的”之间。三把尺子量的都是已经写下来的东西：规定需求、设计叶子；没人写下来、又没写进任何一片的依赖，三把尺子都照不到。

这一站的账，落在验证与确认的证据上：测试记录、确认记录。它们是“这东西行不行”的凭据。

### 17.3.5 架构：定什么、权衡什么、写什么

这一站不排在时间线的某一点上，它穿过每一站。（第 10、11、12、13 章）

前面三站都是接力：上一站交出来，下一站接住。这一站不一样。架构不是时间线上的某一段，是每一段上都要做的那几件事：

- **定什么**：哪些概念和原则要立住；
- **权衡什么**：方案怎么选，定下来之后还能不能改；
- **写什么**：怎么写清楚，让下游能照着做。

先说定什么。每一站动身前，都要问两句：

- 目的关：这个决定守的，是不是“目的不走样”？去掉这个决定，目的是不是就落空了？
- 代价关：它有多难改？按难改的程度，决定给它什么待遇、排在什么时候。

两关的次序不能颠倒。先过目的关，再过代价关。难改不难改，只决定这件事排在哪、审多严；够不够格进架构，看的是目的。恒信那次事故，就是把“编号来源”当成了“改个字段”，先看代价（不贵）就放过去了，目的那一关没人问。

恒信的电子签章里，这两个决定都过了这两关。一个是引入第三方 CA 服务商，那是对外接口；一个是签章编号和台账编号的关系，那是数据模型。两个都在需求那一段就被识别出来，写了决策记录：名称、背景、选项、选择、影响、能不能反悔。

冰链这边，时间戳按谁的时间算，往深里问一层，是温度记录的数据所有权：这批记录以谁的时间为准、谁能动采集算法。石军和林涛也是两关走下来的，结论是把它钉成一条规定需求，让固件、云端两端坐在一起确认。

再说写什么。定下来的东西要写成架构描述：基本概念与属性、管控原则、架构视角与架构视图。这份描述要收得拢，也得放得开：收得拢，是它能回答各方的关切、能被评估；放得开，是它不替设计把话说死。

写完之后还要过两把尺子：架构本身评一遍（它是不是真的回应了那些关切），描述评一遍（写出来的这份东西，别人照着能不能做对）。评估要由不设计它的人来做，这是第 11 章讲过的一条分工。

最后是这一站与下一站的接口。第 14 章讲这件事：架构描述交出去，接住它的是另一份描述，设计描述。两者不是粗细两版，是两类工作：一边是定义出来的解，一边是转换出来的设计。交出去的是一份要遵守的约束，不是另开一条流程。

这一站前后那道缝，靠“两问带进每一站”连上。目的关、代价关不是集中在某一次评审会上过，是散在全程每一段里。需求那一段的业务边界、设计那一段的接口约定、运维那一段的升级策略，都要问这两句。谁在每一段动身前先问目的、再问代价，谁就是在为这条横线缝合。

这一站的账，落在架构描述和决策记录上。它们往下游走的时候，就是约束本身。

### 17.3.6 配置：哪一版是基线

这一站也穿过每一站。（第 15 章）

每一站都会产出东西，产出会改。改来改去，最容易出的问题是：手上这一份，到底是不是大家说好的那一份？

配置管理解决的就是这一问。它不改你的架构，只把“哪一版是定下来的那一份”固定下来。

恒信的电子签章走到上线前两周，CA 服务商通知：验签接口升级，旧接口三个月后停用。搁在以前，这就是群里一句“接口换了，改

一下”。这次这个变更走的是一条完整的路：变更申请、影响评估、批准、实施、建立新的一版。影响评估写的是：改验签接口，涉及 CA 对接那一片，不影响编号映射那一片。

改完之后，罗建说了一句实在话：“关键不是我们多走了几道手续，是每一个变化都能说清'从什么变到什么'。以前部署包叫 final_2 的时候，回滚都找不到上上个版本；现在随便哪一版，都能说清它改了什么、基于哪一版。”

冰链这边，固件 v2.5 的基线是给疾控审计准备的答案。上次审计员问“这批记录是哪版固件采的”，石军查不清，因为升级记录只有一句“已推送 v2.3”。这次 v2.5 这一版底下有批准、有冻结的参照、有逐条记录，写清从 v2.4 到 v2.5 改了哪些。审计员再问，石军能指着文件说：这批记录是 v2.5 采的，补采逻辑是这一条，变更记录在这。

有一点要说清：配置管理这一站，工程这一侧只做一半。把候选挑出来、递过去，是工程这一侧的事；收下、定成基线、维护它，是配置管理这一侧的事。哪些东西算配置项、按什么规矩变更，那是组织的制度（第 15 章）。

配置管理平时做的活，第 15 章列过五项：准备、标识、更改管理、状态记帐、验证与审核。它不说“这是最新版”，它说的是“这是定下来的那一份”，以及“它是怎么变成这一份的”。

这一站前后那道缝，靠“留档”连上。需求那一段的定稿、设计那一段的分配、把关那一段的证据、架构那一段的决定记录，全都汇到这一条线上，变成查得到的参照记录。恒信和冰链上次的失败，本质都是“片与片之间的约定没有被记下来”。

这一站的账，落在配置信息和基线上。

### 17.3.7 治理：谁有资格定，按什么程序

这一站也穿过每一站，管的是那些谁都定不了的事。（第 16 章）

前面几站走到这儿，都还停在“算”这一侧：把需求算清楚、把方案算清楚、把代价算清楚、把哪一版定下来了说清楚。可还有一问没人答：算出来的那些结论，谁点头才算数？

恒信电子签章走到上线前，冒出一个跨部门的问题：签章编号要不要纳入编号标准、由谁归口？法务说该进标准，财务说生成时机得对齐台账，业务说别给我增加步骤。三方各执一词，谁都定不了，因为这件事不在任何一个人的权限里。

恒信那次，委员会裁了一条：签章编号纳入编号标准，由法务归口；台账在签章完成时登记，财务对账口径对齐。一条规则立下去，往后“签章编号进不进台账”不用再开会。顾言说：“这次裁的不是数据，是边界。谁有资格定'编号从哪来'，这是治理问题。”

冰链的治理问题藏在固件升级的边界里。M3 有一条老规矩：固件升级不能毁已出厂的箱子。可这次断链补采要动采集算法。谁能批准动它？不是石军一个人，他有权决定固件怎么改，但没权决定另一件事：改了它，已出厂批次的追溯影响还承受不承受得起。

冰链没有正式委员会，但立了一条规则：凡是动采集算法、影响批次追溯的变更，必须过数据所有权那道裁决关，谁认可谁担责。

第 16 章给这一站留了四句自问，正对着本章的缝：这一版**谁批的、按什么程序、改它要走哪条路**、依据是哪一条。四句答得上，这条缝就有有资格的人管着。

这一站前后那道缝，靠“资格”连上。前面几站的缝合靠手艺：识别、补叶、记录；这一站的缝合靠资格：谁有权为这条缝负责。手艺缝的缝会随人走，资格缝的缝才立得住。

这一站的账，落在治理的裁定与指令上。它们不是技术文件，是“以后这类事按这条办”的依据。

### 17.3.8 运维：上线之后，直到退役

这一站是顺次切片的终点，也是整件事的归处。（第 2 章的外框）

恒信的电子签章上线了。服务器部署了，接口通了，验签能验了。王默盯着刚上的线，说：“服务器、接口、验签都通了，可这只能证明技术这一侧醒了。算不算成功，得看业务那边这一周用起来的样子。”

第 2 章讲过：工程系统包含技术系统，承诺的是整个系统能运转起来。上线第一周，顾言盯着后台看一件事：电子签章的合同，有没有真的从“CA 返回”一路走到“台账登记”、走到“财务对账”。那条跨切片的链路，才是这次上线真正的验收。

他又请三方各答一问：法务的人，你能真用它审合同了吗；财务的人，下个月对账能对上吗；郑亮，你愿意用它，而不是签完章再手工补一遍流程吗。三问都答“能”，这件事才算接上了。

旧流程随后退役：纸质签章不再是主流程。退役不等于销毁，历史合同档案要随时可查，靠的是配置那一站留下的档案。一个系统走到这里不是结束，是开始运营：它从此在时间线上活着，直到下一条需求把它再唤醒。

冰链的 v2.5 通过无线升级推向在役批次。不是所有箱子一起升，M3 有标准版、医疗加强版、工业版，各版本交叉在役。小周管批次，他说：“标准版能升，医疗加强版要过数据所有权那道裁决关，工业版固件锁死，不能硬升。不是我不想升，是边界在那儿。”他翻开批次台账：“每批升不升、什么时候升、升完基线在哪，都记着。”

他把在役升级那份边界摊开，问旁边的小张：“这一版能不能升？”她没答，把封面翻到“签在哪一版”那一页，顺手把笔搁到他手边。他也没再问，在那一行签了字。

按 15288:2023 的分法，这一段其实是三件事：**运行**（把系统用起来、按需要纠偏）、**维护**（换件、修复、保持能力）、**处置**（把它退下来）。日常说的“运维”，通常只指前两件，第三件被漏掉。

再往后还有一站，也最常被整个行业忘掉：退役。系统退役不是失败，是一件事走完了。可退役同样要说清：这一批 M3 为什么退、固件档案归到哪、历史批次还查不查得到。第 15 章那三问（应该是什么、实际是什么、怎么变成这样），在退役这一天问的是一个系统的完整一生。

石军站在仓库里，看着一箱箱 M3 装箱上车，说了一句：“我们改了这么多章的概念，其实是在干同一件事：让一个系统从出生到退役，每一步都说得清、接得上、有人负责。”

这一站前后那道缝，是所有切片的总缝合点。恒信那条“签章→台账→对账”的链、冰链那条“固件→云端→追溯”的链，就是把前几站的接缝一根根串起来验。第 2 章那句“部件全对不等于系统对”，在这一站第一次有了完整的画面：部件就是切片，系统是接缝连上之后的那个整体。

这一站的账，落在运维记录和退役档案上。

---

## 17.4 换一个行业，这套概念还成立吗

### 17.4.1 民机平视显示器：坏了却看着正常

前面走的两个系统，一个是流程，一个是设备，都还在本书自己的两条案例线上。要做最后一道检查，得换到书外去：找两个完全陌生的行业，看这套切法与接缝还在不在。

第一个行业是民用客机驾驶舱里的**平视显示器**。它的作用一句话说得清：把飞行信息投影到飞行员正前方，叠在风挡外的视景上。飞

行员盯着窗外，就能读到速度、高度、姿态，在低能见进近和着陆时也看得见外面。

它的架构里，最要紧的四类视图是：

- **光学成像与眼位**：视场多大、眼位盒覆盖到什么范围、符号和外景对得上吗；
- **接口与时序**：姿态和大气数据从哪来、延迟多少、误差预算怎么分；
- 失效与降级：哪一处坏了会怎样、降级路径事先怎么声明；
- 取证：符合性怎么证明给适航当局看。

看它的失效形态：单点故障如果“看起来正常”，比不显示更危险。屏幕黑掉，飞行员立刻知道出了问题；屏幕上显示的数值错了、符号和外景错位了，飞行员会当成真的。所以它有一条管控原则：单点故障不得误导。这条原则要是没被“失效与降级”那张视图覆盖住，就是一个没人管的安全空洞。

现在把这条链放回七处上看：

- 需求这一处：接的是适航运行最低标准（什么条件下能飞）；
- 设计这一处：把光学、眼位、符号一层层分下去；
- 把关这一处：靠试飞和取证，验证对性能指标、确认对真实飞行；
- 架构这一处：横着管住“共形基准唯一”“失效不得误导”这几条原则；
- 配置这一处：管构型，改一个光学件、换一版软件，都要能追到具体是哪一件、哪一版；
- 治理这一处：搁在适航当局的批准权和构型管理规章上，谁有权改，不由厂家自己说了算。

它自己的接缝也很清楚。“共形基准唯一”这条原则在三个视图里都露面，只有那张记着“符号与外景怎么对应”的清单，才能把它当

成一条约束记下来，并且能核它是不是成立。这就是本章说的那道缝，换到航空行业，样子一模一样，只是代价更硬。

### 17.4.2 企业运营平台：能力固定，用法可变

第二个行业是企业运营体系的信息化平台。常见的做法是一家公司建一套系统；这套平台不是。能力固定在引擎这一侧，用法放在配置那一侧，两边用接口约定接上。

它的架构里最要紧的几类视图是：

- **引擎清单**：哪些能力做进引擎；
- **两类产品架构**：平台那一侧是软件组件，业务那一侧是结构化的配置定义；
- **配置与呈现**：配置怎么组装成某家企业看到的样子；
- 需求分流与上升：一条新要求该进引擎、该进配置，还是该用工具解决。

它的失效形态，正好出在本章的一个词上：“配置”这个词，在两个世界里指的不是一件事。平台这一侧说的“配置”，是把能力组装成某家企业的使用方式；工程这一侧说的“配置管理”，是“哪一版是定下来的那一份”。两边混起来，就会出现一种很典型的结果：各个业务域各配各的，语义各立各的，越配越散，最后配出一堆互不相认的孤岛。

所以它立了一条管控原则：能力缺口要上升成引擎需求，不许用配置绕过去。绕过去一时省事，代价是这条能力永远没有主人。

把这条链放回七处上看：

- 需求这一处：先做分流；
- 设计这一处：划清两类产品架构的边界和接口契约；
- 把关这一处：对的是“配置与引擎匹配不匹配、配置能不能执行、业务是不是真的贯通”；
- 架构这一处：横着管住分界与原则；

- 配置这一处：管配置项的版本与独立性；
- 治理这一处：管的是资产优先的次序和谁来批准分界的改动。

它的接缝也很典型。“配置里的一条链该落到哪个显示区域”，这类信息不属于任何一张视图：硬塞进某一视图里，就会变成同一件事在两处各画一遍、两处对不上。它得作为一条原则单独记下来，写上一个负责的人。这正是本章说的“不属于任何一片，就得有人负责”。

把两条案例线与书外这两例放在一起看，同一批概念在两个行业里各长什么样：

| 概念 | 恒信（流程） | 冰链（设备） | 书外两例（航空／平台） |
| --- | --- | --- | --- |
| 规定需求 | 签章编号必须进台账 | 补采时间戳按设备时间 | 运行最低标准获批；能力缺口上升为引擎需求 |
| 设计 | 一棵树，两片叶子之间补一片 | 三棵树靠接口对接 | 光学与眼位；两类产品架构 |
| 确认 | 法务在真实流程里审一份合同 | 真车在隧道里断链 30 秒 | 试飞取证；配置能不能真跑起来 |
| 基线 | 合同版本下面被批准冻结的参照 | 固件 v2.5 那一版 | 构型件与软件版本；配置项版本 |
| 数据所有权 | 编号标准归法务归口 | 温度记录的时间标准由数据链的持有方定 | 构型管理规章；两类产品架构的分界 |
| 治理 | 编号来源谁有权裁决 | 动采集算法谁有权批准 | 适航当局的批准权；资产优先的次序 |

同一个概念，换一个世界长一副样子，位置却对得上。这就是“概念不挑行业”最实在的证据。

### 17.4.3 认知反转

回到章首。恒信的编号对不上，冰链的时间戳对不上。现在能看清，这两场事故连着错了三层，一层比一层深。

第一层，把“每一段做对”当成“系统做对了”。需求、设计、评审、架构、配置、治理，每一样都做了，每一样都像样，可系统死在段与段之间。需求做得对，解决不了“编号从哪来”没有主人；测试做得对，发现不了“签章到台账”没人验；配置做得对，记录不了“时间戳按谁的时间算”这条口头约定。

第二层，把概念当成彼此不相干的东西。需求管需求，架构管架构，配置管配置，每一样都做到位。可这些词讲的是同一件事：你换个角度看它，就换一套词。不相干的东西之间，各干各的就行；同一件事的不同讲法之间，必须有人去连。把词学懂了，却忘了它们说的是同一个系统，问题就出在这里。

第三层，把缝合当成最后多出来的一道工序。以为这是全书结尾的“总复习”，等所有事情做完，再回头把概念串一遍。

可缝合不是最后一步：需求那一段把没说出口的话写成条款，是缝合；设计那一段把两片之间那条接口补成一片叶子，是缝合；把关那一段把跨段的场景补成一个用例，是缝合；配置那一段把口头约定记进基线，是缝合；治理那一段把“谁有权定”裁清楚，也是缝合。没有哪一段天生带着接缝，接缝都是每一段上有人连出来的。

三层错完，最后一句可以说出口了：前面十六章教的不是十六样东西，是同一件事的十六种讲法。你学到的那些词，每一个都对；它们的用处，是让你在一条真实的链上指得出位置、看得见缝、找得到人。

用第 1 章那把尺子量一下：本章站在**第 ④ 级，解释框架**。前面的概念章把词一个一个说准（① 表征），把结构画出来（② 图式），把机

制推得出来（③ 心智模型）。到了本章，这些合起来回答一个新的问题：这条链上，什么值得相信、凭什么相信、谁为它负责。

---

## 案例进展①｜恒信：把编号来源缝回去

恒信没有重做电子签章。顾言把“编号来源”那条线追出来，穿过六处，一处一处补回去，列成一张清单：

- **需求那一段**：补一条规定需求——“签章编号必须经台账登记，纳入法务归口的编号标准”。原先这是走廊里的一句假设，现在它是验证够得着的一条。
- **设计那一段**：把“签章编号映射”补成一片叶子，挂在“CA 对接”和“台账登记”之间，管“CA 返回的编号怎么变成台账的编号”。原先它只活在会议纪要里，现在它有编号、有描述、有指标。
- **把关那一段**：开发组的苏婷补出“签章→台账→对账”的集成用例，标得格外醒目，旁边写着：“CA 返回的编号落进台账，这一行以前不在任何用例里。按第 8 章那三把尺子，它以前一条也照不到：验证没有对它的用例，确认也没把它放进真实流程。现在它是验收清单上的硬要求。”
- 架构那一段：把“编号来源变更”按数据模型层面的决定重做一遍，按六问补一份决策记录。
- 配置那一段：把“编号来源变更”写进基线的变更记录。原先“编号从哪来”这个事实不在任何记录里。
- 治理那一段：委员会裁一次——编号来源由法务归口，台账在签章完成时登记，财务对账口径对齐。

王默接的是把关和配置两处。集成场景补出来之后，他盯着“签章→台账→对账”这条链重跑了一遍；变更记录写进基线之后，他逐项核对。

他说了一句：“以前这条链在监控里是隐形的。部署看得见、服务看得见，看不见'编号从哪来'。现在把关那一站的集成用例补上了、配置那一站的变更记录写进基线了，这条链才在纸上有据、在监控上也看得见。第 15 章讲的留档就是这个意思：判断落成产品数据，人走了也追溯得到。”

两周后，郑亮又拿来两张表：台账的编号，和签章系统里的编号，对上了。

顾言看着清单说：“六处，一处没少。可你我都清楚：这六处只要缝上任何一处，都不会炸；缝六处，是因为六处都漏了。这件事说明的不是六个问题，是同一件事被漏了六次——每一段都把自己那段活干完了，谁也没抬头看一眼两段之间那条接缝；这条缝该按哪一条判据查、该由哪一层原则管，当时没人答得上来。”

## 案例进展②｜冰链：把时间戳口径钉成接口

冰链也没有推倒重来。林涛把“时间戳按谁的时间算”追到三处，逐一补上：

- **需求那一段**：需求规格里补一条——“补采记录的时间戳按设备本地时间，原样存储，展示时由平台统一换算”。原先固件和云端各按各的理解做这句话，现在它是验证的参照。
- **设计那一段**：在固件和云端两棵树之间，补一片“时间戳对齐”的接口叶子，固件打设备时间，云端原样存、统一换算。原先它只是会议上的一句口头约定，现在它进了接口。

- **把关那一段**：再跑一趟隧道，固件、云端、App 联起来，验“补采时间戳和原始记录时间戳对齐”。

三处补完，系统工程师小张把它们写成清单三行。

她指着最后一行：“三行都有人签字，换个人来，照这张单子也接得住。清单要是只写前两栏、第三栏空着，这条缝还是没主：写不出责任人的那一行，就是下次出事的地方。”

疾控来复查，审计员问的还是那句：“这批补采的记录，时间戳是哪来的？”

石军指着接口规格，一字一句：“设备本地时间，原样存储，展示统一换算。这条写进了需求规格，接口描述上也写着；它是 v2.5 那一版定的，谁批的、改了什么，变更记录都在这。这条口径按需求定，不按习惯定。”

吴浩点开追溯平台：“补采记录和原始记录按同一个时间戳，展示层才换算。需求里写的是'按设备本地时间，原样存储'，我们验的就是这一条——不是功能跑得通就算过。”

小周翻开批次台账，逐批核对，指给审计员看：“补采的记录落在 3 号批次；升了 v2.5 的箱子和没升的分开列，哪批升级、哪批没升，都对得上。每条升级都挂在它那一版基线上，这条判据是配置那一站定下来的，架构那一站也认过这一版。”

他顿了顿：“固件版本跟着箱子走：石军那儿是接口规格，吴浩那儿是追溯平台，我这儿是批次台账——这条缝，三头各自留了档，同一批数据三处对得上，才算连上了。”

审计员点点头，过了。出门时石军说：“上次我们俩说，两个都没问题，中间那条缝有问题。这次我们把中间那条缝变成了一站：需求里写下它，设计里给它一片叶子，验收时验它一条用例——有人签、有人记、有人验。”

## 小结

本章把前面十六章合起来看了一遍。

同一个系统，换一个角度看就换一套词。顺着时间切出来的四段，一站接一站：需求、设计、把关、运维；横着时间切出来的三样，穿过每一段：架构、配置、治理。它们不是彼此不相干的东西，是同一件事的不同讲法。

每一片单独看都合格，片与片之间那道缝还是可能断。这是本章最要紧的一句。四段各自做对，不等于系统做对；问题常常出在段与段之间——那里没有主人，没有记录，也没有尺子照得到。

缝有四条，每条都要有一个人负责。承接：上一段交出来的，下一段能不能直接接住；参照：几段对同一件事的说法，对不对得上；记录：人走了还查不查得到；问责：这条缝有没有名字、有没有主人。四条都答得上，缝才连好了。

缝合不是最后一步，是每一步。本章没有教新概念，它教的是那件最容易被十六个词淹没的事：切开了，就有人把它连回来。

到这里，一条变更从出生到退役走完了一整圈。书还没走完：还有一个“物种”没有回答——**架构师**。下一章回答这本书的书名。

如果只带走一句话，带走这一句：

> 每一片都合格，缝还是会断——所以一路贯通不是最后多出的一道工序，是每一站都在做的事。

运维台前，王默把那张缝合清单钉在了墙上。清单上一行一行写着：哪两处之间、靠哪一种连上、谁负责。以前他盯的是“系统活着没活着”；现在他抬头就能看见——这条链上哪一处有人，哪一处还没人。

# 附录

## A 本章概念总图

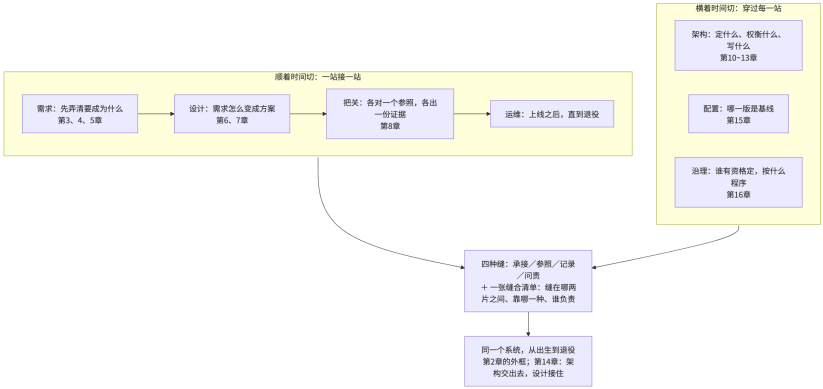



图 17-1

> **读这张图注意**：上下两块不是两套流程。上面那条是一次变更走的四段路，下面那三样是在每一段上都要做的事。三样穿过去，四段接起来，中间那四处连接要靠四条缝和一张清单落实。

## B 本章的能力级

本章以“全书”为单位清点。前面各章各交一段，本章把它们合成一张地图：

① 表征：全书术语的总表——十六个概念章讲过的词，各是什么、不是什么、在哪一段起作用（回指书末附录 A）。

② 图式：本书的骨架——一条时间线加四段路，三样横着切的东西，四处连接；再加第 1 章的能力四级与第 3~16 章立下的那些判据。

③ 心智模型：前面各章那些判据，在这里合起来推得动：每一片都合格不等于系统合格；横切的那几样不能当顺次的一站做；不属于任何一片就等于没人负责；技术上线不等于工程成功。

④ **解释框架（本章主级）**：对“什么值得相信”这件事，本章给四句——概念要织成网，不是收藏切片；要看接缝，不是只看单片；要落成产品数据，不是停在嘴上；要长在人身上，不是停在书上。

再用第 1 章 §1.4.4 那把尺子量一下：**本章以第 ④ 级为主**。前面十六章各交一段，本章把它们放回同一张图——图立起来之后，那些判据才推得动。

**能答全四问**，才算拥有——一条变更摆在面前，你说得出：它要走过哪几段、哪几处缝可能断、每条缝靠哪一种连上、由谁负责。

本章的能力落在哪里？第 9 章那张能力清单，列了六项。第 15 章长的是六项里的第 ⑥ 项：可复现意识；第 16 章长的是第 ④ 项：职业伦理。本章不新增能力项，它给的是这几项能力的**合用地基**：能在一张图上指出位置，你才知道自己守的是哪一段、动的是哪条缝。

## C 自查 · 这些说法对吗？

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “需求、设计、评审、架构、配置、治理都做了，系统肯定没问题” | 把每片做对当成系统做对 | 每片合格不等于系统合格，问题常出在片与片之间 | 这条变更的六处接缝，能指出几处？ |
| “把所有概念汇总一遍，就是这一章” | 把缝合当成总复习 | 本章不汇总概念，只讲接缝：哪两片之间、靠哪条、谁负责 | 你能说出一条缝的名字吗 |

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “架构是设计之后的一个阶段” | 把横切当成顺次的一站 | 架构穿过每一段：需求、设计、把关、运维上都有它的活 | 这条变更在需求那一段，问过目的和代价吗 |
| “配置管理就是发布前打个版本号” | 把版本当受控 | 配置管的是“哪一版是定下来的那一份”，全程都算 | 这一版是谁批的、基于哪一版 |
| “治理是项目结束后的评审” | 把治理当成一个环节 | 治理管的是“谁有权定”，从头到尾都在 | 这条规则的改动，谁有权批 |
| “技术上线了，这个项目就成了” | 把技术上线当工程成功 | 工程系统包含技术系统，要看用的人接没接上 | 法务、财务、业务三方各答一问了吗 |
| “接缝记在心里就行，不用写下来” | 靠默契连缝 | 缝靠记录连上，不靠默契 | 换个人来，这条缝还查得到吗 |
| “不属于任何一片的东西，不归我管” | 把没归属当成没责任 | 落在切片之间的依赖，必须有一个责任人 | 这一处有没有名字 |
| “两个行业差别太大，这套东西不适用” | 把概念当行业行话 | 概念不挑行业：换一个世界，位置和缝都在 | 拿书外的例子走一遍，四段和三样还在吗 |
| “平视显示器坏了大不了不显示” | 不知道失效形态的差别 | 安全关键系统里，“看着正常”的失效比不显示更危险 | 这类失效有没有被专门一张视图盯住 |

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “平台的配置和配置管理是一回事” | 同一个词两个意思 | 平台那侧的“配置”是用法组装，工程这侧的“配置管理”是哪一份算数 | 说的是用法的组装，还是版本的定案 |
| “缝合清单是给检查用的” | 把清单当台账 | 清单是给干活的人看的：先知道缝在哪儿，才干得对 | 这张清单上一次被用来分活，是什么时候 |

## D 练习

1. 画一条你们最近一次真实变更的完整时间线，标出四段路，在每两段之间写上那道缝在哪儿；圈出最弱的一处。
2. 把这本书第 3~16 章的分类写下来（哪几章管顺次、哪几章管横切、哪一章讲人、哪一章讲外框）；再用同样的方法，把你们组织的岗位和流程分一遍：横着切的那几条线，各自是谁在贯穿？有没有一条线没人贯穿？
3. 找一次“每一段都做了、系统还是失败”的经历，指出这条缝断在哪两段之间、断在承接、参照、记录、问责里的哪一条。
4. 用“应该是什么／实际是什么／怎么变成这样”三问，复盘你们系统里的一个模块：三个答案对得上吗？哪一问答不上，哪里就在漏。
5. 挑你们最近一次变更里最难改的那个决定，写清两关：它守的目的是什么（目的关）？它有多难改、给了什么待遇（代价关）？
6. 选一条真实需求，按“完整、无歧义、不冲突、隐含需求已识别”四条过一遍：哪一条最弱？有没有像“编号进台账”那样藏在场景里、没被写出来的东西？

7. 把你们系统的一个模块拆成“框架定义＋详细定义”，查两件事：约束有没有传下来？有没有两片之间缺一片“接口叶”？
8. 用三把尺子复盘一次真实上线：评审对目标、验证对规定需求、确认对预期用途，各自站住了吗？“确认”是不是真在场景里做的？
9. 找一条你们组织里“没归属”的依赖，写一行：缝在哪两处之间、靠承接、参照、记录、问责里的哪一条、责任人是谁。写不出责任人的那一行，标出来。
10. 拿一个书外的行业（航空、医疗、金融都行），把本章的四段路和三样横切的东西套上去走一遍：哪一处最像，哪一处最不一样。

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

**标准**

- [1] **R-04** ISO/IEC/IEEE 42010:2022《架构描述》
- [2] **R-05** ISO/IEC/IEEE 42020:2019《软件、系统与企业 — 架构过程》（§5.4 核心过程〔概念化／评估／细化〕、§5.3 架构治理与管理过程、§6 架构治理过程、§7 架构管理过程）
- [3] **R-28** ISO/IEC/IEEE 42030:2019《架构评估》（引言：实施架构评估的原因）
- [4] **R-37** ISO/IEC/IEEE 15288:2023《系统与软件工程 — 系统生存周期过程》（§6.3.5 配置管理过程；§6.4 各技术过程：§6.4.3 系统需求定义、§6.4.4 系统架构定义、§6.4.5 设计定义、§6.4.12 运行、§6.4.13 维护、§6.4.14 处置）

· [5] **R-38** ISO 9000:2026《质量管理 — 基础和术语》

# 第 18 章 架构师：还是一份能力清单

## 18.1 引言

前面十七章，一章讲一件事。每一件都讲清了：它是什么、凭什么这么说、缺了会怎样。可有一件事，十七章都没有正面回答——**这些事合起来，是一个什么样的人**。

这件事难在：答案不在哪一章里。翻遍任何单独一章，都找不到这个人；只有把十七章摆到一起，才看得见他。上一章末尾留了一句：书还没走完，还有一个“物种”没有回答——架构师。本章回答它。本章不新增概念。它只做一件事：把前面各章交下来的东西，合成一个人。

### 18.1.1 误解现场

恒信集团，年度评优会。主持人念颁奖词：“年度架构师——罗建。全年产出架构图最多，架构文档最全。”掌声。罗建上台，接过那块牌子。

散会以后，需求分析师顾言在走廊上把他叫住：“当选架构师，意味着你现在是什么？”罗建张了张嘴。他试着替自己答了三次。

第一次，他想说“我是画架构图的”——可那面墙上挂着的四张图，图下面没有一条决定。第二次，他想说“我管架构”——可真要动架构，得先提理由、再进基线、最后还得有人批；他手里那点权，只够把话说清楚。第三次，他想说“我技术最好”——那是工程师的尺子，量不出架构师。

三条都走到头，没有一条通到他身上。他答不上来。办公室里，三个人替他答了。

开发说：“架构师就是更高级的工程师——升上去了嘛。”测试部的同事说：“架构师是画架构图的人，图最多的那个。”运维说：“架构师是什么都管的人，出了事都找他。”

三句话，三个“架构师”。两年前，两间会议室为“架构”吵得不可开交；两年后，一间办公室为“架构师”各说各话。

三句都错，错法还一样：**第一句错在把职级当成能力，第二句错在把图当成架构，第三句错在把权力当成责任**。三句话说的都是他坐在哪儿、手里拿着什么，没有一句说的是他做得了什么。

### 18.1.2 本章要回答什么

读完本章，你应该能回答：

1. 架构师这份能力清单由哪几项构成？每一项凭什么说有？
2. 前面各章各交下了什么？为什么说它们是同一个人的不同侧面？
3. 为什么“先是一个工程师”？那几项缺一项，会怎么样？
4. 配置管理和架构治理，为什么也算他的能力？
5. 他每天在做的，是哪几件事？
6. 他和工程师，差别到底在哪儿？
7. 为什么说“架构师”不是标准的术语？标准认的是什么？
8. 他不是哪些角色？他和系统工程师，各背哪本账？
9. 评估和认证，各管什么？
10. 他会怎么失败？
11. 一个人怎么长成架构师？“被任命”为什么不算数？
12. 能力最终体现在哪儿？为什么说落在产品数据上？
13. “换个人，能不能接住”——凭什么用这一句量他？

### 18.1.3 本章的骨架

本章的主线是一句话：**前面各章各交下一样东西，本章把它们合成一个人**。围绕它，本章先合出这份清单，再划开他跟别人的界，最后看他怎么长成、拿什么量。

**架构师会什么，每天做什么（18.2）**：先把前面各章交下的东西摆成一张表——每一章交下的是什么、落在人身上是哪一项；再逐项点名：先是工程师的那几项，再是他比工程师多出来的那几项；然后说清配置管理和架构治理为什么也在这份清单上。最后三节，看他一整天在做的活、看这种活跟工程师的活差在哪儿，再说清标准为什么只认活动、不认头衔。

**架构师的边界在哪儿（18.3）**：把他跟六个最容易混的角色分开；分清他和系统工程师各背哪本账；说清也不是目的的定义者；把评估和认证这两个常被混进来的动作摆到该在的位置；最后说他会怎么失败。

**架构师怎么长成，拿什么量（18.4）**：从工程师到架构师，变的是什么；被任命为什么不算数；来路不同、功课也不同；再往上走，管的是什么。最后落到一处——**能力终究体现为产品数据**：换个人，拿着他签过字、写下来、守得住的那批产品数据，接不接得住。

三块合起来回答一件事：前面各章讲的，是一个人的不同侧面。

> **术语注**：“架构活动”（architecting）用的是 ISO/IEC/IEEE 42010:2022 §3.1 的词条；本书资料库里的中英对照版把 architecting 定名为“架构工作”，本书用“架构活动”（两个译名待统一）。“架构师”这个词，标准没有立条——标准认的是活动，不是职位。本书说的“架构师”，是把这个活动背在身上的人；这一句是全章的名分，18.2.7 展开。

## 18.2 架构师会什么，每天做什么

### 18.2.1 每一条能力，都能追到一章

前面十七章，各交下一样东西。把它们摆到一起，就是这一章要给的那份清单。

| 前面各章 | 它交下什么 | 落在人身上是哪一项 |
| --- | --- | --- |
| 第 1、2 章 | 把词说准的方法；工程是什么、靠什么成立 | 他量自己的方式；科学素养与工程判断的来处 |
| 第 3 章到第 8 章 | 一条路：需求 → 设计开发 → 产品数据 → 评审、验证与确认 | 这条路，他每一段都得懂 |
| 第 9 章 | 工程师的能力清单 | 他的前一段 |
| 第 10 章到第 14 章 | 架构是什么、怎么定、怎么取舍、怎么写下来、怎么交给设计 | 他比工程师多出来的那几项 |
| 第 15 章、第 16 章 | 配置管理、架构治理 | 其中两项，在他这一层的落地 |
| 第 17 章 | 一条链上的接缝与缝合 | 他知道每一处缝上有谁 |

这张表里没有新东西。它只是把散在十七章里的东西摆到一处，让人看见：这不是十七章拼起来的一摞，是同一个人的六个面。

摆完就能看出两件事。一件是：清单里每一项都有来处，没有一项是本章新造的。另一件是：**能追到来处的，才是能力；追不到来处的，多半是习惯、是印象，或者干脆是头衔的影子**。

所以本章给的第一条判据，是冲着清单自己来的：

> 判断一份能力清单立不立得住，只看一条：它的每一条能不能指回某一章。

指得回去，这一条就有出处、有判据、有人检验过；指不回去的，要么是本章凭空加的，要么是从别处听来的，两种都不该留在清单上。后面几节逐项点名，每点一项，都按这一条走一遍。

**先把一个词说清**：本章说“能力”，一律指第 9 章那份清单的意思——先是一个工程师的那几项，再是他比工程师多出来的那几项；不与第 1 章的“能力四级”（表征、图式、心智模型、解释框架，§1.4.4）混，也不指某一段里临时说的“这项能力”。同一个词，量的是两件不同的事：第 1 章量的是概念掌握到哪一级，这里量的是他手里长出了什么。

### 18.2.2 工程师的能力，一样都不能少

先把清单的前一段立起来。这一段不是本章的发明：第 9 章已经逐项论证过，这里只点名，并且给每一项配一句当场能自问的话。

| 能力 | 它管什么 | 当场自问 |
| --- | --- | --- |
| 科学素养 | 判断这个系统靠什么成功 | 我对这个系统的理论是什么？做错什么最致命？ |
| 方法论素养 | 方法不靠人传人 | 我这一套，别人学得会吗？ |
| 实践能力 | 落得了地 | 我最近一次亲手落地，是什么时候？ |
| 职业伦理 | 知道什么不能牺牲 | 压力面前，我有没有为进度悄悄牺牲可靠性？ |
| 工程判断 | 冲突里能取舍 | 我依据什么做这个取舍？ |
| 可复现意识 | 判断换个人也接得住 | 我写架构决策记录（ADR）吗？ |

六项里，前四项来自工程的那几层，第 2 章逐层立过——科学素养、方法论素养、实践能力、职业伦理；后两项横着穿，工程判断管取舍，可复现意识管传承。

六项一条都不能少，理由不在“多一项总比少一项好”，在**缺哪一项，后面多出来的那几项就都落不了地**。

- 缺科学素养：判断不了系统靠什么成功，理论是空的。
- 缺方法论素养：方法只在他一个人脑子里，人一走就带走了。
- 缺实践能力：判断悬在半空，落不到任何能被造出来的东西上。
- 缺职业伦理：判断越准，破坏越大——能力不是中性的。
- 缺工程判断：取舍没有依据，只能看谁的声音大。
- 缺可复现意识：他做出来的东西换个人接不住，等于每次都从零开始。

> 判据：这六项里任何一项答不上来，先别提后面那几项。架构师先是工程师——这句不是资历上的客气话，是清单上的次序。

### 18.2.3 比工程师多出来的能力

工程师那六项，管的是“把部件做对”。架构师在那六项之上多出来的，是四件事。

第一件，给每个候选写一句目的陈述。摆出三个候选，就要写三句“它为了什么”。正本（标准的正式版本）说得更直白：每个解决方案可能有不同的目的陈述（42020:2019 § 8.4.5 e) 及其注 5）。

第二件，把手段立成概念与原则。选出来的做法不能停在“我们打算这么做”，要立成实体的一条概念或属性，或者一条管住它实现与演化的原则（42020:2019 § 8.4.7 b)、e)、h)）。立不成，它就只是一次口头约定。

第三件，适配与调和。既有做法不能照搬。按正本，**要把参考架构实例化并加以特化**（42020:2019 附录 E.3.3.2，资料性）；本书的说法是“特化到眼前这个实体”。几套做法撞在一起，还要调和到“适用于当前情形”（42020:2019 §8.4.6 注 3）。

第四件，把“现状”也摆上桌。现状有时就是备选之一（42020:2019 §9.4.7 注 2）。不写下来，它就默认被选中，而且没人知道它是被选中的。

这四件事有一句共同的判据：

> 做完之后，有没有多出一份别人能用的东西。

写不出这份东西，四件事就都停在脑子里，谈不上能力。

边界要说清。多出来的这四件，管的都是“这个实体是什么”那一层：基本概念或属性，加上管住它实现与演化的管控原则。元素、接口、关系不在这里。把系统需求分配到系统元素、把架构实体与关系转化为设计元素，是设计定义过程的任务（15288:2023 §6.4.5.3 b) 1) 与 b) 2)）；架构侧也有分配，架构师分到的是架构实体（15288:2023 §6.4.4.2 c)）。这不是说架构师不碰元素，是说元素不是他这一角色的专责对象。

还有两项要挪出去。“识别后果严重的决策”和“判断不可逆性”，不在这份清单上。影响面广、难逆转、约束硬，这一族在本书里是代理：它有它的正当用途（排序、定决策时机、配治理强度、做回代检查，第 10 章列过四样），但架构内容的准入只有一条，是“对目的必要”。把它写成能力项，等于让代理坐上准入位——第 10 章和第 12 章都为这件事划过界。

把这几处收成一句角色定义，本章后面都用它：

> **架构师对架构（基本概念或属性与管控原则）的成立与守持负专责**；他不背系统成败的终责，那是系统工程师的账（18.3.2 展开）。本章下文说“架构的成立与守持”，都是这一句的简称。

### 18.2.4 配置和治理，也算架构能力

第 9 章交代过一句：可复现意识会往配置与基线长，职业伦理会往治理长。到本章，这句话要落成清单上的两格。

配置管理那一格。工程师版是“管好自己那几份”；架构师版是“让整套架构描述与决策记录受控，并且让下游必须依据它”。两档之别是尺度，不是两件事——一个人的判断能不能被接住，放大到一整套描述上，就是配置管理。

治理那一格。治理不替你算，它决定你算出来的那份判断算不算数；在这条线上，架构师是出技术判断的那一头，不是签发的那一头。

两格的机制与效力，第 15 章和第 16 章各讲了一整章，这里不重讲。本章只确认一件事：**它们在这份清单上有位置**，不是身外的行政事务，也不是只有组织才需要的东西。嵌入式硬件工程师石军那页老规矩能被别人拿起来改、拿起来验，就是其中配置那一格在他身上的样子。

> 判据：两格合成两句自问。你不在场，它接得住吗？——这一句问配置。要改它，走哪条路、谁批？——这一句问治理。两句都答得上，这两格才算长在他身上。

### 18.2.5 每天在做的活：从构思到改进

清单回答的是“这个物种长什么样”。还有一面：他每天在做什么。ISO/IEC/IEEE 42010:2022 §3.1 给这件事立了词条（architecting,

本书作架构活动）：在所关注实体的整个生存周期内，构思、定义、表达、记录、沟通、认证恰当实现、维护与改进架构。

> **译名注**：这七件事里，两版译名不同的有三处：本书资料库里的中英对照版作“构想／编制文档／证实其正确实施”，本书作“构思／记录／认证恰当实现”；义同，七件事的数量与次序两版一致。

七件事，一件一件点：

| 动词 | 干什么 |
| --- | --- |
| 构思 | 从关切出发，生成候选 |
| 定义 | 画边界、做选择，收敛到一个 |
| 表达 | 把它翻成一组各有其主的视图 |
| 记录 | 把视图与决策固化成架构描述 |
| 沟通 | 让各利益相关方理解，达成一致 |
| 认证恰当实现 | 查实现符不符合架构 |
| 维护与改进 | 对着腐化，把它守住 |

两点要注意。第一，**这七件事贯穿整个生存周期反复发生，不是串行的七步**。今天在构思，下周在定义，明年还在维护——它们不是一条流水线，是一组反复回来的动作。

第二，它和前面那张清单是两套切法：清单是体检切法，看这个人长什么样；这七件事是工序切法，看他每天在做什么。这两套切法咬在同一处：构思那一件的起点，是各利益相关方的**关切**（标准中译作“关注点”）；沟通那一件对着的，也是他们。两套并行，谁也不吞掉谁。

> 判据：当场能问的一句——我现在做的，是这七件里的哪一件？答不出来的，多半正在做第八件；第八件通常不叫架构活动。

### 18.2.6 架构师和工程师，不一样在哪儿

前面几节说的是他手里有什么；把它们和工程师那六项摆在一起，差的到底是什么，是另一件事。

| | 工程师 | 架构师 |
|---|---|---|
| 面对的东西 | 规定需求：明示的、可验证的 | 关切：可以模糊、可以冲突、没有标准格式 |
| 把关方式 | 测试，对着规定需求 | 评审，对着目的；审的是写下来的理由 |
| 判据锚点 | 规定需求 | 目的 |
| 时间尺度 | 这一版交付得对不对 | 这套东西明年、三年后还立不立得住 |
| 手里最要紧的件 | 契约与证据 | 决策与理由 |
| 出的产品数据 | 那棵树上管叶子的那一段：详细定义 | 那棵树上管干枝的那一段：框架定义与架构描述 |

六行差别的根子是一句话：工程师面对的是规定需求，架构师面对的是**利益相关方**的**关切**——一个是明示、可验的条款，一个是可以模糊、可以冲突、没有标准格式的兴趣点。**这不是做得更多，是做得不一样**。

第 14 章那句“是种类之别，不是粒度之别”，在这里同样作数。同一件事里，工程师竖着往深处做，架构师横着把面铺开。一个现成的例，罗建那三句答不上来，卡的就是这一行：他手里只有图，没有理由。

> 判据：看他交出来的件。交的是“做出来了，证据在这”，他在工程师那一侧；交的是“为什么这么做，理由在这”，他在架构师这一侧。罗建那四张图，图下面没有一条决定——交出去了图，没交出去理由。

### 18.2.7 标准只认架构活动，不认头衔

有一件事必须先说清，否则整份清单都悬着。

ISO/IEC/IEEE 42010:2022 第 3 章一共 19 个词条，从 §3.1 排到 §3.19。**里面没有一条词条叫** architect。以 architect 开头的那八条（§3.1 到 §3.8）讲的全是架构与架构描述，不是人——§3.1 architecting 指的是活动。architect 在正本里只作过一次角色名，出现在 §3.17 利益相关方词条的举例里：**有角色名，没有词条**。

这不是标准疏忽，是它的立场：**认活，不认职位**。所以本书说的“架构师”，意思是把这个活动背在身上的人；正式名是“**系统架构师**”，本章下文凡说“架构师”，除特别点出模块那一层外，都指这个角色。这一句是本书的主张，不是标准条文——顺带说清一件事：全书唯一给“架构师”这一角色下定义的是 INCOSE《系统工程手册》。

> 判据：拿一个人来问——构思、定义、表达、认证这些活，他做过哪几件、各留下什么件——不是问他名片上印的什么。

## 案例进展①｜罗建：有头衔，能力没补齐

评优会散了，话题没散。

开发组的苏婷站在那面墙前面。四张图还挂着。“图我们天天看，”她说，“可真做接口的时候，碰上'数据放哪、归谁管'，图上找不着答案。”

运维王默翻出一份变更记录：“上回发版，我照例翻变更。'改了什么'有，'为什么改'没有——就一句'按上级要求'。第 15 章说配置管理管的是'哪一版算数'，可我连它为什么改都不知道；拿这份记录去接别人的班，接不住。”

业务方的郑亮也接了话。他说得慢：“我们业务不看图画得漂不漂亮，只看合同在审批链上别卡壳。'改个编号'这四个字，当年是我随口一提的，我以为是件小事；后来才知道，那是每份合同的唯一身份，台账、审计、签章，整个生存周期都靠它认人。业务规则的每一行，都得有人想清楚它进系统哪块地。”

罗建站在旁边听。三句话他都听懂了，因为每一句都对着他清单上的某一格。

当天下午，他把那四张图从墙上取下来，换成一份决定清单。清单第一栏写“这个实体是什么”，第二栏写管住它实现与演化的那几条原则。写到“这份架构靠什么成功”那一栏时，他停了一会儿，把这一栏空着，先往下写。这一栏他一时填不上。搁以前，他会先填一句像样的话。

他做的第二件事，是给自己写了一份体检：照那份清单，一项一项打勾。

科学素养那一栏，他犹豫了，最后写“存疑”：这两年不碰实现，理论有点悬空。方法论素养、实践能力，有。职业伦理，他想起两年前那次“为进度悄悄牺牲可靠性”的记录，账还没还清，但这次他把代价先摆到了桌面上。工程判断，正在长。可复现意识这一栏，他写的是“以前没有，从这次起写决策记录”。

体检单上没打满勾。罗建有头衔，缺的那几项，就是这次没打上勾的。

但写这张单子本身，是他往前走的第一步：他终于拿清单量自己了，而不是拿名片量自己。

## 18.3 架构师的边界在哪儿

### 18.3.1 不是更高级的工程师

回到评优会那句“架构师就是更高级的工程师”。

错在把职级当成物种。职级是组织给的一个位置：管多少人、拿多少钱、在哪个序列上。物种是能力定的：会做什么、做得了什么。第 9 章立过“头衔不定义工程师”；本章要在这一句上面再加一层：**职级也不定义架构师**。

那么两者的活按什么分？本书一直用一条判据：看谁拥有对该层规定需求的变更权。同一层里，谁有权改那一层的规定需求，那一层就归谁。这是种类之别，不是粒度之别；第 14 章整章讲的就是这件事，这里不重讲。

**这一条要标清楚**：这条划分是本书的读法。标准不替组织分配谁做工程师、谁做架构师，它只管过程；哪一层的变更权在谁手上，是组织自己定的，再拿这条判据去核。

组织可以把任何人任命为架构师，就像可以把任何人印上工程师头衔。但“是不是在做架构活动”，不由任命决定。

> 判据：拿一个决定来问一句——这一层的谁有权改它？权在谁手上，这个决定就归谁那一层。头衔不参与回答。

### 18.3.2 不是系统工程师：专责和终责

这是全书最容易混的一对，因为两个名字都带“系统”，都像统管的角色。其实两本账分得很清：

- 系统工程师对系统成败负终责。整个系统的账，出了事他担着；这本账不可委托。
- 架构师对架构（基本概念或属性与管控原则）的成立与守持负专责。基本概念或属性、管控原则立不立得住、守没守住，这本账既可以委托出去，做砸了也要追到他头上。

一句话：**架构师搞砸了，追责追到架构师；对外向客户、向监管担责的，是系统工程师**。两者的关系不是指挥与执行，是一个回路：委托、协商、验收。

委托：系统工程师把“架构的成立与守持”委托给架构师，架构师以架构描述回馈，系统工程师评审背书。

协商：架构能改变需求。选了一个架构方案，等于给系统加了一批它本来没有的要求（派生需求），要回流去更新规定需求。所以两者是持续的谈判对手，目标不是“完美的正确解”，而是双方都接受的**协商理解**（negotiated understanding）。这层意思，正本写在 42020:2019 §5.2。协商理解不是谈一次就定死的：需求一回流，它就要跟着更新，下游要依据的正是当下那一版。

验收：拍板权在系统工程师，理由权在架构师。你提方案加理由，我拍板，我担着。

有一处要按正本说准：拍板这件事，通常在架构过程之外（正本见 R-05 §5.7 与 §8.4.7 注 7）；15288:2023 §6.4.4.3 e) 2) 给架构定义这一步收口的，是“就架构定义取得一致意见”这句话——它要的是一个答复，不是一个结论。42010:2022 §6.10.1 另有一项**宜加考虑**：架构描述里记录决策权属方或决策责任人。记下它的那一栏，就是治理落下名字的地方（第 13 章、第 16 章）。

模块那一级还有一层：系统架构师把模块的架构委托给模块架构师，通常就是模块工程师自己。

> 判据：出事以后问一句——这口锅扣在谁头上？为系统成败担着的，是系统工程师；被追到“你负责的那部分立没立住”的，是架构师。两问指向同一个人，要么是兼任，要么是有一本账没记清。

### 18.3.3 不是技术负责人，也不是项目经理

技术负责人管的是人：代码评审、任务分配、进度、绩效、团队成长。架构师管的是内容：这个实体在概念与属性上是什么、哪些边界与演化规矩必须守住。

两者可以是一个人兼，但问的问题不同：技术负责人问“这个迭代能不能按时交付”，架构师问“这套东西明天改需求还立不立得住”。

这里有一条硬差别：**管理权可以任命，领导力不能任命**。技术负责人靠职位；架构师多半没有职权可依，他要在没有正式职权、或者职权不足的情况下，靠专业可信度让人自愿地跟着走。可信度积得慢、耗得快，一次判断让人受益，才算攒下一点。罗建被任命那天，拿到的是职位给的那点权；真要动架构，还得先提理由、再进基线、最后有人批。

项目经理管的是日历：里程碑、资源、风险登记、沟通。架构师管的是演化。项目经理说“这个需求排到下个迭代”；架构师说“这个需求会动到那条原则，得先过一遍”。两者冲突时，常常是进度对结构——第 2 章讲的价值层失守，现场多半就在这一刻。

> 判据：看他在压力下护住哪一样。护住交付，他在技术负责人或项目经理那一侧；护住这套内容立不立得住，他在架构师这一侧。

### 18.3.4 不是领域专家，也不是画图匠

领域专家懂业务：审批规则、法规要求、结算流程。架构师懂系统：这些规则怎么被建模、这些流程的数据怎么流动、系统怎么响应变化。

领域专家说“法务要求合同必须有三个审批节点”；架构师说“这三个节点在系统里怎么表示、数据归谁、改起来要动哪里”。好的架构师不抢领域专家的活——他让领域专家的规则在系统里待得住、改得动。

画图匠这一对最容易混，因为两者都画图。差别只有一条：图是决定的表达，还是决定的替代。第 10 章立过：架构是那份内容，不是承载它的那份东西。挪到人身上就是——

> **把图全删掉，决定还在，你是架构师；图删掉就什么都不剩，你是画图匠。**

石军在第 11 章写下的那页老规矩不是图，是一份清单；罗建那四张图曾经挂满一面墙，图下面没有一条决定。

> 判据：删一次图试试。剩下的东西够不够别人接着干，就是答案。

### 18.3.5 不是目的的定义者

角色边界往上走，最后到一条最上游的界：**目的不由架构师定义。**

系统目的与任务的定义权和问责在业务方（赞助人、产品方）。架构师的职责是让架构忠于目的，不是替目的做主。用 RACI 的说法（A＝最终问责，R＝主责，C＝被咨询）：目的定义，A 在业务方，架构师是 C；架构定义，A 和 R 在架构师，业务方是 C。

不定义，但绝不被动。他对目的有四件事要做：

| 职责 | 做什么 |
| --- | --- |
| 理解 | 把目的当作判据吃透——它是架构活动的输入总开关 |
| 澄清 | 目的模糊时向上游回溯，推动业务方表达到可判定的粒度 |
| 质疑 | 用可行性、成本、一致性去检验目的——有质询权，没有改写权 |
| 守护 | 全程盯目的漂移；目的变了，就把架构重审一遍 |

这四件事里，“理解”有一份自己的产品数据：《系统目的或任务理解确认单》。这份单子分三部分：一是目的原文照引，业务方的原话只引不改；二是架构师的复述理解；三是业务方签字。它是纯理解、不含一点架构内容：复述里冒出架构语言，就是越界了。这份单子是架构活动的入场券：签字之前，后面的一切产出都没有判据锚点。

恒信那边，这份单子第一次签字时，产品开发经理曹峰想省一步：“目的还用写？我们心里都清楚。”顾言没让他省：“心里清楚不算数。您把它说到我能判真假，我才好开工。”曹峰把那张单子签了，签完说了一句：“这么一问，我这边倒先把目的想清楚了一遍。”

还要分清一件事：角色分离不等于人员分离。小团队里，一个人可以同时戴产品方和架构师两顶帽子；但做每个决定时，得知道此刻戴的是哪一顶。两顶帽子在脑子里互相将就，是很多架构失控的起点。

目的不是孤零零一个点。它沿一条链往下展开：目的（为什么存在）→ 目标（达成什么可度量的结果）→ 需求（必须满足什么可验证的条款）。评审和确认，最后都要沿这条链追到目的上去。

> 判据：拿一份目的陈述来问——这句话是谁说的？说得出来源和签字人，它在链上；说不出，它只是房间里的一句共识。

### 18.3.6 评估和认证，各管什么

第 8 章把评审、验证、确认三个词钉住，末了划出去两个动作：评估和认证，说好留到这里讲。现在补上。

| 动作 | 管什么 | 依据 |
| --- | --- | --- |
| 评审 | 这版材料准备好了吗——人的判断活动 | ISO 9000:2026 §3.11.2；第 8 章 |
| 验证 | 做出来的东西符不符合规定需求——要客观证据 | 15288:2023 §6.4.9；第 8 章 |
| 确认 | 是不是用户真正要的东西——在真实用途里管不管用 | 15288:2023 §6.4.11；第 8 章 |
| 评估 | 架构本身值不值、够不够——对架构所作的分析与价值评定 | 42020:2019 §9 |
| 认证恰当实现 | 实现符不符合架构 | 42010:2022 §3.1；42020:2019 §3.2 注 2 |

五个动作，各有一个对象：评审对着材料，验证对着规定需求，确认对着预期用途，评估对着架构本身，认证对着造出来的东西。

评估和认证这两个，最容易混进评审桌上。它们跟评审的区别就在对象：评审桌上放的是那版方案；评估问的是这套架构值不值；认证问的是实现有没有按这套架构做。

认证还有一层意思，不出那第五个动作之外。正本把这句话说得很直白，**证实架构的正确实施，有时被表述为架构师向客户或用户作出的正式声明，即所构建的系统满足可投入使用准则**（42020:2019 §3.2 注 2）。也就是说，符合性把关不是内部自娱，它的一头对着交付。

还有一处要标清楚：架构师在技术评审里有没有固定席位，是组织分配的，不是标准赐予的。技术评审的参与者名单里并没有单列“系

统架构师”，他通常落在“其他技术负责人”这一类（这一条是本书据 INCOSE《系统工程手册》那份名单所作的说明，属本书主张，不是标准条文）。谁坐进评审，由组织定。

另有一个动作要说明归属：审计（audit）查的是“系统对运行信息部件与配置信息部件的符合性”（15288:2023 §6.3.5 注 22），在本书里归配置审核（第 15 章）与治理的符合性评定（第 16 章）。本章只点名，不展开。

> 判据：手上这件事，问一句“对着的是什么”——对着材料、对着需求、对着用途、对着架构、还是对着造出来的东西？对象一清，动作就定。

### 18.3.7 架构师会怎么失败

划清边界之后，还有一件要看的事：一个人是怎么从位置上滑下去的。这些失败都不戏剧性，是慢慢掉下去的。

画图匠：图越来越多，决定越来越少。症状是图越画越漂亮、越画越全，可图下面没有一条决定。掉下去的方式不是被免职，是团队发现图不能用来做决定——图上说系统长这样，可真碰上“数据放哪”，没人答得上。于是大家绕开图，各做各的。他缺的是“把手段立成概念与原则”那一件。罗建那四张图正走在这一条上。

什么都管：把该别人定的也定了。每个接口、每行代码、每张表都要过他。看起来非常负责，实际是把专责当成了终责：既背了不该背的账，又把团队判断的能力收走了。精力全花在低价值的决定上，真正重要的那一批反而没人管。

不决策：把决定留给默认。症状不是不管，是不选。不选就是选了默认，而默认往往是最保守、最省事、最像“大家都这么做”的那

一条。架构决策躲不掉，只是早晚；不做决定，系统照样会被某个选项塑形，只是你在事后才认识它。他缺的是工程判断。

远离代码：判断失去锚点。升上去以后不再看实现，只开评审会、只画蓝图。他说“这里该微服务化”，可他不知道现在的服务边界在哪；说“数据该统一”，可不知道数据现在在谁手里。理论还在，但已经不落在任何能被造出来的东西上。他缺的是实践能力。

四种姿态：画图匠死于图代替了决定，什么都管死于把专责当终责，不决策死于不选，远离代码死于看不见实现。

> 判据：四种里对号入座——最近一次，你在哪一格里？每一种都能在这份清单上找到对应的那一格；找到那一格，功课就定了。

## 案例进展②｜石军：没头衔，换个人接不住

磨完边，再量另一个人。石军的名片上印着“嵌入式硬件工程师”。他就没换过名片。

产品经理林涛先在评审会上说了话：“会是我主持的。可一到'数据归谁、接口怎么切'，话筒就得递给石军——他答得比需求清单还准。”

测试季昭说的是另一头：“测试最怕一件事：为什么这样测、凭什么说'测过了'，我常说不清。石军那页老规矩，写明白的地方我能验；没写明白的，我验不了。判断这东西，光在他脑子里不算数——换个人读得懂、照着做得下去，它才是一份能被人验证的东西。”

制造侧的小周说得更实：“他每动一次数据口径，产线的批次追溯就得重对一遍。第 11 章那条'断链就是失效'，我拿着能用；可它凭什么这么定，纸上没写——口径一改，我这边就得把三个月的批次重对一遍。”

系统工程师小张把手上那张清单合上，接了一句：“两本账得分清：系统能不能用，这个账在我这儿；他那页老规矩立没立住，是他的账。合成一本，出了事就没人说得清该问谁。我这边可以替他兜，账不能替他记混。”

石军自己没接话。散会以后，他在楼梯口站了一会儿，才把那页纸从口袋里拿出来看了一眼。

按那份清单量，他缺两格：方法论素养，可复现意识。他是手艺的极致——判断准，落地稳；可那一套在他手里长成了技艺，没有长成别人能接着用的东西。他没有头衔，缺的也不是头衔。

## 18.4 架构师怎么长成，拿什么量

### 18.4.1 从工程师到架构师，变的是什么

绝大多数架构师不是空降的，是从工程师长出来的。这一跳变的不是职级，是管的事换了。石军就是现成的一例：名片上印着模块岗位，做的事却落在系统那一层。

工程师管“把东西做出来”：这段代码怎么写才清楚，这个模块怎么切才不绕，这一版交付得对不对。架构师管“它是什么、凭什么立得住”：哪些内容是基本概念或属性，哪些规矩必须守住，往后的版本还认不认得出这是同一个东西。

判断的依据也跟着换，从“这段代码有没有毛病”，换到第 10 章那三个问题：这个系统靠什么成功？最大压力在哪？做错什么最致命？你关心的问题换了，位置就换了。

两个方向同时扩。一个是面：从部件到整机，从一棵子树到整棵树。一个是时间：从这一版交付，到跨版本演化——今天做一个选择，是在给后年留活路，还是把后年堵死。

有一件事要说清：**这不是升职，是分叉**。管理线（技术经理、项目总监）和架构线（系统架构师、首席架构师）是两条并排的路，走哪条都正当。把分叉当成阶梯，才会问出“升不上去才走架构线”这种话。

想换到这条线上，有几个成本不高的入口：先以评审人的身份坐进架构评审，反复问“为什么选 A 不选 B”，直到能替架构师把理由讲圆；动手写决策记录，从记录别人的理由开始；在系统架构师划好的边界里，独立主责一个子系统的架构，把“构思、定义、表达”三件事各走一遍。

> 判据：拿一个架构师来问一句——“如果需求翻一倍，这套东西还成立吗?”答得上来，他在架构这一侧；只会说“现在这样挺好的”，他还在工程师那一侧。

### 18.4.2 被任命不等于成为

评优会上，罗建被任命为“年度架构师”。任命给了他一个名分，名分不给他能力。这听起来不太客气，其实公平：名分是时间的快照，能力是长出来的东西。任命可以明天收回，能力收不走。

所以看一个人，不看任命书写了什么，看那份清单上亮了几格。罗建被点醒以后做的那两件事——把四张图换成一份清单、拿清单给自己打勾——都不是“补画图技术”，是开始补判断。**成为架构师的过程，就是清单一项一项亮起来的过程**。

> 判据：把任命书摆在桌上问一句——它让清单上哪一项变了？一项也没变，它就只是一个名分。

### 18.4.3 来路不同，功课也不同

长成架构师的人，来路不一样。从开发长出来的、空降的、挂名的，各自缺的东西不同，要补的也不同。

从开发长出来（最常见）。实践能力与科学素养通常不缺，工程师那几项已经有一半在身上。缺的是系统级的判断——哪些内容算基本、哪些决定值得按架构决策来对待。功课是把小团队里的代码判断，练成对整棵树的判断。

空降（带着方法进来）。理论、判断、别的行业的经验，往往都有；缺的是对这个系统的实践，以及可复现——他的判断还没被这个系统检验过，别人也接不住。功课是先落地：把一套判断在这个系统上跑通一遍，再谈蓝图。

挂名（组织设了岗位，人没换）。只有头衔，没有清单。功课只有两条路：要么真的开始补，要么承认这只是一个名头——挂着名的架构师，是组织在用名字骗自己。三条路没有高低，只有功课不同。

> 判据：看他最近三个月在补清单上的哪一项。一项也没补，那他不是走在哪条路上，是站在原地。

### 18.4.4 说“不”：拦下的不是错，是没想清楚

一个人真正长成，往往不是学会画一张更大的图，而是学会对一件“做了也没人反对”的事说不。

这种“不”最难的，是它拦下的通常不是错的东西。云端软件的吴浩想给客户开个直连接口，功能上完全做得到，客户还会满意；石军把它拦下来，理由不是“不行”，是“绕开了追溯域，这条路以后退不回来”（第 12 章）。

罗建想把合同编号字段当“改个字段”改掉，改起来当天就能上。这一次，“不”没来得及在上线前说出来；三个月后连环三爆（第 12 章）。顾言事后点破他，理由同样是“这件事改错了以后，代价不在这一版里”。

石军那次“不”，**拦下的不是错，是没想清楚**。也正因为这样，说这个“不”的人常常不占理——没有一条规定需求可以引，没有一次测试可以证明，只有一句“再等等”。可罗建那次没人说，代价就落在三个月后的三爆里。所以第一次说“不”，说的其实不是别人，是自己：忍住“先上了再说”的冲动。

> 判据：回想你最近一次说“不”。拦下的是错，还是没想清楚？只拦得下前者，说明这把尺子还没长出来。

### 18.4.5 再往上走，管的是组织

架构师是这本书的终点，但不是这个物种的终点。再往上走，不是更深的架构，是关注的东西换了一级：工程师的“用户”是部件，架构师的“用户”是系统，再往上是系统之系统，最后是整个组织。每一次，服务对象换一级，物种也跟着换一次。

系统之系统有一个显眼的特征，这一条是本书的读法：每个子系统都有自己的负责人和生存周期，没有全局负责人。架构师缝的原来是部件之间的缝，再往上缝的是子系统之间、部门之间的缝。

再往上的路，常见的有四条，没有高低，只有管的面大小不同：

- 深度走首席架构师，从“做架构”到“建架构体系”。
- 宽度走企业架构师，从“一个系统”到“系统之系统”；企业架构那几套域划分是行业参考（TOGAF），不是标准条文。
- 组织走技术总监一类的岗位，管的是技术投资与组织能力。
- 权威走首席工程师一类，管的是跨学科的技术裁决。

组织那一层上还有一条分工值得记住：**方向权（目标、政策、战略）在组织层，内容权（原则、架构决策、符合性判断）在架构那一侧**。这一条是本书的读法——标准只说治理在组织的较高层级、向治理机构问责，并没有这一句。

有一件事在这里同样成立：头衔也不定义他。一个人做到技术总监，如果还在用工程师的尺子量自己，他只是换了个更大的名分；反过来，一个人在“系统”这一格上待一辈子，只要那份清单一项项拿得住，他就是这本书说的架构师。

> 判据：看他服务的对象变没变一级。判断、权衡、沟通、担责这几样没换，换的是它们作用的对象有多大。

### 18.4.6 能力最终落在产品数据上

清单上那些东西长在人身上，看不见，也没法直接检验。面试官会看走眼，领导会凭印象打分。所以本领最终要落到一处能被翻、能被查、能被接手的地方：他交出来的产品数据。

第 9 章给工程师立过一句：能力＝开发出高质量的产品数据。这句话在本章同样作数，只是对象换到了架构这一层。罗建把那四张图换成一份决定清单，换的正是“交出去的产品数据”。六项各落在哪件上，一项一件：

| 能力 | 落到哪件上 |
| --- | --- |
| 科学素养 | 架构的依据：为什么这么分（容量、性能、约束条件） |
| 方法论素养 | 架构的方法：视角与视图怎么定、怎么评 |
| 实践能力 | 架构的落地：它造得出来，不是纸上楼阁 |
| 职业伦理 | 决策的真实：取舍如实记，账不藏 |
| 工程判断 | 架构的取舍：系统级的权衡与理由 |

| 能力 | 落到哪件上 |
| --- | --- |
| 可复现意识 | 架构的可接住：决策记录写得清，演化路径接得下 |

多出来的那几项，也各有各的件：给候选写的目的陈述，落在架构目标与关键成功准则、问题空间定义报告上；把手段立成的概念与原则，落在基本概念或属性清单、管控原则上；适配与调和，落在架构权衡报告上；把“现状”摆上桌，落在权衡报告的候选行里。

这些件不是一张纸，是一族，以架构描述为核心：基本概念或属性清单、架构描述、管控原则、架构决策记录、接口控制文档、框架定义、派生需求清单、架构权衡报告、架构基线、演化路线图、系统级验证方案。

按标准明文的清单对一遍，还差几件要补进来：

- 架构评估计划、架构评估报告与分析结果、价值评定结果（42020:2019 §9.5，现有清单里只有“架构权衡报告”）。
- 质量模型、问题空间定义报告（42020:2019 §8.5）。
- 架构目标与关键成功准则（15288:2023 §6.4.4.3 b) 2)）。
- 未处理项与不符合项声明（含理由）（42010:2022 §6.7 d) 及注 2）。
- 符合性正式声明（42020:2019 §3.2 注 2）。
- 基线候选物（15288:2023 §6.4.4.3 e) 6)）。
- 评审问题的处置记录（42020:2019 §9.4.10 h)）。

还有一件职责最容易漏：架构符合性把关。架构活动这个定义里本来就带着“认证恰当实现”；正本还补了一句（42020:2019 §3.2 注 2，18.3.6 已引全）：这份符合性正式声明的一头对着客户或用户，也就是对着交付。也就是说，架构师不只写架构，还要查别人做出来的东西有没有按这套架构造；这份查的义务，责任矩阵里不一定有一格写着它。

那么，什么叫高质量？不是图多、不是文档厚，是这五样：**本质立得住、原则守得住、接口可承接、决策如实、换个人接得住**。

冰链那边，产品开发经理赵刚在评审结论上签字，落笔前总问一句：“这一条谁验的？”问清了才签，签完在背面记一行“为什么”。他说不清的，就不签。那行“为什么”，换个人翻开就接得住。

> 判据：把他的件摊开问一遍——换个人，拿着这些件，接得下去吗？接得下去，能力就在；接不下去，能力还在他脑子里，不在产品数据里。

### 18.4.7 架构师在生存周期里的位置

前面第 17 章把一条链走了一遍：从需求到设计，到把关，到上线，到退役。那一章讲的是这条链本身。这一节补的是另一半：在这条链的每一站上，架构师具体做什么。

按 15288:2023 的技术过程排过去，他在每一站的角色可以归成四个记号：

- 接收输入：这一站交给他、他必须接住的东西。是责任上的接收，不是时间上的先后。
- 开发输出：他编制、并对正确性负责的东西。缺了它，这一站跑不动。
- 出意见：他判断别人的东西——评审、符合性判断——但不签放行。
- 无：标准与组织都没有要求。

“开发输出”和“出意见”必须分成两件事：架构描述与“符合性检查意见”不是一类东西，前者他能定，后者只能提。混成一件，等于说他对自己提的意见也有定夺权，那第 8 章那句“评审通过 ≠ 决策作出”就被抹掉了。

| 过程 | 架构师做什么 |
| --- | --- |
| 业务或任务分析 | 接收输入：优选解类；开发输出：问题空间的架构视图 |
| 利益相关方需要与需求定义 | 接收输入：需要与需求；开发输出：关切清单、利益相关方与角度台账 |
| 系统需求定义 | 接收输入：派生需求回流；开发输出：架构侧对需求的反约束；出意见：需求对架构的充分性 |
| 架构定义 | 开发输出（主责） |
| 设计定义 | 接收输入：分配与对齐的整合或确认；开发输出：架构实体与关系、架构特性、接口定义；出意见：符合性 |
| 系统分析 | 接收输入：分析结果；开发输出：分析命题与判据 |
| 实现 | 接收输入：实现偏差；开发输出：架构约束；出意见：符合性 |
| 集成 | 开发输出：接口定义、架构对集成顺序的约束；出意见：接口问题 |
| 验证 | 接收输入：异常与验证证据；开发输出：验证方法与判据；出意见：方案覆盖性 |
| 转换 | 接收输入：转换约束 |
| 确认 | 接收输入：确认结果；出意见：确认参与 |

| 过程 | 架构师做什么 |
| --- | --- |
| 运行与维护 | 接收输入：运行与维护反馈；开发输出：架构对运维的约束与演化原则 |
| 处置 | 接收输入：处置约束；开发输出：处置的架构侧约定 |

**这张表要说清**：这张表与“接收输入／开发输出／出意见／无”这四个记号是本书的工作清单，不是标准条文——它们是把 15288:2023 各过程的目的与结果，按“架构师接什么、交什么、评什么”重新读了一遍。

表里“运行与维护”合成了一行：标准中运行（15288:2023 §6.4.12）与维护（§6.4.13）是两个过程，故 14 个技术过程在本表作 13 行。

> 判据：拿到一个过程名，答三句——他接什么输入、交什么输出、给什么意见？三句都答不上，说明这一站上没有人。

### 18.4.8 换个人，能不能接住

全章的清单、边界、成长，最后都收到一句话上。这句话和第 9 章量工程师的那句是对称的。第 9 章那句是“换个人、换团队，用同样的需求和约束，能不能交付质量相当的成果”；在这里，对象换成了架构：

> **换个人，拿着他签过字的那些件，接得下去吗？**

接得下去，架构就在；接不下去，再大的图也只是装饰。

这一问还顺手把前面几节的账一起算了。配置管理那一格问“你不在场，它接得住吗”；可复现意识那一项问的是同一件事；架构符合

性把关问“别人做出来的东西，还认得出这套架构吗”；专责与终责问“接的人接的是哪一本账”。拿这一问量罗建：那四张图接不住，因为图下面的决定他没写下来；后来换上的那份决定清单，才是能交出去的东西。

> 判据：这一问不必等到出事才问。换一个人，把手上的产品数据交给他一周，看他要问你几次、问的是不是那些产品数据里本该写着的东西。

### 18.4.9 认知反转

回到评优会。

两年前，两间会议室为“架构”两个字吵得不可开交；两年后，一间办公室为“架构师”三个字各说各话。我们终于看清这两场是同一个病：

**人们把一个本该由能力定义的东西，交给了名分去定义。**

名分回答不了“你现在是什么”。能回答的，是那份清单：先是一个工程师（那几项一项都不能少），再在上面长出把架构立住的那几项；配置和治理也是他的能力，只是长在他这一层；他背的是专责，不是终责；而这一切，最后都落在同一处——他签过字、写下来、守得住的那批产品数据。

最后用第 1 章那把尺子量一遍。对“架构师”这个词，章首的罗建停在“见过”那一级——叫得出头衔、画得出图，却答不上那句“你现在是什么”；到这里，判词把他量到了“拥有”那一级——他能拿清单打勾，认得出自己缺什么。

散会那天，小周和石军一起走出会议室。小周说：“石军，你做的这些，是架构师的活。”

石军没接话。他手里那页纸写着三条规矩，边角已经磨毛了——那页纸从第 11 章写下来，到第 13 章补成规格，到今天还在用。改的人照着它改，验的人照着它验。它不是图，也不是头衔，是这个团队能接着用下去的东西。

走廊尽头，小周把手上那页没写完的批次台账推过去。小张翻了一眼，收进自己的文件夹：“这份我归到批次追溯那一档去。”小周点点头，各自散了。

两个世界，各开各的会，到这里对上了同一个答案。顾言从第 1 章那两间会议室问起的那个问题——“这个产品到底是什么”——问的是系统；今天这一问换到人身上，答案是同一个：这个物种不由头衔认，由他交出去的那批产品数据认。

## 小结

本章没有新增任何概念。它做的是另一件事：把前面各章交下来的东西，合成一个人。

合成之后，那份清单是两段加两格。前一段是工程师那几项，一项都不能少；后一段是他比工程师多出来的那几项，管的是“这个实体是什么”那一层——给每个候选写目的陈述、把手段立成概念与原则、适配与调和、把“现状”也摆上桌。两格是配置管理和治理：一项管判断可复现，一项管判断算不算数。

它给他划了界。他不是更高级的工程师（职级不定义物种）；不是系统工程师（他对架构的成立与守持负专责，系统工程师对系统成败负终责，两本账）；不是技术负责人，也不是项目经理；不是领域专家，也不是画图匠（删掉图，决定还在的才算）；他也不是目的的定义者（目的的定义权在业务方，他有理解、澄清、质疑、守护四件事）。

同一条界上，它还把五个动作分开了：评审对材料，验证对需求，确认对用途，评估对架构，认证对实现。

它讲了成长。从工程师到架构师，变的是管的事，不是职级；被任命不等于成为；来路不同，功课也不同；第一次说“不”，拦下的往往不是错，是没想清楚；再往上走，换的也只是服务对象的大小，判断、权衡、沟通、担责那几样没换。

最后，它把这一切从人身上挪到纸面上：**能力终究体现为产品数据**——本质立得住、原则守得住、接口可承接、决策如实、换个人接得住。拿这句话量两个人：罗建有头衔，缺的那几项没打上勾；石军没头衔，缺的是方法论素养与可复现意识。两个人都还不是完整的架构师，两个人都走在成为的路上。

而这一章也把这本书回答完了：书名问“架构师是个怎样的物种”，答案不在头衔里，在他签过字、写下来、守得住的那批产品数据上。

# 附录

## A 本章概念总图

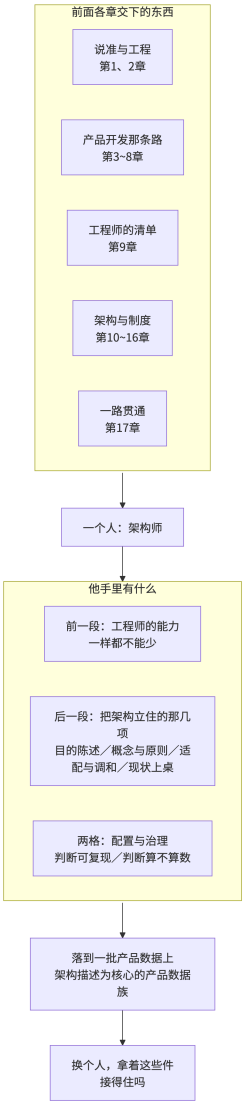

前面各章交下的东西
说准与工程
第1、2章
产品开发那条路
第3~8章
工程师的清单
第9章
架构与制度
第10~16章
一路贯通
第17章
一个人：架构师
他手里有什么
前一段：工程师的能力
一样都不能少
后一段：把架构立住的那几项
目的陈述／概念与原则／适配与调和／现状上桌
两格：配置与治理
判断可复现／判断算不算数
落到一批产品数据上
架构描述为核心的产品数据族
换个人，拿着这些件
接得住吗

图 18-1

> **读这张图注意**：上面那五块不是五摞知识，是同一个人身上的东西。中间那份清单不是本章新造的，它是把上面五块合成之后的样子；下面那一处检验也不是新加的题目，它就是第 9 章量工程师的那句话换了个对象。

## B 本章的能力级

本章以“全书”为单位清点。前面各章各交一段，本章把它们合成一个人：

① 表征：四个角色名各就各位（系统架构师／系统工程师见 18.3.2；模块架构师／模块工程师见 18.3.2 末）；专责与终责；架构活动那七件事；评审、验证、确认、评估、认证五个动作各对什么。

② 图式：一张合图（前面各章交下什么、落在人身上是哪一项）；一份能力清单（前一段＋后一段＋两格）；一张能力与件的对应表；一张角色在生存周期各过程里的参与形态表。

③ 心智模型：前面各章那些判据，在这里合起来推得动——名分不定义人；专责不是终责；能力最终落在产品数据上；“难逆转”那一族只是筛选与排序的代理，坐在准入位上的是“对目的必要”。

④ 解释框架（本章主级）：第 17 章给了四句——概念要织成网，要看接缝，要落成产品数据，要长在人身上。本章兑现最后那一句：那四句不是四个要求，是同一个人的四件事。前面各章立的判据，到这里合起来回答一个新的问题：什么样的人，能把这条链守住。

再用第 1 章 §1.4.4 那把尺子量一下：**本章以第 ④ 级为主**。前面十七章各交一段，本章把它们收回一个人——清单立住之后，那些判据才知道该长在谁身上。

**能答全四问**，才算拥有——拿一个人来，你说得出：这份清单上他亮了几格、他背的是哪本账、他交出去的产品数据换个人接不接得住。

本章的能力落在哪里？第 9 章那张清单列了六项，第 15 章长的是第 ⑥ 项（可复现意识），第 16 章长的是第 ④ 项（职业伦理）。本章不新增能力项：它把前面各章交下的一次合成，给这几项能力一个共同的宿主。

## C 自查 · 这些说法对吗？

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “架构师就是升职后的更高级工程师” | 把职级当物种 | 职级不定义人；分工看那一层的变更权在谁手上 | 拿一个决定问：这一层的谁有权改它 |
| “图画得最多、最全的人，就是架构师” | 把图当架构 | 架构是那份内容，删掉图还在的才算 | 删一次图试试，剩下什么 |
| “架构师应该什么都管” | 把专责当成终责 | 两本账：架构的成立与守持是专责，系统成败是终责 | 出事以后，这口锅扣在谁头上 |
| “图纸都画完了，拍板就是架构师的事” | 把理由权当拍板权 | 拍板权在系统工程师，架构师给理由；拍板通常在架构过程之外 | 这份架构的决策权属方，那一格写了谁 |
| “被任命为架构师，就是架构师了” | 把名分当能力 | 名分是时间的快照，能力是一格一格长出来的 | 这份任命让清单上哪一项变了 |
| “架构师负责把系统的目的定下来” | 越界改写目的 | 目的的定义权在业务方；他有理解、澄清、质疑、守护四件事 | 这份目的陈述，是谁说的、谁签的字 |

| 说法 | 病 | 药 | 判据 |
| --- | --- | --- | --- |
| “评估和认证跟评审是一回事” | 把动作的对象混了 | 评审对材料，验证对需求，确认对用途，评估对架构，认证对实现 | 手上这件事，对着的是什么 |
| “能力是虚的，没法量” | 拿印象当检验 | 能力落到产品数据上，件是可以翻、可以查的 | 换个人，拿着这些件接得住吗 |
| “架构师不该再碰实现” | 判断失去锚点 | 实践能力那一项撑着“判断有没有锚” | 最近一次亲手落地，是什么时候 |
| “做到架构师就到头了” | 把分叉当阶梯 | 管理线与架构线是两条并排的路，再往上换的只是服务对象的大小 | 判断、权衡、沟通、担责，这几样换了吗 |
| “标准里给'架构师'立了词条” | 把本书主张当标准条文 | 42010:2022 只给架构活动立条，不给人立条 | 第 3 章那 19 个词条里，查得到 architect 吗 |
| “配置管理和架构治理是行政事务，跟架构能力无关” | 把清单上的两格看成身外事 | 一项管判断可复现，一项管判断算不算数 | 你不在场，它接得住吗；要改它，走哪条路 |

## D 练习

1. 拿合图量一份简历：把你们团队里最像架构师的那个人的经历，逐条往那张合图上落——哪几块他有证据，哪几块只有头衔。
2. 写一句目的陈述：找一件正在酝酿的事，给两个候选各写一句“它为了什么”。写不出来，说明这件事还没有候选，只有倾向。
3. 把图删一次：把你手上那套架构材料里的图全放到一边，只留文字。剩下的东西够不够别人接着干？

4. 两本账盘一次：在你们最近一个项目里，标出“系统成败的账”和“架构成立与守持的账”各在谁手里。两本账落在同一个人身上，是兼任，还是有一本没人接？
5. 五个动作对号：把你们上个月做过的判定列出来，逐个归到评审、验证、确认、评估、认证里去。归不进去的，多半是“开了个会”。
6. 目的四职责自查：你们系统的目的，最近一次变更谁提的？你当时做的是理解、澄清、质疑，还是直接改了？
7. 失败对号入座：画图匠、什么都管、不决策、远离代码，你更接近哪一种？它对应清单上哪一格？
8. 活数一遍：把你上周做的事，往构思、定义、表达、记录、沟通、认证、维护改进这七件里归。归不进去的，占了多少时间？
9. 换人一周：把手上那批产品数据交给一位同事一周，只回答他提出的问题，不主动补充。一周后看他问的是什么——那些就是产品数据里没写清的。
10. 再往上想一格：如果你现在管的是一个系统，往上一格是系统之系统或组织。写下：判断、权衡、沟通、担责这几样，哪一样要先换？

## E 参考文献

> **编号怎么读**：[k] 是本章内的文献序号；括注的 R-xx 是全书参考文献总表（附录 D）的编号，凭它可回总表查该条的完整说明与条款。

### 标准

· [1] **R-04** ISO/IEC/IEEE 42010:2022《软件、系统与企业 — 架构描述》（§3.1 架构活动及其七件事；§3.7 架构视图；§3.8 架构视角；§6.7 d) 及注 2 未处理项；§6.10.1 记录决策权属方）

- [2] **R-05** ISO/IEC/IEEE 42020:2019《软件、系统与企业 — 架构过程》（§3.2 注 2； §8.4.5 e) 及注 5； §8.4.6 注 3； §8.4.7； §9 架构评估，含 §9.4.10 h)、§9.4.7 注 2、§9.5；附录 G.1 设计权威机构，资料性）
- [3] **R-11** IEEE 1028-2008（评审类型）
- [4] **R-19** TOGAF（企业架构域划分，行业参考）
- [5] **R-28** ISO/IEC/IEEE 42030:2019《软件、系统与企业 — 架构评估框架》（引言：实施架构评估的原因）
- [6] **R-37** ISO/IEC/IEEE 15288:2023《系统与软件工程 — 系统生存周期过程》（§6.4.4 系统架构定义过程，含 §6.4.4.2 c) 与 §6.4.4.3 b) 2)、e) 2)、e) 6)； §6.4.5.3 b) 1) b) 2)； §6.4.9 验证； §6.4.11 确认）
- [7] **R-38** ISO 9000:2026《质量管理 — 基础和术语》（§3.11.2 评审）

**经典文献**

- [8] **R-61** Ralph Johnson（“架构关乎重要的东西”）
- [9] **R-63** INCOSE《系统工程手册》第五版中文版（与 15288:2023 同版同年）
- [10] **R-76** NASA Systems Engineering Handbook（首席工程师角色的权威锚点）
- [11] **R-87** SEBoK（SEBoK 的其余用法）

> **两处曾误列**：协商理解（negotiated understanding）在 **42020:2019 §5.2** 有正本句；ISO 9000:2026 §3.10.6（competence）本章正文无落点，此前提作悬空引用，已删。

# 后记 · 两间会议室，两年后

## 散场后的两间屋

评优会散了。恒信的会议室里，椅子还歪着，投影幕没有收。罗建把那块“年度架构师”的牌子放在桌上，没有带走。

走廊上，顾言问他的那句话还留在墙上：“当选架构师，意味着你现在是什么？”他答了三次——三次都没有落到自己身上。这个问题，两年前在两间会议室里问过一遍，问的是系统；今天问的是人。

两年前，这两间屋为“架构”两个字吵过；今天，一间屋为“架构师”三个字各说各话。

他们用的还是同一个定义：架构是一个实体在其环境中的基本概念或属性，以及管住该实体及其相关生存周期过程怎么实现、怎么演化的管控原则（ISO/IEC/IEEE 42010:2022 §3.2）。两年前，这句话是用来吵的；今天，它是用来交产品数据的。每个角色交的那批产品数据，就是这句话在一个人手上落下的地方。

定义里的两样东西，落到人身上是两件事。基本概念或属性，回答“这个实体是什么”；管控原则，回答“它怎么造、怎么改，谁照着干”。一个人交的产品数据里，这两样都在，他才算把架构立住了；少一样，交出去的只是图，不是架构。

系统有架构，模块也有架构。这句话两年前还只是口号；今天，它在两间屋里都落成了清单。

散场以后，时间倒回两间屋的会场——

## 会场中的两间屋

恒信那间屋，评优会正开着。主持人念到“年度架构师”时，台下有人抬头。

“罗建——全年产出架构图最多，架构文档最全。”掌声响起来。

罗建从第三排站起来，走上台，接过那块牌子。灯光有点晃，他把牌子翻过来看了一眼，又翻回去。台下，顾言坐在靠门的位置，没有鼓掌。

同一天晚上，冰链那间屋的灯还亮着。石军把那页磨毛了边的老规矩放在桌上，旁边是季昭的测试记录、吴浩的接口清单、小周的物料表。没有人提评优会，也没有人提头衔。两间屋的人都到齐了：冰链七个，恒信六个。两年前在那两间屋里说话的十二个人，一个没少；小张是后来进来的，进来就没走。

可坐下之后，他们做的是同一件事——把自己手上那批产品数据摊开，看哪一份立得住。

两间屋的桌上，摆的东西不一样。冰链那边是温控箱的物料表、测试记录、接口清单；恒信这边是合同的审批流、编号规则、验收报告。可每一张纸回答的是同一类问题：这个实体是什么，它凭什么这么造、这么改，换个人接不接得住。

先看冰链。林涛把一张单子推到桌子中间：

“M3 是什么？一台医疗冷链运输的温控箱，不是能制冷的箱子；温度控得准也不够。真正让它成为 M3 的，是'全程可追溯、断链必告警'这条基本属性。这条属性，就是 M3 的架构。立错了，做出来就是台冷柜，不是 M3。”

那张单子上有三样：目的的原话、架构师的复述、业务方的签字。林涛在上面签了字。目的的定义权在业务方，系统架构师负责把目的

吃透、守住“架构忠于目的”；这张单子，就是让目的一直在场的那份产品数据。

这张单子不是走手续。签字之前，后面的一切产出都没有判据锚点；签字之后，目的变了，架构就要重审一遍。

赵刚接过来看了一眼：

“这些产品数据，我每一份都签过字。以前签的是'参数达标'；现在我知道，我签的是'M3 还是不是 M3'。”

石军没有把东西推过来，他把那页老规矩摊平：

“架构不在那张图里。图是画给人看的，跟不上就过时；规矩是大家照着干的，图烂了，规矩还在。冷链一断必须立刻报、固件升级不许搞坏已出厂的箱子，这就是定义里说的管控原则（governing）。规矩也会过时，所以每条后面都补了一句'为什么'，改的人知道往哪改，换个人接得住。”

他不是系统架构师，可模块也有架构。模块架构师通常就是他这样的人：在模块岗位上，干着“这个模块是什么、怎么守”的活。

模块这一层，标准没有单独给模块立一个架构工作产品；可它有独立的判断：这个模块的边界在哪，什么不能改。这些判断不写下来，就只在一个人脑子里。

吴浩指了指接口清单上的一行：

“架构不只看元素，元素数得清；真正断不得的东西，藏在元素之间：温度记录归追溯平台管，这不是数据放在哪儿的问题，是所有权归谁的一条线。第 11 章把这条判断写成了管控原则；第 12 章我想绕过它，被两句问话拦了回来。正因它写死，后面争论都不用再争。”

季昭把测试记录翻开：

“测试最怕一件事：为什么这样测、凭什么说'测过了'，说不清。当年我问，没人答。后来我问了一句更窄的话：'我该验的东西在哪？'问

完，我才明白，答案不在功能清单里，在架构的定义里。所以测之前，我先问'我该验的东西在哪'；测完，我交的是验证与确认报告，还有记录里那一栏基线。”

小张把几个文件袋按顺序摆开，每个袋口贴一条标签：

“我这一份摊不出单张，它是整条链。从需求到退役，每一段都得受控、都追得回去。以前我盯的是'东西做出来没有'；现在我先问'这一段是谁签的字、换个人接不接得住'。第 15 章那句'哪一版算数'，问的其实就是这个：签不上字的段，等于没人管。”

链子最怕的不是哪一段断了，是断了没人知道断在哪一段。

小周把物料表往前挪了挪：

“以前架构描述就一张图，我在里面找'怎么造它'，找不着。后来才懂：利益相关方关心什么，就该有回答那一问的视图。制造关心可制造性，就是系统好不好造；给制造看的架构视图，画的是结构：标准件多不多、接口好不好装、要不要开模。装配、排产，照它来。”

制造这一头，最容易被漏掉。漏了，产线到最后一刻才发现造不出来。

再看恒信。郑亮把合同编号那一条圈出来：

“合同编号规则，应该成为咱们合同系统的架构内容。它是每份合同的唯一身份：台账、审计、签章，整个生存周期都靠它认人；缺了它，系统就不是它了。当初一句'改个编号'，三个部门对了一个月的账。”

曹峰把验收报告合上：

“以前我只看系统上线。现在我知道，架构定义得对不对，看系统的目的是不是被实现。系统的目的，就是用户和客户的目的；实现了，才算立对。”

罗建把那四张图从墙上取下来，换成一叠纸：

“以前我以为，架构师就是把图画好；第 10 章那四张图，我挂了一整面墙，以为结构就在上面了。现在知道：图会过时，结构也会换；真正要紧的，是那些基本概念或属性怎么换、往哪走，系统都还是那个系统。”

他停了一下：

“守住'还是不是它'这一条，归我负责。我交的是一份决定清单，第一栏写这个实体是什么，第二栏写管住它实现与演化的那几条原则；每一行，为什么这么定、改它要走哪条路，另记在架构决策记录里。怎么写清楚、守得住，我还在学。”

顾言接话：

“现在大家开口之前，会自己先问'它是什么'。问清了才发现：两个世界的架构，是同一件事，都在回答'这个产品到底是什么'。M3 答的是'全程可追溯'那几条；合同系统答的是'审批链不断'那几条。我手上这一份，是运行概念和规定需求；需求基线一改，我知道要去找谁。”

苏婷指了指屏幕上的接口：

“以前我只管代码，接口照着写，不问它怎么来的。第 12 章改字段那一回，我才明白：接口不是我们随手定的，它是元素之间怎么对接的约定；架构基线上定下来，开发照它做详细定义。合同编号在约定里锁得最死，改一个字段，先问对接方。它连着两边的结构，不是代码的事。”

王默把那张清单摊开：

“以前我在架构图里找部署和监控，找不着；现在才理解，运维是架构要考虑的利益相关方了。部署在哪、监控什么、怎么回滚，架构描述里都写了它们落在哪儿。第 17 章那张缝合清单，我钉在运维台前，一行一行写着：哪两处之间、靠哪一种连上、谁负责。上线前先问它归谁管。”

两年里，十三个人各长了一截。长在哪儿，不看头衔，看手上那批产品数据。

靠门的位置坐着两位来旁听的同行。他们不是这两间屋的人。

其中一位听到这里，举起手："那……系统架构师、系统工程师、模块架构师、模块工程师，都是工程师，职责到底差在哪？"

另一位接着问："这些责任，最后都得落在纸面上吧？我们想往上走，又从哪一步开始？"

罗建没有口头答。他把桌上的东西往前一推：

"差在两处：管哪一层，干哪种活。架构不止系统有，模块也有：模块既是一个系统元素，也可以是一个被架实体（42010:2022 把被架对象由 system 扩为 entity）。每一层都要先定义'它是什么、必须怎么造怎么变'，再把它做出来。"

"系统架构师负架构的成立与守持的专责，这个工作可以委托出去，责任还是自己的；系统工程师把系统做出来，系统成败的终责他背，不可委托；模块架构师负模块自身架构的成立与守持的专责，通常就是模块工程师自己。"

"往上走，从你手上那一份开始。摊开看。"

会场散了，桌上留下四份产品数据清单。

## 每个角色的清单

每一份只写三到五样；写得下的，才是真交得出去的。每一样后面标了出处：写"标准"的，是标准正文里有的工作产品名；写"本书"的，是这本书的凝练，标准里没有这个名字。

清单上的每一样，都不是新东西。它们在各章里都出现过；这里只是把它们按角色分开，看谁该交什么。清单短，不是省事；是让每个人记得住自己该交什么。

## 系统架构师

这一份最要紧的，是**决策与理由**。系统架构师交的不是图，是图下面那些决定：为什么这么分、为什么这么定、改它要走哪条路。

- **目的与本质**：系统目的或任务理解确认单，系统基本概念和基本属性清单，管控原则。确认单由业务方签字；清单和原则写的是“这个实体是什么”（确认单与清单为本书；管控原则为标准，定义句见 42010:2022 §3.2）。
- **架构描述**：视角、视图、对应关系，交得出去的那一份（标准，42010:2022 §3.3 与 §6）。
- **决定与理由**：架构决策记录，加上架构权衡报告。每一行为什么这么定，改它走哪条路（标准；架构权衡报告为本书。42010:2022 §6.10；15288:2023 §6.4.4）。
- **接口与基线**：接口控制文档，基线候选物，架构基线。两边怎么对接、哪一版算数（标准；接口控制文档为本书，15288:2023 §6.4.4 与 §6.3.5）。
- **评估与符合性**：架构评估报告，符合性正式声明，未处理项与不符合项声明，治理决策与符合性状态报告（标准；评估见 42020:2019 §9.5，治理见 §6.4.4 与 §6.5，符合性见 §3.2 注 2，未处理项见 42010:2022 §6.7）。

细目见第 18 章 §18.4.6 那一族；这里只写每一份最要紧的几样。

他不交的，是“更完整的图”。图是表达，不是内容；删掉图，决定还在的，才是架构师。

这几样里，最容易被漏掉的是“决定与理由”。图画得再全，图下面没有理由，换个人就得从头再想一遍。

## 系统工程师

这一份最要紧的，是**整条链受控、可追溯**。系统工程师交的不是一份，是整条链：从需求到退役，每一段都受控、都可追溯。

- **需求与运行概念**：运行概念，规定需求，操作目标清单，需求追溯矩阵（标准；操作目标清单为本书）。
- **计划**：系统工程计划，验证与确认计划，配置管理计划。先计划，后执行（标准，15288:2023 §6.3）。
- **集成与验证确认**：集成方案，集成测试报告，验证与确认报告，测试记录（含基线栏），缝合清单（标准；测试记录与缝合清单为本书，15288:2023 §6.4.8、§6.4.9、§6.4.11）。
- **基线与变更**：技术基线集，变更申请与评审记录，评审记录。哪一版算数、为什么改（标准，15288:2023 §6.3.5）。
- **部署、运维与退役**：部署报告，运维报告，退役计划（本书；过程见 15288:2023 §6.4.10、§6.4.12 至 §6.4.14）。

他不交的，是“架构本身”。架构的成立与守持，他委托给系统架构师；他交的是整条链的受控与可追溯。

这几样里，最容易被低估的是“计划”与“基线”。没有它们，前面那些报告都只是一次性的。

## 模块架构师

这一份最要紧的，是**模块自身的成立与守持**。标准不认这个头衔，只认这件事；模块架构师通常就是模块工程师自己；模块大的，就是那个模块的团队。这是本书立的名。

- **模块框架定义**：这个模块是什么，对外边界在哪（本书）。
- **模块级基本概念或属性**：拿掉它，这个模块就不是它了（本书）。
- **模块级管控原则**：模块内部怎么造、怎么改，谁照着干（本书）。
- **模块内接口**：与系统接口契约怎么对上，改一处要通知谁（本书）。

他不交的，是“模块实现”。实现是模块工程师的活；他交的是这个模块的“是什么”和“怎么守”。

模块这一层的架构，常常没有人写；不写，接口就只能靠两边猜。

## 模块工程师

这一份最要紧的，是**契约与证据**。按约定实现，并证明它成了。

- **模块详细定义**：含仿真计算，做到什么程度算完（本书）。
- **模块实现方案与实现报告**：按约定做出来，做出来的是什么（本书）。
- **模块测试与验证记录**：凭什么说它成了（本书）。
- **模块版本与基线记录**：哪一版算数，改过什么（标准，15288:2023 §6.3.5）。
- **模块接口记录**：与模块内接口怎么对上，改一处记在哪（本书）。

他不交的，是“架构决定”。接口怎么定，先问对接方；他只管按约定做出来，并拿出证据。

这几样里，最要紧的是“版本与基线记录”。做出来了，却说不清哪一版算数，等于没做。

四份清单不是四份孤立的文件，是一条链上的几处交接点；链上还有别的缝，见第 17 章。系统工程师把架构的成立与守持委托给系统架构师，系统架构师把模块的架构委托给模块架构师；委托、协商、验收，一圈走完。

拍板权在系统工程师，理由权在系统架构师。目的的定义权在业务方，确认单上签的是业务方的字。接口契约连两边，缝合清单钉责任人。

四份清单合起来，就是产品数据在一条链上的全部去处。每一处去处，都是一个交接点；每一处断了，链上就有一处没有人。这四份清

单里，没有一份写着“全部”。每一份只写自己那一层；四份合起来，才是全部。

清单不用背。你要找的那份产品数据，按角色翻到对应的那一份，看它在不在。

## 明天你要做的事

清单摊完，屋里静了一会儿。到这里，三面合到一处：概念看准了，产品数据写下来了，人长出来了。这三面不是三样，是一件事的三面。

两间屋，两个世界，用的是同一把尺子：换个人，拿着这些产品数据，接不接得住。一个造有形的箱子，一个做无形的系统；有形的靠物料表，无形的靠那份决定清单。

冰链那边，换个人接过那页老规矩，冷链一断还是立刻报，固件升级还是不许搞坏已出厂的箱子，M3 还是 M3。恒信那边，换个人接过那份决定清单，合同编号还是那一条，审批链还是不断，合同系统还是它。两个世界的答案不一样，判据是同一个。

这把尺子不挑行业，也不挑头衔。它只问一件事：你交出去的那批产品数据，换个人接不接得住。

你手上也许没有“架构师”这个头衔；不要紧。这四份清单里，总有一份是你的，或者有一份是你明天要接的。

明天你要画的那张图、要写的那份清单、要做的那条决定，先问三句：

**归属**——这是谁的事？

**内容**——它对应架构定义里的哪一样内容，是基本概念或属性，还是管控原则？

**载体**——它由哪一件产品数据记下来？

三句都答得出，这件事才不是“大家都以为有人管”。

这三句不是新规矩。第 1 章教过四问，第 9 章量过工程师，第 18 章量过架构师；到这里，问的对象从词换成了产品数据。

问清了，再落笔。

这本书讲到这里，能交给你的，就是这把尺子。尺子不是用来供奉的，是用来较量的。

书里那些概念，讲到这里就算讲完了。剩下的不是再读一遍，是拿一件真东西去量。

合上书，回你的工位，挑一件明天就要交的东西，先问那三句。

# 附录：把尺子收进五个抽屉

> 正文十八章，发的是概念、判据和案例。本附录把它们归档成五个抽屉，供你随时回来取用：**附录 A 术语表**——全书概念的英中对照（术语“宪法”，正文用词的唯一权威）**附录 B 概念档案集**——33 张最常被查、最容易误解的九格档案完整版（字典式查询）**附录 C 误解清单**——全部“反直觉命题”速查（18 章病/药/判据三栏）**附录 D 参考文献总表**——正文引用的标准与经典文献 **附录 E 练习答案**——各章练习与自查的答案要点（与训练教材题卡同源）
>
> **用法**：遇到说不清的词，翻 A 或 B；“这个误解错在哪”，翻 C；“这条原理出自哪个标准”，翻 D；做完练习想对答案，翻 E。第 1 章把四问交到你手里，把“查全”这一步交给附录（§1.7.5）——这五个抽屉，就是那个“查”字落地的地方。

---

## 附录 A 术语表：全书概念的英中对照

> 术语表是本书的“术语宪法”——凡是正文用词，以本表为准。遇到同词异义，回到本表看指称的是哪个概念；遇到跨域对不上的词（如恒信的“合同编号”与冰链的“数据所有权”），先查本表，再谈别的。

**T-01　概念**（concept）　一类对象共同特征的抽象概括；严谨定义＝由特性的独特组合而形成的知识单元（ISO 1087:2019 §3.2.7）〔第 1 章〕

**T-02　术语**（term）　概念的能指——同一个概念可以有多个词〔第 1 章〕

**T-03　符号**（sign）　能指 + 所指 的统一体（索绪尔）〔第 1 章〕

**T-04　能指 / 所指**（signifier / signified）　符号的形式（声音/文字）/ 符号的内容（概念）〔第 1 章〕

**T-05　涵义**（sense（Sinn））　符号呈现对象的方式（弗雷格）——“清晨天边那颗亮的”〔第 1 章〕

**T-06　指称**（reference（Bedeutung））　符号实际指向的对象——金星〔第 1 章〕

**T-07　同物异名**　同一个指称、多个涵义——“漏”的根源〔第 1 章〕

**T-08　同名异物**　同一个符号、多个所指——“吵”的根源〔第 1 章〕

**T-09　词汇病**　误解的三个形态：说不出定义 / 同词异义 / 翻译伪朋友〔第 1 章〕

**T-10　翻译伪朋友**（false friend）　英文原词不同、中文译成形近字的陷阱（评审/验证/确认）〔第 1 章〕

**T-11　概念档案**（concept profile）　按九格为概念建立的完整档案（定界·溯源·致用·纠误）〔第 1 章〕

**T-12　九格尺子**　①定义⑥对立⑦辨析⑨英文 · ②背景③历史 · ④问题⑤场景 · ⑧误区（概念档案 B 区的固定栏目）〔附 B〕

**T-103　枢纽**（hub）　解释别的概念要用它、解释它自己却不需要太多前置知识的概念——领域的核心概念〔第 1 章〕

**T-104　四问法**（four questions）　真正理解一个概念的四问：是什么/不是什么/从哪来/往哪去——能答全四问才算拥有〔第 1 章〕

**T-105　五环模型**（five-ring model）　掌握专业知识的循环：立枢纽→织关系→逼输出→勤校准→落实践；掌握线在“应用”之上〔第 1 章〕

**T-106　压缩学习**（compressive learning）　把全量信息转化成关键信号的学习——先立枢纽越学越轻松；人脑是压缩机器不是存储器〔第 1 章〕

**T-107　核心原理**（core principle）　概念之间反复出现的规律（边/规则），用“当……必须……”表达——比概念更高级的枢纽〔第 1 章〕

**T-108　三步识别法**（three-step identification）　识别枢纽的三步：收集候选→硬判据筛选（词频/被引/前置数/反事实）→往上挂〔第 1 章〕

**T-109　概念体系能力**（conceptual ability）　以概念为单位的压缩与组织能力——识别枢纽×织关系×迁移×辨边界（简称“概念能力”；判据：转述测试+辨伪测试+迁移测试）。**章题用法**：第 1 章章题作“概念体系：学习能力的杠杆”——题面省去“能力”二字，所指仍为本条；“概念体系”是名词、本条是主词，正文首现处须有一句把“体系”与“能力”接上〔第 1 章〕

**T-110　根概念能力**（root-concept ability）　识别、验证并运用根概念的能力——概念体系能力的子能力（方向盘：先抓承重柱，再展骨架）；判据：20 分钟给出根概念+核心概念清单及其关系，并能用判定矩阵与演绎链辩护〔第 1 章〕

**T-111　杠杆模型**（lever model）　学习产出＝根概念能力（方向）×概念体系能力（力臂）×学习投入（施力）×实践场景（支点）；**两个归零项是方向与表征（两根轴），力臂·施力·支点只负责放大**；杠杆的第一性是方向（§1.4.1~§1.4.5）〔第 1 章〕

**T-112　框架**（framework）　对某一类事物/活动预制的骨架+位置+组织规则，内容后填——用框架 ≠ 有架构（见 §10.4.5）〔第 10 章〕

**T-13　工程**（engineering）　科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西〔第 2 章〕

**T-14　手艺**（craft）　靠个人经验与技巧、结果依赖人的做事方式（工程的方法论对立面）〔第 2 章〕

**T-15　科学**（science）　发现现存的世界（工程的认识论对立面——创造未存在的世界）〔第 2 章〕

**T-16　艺术**（art）　表达自我（工程的目标对立面——满足用户）〔第 2 章〕

**T-17　方法论**（methodology）　关于“怎么组织工作、怎么选方法”的系统学说（工程的一层，非全部）〔第 2 章〕

**T-18　系统工程**（systems engineering）　把部件整合成整体的元工程——不造部件、管接口、做系统级验证与确认。**口径**：本书“**部件**”＝正本的 system element（ISO/IEC/IEEE 15288:2023 **§3.47** 正本名“**系统元素**”：discrete part of a system that can be implemented to fulfil specified requirements）；“**接口**”＝部件之间的约定——接口契约、数据定义、时序依赖，正式条见 **T-136**〔第 2 章〕

**T-19　工程系统**（engineered system）　被设计/改造、与环境交互、有目的、守约束的系统（INCOSE 2019：“a system designed or adapted to interact with an anticipated operational environment to achieve one or more intended purposes while complying with applicable constraints”）。

**INCOSE 另说明：工程系统可由人、产品、服务、信息、流程，甚至自然元素组成**〔第 2 章〕

**T-20　技术系统**（technical system）　仅由人工技术组件构成的系统，人只是使用者〔第 2 章〕

**T-21　社会技术系统**（socio-technical system）　技术子系统 + 人的子系统 + 组织子系统耦合的整体〔第 2 章〕

**T-22　涌现属性**（emergent behavior / emergent property）　部件全对 ≠ 系统对的整体行为（系统的“缝”里的事）——**“部件全对、系统未必对”正是确认侧的题眼**（§2.4.2）〔第 2/8 章〕

**T-23　系统化 / 系统性 / 算法化**（systematic / systemic / algorithmic）　做事有章法 / 看问题有全局 / 确定性自动执行〔第 2 章〕

**T-24　试金石**　换个人、换团队，用同样的需求和约束，能交付质量相当的成果——能是工程，不能是手艺。**熟语式命名**〔第 2 章〕

**T-25　工程师**（engineer）　让工程成立的人——一份能力清单，里面既有能力也有活：能力＝工程科学、方法论、工具与实践、价值与伦理各长一项（科学素养／方法论素养／实践能力／职业伦理），另加工程判断（横切）与可复现意识（元能力）；活＝需求、设计开发、产品数据、评审验证确认，按第 3 章那条路径依次走一遍；全清单只指向一条判据（换个人、用同样的需求和约束，能不能交付质量相当的成果）；同一个统称底下站着好几种粒度（模块工程师／系统工程师／系统架构师）〔第 9 章〕

**T-26　需求**（requirement）　明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）。**★ 术语口径：requirement 一律作「需求」；「要求」是另一个词（demand），不用来指 requirement**——正本中译在 §3.3.11／§3.5.2 等处把

requirement 译作「要求」，本书引用时按本书术语写作「需求」，条款号不变〔第 3、9 章〕

**T-27　需要 / 期望**（need / expectation）　硬性诉求（缺了会痛）/ 软性诉求（缺了会失望）——两者是**内容**，与“明示 / 通常隐含 / 必须履行”这个**呈现方式**交叉，才是需求的完整样子（ISO 9000:2026 §3.5.1；两维画法是本书的梳理主张）〔第 3、9 章〕

**T-28　目标**（objective）　要实现的结果（ISO 9000:2026 §3.7.11），系统语境即系统预期达成的可度量结果；主语是结果，可落在战略/战术/运行层面、可针对整个组织或其过程/产品/服务（§3.7.11 注 1/注 2），不必是系统自身属性；目的回答“为什么做”、目标回答“达成什么可度量结果”、需求回答“做成什么样”；操作目标=目的在运行环境的可度量投影≈INCOSE MOE 层〔第 3/6 章〕

**T-29　客体**（object）　可感知或可想象的任何事物——被设计的东西（ISO 9000:2026 §3.5.3）〔第 3 章〕

**T-30　特性**（characteristic）　可区分的特征——需求落到特性上才可测量、可验证（§3.10.1）〔第 3 章〕

**T-31　设计开发**（design and development）　将某一客体的需求转换为该客体更详细需求的一组过程（ISO 9000:2026 §3.3.11）——“更详细”是它有没有发生的唯一判据〔第 3、9 章〕

**T-32　质量**（quality）　客体的一组固有特性满足需求的程度（ISO 9000:2026 §3.5.2）——是程度，不是除法〔第 3 章〕

**T-33　符合性**（conformity）　满足了吗——二者取一（符合 / 不符合，ISO 9000:2026 §3.5.9；注 1 明示 conformance 已废弃）。**★ 正本中文定名作「合格」**——本书行文保留「符合性」，为的是与“质量”“等级”两个近亲在词面上分得开〔第 3 章〕

**T-34　等级**（grade）　对功能用途相同但需求不同的客体所做的类别或级别划分——高档 / 低档（ISO 9000:2026 §3.5.4）〔第 3 章〕

**T-35　过度设计**（over-engineering）　特性超出需求范围——只加成本，不加质量〔第 3 章〕

**T-36　接收准则**（acceptance criteria）　提前写在测试方案里的判定标准——质量“程度”能认定的前提（又称“验收标准”——判据义惯用语：验收标准是判据，验收是评审、验证、确认依次签字，两义区分）〔第 3 章〕

**T-37　需求工程**（requirements engineering）　需求开发 + 需求管理（总称，两段并列）——开发负责做出来，管理负责让它一直是那一份〔第 4 章〕

**T-38　需求开发**（requirements development）　把需要与期望转化为规定需求——获取、分析、规格说明、需求校核（评审）〔第 4、9 章〕

**T-39　需求管理**（requirements management）　对规定需求进行基线化治理——基线、变更控制、追溯、状态与度量〔第 4 章〕

**T-40　规定需求**（specified requirement）　被明示的需求——验证的参照；**成文与基线是它的可采条件，不是构成条件**（§3.5.1 注 2 的 e.g. 是举例）〔第 4 章〕

**T-41　运行概念**（concept of operations（ConOps））　系统怎么被用的叙事文档——需求工程里唯一一份叙事性的正式输出（IEEE 1362）〔第 4 章〕

**T-42　需求规格说明书**（SRS）　规定需求的载体——六章骨架（引言·总体描述·功能·非功能·外部接口·验证方法）。**章题用法**：第 4 章章题作“把诉求变成需求规格”——题面的“需求规格”即本条简称

（正式名“需求规格说明书”）；**须与“规定需求”（T-40）分开**：规格是载体、规定需求是内容与枢纽〔第 4 章〕

**T-43　追溯矩阵**（traceability matrix（RTM））　需求从哪来、到哪去的对照——变更影响分析的依据、测试覆盖的对照、合规审计的证据〔第 4 章〕

**T-44　需求管理计划**（requirements management plan（RMP））　项目级“怎么管需求”的执行计划（程序文件的项目版）〔第 4 章〕

**T-45　好需求的判据**　完整 · 一致 · 无歧义 · 可验证 · 可追溯 · 有优先级（**旧称“需求质量六性”，数字名已废**；第 4 章节题作“一条好需求的判据”、正文以“六条”计数称之；第 8 章 §8.2.7 两处旧称待随该章修订同步）〔第 4 章〕

**T-46　变更请求单 / 变更控制委员会**（change request（CR）/ change control board（CCB））　变更的立案凭证 / 裁决变更的机构〔第 4/12 章〕

**T-47　功能需求**（functional requirement）　系统所具备的行为特征，以 IPO（输入/处理/输出）定义，及其指标〔第 5 章〕

**T-48　环境适应性需求**（environmental adaptability requirement）　对气候/国家/文化/风俗/法律等各类环境的适应性，及其指标〔第 5 章〕

**T-49　可实现性需求**（realizability requirement）　全生命周期'方便实现'+经济可承受性（八项）及其指标；**八个方面＝物料可采购性／可制造性／可测试性／可交付性／可部署性／可运维性／可回收性＋经济可承受性**（第 5 章 §5.2.5）；**八个方面＝物料可采购性／可制造性／可测试性／可交付性／可部署性／可运维性／可回收性＋经济可承受性**（第 5 章 §5.2.5）；**八个方面＝物料可采购性／**

**可制造性／可测试性／可交付性／可部署性／可运维性／可回收性＋经济可承受性**（第 5 章 §5.2.5）〔第 5 章〕

**T-50　验证载体**（validation vehicle）　每类需求的专属验证交付物：原理样机/环境样机/**真正的产品**——英文名沿用 validation vehicle，按第 8 章严格口径部分载体更近确认（见第 5 章辨析）〔第 5 章〕

**T-51　承制方主导**　需求开发由承制方主导（引导范围·形成条目），利益相关方保留确认终审权〔第 5 章〕

**T-52　框架定义**（preliminary definition）　设计树的干枝——回答“结构是什么、承接了谁、往下分解出什么”；＝**框架**这一形态（根→枝梢，叶只以展开项露名）；同一份产品数据按定义到多深分**框架／概要／定义**三形态，最深的形态才是“需求-方案”这一对的端点（旧“完整态”一词已下线）；枝梢节点物理意义双向可用（承接判据＋分解源判据）〔第 6 章〕

**T-53　详细定义**（detailed definition）　设计树的**叶节点**——回答“每个模块具体做什么”，每条可独立定义、可独立验证〔第 6 章〕

**T-54　分解分析**（decomposition & analysis）　**节点**按分解规则分解为方案**叶节点**（切点沿消费方语义范围），并做承接覆盖检查——“接住的需求一定被实现”（树内动作·阶段二）〔第 6 章〕

**T-55　分配生成**（allocation & generation）　需求**叶节点**落到方案侧能负责它的**节点**上（复用→改进→新增；**落点以语义定级**，干枝或枝梢由语义定），并为新节点自下而上调树（树间+树上动作·阶段一）〔第 6 章〕

**T-56　符合性分析**（compliance analysis）　检查每个方案**节点**是否充分满足所承接的全部需求（语义＋指标）——承接覆盖检查的落纸形态〔第 6 章〕

**T-57　文档结对**（document pair）　每对“需求-方案”：上层叶节点＝需求方，下层节点＝方案方〔第 6 章〕

**T-196　分析迭代**（analysis & iteration）　检查分配与生成的正确性（单承接、承载量、影响面），并按检查结果迭代——迭代的主要对象是需求分解；修正方向单向：节点是基准、需求是适配方；改进必做影响面复查〔第 6 章〕

**T-197　两个存在（物理存在／信息存在）**（physical existence / information existence）　产品同时有两个存在：**物理存在**（实物，会老化、会报废）与**信息存在**（描述它“是什么、为什么、怎么做、怎么证明对”的全部数据，可复制、可传递、可留存）；**实物是信息存在的复制品**——从开发角度看，你真正“造”的是信息存在。本书凝练（产品数据研究报告 14 号 §4.1 作“一个产品有两个存在”）〔第 7 章〕

**T-198　六类数据**（six categories of product data）　**本书工作名**：产品数据的工程实践分类——需求数据／设计数据／制造数据／质量数据／服务数据／管理数据；每类各答“从哪来、到哪去、它的质量决定什么”，可按行业增删。判据：覆盖产品从“该做什么”到“做对了吗”到“用起来怎么样”的全程〔第 7 章〕

**T-199　数据质量**（data quality）　拿产品数据这组“固有特性”满足要求的程度（承第 3 章质量定义）；**开发阶段可评价的质量就是数据质量**。评价取五把尺子：准确性／完整性／一致性／**现时性**／可信度（据 ISO/IEC 25012，经 ISO/IEC/IEEE 42030:2019 表 A.3 可达）〔第 7 章〕

**T-200　实现方案**（realization scheme）　**本书工作名**：设计数据与制造数据合起来的那一整套——实物产品按“止于可实现”具体化为**八个方案**（技术／物料采购／制造／测试／交付／部署／操作／运维），每个方案＝给一个下游环节的“可实现需求”；**方案齐备**＝

**可实现＝设计开发的终点**。依设计开发概念研究报告（01 号）§4.3〔第 7 章〕

**T-201　增值判据**（value-added criterion）　**本书工作名**：判一个活动值不值的口径——**看它有没有让产品数据增值**。两型：**内容增值**（数据更准更对，且须顾客需要）与**状态增值**（内容没变、状态变了：草稿转受控、合规打标注）；**不增值 ≠ 浪费**（必要非增值要压缩保留，纯浪费才消除）。判据双维：工件增量 × 流动效率〔第 7 章〕

**T-202　流水线（工件·工位·增值）**（pipeline (work item · workstation · value added)）　**本书工作名（数据流动视角）**：产品开发＝一条流水线——**工件＝产品数据、工位＝需求定义／设计／实现／验证／移交、增值＝每个工位往数据上添加价值（也可能添加缺陷）**；与“瀑布流程”（流程组织方式）不是一回事。用于看清“数据在不在流动、有没有增值”〔第 7 章〕

**T-203　设计态同一性**（identity in the as-designed state）　**本书工作名**：在设计态，**开发产品 ≡ 开发产品数据**（产品无物理实体，全部存在＝那组受控产品数据）；严格说法是“**开发 = 定义生成**”。两条边界：只在设计态、对纯信息产品严格成立；不等于“产品就是数据”。**与“同一性目的论”（见 T-144）是同词两概念，不可混**〔第 7 章〕

**T-204　架构定义**（architecture definition）　**过程名**（15288:2023 §6.4.4 System architecture definition process）**兼动作名**（本书）：这个过程把一组**输入**（相关架构／组织与项目的方针与指令／生存周期概念与约束／利益相关方关切与需求／系统需求与约束；**目的与目标是准绳，不在其中**）**变换**成两样内容（**基本概念或属性＋管控原则**）——**不是“给架构下定义”**（那是第 10 章已完成的事），是一次看得见进出口的变换；本章只走到“立成内容”（选定与背书归第 12 章、表达成件归第 13 章）〔第 11 章〕

**T-205　生存周期概念与约束**（life cycle concepts and constraints）　15288:2023 §6.4.4.1 目的句里的一类输入——运营概念、保障概念这一类东西，**给出场景与边界**；本书旧译“生命周期”一律作**生存周期**〔第 11 章〕

**T-206　资产稀缺**（asset scarcity）　**本书归拢**：手上没有可用资产（或资产仍在、但不该沿用）时怎么走——先**分流**——**42020:2019 E.3.1 共六种情形**（E.3.1.2 全新开发／E.3.1.3 产品线中的新产品／E.3.1.4 遗留系统演进／E.3.1.5 遗留系统革命／E.3.1.6 增量式启动／E.3.1.7 产品线式启动），**本书在“资产稀缺”题下取四种**（全新开发／产品线启动／遗产革新／增量启动；未取的两种手上有资产）＋**五处适配**（走法／取材／输出／迭代策略／收窄口径）＋**三条红线**（不许拿“没资产”当不写架构的理由／不许把参考架构当答案／不许把“不熟”写成“不该”）〔第 11 章〕

**T-207　候选（架构候选）**（candidate）　过了**四道准入**的潜在解——**有类可归·有物可依·有径可循·有辨可评**；四项全过＝**候选**，任何一项不过只是一个“**想法**”；来路三条（上游给的偏好解类／既有解与既有架构／模式·战术·启发式），**动词是综合与识别，不是发明**；每个候选写一句**目的陈述**〔第 11 章〕

**T-58　评审**（review）　为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（ISO 9000:2026 §3.11.2）——对目标〔第 8 章〕

**T-59　验证**（verification）　通过提供客观证据，证实规定需求已得到满足（ISO 9000:2026 §3.11.12）——对需求〔第 8 章〕

**T-60　确认**（validation）　通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（ISO 9000:2026 §3.11.14）——对用途〔第 8 章〕

**T-61　复盘**（retrospective）　干完之后的经验沉淀与流程改进——不是评审（评审管把关）〔第 8 章〕

**T-62　用户验收测试**（UAT）　真实用户 + 真实任务 + 真实场景的确认——不是让业务方当测试员〔第 8 章〕

**T-63　架构**（architecture）　一个实体（如系统）在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（governing principles；ISO/IEC/IEEE 42010:2022）；本书读作系统不可还原的涵义（落前半句；**不是“一组决定”**——决定只是抓手，正位在 §5.2.12／§6.10）。**别名**：第 10 章题面用“**架构概念**”指“架构”这个词所指的那件事本身（与 42020:2019 §8.4.8 的 architecture concepts＝架构里的概念与属性，**同名不同指**）〔第 10 章〕

**T-64　架构描述**（architecture description）　**承载架构那份内容的工作产品**：能签收、能入库、能版本化（**内容与载体**，本章 §10.2.3）；内部组织链“利益相关方 → 关注点 → 架构视角 → 架构视图”归第 13 章〔第 10、13〕

**T-65　利益相关方 / 关注点**（interested party / stakeholder / concern）　**关切＝关注点**（本书正文用「关切」，第 13 章 13.2.2 立户；标准中译作“关注点”）——对系统有利益关系的任何个人/团队/组织 / 对系统的任何兴趣〔第 10、13〕

**T-66　可复现意识**（reproducibility mindset）　元能力：让判断被接住——能力＝开发出高质量的产品数据（第 2 章那把试金石落到一个人身上的版本）〔第 9 章〕

**T-68　架构设计**（architecture design）　把需求翻译成“架构”的活动——产出两样内容（**基本概念或属性＋管控原则**），并把它表达为可评审、可交付的**架构描述**；**元素与关系是架构“可以关乎”的**

**对象**（不在定义句内），其规格归设计定义过程（15288:2023 §3.13）〔第 11 章〕

**T-69　基本概念**（fundamental concepts）　系统“怎么想”的根源——组织范式/核心抽象/根本机制/根隐喻；90% 选 + 10% 逼，幸存下来〔第 11 章〕

**T-70　基本属性**（fundamental properties）　系统“具备什么”的本质特征——涌现属性，不能直接设计，只能设计结构让涌现〔第 11 章〕

**T-71　管控原则**（governing principles）　架构师自愿立法、立了就有约束力：约束实现（怎么造对）与演化（怎么变不破坏）及其生存周期；本书早期稿件作「治理原则」——与 governance principles（组织治理原则）不是同一概念；必须/禁止/应当 + 理由；可判定/有理由/少而精/可入评审〔第 10、11 章〕

**T-72　方面体**（aspect）　实体特性的一个客观切面（functional/structural/informational/behavioral 等，42010:2022 §3.9；GB/T 45630-2025 定名「方面体」）；基本概念+属性清单可按方面体分块——用于系统化覆盖关注点〔第 10、11、13 章〕

**T-73　关系**（relationships）　元素之间那些断不得的约定——关系是成本：克制、往弱推、契约化。（**元素与关系不在架构定义句内**：架构“可以关乎”它们〔42010:2022 §5.2.2〕，其规格归设计定义过程）〔第 11 章〕

**T-74　架构决策**（architecture decision）　**在架构总体语境中所做选择的一个集合**（42010:2022 §5.2.12 a collection of choices made in the overall context of an architecture）——**不是一个动作、不是拍板那一下。准入看目的必要性**（承第 10 章）；影响半径／持续时间／修改成本三标志只是**诊断与排序的代理**，不是准入判据。**汉**

**语义项错位提示**：日常汉语的“决策”多指“拍板那一刻”，标准义的“决策”是一组选择加它的理据——本书正文按标准义用〔第 12 章〕

**T-75　不可逆性**（irreversibility）　修改成本高到理性的决策者宁愿忍受现状、也不掏钱去改——经济属性不是技术属性；作判据时只是**代理**：会**漏判**（不贵但承重，如“日志须可被外部审计员独立读取”）、也会**误纳**（很贵但与目的无关，如框架版本、组织惯例技术栈）。**本章口径**：不可逆只管**排序与时机**，不管准入；量它用**本书凝练的三样**——影响半径／信息成本／修改风险（见 12.4.2）〔第 12 章〕

**T-76　架构决策记录**（architecture decision record（ADR））**标准给内容项、不给定栏位**：42010:2022 §6.10.1 列**十一项内容**（唯一标识／清晰陈述／**决策权属方或决策责任人**／约束和假设／连关切或方面／连受影响的 AD 元素／连理据／**与其他决策的关系**／后果／**作出、获批、修改三个时间戳**／来源引用）。**把十一项压成六问，是本书的做法**（名称／缘由／候选／定者与时间／影响／反悔难度）。理据随行（§6.10.2 shall）；被否的备选及其理由宜记（§6.10.1 第二句 should）。**记录不是一次定死**：可改，改了留痕〔第 12 章〕

**T-77　期权价值**（option value）　推迟低不可逆决策、保留选择权——“现在不做”本身是价值。★ **本章不用此名**：正解＝ISO/IEC/IEEE 42020:2019 §8.3 的**承诺不对称**（早期收窄设计空间的收益即时、改起来的代价延后且高，见 12.4.3）〔第 12 章〕

**T-78　康威定律**（Conway's Law）　系统结构是组织沟通结构的副本——构件边界就是团队边界〔第 11、17 章〕

**T-79　配置**（configuration）　在配置信息中规定的产品（或服务）的相互关联的功能特性和物理特性（ISO 9000:2026 §3.10.8）——**是一套特性，不是一套设置**：参数只是配置信息里的一个条目，“改一下配置”在日常里指调参数、在正本里指这台设备的样子变了。

三个词各有各的所指、互相引用成一个环（配置用配置信息定义、配置项用配置定义、配置信息又指回配置项）：配置＝**特性**、配置项＝**实体**（T-80）、配置信息＝**记录**（T-81）（§15.2.1）。**判据**：问它答的是“它是什么样”还是“它怎么调”——答“怎么调”的是设置，答“是什么样”的才进得来。**译名提醒**：GB/T 19000—2016 把 configuration 一族译作“技术状态”（技术状态项／技术状态信息／技术状态基线），说的是同一件事〔第 15 章〕

**T-80　配置项**（configuration item）　配置之中满足某一最终使用功能的实体（ISO 9000:2026 §3.10.9）——配置这件东西被切出来的**管理单位**（对象，被管起来的那个东西）。**靠“选定”得到，不是从天上掉下来**：选定要有人做、有记录；选定之后要有一份配置项清单，清单上每一项都答得出“它是什么、归谁管”——答不出的那一样，多半还只是某个人的习惯（§15.2.2）〔第 15 章〕

**T-81　配置信息**（configuration information）　关于产品或服务的设计、实现、验证、运行和支持的**要求**（ISO 9000:2026 §3.10.10）——把配置那套特性写下来的那一份（记录，它的档案）。**不等于参数**：参数只是这本账里的一页，配置信息不止参数（§15.3.1）；**一版经批准的配置信息就是配置基线**（§3.10.12，见 T-86）〔第 15 章〕

**T-82　版本**（version）　配置项的初始发布或其后基于完整构建的重新发布（定义句取 ISO/IEC/IEEE 24765，见 R-20）——时间轴上的演进刻度。**正本四份未给“版本”词条**：15288:2023 把力气花在**版本标识**上——架构视角、架构视图、视图组件三处各有一条 shall，“每一张架构视角应含由组织和／或项目规定的版本标识”（Each architecture viewpoint shall include version identification as specified by the organization and/or project.；中译：每一张架构视角应含由组织和／或项目规定的版本标识。ISO/IEC/IEEE 42010:2022 §6.6~§6.8），同一组要求另落在架构描述框架（§7.1）

与模型种类（§8.2）下、那两处用 should（宜）。**第 15 章用法**：版本最朴素的样子就是给每一次改动一个号（“主.次.修订”三段式是业界惯例、不是标准条文），它给出的是**位置**——答得出先后，答不出资格；最新版不等于算数的那一份（§15.2.3）〔第 15 章〕

**T-83　修订**（revision）　内部变更次数（Rev A/B/C）——比版本更细的一档。**第 15 章用法**：就是三段式编号里“修了缺陷进修订号”的那一位（业界惯例，不是标准条文）；修订与版本一样只给**位置**，不回答“哪一版算数”（§15.2.3）〔第 15 章〕

**T-84　变体**（variant）　**空间**并存的形态差异（−20℃ 冷链型／2~8℃ 冷藏型）——与版本正交（版本是**时间**：同一条线上的一次次改动）。**第 15 章用法**：变体各有自己的配置项，各自往下走自己的版本线，基线也各自立——变体管“哪一种”、版本管“哪一次”、基线管“哪一版算数”，三样各占一个位置，缺一个另外两个就顶不住（§15.3.3）〔第 15 章〕

**T-85　基线**（baseline）　配置项不论介质如何、在配置项生存周期内的特定时刻被正式指定并固定的、经正式批准的版本（ISO/IEC/IEEE 15288:2023 §3.8；该条注明出处为 IEEE Std 828-2012，见 R-41）——**第 15 章的三特征，缺一就散架**：**批准**给资格、**冻结**给状态、**参照**给用途；最容易漏的是参照（会开了、字签了，大家接着用自己电脑上那份“顺手改过一点”的干活，批准与冻结都有、参照没有，基线于是名存实亡）。**承载的只是“明示的”那一部分**——评审与验证的判据长在这条链上，确认（validation）撑不住。**与版本的关系**：版本是刻度、基线是资格（T-82）；正本另给两个名字，管受控内容本身的**配置基线**与管产品定型的**产品基线**（15288:2023 §6.3.5.2 b)／§6.3.5.3 e) 4)），从信息一侧的同义句见 ISO 9000:2026 §3.10.12 ——都归 T-86；架构这一侧被选定并冻结的那一版见 T-217（§15.2.4）〔第 15 章〕

**T-86　配置基线 / 产品基线**（configuration baseline / product baseline）　**正本给的两个名字，一前一后两步**（§15.2.5）。**配置基线**（15288:2023 §6.3.5.2 b）：“配置基线（包括已批准的配置）得到维护”；ISO 9000:2026 §3.10.12 从信息一侧的同义句是“经批准的配置信息，它确定产品或服务在某一时间点上的特性，并作为该产品或服务生存周期内各项活动的参照”）——管的是**受控内容本身**：配置信息与它的基线被建立起来、被维护住。**产品基线**（15288:2023 §6.3.5.3 e) 4）：“执行配置及配置管理验证活动与审核，以建立产品基线”）——管的是“能不能定型”：审核做完了，产品这一版才可以建立起来、进入发布；该条底下的注把两种审核点了名，**功能配置审核**（functional configuration audit，FCA）看功能与性能能力、**物理配置审核**（physical configuration audit，PCA）看系统对运行信息部件与配置信息部件的符合性，执行审核的是验证过程（注 22）。“功能基线／分配基线”的行业出处是 ISO 10007／SAE EIA-649／MIL-STD-973 等（见 R-08、R-42；15288:2023 附录仅把 SAE EIA-649 列为配置管理附加指南）；这两个名字只作历史名说明，**不当正本讲**〔第 15 章〕

**T-87　配置管理**（configuration management（CM））　**过程名**：在系统与系统元素的生存周期内管理其配置，管理包括“建立并保持一致性、完整性、可追溯性与控制”（ISO/IEC/IEEE 15288:2023 §6.3.5.1）；做到位的样子是六条可当场检查的状态（配置得到管理／配置基线得到维护／纳入项的更改得到控制／配置状态信息可获取／所需审核得以完成／系统发布得到批准，§6.3.5.2 a)~f)）。**五项活动**（§6.3.5.3 a)~e)）：**准备**（定角色、职责、问责与职权，定要建立哪些基线，定存储位置与条件，定何时启动配置控制）／**配置标识**（选进来、给标识、写描述）／**更改管理**（识别并记录更改请求与偏离请求，协调、评估并处置）／**配置状态记帐**（T-219）／**验证与审**

**核**（验证配置满足要求、监视已批准更改的纳入、执行审核以建立产品基线）。**与前面各章并行，不是它们的下一步**——15288:2023 十四个技术过程每一个都“提供已选定用于基线的关键人工制品”（Provide key artefacts that have been selected for baselines；中译：提供已选定用于基线的关键人工制品），架构定义只是这十四个供料口里的一个（§15.1.3）。正本是**五项配置管理活动**，“准备”最常被跳过、一缺后四项即成临时应付；“配置标识／配置控制／状态记录／配置审计”这条四段链属 ISO 10007 系旧说法（见 R-08）；另**配置审计 ≠ 审核**：前者对象是配置项与产品、判据是基线与配置信息（§6.3.5.3 d) 注 19），后者对象是组织与体系、判据是审核准则（ISO 9000:2026 §3.12.1）——两个词只共用了中文一个“审”字〔第 15 章〕

**T-88　变更控制**（change control）　变更申请 → 影响评估 → CCB 批准 → 实施 → 建立新基线（冻结→变更→再冻结）；**需求侧另含“再验证”节点**——CR → 影响分析 → CCB 评审 → 实施变更 → 再验证 → 关闭（第 4 章 §4.2.6，闭环）〔第 15/4 章〕

**T-89　契约链**（chain of contracts）　每一对相邻环节都是局部甲乙方——基线是冻结端、版本是交付端〔第 17 章〕

**T-90　治理**（governance）　**本书主口径取正本**：架构治理＝在组织范围内对架构进行管理和控制所依循的规程与取向（ISO/IEC/IEEE 42020:2019 附录 G.1，资料性），过程侧定义权在第 6 章（**须标“本章定义权在 42020，不在行业框架”**）；它是**一个词的两面**——既是一套制度系统（谁在治、拿什么治、治出了什么），也是那套系统一直在做的动作（§6.4 六个活动、共 49 条任务）；通义侧另有“以人为基础的系统”一解（**ISO 37000:2021 §3.1.1／ISO/IEC 38500:2024 §3.3）；“掌舵不是统治”是**本书比喻**，实质说“守不变量”〔第 16 章〕

**T-91　管理**（management）　在既定框架内把事做好、做对、做快的执行活动（**架构管理的目的句**：实施架构治理指令，及时、高效和有效地实现架构集合目标——ISO/IEC/IEEE 42020:2019 §7.1）；**“守不变量、调变量”是本书对治理／管理两句目的句的概括**（“划桨”是本书比喻）；**治理建机制（§6.4.2 k)）、管理执行过程（§7.4.4 m)）**——机制≠过程〔第 16 章〕

**T-92　统治**（rule）　预设“主权者-臣属”关系、靠强制自上而下压下来的支配〔第 16 章〕

**T-93　控制论**（cybernetics）　通过反馈调节方向、修正偏差（与 governance 同根）〔第 16 章〕

**T-94　委托必有治理**（delegation & accountability）　责任可委托、**担责不可委托**（ISO 37000 §4.2.2／§6.5.2、ISO/IEC 38500 §6.1 ——治理机构可授权他人，仍对整个组织的作为、不作为和渎职负责）；授权有效五条件（结果协商一致／资源可用／责权匹配／定期报告并举证／后果可执行，§4.2.2）；委托不是治理的前提，无委托时治理同样是组织常态（§3.3）〔第 16 章〕

**T-95　决策系统 / 执行系统**（四活动 / PDCA）　治理跑四活动循环（利益相关方参与→评估→指导→监督，ISO 38500:2024 §6），管理跑 PDCA（计划→执行→检查→改进）；**评估判“该不该”（批准之前）、监督复核“是不是”（批准之后）；oversight ⊃ monitor**〔第 16 章〕

**T-96　治理目的**（governance purpose）　三项治理结果：**有效的绩效 / 尽责的管理 / 合乎伦理的行为**（ISO 37000:2021 §5；ISO/IEC 38500:2024 §4 原封搬入）——标准认“结果”，“有没有董事会”“有没有制度文件”都不算数〔第 16 章〕

**T-97　力量倍增器**（force multiplier）　委员会定规则自动处理 990 个争议，只裁 10 个例外〔第 16 章〕

**T-98　术语膨胀**　用治理的名字干管理的活——伞是“管理”，治理只是伞下的子域〔第 16 章〕

**T-99　顺次切片**　需求→设计→把关→运维：时间上接力的一站（第 3~8 章）；设计站的产出形态是产品数据（第 7 章）——不另占一站，挂在站点②下〔第 17 章〕

**T-100　横切切片**　架构（**第 10~13 章**）/配置（**第 15 章**）/治理（**第 16 章**）：垂直穿过每一站，不占时间线上的一段——**不能当顺次的一站做**（第 17 章的**工程师**不在这两种切法里，第 14 章是“架构交出去、设计接住”那道题的例子）〔第 17 章〕

**T-101　接缝**　切片之间靠承接/参照/记录/问责四条缝合的关系——切了就要缝回来〔第 17 章〕

**T-227　切片　本书立名**：把同一件事按一种切法分出来的那几片——本书只两种切法，顺次切片（顺着时间）与横切切片（横着时间）；**两条线作为内容的分类成立，作为流程的分类不成立**〔第 17 章〕

**T-228　缝合　本书立名**：把切开之后断掉的地方连回去的那件事——四种缝（承接／参照／记录／问责）就是四种连法；**缝合不是最后多出的一道工序，是每一段上都在做的事**〔第 17 章〕

**T-229　缝合清单　本书立名**：把接缝落到纸上的一张表，每一行只写三件事——缝在哪两片之间／靠哪一种连上／责任人是谁；**写得出来，才缝得上**（不是给检查看的台账，是给干活的人先知道缝在哪儿）〔第 17 章〕

**T-102　架构师**（architect）　**把架构活动背在身上的人**——正式名是**系统架构师**（T-115）。能力清单＝**两段加两格**：前一段是工程师那六项（科学素养／方法论素养／实践能力／职业伦理，另加横切的工

程判断与可复现意识）；后一段是他**比工程师多出来的四件事**（给每个候选写一句目的陈述／把手段立成概念与原则／适配与调和／把“现状”也摆上桌）；两格是**配置管理**与**架构治理**。**对架构（基本概念或属性与管控原则）的成立与守持负专责**（responsibility，可委托可追责）；**不背系统成败的终责**（accountability），那是系统工程师的账。旧“四项增量”（识别后果严重决策／对系统的理论／把架构说清楚／判断不可逆性）已作废：其中“识别后果严重的决策”与“判断不可逆性”属**代理**，只管排序与时机，不占能力清单的准入位〔第 18 章〕

**T-113　系统工程师**（systems engineer）　实践系统工程学科的个人（与头衔无关；**类推**——24765 **无**该词条，系据同书 `systems security engineer` 词条句式，见 R-20）——对系统全**生存**周期技术事务负终责（accountability，不可委托）；转化/统筹/把关/守护〔第 9 章〕

**T-114　模块工程师**（module engineer）　系统元素/模块的实现责任者——模块级闭环（需求→设计→实现→验证）；系统工程师的分形（迷你系统工程师），分形不对称（模块级没有**对外的那部分架构**与确认／部署／集成方案；**模块自身的架构**归自己）〔第 9 章〕

**T-115　系统架构师**（systems architect）　工程师角色体系中专责“架构定义”的专门角色——**对架构（及其描述）的正确性**负专责（responsibility）；本书“架构师”的正式名〔第 9、18 章〕

**T-116　粒度**（granularity）　工程实践横跨的尺度（**元素级／系统级**）＋责任性质（专责／终责）——粒度 × 责任 决定角色。（**粒度只切角色账，不切架构与设计**——架构与设计的分界是**种类**，不是粒度）〔第 9 章〕

**T-117　终责 / 专责**（accountability / responsibility）　终责 = 系统成败的账（不可委托）；专责 = 划得清的账（可委托可追责）〔第 9、18 章〕

**T-118　架构治理**（architecture governance）　**在组织范围内对架构进行管理和控制所依循的规程与取向**（ISO/IEC/IEEE 42020:2019 附录 G.1，资料性；该附录另列 a)~d) 四条要求）；过程侧的定义权在**第 6 章**（“建立并保持架构集合中的架构与企业目标、策略和战略的一致”）；**“方向权属组织层、内容权属架构师”是本书读法**——正本只说治理层级与问责对象，并为治理指定**设计权威机构（Design Authority，附录 G.1）**，必要时逐级上报（**该机构名出自 39 号报告 A2 项，属本书补入的正本名）；系统级文档是“作品”、治理产出是“制度”〔第 16、18 章〕

**T-119　产品数据**（product data）　关于产品的信息的、形式化的表示，适合由人或计算机处理；产品的信息存在、开发工作的全部对象（六类：需求/设计/制造/质量/服务/管理）〔第 7 章〕

**T-120　两域三态**　产品在两个域（信息域/物理域）、三种状态（设计态/制造态/使用态）中存在；设计态产品=纯数据〔第 7 章〕

**T-121　设计态**（as-designed）　产品尚无物理实体、全部存在=受控产品数据的状态——设计态里开发产品 ≡ 开发产品数据〔第 7 章〕

**T-122　制造态**（as-built）　产品=数据+实物（MBOM、序列号、批次、制造记录）——数据被复制成实物的状态〔第 7 章〕

**T-123　使用态**（as-maintained）　产品=实物+实况（维护记录、配置实况、现场变更）——数据被实况反馈的状态〔第 7 章〕

**T-124　价值三层**　利益相关方对产品贡献价值的沉淀层（物化/决策/过程三层）；产品数据是价值的投影与锚，不是本体〔第 7 章〕

**T-125　根概念**（root concept）　领域中不可再向下归约的源头概念，其余概念（含核心概念）都由它导出——领域的“承重柱”（判据：原子/生成/不可替代/递归；通常 1~3 个）〔第 1 章〕

**T-126　基本原理**（root principle）　原理侧的“源头”——领域中不可再被推导的法则（命题），是根概念（母体）的语义内蕴被展开成的显式命题（判据：不被推导/生成规则/去之崩塌/多层生效）〔第 1 章〕

**T-127　三层体系**（three-layer concept system）　一般概念 ⊃ 核心概念 ⊃ 根概念——层级越深、判据越严、数量越少；根概念 ⊂ 核心概念 ⊂ 一般概念〔第 1 章〕

**T-128　架构活动**（architecting）　在系统整个**生存周期**内构思、定义、表达、记录、沟通、认证恰当实现、维护与改进架构的过程（ISO/IEC/IEEE 42010:2022 §3.1）；架构师能力清单的母词（第 18 章）。**两译名并存，全书统一另案**：库内 42010 中英对照版把 architecting 定名为「**架构工作**」，本书用「架构活动」（属既定语汇）〔第 18 章〕

**T-129　构思**（conceiving）　从关切出发生成多个候选架构概念的发散性概念活动——“打开可能性”，产物是“可能是什么”、不是“决定是什么”〔第 11 章〕

**T-130　定义**（defining）　在候选之间画边界、做选择，使方案从多义变单义、从漂浮变确定——“终结漂浮”；三重身份：动作义 / 流程义（15288 §6.4.4）/ 概念框架义（ISO 1087-1）〔第 11 章〕

**T-131　设计（一词两义）**（design）　广义 = ABET 容器（迭代决策与创造，⊃ 构思+定义）；狭义 = 以生效架构基线为输入的下游细化流程（15288 §6.4.5）——“转换到底”〔第 11 章〕

**T-132　创新**（innovation）　创造或重新分配价值的新对象或变更对象（结果义：新×成，缺一归零）；链尾的追认，不在这条动词链上〔第 11 章〕

**T-133　构架**（skeleton）　名词的动词化，本义“搭建承重骨架”；口语“把 XX 构架出来”≈ 架构生成①~⑤的投影，默认不含⑥评审背书与 ADR〔第 10 章〕

**T-134　架构驱动因素**（ASR）　从关切中筛出的值得架构的问题；筛选三标准（影响面广/难逆转/约束硬），6±3 个〔第 11 章〕

**T-135　效用树**（utility tree）　质量属性→场景，按重要性×难度（H/M/L）打分的权衡工具（ATAM）〔第 11 章〕

**T-136　接口控制文档**（ICD）　模块间接口契约——接口是**槽位与填料的多对多映射**（本书口径；1:1 搬入已废）〔第 11、18 章〕

**T-137　架构生成**（architecture generation）　借代用法（D 层）：构思①②→定义动作义③④→表达⑤，补认证⑥成完整闭环；设计（狭义）在基线生效后接棒〔第 11 章〕

**T-138　目的判据**（purpose criterion）　fundamental 的裁决者——对系统在其环境中的目的而言必要且充分（该读法的锚 **IEV 831-01-01 Note 2 系二手派生表述**——42010:2022 正本未引 IEV、书目仅列 IEC Electropedia，它不是任何标准的准入判据条款）；目的定义=环境侧/为利益相关方/解决什么问题创造什么价值，来路三条（战略承接/业务或任务分析/关注点反推）+ 收口确认单，到位判据三（可作判据/可分解/可追溯）；先过目的关（是不是根本），再过代价关（值不值得现在定死）〔第 10、11、12 章〕

**T-139　不变量判据**（invariance test）　fundamental 的操作化——演化/重实现/实例替换中必须守住、否则系统沦为另一个系统的概念与属性；判别问句“这东西变了，系统还能达成它的目的吗”〔第 11 章〕

**T-140　目的漂移**（purpose drift）　目的悄悄变而架构未跟随（与架构漂移方向相反）；架构失效第一病因；目的变更 = 架构重审的强制触发器〔第 10、12、16 章〕

**T-141　系统目的或任务理解确认单**　架构设计的入场券、判据侧首类输入「目的与任务」的确认形态——原文引用 + 复述理解 + 业务方签字，纯理解零架构；兼为架构师「理解」职责的产品数据〔第 11、17 章〕

**T-142　被架对象**（entity of interest（EoI））　其架构被描述的实体——42010:2022 由 system of interest 扩展而来（企业/产品/流程/数据/应用皆可）；“一个被架对象，一个架构”，分析期可多候选、采纳后收敛一个；42020:2019 §3.6 给同一角色另立一名**架构实体**（architecture entity）＝**“被架构的那个事物”——不是“架构的构成物”**（中文圈的常见误读）；它与**系统元素**（系统的一个离散部分，可被实现以满足规定要求；15288:2023 §3.47 discrete part of a system that can be implemented to fulfil specified requirements）**是两个不同的概念**（15288:2023 §5.9 明写 two different notions：一个是被架构的，一个是构成系统的）〔第 10、12 章〕

**T-143　豁免**（dispensation）　有期限、有补救计划的受控偏离——治理的安全阀，没有豁免通道的治理会逼出隐瞒；豁免数量 = 治理框架的健康指标〔第 16 章〕

**T-144　同一性目的论**（purpose-based identity）　人造系统的同一性跟着目的走、不跟着物质走（15288 界定系统为 man-made）——忒修斯之船：木板全换仍是那条船（目的延续），木板不换拖进博物馆反而不是那条船（目的改变）；目的漂移=架构失效第一病因（见 T-140）；判据链「目的→同一性→不变量→fundamental→架构」第二环〔第 11、16、17 章〕

**T-145　不变量链**（invariant chain）　架构治理守护的对象——「目的→不变量→根本决策→遵从与演化」四层（架构决策/决策遵从/架构演化/架构资产）；治理管“该不该调、往哪调、什么不许变”，管理管“怎么调”（区别于 T-139 不变量判据：一为治理对象链、一为 fundamental 的操作化检验）〔第 16、17 章〕

**T-146　目的所有权**（purpose ownership）　系统目的与任务的定义权和问责在业务方（赞助人/产品管理）——架构师是目的的“理解与守护者”：对目的有质询权（澄清/可行性权衡反哺）与守护责（监控漂移、触发重审），没有改写权；目的变更由业务方行使，架构师负责触发重审（确认单=目的在场化的载体，见 T-141）〔第 16、17 章〕

**T-147　系统基本概念和基本属性清单**　fundamental 成员资格登记表——每条 = 定义 + 对目的的必要性论证（不变量判据判别问句的答案）+ 到目的/需求的追溯链；目的判据的裁决产物、评审第一核对单；清单自身基线化、条目增删走变更控制（命名取“清单”后缀=受控枚举）〔第 11、17 章〕

**T-148　操作目标清单**（operational objectives list）　系统操作目标的可判定集合（对象+时限+可衡量结果，第 8 章配方）——需求工程上游输入、评审通过标准的来源；目的落纸=确认单（归系统架构师）、目标落纸=操作目标清单（系统工程师需求侧编制·业务方确认）〔第 4、8、17 章〕

**T-149　判据锚点**（judgment anchor）　工程师的判断锚在规定需求上（对就是对、错就是错，验证参照明确）；架构师的判断锚在目的上——每天回答“什么才是 fundamental 的”（目的判据，见 T-138）；“六个更锋利的差异”之六（规定需求→目的）〔第 17 章〕

**T-150　四职责**　架构师对目的的权责——理解（把目的当 fundamental 判据吃透）/ 澄清（模糊时向上游回溯）/ 质疑（可行

性·成本·一致性检验反哺）/ 守护（监控目的漂移、目的变更触发架构重审）；有质询权无改写权，目的定义权在业务方（见 T-146）；角色分离 ≠ 人员分离〔第 18 章〕

**T-151　治理基线**（governance baseline（本书梳理主张））　按“谁批准、谁变更、变了意味什么”把“基线”再切一层的复合词——目的/架构原则/治理红线入治理级（治理层批准·一变=系统该不该存在的危机）、目标/成功判据入协议级（需供双方约定）、需求及以下入产品 CM 级（CCB 批准）；与功能／分配／产品基线（成熟度轴）正交（见 T-86）；非 ISO 术语（ISO 10007 只有“基线”）〔第 16 章〕

**T-152　架构视角**（architecture viewpoint）　创建、解读与使用架构视图、以框定一项或多项关注点的**一组惯例**（42010:2022 §3.8 正本逐字 set of conventions for the creation, interpretation and use of an architecture view to frame one or more concerns）——是**规则**不是产物，也不是“一种立场”（立场是利益相关方角度）；**正本该定义句无属概念**（正本凡需属概念必明写：§3.3 架构描述＝工作产品、§3.7 架构视图＝信息部件），故照正本写「一组惯例」、**不得替它补一个标准没写的属概念**；可版本化、可入视角库的依据是 §3.8 注 3 ——惯例记载在该视角的**规格**里（见 T-157）〔第 13 章〕

**T-153　架构视图**（architecture view）　组成架构描述的一部分的信息部件——是**产物**；每张视图恰有一个治理它的架构视角（1:1；**1:N 为 §5.2.7 规范句，1:1 见资料性附录 A.5**），一个视角可治理多张视图（1:N）；视图与架构之间隔着 represent（描述），与实现之间隔着 realize（实现）〔第 13 章〕

**T-154　视图组件**（view component）　视图的可分离构成部分；基于模型的视图组件由**模型种类**管辖，非模型的视图组件只由**图**

**例（legend）**记录惯例——“图”只是视图组件里最显眼的一种，清单、表格、决策记录片段同属视图组件〔第 13 章〕

**T-155　模型种类**（model kind）　以其**关键特性**和建模约定相区分的模型类别（42010:2022 **§3.15 正本逐字** category of model distinguished by its key characteristics and modelling conventions）；与架构视角在 42010 §4 中并列为独立的符合性对象，故一个模型种类可被多个视角复用——记法（模型种类）与看法（视角）由此解耦〔第 13 章〕

**T-156　架构描述框架**（architecture description framework（ADF））　在特定应用领域或利益相关方共同体内**已确立**的、用于描述架构的惯例、原则与实践（GERAM／RM-ODP／UAF／NAF 皆是）；一个 ADF 的核心内容就是一套预定义的架构视角库——标准故意不规定视角清单，缺领域 ADF 就自建一个（ADF 取代 2011 版的「架构框架」一词，以与 42030 的架构评估框架相区分）〔第 13 章〕

**T-157　视角规格**（viewpoint specification）　记录一个架构视角之惯例的**信息部件**（42010:2022 **§3.16 正本逐字** information part (3.14) that identifies, in a complete, precise and verifiable manner, the requirements, design, behaviour, or other expected characteristics of an entity；〔SOURCE: ISO/IEC/IEEE 15289:2019, 3.1.26〕），须写明**七项（正本 §8.1 a)–g) 七项 shall + 第二段一句 should（视图方法）**）：a) 与本视角相关的利益相关方角度／b) 由本视角框定的关切／c) 与这些关切相关的方面体／d) 持有这些关切的已知典型利益相关方／e) 构造视图时使用的模型种类与图例／f) 捕获视图内及视图组件之间关系的**对应方法**／g) 对本视角任何信息来源的引用；另有一句“宜”：标识**视图方法**（细则见 §8.3）。**本书归并的末一行为“必需的模型·元素、关系、约束与属性”**（其内容出自附录 B.2.9.2），标准没有这一栏、**不删行**；规

格不全的视角，产出的东西不构成架构视图；视角的惯例正记载在它自己的规格里，故视角可版本化、可入视角库（§3.8 注 3，见 T-152）〔第 13 章〕

**T-158　架构评估**（architecture evaluation）　对照架构的**预期目的（intended purpose）**评估其质量与价值（ISO/IEC/IEEE 42030）；三层＝评估综合／价值评估／架构分析，两个工作产品＝评估计划／评估报告；**与架构评审（第 8 章，尺子=适宜·充分·有效）不是一回事**；评描述适宜性归 42020 §10.4.6〔第 16、18 章〕

**T-159　关切台账**（concern register（本书形式化））　把利益相关方关切逐条登记的台账：每条须给出来源（谁在乎）、后果（可度量的期望值与可接受区间）、分流类型（承重型／约束型／偏好型／交付型）与出路——**悬空不是选项**；举证三件套不齐则“退回补件”，不是驳回〔第 11 章〕

**T-160　四级映射表**（four-level mapping table（本书形式化））判据链的对账承重件：关切点 → 架构目标 → 关键成功准则 → **基本条目** → 视图落点 → 依据决策（ADR）；三条规则——每行必须打通、每**个基本条目**至少一行、每行至少一条 ADR〔第 13 章〕

**T-161　判据与门槛分家**　判“算不算治理”看三**条判据**（层级／对象／不可自裁，逐段定性）；判“该不该上治理机构”看一**个门槛**（跨出单管理层权限，这是升级的理由）——**判据管分类，理由管升级**，把“跨部门”当判据会推出“不跨层的架构治理不算治理”的错结论（本书形式化，对齐 ISO/IEC/IEEE 42020 §6 与 ISO 37000 §4.1）〔第 16 章〕

**T-162　治理系统的七类元素**　主体（谁）/ 意图（为了什么）/ 规矩（依据什么）/ 结构（怎么搭）/ 关系（怎么连：授权→报告→担责）/ 行为（怎么动）/ 保证（怎么确信）——**由 ISO 37000 内容推**

**导，非条款原文**；原则不是要素、结果也不是要素；“保证”（使自己确信治理系统设计得当并按预期运行）最常被整个跳过〔第 16 章〕

**T-163　担责三名字**（accountability）　**担责**＝术语位（那项义务：对他人说明并解释责任履行情况的义务，ISO 37000 §3.2.2）／**问责**＝动词位（治理机构行使的那项动作）／**述责**＝动作层（把账摆出来）——三者不混用；中文没有一个天然能与“责任”切开的单词，分界只能靠定义句与首次括注承载〔第 16 章〕

**T-164　设计定义**（Design Definition（ISO/IEC/IEEE 15288:2023 §6.4.5））　与**架构定义**并列的**过程名**（非同一工作的粗细两版）：目的是“提供关于系统和其元素的足够详细的数据和信息，以便根据系统需求和架构实现解决方案”（§6.4.5.1 Purpose 句）；作用面是“整体系统设计＋细节充分”；其产物＝系统设计描述。三条易混口径：**design 双义**（宽义见 ISO/TS 14812 §3.1.15.1，含 define the architecture；窄义＝架构之外这一段）；**“合规实现”的一手出处是 15288:2023 §3.13**（design, noun: specification of system elements (3.47) and their relationships, that is sufficiently complete to support a compliant implementation of the architecture (3.5)——主语是信息／规格，关系词是 support，承担者在 implementation 侧；24765:2017／14812 §3.1.15.1 降为**术语学旁证**）；**§5.9 给出两过程的分工对句**——架构管适宜性·存续性·合意性，设计管与技术及其他设计元素的兼容性、构建与集成的可行性；且**架构实体与系统元素是两个不同的概念**（§5.9 two different notions，见 T-142）〔第 14 章〕

**T-165　系统设计描述**（system design description）　设计定义的工作产品：“对系统或元素的设计的描述”（据 ISO/IEC/IEEE 24765:2017；SEH 中文作“系统设计**说明**”，同物异译）；涵盖设计的组成、属性和特性，用**方法·框架·图示形式·符号·其他模型形式**表达；

出口＝**开发为规范，用于采购或实现——停在〈待定〉即未过出口**（该出口的正本位＝**15288:2023 §6.4.5.3 b) 8) Capture the design**＋**注 12**：…in a form that can be used to either procure or realise the system elements that comprise the system；“开发为规范”是中译加工）；与**系统元素说明**分层配对（后者＝将系统架构描述应用于低级别配置项与元素，详细程度足以支持设计、实现和测试）〔第 14 章〕

**T-166　设计描述符**（design descriptor）　**出处订正**：INCOSE SEH 5.0（**同版同年旁证**）；正本 15288:2023／42010:2022／42020:2019 检索 `descriptor`。原文续：一组（1）**设计特性**和（2）**其可能的值**；由自上而下的**分配与配置**与自下而上的**选择与测量**相结合确定。“将系统元素及其特定的设计描述符与**整体系统的设计描述符相匹配**是系统设计的关键部分”——完备这条判据最终落在描述符上〔第 14 章〕

**T-167　原子元素**（atomic system element）　递归的**换手点判据**（两条同时满足）：**只需要用黑盒（不透明箱）表示法来描述它的需求**、且**能够自信而明确地定义其在现实世界中的解决方案**——此时可在不进一步详细说明的情况下做出**购买／制造／重用**决策。**性质是信息充分性判据，不是深度／层级判据**；7±2 是可管理性规则，不是停止规则。**出处说明**：该两条件的中文逐字取 **INCOSE SEH 5.0 p33**（同版同年旁证）；正本**邻近句＝§5.2.2**（生存周期过程**递归**应用于所关注系统，直到结构分解到“可理解且可管理的系统元素可被实现（制造、购买或复用）”的程度）〔第 14 章〕

**T-168　架构落点　本书工作名**：指**架构“可以关乎”的对象**——实体侧的 components 与 relationships（42020:2019 §3.3 Note 1，**注释级、无术语地位**），由架构视图经 represent（描述）指代、再由实现经 realize（实现）兑现为**实现层的系统元素**；用来把**实体侧／**

**实现侧**两界分开（视图组件在描述层、系统元素在实现层，两者**同形不同界**）。★ 与标准的**架构实体**（见 T-142）**不是一回事**〔第 10、12 章〕

**T-169　坐标系（位次轴 × 深度轴）　本书工作名**：给一个概念定位的两根正交的轴——**位次轴**＝概念在知识网络里站在哪（一般概念 ⊃ 核心概念 ⊃ 根概念，§1.2）；**深度轴·能力四级**＝同一种能力练到第几级（每级按作业对象命名：① 表征 ⊃ ② 图式 ⊃ ③ 心智模型 ⊃ ④ 解释框架，§1.4.4）。两轴正交，报坐标须两根一起报（§1.4.5）〔第 1 章〕

**T-170　价值律（1 和 0）　本书工作名**：两轴作业的权重律——**表征是 1，图式／心智模型／解释框架是 0；1 定成立，0 定量级**（“1 和 0”说的是成不成立与量级，不是分数）。它与方向同属一条乘法律的两个归零项（方向管往哪使劲，表征管拿的是不是那个东西）；定义位＝§1.4.5〔第 1 章〕

**T-171　乘法律**（multiplicative law）　**本书工作名**：学习产出是四个部件**相乘**——方向 × 力臂 × 施力 × 支点（§1.4.1）：方向（根概念能力）／力臂（概念体系能力）／施力（学习投入）／支点（实践场景）。相乘的后果：**任一部件取零，整条式子都是零**；其中**一错就让别的力气全白费的，只有方向与表征这两根轴**——力臂·施力·支点只负责放大（§1.4.1、§1.4.5）〔第 1 章〕

**T-172　归零项**（zeroing factor）　**本书工作名**：乘法律里**一错就让别的力气全白费**的那两项——**方向**（先抓错了词，§1.4.3）与**表征**（话说得不准，§1.4.5）。它们是**两根轴**（先抓哪个词／说到多准），**不是四部件里挑出的两个**；两项都过，才轮到力臂与投入去放大〔第 1 章〕

**T-173　软件工程**（software engineering）　**把工程应用于软件**（IEEE 610.12-1990：“the application of a systematic,

disciplined, quantifiable approach to the development, operation, and maintenance of software”）——它不是一个与“工程”并列的东西，而是**工程在软件这一载体上的分科**；别拿它当“工程”本身（§2.2.3 术语注）。**一处常被混用**：敏捷 + CI/CD + 测试只落在方法论与工具与实践上，不构成“在做软件工程”（§2.3.3）〔第 2 章〕

**T-174　约束**（constraint）　工程那句定义里的一样（“框”）——成本、工期、资源、风险、法规、物理极限；**不是工程的附加项，而是定义的一部分**（1941 ECPD 定义里即有“运行经济性”与“生命财产安全”两条）。判据：**它得是一条有人守的规则**——只报数据不报约束，等于没有约束。反面：无意义的限制与纯官僚式流程不产生判断价值，不算约束（§2.2.2）〔第 2 章〕

**T-175　系统方法**（systematic approach）　工程那句定义里的一样（“路”）——需求 → 设计 → 实现 → 验证 → 维护，保证工作**可计划、可度量、可复现**。它是定义里“挪得走”的那一半：可以写成文档、做成流程、交给工具，让别人接住；判断不在此列（§2.2.2、§2.3.4）〔第 2 章〕

**T-176　能力四级　本书工作名**：同一种能力的四种粗细——① 表征（现实事物在头脑中的代用品，最小单位＝概念、名词、对象、关系和边界）⊃ ② 图式（表征聚起来、认得出的一类结构）⊃ ③ 心智模型（会动的图式）⊃ ④ 解释框架（对一整个领域的整体看法）；每章末「本章的能力级」按此四级清点（§1.4.4；底本＝R-97 第 41 讲）〔第 1 章〕

**T-177　图式**（schema）　**能力四级第二级**：表征聚起来、认得出的一类结构——一眼能认出的套路；判据：看事情一块一块看，一

听就知道这是哪一出，还能预判下一步（§1.4.4）。第 3 章章题的“图式”即此义：**三个词是表征，聚起来的那条路径才是图式**〔第 1/3 章〕

**T-178　判据主义　本书工作名**：不给没有判据的定义、不给没有失效边界的工具、不给没有反例的概念——判据要配“违反即崩”的画面，让失效边界可辨认（序言列为全书底色；**命名位＝§1.8.3**，正式定义＝方法论的底座 §一“底色”与 §五 规则 3“判据主义”）〔第 1 章〕

**T-179　定需求／出规格／判满足　本书工作名**：需求、设计开发、质量三者在产品开发这件事上的位置——**定需求**＝定义“要什么”（设计开发的输入、质量判定的依据）／**出规格**＝逐级出规格，直到可实现——把需求转化为更详细的需求，一级出一份规格，直到需求足以支撑客体实现与验证（规格＝规定需求的集合）／**判满足**＝认定客体固有特性满足需求的程度（§3.1.3、§3.4.4）〔第 3 章〕

**T-180　架构治理的制度面与动作面**（the governance / governing）　架构治理的两面：**制度面**是那套常设的东西（机构与职权、指令与原则、程序、论坛、登记载体、度量），标准里落在 **42020:2019 §6.4.1 d)~k)** 与附录 G.1；**动作面**是实际发生的事（划范围、配权、立规矩、监视、评定、决定、传达、查符合、处置偏离、纠偏自身），落在 **§6.4.1~§6.4.6 六项活动与 49 条任务**。两面缺一的表现不同——**有制度无动作**＝橡皮图章（委员会照开、台账照建，没有人真的作出过决定）；**有动作无制度**＝某人临时拍了一次板，下次不保证还有（本书读法）〔第 16 章〕

**T-181　配置控制管理机制**（configuration control management mechanism）　治理用来管控**架构集合中的变更**的机制（**ISO/IEC/IEEE 42020:2019 §6.4.2 k) 正本逐字** Establish configuration control management mechanism to govern the

changes in the architecture collection.）——它是“架构描述改了要不要重新认定、认定过的东西还算不算数”这条链的制度落点；**与配置管理（T-87）不同层**：T-87 是那五项活动（准备／配置标识／更改管理／配置状态记帐／验证与审核），本条是治理要求建立的那道**管控机制**；第 15 章只把基线候选物挑出来递过去，**批不批、谁批由治理这一侧定**——“谁有权批这一版”与“谁有权建立这道机制、谁裁定架构集合中的变更”是同一个问题（42020:2019 §6.4.2 k)），本书在第 16 章讲（§15.4.5）〔第 15 章〕

**T-182　可转化／可实现　本书工作名**：设计开发起点与终点的两条判据——起点“**可转化**”：完整、无歧义、不冲突、隐含需求已识别；终点“**可实现**”：需求被细化到足以支撑客体实现、生产和服务提供可直接执行的程度，操作化为下游各方案配齐（技术、物料采购、制造、测试、交付、部署、操作、运维，可按组织下游环节增删）（§3.3.2、§3.3.5）〔第 3 章〕

**T-183　科学原理 × 判断创造**（scientific principles × judgement-based creation）　工程那句定义里的“料”——**科学原理**回答“为什么这样能行”（应用原理，不是发现原理）；**判断创造**是把原理用活的那个动作（选哪个模型、怎么取舍、何时偏离标准做法），**不单独占位却贯穿每一步**。**本书工作名**，取 ABET 1979 定义 applied with judgment 与 ECPD 1941 定义 creative 两处合成（§2.2.1、§2.2.2①）；与 §2.3.3 第四层守的“**底线**”分开放——**判断是每一步的动作，底线是判断不可越过的界限**〔第 2 章〕

**T-184　工程化**（engineering process characteristics）　“工程做起来是什么样”——**本书工作名**，指工程的过程特征，落实为有结构、有方法、可重复、全覆盖、受控、可验证；**它不是“工程”的定义**：直接出处是 IEEE 610.12-1990 对**软件工程**的定义（systematic / disciplined / quantifiable 三个形容词的展开），严谨写法＝“据

IEEE 610.12 对软件工程的定义，工程化的过程表现为……”（§2.3.4，引法见 R-14）；**再往上还有两级**：**再固化是流程化、系统承载是信息化——流程化 ≠ 工程化，有流程不等于有工程**〔第 2 章〕

**T-185　开发欠账 / 管理欠账　本书工作名**：需求乱的两笔账。**开发欠账**出在源头——没调研（来源不清）、没分析（话没说清）、没写可验证的规格；**管理欠账**出在过程——没基线、没变更流程、没追溯。判法：先问是“当初没说清”还是“定了之后没人管”，再开药；**顺序不能反**——管理再严，也只是把没说清的东西看住（§4.2.2、§4.3.2）〔第 4 章〕

**T-186　可采性（正文作“可采”）　本书工作名**：一条规定需求要**当依据用**，除“明示”（资格）之外还得满足的**四个条件——相关性**（能回溯到目的，过必要性判据）／**可检验性**（已明示且自带判定方法与准则）／**时效性**（属已冻结的基线版本）／**同一性**（有唯一标识、被唯一引用）。前置一步是**可判定**（ID、对象与适用范围、断言、判定方法——“可判定四件套”）；**明示是资格、可采才作数**：口头明示仍是规定需求，但缺判定方法、不可复现、无版本，所以当不了验证的参照（§4.2.5，§4.3.1，四条件为本书工作名）〔第 4 章〕

**T-187　术语锁定三问**（term-lock three questions）　锁住一个词的身份的三问：① 这个词出自哪部标准的哪个条款？② 那部标准的术语章里有没有定义它？③ 没定义的话，它是按通用语意用的吗（§1.8.2）〔第 1 章〕

**T-188　自动化**（automation）　**本书工作名**：把执行交给机器、判断留给人——自动化不是减少判断，而是**把判断提纯**，同时**拿走人阻止事故的能力**。分野：工程要的是“错误可见”，自动化追求的是“零错误”（§2.3.4）〔第 2 章〕

**T-189　目的**（purpose）　**本书工作名**：系统为什么被研制的理由——落在价值侧、可以不量化，是目标与需求的来源，在最上游；确立过程对 ISO/IEC/IEEE 15288:2023 §6.4.1 业务或任务分析（任务与目的在那里定义），**标准未给它术语级定义**（§3.2.3）〔第 3 章〕

**T-190　需求池　本书工作名**：需求的统一入口——**无来源需求不得入池**；分析类文档在这里聚成可排序、可追溯的条目（素材类留原始记录，见 §4.4.5 文档体系表），是骨架五份之一（§4.2.3、§4.4.5）〔第 4 章〕

**T-191　需求校核（评审）**（review）　**本书工作名**：需求开发的第四个活动——对需求这份**工作产品**判断适宜、充分、有效；**需求文档做不了确认**（确认须在使用中兑现），但**评审与验证都做得了**（验证它满足文档自身应满足的规定需求；§4.2.3、§4.3.3；ISO 9000:2026 §3.11.2、15288:2023 §6.4.9）〔第 4 章〕

**T-192　工作产品**（work product）　ISO/IEC/IEEE 15288:2023 用它**统称过程产出**（表 B.1：一组可能的工作产品，包括人工制品与信息部件）；**人工制品是其中一类**——§3.6 把人工制品定义为“在项目期间产生并用于捕获和传递信息的工作产品”（**该标准无 work product 独立词条**）。**需求文档是工作产品**；**人工制品（含需求文档、架构描述、设计描述）在验证的射程内**（15288:2023 §6.4.9 的验证对象含 artefact），而**确认须在使用中兑现**（§4.2.3、§4.3.3、§8.3.6）〔第 4、8〕

**T-193　客观证据**（objective evidence）　验证与确认定义里的要件——**通过提供客观证据**，证实规定需求或特定预期用途的需求已得到满足；测试只是提供证据的手段之一（ISO 9000:2026 §3.8.6；§4.2.5、§4.3.3、§4.4.4）〔第 4、8〕

**T-195　指标**（indicator）　**需求＝描述＋指标，缺指标不成立**（第 5 章 §5.2.6）；治理侧另有“治理框架的健康指标”（第 16 章），两处不同物〔第 5 章〕

**T-194　三类内容（底本作“三类需求”）**（three types of requirements）　**本书工作名**：一份产品需求要“全”所覆盖的三类内容——**功能需求**（T-47）、**环境适应性需求**（T-48）、**可实现性需求**（T-49）；判据是“谁在问”（客户、环境、承制方），每类**及其指标**、每类各配一个**专属验证载体**（T-50）。**它是本书对军品与工程界“完整需求”实践的凝练与命名，不是任何标准的原文**（一手来源＝20250326 正向设计 §7.2；§5.2.1 术语注）〔第 5 章〕

**T-208　架构理由（理据）**（architecture rationale）　记录关于架构决策的**解释、论证或推理**（42010:2022 §5.2.12）；与决策**并排**、同为架构描述元素（§5.2.9）。可含六项：做出决策的依据／对质量属性的影响／所考虑的备选方案与权衡／潜在后果／架构原则／附加信息来源的引用。**本书行文简称“理据”**〔第 12 章〕

**T-209　权衡空间**（tradespace）　与满足架构目标与利益相关方关注点相关的**参数、性质与特征的范围与程度**（42020:2019 §8.4.6 注 2）；其构成＝权衡考虑因素的集合连同各解之间的影响与相互作用（同节注 5）。**标准未给“权衡”（trade-off）定义句**——四份正本第 3 章零词条，故本书讲“权衡”时一律指到 tradespace 上〔第 12 章〕

**T-210　强制项（强制项与要求）**（mandates and imperatives）　从需求那一侧来的**给定**：相关的方针、法规与标准（42020:2019 §8.3 f)），评估活动第一件事就是把它识别出来（§9.4.3 a)）。**是给定不是选项**——候选不满足它，不是“差一截”，是要先把“给定”改掉（减免 §9.4.8 d) 注 4／豁免与改需求 §9.4.10 f) 注）。**“留得下”与“选得中”是两件事**（§9.4.7 c)、§9.4.8 d) 注 5）〔第 12 章〕

**T-211　影响半径／信息成本／修改风险**（impact radius / information cost / risk）　**本书凝练**（四份正本无对应词条）：量一条决策不可逆程度的三样——影响半径（结构／时间／组织耦合）、信息成本（知识深度／隐含上下文）、修改风险（关键路径交叉／是否动已存在的数据）。**只作排序与时机，不作准入判据**；准入看目的必要性〔第 12 章〕

**T-212　决策权属方**（decision authority or owner）42010:2022 §6.10.1 十一项内容之一——**“决策权属方或决策责任人的标识”**。记录不产生授权：谁有权定要单独写（42020:2019 §6.2 c) 由适当权威作出并传达；§6.4.4 e) 其余决策上报企业决策论坛）〔第 12 章〕

**T-213　未决议题**（open issue）　42010:2022 §6.7 d) 注 2：Open issues can lead to decisions to be made.（中译：未决问题可以导致待作的决策。）**本书用法**：把“暂不决定”的合法去处记成未决议题——它**不进决策记录、不占决策位置**，要写明现在为什么不做、等什么信息、何时再评估〔第 12 章〕

**T-214　同解**（same reading（**本书立名**））　**本书立名，正本无对应词条**：一份架构描述的价值在于把“怎么读”从各人脑子里写进公开规则，让**不同的人、在不同的时间读出同一个意思**（机理＝42010:2022 §3.8 的“一组惯例”句与 §3.16／§8.1 的约定句：视角的规格就是为创建、解释、呈现和分析视图而确立的那套约定）。**边界**：它管**读法**能不能对齐，不管内容对不对（内容对不对比的是目的，见第 11、12 章）；“使架构可判定”是它的**结果**，不是价值本身，“分面加同一”说的是组织方式、也不是价值。**依据**＝对四套蓝本与 42010 的逐条对读〔第 13 章〕

**T-215　可交付物**（deliverable）　**过程产出里被交出去的那一份**——正本用词见 42020:2019 §6.4.2 任务句 architecture deliverables（架构可交付物）；与工作产品（T-192）的分界是**交不交**：工作产品统称过程产出、含大量不交付的中间产物，可交付物只指其中交出去的那一份（30 号报告即以此两词相区别）。**本书用法**：架构定义交出去的那一份是**架构描述**（第 13 章题面“把架构定义交出去”取此用法；架构本身无形，交不出去）〔第 13 章〕

**T-216　系统分解结构**（system breakdown structure（SBS））　标准把“建立适当的分解结构”写进**项目规划过程**的任务（15288:2023 §6.3.1.3 任务 4：“Establish appropriate breakdown structures.”），其注 4 把四种典型分解结构并列——工作分解结构、功能分解结构、**系统分解结构**、组织分解结构。三条口径：① **户口在项目规划过程**，不在架构过程——它是一份**规划台账**（组织工作、分派责任、管理进度），**不是架构的产物，也不是架构描述的一部分**；② **系统结构本身不必是层级**（§5.2）——分解结构长成一棵树是“台账要好查”这个用途逼出来的，把“结构是树”当成系统的性质，会看漏横切、共享、循环的关系；③ **“产品分解结构”（product breakdown structure，PBS）不是标准术语，本书不用**——讲分解结构的那条注里也没有它；遇到这个词一律改说 SBS，或直接说“哪一份台账”。与**物理视图**的关系：两者可以互相参照，但不是同一份东西——一份是架构的表达，一份是规划的台账（§14.4.6）〔第 14 章〕

**T-217　架构基线**（architecture baseline（**本书用法**））　正本无独立词条，取“基线架构”那一角色的实指：**架构描述被选定并冻结的那一版**——被冻结的是描述，不是架构本身（架构无形而抽象，能被选定和冻结的永远是承载它的那份东西）；跟着一起进去的还有那份描述所依据的记录（架构描述与架构决策记录一起纳入配置管理）。正本对它那句角色话是“**充当同步关键点，用于架构的评审以及将**

**架构作为参考予以提供**”（角色依据＝ISO/IEC/IEEE 42020:2019 附录 E.4.1.6；“选定并冻结”的资格依据＝ISO/IEC/IEEE 15288:2023 §3.8，见 T-85）。两个用途正好对着“设计依从的是哪一版”这一问：**设计依从的是那一个同步点上的那一版**——设计手里的版本比基线新，可能在依从一个还没被批准的改动；比基线旧，在依从一个已经作废的版本（两种错都不在内容上，而在资格上）。**用法**：评审写清“按哪一版评的”、引用写清“引的是哪一版”——纪要上只写“架构评审通过”，等于把一个同步关键点丢掉了（§15.4.7）〔第 15 章〕

**T-218　正式化**（formalisation）　**本书承 41 号报告 §6.6 命名**：把一个时点上的一组内容变成“算数的那一份”的那个动作——正本句是“基线的内容通过技术过程开发，但在某一时点经由配置管理过程予以正式化”（The content for the baselines is developed through the technical processes, but is formalised at a point in time through the configuration management process.；中译：基线的内容通过技术过程开发，但在某一时点经由配置管理过程予以正式化。15288:2023 §6.3.5.3 b) 4) 注 8），一次含三件事：**获批、冻结、作参照。要点是它不在技术过程里**——技术过程开发内容，正式化是技术过程与配置管理之间的那道交接动作，所以“架构定稿了”与“架构基线建立了”不是同一件事。**宾语可以不同**：对架构描述正式化得到架构基线（T-217），对需求规格书正式化得到需求基线，对治理指令正式化得到治理那一层的基线。**与评审的界**：评审评的是内容够不够好，正式化管的是这一份东西的资格；评审合格是正式化的常见前提，但不等于正式化——一份评审合格的草稿仍然是草稿（§15.4.1）〔第 15 章〕

**T-219　配置状态记帐**（configuration status accounting）　记录并报告配置信息、所提议更改的状态、已批准更改的实施状态（ISO/IEC/IEEE 15288:2023 §6.3.5.3 d）；该句逐字引自 ISO 9000:2026

§3.10.11）——**五项活动里的第四项**（T-87），让“现在是什么状态、曾经是什么状态”随时查得到；**不是给人看的台账，是给追溯用的底账**：出了问题要精确召回哪几项，答案就在这一项里，这一项没做就只剩全量召回（§15.4.3、§15.4.8）。**译名**：同一件事 15288:2023 中译作“配置状态记帐”、ISO 9000:2026 中译作“配置状态纪实”，第 15 章随 15288 用“记帐”；GB/T 19000 系把 configuration 一族通译作“技术状态”。**一条相邻边界**：记录必须比产品存续得更久（§6.3.5.3 d) 注 18）——存档的寿命通常跟着项目走，配置记录要跨过项目〔第 15 章〕

**T-220　基线候选物**（baseline candidate（**本书工作名**））　技术过程这一侧交给配置管理的那一份——正本句是“识别基线的候选对象，随后把人工制品提供给配置管理”（ISO/IEC/IEEE 15288:2023 §6.4.4.3 e) 6) 与注 12；设计定义过程照同一句式，见 §6.4.5.3 注 20），十四个技术过程每一个都“提供已选定用于基线的关键人工制品”（Provide key artefacts that have been selected for baselines；中译：提供已选定用于基线的关键人工制品）。对着本书讲过的那些东西：架构这一侧是架构描述本身（含各张视图与模型）、架构决策记录与它的理据、各份清单（关切台账、视角清单、对账表这一类）；设计那一侧是系统设计描述、设计侧的决策与理由、设计特性的取值与依据。**挑候选与定基线是两件事、分属两只手**：够不够格当候选由技术过程判断（不够格的留在过程里继续改），收下并正式化的是配置管理；这一递不是“交出去存档”，它是进入受控范围的第一步（§15.2.2、§15.4.5）。依据＝41 号报告 §四、39 号报告〔第 15 章〕

**T-221　架构管理**（architecture management）　**过程名**：实施架构治理指令，及时、高效和有效地实现架构集合目标（ISO/IEC/IEEE 42020:2019 §7.1）；活动六项（§7.4 a)~f)）；它站在治理与核心过程之间——往上接治理的指令与指导，往下给架构概念化、评

估、细化下指示；**“架构定义产出判断、架构管理执行指令、架构治理给效力”是 41 号报告的三段**，与正本一句不改〔第 16 章〕

**T-222　架构治理计划**（architecture governance plan）　正本 §6.5 明列“应产生”的四项工作产品之一（The following work products shall be produced）；本书把治理的“制度系统”归为五样，它是其中一份骨架性产物（治理工作要覆盖哪些架构、需要什么资源与节奏）〔第 16 章〕

**T-223　架构治理指令与指导**（architecture governance directives and guidance）　正本 §6.5 工作产品之一；来源是 §6.4.1 f)“建立指导原则、策略与指令”，须**由具有适当职权者作出并传达给利益相关方**（§6.2 c)）；**正本中译另处作“指南”，与 guidance 同词——本书统一写“指导”**〔第 16 章〕

**T-224　架构治理符合性状态报告**（architecture governance compliance status report）　正本 §6.5 工作产品之一；把“有没有照办”变成一份可看的记录，使符合性**可被评定、因而可审计**（§6.4.5 监视并评定对治理指令与指导的符合性；§6.2）——它是“给出指导—评定符合性—对结果问责”三抓手里第二样的产物〔第 16 章〕

**T-225　架构集合目标**（architecture collection objectives）　正本 §6.5 工作产品之一，也是 §6.4.3“确立架构集合目标”的产物；治理盯着的那把尺（这一批架构要达成什么、达成到什么水平）；§6.2 a) 要求它满足企业及其客户、供应商当前与预期的业务需求〔第 16 章〕

**T-226　架构判断的效力**（—（**本书读法**））　一条架构判断“**算不算数**”的那一维——**正本里没有这个词**，它讲“有效性”用的是 effectiveness；本书用它把 41 号“判断—正式化—效力”三段的**第三段立名**（ch15＝正式化／ch16＝效力）；**由治理给出，不由算的人**

**自己定**；判据四条＝谁批的／按什么程序／改它要走哪条路／依据是哪一条〔第 16 章〕

**T-230　架构评估报告（含分析结果）**（architecture evaluation report）　**本书正文用作产品数据清单的一件**：评估工作的可交付件之一，把评估结论连同支撑它的分析结果写下来（ISO/IEC/IEEE 42020:2019 §9.5）〔第 18 章〕

**T-231　价值评定结果**（value assessment result）　**本书正文用作产品数据清单的一件**：评估三层里“价值评估”的产物（ISO/IEC/IEEE 42020:2019 §9.5）〔第 18 章〕

**T-232　质量模型**（quality model）　**本书正文用作产品数据清单的一件**：正本明列的工作产品（ISO/IEC/IEEE 42020:2019 §8.5）〔第 18 章〕

**T-233　问题空间定义报告**（problem space definition report）　**本书正文用作产品数据清单的一件**：正本明列的工作产品（ISO/IEC/IEEE 42020:2019 §8.5）〔第 18 章〕

**T-234　符合性正式声明**（formal statement of compliance）**本书正文用作产品数据清单的一件**：证实架构的正确实施，有时被表述为架构师向客户或用户作出的正式声明，即所构建的系统满足可投入使用准则（ISO/IEC/IEEE 42020:2019 §3.2 注 2）〔第 18 章〕

**T-235　未处理项与不符合项声明（含理由）**（statement of untreated items and non-conformances）　**本书正文用作产品数据清单的一件**：把没处理的项与不符合的项连同理由一并写明（ISO/IEC/IEEE 42010:2022 §6.2、§6.7 d) 及注 2）；它与**基线候选物**（T-220，ISO/IEC/IEEE 15288:2023 §6.4.4.3 e) 6)）同属 18.4.6 那一格，后者附录 A 已有户〔第 18 章〕

**T-236　评审问题的处置记录**（record of disposition of review issues）　**本书正文用作产品数据清单的一件**，不在六格之内的第 7 件：把评审提出的问题怎么处置记下来（ISO/IEC/IEEE 42020:2019 §9.4.10 h)）〔第 18 章〕

**T-237　模块架构师**（module architect（**本书立名**））　模块这一层“架构自身的成立与守持”的专责角色——通常即模块工程师自己或该模块的团队；标准只认架构活动（ISO/IEC/IEEE 42010:2022 §3.1），不认此头衔；与系统架构师的分界＝管哪一层（模块自身的架构 vs 系统级的架构）。**本书立名**：模块工程师＋模块级迷你架构师〔第 18 章、后记〕

**T-238　架构权衡报告**（trade study report（**本书工作名**））　权衡空间与候选×判据矩阵的记录件——标准未给“报告”名：ISO/IEC/IEEE 42020:2019 §8.4.6 给权衡空间（T-209）；本书把“候选×判据＋理由”那一份记录叫架构权衡报告（ch18 §18.4.6 与后记系统架构师清单用）〔第 18 章、后记〕

> 尺子＝本书既用的量度用语（判据／量具之意），不单独立条

---

## 附录 B 概念档案集：33 张完整九格

> 全书核心档案约 45 张。下面 33 张是最常被查、最容易误解的，给出**完整九格**（字典式查询）——前六张为第 1 章学习方法论档案（概念/核心概念/根概念/基本原理/概念体系能力/根概念能力），后二十七张为工程档案。其余 12 张（工程系统、需求工程、运行概念 ConOps、框架定义、详细定义、分配生成、分

> 析迭代、分解分析、架构描述、不可逆性、版本、管理）的九格完整版散见各章“概念卡片”，一句话定义见附录 A。用法同第 1 章：薄格可省，不可跳过——本表每个概念都填满了能填的格；查的时候，从①定义开始，到⑧误区为止。

**怎么读这 33 张卡：九格 = 四问的展开**（四问法见第 1 章；格号与栏目见附录 A T-11／T-12）：

- **它是什么**＝① 定义（定界：一句话钉它是谁）
- **它不是什么**＝⑥ 对立 · ⑦ 辨析（定界：远界对立面 + 近界辨析）
- **它从哪来**＝② 背景 · ③ 历史 · ⑨ 英文（溯源：诞生处境 + 演化 + 词源）
- **它往哪去**＝④ 问题 · ⑤ 场景 · ⑧ 误区（致用 + 纠误：为什么造它、什么时候用、最常见的误解）

B 区保持字典式九格顺序（①⑥⑦⑨ → ②③ → ④⑤ → ⑧），不另作四问化改写；薄格可省，不可跳过。

概念档案 • 概念

① 定义　对一类对象共同特征的抽象概括；严谨定义（ISO 1087:2019 §3.2.7）：由特性的独特组合而形成的知识单元（可指称 + 可界定）

⑥ 对立　个别（感知 percept / 实例 individual）——概念是"一般"，把无限具体压缩成可沟通的有限单元

⑦ 辨析　术语 term（指称概念的语言符号）vs 概念（被指称的知识单元）；定义 definition vs 概念本身

⑨ 英文　concept ← 拉丁 conceptus（concipere：con-"一起" + capere"抓取"，本义"被把握住的东西"）

② 背景　人需要把无限具体经验压缩成可沟通的有限符号单元——概念是压缩的产物；术语学由此成为学科

③ 历史　亚里士多德范畴论 → 逻辑学/认识论 → 术语学标准（ISO 1087）→ 认知科学（原型理论 Rosch）→ 本方法论（概念=网络节点）

④ 问题　没有概念之前，人只能用具体事例交流（"那个红的圆的"），无法抽象沟通——如何用最少符号承载最多经验？

⑤ 场景　一切思维与沟通的前提；需精确指称个体时改用专名

⑧ 误区　✗ "概念=词语"——词语是符号，概念是意义；✗ "概念=事物"——是对事物的抽象，不是事物本身；✗ "概念有绝对正误"——只有共识强弱（燃素曾是共识）

判据　可指称（一个词/短语）+ 可界定（内涵与外延）；依据 ISO 1087:2019 §3.2.7

概念档案 • 核心概念

① 定义　概念网络中处于枢纽地位的概念——解释领域内其他概念都要用到它，理解它自己只需很少前置知识

⑥ 对立　边缘概念（叶子 leaf）——处于网络末端、被枢纽解释、不解释别的东西（维度：网络地位）

⑦ 辨析　基本概念（入门次序，偏教学先后）vs 核心概念（结构地位，偏被依赖程度）；关键概念（重要但未必枢纽）vs 核心概念（强调"被依赖"）

⑨ 英文　core concept / hub concept；hub 原义"轮毂"——辐条都连到它；图论无标度网络的枢纽节点

② 背景　压缩学习实验发现无标度网（少数枢纽占据大量连接）最易学——识别枢纽成为学习的第一步

③ 历史　图论 hub（Barabási 无标度网络）→ 认知科学压缩学习（北大 2026）→ 学习方法论"先立枢纽" → 本方法论

④ 问题　没有核心概念识别之前，学习不知道从哪下手（信息过载、细节淹没）——领域知识的四梁八柱是什么？

⑤ 场景　学习新领域的第一步（20 分钟立枢纽）、课程设计、教材编写（先枢纽后细节）；领域已有共识框架时直接采用

⑧ 误区　✗ "核心概念=看起来最重要的概念"——是"被依赖最多"，不是"看起来重要"；✗ "一个领域只有一个核心概念"——通常 3~5 个构成骨架；✗ "核心概念固定不变"——随范式更替

判据　反事实检验（去掉它领域还成立吗?）+ 词频（领域文本里被反复提到）+ 被引用度（其他概念定义引用它最多）+ 前置数（理解它所需前置最少）；后三条为代理判据

概念档案 • 根概念

① 定义　概念网络的"承重柱"——领域中不可再向下归约的源头概念，其余概念（含核心概念）都由它导出；根概念 ⊂ 核心概念（承重柱 ⊂ 骨架），通常 1~3 个

⑥ 对立　派生概念——可由根概念导出（如流程 = 产品数据在时间上的投影）；根概念不可由任何域内概念导出（维度：归约方向）

⑦ 辨析　核心概念（骨架，管"连接多"，3~5 个）vs 根概念（承重柱，管"是源头"，1~3 个）；枢纽 hub（认知层，压缩学习术语）vs 根概念（知识层，源头术语）——同一结构原理的两层表现

⑨ 英文　root concept；root 本义"植物的根"——营养从根流向枝叶；图论有向图的根（root）= 无入边的源节点，恰对应"被全部依赖、不依赖别人"

② 背景　早期用"被引用度=出度、前置数=入度"测核心概念，实际测的是 root（有向源）属性而非 hub（无向中心）属性——名称与判据错位，逼出"根概念"这一层

③ 历史　图论 root（有向源）→ 存在物的不可归约者→ 本方法论四条判据（原子/生成/不可替代/递归）→ 判定矩阵 + 演绎链（§1.7.3）

④ 问题　没有根概念之前，"核心概念清单"只是凭感觉的枢纽列表，不知道哪根柱子最承重——这个领域，什么存在物是不可再归约的源头？

⑤ 场景　确定学习与治理的第一对象、解释"为什么是这个概念"；只需"学得快"时用核心概念就够，不必每次都挖到根

⑧ 误区　✗ "根概念=最重要的概念"——是最"源头"的，未必最"重要"（质量很重要，但它是需求/系统的派生）；✗ "根概念与核心概念矛盾"——分层不冲突（承重柱 ⊂ 骨架）；✗ "根概念凭直觉找"——必须过判定矩阵+演绎链；✗ "根概念永恒"——随范式更替

判据　原子性（不可由域内概念定义）+ 生成性（其余概念从它派生）+ 不可替代性（去掉它领域崩塌）+ 递归性（多个抽象层级反复生效）

概念档案 • 基本原理

① 定义　原理侧的"源头"——领域中不可再被推导的法则（命题），是根概念（母体）的语义内蕴被展开成的显式命题；基本原理 ⊂ 核心原理（公理 ⊂ 骨架规则），与根概念同处源头层

⑥ 对立　派生命题——可由基本原理推导（如测试用例设计规则由"验证以规定需求为参照"推出）；基本原理不可由域内任何命题推导（维度：推导方向）

⑦ 辨析　核心原理（枢纽命题，骨架规则）vs 基本原理（源头命题，公理）；原理 principle（一般规律）vs 命题 proposition（可判真假的陈述，弗雷格）；"原理"日常还指"原则/准则"，本方法论取"源头法则"义

⑨ 英文　root principle / axiom；root 与根概念同源（图论有向源：无入边）；axiom 希腊语 axiōma"被认为值得的东西"——自明、无需证明的起点

② 背景　诞生于"原理也有根"的追问：概念侧分出根层后，原理侧是否同样有源头？2×2 层级矩阵（节点/边 × 核心/根）落位后，发现原理侧的源头恰是根概念定义的语义内蕴

③ 历史　欧氏几何公设法 → 弗雷格"概念无真假、命题才有真假" → 本方法论 2×2 层级矩阵 → 母体—展开修正（基本原理 = 根概念的展开运算，演绎链即展开器）

④ 问题　没有基本原理之前，领域推理没有公理层——规则从哪来、凭什么站得住，全凭经验与权威：这个领域，什么法则是不容再推导的源头？

⑤ 场景　建立领域公理层、把概念内蕴显式化为工程规则（评审准则/测试用例/追溯矩阵）、用原理作探针检验对根概念的理解；只需"学得快"时用核心原理即可

⑧ 误区　✗ "基本原理=最重要的原理"——是最"源头"的，未必最"重要"；✗ "搞懂根概念就自动搞懂基本原理"——跨概念原理在接缝里，且内蕴≠显式、须展开成命题才可检验；✗ "基本原理凭直觉找"——须过四判据；✗ "基本原理永恒"——随范式更替

判据　不被推导（域内无更基本命题推它）+ 生成规则（派生命题与工程规则从它导出）+ 去之崩塌（去掉它领域推理失去依据）+ 多层生效（多个抽象层级反复适用）

概念档案 • 概念体系能力

① 定义　以概念为单位的压缩与组织能力（简称"概念能力"）——识别枢纽 × 织关系 × 迁移 × 辨边界

⑥ 对立　机械执行（不动脑的重复）与碎片化记忆（无结构的信息堆积）——组织信息 vs 存储信息

⑦ 辨析　概念技能 conceptual skill（Katz，管理视角，偏"看全局"）；抽象思维（更宽，含演绎推理）；理解力（偏接收端，概念体系能力还含输出端：迁移与辨伪）；根概念能力（子集：方向盘）vs 概念体系能力（全集：四构件）

⑨ 英文　conceptual ability；concept ← 拉丁 conceptus（"抓取"）——概念体系能力字面即"抓取的能力"

② 背景　诞生于"AI 时代人还剩什么"的处境：记忆与信息处理外包给机器，人剩下的稀缺能力是压缩与组织（北大压缩学习 2026；Katz 1955 概念技能）

③ 历史　conceptual skill（Katz 1955，管理）→ conceptual knowledge（布鲁姆修订版 2001，教育）→ concept learning（认知科学）→ 本方法论整合为"概念体系能力"（五环模型第 1、2 环）

④ 问题　没有它之前：学习是碎片堆砌、表达是复读、跨领域是重来——如何把混沌信息变成可用结构？

⑤ 场景　学习新领域（立枢纽）、诊断概念误解（辨边界）、写作与教学（讲给外行听）、跨域转型（迁移）；纯程序性操作为主时不适用

⑧ 误区　✗ "概念体系能力强=懂得多"——组织得好才关键；✗ "概念体系能力强=

口才好"——转述是测试，结构是本体；✗ "概念体系能力=核心竞争力"——是枢纽不是全集；✗ "概念体系能力是天生的"——五环模型就是训练流水线；✗ "概念体系能力=根概念能力"——根概念能力只是方向盘（识别承重柱），全集还含织关系/迁移/辨边界

判据　转述测试（陌生领域三五句话讲清）+ 辨伪测试（能举反例划边界）+ 迁移测试（换个情境还认得出同一个结构）

概念档案 • 根概念能力

① 定义　识别、验证并运用根概念的能力——概念体系能力的子能力（方向盘：先抓承重柱，再展骨架）

⑥ 对立　细节优先的学习习惯（LeafToHub）——从边缘概念入手、不看骨架、更不辨承重柱，被细节淹没

⑦ 辨析　概念体系能力（全集：识别/组织/迁移/辨边界）vs 根概念能力（子集：承重柱识别）；判断力/洞察力（更宽，含价值判断）vs 根概念能力（特指结构性判断）

⑨ 英文　root-concept ability；root（根）——植物的根、图论有向源的"根"；能力对象从"hub（枢纽，连接多）"收窄为"root（源头，被依赖）"

② 背景　压缩学习实验 HubToLeaf 组优势（预学习 200 轮，优势扩散至 800 轮）——"先识别枢纽"可训练且回报极高；进一步追问"枢纽的深层形态"，落到根概念

③ 历史　专家直觉（老手知道什么重要）→ 压缩学习实验证实其可训练 → 三步识别法（收集候选/硬判据/往上挂）→ 三步识别法 + 判定矩阵/演绎链（§1.7.2/§1.7.3）→ 本方法论第 1 环（立枢纽）

④ 问题　没有它之前，新手被细节淹没、学得慢而累，且分不清"重要"与"源头"——面对陌生领域，20 分钟内找到那根承重柱与整副骨架

⑤ 场景　学新领域、做课程大纲、审一份陌生标准、跨界转型；已有权威框架时直接采用，不必每次重找

⑧ 误区　✗ "根概念能力=知识渊博"——知识多不等于会抓根；✗ "是天赋"——判定矩阵+演绎链可训练；✗ "一次定终身"——骨架必须勤校准（五环第 4 环）；✗ "管住根就管住一切"——根概念能力只是方向盘，织关系/迁移/辨边界还要靠全集

判据　20 分钟对陌生领域给出"根概念 + 核心概念清单及其关系"，并能用判定矩阵与演绎链为每个候选辩护

概念档案 • 工程

① 定义　科学原理 × 判断创造，在约束下，用系统方法，造出有用且安全的东西

⑥ 对立　手艺（方法论维度）• 科学（认识论维度）• 艺术（目标维度）——工程没有单一对立面

⑦ 辨析　工程 ≠ 方法论（方法论只是四层之一）；工程 ≠ 科学（创造 vs 发现）
⑨ 英文　engineering ← 拉丁 ingenium（天生的才智、巧思）
② 背景　1941 ECPD 首次给出权威定义，从"造东西"里提炼出可检验的要素
③ 历史　80 年、四个机构（ECPD/ABET/INCOSE/IEEE 610.12）——**ECPD、ABET、INCOSE 三家收敛到同一组要素；IEEE 610.12 另给软件工程一侧的过程特征**
④ 问题　工程和手艺不分 → 交付依赖具体的人，换人就走样
⑤ 场景　团队交接/招聘/复盘时用试金石；"在不在做工程"用四层检验
⑧ 误区　✗ "有流程有工具 = 在做工程"——四层缺一不可
　　✗ "工程 = 写代码 / 画图纸"——载体不是判据
判据　试金石：换个人、换团队，用同样的需求和约束，能交付质量相当的成果——能，是工程；不能，那是手艺

概念档案 • 需求
① 定义　明示的、通常隐含的或必须履行的需求或期望（ISO 9000:2026 §3.5.1）
⑥ 对立　点子 / 创意——点子不表述为需求，就进不了设计开发的流程
⑦ 辨析　需要（内容•硬性）→ 期望（内容•软性）→ 需求（需要或期望 × 呈现方式）；需求 ≠ 目标
⑨ 英文　requirement ← 拉丁 requirere（要求、寻求）
② 背景　需要或期望的表述——"往往是研究的成果"
③ 历史　从"要的东西"漂移出"明示/隐含/必须履行"三形态——ISO 9000:2026 把这场漂移固化为标准定义（ISO 9000:2026 §3.5.1）
④ 问题　需求与目标、点子混用 → 设计开发转化了错误需求
⑤ 场景　设计开发的输入；三种形态都要识别，隐含需求要主动去挖
⑧ 误区　✗ "需求 = 用户说出来的"——隐含需求、必须履行的需求漏掉是缺陷
　　✗ "需求 = 目标"——目标回答"达成什么可度量的结果"，需求回答"做成什么样"（答"为什么做"的是目的）
判据　起点判据"可转化"：完整、无歧义、不冲突、隐含需求已识别

概念档案 • 设计开发
① 定义　将某一客体的需求转换为该客体更详细需求的一组过程（ISO 9000:2026 §3.3.11）
⑥ 对立　走流程——输入输出同样详细，没发生设计开发
⑦ 辨析　设计、开发、设计开发三个词的关系（§3.3.11 注 2）；需求开发（ISO 9001:2015 §8.2）与设计开发（§8.3）的边界——看"特性有没有被规定"（第3章 §3.3.3 立此边界；第4章 §4.2.5 承接句改挂该节，不引 ISO 9001 条款号）
⑨ 英文　design and development；design ← 拉丁 designare（标注、指明）

② 背景　起点是研究的结果——需求往往是研究的成果，设计开发天然带不确定性
③ 历史　输入比输出更宽泛、更概括；一个项目可有多个设计开发阶段（递归）——输出是更详细的需求，且逐级更细
④ 问题　何时开始、何时结束说不清 → 只做技术方案就宣布完成 → 下游环节停摆
⑤ 场景　从需求到可实现的转化；"这算不算设计开发"用"更详细"判据
⑧ 误区　✗ "设计开发 = 画图 / 创意活动"——是需求转化
✗ "技术方案做完 = 设计开发完成"——下游方案配齐才算
判据　止于"可实现"；"还有哪个环节会在执行时反问'这到底怎么做'？有，设计开发就没结束"

概念档案 • 质量
① 定义　客体的一组固有特性满足需求的程度（ISO 9000:2026 §3.5.2）
⑥ 对立　等级 grade——高档车可以是差质量，低档车可以是好质量
⑦ 辨析　符合性（满足了吗，二元）→ 质量（满足到什么程度，评价）→ 等级（按哪套需求分类）
⑨ 英文　quality ← 拉丁 qualitas（"如何"）
② 背景　程度是被"认定"出来的，不是被"计算"出来的——靠验证/确认用客观证据判定
③ 历史　（薄）从制造业"规格档位"来；标准通篇没有给质量一个单点分值
④ 问题　等级高 ≠ 质量好；"质量 vs 进度"被误判成打架
⑤ 场景　判定满足程度时用；特性超出需求范围的不是加分，是过度设计
⑧ 误区　✗ "特性越多/越高，质量越好"——出了需求区间，只加成本、不加质量
✗ "质量能算分（85 分）"——关键特性不能被加权平均摊平
判据　特性超出需求范围的不是质量加分，是过度设计——只加成本，不加质量

概念档案 • 规定需求
① 定义　被明示的需求——验证的参照；**成文与基线是它的可采条件，不是它的构成条件**（ISO 9000:2026 §3.5.1 注 2 的 e.g. 是举例）
⑥ 对立　不可采的明示——口头说了，但没判定方法、不可复现、无版本，够不上"依据"
⑦ 辨析　明示 = 规定（一个等号）；成文 ⊂ 明示（最常用的一种明示方式）
⑨ 英文　specified requirement；specified ← 拉丁 specificare（明确指明）
② 背景　需求开发四活动的输出——需求被"生"出来的成品
③ 历史　（薄）"可验证"是从验证定义（§3.11.12）推出来的，不是定义原话
④ 问题　没有规定需求，验证无从谈起（验证三要素的第一要素）

⑤ 场景　验证的参照；需求评审的对象；验收与责任的依据——评审、验证、确认都绕不开它

⑧ 误区　✗ "客户说了就是规定需求"——那是资格，不是可采；✗ "没写成文档就不算规定需求"——不算的是可采，不是资格；✗ "口头说了就能当依据"——缺的是可采

判据　一条需求能提供客观证据吗（测试只是其一）？能，就可采、可当依据；只能"感觉"，就还得写下来、定下来

概念档案 • 评审

① 定义　为达成既定目标，对客体的适宜性、充分性或有效性所作的确定（ISO 9000:2026 §3.11.2）

⑥ 对立　验证——评审对目标（方向对不对，判断），验证对需求（做对了吗，证明）

⑦ 辨析　评审 ≠ 核对需求——核对需求是验证的姿势；评审是唯一在纸面阶段抬头看目标的环节（确认是建成后的实证终审，方向上的把关是前后两处，见§8.3.6）

⑨ 英文　review ← re-（再）+ view（看）——再看一遍；第一遍是做，第二遍是看

② 背景　2026 版与验证/确认同属 §3.11"有关确定的术语"组——分野在动作词：评审用"确定"（determination，§3.11.1）定义，验证/确认用"证实"（confirmation）定义

③ 历史　IEEE 1028 把评审做法分成五类：管理/技术/检查/走查/审计

④ 问题　把评审做成核对需求 → 需求错了没人发现；目标偏了没人看

⑤ 场景　五个把关节点处的评审洒全程（需求/设计/代码/测试/发布）；复盘是单独的节点、不是评审（评审对目标，复盘查偏差）；目标具体化才评得起来

⑧ 误区　✗ "评审就是开会过一遍"——没结论、没记录，最多算聊天

✗ "评审目标太抽象，评不了"——目标具体化配方：对象 + 时限 + 可衡量结果

判据　三问过了吗（适宜/充分/有效）？通过标准会前定了吗？团队、结论、记录三属性齐吗？

概念档案 • 验证

① 定义　通过提供客观证据，证实规定需求已得到满足（ISO 9000:2026 §3.11.12）

⑥ 对立　评审——验证对需求（做对了吗，证明），评审对目标（方向对不对，判断）

⑦ 辨析　验证 ≠ 测试——测试是提供证据的手段之一；验证 = 对照规定需求 + 客观证据 + 证实

⑨ 英文　verification ← verus（真）——求真：对照规定需求，证明为真

② 背景　2026 版与评审同属 §3.11"有关确定的术语"组——分野在动作词：验证用"证实"（confirmation）配客观证据定义

③ 历史　V 模型的右半边——需求在哪一级分解，验证在哪一级回收（左右镜像）

④ 问题　只验证不确认 → 隐含需求系统性漏失 → 测试全过、没人用

⑤ 场景　对照规定需求提供客观证据；测试/检验/审查/计算/文档评审都是证据（符合性分析=设计层的验证）

⑧ 误区　✗ "测试全绿 = 验证过了"——测试只是证据之一，验证要对全部规定需求做证实

✗ "隐含需求靠验证兜住"——验证的尺子在文档里，够不着没写下来的需求

判据　三要素齐吗（对照 / 证据 / 证实）？逐级回收（单元 → 集成 → 系统）了吗？

概念档案 • 确认

① 定义　通过提供客观证据，证实针对特定预期用途或应用的需求已得到满足（ISO 9000:2026 §3.11.14）

⑥ 对立　验证——确认对用途（做对的东西了吗），验证对需求（做对了吗）

⑦ 辨析　确认 ≠ UAT 走流程——UAT 必须是真实用户 + 真实任务 + 真实场景；让业务方点测试环境是把确认做成了验证

⑨ 英文　validation ← validus（强壮、有效）——放进真实场景，看它"立得住"吗

② 背景　参照在场景里不在文档里——预期用途三构成：目标用户 / 使用场景任务 / 环境条件

③ 历史　（薄）"使用条件可以是真实的或模拟的"给了灰度发布、仿真测试的合法性

④ 问题　确认的存在价值建立在非明示需求上——写不下来的需求，只有真实场景能裁决

⑤ 场景　UAT / Beta / 灰度 / 试运行 / 仿真测试；兜住隐含需求，也兜住体验类需求

⑧ 误区　✗ "测试全过 = 能用"——测试证明做对，不证明能用

✗ "隐含需求先写文档再验证"——很多隐含需求写不出来，只有真实场景能裁决

判据　真实用户 + 真实任务 + 真实场景齐了吗？隐含需求在真实场景里显现并处理了吗？

概念档案 • 架构

① 定义　一个实体（如系统）在其环境中的基本概念或属性，以及用于该实体及其相关生存周期过程的实现和演化的管控原则（governing principles；ISO/IEC/IEEE 42010:2022；本书读作系统不可还原的涵义）——这些基本概念与属性构成演化中必须守住的**不变量集合**，是第14章不变量链（目的→不变量→根本决策→遵从与演化）的第二环

⑥ 对立　详细设计——架构是"去掉就不是这个系统"的那组**基本概念或属性与管控原则**（不可还原）；详细设计是"改了系统还是它"的细节（可还原）

⑦ 辨析　架构 ≠ 一张图——图是架构描述；架构是那份内容、**不是交付物**（能交付、能评审、能版本化的是架构描述）；**元素与关系不是定义要素**（2022版已从定义句删去，降为 42020:2019 §3.3 注 1；见 T-63 行）；框架 ≠ 架构——框架是"这一类"的组织方式（骨架+位置+组织规则，内容后填），架构是"这一个"实体在其环境中的基本概念或属性与管控原则，用框架 ≠ 有架构

⑨ 英文　architecture ← 希腊语 arkhitekton（首席工匠）；fundamental ← fundus（底部、地基）

② 背景　系统太复杂，一个人装不下 → 靠"**哪些才是基本的**"这一问把复杂度压成可管理

③ 历史　早期"架构=图纸" → 1970s 结构化设计 → 1990s 软件架构热 → 后由 ISO/IEC/IEEE 42010（现行 2022）定稿

④ 问题　架构 = 一张框图 → 图过时了架构也"过时"了（其实那组基本概念或属性一直活着）；架构 = 那组改起来贵的决定 → 漏判"不贵但承重"的，误纳"很贵却与目的无关"的

⑤ 场景　在不可逆决策做出来之前；变更影响评估时；团队边界定义时

⑧ 误区　✗ "架构就是那张架构图"——图会过时，**那组基本概念或属性与管住它的管控原则（架构）**可以一直有效

✗ "架构等于技术选型"——技术选型是"概念或属性"的一种体现，不是全部

判据　**必要性**（去掉它，目的落空吗；"不可再分 / 不可缺失 / 不可替代"三问是它的顺口问法）＋**充分性**（齐备之后，剩余问题是否都不影响目的的可达）——**不可逆性一类只是代理，管排序与时机，不管准入**

概念档案 • 架构描述

① 定义　用来表达一个架构的**工作产品**（work product；42010:2022 §3.3）

——架构是那份内容，不是交付物；能交付、能评审、能版本化的是它

⑥ 对立　架构——架构是抽象的、不可交割；架构描述是有形的、可交付、可版本化的

⑦ 辨析　≠ 架构本身；≠ 一张图（图只是它的**视图**之一）；≠ 交付物清单——它按"利益相关方 → 关注点 → 架构视角 → 架构视图"组织，含 11 类架构描述元素（§3.4 Note 1）

⑨ 英文　architecture description（AD）；description ← de-（彻底）+ scribere（写）

② 背景　架构无形，要被人使用、评审、冻结基线，必须落成有形的信息载体

③ 历史　2011 版定义句曾把架构"体现于元素与关系"，易被读成"结构即架构"；2022 版删去该分句，并把 AD 明写为 work product（§3.3）

④ 问题　把架构描述当架构 → 图过时就说"架构过时了"；把描述当交付物 → 以为交付了描述就等于交付了架构

⑤ 场景　评审、入库、基线冻结、交付与沟通的**唯一有形标的**；架构符合性检查的输入

⑧ 误区　✗ "我们的架构在这台服务器上"——说的是描述；✗ "画完图架构就有了"——图只是视图，架构要靠两样内容立住

判据　能不能被**单独识别**、**存储**、**交付**给别人用（§3.14 信息部件的三个动词）；**受控状态**（进配置管理、可冻结基线）是它成为"参照"的条件

概念档案 • 架构定义

① 定义　把一组**输入**变换成架构**两样内容**的那次动作（兼 15288:2023 §6.4.4 的**过程名** *System architecture definition process*）——输入五组（相关架构 / 组织与项目的方针与指令 / 生存周期概念与约束 / 利益相关方关切与需求 / 系统需求与约束），产出两样（**基本概念或属性**＋**管控原则**），落成的是**清单**不是描述；**不是"给架构下定义"**（那是第10章已完成的事）

⑥ 对立　给架构下定义（一个语义动作：把词说准）——**定义是动作，不是语义动作**；也对立于"设计"（另一样工作，在换手点的另一侧，归第14章）和"表达成件"（把内容按视角落成视图、写成描述，归第13章）

⑦ 辨析　架构定义 ≠ 汇总需求（需求只占输入里两样，且只有 *key architecturally-relevant* 的那部分留下）；≠ 整理（整理不产生内容，变换必然做选择）；≠ 发明（动词是综合与识别：偏好解类 / 既有解 / 模式战术与启发式）；≠ 画图选型定规矩（三样分别是描述、候选、原则的雏形，没有一样是变换本身）

⑨ 英文　architecture definition；15288:2023 §6.4.4 *System architecture

definition process*；动词 *transforms*（**不是** collect / summarize）；42020:2019 §8 概念化是它内部第一件事

② 背景　第10章立了"架构是什么"（两样内容），却留下一个空：**这些内容不会自己长出来**——"这几份清单怎么长出来，是第11章的事"；15288 只有"架构定义过程"没有"架构设计过程"，**过程以产出物命名**

③ 历史　42010:2022 §3.2 定义句（两半）→ 15288:2023 §6.4.4 过程化（目的句 / 成果八条 / 四活动）→ 42020:2019 §8~§10 三核心过程（概念化 / 评估 / 细化）＋§6 / §7 两并行过程；**本书读法**：四环（判断→立成内容→选定并背书→下游必须依据它）＋需求进架构三条去向＋资产 • 知识 • 经验各占一位

④ 问题　没有这个动作之前，"开始做架构设计"一句话罩着好几件事（定义 / 选定 / 表达），责任与交接都无处落；把入口说小成"一份需求"、把产出说成"一份文档"，都会让后面的人无从依据

⑤ 场景　章首现场（带图 / 选型 / 老规矩三样东西开会，三问都答不上）→ 吃进什么、吐出什么、中间加什么（入口五样＋一条例外；三条去向；产出只有两样；清单≠描述）→ 一句判断，凭什么让人非依不可（四环，**本章只走到第二环**）→ 素材：资产、知识与经验（三样各占一位；都没有时怎么办）

⑧ 误区　✗ "架构定义 = 给架构下定义"——那是第10章做完的事，本章讲的是一个动作；✗ "我是从需求出发的，所以输入就是需求"——输入五组，需求只占两样；✗ "送进来的需求最后都会变成架构内容"——只有关键架构相关的那部分会，其余各有去向但**不许消失**；✗ "清单写完，架构描述就有了"——清单属内容侧、描述属描述侧；✗ "手上没资产就没法做"——缺资产改变取材方式与细化深浅，不改变有无

判据　入口那一句问法（**这条需求是不是只有在"这个实体是什么"定下来之后才判得了**）＋四道准入（有类可归 • 有物可依 • 有径可循 • 有辨可评；四项全过＝候选，任何一项不过＝"想法"）＋三条红线（不许拿"没资产"当不写架构的理由 / 不许把参考架构当答案 / 不许把"不熟"写成"不该"）

概念档案 • 基本概念

① 定义　系统在其环境中"怎么想"的根源——组织范式 / 核心抽象 / 根本机制 / 根隐喻（42010 定义里的 concepts——本质 • 怎么想）；**来历一句话：概念是幸存下来的**——一个实体身上可以说的概念很多，能进架构的只是极少数，它们经历一轮又一轮取舍，最后活下来；活下来的理由只有一个：**去掉它，这个实体就不再是它自己**（第11章 §11.3.6）

⑥ 对立　详细实现——基本概念是"去掉就不是它"的层，改它等于换一个系统；详细实现是"改了系统还是它"的层

⑦ 辨析　基本概念 ≠ 技术选型——选型是候选，概念是幸存下来的承诺；命名是概念化的完成仪式

⑨ 英文　fundamental concepts；concept ← 拉丁 concipere（一起抓住、成孕）——把分散的东西"抓住"成一个整体

② 背景　概念不会自己长出来，也不靠灵感——**它从一轮又一轮的取舍里幸存**；可参的来路是既有解、参考架构、模式战术与启发式（第11章），过不了承重测试的就被筛掉

③ 历史　"基本概念"来自 42010 定义中的 fundamental concepts；生成路径五条（模式选择/领域抽象/质量反推/类比迁移/第一原理）

④ 问题　把概念当发明 → 每代系统从零开始；概念没命名 → 无法共享、讨论、传承

⑤ 场景　组织范式定形态、核心抽象定实体、根本机制定机理、根隐喻定眼光；质量属性反推是"驱动"机制

⑧ 误区　✗ "概念是架构师发明出来的"——候选要过准入检验才配叫"基本"，概念是**幸存**下来的（去了它这个实体就不是它自己）

✗ "画了架构图 = 定义了概念"——概念长在决策里，不长在图上

判据　四问：它是组织范式还是核心抽象？走哪条生成路径来的？过了检验吗？它有名字吗？

概念档案 • 基本属性

① 定义　系统具备的本质特征——可观察、可度量的表现；涌现属性，由结构决定

⑥ 对立　基本概念——概念定"系统是什么"（可切可选）；属性定"系统具备什么"（涌现，只能让结构去催生）

⑦ 辨析　属性 ≠ 需求指标——需求指标是目标，属性是结构运转后涌现的结果；场景化（QAS）是两者的桥

⑨ 英文　fundamental properties；property ← 拉丁 proprius（自身的、固有的）——系统固有的东西

② 背景　你设计的是结构，属性是结构运转之后的宏观效果——不能直接设计属性，只能设计结构让属性涌现

③ 历史　SEI 属性驱动设计（ADD）把属性设计拆成四步：QAS 场景化 → 战术选择 → 结构落位 → 验证度量

④ 问题　直接设计属性 → 画不出"可用性模块"；属性不场景化 → 无法进入设计、无法验收

⑤ 场景　质量属性是架构的主要驱动；属性之间天然冲突，设计实质是排优先级（ATAM）；生命线属性 = 关键性分析

⑧ 误区　✗ "直接设计可用性"——不能设计强度，只能设计晶格；不能设计可用性，只能设计冗余

✗ "属性可以分配给某个模块"——质量属性是涌现的，不能"分"，只能"译"成部件约束

判据　四问：目标属性场景化了吗（QAS）？选了哪个战术？战术落在哪个结构位置？度量过、涌现达标了吗？

概念档案 • 架构决策

① 定义　在架构总体语境中所做选择的一个集合（42010:2022 §5.2.12）——不是一个动作，不是拍板那一下；

它的理据与它并排，同为架构描述元素（§5.2.9）

⑥ 对立　实现决策——架构决策要按架构待遇办（识别→判断→选定→记录）；实现决策改得动、改得起。

另两个常见对立：不是"架构风格"（微服务/事件驱动是套路），不是"画图时做的决定"

⑦ 辨析　与"权衡"分家：权衡管能不能把东西分开，决策管谁把它定下来；与"一组决定"分家——

架构定义句里没有"决定"，决策是抓手不是架构本身（T-63）

⑨ 英文　architecture decision；decision ← 拉丁 decidere（切断、裁断）；rationale ＝ 解释、论证或推理

② 背景　42010:2022 §5.2.12 给定义句与示例（含"所要考虑的选项集的选择"）；§6.10 管记录与理据；

Nygard 2011 提出 ADR 的实践惯例

③ 历史　从模糊说法变成可记录、可管理、可审查的工程对象

④ 问题　架构决策不可避免——不是做不做，是早晚（"我们没做架构决策"是伪选项）；

拍板通常在架构过程之外——评估出"最佳"是判断，处置是决定

⑤ 场景　恒信编号事故（把四条选择打包成"改个字段"）；把谁列为候选本身是一条决策；

量不可逆性用三样（影响半径/信息成本/修改风险）；记录用六问（十一项的本书压法）

⑧ 误区　✗ "改字段是小事"——数据模型可能正是最难反悔的那一类

✗ "架构决策是评审会上才有的"——没被识别出来的架构决策依然承担全部代价

✗ "贵的东西就算架构"——"改起来贵"只出现在"要不要记"那张单子上，

不构成准入判据

✗ "记下来就算拍板了"——记录不产生授权，决策权属方要单独写

✗ "写了就一次定死"——三个时间戳里有一个专记修改

✗ "记不记随我"——标准分 shall 与 should 两档（决策与理由必须记）

判据 ① 准入：它对目的是必要的吗（承第10章"承重测试"）；

② 识别：它是不是一组选择、而不是一个动作；

③ 排序与时机：三样（影响半径/信息成本/修改风险）各看几个来路指高——**只定先后，不发身份证**

概念档案 • 配置

① 定义 实体被记录在配置信息里、相互关联的功能特性与物理特性——实体"被记录下来的样子"

⑥ 对立 实体的全部客观特性——特性客观存在，配置是"被管理起来的那部分特性"（三个隐形过滤器切出来的子集）

⑦ 辨析 配置 ≠ 配置项（对象 vs 属性：配置项是"人"，配置是"人的身高体重"）；配置 ≠ 配置信息（属性 vs 记录：配置是"体检结果"，配置信息是"体检档案"）；配置 ≠ 参数文件（参数是零件，配置是装好的整机）

⑨ 英文 configuration——con-（共同）+ figurare（塑造成形）＝共同塑成的形态；中文"配+置"＝配合+安置＝搭配并安置成形，两语同源

② 背景 变与不变的分离——核心只写一遍，变化的部分外置；"改一个参数就要改代码、重编译、重发布"是没分离的信号

③ 历史 从"多"到"一"的旅程——多个组件经"配"与"置"塑成一个整体形态（as-designed / as-built）；选配有 requires / excludes / mandatory 约束

④ 问题 把配置当参数文件 → 拿零件清单当整机图纸，追溯时"这台箱子当时是什么样"答不上来

⑤ 场景 配置项清单（管哪些）；类别/记录/关联三个隐形过滤器（哪些特性算配置）；追溯温度记录必须知道固件版本

⑧ 误区 ✗ "配置 = 参数文件 / 个性化设置"——参数只是记录形态的图纸，形态才是配置

✗ "配置只跟软件有关"——硬件、图纸、文档、数据都要配

✗ "配置就是客户能改的那几个开关"——选配、约束、组合都是配置

判据 问三句：它被记录了吗（记录边界）？它是功能/物理特性吗（类别边界）？它和其他特性关联成整体了吗（关联边界）？——三句全"是"，才是配置

概念档案 • 基线

① 定义 经批准、确立了产品或服务某一时点特性、作为生存周期参照的配置信息

（ISO 9000:2026 §3.10.12）——被批准冻结的参照

⑥ 对立　流动状态——基线把"流动"升格为"受控"：线左版本自由流动，线右任何变化必须走变更控制

⑦ 辨析　基线 ≠ 版本（版本是刻度，基线是有"资格"的那一版——把里程桩当收费站）；基线 ≠ 发布；基线 ≠ 快照

⑨ 英文　baseline——base（基础）+ line（线）＝"底线/基线"；本指测绘基准线，转义为"作为基准的那一条"

② 背景　让一直在变的东西说得清——变化需要一个"从什么变起"的原点

③ 历史　功能 → 分配 → 产品三种正式基线（不当正本讲，见 T-86）→ 正本给的两个名字：配置基线（内容受控）与产品基线（产品定型）；"版本顺着流、基线垂直切"（里程桩 vs 收费站）

④ 问题　没有基线，变化就不可言说——每一次变更都查无实据

⑤ 场景　配置基线（内容受控）与产品基线（产品定型）；变更控制循环（冻结→变更→再冻结）的支点；批准 • 冻结 • 参照三动作缺一，基线即名存实亡

⑧ 误区　✗ "版本号就是基线"——版本是刻度，基线是资格

✗ "基线建立了就不能改"——基线可以变更，但必须走变更控制，改完建立新基线

判据　三特征问全：被批准了吗？被冻结了吗？生存周期拿它作参照吗？——三问全"是"，才是基线

概念档案 • 治理

① 定义　**在组织范围内对架构进行管理和控制所依循的规程与取向**（ISO/IEC/IEEE 42020:2019 附录 G.1，资料性；该附录另列 a)~d) 四条要求）；过程侧的定义权在第 6 章——"建立并保持架构集合中的架构与企业目标、策略和战略的一致"。它是**一个词的两面**：一套**制度系统**（本书归为五样＝机构与程序 / 治理计划 / 治理指令与指导 / 符合性状态报告 / 架构集合目标）＋那套系统一直在做的**动作**（§6.4 六个活动、共 49 条任务）

⑥ 对立　管理——治理**守不变量**（企业的目标、策略、战略，架构的原则与红线），管理**调框架内的变量**（"守不变量、调变量"是本书对两句目的句的概括；"掌舵 / 划桨"是本书比喻）；**两者不是上下级**：治理建**机制**（§6.4.2 k) 配置控制管理机制），管理**执行过程**（§7.4.4 m)）——**机制≠过程**

⑦ 辨析　治理 ≠ 审批岗（机制不等于过程；正本那六项活动里没有"审批"）；治理 ≠ 一串会（"作出架构治理决策"只占六项之一，日常动作是看与立）；治理 ≠ 清洗工具或 IT 系统（定规则、裁例外是治理，清洗是执行层的活）；治理 ≠ 架构做完之后的第三步（§5 各过程可并发、指令与状态双向流动）

⑨ 英文　governance ← govern ← 拉丁 gubernare ← 希腊 kybernân（掌舵、领航）；本书**同源三槽分开译**——`governing` 译**管控**（作定语：管控原则 / 管控指令，说的是"这套东西是管事的"）、`governance` 译**治理**（就是本章这两义）、`governs` 译**管控**（作谓语：视角管控视图，说的是"谁管谁"）

② 背景　这个词在正本里的户口有两处：**附录 G.1（资料性定义）与第 6 章（过程正本）**——**定义权在这两处，不在行业框架那儿**，TOGAF 只作参考。为治理指定的组织权威机构称**设计权威机构（Design Authority）**，按架构原则被指定、必要时逐级上报（附录 G.1）

③ 历史　英文从"掌舵"来（同根词 cybernetics 保留"反馈纠偏"本义）；中文"治理"原是治国理政的常用词（"治"＝治水疏导、"理"＝治玉顺纹），1989 世界银行"治理危机" → 1995"治道" → 2000 俞可平定译 → 2013 国家治理现代化——**一次有意的定译**

④ 问题　治理与管理混成一个词 → 用治理的名字干管理的活（季度"数据治理委员会"只审不定）；治理与架构定义的关系被读成串行 → 以为"架构定完了交给治理审"；**判据本身没人管** → 会开了、字签了、纪要写了，可没人有权说"这条从现在起算数"

⑤ 场景　与架构定义**并行**——在架构定义的一头一尾朝里伸手（起手为治理做准备、收尾为治理提供支持），同时管着一件架构自己管不了的事：**谁能改架构本身**；三种"评"各评各的（符合性评定 §6.4.5 / 架构评估 §42020 §9 / 配置审核 15288:2023 §6.3.5），一个不能替另一个

⑧ 误区　✗ "治理就是把管理层升一级"——治理守不变量、管理调变量；判据是这话里有没有"给出指导"那一半

✗ "架构定完了，交给治理去审"——正本明写各过程可并发，指令与状态双向流动

✗ "治理委员会审批变更"——机制不等于过程；这条变更按哪一版规矩办？答得出，就不必上治理

✗ "治理越早介入越好"——该不该进治理，看它动不动目的、原则、跨系统约定

判据　问四样：这条判断**谁给它效力**（谁有权批）？**谁能改它**（判据本身由谁定）？**按什么程序改**？**依据是哪一条**？——四样答得出，治理才算真的在

概念档案 • 三类需求

① 定义　产品需求要全 = 功能需求 + 环境适应性需求 + 可实现性需求，每类"及其指标"（功能以 IPO 定义行为；环境对气候/法规/文化的适应；可实现为生存周期八方面）

⑥ 对立　功能清单——只写"要做什么"，把产品需求当成功能需求，缺了环境适应性与可实现性

⑦ 辨析　功能/非功能二分 ⊂ 三类需求："非功能需求"是个大筐，装着环境适应性和可实现性两类主人、来源、验证载体都不同的需求；分类的判据不是特性本身，是"谁在问"

⑨ 英文　functional / environmental adaptability / realizability requirement；realizability ← realize（实现）

② 背景　产品的终点是"可实现"——只有功能需求，交付物停在原理样机；补上环境适应性需求，停在环境样机；补上可实现性需求，才是产品。分类判据是"谁在问"（客户、环境、承制方），不是特性的名字（第5章 §5.2.2）

③ 历史　以"产品 vs 样机"为界：功能/环境/可实现三对三对应原理样机/环境样机/产品（需求类别 → 验证条件 → 交付物）

④ 问题　需求"全不全"说不清 → 靠"功能写得多"自欺 → 量产翻车

⑤ 场景　相关方需求捕获、需求评审、SRS 编写；每一类都要答出来源、相关方、指标、验证载体

⑧ 误区　✗ "需求写全了 = 功能写全了"

✗ "可实现性需求是后续工艺的事"——它是产品的固有特性，必须写进相关方需求

判据　每一类都能答出"谁在问"（客户/环境/承制方）+ "用什么交付物验证它"（原理样机/环境样机/产品）

概念档案 • 验证载体

① 定义　每类需求的专属验证交付物：功能→原理样机、环境适应性→环境样机、可实现性→真正的产品

⑥ 对立　用一个载体验证全部需求——用原理样机的通过宣布产品成功

⑦ 辨析　验证载体 vs 评审/验证/确认（第8章）：载体回答"每种需求用什么证明"，判定回答"验证时对照什么"；载体验证的是需求，不是"做对了吗"的动作

⑨ 英文　prototype（原型、样机）——先造的、用于检验的实物；validation vehicle（沿用译名；按第8章严格口径，环境样机入真实环境更近确认，见⑥⑦）

② 背景　三类需求需要三种互不相容的验证条件：原理要脱离环境/成本单独验，环境要在真实环境验，可实现要在量产经济性约束下验

③ 历史　"样机→环境样机→产品"逐级完备，让"样机通过"被误读为"产品成功"——但每类需求只有一个专属载体

④ 问题　原理样机通过就宣布产品成功 → 环境适应性未验、可实现性根本没进需求清单 → 量产翻车

⑤ 场景　验收、里程碑决策、需求评审；判定"交付物是什么"用三类载体的验证完成度

⑧ 误区　✗ "样机验证通过了 = 产品成功了"——只验证了功能需求

判据　每一类需求都答出"用什么交付物验证它"——答不上来的那类，就是需求没写全的证据

概念档案 • 承制方主导

① 定义　产品需求开发由承制方主导（捕获、分析、建模、引导范围、形成条目），利益相关方只保留确认终审权

⑥ 对立　客户主导——把需求开发当成"客户说、承制方记"；承制方对产品更专业，是主导方而非转述方

⑦ 辨析　承制方（主导 • 加工）vs 来源方（客户/环境/资产 • 原料）vs 确认方（相关方 • 终审）——三分工；引导可以激进，确认不能缺席

⑨ 英文　承制方 = 产品的实现方（realizing party / developer）；主导 ≈ lead / drive，区别于 guide（引导）

② 背景　客户知道"要什么"但不知道"什么可能/什么成熟/什么惯例"——专业不对称让承制方必须引导

③ 历史　需求捕获的"隐含性"（客户的需要连他自己都没意识到）→ 需要专业需求分析师（承制方团队）去挖——"承制方引导"在需求分析师角色上显形

④ 问题　承制方被动接单 → 需求不全 → 责任甩给客户（"没提清楚"）

⑤ 场景　需求访谈、范围沟通、需求评审；"需求全不全"的问责对象永远是承制方

⑧ 误区　✗ "承制方就是听客户要什么就做什么"

✗ "需求不够全是客户没提清楚"——需求全不全是承制方的责任

判据　需求全不全、明不明确、可不可度量——问责承制方；引导可以激进，利益相关方确认一次不能省

概念档案 • 产品数据

① 定义　关于产品的信息的、形式化的表示，适合由人或计算机处理；产品的信息存在、开发工作的全部对象（六类：需求/设计/制造/质量/服务/管理）

⑥ 对立　产品本身（物理存在）——描述与被描述者；数据管不到实物那一半

⑦ 辨析　不是文档（数据是内容、文档是容器）；不是配置项（受控的才是）；不是产品（验证要求两者一致）；不是架构本身（可交付与可冻结的是架构描述 AD）

⑨ 英文　product data ← product（产品）+ data（数据）；ISO 10303（STEP）为数据交换而定义

② 背景　产品数据交换困难催生 STEP 标准；开发全程在数据上工作，实物只是数据的复制品

③ 历史　ISO 10303（STEP）1980s 起规范产品数据表达；与 ISO 10007"配置信息"同族

④ 问题　没有它，开发="把实物做出来"、数据="开发完再补的文档"——追溯、验证、交接全断

⑤ 场景　六类数据的定义、组织、交接；评价锚点、问责落点、协作的对象

⑧ 误区　✗ "数据是文档，开发完再补"——数据是开发工作的全部对象

✗ "产品=实物"——设计态里产品=纯数据

判据　六类数据覆盖产品全程、可验证可追溯；设计态里开发产品 ≡ 开发产品数据

概念档案 • 两域三态

① 定义　产品在两个域（信息域/物理域）、三种状态（设计态/制造态/使用态）中存在；设计态产品=纯数据

⑥ 对立　物理存在（实物）——数据管不着的那一半（制造偏差、使用体验）

⑦ 辨析　三态是同一产品的三种存在方式，不是三个产品；设计态同一性 ≠ 产品就是数据

⑨ 英文　as-designed（设计态）/as-built（制造态）/as-maintained（使用态）；数字孪生=信息域对物理域的实时映射

② 背景　开发/制造/使用三个阶段，产品的存在形态不同；PLM 用三态组织数据

③ 历史　PLM/STEP 数据管理实践；EBOM（工程 BOM）→ MBOM（制造 BOM）→ 维护记录

④ 问题　把三态混为一谈，就会问错"产品是什么"——设计态里没有实物可看

⑤ 场景　判断"开发产品=开发产品数据"的成立范围；组织产品数据的分态管理

⑧ 误区　✗ "质量在车间里"——开发环节的质量就是数据质量（数据质量决定上限）

判据　设计态产品=纯数据（开发≡开发数据）；制造态=数据+实物；使用态=实物+实况

概念档案 • 价值三层

① 定义　利益相关方对产品贡献价值的沉淀层（物化/决策/过程三层）；产品数据是价值的投影与锚

⑥ 对立　价值本体（人的判断与协作）——数据管不到的源头

⑦ 辨析　三层 ≠ 全部价值：决策层（ADR）部分沉淀、过程层只留结果

⑨ 英文　ADR = architecture decision record（决策层沉淀）；Polanyi"我们知

道的多于我们能说出的"

② 背景　工件是判断的投影：完美工件可能是错误决策的产物（验证全绿、确认全红）

③ 历史　源自"决策要记录"（决策记录六问）与知识管理实践

④ 问题　以为"数据全=价值全"——决策与过程层的价值流失，产品数据装不下

⑤ 场景　评价角色价值（工件质量×判断质量×下游影响）；职责挂在工件上

⑧ 误区　✗ "产品数据装下了全部价值"——物化层全在，决策/过程层只部分沉淀

判据　物化层完全在产品数据中、决策层部分沉淀（ADR）、过程层结果记入；数据是锚，判断与协作是本

概念档案 • 构思

① 定义　从关注点出发生成多个候选架构概念的发散性概念活动——"打开可能性"；产物是"可能是什么"，不是"决定是什么"

⑥ 对立　定义——构思发散、定义收敛；构思停在候选，定义画定边界

⑦ 辨析　构思 ≠ 灵感（构思有方法：模式库/参考架构/第一性原理）；≠ 设计本身（构思是广义设计的发散半场，设计还负责收敛与转换到底）

⑨ 英文　conceiving ← 拉丁 con（收进来）+ capere（握取）——"怀在脑中"；基因是孕育可能性，不是做决定

② 背景　ISO/IEC/IEEE 42010 architecting 动词链第一词；标准用了它却未单独立词条，操作化在 15288 §6.4.4"生成候选架构"任务

③ 历史　构思作为工程概念长期被"设计"吞并——42010 把 architecting 拆成七动词后，构思才从设计的影子里走出来

④ 问题　直接把第一个想法当唯一解——"唯一解 = 未审视的解"，没有候选就没有权衡

⑤ 场景　架构生成六步的①②步（定驱动因素→生成备选）；纪律：备选 ≥2 个；陷阱：稻草人备选（造注定输的陪跑，让预定方案"胜出"）

⑧ 误区　✗ "构思=等灵感"——构思有方法，方法正是创新可工程化的核心证据

判据　这一个动作产出了 ≥2 个候选概念吗？每个候选都能说清它回应谁的什么关注点吗？——是，才是构思；只有一个解的"构思"，等于没构思（本书凝练）

概念档案 • 定义

① 定义　在候选之间画边界、做选择，使方案从多义变单义、从漂浮变确定——"终结漂浮"

⑥ 对立　构思——定义终结构思打开的漂浮；一个是发散、一个是收敛

⑦ 辨析　定义 ≠ 陈述一句话（陈述是定义的产出，定义是画边界的过程）；≠ 设计狭义（定义收敛出被承诺的架构，设计把它转换到底）

⑨ 英文　defining ← 拉丁 de（往下）+ finis（边界/终点）——"划到边界为止"；同根：finish、final；ISO 1087-1：definition = 把概念与相关概念区分开来的表述

② 背景　ISO/IEC/IEEE 42010 architecting 动词链第二词；15288 §6.4.4"架构定义过程"内部第一件事就是构思——流程义的"定义"包住了动作义的"定义"的上游

③ 历史　定义是"口径事故"高发词：三重身份共存（动作义=画边界 / 流程义=15288 §6.4.4 技术流程 / 概念框架义=ISO 1087-1 界定表述）——同一句"架构定义"可能指三种东西

④ 问题　候选停留在多义、漂浮——没有边界就没有"被承诺的架构"，下游无法接棒

⑤ 场景　架构生成六步的③④步（权衡选定→分配与接口）；效用树+ADR 落定选择；partition/align/allocate 把边界落到元素与契约

⑧ 误区　✗ "架构定义 = 给架构写一句描述"——动作义与概念框架义混用，把"画边界"读成"写词条"

判据　选定方案有边界吗？被否掉的备选"死因"说得清吗（ADR）？——边界画定、漂浮终结，才是定义（本书凝练）

概念档案 • 设计（一词两义）

① 定义　一词两义：广义 = ABET 容器（迭代决策与创造的容器过程，⊃ 构思+定义+表达）；狭义 = 以生效架构基线为输入的下游细化流程（15288 §6.4.5）——"转换到底"

⑥ 对立　广义设计 vs 狭义设计——容器（包含构思定义）vs 下游（基线生效后才接棒）；并列使用时"设计"必须降为狭义义

⑦ 辨析　设计 ≠ 画图/选型/定规矩（那些是产物与候选，见架构设计卡）；≠ 构思（构思是设计的发散半场）；≠ 定义（定义收敛，设计转换）

⑨ 英文　design ← 拉丁 de（出来）+ signare（做记号）——"做记号标出来"，把意图标记成可传递的形式；ISO 24765：design 兼指过程与结果

② 背景　ABET engineering design = devising a system/component/process to meet desired needs（iterative/creative/open-ended）——"容器"出处；ISO 9000 §3.4.8：设计开发 = 把需求转换为更详细需求（可追归）

③ 历史　广义/狭义两义长期并存：工程教育（ABET）取容器义，系统生命周期（15288）取流程义——"设计是动词（构思）"是广义的通俗说法，"设计定义过程"是狭义的正式名称

④ 问题　不声明取哪一义，"设计"就在容器与下游之间漂移——说"我们在设计"，可能是构思阶段，也可能是基线后的细化

⑤ 场景　广义：设计全过程的统称（构思→定义→表达→…）；狭义：架构基线生效后 15288 §6.4.5 接手，把架构逐层细化为可实现元素与契约

⑧ 误区　✗ "设计 = 把想法画出来"——忽略广义容器（构思定义在内）；✗ "设计 = 架构基线前的一切"——忽略狭义下游（基线生效后设计还在继续）

判据　说"设计"时声明了取广义还是狭义吗？狭义设计的输入是生效的架构基线吗？——一词两义分得清，才是设计（本书凝练）

概念档案 • 架构活动

① 定义　architecting——在系统整个生存周期内构思、定义、表达、记录、沟通、认证恰当实现、维护与改进架构的过程（ISO/IEC/IEEE 42010 §3.1）；架构师能力清单的母词（第18章）

⑥ 对立　架构设计（本书用语；15288 的规范过程名是 **System architecture definition process**〔§6.4.4〕，正本无 `architectural design` 一词，见 ch11 §11.2.2）——架构活动覆盖整个生存周期，架构设计只是其中一段

⑦ 辨析　架构活动 ≠ 架构师头衔——42010 只认活动不认头衔；本书讨论"从事架构活动的人"（架构师），标准讨论"架构活动"本身

⑨ 英文　architecting = 42010 给 architecture 造的过程词（architecture 当动词用会硌牙，才造出 architecting）

② 背景　六块能力增量（架构决策/关注点管理/质量权衡/抽象建模/技术领导/演化对齐）收敛到同一个母词——动词链的投影

③ 历史　ISO/IEC/IEEE 42010 立"architecting"（架构活动）词条，把原本混在"架构设计"里的活动拆成七动词；中文定译"架构活动"（封面译法源同步 42020）

④ 问题　没有母词，架构师的能力只能罗列成碎片清单——有了动词链，每一项能力都能在链上找到位置

⑤ 场景　贯穿系统生存周期反复发生（不是串行阶段）；ch14 治理取链尾两项（认证=符合性监控、维护改进=演化治理），ch18 架构师取全链

⑧ 误区　✗ "架构活动 = 架构设计"——设计只是七动词之一；✗ "架构师 = 42010 头衔"——标准只认活动，不认职位

判据　一个动作能在七动词链上找到位置吗？在生存周期里反复发生吗？——能，才是架构活动（本书凝练：动词链为母词映射，非标准原文）

概念档案 • 创新

① 定义　创造或重新分配价值的新对象或变更对象（ISO 56000:2020）——结果义：新 × 成，缺一归零；只有新没成=点子/发明，只有成没新=优化/复现

⑥ 对立　发明——发明是新想法的诞生，创新是新组合的兑现（熊彼特）；发明在链上（构思），创新在链尾（评分席）

⑦ 辨析　创新 ≠ 构思（构思供给"新"的因子，"成"的因子大头在定义与设计）；≠ 创意/点子（没落地的改进不叫创新，奥斯陆手册）
⑨ 英文　innovation ← 拉丁 innovare（更新）；ISO 56000 取名词义（new or changed entity），熊彼特取活动义——本书统一取结果义
② 背景　创新管理是经济学/管理学话题（熊彼特、ISO 56000 家族），工程动作（构思/定义/设计）是它的生产现场
③ 历史　熊彼特 1912《经济发展理论》把创新从发明中剥出；Henderson & Clark 1990 四分型（突破式/架构式/模块式/渐进式）指出主导动作随类型在构思/定义/设计间滑动
④ 问题　把"创新"当动词用（"我们去创新吧"= 语病）——你能做的是构思/定义/设计/开发，创新是对动作结果的追认
⑤ 场景　链上的活干完了，够新又成了，才被追认为创新；工程化对象是流程/方法/组织/组合（ISO 56001 可认证），单点命中不可承诺
⑧ 误区　✗ "创新 = 灵感 = 构思"——架构式创新的主导动作恰是定义（元件不动、连接重排、重新画边界）；✗ "创新可被保证"——工程化管漏斗的形状与流量，不管哪一滴水通过
判据　够新吗（新颖性）× 成了吗（价值兑现）？缺一归零——两个都答"是"，才追认创新（本书凝练）

---

# 附录 C 误解清单：全部反直觉命题速查

> 每个概念一章，把“流行误解”和“精确概念”并排钉死——本书的误解清单完整版。读完正文，用这一页检验自己有没有把话说准；读正文之前，也可以用它当预习。

**第 1 章　误解（病）**：“背熟定义、记住更多词 = 懂了这个概念；懂概念靠天赋”　**精确概念（药）**：**把话说准，是学习能力里回报最高的那一步**：这条乘法律的两个归零项是**方向**（先抓哪个词）与**表征**（表征是 1、上面三级是 0 ——1 定成立，0 定量级）；概念是网（一般 ⊃ 核心 ⊃ 根）；根概念能力是方向盘；概念体系能力 = 识别枢纽×织

关系×迁移×辨边界；能力四级（表征 ⊃ 图式 ⊃ 心智模型 ⊃ 解释框架）；四问答全才算拥有　**一句话判据**：报出一个概念的两个坐标：**位次在哪一层、能力说到第几级**；先立枢纽再填细节；能答全四问才算懂；表征没拿到，上面盖多少层都白搭

**第 2 章　误解（病）**：“用了敏捷 + CI/CD + 测试，我们就在做软件工程”；“工程的对立面就是科学”；“系统化就是把流程固化成算法、让机器自动跑”；“软件/系统上线了 = 工程成功”；“老工程师经验丰富，这就是工程”；“自动化程度越高，系统越安全”；“微服务每个都健康，系统就是健康的”；“需求全部验证通过了，这个系统就成了”　**精确概念（药）**：工程是**工程科学、方法论、工具与实践、价值与伦理同时在场**的人类实践——一句话主张：**把交付从人身上挪到方法上，挪不走的是判断，没有单一对立面**（认识论对科学·方法论对手艺·目标对艺术）；**工程 ≠ 方法论**（敏捷 + CI/CD 只占其中一部分）；**系统化 ≠ 机械化**（有章法的判断 ≠ 确定性的执行）；**系统工程 = 元工程**（整合部件、管接口，涌现属性：部件全对 ≠ 系统对）；**工程系统包含技术系统**（承诺的是整个社会技术系统能运转）　**一句话判据**：**试金石**：换个人、换团队，用同样的需求和约束还能交付质量相当的成果才是工程；**层次同测**（缺价值与伦理＝有组织的赌博）；**工程化的成色**（有结构／有方法／可重复／全覆盖／受控／可验证）＋**系统化自问**（有输入输出／有检查点／换执行者稳定）；**价值失守问什么**（利益相关方知不知情／有没有评审／有没有记录）；**验收参照＝系统被用起来**，不是系统上线；**验证与确认不可互推**（验证对规定需求 · 确认对预期用途）

**第 3 章　误解（病）**：“参数比竞品高，质量绝对没问题”；“技术方案做完了，设计开发结束”；“质量进度冲突，砍验证先上线”；“质量能打 85 分”　**精确概念（药）**：那是**等级**，不是质量；设计开发止于**可实现**；验证与确认可缩不可砍；程度是“认定”不是“计算”　**一句话判据**：质量 = 固有特性满足需求的程度（可检验）；终点由“最

后一个配齐的方案”决定；等级高不等于质量好，超出需求范围就是过度设计；起点判据“可转化”——完整、无歧义、不冲突、隐含需求已识别；需求开发与设计开发的边界看特性有没有被规定；质量与进度是同一份需求集合内部打架，正道是与利益相关方一起剪裁（排序、留痕、挂账）

**第 4 章　误解（病）**：“需求管理包含需求开发，管好需求就开发好了”；“说了不等于规定，所以口头需求不算需求”；“需求评审开完就是确认通过了”；“改名留档就是版本管理”　**精确概念（药）**：**需求工程 = 需求开发 + 需求管理，两段并列**（开发负责做出来，管理负责让它一直是那一份；管理的严格补不了开发的欠账）；**明示即规定**（§3.5.1 注 2 是一个等号，成文属**可采条件**）——口头需求是规定需求，但**不可采**（没有判定方法、不可复现、没有版本），所以当不了验证的参照；**需求文档是工作产品；评审与验证都做得了**——评审对目标（§3.11.2）、验证对规定需求（§3.11.12，其对象含人工制品：15288:2023 §6.4.9）；**确认对照特定预期用途或应用的需求（§3.11.14），须在使用中兑现**；**改名留档 ≠ 版本管理**（要有基线、变更控制、追溯）　**一句话判据**：一条需求能提供客观证据吗（测试只是其一）？能，就可采、能当依据；需求乱先分清是“当初没说清”（开发欠账）还是“定了没人管”（管理欠账）——先补开发的课，再谈管理的严；无 CR 不得变更、不分析不得上会、验证通过方可关闭；改名留档要能答出“哪一版是基线、这次为什么改、从什么变到什么”

**第 5 章　误解（病）**：“原理样机验证通过了，产品就成功了；需求写全了 = 功能写全了；”非功能需求“是功能之外的另一类；隐含需求是三类之外的又一类”　**精确概念（药）**：需求要全，全在三类内容上（功能需求、环境适应性需求、可实现性需求），每类“及其指标”、每类各配一个专属验证载体；“非功能需求”是排除法划出来的名目，底下装着环境适应性需求与可实现性需求；隐含是“怎么呈现”

的一种形态，与“谁在问”这条内容分类是两条轴　**一句话判据**：每一块内容都能答出“谁在问”与“用什么交付物验证它”；答不出来的那一块，就是需求没写全的证据

**第 6 章　误解（病）**：“设计方案就是一篇大文章，写得长就负责”；“把需求拆细就算设计开发完了”　**精确概念（药）**：设计开发的单元是一对“需求-方案”——一次 Y=F(X) 求值（由需求开发出方案、并由方案迭代需求）；设计文档是树不是散文；每对先定规则：分解规则以消费方的语义范围为制定输入，单承接由分解规则直接保证；每个文档结对走完接·验·想·对·裁·签——需求就位验收后，方案侧还要长骨架、切叶节点、定稿　**一句话判据**：框架定义 ≠ 架构定义；单承接＝函数单值性（可 N:1、不可 1:N）；**落点以语义定级**；**链上只有分配步在做承接**；语义四查＋分配三自检＋承接覆盖检查／符合性分析

**第 7 章　误解（病）**：“产品是实物，数据是文档，开发完再补；质量在车间里”　**精确概念（药）**：产品有两个存在（物理/信息），开发造的是信息存在；开发产品=开发产品数据（设计态）；数据质量决定上限不决定达成；产品数据是协作的对象、职责的落点与企业的资产　**一句话判据**：设计态里开发产品 ≡ 开发产品数据；产品数据 ≠ 文档／≠配置项／≠产品／≠架构本身；产品会死，数据体系活着

**第 8 章　误解（病）**：“评审 = 核对需求；UAT 跑了 = 确认过了；测试全绿 = 验证过了；测试全绿 / 操作目标全绿 = 确认通过；一份需求文档或设计描述只能评审、谈不上验证”　**精确概念（药）**：**评审对目标、验证对规定需求、确认对预期用途**——三处不同参照；方向上的把关是前后两处（评审早拦·确认终审）；**对象不限**：实现与人工制品（产品数据）都在射程内（15288:2023 §6.4.9），数据交到哪一级验证就在哪一级回收；MOE 全绿 ≠ 确认通过（操作目标＝预期用途要求的显性子集，确认兜住操作目标没覆盖的使用期望）　**一句话判据**：只验证不确认 → 隐含需求系统性漏失 → 测试全过、没人用；

评审通过可被确认推翻（前后两处缺一不可）；一屋子专家对着错目标评审只会一致同意那个错目标

**第 9 章　误解（病）**：“会写代码 / 有职称 / 经验多 = 工程师；工程师是一个统一的物种”　**精确概念（药）**：工程师＝一份能力清单：能力来自第 2 章那几层，各长一项，另加横切的工程判断与元的可复现意识；活来自第 3 到第 8 章，需求、设计开发、产品数据、评审验证确认，按第 3 章那条路径依次走一遍；头衔不定义工程师；同一个统称底下站着好几种粒度（模块工程师／系统工程师／系统架构师）　**一句话判据**：**判据只有一条**：换个人、用同样的需求和约束，能不能交付质量相当的成果——缺哪一项，就问哪一项；能力问“我有没有”，活问“我会不会干”，两样都答得上来才叫工程师；能力不中性，职业伦理定方向；终责甩不掉、专责划得清；分形不对称（模块级没有对外架构／确认／部署／集成方案）

**第 10 章　误解（病）**：“架构 = 一张框图，画完就完事”；“架构就是那组改起来贵的决定”；“很贵、改不了的就该当架构”；“把它构架出来 = 骨架可见就交付了”；“架构”两个字本义就是架木造屋，所以架构就是结构　**精确概念（药）**：架构是实体在其环境中的**基本概念或属性**，加上管住它及其相关生存周期过程怎么实现、怎么演化的**管控原则**（42010:2022 §3.2）；架构**是那份内容，不是交付物**——能交付、能评审、能版本化的是**架构描述**；**元素与关系不是定义要素**（2022 已从定义句删去，降为 42020:2019 §3.3 注 1）；系统 ≠ 架构 ≠ 架构描述；构架立骨架、架构定本质（基本概念或属性与管控原则）、框架给半成品；**“基本”的准入判据是对目的必要且充分**——不是“不可再分／不可缺失／不可替代”（那三条只是必要性侧的问法），也不是不可逆性一类**代理**；**字面不承担定义**——“架构”的字面落在结构骨架上，而那正是 2022 已删的那一层：结构是架构“**可以关乎**”的对象（§5.2.2 五类之一），不是架构的内容　**一句话判据**：承重测试

＋**两遍跑**（兑现侧＋演化侧）＋关注点对账；约束即解放；骨架立起来了、理由还悬在空中 = 画图匠；见“结构／骨架／蓝图”先问一句：它说的是架构，还是架构可以关乎的对象

**第 11 章　误解（病）**：“架构定义 = 给架构下定义”；“我是从需求出发的，所以我的输入就是需求”；“送进来的需求最后都会变成架构内容”；“这一步就是把需求整理一遍、汇总一下”；“架构师往里加的是灵感”；“参考架构拿过来就能用”；“清单写完了，架构描述就有了”　**精确概念（药）**：**架构定义不是“给架构下定义”，是一次看得见进出口的变换**：吃进五组东西（**目的与目标是准绳、不在其中**），只把 key architecturally-relevant 的那部分变换成**两样内容**（**基本概念或属性＋管控原则**）；**动词是变换，不是汇总也不是整理**——同一批输入可以立成不同架构，**变换必然做选择**；**架构师往里加的是四样**（给每个候选写目的陈述／把手段立成概念与原则／把既有做法适配到眼前／把“现状”也摆上桌）；**候选是取材与综合来的，不是发明的**（三条来路＋四道准入：有类可归·有物可依·有径可循·有辨可评）；**清单是内容侧、视图是描述侧**——“清单写完”不等于“描述有了”（落成视图与描述归第 13 章）；**元素与关系不在产出里**（架构“可以关乎”结构，那是外延不是产出）　**一句话判据**：判据一串：入口那一句问法（**这条需求是不是只有在“这个实体是什么”定下来之后才判得了**）／四道准入（四项全过＝候选，任何一项不过＝想法）／三条红线（不许拿“没资产”当不写架构的理由、不许把参考架构当答案、不许把“不熟”写成“不该”）／**经验只帮着找、不帮着选**

**第 12 章　误解（病）**：“改个字段是小事；架构决策可以以后再说”；“改起来贵的选择都算架构”；“记下来就算拍板了”；“写了就一次定死”　**精确概念（药）**：**架构决策不是一个动作，是一个集合**；**架构决策不可避免**——“我们没做架构决策”是伪选项；**“贵”只出现在“要不要记”那张单子上，不构成准入判据**（**漏判**：不贵却承重，

如“日志须可被外部审计员独立读取”；**误纳**：很贵却与目的无关，如框架版本、组织惯例技术栈）：真准入判据是**目的必要性**，三样（影响半径／信息成本／修改风险）只管排序与时机；**拍板通常在架构过程之外**——评估出“最佳”是判断，处置是决定；**记录不产生授权、也不是一次定死**（决策权属方与三个时间戳）　**一句话判据**：先过目的关、再过代价关（次序不可颠倒）｜识别→判断→选定→记录，四步各有依据

**第 13 章　误解（病）**：“架构描述就是那摞图”；“架构本身有形态，能签收的是架构”；“视角是一种立场（或一种画图习惯）”；“内容都写全了，视图是重复劳动”；“先画图，再找视角挂上去也行”　**精确概念（药）**：**架构本身无形，交得出去的是架构描述**（工作产品，42010:2022 §3.3 ——能签收、能入库、能版本化）；**视角是一组惯例**（§3.8 正本定义句无属概念，不替它补）、**视图是信息部件**（§3.7）——**图只是视图组件的一种形态**；描述交得出去，不在把内容写全，在把“怎么读”写成公开规则：**视角先订、视图后造**，关切由视角框定、由视图应对；账（对应关系·对应方法·已知问题）与“视图之外那一档”（§6.1 标识与概述、§6.2 末两句未处理项声明）同样要交；缺的不是视角标准而是**视角规格**——标准故意不规定视角清单，正解是自建 ADF　**一句话判据**：判一份东西是不是视图就三步：在描述里吗／受某视角治理吗／标明模型种类或给了图例吗；七类视图只给“常用”（42020 附录 E，资料性），出哪几类由关切反推；写到哪儿为止——物理视图到设计解之前、逻辑视图到表示方式之前；价值是同一个词：**同解**

**第 14 章　误解（病）**：“架构就是粗一点的设计，我们只是把它做细”；“接手架构就是接收文档、照着做”；“设计里多出来的东西就地补上就行”；“视图上每个图标都得满足'基本'”；“能不能进视图、它表达的是谁、它够不够格，这三件事是一件事”；“系统分解结构是架构的产物”　**精确概念（药）**：**架构与设计是两类工作，不是同一件**

**事的两档精度**——判据是“**种类**”不是“**粒度**”：目的句不同、作用面不同（架构管适宜·存续·合意，设计管兼容·可造·可集），另加独立性要求“有效的架构尽可能不依赖于设计”；design 有宽窄两义——宽义句含 define the architecture，推不出“粗细两版”；**架构不是基于需求设计出来的**（架构＝**定义**解，设计＝**转换**；需求是必要且充分的**输入**）；交接的入口是**检查架构、取约束**（不是接收文档），中间是**两条转化**（架构实体与关系→设计元素、架构特性→设计特性）＋一**条分配**（需求→元素，须同时对着系统需求和架构目标）；设计会长出新元素（正常），要管的是**有没有被认领**；视图上的元素分**四层**、隔**两道界**，架构实体不必是实物、也不必逐一对上；**“基本”修饰的是架构的内容**（只从视角这一环起作用）；**系统分解结构属项目规划过程**，不是架构的产物　**一句话判据**：不是“细了没有”，是“**转没转过去**”；**交文档不算交接，取到约束才算**；**不必基本 ≠ 不必相关**；三关各管一段（描述关／实体关／目的关），谁也替不了谁；出口＝开发为规范用于采购或实现，**停在〈待定〉即未过出口**；“接口只细化不新增”**已废**（15288:2023 §6.4.5.3 注 10 明许新增接口）——不合格的是**追不到架构依据、又不回流**；换手点＝原子元素两条件（黑盒可定义＋自信定解）→ 买／造／重用；**无主比违规更危险**；系统结构不必是层级

**第 15 章　误解（病）**：“配置 = 参数设置页”；“配置项就是我想管的那几样”；“有版本号，配置管理就做好了”（“最新版就是基线”）；“变体就是高一级的版本”；“配置管理是定稿之后的事、是配置管理员的事”　**精确概念（药）**：**配置回答“它是什么样”、版本回答“走到哪一步”、基线回答“哪一版算数”**——配置是相互关联的功能特性与物理特性（ISO 9000:2026 §3.10.8），参数只是配置信息里的一个条目；配置项**靠选定得到**（§3.10.9），选进来要有清单、有归属；基线靠**批准·冻结·参照**三个动作（批准给资格、冻结给状态、参照给用途；15288:2023 §3.8、ISO 9000:2026 §3.10.12）；**版本顺着流、基**

**线垂直切**——“有版本管理、没有配置管理”是最常见的坏法（版本是回滚的地址，基线是变更的参照）；变体是空间（同时存在）、版本是时间（先后出现）；受控**与编写同时开始**，不是定稿之后才交出去管（42020:2019 §10.2 e)、§10.4.2 h) n)）；**正式化**这个动作不在技术过程里（15288:2023 §6.3.5.3 b) 4) 注 8）——挑候选的是架构与设计、收下并正式化的是配置管理（§6.4.4.3 e) 6) 与注 12）；配置管理是**五项活动**（准备／配置标识／更改管理／配置状态记帐／验证与审核，§6.3.5.3 a)~e)），它与前面各章**并行**——十四个技术过程都在识别基线候选物交给它；“功能基线／分配基线”是行业惯例，讲哪一套得先说自己讲的是哪一套的依据　**一句话判据**：拿三个问题对着任何一份 v1/v2/final 问：**它是哪一版？谁批的？什么时候？**——答不出就只是留了副本；再问一句“改它要不要过门、追不追得到凭什么”。五项目常活动点一遍，哪一项没有真实发生，那里就是下一次事故的入口

**第 16 章　误解（病）**：“治理 = 管理；买一套治理平台 = 治理落地”　**精确概念（药）**：治理**守不变量**、管理**调框架内的变量**；**机制不等于过程**；**判据本身不可自裁**　**一句话判据**：治理是**一个词的两面**（一套制度系统＋那个一直在做的动作）；它与架构定义**并行**、不是“做完之后的第三步”；两对活动同名而对象不同；三种“评”各评各的；**“定了”**与**“算数”**不是一回事

**第 17 章　误解（病）**：“每件事都做对了，系统就不会出问题”**精确概念（药）**：所有概念是同一个系统的不同切片；**切片合格 ≠ 系统合格**——问题常常出在片与片之间　**一句话判据**：四种缝（承接／参照／记录／问责）＋一张缝合清单（缝在哪两片之间、靠哪一种连上、责任人是谁）——切了，就要缝回来（**概念链在本章收束；第 18 章量的是“架构师”这个人**）

**第 18 章　误解（病）**：“架构师就是升职后的更高级工程师”；“图画得最多、最全的人，就是架构师”；“架构师应该什么都管”；“图纸都画完了，拍板就是架构师的事”；“被任命为架构师，就是架构师了”；“架构师负责把系统的目的定下来”；“评估和认证跟评审是一回事”；“能力是虚的，没法量”；“架构师不该再碰实现”；“做到架构师就到头了”；“标准里给'架构师'立了词条”；“配置管理和架构治理是行政事务，跟架构能力无关”　**精确概念（药）**：架构师是**把架构活动背在身上的人**（正式名＝系统架构师），不是职级：**能力清单＝两段加两格**，前一段是工程师那六项（科学素养／方法论素养／实践能力／职业伦理，另加横切的工程判断与可复现意识），后一段是他比工程师多出来的四件事（给每个候选写一句目的陈述／把手段立成概念与原则／适配与调和／把“现状”也摆上桌），两格是**配置管理**与**架构治理**；他对**架构的成立与守持负专责**，不背系统成败的终责（那是系统工程师的账）；**评审对材料、验证对需求、确认对用途、评估对架构、认证对实现**；**能力最终落在产品数据上**——删掉图决定还在的才算，名分不是能力、职级不定义物种　**一句话判据**：拿一个决定问这一层的谁有权改它；删一次图，看剩下什么；出事以后这口锅扣在谁头上；这份架构的决策权属方那一格写了谁；这份任命让清单上哪一项变了；这份目的陈述是谁说的、谁签的字；手上这件事对着的是什么；换个人拿着这些件接得住吗；最近一次亲手落地是什么时候；判断、权衡、沟通、担责这几样换了吗；第 3 章那 19 个词条里查得到 architect 吗；你不在场它接得住吗、要改它走哪条路

# 附录 D 参考文献总表

> 全书参考文献的总表：**D.1 标准**（R-01~R-42）、**D.2 经典文献**（R-43~R-97），编号连续、按号即读序；每条的末尾方括注「出现章节」。各章章末的「参考文献」就是本表按章列出的那一段，两处同号。查“这条原理出自哪个标准”到这里；查“这个概念怎么定义”回附录 A/B。版本号为正文实际引用的版本。
>
> 「出现章节」记该条在哪几章的章末「参考文献」里出现；末尾记〔**全书正文未引·参考书目**〕的 33 条，是本书选型时查阅过、正文未引用，保留作参考书目、不作引用依据。

## D.1 标准

**R-01　ISO 1087:2019**　主题：术语学·词汇｜本书锚定：概念。概念定义（§3.2.7）：“由特性的独特组合而形成的知识单元”；definition = 用以把一个概念与相关概念区分开来的表述（“画边界”的跨域印证）〔第 1 章〕

**R-02　ISO 9000:2015**　主题：质量管理体系·基础与术语｜本书锚定：需求/设计开发/质量/评审/验证/确认/目标/特性/等级/符合性/产品/服务/数据/信息/对象/顾客。§3.6.1 对象；§3.6.4 需求与规定需求；§3.4.8 设计开发；§3.6.2 质量；§3.7.6 产品；§3.7.7 服务；§3.8.1 数据；§3.8.2 信息；§3.2.4 顾客；§3.8.12 验证；§3.8.13 确认；§3.11.2 评审；§3.7.1 目标；§3.10.1 特性；§3.6.3 等级；§3.6.11 符合性〔第 6、7 章〕

**R-03　ISO 9001:2015**　主题：质量管理体系·要求｜本书锚定：需求确定、设计开发过程、设计评审与验证确认、变更。§8.2 需求确定；§8.3 设计开发（§8.3.3 设计开发输入的可转化判据/§8.3.4 评审/§8.3.5 设计输出/§8.3.6 设计更改）；§8.4 外部提供（物料采

购方案的条款落点）；§8.5.1 受控条件；§8.5.2 标识和可追溯性（产品标识与可追溯性）；§9.3 管理评审〔第 3、4、6、7、8 章〕

**R-04　ISO/IEC/IEEE 42010:2022**　主题：架构描述｜本书锚定：架构/架构描述/利益相关方/关注点/架构视角/架构视图/架构决策/基本概念/基本属性/管控原则。架构定义（实体的基本概念或属性 + **管控原则**〔governing·约束实现与演化及生存周期〕）；**利益相关方**→关注点→架构视角→架构视图链（四动词 has／frames／governs／addresses）；架构决策是**架构描述必须记录的内容**（§5.2.12／§6.10），不是架构本身；架构设计产出＝本质（concepts/properties）＋管控原则（落地见 ch10 §10.2.2）；architecting 七动词链定义（构思→定义→表达→记录→沟通→**认证恰当实现**→维护改进）= 架构师能力增量的**母词**，贯穿**生存周期**反复发生；“基本”判据的锚系二**手派生表述** IEV 831-01-01 Note 2（对目的而言必要且充分；**42010:2022 正本未引 IEV，书目仅列 IEC Electropedia**，它不是任何标准的准入判据条款）；EoI／多候选见 ch10 §10.4.4、ADF 与视图组件见 第 13 章（视图组件节）、1:1 与 1:N 见 第 13 章（视角节）术语注；三**条属概念（：架构描述＝工作产品（§3.3）、架构视图＝信息部件（§3.7）、架构视角＝一组惯例（§3.8，正本无属概念，不补）**；架构视角与架构视图的规格（§8.1 与附录 B）、ADF 规格（§7.1）、架构描述成分清单（§6.2~§6.10）、架构决策与理据的记录（§6.10）见第 13 章；**第 15 章用法——版本标识**：架构视角、架构视图、视图组件三处的条目下各有一条 shall「Each architecture viewpoint shall include version identification as specified by the organization and/or project.」（中译：每一张架构视角应含由组织和／或项目规定的版本标识。§6.6~§6.8），同一组要求另落在架构描述框架（§7.1）与模型种类（§8.2）下、那两处用的是 should（宜）；架构描述＝**工作产品**（§3.3）是“能被选定、被冻结、被引用

的永远是承载它的那份描述”的依据；AD 元素（§5.2.9）与决策及理由的记录（§6.10）随描述同版定格〔第 1、6、7、9、10、11、12、13、14、15、16、17、18 章〕

**R-05　ISO/IEC/IEEE 42020:2019**　主题：架构过程｜本书锚定：架构设计（§8.4.5 综合潜在方案／§8.4.7 形成候选架构）；架构治理。**§8.4.5 综合潜在方案／§8.4.7 形成候选架构**——候选架构概念由此生成，后接评估、选择（“架构综合”architecture synthesis 系本书早期措辞，正本无此词）——“42010 是语法书，42020 是创作手册”；架构治理流程 §6（**§2.1 目的句：建立并保持架构集合中的架构与企业目标、政策和战略以及各相关架构之间的一致**）、治理 §6 vs 管理 §7（**§6.2 注 2／§7.2 注同文：治理结果适用于整个组织、管理结果适用于架构集合；§5.3 另说治理过程“适用于组织内的全部架构工作”**）；**六个过程及其分组（正本 Table 1 与 §5.3–§5.5）**——治理 §6／管理 §7／概念化 §8／评估 §9／细化 §10／**使能 §11**；**§5.3 架构治理与架构管理同组、§5.4 架构概念化·评估·细化为三核心过程、§5.5 架构使能单列**（六者常被漏掉第六个）；**治理过程正本—— §6.4.1~§6.4.6 六项活动**（准备并规划 15 条／监视、评定和控制 11 条／确立架构集合目标 5 条／作出架构治理决策 6 条／监视并评定符合性 5 条／评审指令与指导的实施 7 条，**共 49 条任务**）＋**§6.3 三层规范力度**（活动 shall·任务 should·can 为许可）＋**§6.5 工作产品四项**（治理计划／治理指令与指导／符合性状态报告／架构集合目标）＋**§6.2 预期结果六条**（c) 决策由具有适当职权者作出并传达）；**§6.4.2 k) 配置控制管理机制**与**§6.4.5 d) 及注 4**（适当职权者可**直接访问按设计构建的实体**验证；注 4：**仅检查架构的符合性可能不够充分**，因为架构特征可能未能恰当转化为该实体的设计或运行）见第 15、16 章；**附录 A（规范性）裁剪过程**——组织的治理与指令可用于裁剪治理与管理过程，**裁剪须给出理由**（A.3.2

b)）；**附录 B.2.1（资料性）**给治理度量的三组指标（业务资源指标／各架构相对于业务目标的状况与演进／各架构之间的关联模型）；**第 15 章用法**：架构描述处于**配置控制**之下、并提供给所有相关方（§10.2 e)）、**确保架构描述处于适当的变更控制之下**（§10.4.2 h)）、以配置管理过程为指南管理架构描述、视图与模型的变更（§10.4.2 n)）、把产出置入存储库（§10.4.4 m)）、存储库的条目宜处于配置控制之下（§3.19 注 1）、**基线架构“充当同步关键点，用于架构的评审以及将架构作为参考予以提供”**（附录 E.4.1.6）——；**术语两处被 42010:2022 转引、第 10 章引用**——架构实体（§3.6 thing being considered, described, discussed, studied or otherwise addressed during the architecting effort，即“被架构的那个事物”，非“架构的构成物”）与关注点（§3.8 matter of interest or importance to a stakeholder）〔第 10、11、12、13、14、15、16、17、18 章〕

**R-06　ISO/IEC/IEEE 25010:2011**　主题：系统与软件质量模型（SQuaRE）｜本书锚定：基本属性（质量属性清单）。八大质量特性——功能适用性/性能效率/兼容性/易用性/可靠性/安全性/可维护性/可移植性（concern 提取的防漏检查清单）〔全书正文未引·参考书目〕

**R-07　SEI ADD / ATAM**　主题：属性驱动设计 / 架构权衡分析方法｜本书锚定：基本属性设计。ADD：QAS 场景化 → 战术选择 → 结构落位 → 验证度量；ATAM：把属性冲突摆上桌、逼出取舍；效用树（utility tree）= 质量属性→场景按重要性×难度打分〔全书正文未引·参考书目〕

**R-08　ISO 10007:2017**　主题：配置管理指南｜本书锚定：配置／配置项／配置信息／基线／变更控制（**只作行业出处指引**）。第 15 章把本标准列为行业出处（该章附录 E），正文不引其条款；**五项活动的一手出处是 ISO/IEC/IEEE 15288:2023 §6.3.5.3 a)~e)**（准备／配置标识／更改管理／配置状态记帐／验证与审核，见 T-87），“配

置标识→配置控制→状态记录→配置审计”这条四段链属本系旧说法、库内正本无此归并；配置家族的术语定义一律走 ISO 9000:2026 §3.10.8§3.10.12（T-79T-81、T-86）〔第 1、15 章〕

**R-09　ISO/IEC/IEEE 15288:2015**　主题：系统生命周期｜本书锚定：工程系统生命周期、契约链、集成、架构师角色（specialization）、系统工程师职责 **版本说明**：本条锚 **15288:2015**，而现行引用（含第 2 章新增的 §3.46／§3.50／§5.2.1／§3.54／§3.55）一律走 **15288:2023（R-37）**；本条为历史版本行。。系统生命周期过程；stakeholder 定义（§3.1）；需方-供方协议（契约链）；架构定义过程（6.4.4：生成→选择→表达）vs 设计定义过程（6.4.5）；架构师=工程师角色体系的一种 specialization（与 12207 并提）；角色定义；系统元素（可递归视为系统）〔第 9 章〕

**R-10　NASA TRL（技术就绪水平）**　主题：技术成熟度评估｜本书锚定：专属验证载体。九级刻度：基本原理观察（1）→ 真实环境运行（9）；原理样机演示通常对应 5~6 级、真实环境运行对应 8~9 级——“样机通过 ≠ 产品就绪”的工程刻度（生产就绪是 MRL 另一把尺子，TRL 只管技术成熟度）〔第 5 章〕

**R-11　IEEE 1028-2008**　主题：软件评审与审计｜本书锚定：评审、五类评审、系统性评审。系统性评审三属性（团队/结果文档化/过程文档化）；管理/技术/检查/走查/审计五类〔第 8、18 章〕

**R-12　ISO/IEC/IEEE 29148:2018**　主题：需求工程｜本书锚定：需求开发与需求管理；好需求判据。需求工程过程（开发与管理并列）；单条需求 11 项质量特性（必要/恰当/无歧义/完整/单一/可实现/可验证/正确/符合/一致/可追溯）+ 需求集 5 项 + shall 语句形式规则〔第 4、6 章〕

**R-13　IEEE 1362**　主题：运行概念（ConOps）｜本书锚定：运行概念 ConOps。ConOps 六部分结构（运行场景为核心）〔第 4 章〕

**R-14　IEEE 610.12-1990**　主题：软件工程术语｜本书锚定：软件工程定义、工程化过程特征。“把工程应用于软件”；systematic/disciplined/quantifiable → 工程化六方面〔第 2 章〕

**R-15　ECPD 1941 / ABET 1979 / INCOSE 2019**　主题：工程权威定义｜本书锚定：工程四要素；技术领导力。三家权威定义（ECPD／ABET／INCOSE）收敛到同一组要素（科学原理/判断创造/约束/系统方法/造福目的），IEEE 610.12-1990 另给软件工程一侧的过程特征；ABET engineering design 容器定义（devising a system/component/process to meet desired needs，iterative/creative/open-ended——“构思+定义”的容器）；ABET 工程师成果七条（“运用判断力加以应用” applied with judgment）；INCOSE 把技术领导力列为系统工程师核心能力〔第 2、9 章〕

**R-16　ISO/IEC/IEEE 12207:2017**　主题：软件生命周期｜本书锚定：联合评审；架构师角色（specialization——工程师角色体系中的一种）。评审是支持过程；architect 是工程过程中的角色〔第 8 章〕

**R-17　ISO/IEC 38500:2024**　主题：IT 治理｜本书锚定：治理模型四活动。engage stakeholders / evaluate / direct / monitor（利益相关方参与/评估/指导/监督）；§3.3 定义三职能（指导/监督/**担责**——第三成分英文为名词 accountability）；§3.2 评估=考虑并作出有依据的判断、§3.8 监督=复核作为适当决策与调整的依据（**评估判“该不该”、监督复核“是不是”**）；**§6.1 担责不可授权**；§7.2.5 授权；§7.2.7 展示担责的机制〔全书正文未引·参考书目〕

**R-18　CMMI（REQM / RD）**　主题：能力成熟度｜本书锚定：需求管理/需求开发；工程方法论举例。需求开发与需求管理是并列过程域（ch4）；作为工程/开发方法论之一举名（ch2 等：敏捷、瀑布、CMMI）〔第 2、4、9 章〕

**R-19　TOGAF / DAMA-DMBOK / ABPMP CBOK**　主题：企业架构/数据管理/流程管理｜本书锚定：治理伞结构、术语膨胀；架构描述框架 ADF（预定义的架构视角集）；企业架构四域。领域层面“管理”是伞、“治理”是子域；架构治理管控制与合规；4+1/TOGAF 等预定义的架构视角集（即架构描述框架 ADF）；系统之系统的企业架构四域（业务/数据/应用/技术）——**四域≠四个 42010 视角（框架内自述、非完整视角定义）**〔第 10、18 章〕

**R-20　ISO/IEC/IEEE 24765**　主题：系统与软件工程词汇｜本书锚定：版本/修订；系统工程师角色；模块。版本定义（初始发布或重新发布）；产品=由过程产出的工件（artifact）；系统工程师=实践系统工程学科的个人（与头衔无关）〔**按：24765 无该词条，系据同书** `systems security engineer` **词条句式类推，见 R-20**〕；module 定义与层级（system→subsystem→module→component）；design 双义（过程与结果）〔第 9 章〕

**R-21　SemVer**　主题：语义化版本｜本书锚定：版本号规则。主版本·次版本·补丁号三段升级规则——第 15 章 §15.2.3 只把它作**业界惯例**举例（改了不兼容的进主号、加了功能进次号、修了缺陷进修订号），**明记不是标准条文**；版本号给的是**位置**（答“这批记录是哪一版采的”），答不出**资格**（答不了“哪一版算数”）〔第 15 章〕

**R-22　ISO 10303-1（STEP）**　主题：产品数据表达与交换｜本书锚定：产品数据。产品数据 = 关于产品或产品族、与其整个生命周期相关的信息的表示〔第 7 章〕

**R-23　ISO/IEC 25012**　主题：数据质量模型｜本书锚定：数据质量特性（十五项，本章取五把尺子）。准确性/完整性/一致性/现时性/可信度〔第 7 章〕

**R-24　PMBOK® Guide**　主题：项目管理知识体系｜本书锚定：项目。项目 = 创造独特产品/服务/成果的临时性工作（产品 vs 项目辨析）〔第 7 章〕

**R-25　ISO 56000:2020**　主题：创新管理·基础与术语｜本书锚定：创新。创新 = 创造或重新分配价值的新对象或变更对象（结果义：新×成，缺一归零）——创新不在这条动词链上，是链尾的追认〔全书正文未引·参考书目〕

**R-26　ISO 56001:2024**　主题：创新管理体系·要求｜本书锚定：创新管理。可认证的创新管理体系（与 9001 同构）——创新工程化“管流程/组织/组合、不管单点确定性”的可认证证据〔全书正文未引·参考书目〕

**R-27　ISO 56002/56003/56005/56006/56007**　主题：创新管理家族｜本书锚定：创新管理。指南/创新伙伴关系/知识产权管理/战略情报/创意管理——创新工程的工具箱〔全书正文未引·参考书目〕

**R-28　ISO/IEC/IEEE 42030:2019**　主题：架构评估框架｜本书锚定：以目的检验架构（评估目的）。评估目的含“评估架构是否针对其预期目的（intended purpose）”——以目的检验架构的最权威背书；42010 家族分工：42010 管描述、42020 管过程、42030 管评估；2019 首版、**库内正本无版本状态记载**（32 号记 **AWI 修订在研**，ISO 号 93814）；**评估依据是目的、判定靠目标；评架构≠评描述（≠架构评审）见第 16 章**〔第 10、11、12、14、16、17、18 章〕

**R-29　IEEE 830-1998**　主题：软件需求规格说明书推荐实践｜本书锚定：好 SRS 特性。前身经典：正确/无歧义/完整/一致/按重

要度排序/可验证/可修改/可追溯八条特性——29148 单条 11 特性的历史前身（8 条 ⊆ 11 条）〔第 4 章〕

**R-30　ISO/TS 14812:2022**　主题：智能运输系统·词汇｜本书锚定：实物产品领域的四条明文视角。功能视角（§3.1.4.6）／物理视角（§3.1.4.8）／通信视角（§3.1.4.2）／企业视角（§3.1.4.4）——唯一能取到逐字定义句的实物产品视角落点；定义句带 ITS 领域限定词，迁移时只作**结构性迁移**（功能→物理→接口→企业四段结构），不得当通用产品标准原句〔全书正文未引·参考书目〕

**R-31　RM-ODP（ISO/IEC 10746 系列, ITU-T X.901）**　主题：开放分布式处理参考模型｜本书锚定：五视角蓝本。企业／信息／计算／工程／技术五视角，并带企业视角规格模板——自建 ADF 的四份蓝本之首〔全书正文未引·参考书目〕

**R-32　GERAM／ISO 15704（含 ISO 19439:2006）**　主题：企业与制造系统建模框架｜本书锚定：四视图蓝本。功能／信息／资源／组织四视图（ISO 19439 范围明写“针对特定用户关注点的附加视图可生成，但不属于本标准”）——自建 ADF 的蓝本之一（视图名称与条款号未核）〔全书正文未引·参考书目〕

**R-33　DoDAF 2.02（美国国防部架构框架）**　主题：作战任务系统架构框架｜本书锚定：八视角蓝本。全景／能力／作战／服务／系统／数据与信息／标准／项目；裁剪纪律逐字：“不规定任何模型”，力求“适合用途（Fit-for-Purpose）”，并非所有所述模型都必须创建；系统视角已标为**遗留支撑**〔全书正文未引·参考书目〕

**R-34　NAF v4.1（北约架构框架）**　主题：作战任务系统架构框架｜本书锚定：网格视角蓝本。行（关注主题 Concepts／Service／Logical／Physical Resource／Architecture Foundation）× 列（关注方面 Taxonomy／Structure／Connectivity／Behaviour／

Information／Constraints／Roadmap），“行列相交处的每一格就是一个视角”；视角选择必须针对具体架构工作裁剪〔全书正文未引·参考书目〕

**R-35　OMG UAF 1.3（ISO/IEC 19540-1/-2:2022）**　主题：统一架构框架｜本书锚定：十一视角蓝本。11 视角 × 11 方面；自述其视角命名取 42010 语境（视角是视图的规格），网格用于显示各视图规格与视角的对应〔全书正文未引·参考书目〕

**R-36　ISO 37000:2021**　主题：组织治理 指南（治理**本体**权威）｜本书锚定：治理/宗旨/担责/授权/治理方针/治理机构。§3.1.1 定义（human-based system／directed-overseen-held accountable 三职能／for achieving its defined purpose；中译定稿“以人为基础的系统”——不译“以人为本”，因 human-based 说的是**构成**不是价值观）；§3.2.2 担责定义与两注（注 1 告知和解释；注 2 后果由责任方承担）＋§6.5.3.2 披露八项（含“解释和证明”与“不履行责任的后果”）、§6.5.1 方向双向；§3.2.3 责任 vs §3.2.4 授权；§3.2.10 组织宗旨（注 2 宗旨指导表现目标——**不能用来做决策的宗旨是标语**）；§3.3.1–3.3.6 角色族（治理机构／治理小组／人员／利益相关方）；**§4.1 治理核心五条**（设定并致力于宗旨与价值／确定价值创造方法／指导和参与战略／监督按期望执行／表达责任感）；§4.2.2 授权五条件与那条禁令；§4.2.3 治理与管理之分（**定参数是治理、在参数内选最优是管理**）；**§5 三项治理结果**（有效的绩效／尽责的管理／合乎伦理的行为）；**§6 十一条原则三分类**（首要＝宗旨；基础 4＝价值创造·战略·监督·担责；赋能 6）；§6.4 监督（含 assurance 与内部控制体系）；§6.3.3.1.2 d 指导的止境；**治理系统的七类元素（主体/意图/规矩/结构/关系/行为/保证）系由本标准内容推导**，非条款原文〔全书正文未引·参考书目〕

**R-37　ISO/IEC/IEEE 15288:2023**　主题：系统与软件工程·系统生命周期过程｜本书锚定：需求侧过程（§6.4.1~§6.4.3）／**系统架构定义**（§6.4.4）／设计定义（§6.4.5）／人工制品（§3.6）。**需求侧三个过程**（第4章 §4.2.1的“需求工程过程锚点”）：§6.4.1业务或任务分析（**任务与目的**在那里定义）、§6.4.2利益相关方需要与需求定义（其注1指向29148：该标准给出更低层级细化与三份文档的带注释大纲）、§6.4.3系统需求定义——需求工程做的是后两步，第一步是它的输入；**§6.4.2.3的三条注**（第4章用）：注5「开发运行概念及其他生存周期概念」是 §6.4.2 的一项活动（系统运行概念的带注释大纲在该标准附录A）、注24需求以适合需求管理的形式记录→建立需求基线→保留需要的变化及其来源→四向追溯（第4章 §4.2.6）、注26单条需求特性与需求集特性（第4章 §4.4.4六条判据的库内邻近表述）；**第9章 §9.2.8的“把需要与期望开发成规定需求、再管住它一直是那一份”口径取自 §6.4.2.2 e)／i) 与 §6.4.2.3 注24**；**两个过程的目的句**：架构定义＝“生成系统架构的不同方案，选择一个或多个能够满足利益相关方关切和系统需求的方案，并以一致的视图和模型进行表达”；设计定义＝“提供关于系统和其元素的足够详细的数据和信息，以便根据系统需求和架构实现解决方案”；设计定义的四组活动（准备／开展／评估／管理，§6.4.5.3 a)~d)）、**两条转化**（架构实体与关系→设计元素、架构特性→设计特性，b) 2)、b) 3) 逐字）、分配（须满足系统需求**和架构目标**，b) 1)＋注4）、设计演化原则（a) 4)）；**注意分清出处——设计描述符**（design descriptor）、**原子元素**两条件（atomic／black box）、**横向与纵向集成**三项，属**INCOSE SEH 5.0专有表述**（正本只给邻近句 §6.4.5.3 a) 3) 注1／b) 4) 注7／§5.2.2，见R-63）；**§5.9（概念与系统定义）另有一句为两概念辨正的依据**——an architecture entity … and a system element … represent two different notions（架构实体≠系统元素，见第10、15、16章）；

**§5.9 还给出两过程的分工句与「有效的架构尽可能不依赖于设计」的规范位**——Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and on feasibility of construction and integration. An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space.（见第 16 章）；**§3.13** design, noun **＝“合规实现”的一手出处**——specification of system elements (3.47) and their relationships, that is sufficiently complete to support a compliant implementation of the architecture (3.5)（见第 16 章）；§3.47 系统元素＝discrete part of a system that can be implemented to fulfil specified requirements（§5.9 行文中的简式作 a discrete part of a system that fulfils specified requirements）；**§3.6** artefact **＝工作产品**——work product (3.32) that is produced and used during a project to capture and convey information（与 42010:2022 §3.3 注 1 的 work product **互为属概念、方向相反**，分流译名依“所属标准不同、属概念层级不同”；见第 8 章 §8.3.6 术语注）；**§6.4.14 处置过程**（退役与回收，第 3 章 §3.3.5）；**第 15 章用法**：**§3.8 基线**＝“配置项不论介质如何、在配置项生存周期内的特定时刻被正式指定并固定的、经正式批准的版本”（该条注明出处为 IEEE Std 828-2012，见 R-41）；**§3.11 配置项**；**§6.3.5 配置管理过程**——§6.3.5.1 目的句、§6.3.5.2 a)~f) 六条预期结果、§6.3.5.3 a)~e) 五项活动（准备／配置标识／更改管理／配置状态记帐／验证与审核）与 b) 4) 注 8 正式化、d) 注 18 记录比产品存续更久／注 19 配置审计的判据、注 13 偏离请求、注 15 更改请求与 CCB、注 22 两种配置审核由验证过程执行；**§6.4.4.3 e) 5) 与注 11**（架构实体与需求、接口、分析结果、验证方法的双向追溯）、

**e) 6) 与注 12**（识别基线的候选对象、随后把人工制品提供给配置管理）；**§ 6.4.5.3 注 20**（设计定义照同一句式交候选物）；**附录**把 SAE EIA-649 列为配置管理附加指南（见 R-42）；**上述条款均直核 2023 版正本**，INCOSE SEH 5.0 中文版（与 15288:2023 同版同年）退为中文对读旁证〔第 2、3、4、5、6、7、8、9、10、11、12、13、14、15、16、17、18 章〕

**R-38　ISO 9000:2026（第 5 版）**　主题：质量管理体系·基础与术语｜本书锚定：评审／验证／确认；设计和开发；目标。§ 3.11.2 评审＝“为达成既定目标，对客体的适宜性、充分性或有效性所作的确定”（设计定义是工作产品：**可评审，也可验证**——15288:2023 § 6.4.9 的验证对象含 artefact；**做不了确认**）；§ 3.11.12 验证对“规定的需求”；§ 3.11.14 确认对“针对特定预期用途或应用的需求”（注 3：使用条件可真实可模拟）——三把尺子的界分是本章审核动作定名的依据；**§ 3.5.1 注 2 规定的需求是明示的需求，例如写在成文信息中的需求**（第 4 章：e. g. 是举例——**明示即规定，成文属可采条件**，第 4 章 § 4.2.5 与 § 4.3.1 据此改写理由）；**§ 3.3.11 设计和开发＝“将某一客体的需求转换为该客体更详细需求的一组过程”**（第 4 章 § 4.2.4 目标→需求“转化”的依据）；**§ 3.7.11 objective＝“要实现的结果”**；**§ 3.5.2 质量、§ 3.5.3 客体、§ 3.5.4 等级、§ 3.5.5 法定要求、§ 3.5.8 法规要求、§ 3.5.9 合格**（第 3 章三个词与三处边界的条款依据）；**§ 3.10.6 competence**（第 9 章**：应用知识和技能以取得预期结果的能力）**、§ 3.3.11 设计和开发（第 9 章 § 9.2.3**：引它说“一组过程就是一套方法”，本书读法）**、§ 3.5.1＋注 2（第 9 章 § 9.2.8**：“需要与期望→规定需求”三词口径）**、§ 3.11.1 确定＋§ 3.11.2／§ 3.11.12／§ 3.11.14 三判定（第 9 章 § 9.2.8／§ 9.4.7**：“判定”简称的锚）**、**§ 3.10.1 特性、§ 3.8.6 客观证据、§ 3.2.11 项目**（第 3 章章末切片的其余五条）；**第 15 章用法——§ 3.10.8 配置**＝“在配置信息中规定

的产品（或服务）的相互关联的功能特性和物理特性”、**§3.10.9 配置项**＝“配置之中满足某一最终使用功能的实体”、**§3.10.10 配置信息**＝“关于产品或服务的设计、实现、验证、运行和支持的**要求**”、**§3.10.11 配置状态记帐**（15288:2023 §6.3.5.3 d) 逐字引此句）、**§3.10.12 配置基线**＝“经批准的配置信息，它确定产品或服务在某一时间点上的特性，并作为该产品或服务生存周期内各项活动的参照”；**§3.12.1 审核**（判据是审核准则，与配置审计分家）〔第 1、2、3、4、5、6、7、8、9、11、14、15、16、17、18 章〕

**R-39　ISO/IEC/IEEE 24765:2017**　主题：系统与软件工程·词汇｜本书锚定：design（宽义·术语学旁证）。**术语学旁证**：本章“合规实现”一语的**一手出处已改挂 R-37／ISO/IEC/IEEE 15288:2023 §3.13**，不必绕道本标准；本标准保留的理由只剩“**宽义 design** 的术语学来源”——其 design 定义句含 define **the architecture**（经 ISO/TS 14812 §3.1.15.1 design 注 1 转引，页面自标源即本标准，逐字 It is information including specification of system elements and their relationships, that is sufficiently complete to support a compliant implementation of the architecture）〔全书正文未引·参考书目〕

**R-40　ISO/TS 14812:2025**　主题：智能运输系统·词汇（术语批次）｜本书锚定：实现／兑现／设计（宽义）——**术语学旁证**。§3.1.15.2 implementation＝把设计翻译为硬件部件、软件部件或两者的过程（或该过程的结果）；§3.1.15.4 realization＝**implementation of a system concept**（注 1：兑现需要开发出一份设计、并把该设计翻译为部件）；链条为 concept → design（开发出规格）→ implementation（翻译成部件）→ realization（系统概念实现）——**“合规实现”的承担者在 implementation 侧**即据此旁证；该批术语 History note 记“Introduced in ISO/TS 14812:**2025**”，

与 2022 批（architecture／concern／viewpoint 等）并存于同一词汇站〔全书正文未引·参考书目〕

**R-41　IEEE Std 828-2012**　主题：系统与软件工程·配置管理标准｜本书锚定：基线的定义出处。ISO/IEC/IEEE 15288:2023 **§3.8 基线**那一条注明其出处即本标准；第 15 章由它取到“配置项不论介质如何、在配置项生存周期内的特定时刻被正式指定并固定的、经正式批准的版本”一句（见 T-85）〔第 15 章〕

**R-42　ISO/IEC/IEEE 24748-7／SAE EIA-649／MIL-STD-973**　主题：配置管理指南与军用配置管理标准｜本书锚定：功能基线／分配基线的行业出处。第 15 章只把这一族列为“功能基线、分配基线”三分法的**行业出处**，并明记；15288:2023 附录仅把 SAE EIA-649 列为配置管理附加指南——**三分法不当正本讲**，用须当场标明来源与状态（§15.2.5，见 T-86）〔第 15 章〕

## D.2 经典文献（非标准）

**R-43　索绪尔《普通语言学教程》**　符号 = 能指 + 所指；语言（langue）与言语（parole）〔第 1 章〕

**R-44　弗雷格《论涵义与指称》（1892）**　符号 → 涵义 → 指称；系统≠架构≠架构描述的哲学底座〔第 1 章〕

**R-45　Xiangjuan Ren 等《Compressive learning scaffolds higher-order network structure to enhance human knowledge acquisition》（Nature Communications, 2026）**　压缩学习：枢纽优先的学习结构；人脑是压缩机器不是存储器〔第 1 章〕

**R-46　Bloom 教育目标分类学（Anderson & Krathwohl 修订版，2001）**　五环模型掌握线——“会用才算掌握”：掌握线划在“应用”之上〔第 1 章〕

**R-47　Roediger & Karpicke（2006）**　测试效应：检索比重读记忆更强——五环第 3 环“逼输出”的依据〔第 1 章〕

**R-48　Posner 等（1982）**　概念转变：错误概念须被更准确解释主动替换——五环第 4 环“勤校准”的依据〔第 1 章〕

**R-49　Whitehead《The Aims of Education》(1929)**　惰性知识：不拿来解决问题的知识是死的——五环第 5 环“落实践”的依据〔第 1 章〕

**R-50　Lave & Wenger《Situated Learning》(1991)**　情境学习：技能＝概念结构×情境经验——五环第 5 环“落实践”的依据〔第 1 章〕

**R-51　Katz《Skills of an Effective Administrator》(1955)**　概念技能（conceptual skill）——“概念能力”概念的管理学源头（识别枢纽×织关系×迁移×辨边界）〔第 1 章〕

**R-52　Kruchten, “The 4+1 View Model of Architecture” (IEEE Software, 1995)**　4+1 视图模型——场景是检验其余视图的胶水〔第 10 章〕

**R-53　Simon Brown, “C4 model”**　C4 模型——软件架构描述的四层缩放（Context/Container/Component/Code），给架构描述一个统一的分层镜头〔第 10 章〕

**R-54　John Zachman, “A Framework for Information Systems Architecture” (1987)**　Zachman 框架——企业架构 6×6 分类矩阵（6 个视角 × 6 个 W 问题），最经典的架构描述框架（ADF）之一〔第 10 章〕

**R-55　Len Bass 等《Software Architecture in Practice》**　非功能需求（质量属性）才是架构的主要驱动〔全书正文未引·参考书目〕

**R-56　Nygard, “Documenting Architecture Decisions”（2011）**　架构决策记录（ADR）——决策记录惯例的起源（六问格式为本书适配，非 Nygard 原版字段）〔第 12、13 章〕

**R-57　Tricker, Corporate Governance（1984）**　掌舵/划桨：“有舵无桨到不了岸，有桨无舵原地转圈”〔第 16 章〕

**R-58　俞可平《治理与善治》（2000）**　governance 定译为“治理”；统治 vs 治理五维度〔全书正文未引·参考书目〕

**R-59　Rosenau, Governance without Government（1992）**　全球治理——“没有政府的治理”试金石〔全书正文未引·参考书目〕

**R-60　冯·卡门（von Kármán）**　“科学家研究现存的世界，工程师创造从未存在的世界”〔第 2 章〕

**R-61　Ralph Johnson**　“Architecture is about the important stuff. Whatever that is.”〔第 18 章〕

**R-62　NSPE Code of Ethics（美国国家专业工程师协会伦理准则）**　工程师伦理责任：服务必须致力于保护公众健康、安全与福祉；对公众负责，不是对老板负责〔第 9 章〕

**R-63　INCOSE Systems Engineering Handbook　工程系统**（书内为**译者注中文释义**，PDF p46／印刷页 31：工程化系统＝应用系统工程原理和实践进行设计、实现和验证的复杂集成体；被设计/改造、与环境交互、有目的、守约束，可含人/流程/组织/信息——**非 INCOSE 2019 定义句原文**）；技术领导力；系统工程师日常六件事；MOE/MOP/TPM 三层度量——MOE（效能度量）=运行环境中的成功指标≈操作目标层、MOP（性能度量）=系统级、TPM（技术性能测量）=关键部件级。**第 16 章用法**：① **已直核 15288:2023 正本、SEH 退为中文对读旁证**的——架构定义／设计定义的分立说明（逐

字即 §5.9）、四组活动（§6.4.5.3 a)~d)）、两条转化（b) 2)、b) 3)）、分配（b) 1)＋注 4)、设计演化原则（a) 4)）、目的句（§6.4.5.1）；② **仍属 SEH 5.0 专有表述**的——**设计描述符、原子元素两条件、横向集成与纵向集成、“衍生需求应当纳入系统需求基线”一句**；这四项**以 SEH 5.0 为出处**，正本只给**邻近句**（设计特性类别 §6.4.5.3 a) 3) 注 1、参数值 b) 4) 注 7、递归分解止点 §5.2.2、系统需求基线 §6.4.3.3 注 13）〔第 2、14、15、18 章〕

**R-64　World Bank, Sub-Saharan Africa: From Crisis to Sustainable Growth（1989）**　“治理危机”（crisis of governance）一词的起点〔全书正文未引·参考书目〕

**R-65　Ropohl（社会技术系统理论）**　技术子系统+人的子系统+组织子系统耦合的整体——单独优化技术部分没有意义；“用不起来”往往卡在人/流程/组织那一层〔第 2 章〕

**R-66　维特根斯坦《哲学研究》（1953）**　意义即用法（meaning is use）——“假懂 vs 真懂”的判定〔第 1 章〕

**R-67　库恩《科学革命的结构》（1962）**　范式转换——“共识 ≠ 真理”的例证〔第 1 章〕

**R-68　Kahneman《思考，快与慢》（2011）**　系统一/系统二、偏见——“找枢纽”的读者参照〔第 1 章〕

**R-69　MoSCoW 优先级方法（源自 DSDM）**　需求优先级排序（Must / Should / Could / Won't）〔第 4 章〕

**R-70　Kano 模型（1984）**　基本型/期望型/兴奋型需求分类——“与利益相关方剪裁需求”〔第 4 章〕

**R-71　Scrum Guide（Schwaber & Sutherland）**　敏捷实践：复盘归属对照（Scrum 事件）〔第 8 章〕

**R-72　Conway's Law, “How Do Committees Invent?” (1968）** 组织沟通结构与系统设计同构（康威定律）〔全书正文未引·参考书目〕

**R-73　Evans《领域驱动设计：软件核心复杂性应对之道》(2003）** DDD：bounded context / aggregate〔全书正文未引·参考书目〕

**R-74　迪米特法则（Law of Demeter, 1987）** 最少知识原则——“talk to your friends, not strangers”〔全书正文未引·参考书目〕

**R-75　ITIL 4 / COBIT 2019** IT 服务管理 / IT 治理框架——PDCA 对照与“运营”用语（治理模型以 ISO 38500:2024 §6 四活动为准）〔全书正文未引·参考书目〕

**R-76　NASA Systems Engineering Handbook（NASA/SP-2016-6105 Rev2, 2016）** 首席工程师角色的权威锚点〔第 18 章〕

**R-77　许慎《说文解字》(东汉）** “治”=治水疏导（三点水）、“理”=治玉顺纹——治理字源；“构（構）”=盖也、从木——构架=搭建承重骨架的字源〔全书正文未引·参考书目〕

**R-78　Miller, “The Magical Number Seven, Plus or Minus Two”（Psychological Review, 1956）** 工作记忆容量的经典文献（“7±2”的来路）。该章“4 到 7 个”与压缩单位“图式”的说法依据均在 **R-97 第 38 讲**。〔第 1 章〕

**R-79　IEEE Std 1471-2000（架构描述推荐实践）** ISO 42010 的前身（2000s IEEE 1471 → 并入 ISO/IEC/IEEE 42010）〔第 10 章〕

**R-80　V 模型（V-Model）** 开发过程模型：验证与确认的 V 形对应（从这条路径推出的主流方法）〔第 3 章〕

**R-81　IPD（集成产品开发）**　技术评审点 TR / 决策检查点 DCP（从这条路径推出的主流方法）；TR2/TR3 作架构评审放行点（架构六步第⑥步评审背书）〔第 3 章〕

**R-82　MRL（制造就绪水平）**　生产就绪的另一把尺子（与 NASA TRL 对应：TRL 管技术成熟度、MRL 管制造就绪）〔第 5 章〕

**R-83　COTS（Commercial Off-The-Shelf）**　货架商用产品——组织资产的复用来源〔第 5 章〕

**R-84　依赖倒置原则（DIP, Martin）**　面向接口编程、依赖倒转（SOLID 之一）〔全书正文未引·参考书目〕

**R-85　邓巴数（Dunbar, 1992）**　群体规模上限约 150 人——"邓巴数式的直觉"〔全书正文未引·参考书目〕

**R-86　INCOSE Systems Engineering Vision 2035**　角色与能力五组（Core SE Principles / Professional / Technical / SE Management / Integrating）；模块→系统成长的能力缺口〔第 9 章〕

**R-87　SEBoK（Systems Engineering Body of Knowledge）**　派生需求（derived requirements：架构实体的必要实例引发）；协商理解（negotiated understanding）；架构定义活动（partition / align / allocate）；架构描述须说明"何为纳入、何为排除"，fundamental 的详细程度取决于使用语境与设计目的（第 16 章 §16.2.2／§16.2.5）〔第 6、18 章〕

**R-88　Polanyi《个人知识》(1958）**　隐性知识——"我们知道的多于我们能说出的"（价值三层·决策/过程层不完整沉淀）〔第 7 章〕

**R-89　Henderson & Clark,"Architectural Innovation: The Reconfiguration of Existing Product Technologies and the Failure of Established Firms"（Administrative Science Quarterly 35(1), 1990）**　创新四分型（突破式/架构式/模

块式/渐进式）——架构式创新主导动作恰是“定义”（元件不动、连接重排、重新画边界），“创新主靠构思”的最锋利反例〔全书正文未引·参考书目〕

**R-90　熊彼特《经济发展理论》（1912）**　创新 = 执行生产要素的新组合（发明≠创新——创新是新组合的兑现）；“创新=活动义”的出处〔全书正文未引·参考书目〕

**R-91　奥斯陆手册（OECD，创新度量）**　创新须“已被提供给潜在用户或投入使用”——没落地的改进不叫创新〔全书正文未引·参考书目〕

**R-92　Parnas，“On the Criteria To Be Used in Decomposing Systems into Modules”（1972）**　信息隐藏——partition 按“会一起变的”分组〔全书正文未引·参考书目〕

**R-93　Perry & Wolf，“Foundations for the Study of Software Architecture”**　架构腐蚀（drift / erosion）——代码改图不改，动词链“维护与改进”失守的结果〔全书正文未引·参考书目〕

**R-94　Cooper, Winning at New Products（Stage-Gate门径管理）**　Stage-Gate——创新流程可工程化的代表（每步有判定、有证据）〔全书正文未引·参考书目〕

**R-95　Altshuller（TRIZ，创造性问题解决理论）**　矛盾矩阵/40 条发明原理/ARIZ——构思不靠等灵感（“有方法”的证据）〔全书正文未引·参考书目〕

**R-96　Boehm & Basili，“Software Defect Reduction Top 10 List”（IEEE Computer, 2001）**　需求缺陷的实证溯源：约半数软件错误到后期才被发现、且多发生在写代码之前——需求判据“从失败反推”的经验数据锚点〔第 4 章〕

**R-97　万维钢《现代思维工具 100 讲》（得到 App 课程，第 38—47 讲「教育与学习」板块）　能力四级**：表征 → 图式 → 心智模型 → 解释框架（第 41 讲）；“图式”＝把多个元素打包成占 1 个槽位的压缩包（第 38 讲）；参与度 P<A<C<I、“讲给外行听”只到 C 级（第 39 讲）；迁移是验收标准与三道锁（第 47 讲）；默会知识是显性知识的地基（第 43 讲）；落点＝§1.4.4（全书 18 章章末清点块皆回指此节）〔第 1 章〕

---

# 附录 E 练习答案

> 各章“自查 · 这些说法对吗”的答案与理由已随章给出，下表只给**对/错**速查；“练习”的**答案要点**在下。要点给的是“往哪个方向想、用哪把尺子”，不是唯一答案——它们与训练教材题卡同源，考的是你有没有把话说准。

## 第 1 章 概念体系：学习能力的杠杆

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗ ⑭ ✗ ⑮ ✗ ⑯ ✗（背熟定义≠懂，懂=能答四问；概念能力=组织得好非渊博；根概念能力是概念体系能力的子能力非全集；乘号里有两个归零项（方向与表征），力臂长≠产出大；先学细节越学越累；转述=压缩测试；四问答出“是什么”不够；概念是共识不是真理；掌握=结构不是记忆；识别枢纽有可操作的三步法（收集候选→硬判据→往上挂）；多数概念是凝结不是发明；枢纽是骨架还有根概念/原理层；根概念是最源头非最重要；概念是节点、原理是边，核心原理之上还有基本原理；表征是 1、上面三级是 0 ——先建图式不能替代

表征；答全四问只是**入场**——四问是第一级（表征）的作业工具，上面还有心智模型与解释框架）

**练习答案要点（10 题**，与正文 §附录 D 逐条对位）：1. 用**承重测试**筛：**把这一条内容拿掉，这个实体还能达成它的目的吗**——写下至少三条“拿掉就不再是它”的。2. 逐条跑**两遍**：兑现侧（实现目的需要它吗）＋演化侧（目的变了它守不守得住）。3. 判一条“改起来贵”的决定：它是不是因为“贵”才被叫架构？拿掉它，目的还达得成吗（**不可逆性只是代理**，会漏判也会误纳）。4. 按五类归位：**关注域**（业务／数据／应用／技术）／**描述层次**（概念／逻辑／物理）／**架构种类与范围**（企业／系统／参考／总体（战略）／SoS；基线／目标）／**架构风格**（微服务等）／**框架**（Spring 等）；五类叠在一起才是“一堆架构”的病根。5. 一条规矩的**根据在内容侧、不在人**：说得出“不合它就不叫这个系统了”＝架构在起作用；只说得出“有人这么定”＝命令。6. 评审的标的是**工作产品**：图会过时，能签收与冻结的是**架构描述**（不是“那张图”，也不只是那组基本概念或属性）。7. 框架替你定的是“这一类”的组织方式；它**没替你定**服务边界、数据归属、接口契约——那些正是**架构内容**。8. **被默认的架构**也是一份架构：它只是没被评审、没被记录、没人对它负责（这正是最贵的一种状态）。9. 四问量“来路”：建筑图纸 → 最高层构想 → fundamental organization → 概念或属性（含 embodied-in）→ **2022 的 entity ＋两样内容**。10. 只问两件事：**那句“它是什么”说清了没有；它排掉了什么**——答不上第二个，就还没有架构，只有主张。

## 第 2 章 工程：让产品开发可重复

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ◐（显性化是第一步，还要可检验）⑧ ✗

**练习答案要点**：1. 试金石检验 → 换三个人做，能否同等交付；不能 → 缺的是可复制性，对照四层定位（多是方法论层或价值与伦理层）。 2. 敏捷+CI/CD+测试覆盖方法论、工具与实践两层；缺工程科学（“为什么能行”）与价值与伦理（压力下守不守）。3. systematic 答“怎么做事有章法”、systemic 答“怎么看问题有全局”、algorithmic 答“能否自动执行”；例：敏捷迭代是有章法的判断，不是算法执行。4. 系统工程 = 元工程（整合部件、不造部件）；“部件全对、整体出事” = 涌现属性，去查没人管的接口/数据口径。5. 技术系统看：上线 = 成功；工程系统看：人会用、流程顺、组织撑得住 = 成功；答案不一致 → 缺的是人/流程/组织/数据。机场就是现成例子：物理、技术、人、流程、信息协同起来，才产生“顺利运行”的涌现行为，工程系统的边界画在整个运行环境上。6. 验收记录：对上“规定需求”的是验证，对上“预期用途”的是确认。最常见的病是“验证全过、确认没做”——测试全绿只说明“做对了”，没人用仍然可能失败（§2.4.4）。 7. 三大事故共同点 = 价值与伦理这一层先松手（不是不会做，是没守住）；找“为进度牺牲可靠性”的真实例子。 8. 石军 = 手艺（依赖个人）；工程化改造第一步 = 把“心里有数”的判断显性化成可检验的规则/文档。9. 六方面打分：恒信有结构/有方法/可重复**沾了边**；全覆盖/受控/可验证不合格（缺三个）。10. 最值钱的手艺 → 显性化：把“心法”写成可检验的规则，再验证换个人能按它交付。 11. 四要素打分：科学原理、约束、系统方法、有用且安全的人造物；缺“有用且安全”（价值与伦理层失守）最致命。12. “工程 ≠ 手艺”转述测试 → 量的是“工程”的“不是什么”（对立：手艺），用第 1 章转述测试检验——讲不清“换个人也能同等交付”就卡在“试金石”上；溯源：engineering ← 拉丁 ingenium（天生的才智、巧思），ECPD 1941 首次把“造东西”提炼成可检验的要素。

## 第 3 章 需求、设计开发、质量：产品开发的图式

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✓ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗

**练习答案要点**：1. 点子不算输入 → 要经研究确认成需求；分界线 = 是否表述为“明示 / 通常隐含 / 必须履行”的需求或期望（ISO 9000:2026 §3.5.1）。2. 技术方案做完不够 → 缺采购/制造/测试/交付/部署/操作/运维等方案；终点由“最后一个配齐的方案”决定。3. 特性超出需求范围 = 过度设计，只加成本、不加质量（质量 = 固有特性满足需求的程度，超出需求范围的特性对质量没有贡献）。4. 本质 = 需求集合内部打架（交付时间也是需求）；砍验证 = 不度量就宣布合格，风险后移成现场返工；口头改需求 = 偷换靶子。5. 一起剪裁 = 需求所有权在利益相关方 + 走正式变更（排序/留痕/挂账）；单方面砍 = 偷换。6. 等级答“按哪套需求分类”、质量答“满足到什么程度”、符合性答“满足了吗”；高档车可以是差质量，因为等级高 ≠ 质量好。7. 特性还没开始规定 = 需求开发（ISO 9001:2015 §8.2）；特性开始被规定 = 设计开发（§8.3）——需求规格就是那道分界。8. 起点判据“可转化”的四条前提：完整、无歧义、不冲突、隐含需求已识别；最弱那条的后果 = 错误需求被完整转化。9. 用四问的“它从哪来”量三个词 → 需求 ← 拉丁 requirere（要求、寻求），原指“要的东西”，后漂移出“明示/隐含/必须履行”三形态；设计开发 ← designare（标注、指明），ISO 9000:2026 以 design and development 完整表述“需求转化”一件事，不拆两半；质量 ← 拉丁 qualitas（“如何”），从“档次、用料”漂移成“符合需求的程度”——漂移的那个词是“质量”，正是本章最大反转。10.“定需求 → 出规格 → 判满足”走项目 → 定需求=需求（“可转化”的四条前提，隐含需求最易漏）；出规格=设计开发（方案配齐才到“可实现”，只到技术方案=没走完）；判满足=质量（靠测试方案里的接收准则认定，没有准则只能“感觉”）；三处边

界易混处——起点看“特性有没有被规定”、终点看“还有哪个环节反问这到底怎么做”、参照看“超出需求范围的特性是过度设计”。

## 第 4 章 需求工程：把诉求变成需求规格

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗

**练习答案要点**：1. 开发欠账 = 没调研、没分析、没写可验证的规格；管理欠账 = 没基线、没变更流程、没追溯——两张单子按“源头／过程”归类，“加强需求管理”只治得着后一张。2. “明示但不可采”→ 补上判定方法与版本：例“重要合同要审批”→“金额 ⩾ 500 万或三年期以上须审批”（注意它本来就是规定需求，缺的是可采，不是资格）。3. 跳过分析直接进规格的代价：澄清不足（“重要”没定义）、分解没做、排序没做、建模没做 → 验证没有可对照的靶子。 4. 改口叫评审的差别：评审对的是**工作产品**（判适宜、充分、有效），签的是“这份需求立得住”；叫成“确认”等于给还没实现的东西签了确认的字，验收时那句“确认通过”就成了扯皮的证据。5. ConOps 讲一段过程（系统怎么被用）、SRS 讲一条条条款（必须做什么）；ConOps 还兼着确认判据的载体（隐含需求的正文写在运行场景里）；先条款后故事，条款就悬空。6. 用六条判据打分 → 最弱一条及其后果：如“可追溯”最常被牺牲——它不立刻显形，变更时答不上“从哪来、影响谁”。7. 需求基线在哪；没有基线，最近一次变更“从什么变到什么”说不清。8. 最常被跳的一环是影响分析（口头改需求）；代价 = 查无实据的暗改，出事时拿不出凭证。9. “25 号之后签的合同算下个月”：访谈随口一句 → 挖出来 → 显性化进需求池 → 写成规定需求（隐含需求不会自己开口）。10. “合同数据留档十年” = MoSCoW 的 Must / Kano 的基本型——走的是环境与法规那条来路，不经组织目标，所以不在被砍清单里。11. 三条来路 = 目标转化（主流，明示）、法规与环境约束（**法规强制**那一支属“必须履行”、不经目标；“环境”那一支另立一类，

第 5 章承接）、利益相关方的隐含需要（通常隐含，未必被目标覆盖）；最容易被漏的是第三条——它不开口，也不在目标的覆盖范围里，只能靠需求工程自己补齐。

## 第 5 章 需求的完整性：全不全，责任在承制方

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗ ⑭ ✗

**练习答案要点**：1. 冰链八条 → 八条可实现性需求一一对应：定制芯片＝物料可采购性、注塑做不出来＝可制造性、手工校准＝可测试性、无认证＝可交付性、无远程升级＝可部署性、无备件模块＝可运维性、电池回收＝可回收性、成本超四成＝经济可承受性。重做需求时这八条都该在需求阶段写出来，尤其经济可承受性——它是前七条共同堆出来的总账。2. 三类各三条 → 最少的那一块通常是可实现性；原因从四个错位里找（相关方、时间、话语权、认知），多半是承制方的代表没在需求阶段发声。3. 逐块问验证载体那条判据：功能需求用什么验（原理样机）、环境适应性需求用什么验（环境样机）、可实现性需求用什么验（真正的产品）——答不上来的那一块，就是欠账。4. 症状反推：每一个“不方便”对应一条可实现性需求；当时评审没人提，多半因为提它的那些人（承制方各业务代表）不在需求阶段的桌上。5. 测试性与可测试性：同一个特性两个主人——使用环境问“在役测得出吗”，归环境适应性需求；承制方问“产线上测得方便吗”，归可实现性需求；判据是谁在问，叫法跟着主人走。6. 访谈对象清单：采购、制造、测试、交付、部署、运维、回收的业务代表，加市场代表，有没有在需求阶段发过声？没有，可实现性需求就会系统性漏失。7. 回路诊断（问题、转化、入库、复用）：卡点最常见在“转化”（问题停在口头，没提炼成带指标的需求条目）与“入库”（没人维护资产库）。8. 经济可承受性：提出者应是市场代表（依据对竞品与替代

产品的价格成本研究），要设目标成本一类的指标，并经相关方确认；若没有，“降本大会”反复开就是证据。9. 恒信镜照：原型＝原理样机、试点＝环境样机、正式产品＝真正的产品；“上线成功”只走到功能那一块，部署、运维、成本这些属可实现性需求，要等真正的产品跑起来才验得了。10. 四问量“验证载体”或“承制方主导”：答全四问才算拥有。最薄的那一问通常就是最容易出错的地方——答不出“往哪去”，验收时会拿错交付物；答不出“从哪来”，就说不清为什么每一块内容要各配一个载体。

## 第 6 章 设计开发：需求与方案结对

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗ ⑭ ✗ ⑮ ✗

**练习答案要点**：1. 树 vs 散文判断：查“不可定位/不可承接/不可追溯”三罪各一个具体例子。2. 画框架树：标分类节点（文件夹）与枝梢节点；给一个枝梢节点写承接判据（可承接哪类需求）＋分解源判据（分解产出哪类叶节点）——双向可用。3. 语义四查给证据：覆盖完整（父要求都有对应）、不超出（无影子需求）、逻辑一致（不打架）、约束继承（约束有叶子承接）。4. 指标三情形：判断“可拆分（函数关系）/ 下放（同值继承）/ 暂不分解（标注向下一层传递）”，写出判断过程。5. 分配三自检：全覆盖（叶节点数对账）、单承接（一个叶节点一个归宿）、N:1 合理（每个节点 2~5 条较优，以语义为准）。6. 悬空叶节点判断：叶节点切粗了（迭代重切／拆条）vs 方案树缺节点（生成新节点）——用“拆得开吗”区分。7. 覆盖检查＋符合性分析：先核“该节点承接的所有需求的语义 ⊆ 其叶节点语义之并”，再按“针对需求 A，本节点通过 X 机制满足 Y，指标 Z 满足/降级/丢失”落句——写不出就是没分析。8. 框架定义 vs 架构：一个是结构骨架（一份文档），一个是系统不可还原的涵义；“架构 = 一份文档”是词汇

病。 9. 干枝叶子独立演进：修剪枝条只动框架、长新叶只动详细 → 变更成本下降，可分别纳入基线（第 15 章）。10. 结对走一遍：上层叶节点落到下层节点（复用／改进／新增各试一次）+ 覆盖检查/符合性分析，记录每步自检结果。 11. 四问量“分配生成”（或“分解分析”）→ 分配生成是什么（叶节点落到能负责它的节点上＋自下而上调树：复用→改进→新增）、不是什么（不是简单分派——单承接是硬约束、资产优先有序）、从哪来（从分配实践与资产复用原则凝结）、往哪去（阶段一长骨架，是分解分析的前提）；最薄的一问通常是“不是什么”——把“分配=简单分派”当成它，正是最容易混用的地方。13. 判步型：两侧的叶节点不是同一批 → **分配步**（做承接，Y=F(X) 在此求值）；两侧叶节点是同一批 → **整段搬入步**（不做映射，活是长干枝并逐级判定落位）；同一份产品数据在两个形态之间 → **形态轴步**（不是承接）。12. 六步走一遍：①接需求（需求就位：链首自原始材料结构化/链上验收上一对移交）、②验就位（三把尺子：可转化/可分配/边界不越位 + 两把数值尺子：指标闭合/分配预判）、③出方案（两个以上候选骨架）、④对需求（复用→改进→新增）、⑤做裁决（写下理由）、⑥签定稿（分解分析：节点→方案叶节点＋覆盖检查／符合性分析 → 利益相关方确认 → 冻结基线）——六步 = 三动作的展开：①②=需求就位（做+验）、③④⑤=分配生成（目标+做+裁决）、⑥=分解分析（做+核+封存）。

## 第 7 章 产品数据：开发产品就是开发数据

**自查速查**：① ✗ ② √ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ √ ⑧ ✗ ⑨ ✗ ⑩ √ ⑪ √ ⑫ ✗ ⑬ ✗ ⑭ ✗ ⑮ ✗ ⑯ ✗

**练习答案要点**：1. 六类数据各两个工件：需求（需求规格/验收准则）、设计（图纸/架构描述）、制造（工艺文件/BOM）、质量（测试报告/验证证据）、服务（维护手册/故障数据）、管理（变更记录/基线标识）。

2. 把数据当文档的例子：如“需求在共享盘里三个版本分不清”——数据是内容、文档是容器；开发全程在数据上工作，实物只是数据的复制品。3. 两域三态图：信息域（开发/验证）↔物理域（制造/使用），靠数字孪生映射；设计态（EBOM·纯数据）、制造态（MBOM·数据+实物）、使用态（维护记录·实物+实况）。4. 多环相乘分析：最终质量=需求正确性×数据质量×制造质量×使用质量；找出断掉的那一环，数据质量在其中起“上限”作用。5. ISO 25012 的五把尺子逐项给证据：准确性（数据反映真实对象）/完整性（无遗漏）/一致性（不矛盾）/现时性（跟上现实）/可信度（经得起验证）。6. 出题人=客户（需求源头）、判卷人=用户（验收裁判）；四条路径：需要（源头）/使用（新数据）/反馈（修正）/验收（把关）。7. 价值三层：物化层（需求/架构描述 AD/实现工件，完全在数据里）、决策层（选型判断，ADR 部分沉淀）、过程层（沟通/评审，只留记录）；漏掉的那层怎么补（如决策补 ADR、过程补复盘）。8. 职责三步：识别工件（配置项清单）→ 指定 owner（一件只一人）→ 挂机制（权限/签字/审批/指标挂到 owner）。 9. 数据资产四类：需求资产（该做什么）/设计架构资产（怎么做得好）/验证资产（怎么证明对）/管理资产（为什么这么做）；判断能否跨产品复用。10. 实现方案八方案：技术/物料采购/制造/测试/交付/部署/操作/运维；最易漏的通常是交付/部署/操作/运维——只到技术方案就宣布终点，下游空转。 11. 流水线工位/工件：需求定义（需求数据）→设计（设计数据）→实现（实现产物）→验证（验证证据）→移交（基线）；缺陷放大例：需求错→设计跟着错→验证测错→量产全错，修复成本从 1 涨到 30~70 倍。12. 增值判据分格：完善需求（内容增值）/加顾客不要的特性（过度加工）/审批草稿转受控、合规打标注满足规则（状态增值）/走过场、数据没变（必要非增值）/重复造轮子（纯浪费）。

## 第 8 章 评审、验证与确认：各有一个参照

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗

**练习答案要点**：1. 四维表：参照（目标/规定需求/预期用途）、动作（确定/证实/证实）、证据（不要求/必须/必须）、时机（事前事中/对结果/真实场景）各填一行。 2. 评审重做：先写目标（对象 + 时限 + 可衡量结果），再问适宜/充分/有效三问。3. “测试全绿”缺哪一要素：多为“证实”缺（只有结果没结论），或“对照”缺（需求没成文，尺子没有）。 4. 评审对目标：写出这条需求对应的目标，拿目标照需求——需求够吗？5. “测试全过、没人用”病根 = 隐含需求系统性漏失；验证射程在文档、确认射程在场景。6. 口头改的需求：定位它漏在哪一步（没进文档 → 验证够不着 → 确认没场景）。 7. 设计一次确认：真实用户 + 真实任务 + 真实场景 + 真实数据，写一份确认方案。8. V 模型：需求在哪一级分解，验证在哪一级回收（左右镜像）；只做最后一次性验证 = 中间层级缺陷藏身。 9. 评审管把关（事前/事中），复盘管改进（事后、绕回起点）；复盘驱动下一轮评审的目标。10. 通过标准会后现想 = 没有尺子；改成会前发出去、会后逐条打勾。 11. 三关走查：评审（对目标）过了吗、验证（对需求）过了吗、确认（对用途）过了吗；缺每一关的代价不同（白做/返工/没人用）。 12. 架构评审四看（概念陈述 / 选定 + ADR / 视图·基线 / 回炉痕迹）缺哪样 = 材料半成品；对质三问答不上 = 没权衡；没背书 = 只是架构师个人意见（评审变汇报、异常不闭环 = 放行半成品）。

## 第 9 章 工程师：一份能力清单

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ◐（显性化只是第一步，还要可检验）⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗ ⑭ ✗ ⑮ ✗ ⑯ ✗ ⑰ ✗

**练习答案要点**：1. 体检：逐项打勾，指出缺项，并说出缺这一项会走岔成哪一种（只会做不会说／只会说不会做／守不住底线／能力失控）。判据只有一条：换个人接得住——缺哪一项，就问哪一项。2. 粒度定位：按谱系（模块工程师／系统工程师／系统架构师）定位——背的是专责还是终责；有没有“名片一个粒度、实际另一个粒度”的人（如石军＝模块岗位、干系统级的事）。3. 起点：最近一次亲手把东西做出来是什么时候？答不上来，先做这一件事。4. 第一道关口：把最近一次做对的事写成可交接的记录（步骤、依据、坑），交给没参与过的同事照做一次——他卡在哪一步，就是你还没显性化到位的地方。5. 第二道关口：写下最近一次重要取舍（当时哪些约束在打架、选了哪个、依据是什么、后果如何、重来会怎么选）。6. 第三道关口：写下不可越线清单（安全、合规、隐私、数据真实性各一条），再找出最近一次“为进度让路”的现场——利益相关方知情吗？有记录吗？7. 证据：拿你负责的那套产品数据问一句“换个人拿着它，能不能做出来、继续演进”——答不出的部分，就是你的缺口。8. 拓宽自测：现在的视角是模块视角还是系统视角？最近一次“为系统牺牲自己的模块”是什么时候？9. 清单对表：把你手上这摊活按“需求 → 设计开发 → 产品数据 → 判定”摆一遍——摆不出来的那一段，就是你的缺口；断在哪一段，回头读哪一章（需求→第 3、4、5 章／设计开发→第 3、6 章／产品数据→第 6、7 章／判定→第 8 章）。

## 第 10 章 架构概念：定义、价值与误解

**自查速查（14 题**，与正文 §附录 C 逐条对位）：① ✗ ② ✗ ③ ✗ ④ ◐（不准确：微服务是**架构风格**——决策的套路，不是“架构”）⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗（清单是“表达”，不是架构；能签收的是那份描述） ⑩ ✗（需求可判，但判定宾语不是做法） ⑪ ✗（指派是授

权，不是能力）⑫ ✗ ⑬ ✗ ⑭ ✗（不补这一层会落空判据／责任／留痕三组）

**练习答案要点（10 题，**与正文 §附录 D 逐条对位）：1. 用**承重测试**筛：去掉它，这个实体还能达成它的目的吗——写下至少三条“去掉就不再是它”的。2. 逐条跑**两遍**：兑现侧（实现目的需要它吗）＋演化侧（目的变了它守不守得住）。3. 判一条“改起来贵”的决定：它是不是因为“贵”才被叫架构；去掉它，目的还达得成吗（**不可逆性只是代理**，会漏判也会误纳）。4. 按五类归位：**关注域**（业务／数据／应用／技术）／**描述层次**（概念／逻辑／物理）／**架构种类与范围**（企业／系统／参考／总体（战略）／SoS；基线／目标）／**架构风格**（微服务等）／**框架**（Spring 等）。被当成“四个架构”的，正是把第一类当成了四个实体。5. 一条规矩的**根据在内容侧、不在人**：说得出“不合它就不叫这个系统了”＝架构在起作用；只说得出“有人这么定”＝命令。6. 评审的标的是**工作产品**：图会过时，能签收与冻结的是**架构描述**（不是“那张图”，也不只是那组基本概念或属性）。7. 框架替你定的是“这一类”的组织方式；它**没替你定**服务边界、数据归属、接口契约——那些正是**架构内容**。8. **被默认的架构**也是一份架构：它只是没被评审、没被记录、没人对它负责（这正是最贵的一种状态）。9. 四问量“来路”：建筑图纸 → 1996 最高层级的构想 → IEEE 1471 根本组织 → 42010:2011 基本概念或属性（体现于元素与关系）→ 42010:2022 实体＋删去该层。10. 只问两件事：**那句“它是什么”说清了没有；它排掉了什么**——答不上第二个，就还没有架构，只有主张。### 第 11 章 架构定义：把判断变成判据

**自查速查**：① ✗ ② ◐（不准确） ③ ✗ ④ ✗ ⑤ ✗ ⑥ ◐（不准确） ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗（“不成立”） ⑬ ◐（不准确） ⑭ ✗ ⑮ ✗ ⑯ ✗ （⑤ 动词是变换、不是整理：整理不产生内容；⑨ 清单属内容侧、描述属描述侧；⑩ 架构可以关乎结构，那是外延不是产出，

且架构实体≠系统元素；⑪ 往里加的是四样；⑫ 判断要成为依据，中间四个环节缺一不可；⑮ 经验只帮着找、不帮着选）

**练习答案要点**：1. 逐项归位：相关架构／方针与指令／生存周期概念／利益相关方关切与需求／系统需求。**目的与目标不在清单里**——它们是**准绳**；而**架构目标与关键成功准则**才是这一步的**产物**（别把准绳当输入、也别把产物当输入）。2. 三条去向：**落成架构内容**（变成某一则基本概念／基本属性／管控原则）／**被既有架构覆盖**（上一轮已把它管住）／**不在架构层承接**（归设计定义或实现）。判据只有一句：**这条需求是不是只有在“这个实体是什么”定下来之后才判得了**。去向不许是“消失”，要能倒查回去（42010:2022 §6.2 未处理项声明即此意在标准里的着落）。3. 归到“不在架构层承接”的，各写它该去的那一层（设计层／流程配置／实现）。写不出来，说明它可能没人管——这正是“不许消失”要防的。4. 四环体检：判断“这个实体是什么”／把它立成内容／**选定并背书**／**下游必须依据它**。冰链那间会议室四环全空（没有可划的界／没有可引用的内容／没有可背书的定案／没有可立得住的依据）；绕过去的方式通常是**从第四环倒着补**（先定了再说）。5. 三条“必须遵守”的规矩各问一句：它是从哪条需求或目的读出来的？读不出来 → 它可能是**该立的管控原则**（自加），也可能只是一句习惯。分辨点：原则管的是**演化**（将来新增什么功能都不许破坏它），习惯只管这一次。6. 四项检查逐条过。过不了“有物可依”＝用了哪件既有解答不出、**适配理由没写**（＝把没验证过的做法直接搬进来）；过不了“有辨可评”＝它跟别的候选在权衡空间上分辨不出差别（42020 §9.4.8 NOTE 1：权衡完全相同的两条不予考察）。7. “现状照旧”要写成正式候选并配一句目的陈述（42020 §9.4.7 NOTE 2：the status quo is sometimes one of the alternatives）。写不出来，说明你还没认清**现状在保什么**。8. 三句话：它是什么（**概念**）／它必须守什么（**原则**）／它为什么适用（**适配理由**）。最难写的通

常是第三句——前两句靠标准与经验，第三句只能靠你对**这个局面**的判断。9. 经验（启发式）在**生成那一环**最有用（想不出候选时它是起头的本事）；在**选定那一环**则是把偏好偷渡成理由。给一个具体例子，并指出它出错时**位置**错在哪一步。10. 先分流：全新开发（no prior system）／产品线启动（一次为多次服务）／遗产革新（有资产但不打算沿用，重点在**重判资产可用性**）／增量启动（不确定性大，产出**后续步骤的选项**）。五处适配：走法从“资产优先”改“前向为主”／取材换来源／输出加两件／迭代策略换／**收窄的口径也要调**（去掉明显走不通的，不是去掉不熟的）。11. 每划一条界写一句理由：**是因为它威胁目的，还是因为我们不熟**？说不出理由的界，就不该划——三条红线即由此来。12. 四个问题：它说清“这个实体是什么”了吗？它是一条概念、一条属性还是一条原则？去掉它这个实体还能达成目的吗（**承重测试**）？它凭什么管住下游？——四问都答不出，它多半只是一句口号（“我们是微服务架构”就属这一类）。

## 第 12 章 架构决策：在承重处做选择

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ◐（不准确） ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ◐（半对） ⑨ ✗ ⑩ ✗（①②③⑧⑨ 的错因见附录 C 第 12 行；④“越早定越好”错在一刀切——按不可逆性定节奏；⑧“记录写全六问就够了”错在漏了标准分 shall／should 两档与“决策理由要单独记”）

**练习答案要点**：1. 把一次改动拆成“一组选择”：这次改动里其实含几条独立的选择？各有几个选项？（拆不开，多半是还没认出它是一条架构决策）2. 数据模型级决定的联动清单：主键／状态枚举／字段口径这类，列出重做时要动到的下游（结构上的、时间上的、组织上的）。3. 三样各打高／中／低：影响半径（结构／时间／组织耦合）、信息成本（知识深度／隐含上下文）、修改风险（关键路径交叉／是否动已存在的数据）——**只用来定先后，不用来定资格**。4. 找“不知为

何这么定、不敢改”的决定 → 大概率信息成本极高 → 先补写“背景”。5. 用六问写一份记录（名称／缘由／候选／定者与时间／影响／反悔难度）——它的骨是 §6.10.1 的十一项内容；找不到“背景／候选”= 这个决策从没被认真记录过，这本身就是发现。6. 五个决策按不可逆性排序；对照你们投入的审查资源是否跟着排序走。7. 显式推迟三句：现在为什么不做？等什么信息？什么时候再评估？（记成未决议题，不是记成决策）8. 高不可逆的决策：先过目的关（它守住的是对目的必要的不变量吗），再按三样定节奏与审查深度。9. 高不可逆的决策，先问“谁有权定”——评估给出判断，处置要有权的人作出。10.“把谁列为候选”也是一条决策：你们这一轮的候选是怎么定的？范围定窄了没有？现状照旧上桌了吗？ 11. 复盘一次被测序压住的决策：那一次是“威胁目的”，还是“我们不熟”？两个理由要分开写。12. 你写下的那条记录，别人（没参与的人）读完能不能接着往下做决定？读不懂，缺的多半是理据与决策权属方。 13. 恒信“六问写不全的不进评审”这条规矩，管的是记录写没写全，还是这条规则谁有权改？（答案：只管前者。前者是团队自己立的关，自己也能撤；后者要有人有权，归第 16 章。）

## 第 13 章 架构描述：把架构定义交出去

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗（① 图只是视图组件的一种形态，描述才是信息部件；② 能签收、能入库、能版本化的是描述，不是架构；③ 立场是利益相关方角度，视角是一组惯例；④ 内容与读法是两件事——读法不写下来，五个人读出五个意思；⑤ 规格缺项，产出的东西不成其为视图；⑥ 关切没有视角框定，它在这份描述里就没有位置；⑦ 生成顺序不可颠倒：先订视角、后造视图；⑧ 非模型式组件带一份图例即可，标准不规定形式化

程度；⑨ 功能视图不是规范性要求，但几乎总会被包含；⑩ 决策权属方那一格补不回来——补上去的名字不是当时拍板的人。）

**练习答案要点**：1. 三问落点都在“能不能上账”：不在描述里、不受某视角治理、组件没标模型种类也没有图例——任一问过不去，它是目录页或草图，不是视图。2. 逐项数 §8.1 a)~g) 七项（外加第二段一句“宜”）；最容易缺的是**对应方法**与**视图方法**——缺了它，换个人复现不出同一张图。3. 判视图看“受不受某视角治理”；组件可以同时进另一张视图（§3.19）——同一张数据流图既能在功能视图里，也能进信息安全视图。4. 按附录 B.2 模板的次序走；自查一句＝“换个人拿着它，能不能产出同一张图”。5. 两堆的分法看有没有管控模型种类；不基于模型的那一堆必须配图例（§3.19 注 1），否则读法没写下来。6. 对照十一条自检逐条过；过不去的条目按出处回补——多半落在 §6.6／§6.7／§6.9 三处。7. 由关切反推：每条关切至少一个视角框定、每个视角至少一张视图、每张视图应对它那个视角框定的全部关切；用不上的那一类，往往是没有对应关切（或关切没写下来）。8. 那一格问的是“谁拍的板”；空着就写清为什么空——它是治理在描述里落款的地方，事后补名字等于把记录做假。

## 第 14 章 架构与设计：不是概要和详细

**自查速查**：① ✗ ② ◐（前半对、后半是偷换）③ ✗ ④ ✗ ⑤ ✗ ⑥ ◐（漏了一半：还须对着架构目标）⑦ ✗ ⑧ ✗（**已废**——不合格的是追不到架构依据、又不回流）⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗（SBS 属项目规划过程）⑬ ✗ ⑭ ✗ ⑮ ✗

**练习答案要点**：1. 交接体检：列不出这两栏，说明手上那一版是“收到文档”而不是“取到约束”——把“本次从架构取到了哪几条约束”抄成两栏（左栏适用于本层的基本概念或属性，右栏指导设计及其演化的原则），抄不出来的地方就是缺口。2. 两类处置要分开走：**架**

**构漏了**（既有职责没交代）→ 回架构补一次识别、重新走冻结；**需求变了**（新出现的功能）→ 把衍生需求纳入系统需求基线、走变更流程。两条路都叫**认领**；不分清就走，它就成了没人认领的线。3. 追一条架构特性这一路：它在架构侧是概念或属性（或管控原则），到设计侧变成哪几个**设计特性**、落到哪个**设计元素**上、取值定在哪里；追不动时先判在哪一侧断的——架构侧没立住（特性本身不可判定），还是设计侧没转（有元素无特性落点）。4. 用**四层两界**给十个元素分一次层：描述自己的零件／架构实体／设计元素／系统元素；界一＝描述自身的零件与被描述者，界二＝架构实体与系统元素（两者是 two different notions）。第一遍分错的那些，多半是把“描述里的零件”当成了“被描述的东西”。5. 物理视图的收笔处：**型号、尺寸、材料、数量、实例数**一出现，就属设计定义（15288:2023 §3.13 注 1／§6.4.5.3 注 6）；物理视图只到“系统元素与物理接口的一种安排”为止，再往下是设计解的活。6. 第四问就是“基本”唯一的那把尺：**拿掉这条内容，这个实体还达得成目的吗**——不是“高不高层级”，也不是“改起来贵不贵”；且它是**相对所关注实体**的（同一条内容在上层可能是基本属性，在下层只是设计取值）。7. 四句问的是同一层里的四件事：它服务于本视角框定的哪条关切／它按哪个模型种类或图例的约定被解释／它表达的是不是被架构实体的架构／它排掉了什么。四句答不全，它这份**登场**就没交代清楚。8. 那一段话该说的是**取走什么**，不是“文档在哪儿”：这一版架构交出去，接住的人要取走的是约束（适用本层的基本概念或属性／指导设计及其演化的原则）与它必须守的那几条约定；写不出来的，就是这次交接还没办完的部分。### 第 15 章 配置管理：让架构判断受控、可复现

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗ ⑬ ✗ ⑭ ✗ ⑮ ✗ ⑯ ✗（③ 有版本管理、没有配置管理正是最常见的坏法；⑤ 最新只说明它在最右边，不说明它被批准过；⑨

评审过的是内容，算不算数要有正式化这个动作；⑪ 配置审计的对象是配置项与产品、判据是基线，审核的对象是组织与体系、判据是审核准则；⑫ 基线承载的是“明示的”那一部分，确认一侧它撑不住；⑬ “功能基线／分配基线”是行业惯例，用须标依据）

**练习答案要点（10 题，**与正文 §附录 D 逐条对位）：1. 没进。缺正式化——要有批准的权威、固定下来的时点、此后被当作参照；版本号与评审通过都只是它的前提。2. 第二句。第一句改的是可调的一个数，第二句改的是设备的形态，形态一变，采出来的记录怎么读都跟着变。3. 两个变体、两条版本线；−20℃ 这条走到了 v2.3，2~8℃ 那条停在 v2.0。4. 缺“哪一版、谁批的、什么时候”——没有参照，“最新”不是答案。5. 判不了，先要问 v1.3 是不是当时的基线、v1.4 有没有正式化。补一句“按哪一版、那一版是不是基线”就能判。6. 属于“有版本管理、没有配置管理”（前一半还是假的：文件名不是版本）。它比“完全没有版本号”更危险，因为它看起来是做完的。7. 那三个下游是在按一个会变的版本干活，改动发生时他们不会被告知；等定稿统一发布，等于让他们返工。8. 因为它是治理的事——“谁有权建立配置控制机制、谁裁定架构集合中的变更”归架构治理，本书第 16 章讲；本章只把候选递出去。9. 不算。纪要与归档都只留了副本：缺参照（指不回是哪一版）、缺门（改它不必过手续）、缺理由（追不到“凭什么”）。10. 缺“基线”一栏——对哪一版测的、谁批的、测的时候是什么状态。没有这一栏，“通过”只对当时在场的人成立。

## 第 16 章 架构治理：立规矩，对结果问责

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗（**十二说全是病**；几处要点——⑤ 正本 §6.4.5 注 4 明说“**仅检查架构的符合性可能不够充分**”，实物那一层要另评；⑦ 三层基线（治理／协议／产品 CM）是与“功能／分配／产品基线”**正**

**交的另一条轴**——前者问“谁有权改”，后者问“成熟到哪一步”；⑧“作出决策”只占六项活动之一，日常动作是**看与立**，所以“上次治理会产出了哪一条指令”是最省事的判据；⑪ 该不该进治理看它**动不动目的、原则、跨系统约定**，不看早晚；⑫ 技术权威位不等于治理位，正本 §6.2 c) 要的是“**具有适当职权者**”）

**练习答案要点（10 题**，与正文 附录 D 练习 逐条对位）：1. 四句＝谁批的／按什么程序／改它要走哪条路／依据是哪一条。**写不出的那一格，就是效力没落地的位置**——不是你的本事问题，是那一格上没有人。2. 五问逐条对（进范围了吗／谁有权定／依据是哪一条／谁该知道／改这条要什么手续）；答不出的那一问，就是这次变更的缺口。3. 五样＝机构与程序／治理计划／治理指令与指导／符合性状态报告／架构集合目标。举不出哪一样，缺的就是那一样——只说“我们建了治理机制”，等于一样都没答。4. 判据是治理的事（**判据本身不可自裁**）。只是一个项目组自己定的，它不构成跨系统的架构约束，只在本组生效；要成为架构约束，须由治理定并落到指令与指导上。5. 看它**动不动目的、架构原则、跨系统约定**：动了＝治理决定；在既定参数内选最优＝管理决定。别拿“跨部门”当判据。6. 一句话：治理**守不变量**（企业的目标、策略、战略，架构的原则与红线），管理**调框架内的变量**；治理建**机制**、管理**执行过程**；两者是“给出与实施”的两端，不是一个在另一个上面。7. 该通知的是“依据这条规矩办事、并且会被这次改动影响”的那些人。依据＝§6.2 c) 由具有适当职权者作出并**传达给利益相关方**——通知不是礼貌，是效力的一部分。8. 升级的理由只有一条：**超出本层的权限**（§6.4.4 要求把不属于治理决策的其他决策上报至适当的企业决策论坛）。报到哪里、结果谁传达，是“资格”能不能被查到的判据。9. 第一条指令应当是一条**判据**（什么变动、按哪一条、由谁裁），并且自带“**怎么被改**”——§6.4.6 要求一轮收尾时评审并修订指令；**指令自己也是受控的**。10. 三种假各举

一例并写明判据（动了哪一层）：**名字上的**（拿管理叫治理）／**缺位的**（三样缺一样）／**“越早越好”的**（把日常变更送上去，把治理用废）。

## 第 17 章 一路贯通：从需求到上线运行

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗

**练习答案要点**：1. 画完整时间线：**顺次四段**（需求〔第 3、4、5 章〕→设计〔第 6、7 章〕→把关〔第 8 章〕→运维〔第 2 章的外框〕）＋**横切三样**（架构〔第 10~13 章〕／配置〔第 15 章〕／治理〔第 16 章〕）＝**七处**；每两处之间写一道缝，圈出最弱的一处。2. 分类结果同上一题：**顺次＝第 3₈ 章，横切＝第 10 13、15、16 章**；**第 9 章讲的人不在这两种切法里**（不占任何一段），**第 14 章**是“架构交出去、设计接住”那道题的例子。组织侧照此分一遍岗位与流程：横着切的那几条线各自谁在贯穿，有没有一条线没人贯穿。3. 复盘三问：缝断在**哪两片之间**、断在**承接／参照／记录／问责**里的哪一条、**责任人**写不写得出来。（恒信断在承接与问责——“编号从 CA 来”没被写成条款，也没人被授予裁决来源的资格；冰链断在承接与记录——“两端时间戳要对齐”谁也没交出去。）4. “应该是什么／实际是什么／怎么变成这样”三问＝**需求侧／验证侧／配置记录侧**各答一问；三个答案对得上，才是同一个系统，对不上先把三个答案放回同一件事上核。5. 两关分开答：**目的关**裁它够不够格进架构（它守的是目的之下的不变量吗），**代价关**量它得到什么待遇（影响半径／信息成本／修改风险——三层框架只作**代理**，管排序与时机、不管准入）；判断它是被提前投资了，还是等爆雷才做。6. 起点判据四条：**完整、无歧义、不冲突、隐含需求已识别**——哪一条最弱；有没有像“编号进台账”那样藏在场景里、没被写成条款的东西。7. 拆“框架定义＋详细定义”两件事：**约束有没有传下来**？两片之间缺不缺那片过渡——也就是“接口叶”？8. 三把尺

子复盘上线：**评审对目标、验证对规定需求、确认对预期用途**，各站住了吗；“确认”是不是真在场景里做的。9. 写一行缝合清单：缝在哪两处之间、靠承接／参照／记录／问责里的哪一条、责任人是谁（一个能写字的名字，不是一个部门）——**写不出责任人的那一行，就是将来的事故现场**。10. 书外行业走一遍：四段路与三样横切还在不在，哪一处最像、哪一处最不一样（本章自己走过两遍——民机平视显示器、企业运营平台，结论是概念不挑行业）。

## 第 18 章 架构师：还是一份能力清单

**自查速查**：① ✗ ② ✗ ③ ✗ ④ ✗ ⑤ ✗ ⑥ ✗ ⑦ ✗ ⑧ ✗ ⑨ ✗ ⑩ ✗ ⑪ ✗ ⑫ ✗（**十二说全是病**；几处要点——③ 专责不是终责，架构的成立与守持是专责、系统成败是终责，两本账；④ 拍板权在系统工程师、理由权在架构师，拍板通常在架构过程之外；⑥ 目的的定义权与问责在业务方，架构师做理解、澄清、质疑、守护四件事，有质询权无改写权；⑦ 五个动作各有一个对象，评审对材料、验证对需求、确认对用途、评估对架构、认证对实现；⑧ 能力不是虚的，它最终落在产品数据上，件能翻、能查、能被接手；⑪ 42010:2022 只给架构活动立条、不给人立条，标准认活不认职位；⑫ 配置管理与架构治理是清单上的两格，一项管判断可复现，一项管判断算不算数）

**练习答案要点（10 题**，与正文 附录 D 练习 逐条对位）： 1. 逐条往那张合图上落（§18.2.1 那张“前面各章交下什么”的表）：有证据的格子是能力，只有头衔的是名分；清单本身是两段加两格（§18.2.2~§18.2.4）。2. 给两个候选各写一句“它为了什么”（§18.2.3 第一件）。写不出来，说明这件事还没有候选，只有倾向。 3. 架构是那份内容，不是承载它的东西，删掉图、决定还在的才算（§18.3.4）；剩下的够不够别人接着干，就是答案。 4. 两本账：系统成败的终责不可委托，架构成立与守持的专责可委托可追责

（§18.3.2）。落在同一个人身上，是兼任，还是有一本没人接。5. 对号入座：评审对材料、验证对需求、确认对用途、评估对架构、认证对实现（§18.3.6）；归不进去的，多半只是“开了个会”。6. 四件事：理解／澄清／质疑／守护（§18.3.5）；目的的定义权和问责在业务方，架构师有质询权无改写权，越界改写就是病因。7. 四种失败各缺一格：画图匠缺“把手段立成概念与原则”，什么都管是把专责当终责，不决策缺工程判断，远离代码缺实践能力（§18.3.7）。8. 七件事：构思／定义／表达／记录／沟通／认证恰当实现／维护与改进（§18.2.5）；归不进去的，通常不叫架构活动。9. 换个人，拿着那些件接得下去吗（§18.4.8）；他问的那些，就是件里本该写着的东西。10. 往上一格是系统之系统或组织，服务对象换一级（§18.4.5）；判断、权衡、沟通、担责这几样里，先看哪一样换。

---

## 附录收束

五个抽屉合上，回到全书那句收束——第17章说过，它仍是这本书想让你带走的一句话：

> **每一片都合格，缝还是会断——一路贯通不是最后多出的一道工序，是每一站都在做的事。**

附录A是对齐用词的目录，B是概念的规格，C是“哪个词最容易用错”的警示，D是尺子的出处，E是“判据练得对不对”的对照答案。它们不新增任何概念——它们只是把正文发过的尺子，收进了一个随时能回来取用的地方。训练教材的题卡与讲师版，与附录E同源；遇到任何一题答不上来，先回对应章节，再回这五个抽屉。

把那些你天天挂在嘴边、却从没真正想清楚的词，放到你有把握判定的位置。